WEBSITE BOOK EDITION

Pixels → Perception

From Pixels to Understanding

Foundations • Vision • Language • Learning • Theory

Volume 3 of 4

Machine Learning and Deep Learning for Vision

English Edition • Volume 3

2026-10-17

https://blog.shyandram.dev/

About this edition

This is volume 3 of 4 of the Pixels → Perception course, *Machine Learning and Deep Learning for Vision*. It contains 22 published tutorials and 0 blog posts, in the order of the website's learning tracks. This is a typeset edition of the website, not a separately peer-reviewed publication.

The prose, code, equations, and references retain the website's content. Exercise hints are expanded for print. Static figures remain in the book, image comparisons show their input and output images, and interactive demonstrations link to the website. Reference numbers are local to each tutorial.

The website is still under review. AI tools assisted its production, and errors or omissions may remain. Read it alongside the original sources and consult the website for subsequent corrections.

English and Traditional Chinese, and each volume, are separate PDFs. Links to tutorials in this volume navigate within the PDF. Links to tutorials in other volumes, to specific web sections, interactive demonstrations, notebooks, and external sources remain online. Content snapshot: 2026-10-17

Source revision: `6b02ce5`

https://blog.shyandram.dev/

Contents

I Machine Learning 1

ML 01 ·Introduction to Machine Learning 2

ML 01A ·Linear Algebra for Learning 32

ML 01B ·Probability and Information Theory 56

ML 01C ·Numerical Computation and Gradient Checks 82

ML 02 ·Neural Networks from Perceptron to Deep Networks 109

ML 03 ·Convolution, Image Filtering and CNNs 138

ML 03A ·Recurrent and Recursive Networks 163

ML 04 ·Attention and Vision Transformers 187

ML 05 ·Representation Learning and Self-Supervision 208

ML 05A ·Linear Factor Models and Autoencoders 232

ML 06 ·Generative Models: Autoencoders, VAEs, GANs and Flows 257

ML 07 ·Diffusion Models and Flow Matching 283

ML 08 ·Autoregressive Vision: Pixels, Tokens and Scales 317

ML 09 ·Foundation Models and Vision–Language Learning 340

ML 10 ·Evaluation, Efficiency and Deployment 364

II Deep Learning for Computer Vision 392

DLCV 01 ·Deep Learning for Image Processing and Computer Vision 393

DLCV 02 ·Deep Image Classification 420

DLCV 03 ·Object Detection: From R-CNN to DETR 462

DLCV 04 ·Deep Image Restoration: Low-Level Vision 493

DLCV 05 ·Low-Light Image Enhancement 544

DLCV 06 ·Object Tracking: From Single-Object to Multi-Object 583

DLCV 07 ·World Models: Video, Latent Dynamics and Planning 613

Part I

Machine Learning

ML 01 • Introduction to Machine Learning

What it means for a computer to learn from data: the learning paradigms, linear and logistic regression, cost functions, gradient descent, overfitting and evaluation, built from scratch in NumPy.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-01-introduction/

What you'll learn

- What "learning from data" means, and how it differs from writing rules by hand, using small images of handwritten digits as the running example.
- The main learning paradigms: supervised (regression and classification), unsupervised (clustering, dimensionality reduction, anomaly detection), self-supervised, and reinforcement learning, including how reinforcement learning from human feedback (RLHF) tunes chat models.
- The vocabulary of a learning problem: model $f_{w,b}(x)$, parameters, features, targets, predictions, training/validation/test sets and cost functions.
- Linear regression with the mean squared error cost, and gradient descent to minimize it: the update rule, the learning rate, convergence, and batch versus mini-batch versus stochastic updates.
- Why feature scaling matters, how logistic regression turns a line into a classifier with the sigmoid and cross-entropy loss, and how to detect and fix overfitting with validation data and L2 regularization.
- How to report results honestly with accuracy, precision, recall and MSE.

The big picture

Most programs are recipes: a person thinks hard about a problem and writes down every step. Machine learning flips this. We write a *flexible* program with many adjustable numbers, show it many examples of the job done correctly, and let an algorithm adjust the numbers until the program does the job well. The examples replace the recipe.

> **Explain like I'm five**
>
> Think about how you learned to tell cats from dogs. Nobody gave you a rulebook ("pointy ears plus whiskers minus barking equals cat"). Grown-ups pointed and said "cat", "dog", "cat" many times, and you made mistakes and got corrected until you just *knew*. Machine learning is the same: we show the computer lots of examples with the right answer, it guesses, we tell it how wrong it was, and it nudges its guesses a tiny bit each time until it is rarely wrong.

Why this matters: almost every modern computer-vision system, from phone face unlock to medical image analysis, is a learned model, not a hand-written program. The ideas in this tutorial (a model with parameters, a cost that measures error, gradient descent that lowers the cost, and separate data to check the result) are the same ideas that train today's largest neural networks. Only the size of the model changes. Once these few ideas are clear, the rest of the series (ML 02 and the deep-learning tutorials) is mostly about *which* model and *which* cost.

About this tutorial

This tutorial is an expanded, formal version of the earlier notes Introduction to Machine Learning [1], which followed Andrew Ng's Machine Learning Specialization [4] and the *Deep Learning* book [3].

What is machine learning?

Plain version. A machine-learning system improves at a task by looking at data, instead of having every decision written by a programmer.

Learning instead of programming

The idea is old. In 1959, Arthur Samuel described a program that learned to play checkers better than he did, by playing many games and adjusting how it scored board positions [2]. His point was that a computer which learns from experience removes much of the detailed, hand-written programming the task would otherwise need. The famous one-line definition, machine learning as the "field of study that gives computers the ability to learn without being explicitly programmed", is a later paraphrase of this idea, usually attributed to Samuel, rather than a sentence from his paper.

A more operational definition, common in textbooks, has three parts: a **task** (for example, recognizing digits), a **performance measure** (the fraction recognized correctly), and **experience** (a collection of labeled digit images). A program *learns* if its performance on the task improves with experience [3], [7].

An image example: rules versus learning

Our running example is the scikit-learn digits dataset [5], [6]: 1,797 grayscale images of handwritten digits, each only 8×8 pixels with values from 0 to 16. As in Images as Arrays, each image is just a small array of numbers, and flattening it gives a vector of 64 numbers.

Suppose we only need to tell a **0** from a **1**. A programmer might write a rule: "a 1 is thin, a 0 is wide, so count how many columns contain ink; five columns or fewer means 1." This rule is easy to read, and it is right for most upright 1s. But people write 1s with slants, serifs and bases, and

a slanted 1 covers as many columns as a 0 (Figure 1). The rule gets about 79% of the cases right. Fixing it means more rules ("unless it slants..."), and each new rule creates new exceptions.

The machine-learning approach writes *no* digit-specific rule. It picks a flexible family of functions (here, logistic regression, which we build later), and lets an algorithm choose the 65 numbers inside it from examples. On held-out images, it separates 0s from 1s perfectly, and exactly the same code handles all ten digits with no new rules:

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

digits = load_digits()                        # 1,797 images, 8x8 pixels, values 0..16
X, y = digits.data, digits.target             # X: (1797, 64) flattened pixels; y: 0..9

# Hand-written rule for "0 versus 1": a 1 is thin, a 0 is wide.
keep = (y == 0) | (y == 1)
imgs, labels = digits.images[keep], y[keep]
width = (imgs.max(axis=1) > 4).sum(axis=1)   # how many columns contain ink
rule = np.where(width <= 5, 1, 0)
print("hand rule, 0 vs 1:", round((rule == labels).mean(), 3))           # 0.789

# Learned rule for the same task, judged on images it never saw.
X01 = imgs.reshape(len(imgs), -1)
Xa, Xb, ya, yb = train_test_split(X01, labels, test_size=0.3, random_state=0)
print("learned, 0 vs 1:", LogisticRegression(max_iter=1000).fit(Xa, ya).score(Xb, yb))   #
 ↪  1.0

# The same recipe scales to all ten digits with no new rules.
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0)
print("learned, 10 digits:",
      round(LogisticRegression(max_iter=5000).fit(X_tr, y_tr).score(X_te, y_te), 3))  # 0.953
```

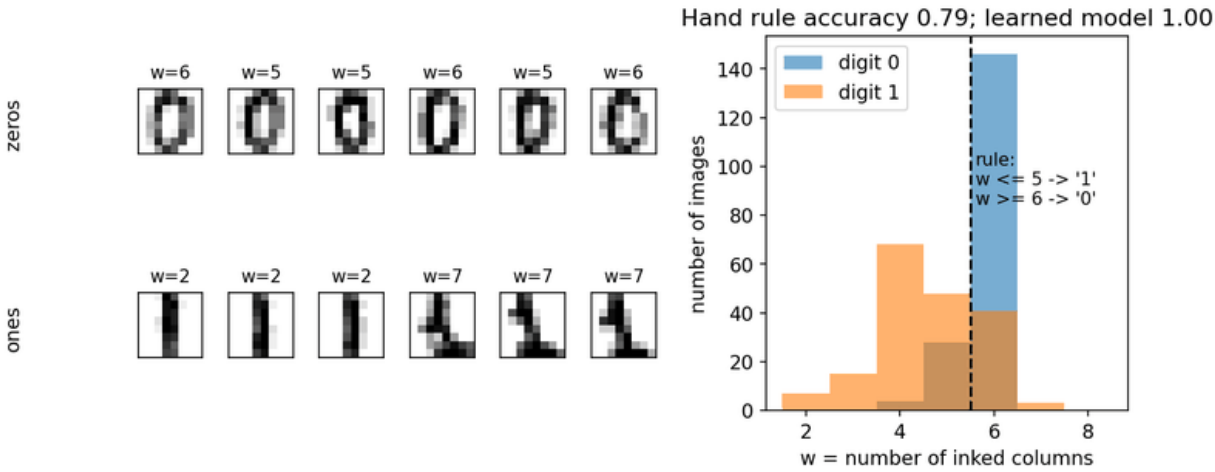


Figure 1 — Why hand-written rules are brittle. Left: zeros and ones from the digits dataset; slanted ones are as wide as zeros. Right: the 'count inked columns' feature overlaps heavily between the two classes, so any threshold on it makes many mistakes. A learned model uses all 64 pixels at once.

This small experiment contains the whole argument for machine learning. When the rule is hard to put into words but examples are cheap, learn the rule from examples. It also shows the price: the learned model is a list of 65 numbers that is much harder to read than "count the columns."

The learning paradigms

Plain version. Learning problems differ mainly in *what feedback the learner gets*: the right answer for every example, no answers at all, answers it can manufacture from the data itself, or only an occasional score for how well it did.

Paradigm	What the data looks like	What is learned	Typical example
Supervised	pairs (x, y): input and the "right answer"	a function from x to y	digit image → digit label
Unsupervised	inputs x only	structure: groups, directions, outliers	grouping similar digits
Self-supervised	inputs x only, but part of x is hidden and used as the target	a general-purpose representation	predicting a masked word or image patch
Reinforcement	an agent's actions and the rewards they earn	a policy: which action to take in each situation	game playing, tuning chat models

My original notes listed the first, second and fourth; self-supervised learning has since become the main way large models are pre-trained, so it gets its own row [9], [10].

Supervised learning

In supervised learning, the learner is given the right answers. The data is a set of examples $\{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$, where $x^{(i)}$ is an input, $y^{(i)}$ is the desired output (the **label** or **target**), and m is the number of examples. The goal is a function f such that $f(x)$ is close to y, *also for new inputs not in the data*.

That last part, called **generalization**, is the real goal; memorizing the examples is easy.

Unsupervised learning

In unsupervised learning there are no answers, only inputs $\{x^{(i)}\}$. The goal is to find something interesting in the data: groups of similar examples, a compact description, or examples that do not fit. There is no single "correct" output, which makes these methods useful for exploration and harder to evaluate.

Self-supervised learning

Self-supervised learning uses unlabeled data but turns it into a supervised problem by hiding part of each example and asking the model to predict the hidden part. In text, hide a word and predict it from the rest of the sentence. In images, hide most of the patches and reconstruct them: masked autoencoders hide 75% of an image's patches and train a network to fill them in [12]. LeCun and Misra describe self-supervised learning as a way for models to acquire broad background knowledge from raw observation, because the labels come for free [9].

The point is not the fill-in-the-blank task itself; it is that a model which can fill in blanks well must have learned a lot about the structure of the data. The learned internal **representation** can then be adapted to many other tasks with few labels. This is the recipe behind **foundation models**: models trained on broad data at scale (usually self-supervised) and then adapted to a wide range of downstream tasks [10]. GPT-3, trained simply to predict the next word on a large text corpus, was able to perform many language tasks from a few examples given in the prompt, without further training [11].

Reinforcement learning

In reinforcement learning (RL), an **agent** interacts with an **environment**. At each step it observes a **state**, chooses an **action**, and receives a **reward** (a number) and a new state. Nobody tells it the right action; it must discover which actions lead to high total reward over time, often when the reward arrives long after the action that earned it. What is learned is a **policy**, a rule for choosing actions in each state [8]. Samuel's checkers player is an early example of this setting [2].

Today, RL is best known for tuning large language models. In **reinforcement learning from human feedback** (RLHF), as described by Ouyang et al. [13], a pre-trained language model is first fine-tuned on demonstrations written by people (supervised learning). People then rank several model answers to the same prompt, and a separate **reward model** is trained to predict those rankings. Finally, the language model is optimized with RL to produce answers that the reward model scores highly. The authors report that labelers preferred the outputs of their 1.3-billion-parameter tuned model over those of the 175-billion-parameter GPT-3 [13]. More recently, DeepSeek-R1 used large-scale RL with automatically checkable rewards (such as whether a math answer is correct) to improve step-by-step reasoning [14].

Notice that RLHF chains three paradigms: self-supervised pre-training, supervised fine-tuning, and reinforcement learning. Real systems mix paradigms freely; the table above is a map, not a set of walls.

Supervised learning: regression and classification

Plain version. Supervised learning comes in two main kinds. **Regression** predicts a number; **classification** predicts a category.

Regression

In regression the target y is a real number, so there are infinitely many possible outputs. Examples: predicting a stock price, a student's final grade from earlier test scores, or a person's age from a face image. The last one shows that the input can be an image while the output is still a single number.

Classification

In classification the target is one of a small, fixed set of categories (classes). Examples: spam versus not spam (two classes, **binary classification**), the shape category of a face, or a tumor type such as benign, malignant type 1 or malignant type 2 (**multi-class classification**). Classes are names, not quantities: "class 2" is not twice "class 1". A classifier usually outputs a probability for each class and then picks the most likely one.

```
from sklearn.datasets import make_regression, make_classification
from sklearn.linear_model import LinearRegression, LogisticRegression

# Regression: the target is a real number.
X_r, y_r = make_regression(n_samples=100, n_features=1, noise=10.0, random_state=0)
reg = LinearRegression().fit(X_r, y_r)
print(reg.predict(X_r[:3]).round(1), y_r[:3].round(1))   # numbers vs numbers

# Classification: the target is one of a few labels.
X_c, y_c = make_classification(n_samples=100, n_features=2, n_redundant=0,
                               n_clusters_per_class=1, random_state=0)
clf = LogisticRegression().fit(X_c, y_c)
print(clf.predict(X_c[:5]), y_c[:5])                     # labels vs labels
print(clf.predict_proba(X_c[:2]).round(3))               # class probabilities
```

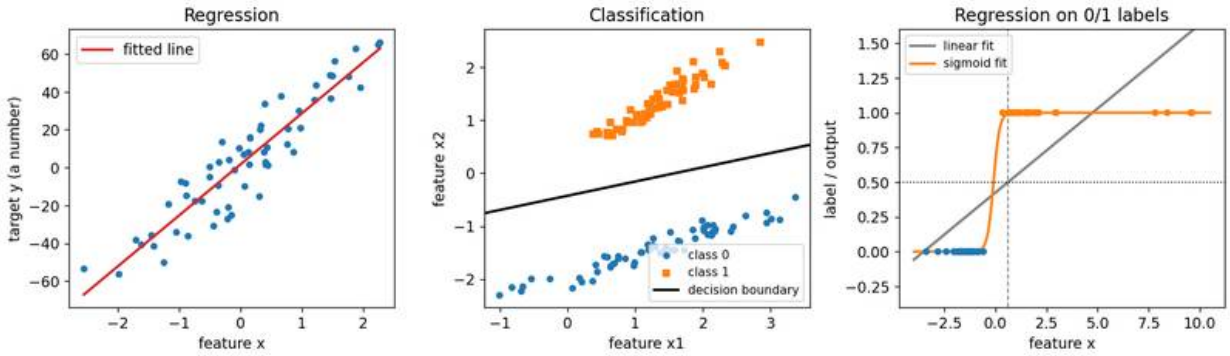


Figure 2 — Left: regression fits a curve through numeric targets. Middle: classification separates labeled points with a decision boundary. Right: fitting a straight regression line to 0/1 labels (gray) gives outputs outside $[0, 1]$, and far-away points drag its 0.5 crossing; a sigmoid (logistic regression, orange) is the right tool.

Why not use regression for classification?

It is tempting to code the classes as 0 and 1, fit a regression line, and call anything above 0.5 a 1. This can work on easy data, but it usually performs badly, for two reasons visible in Figure 2 (right). First, the line outputs values like -0.4 or 1.7, which are not probabilities. Second, the squared error punishes a point that is *correctly* classified but far from the boundary, so a few extreme but easy examples pull the line and move its 0.5 crossing to the wrong place. Logistic regression (later in this tutorial) fixes both problems.

Unsupervised learning

Plain version. Without any answers, we can still ask: which examples belong together, how can each example be described with fewer numbers, and which examples are strange?

Clustering

Clustering groups similar examples. The classic algorithm is **k-means** [15]. Choose the number of groups K. Start with K center points, then repeat two steps until nothing changes: assign each example to its nearest center, and move each center to the mean of the examples assigned to it. Each step can only lower the total squared distance from examples to their centers,

$$\sum_{i=1}^{m} \|x^{(i)} - \mu_{c^{(i)}}\|^2,$$

where μ_k is the center of cluster k and $c^{(i)}$ is the cluster that example i is assigned to. So the algorithm always stops, but possibly at a poor local solution, which is why it is run from several

random starts. Jain's review of fifty years of clustering research [15] stresses that there is no best clustering algorithm in general: the "right" groups depend on what similarity means for the data.

A common confusion: **k-nearest neighbors** (KNN) is *not* a clustering method. KNN classifies a new example by a vote among the k most similar *labeled* training examples, so it is a supervised method [5]. The "k" in k-means counts clusters; the "k" in KNN counts neighbors.

Dimensionality reduction

Dimensionality reduction describes each example with fewer numbers while keeping as much useful information as possible. **Principal component analysis** (PCA) finds the directions along which the data varies most and projects the data onto the top few of them; the new coordinates are uncorrelated and successively capture as much variance as possible [16]. Two PCA coordinates can turn 64-pixel digit images into points on a plane that we can look at (Figure 3, middle).

Anomaly detection

Anomaly detection finds examples that do not fit the pattern of the rest: a fraudulent transaction, a defective part, a corrupted image. Chandola, Banerjee and Kumar's survey [17] organizes the many techniques by the assumption each one makes about what "normal" looks like. A simple version uses PCA: fit a few components on the data, rebuild each example from them, and flag those with a large reconstruction error, because the components only describe what is common.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

X, y = load_digits(return_X_y=True)             # y is used only to check, never to fit

# Clustering: ten groups, found without looking at y.
km = KMeans(n_clusters=10, n_init=10, random_state=0).fit(X)
purity = sum(np.bincount(y[km.labels_ == k]).max() for k in range(10)) / len(y)
print("cluster purity:", round(purity, 3))      # 0.792: most clusters are one digit

# Dimensionality reduction: 64 numbers per image -> 2 numbers.
Z = PCA(n_components=2).fit_transform(X)        # shape (1797, 2)

# Anomaly detection: images that a 20-component PCA rebuilds badly are unusual.
pca20 = PCA(n_components=20).fit(X)
err = ((X - pca20.inverse_transform(pca20.transform(X))) ** 2).sum(axis=1)
print("most unusual images:", np.argsort(err)[-5:])
```

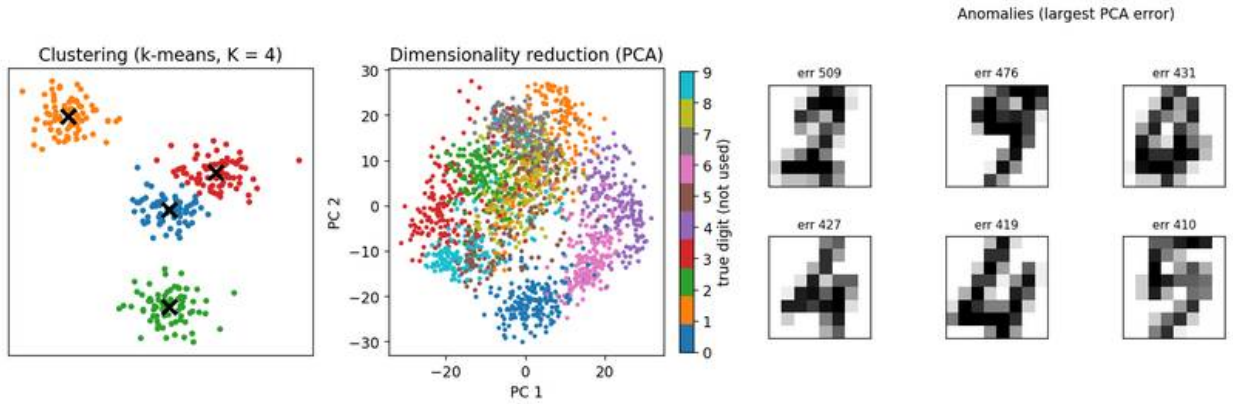


Figure 3 — Unsupervised learning. Left: k-means groups points around K centers. Middle: PCA compresses each 64-pixel digit to two numbers (colors show the true digits, which PCA never saw). Right: the digits with the largest PCA reconstruction error are the oddest-looking ones.

The purity check in the snippet uses the labels only *after* clustering, to see whether the discovered groups match the digits. About 79% of the images fall in a cluster dominated by their own digit, even though k-means never saw a label.

The vocabulary of a learning problem

Plain version. Before any math, we need names for the parts: what goes in, what comes out, what the right answer is, what the model is, and how we measure its mistakes.

Inputs, outputs and targets

- **Features** (inputs), x: the numbers that describe one example. For a digit image, the 64 pixel values; for a house, its size and number of rooms. With n features we write $x = (x_1, \dots, x_n)$, and x_j is the j-th feature.
- **Target**, y: the correct output, also called the label or **ground truth**. Only supervised learning has targets.
- **Prediction**, $\hat{y}$ ("y-hat"): what the model outputs for an input. The hat means "estimated".
- **Training example** i: the pair $(x^{(i)}, y^{(i)})$. The superscript in parentheses is an index, not a power. $x_j^{(i)}$ is feature j of example i.

The model and its parameters

The **model** is the function that maps features to a prediction. We write it $f_{w,b}$ to show that it depends on adjustable numbers called **parameters**, here w (the **weights**) and b (the **bias**):

$$\hat{y} = f_{w,b}(x).$$

Choosing the *form* of f (a line, a curve, a neural network) is a design decision. Choosing the *values* of w and b is what learning does. Numbers we set by hand, such as the learning rate or polynomial degree, are called **hyperparameters** to distinguish them from the learned parameters.

Cost function

A **loss** $L(\hat{y}, y)$ measures how wrong one prediction is. The **cost function** $J(w, b)$ averages the loss over the training set, so it is a single number that says how wrong the whole model is for a given choice of parameters. The model is f; the cost is J. Keeping these two apart avoids a lot of confusion: $f_{w,b}(x)$ takes an input and returns a prediction, while $J(w, b)$ takes parameters and returns an error.

The **objective** of training is then precise: find the parameters that make J as small as possible, so that $\hat{y}^{(i)}$ is close to $y^{(i)}$ for all training examples.

Training, validation and test sets

The data is split into three parts with different jobs:

- the **training set** is used to fit the parameters;
- the **validation set** is used during development to compare models and choose hyperparameters;
- the **test set** is used once, at the end, to estimate how the final model will do on new data.

The test set must not influence any decision, or the estimate becomes optimistic. We come back to *why* this split matters in the section on overfitting.

Linear regression

Plain version. Fit a straight line through the data. The line has two knobs: its slope and its height. Learning means turning the knobs until the line passes as close as possible to the points.

The model

With one feature, the linear regression model is

$$f_{w,b}(x) = wx + b,$$

where x is the feature, w is the slope (how much the prediction changes when x grows by 1) and b is the intercept (the prediction when $x = 0$). Changing w tilts the line; changing b moves it up and down (Figure 4, left).

With n features, the same model is a weighted sum,

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b = \sum_{j=1}^{n} w_j x_j + b,$$

where $\mathbf{w} = (w_1, \dots, w_n)$ holds one weight per feature. A common trick appends a constant feature $x_0 = 1$, so the bias becomes just another weight w_0 and the model shortens to $\mathbf{w} \cdot \mathbf{x}$. Either way, "linear" means linear in the *parameters*; we will later feed it squared or cubed features and it remains a linear regression.

The cost function: mean squared error

For each example, the **error** (residual) is $f_{w,b}(x^{(i)}) - y^{(i)}$. Squaring makes errors positive and punishes big ones more than small ones. Averaging over the m training examples gives the **mean squared error** (MSE) cost:

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2,$$

where m is the number of training examples and $(x^{(i)}, y^{(i)})$ is the i-th example. The extra factor $\frac{1}{2}$ is a convenience: it cancels the 2 that appears when we differentiate the square, and it does not change where the minimum is.

Picture the cost as a landscape over the parameters. If we fix b and vary only w, J is a parabola: lines that are too flat or too steep both have large error, and one slope in between is best (Figure 4, middle). With both w and b, J is a bowl, which we draw as a contour map of ellipses (Figure 4, right). The best line sits at the bottom of the bowl.

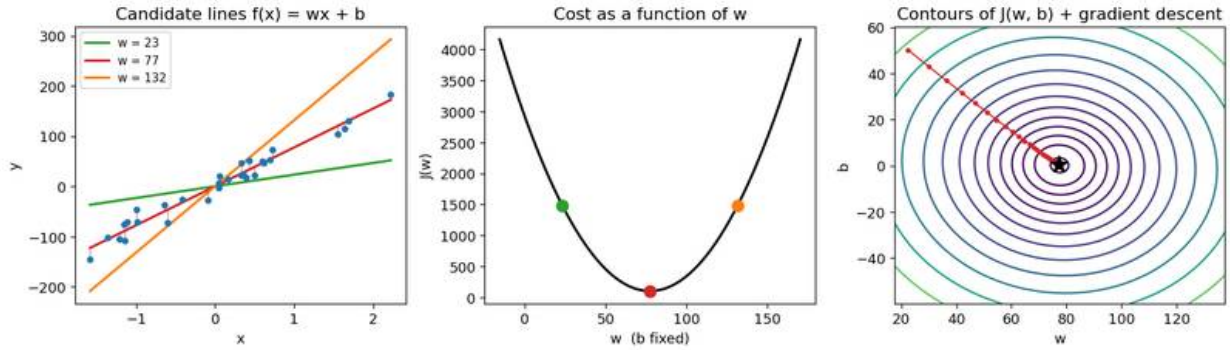


Figure 4 — From lines to costs. Left: three candidate lines; vertical segments are the residuals of the best one. Middle: with b fixed, $J(w)$ is a parabola, and each line is one point on it. Right: $J(w, b)$ is a bowl (contours), and gradient descent walks downhill to its bottom.

Why this bowl has one bottom

For linear regression, the MSE cost is a **convex** function of (w, b): any straight segment between two points on its surface lies on or above the surface. A convex function has no separate local minima; every minimum is the global one. This is a special, friendly case. For many other models, including the neural networks of ML 02, the cost landscape has many valleys, and an optimizer can stop in one that is not the lowest. The original notes make exactly this point: in general, the best we can hope for is a good local minimum, close to the global one.

For linear regression we could even solve for the bottom directly with a formula (the normal equations, which `np.polyfit` uses). We use gradient descent anyway, because it is the method that still works when no formula exists.

Gradient descent

Plain version. Stand somewhere on the cost landscape in fog. Feel which way the ground slopes, take a small step downhill, and repeat. When the ground is flat, you are at the bottom.

The update rule

The **derivative** $\partial J/\partial w$ tells us how J changes when w increases a little: positive means "uphill to the right", negative means "uphill to the left". Its sign gives the direction, and its size says how steep the slope is. Gradient descent moves each parameter a small step *against* its derivative:

$$w \leftarrow w - \alpha \frac{\partial J(w,b)}{\partial w},$$
$$b \leftarrow b - \alpha \frac{\partial J(w,b)}{\partial b},$$

where $\alpha > 0$ is the **learning rate** (step size) and $\leftarrow$ means "replace with". Both derivatives are computed at the *current* (w, b) before either is changed; this is called a **simultaneous update**. The vector of all partial derivatives is the **gradient** ∇J, which points in the direction of steepest increase, so the update moves in the direction of steepest decrease [3].

For the MSE cost of linear regression, the chain rule gives

$$\frac{\partial J}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)x^{(i)}, \qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right).$$

Read them in words: the gradient for w is the average error times the feature; the gradient for b is the average error. If predictions are too high on average, b goes down. If predictions are too high mainly where x is large, w goes down. With n features, $\partial J/\partial w_j$ uses $x_j^{(i)}$ in place of $x^{(i)}$.

Implementing it from scratch

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=100, n_features=1, noise=10.0, random_state=0)
x = X[:, 0]                                  # one feature, shape (m,)
m = len(x)

def cost(w, b):
    return ((w * x + b - y) ** 2).sum() / (2 * m)

w, b, alpha = 0.0, 0.0, 0.1
for step in range(200):
    err = w * x + b - y                      # f_wb(x_i) - y_i for every i
    dj_dw = (err * x).sum() / m
    dj_db = err.sum() / m
    w, b = w - alpha * dj_dw, b - alpha * dj_db   # simultaneous update
    if step % 50 == 0:
        print(f"step {step:3d}  J = {cost(w, b):8.2f}")

print("gradient descent:", round(w, 3), round(b, 3))   # 42.619 -0.814
print("closed form     :", np.polyfit(x, y, 1).round(3))  # [42.619 -0.814]
```

The cost drops from about 800 to 57 within 50 steps and then stops changing. Gradient descent and the exact formula agree to three decimals. The remaining cost is not zero, because the data contains noise that no straight line can explain.

Convergence: where the derivative is zero

"Repeat until convergence" needs a definition. At a minimum, the landscape is flat, so the **derivative is zero** there, and the update $w - \alpha \cdot 0$ leaves w unchanged. Note that it is the *derivative*, not the cost, that becomes zero: the cost at the minimum is just the smallest value available, 57 in the example above. In practice we stop when the cost decreases by less than a small tolerance between iterations, or when the gradient's size falls below a threshold, or after a fixed budget of steps.

A useful consequence: as we approach a minimum, the derivative shrinks, so even with a *fixed* α the steps $\alpha\, \partial J/\partial w$ get smaller automatically. Gradient descent naturally slows down near the bottom.

Choosing the learning rate

The learning rate is the most important hyperparameter of gradient descent (Figure 5):

- **Too small:** every step is tiny. The algorithm does go downhill, but it may need a very large number of steps.
- **About right:** the cost falls quickly and settles.
- **Too large:** a step jumps past the bottom to the other side of the valley, possibly higher than where it started. The parameters bounce back and forth, may never settle (**overshoot**), and can fly off to infinity (**divergence**).

The standard diagnostic is a **learning curve**: plot J against the iteration number. With a good α, J decreases at every step. If J ever goes *up*, α is too large (or there is a bug in the gradient). A practical recipe is to try values spaced by about a factor of 3, such as 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, and pick the largest one whose curve decreases smoothly.

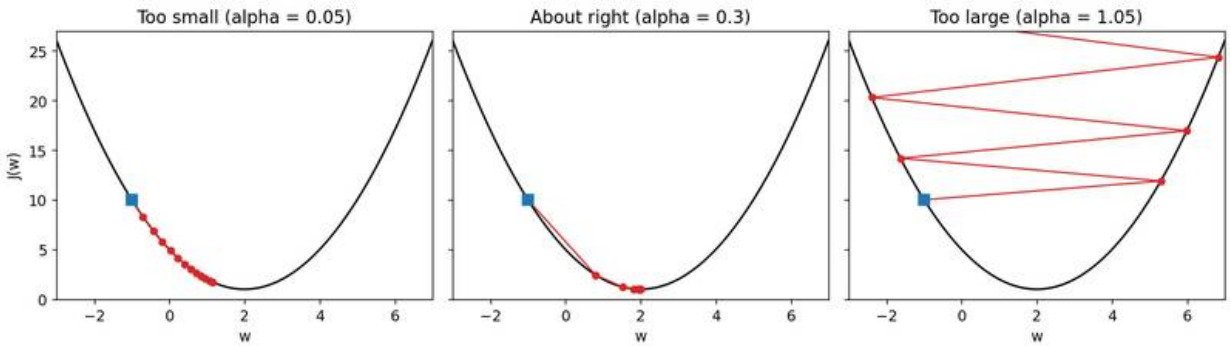


Figure 5 — The same cost $J(w)$, starting from the same point, with three learning rates. Too small: slow progress. About right: fast convergence. Too large: each step overshoots further, and the iterates diverge.

Many optimizers adapt the step size during training, either by a **schedule** that lowers α over time or by scaling each parameter's step using running statistics of its past gradients, as Adam does [19]. These make "the steps become smaller near the minimum" an explicit design choice rather than just a side effect.

Batch, mini-batch and stochastic gradient descent

The gradient formula sums over all m examples. That is fine for 100 examples and a problem for 100 million. The key insight, explained clearly in [3] and analyzed in depth by Bottou, Curtis and Nocedal [18], is that the gradient is an *average*, that is, an expectation over the data, and an average can be estimated from a sample.

- **Batch gradient descent** uses the *whole* training set for every update. Each step is exact and expensive.
- **Mini-batch gradient descent** shuffles the data and, for each update, uses a small random subset (a **mini-batch**) of B examples, typically 32 to a few hundred. Each step is a noisy but unbiased estimate of the full gradient, and costs B/m as much.
- **Stochastic gradient descent** (SGD), in the strict sense, uses one example per update ($B = 1$). In practice, "SGD" often refers to the mini-batch version too.

One full pass through the training data is an **epoch**. Beware of naming in code: the `batch_size` argument in deep-learning libraries means the *mini-batch* size, not the batch-gradient-descent "whole set".

The noise in mini-batch gradients is not only tolerable but often helpful: in the same number of epochs, mini-batch descent makes many more updates than batch descent. Bottou et al. argue that this is why stochastic methods dominate large-scale learning: they reach a useful accuracy with far less computation than exact methods [18].

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=10_000, n_features=5, noise=10.0, random_state=0)
m, n = X.shape
rng = np.random.default_rng(0)

def minibatch_gd(batch_size, alpha=0.05, epochs=5):
    w, b = np.zeros(n), 0.0
    for epoch in range(epochs):
        order = rng.permutation(m)                    # shuffle once per epoch
        for start in range(0, m, batch_size):
            idx = order[start:start + batch_size]
            err = X[idx] @ w + b - y[idx]
```

```
            w -= alpha * X[idx].T @ err / len(idx)    # gradient estimated on the batch
            b -= alpha * err.mean()
    return ((X @ w + b - y) ** 2).mean() / 2

for bs in [m, 64, 1]:                                 # batch, mini-batch, stochastic
    print(f"batch size {bs:5d}: J = {minibatch_gd(bs, alpha=0.05 if bs > 1 else 0.005):.2f}")
# batch size 10000: J = 6127.00   (only 5 updates in 5 epochs)
# batch size    64: J = 49.93     (785 updates)
# batch size     1: J = 50.64     (50,000 noisy updates)
```

With the same five passes over the data, batch descent has made only five steps and is still far from the bottom, while mini-batch descent has essentially converged (the noise level of this data puts the best achievable J near 50). Single-example SGD gets there too, but needs a smaller learning rate because its gradients are noisier, and it cannot use vectorized arithmetic efficiently.

Feature scaling

Plain version. If one feature is measured in thousands and another in fractions, the cost landscape becomes a long, narrow valley, and gradient descent zigzags. Putting all features on similar scales turns the valley into a round bowl that is easy to descend.

Why scale matters

Suppose feature x_1 (say, house area in square feet) ranges over thousands and feature x_2 (number of bedrooms) ranges from 1 to 5. A small change in w_1 changes the predictions a lot; a similar change in w_2 barely matters. So J is extremely steep along w_1 and very flat along w_2: the contours are thin ellipses. A learning rate small enough not to diverge along the steep direction makes almost no progress along the flat one (Figure 6, left).

Standardization and min–max scaling

The most common fix is **standardization** (z-score normalization):

$$x'_j = \frac{x_j - \mu_j}{\sigma_j},$$

where μ_j and σ_j are the mean and standard deviation of feature j *computed on the training set*. Each scaled feature then has mean 0 and standard deviation 1. An alternative is **min–max scaling**, $x'_j = (x_j - \min_j)/(\max_j - \min_j)$, which maps the training range to $[0, 1]$ and is the usual choice for pixel intensities (divide by 255 for 8-bit images). Both are available in scikit-learn as `StandardScaler` and `MinMaxScaler` [5].

Two rules prevent subtle bugs. Compute μ_j, σ_j on the training set only, then apply the *same* numbers to validation, test and future data. And remember that scaling changes the meaning of the learned weights: a weight now says how much the prediction changes per standard deviation of the feature.

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=200, n_features=2, noise=5.0, random_state=1)
X = X * np.array([1000.0, 1.0])               # feature 1 now has a ~1000x wider range

def gd(X, y, alpha, steps=500):
    m, n = X.shape
    w, b = np.zeros(n), 0.0
    for _ in range(steps):
        err = X @ w + b - y
        w -= alpha * X.T @ err / m
        b -= alpha * err.mean()
    return ((X @ w + b - y) ** 2).mean() / 2

print("raw,    alpha=1e-7:", round(gd(X, y, 1e-7), 2))    # 158.01; alpha=3e-6 diverges
mu, sigma = X.mean(axis=0), X.std(axis=0)
Xs = (X - mu) / sigma                          # z-score standardization
print("scaled, alpha=0.1 :", round(gd(Xs, y, 0.1), 2))    # 13.47
```

On the raw features, any learning rate above about 2×10^{-6} diverges, and with a safe one the cost is still stuck near 158 after 500 steps (it is still near 158 after 5,000), because the second weight barely moves. After standardization, $\alpha = 0.1$ reaches the minimum (about 13, the noise level) easily.

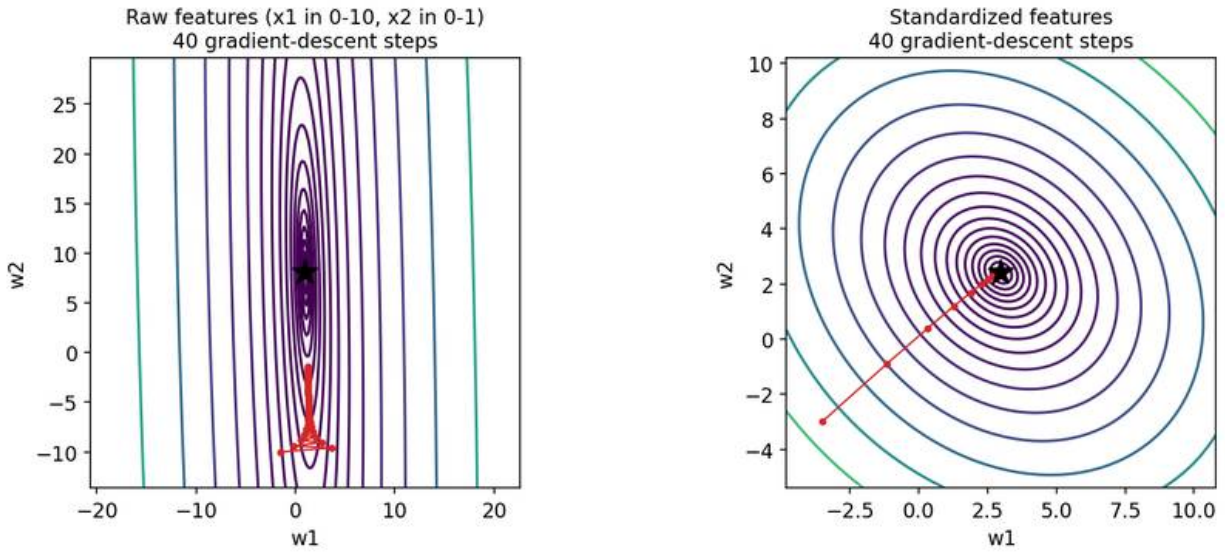


Figure 6 — Feature scaling reshapes the cost landscape. Left: features on very different scales give a narrow

valley, and gradient descent zigzags. Right: after standardization, the contours are nearly circular, and the same number of steps reaches the minimum.

Logistic regression for classification

Plain version. Keep the weighted sum from linear regression, but squash its output into the range 0 to 1 so it can be read as a probability. Then choose the weights so that the probability is high for the true class.

The sigmoid function

The **sigmoid** (logistic) function is

$$g(z) = \frac{1}{1 + e^{-z}},$$

where z is any real number. It maps $-\infty$ to 0, $+\infty$ to 1 and 0 to exactly 0.5, with an S-shaped curve in between (Figure 7, left). It is also symmetric, $g(-z) = 1 - g(z)$, and has the convenient derivative $g'(z) = g(z)\,(1 - g(z))$.

The model and the decision boundary

Logistic regression feeds the linear model into the sigmoid:

$$f_{\mathbf{w},b}(\mathbf{x}) = g(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}\cdot\mathbf{x}+b)}},$$

and interprets the output as the estimated probability that the label is 1, $f_{\mathbf{w},b}(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$. Despite the name, it is a classification method. To make a decision, predict 1 when this probability is at least 0.5. Since $g(z) \geq 0.5$ exactly when $z \geq 0$, the rule is

$$\hat{y} = 1 \quad \text{if} \quad \mathbf{w} \cdot \mathbf{x} + b \geq 0, \qquad \hat{y} = 0 \text{ otherwise.}$$

The set where $\mathbf{w} \cdot \mathbf{x} + b = 0$ is the **decision boundary**. With two features it is a straight line; in general it is a flat hyperplane, with $\mathbf{w}$ perpendicular to it. Points far from the boundary get probabilities near 0 or 1; points near it get probabilities near 0.5. To obtain a curved boundary, feed the model nonlinear features such as x_1^2 or $x_1 x_2$; the model stays linear in its parameters.

The cost: cross-entropy (log loss)

Why not reuse squared error? Combined with the sigmoid, the squared-error cost is *not* convex in $(\mathbf{w}, b)$: it develops flat plateaus and multiple valleys. Instead, logistic regression uses the **log**

loss, also called **binary cross-entropy**:

$$L(f, y) = -y \log f \; - \; (1 - y) \log(1 - f),$$

where $f = f_{\mathbf{w},b}(\mathbf{x})$ is the predicted probability and $y \in \{0, 1\}$ is the true label. Only one of the two terms is active for each example. If $y = 1$, the loss is $-\log f$: zero when $f = 1$, and growing without bound as $f \to 0$. If $y = 0$, the loss is $-\log(1 - f)$, the mirror image (Figure 7, middle). Confident wrong answers are punished very heavily; confident right answers cost almost nothing.

The cost is the average over the training set:

$$J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \Big[- \, y^{(i)} \log f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - (1 - y^{(i)}) \log \left(1 - f_{\mathbf{w},b}(\mathbf{x}^{(i)})\right) \Big].$$

This choice is not arbitrary. If we treat each label as a coin flip with probability $f_{\mathbf{w},b}(\mathbf{x}^{(i)})$ of coming up 1, then J is exactly the negative log-likelihood of the training labels, divided by m. Minimizing it is **maximum likelihood estimation** [3]. And with the sigmoid, this cost is convex, so gradient descent again finds the single global minimum.

Gradient descent, again

Using $g' = g(1 - g)$, the derivative of the log loss simplifies remarkably:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}, \qquad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)} \right).$$

This has *exactly the same form* as the linear-regression gradient: average error times feature. The only difference is inside f. This is not a coincidence; it holds for a whole family of models that pair an output function with its matching likelihood, and it is one reason gradient descent code is so reusable.

```
import numpy as np
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=200, n_features=2, n_redundant=0,
                           n_clusters_per_class=1, class_sep=1.5, random_state=3)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def log_loss(p, y, eps=1e-12):
    p = np.clip(p, eps, 1 - eps)              # avoid log(0)
    return -(y * np.log(p) + (1 - y) * np.log(1 - p)).mean()

w, b, alpha = np.zeros(2), 0.0, 0.5
for step in range(1000):
    p = sigmoid(X @ w + b)                    # predicted P(y = 1 | x)
    w -= alpha * X.T @ (p - y) / len(y)       # same form as linear regression
```

```
    b -= alpha * (p - y).mean()

p = sigmoid(X @ w + b)
print("log loss:", round(log_loss(p, y), 3))  # 0.06
print("accuracy:", ((p >= 0.5) == y).mean())  # 0.99
print("boundary: %.2f*x1 + %.2f*x2 + %.2f = 0" % (w[0], w[1], b))
```

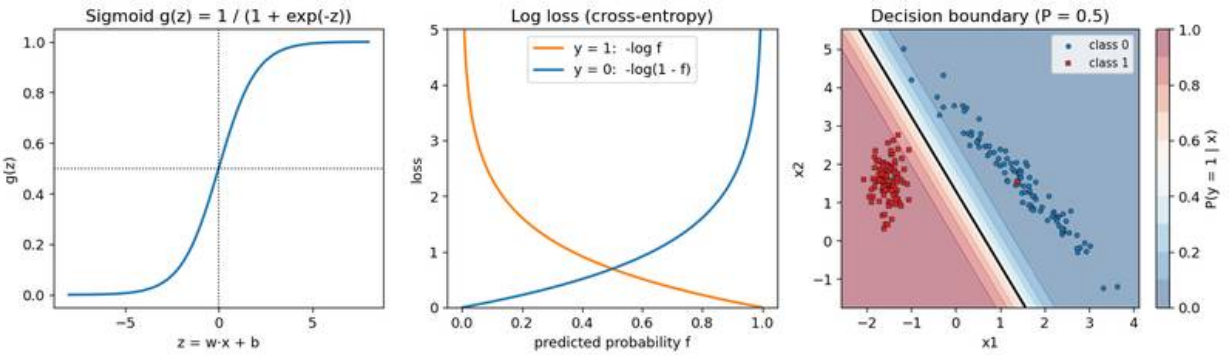


Figure 7 — Logistic regression. Left: the sigmoid squashes any score into $(0, 1)$. Middle: the log loss for $y = 1$ and $y = 0$ punishes confident mistakes heavily. Right: a model trained from scratch; shading is the predicted probability, and the black line is the decision boundary $w \cdot x + b = 0$.

More than two classes

For $K > 2$ classes, the standard generalization gives each class its own weight vector and score $z_k = \mathbf{w}_k \cdot \mathbf{x} + b_k$, and turns the scores into probabilities that sum to 1 with the **softmax** function, $P(y = k \mid \mathbf{x}) = e^{z_k} / \sum_{l=1}^{K} e^{z_l}$. The cost is again the average negative log probability of the true class (categorical cross-entropy) [3]. This is what scikit-learn's `LogisticRegression` did for the ten digits at the start of this tutorial, and it is exactly the output layer of most classification networks in ML 02.

Overfitting, underfitting and regularization

Plain version. A model that is too simple misses the pattern (**underfitting**). A model that is too flexible memorizes the training examples, noise included, and fails on new ones (**overfitting**). We detect the difference with data the model did not train on, and we tame an over-flexible model by penalizing large weights.

Seeing it happen

Take 25 noisy samples of a smooth curve and fit polynomials of different degrees, which is linear regression on the features $x, x^2, \ldots, x^d$ (Figure 8):

- **Degree 1 (underfitting, high bias).** A straight line cannot follow the curve. The error is large on the training set *and* on new data.
- **Degree 3 (a good fit).** The curve follows the trend and ignores the noise. Errors are small on both.
- **Degree 15 (overfitting, high variance).** The curve passes almost exactly through every training point, so the training error is nearly zero. Between and beyond the points it swings wildly, and the error on new data explodes.

The training error alone cannot tell the good model from the overfitted one; it even prefers the overfitted one. Only data that the model did not see can reveal the difference. This is why we keep a validation set.

Using the validation and test sets

The workflow is:

1. Fit each candidate model (each degree, each regularization strength) on the **training set**.
2. Compare the candidates on the **validation set** and keep the best one.
3. Report the chosen model's error on the **test set**, once.

Why three sets and not two? Because picking the best of many candidates on the validation set is itself a form of fitting: the winner is partly the one that got lucky on those particular examples.

The untouched test set gives an unbiased final estimate. When data is scarce, **cross-validation** replaces the single validation set: split the training data into k folds, train k times, each time validating on a different fold, and average.

The pattern to look for: if training and validation errors are both high, the model underfits (use a more flexible model or better features). If training error is low but validation error is much higher, the model overfits (get more data, simplify the model, or regularize).

L2 regularization

Regularization keeps a flexible model but discourages extreme parameter values. The wild curve of the degree-15 fit needs huge, finely balanced weights. **L2 regularization** adds a penalty on the

squared size of the weights to the cost:

$$J_{\text{reg}}(\mathbf{w}, b) = J(\mathbf{w}, b) + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2,$$

where J is the original cost (MSE or log loss), $\lambda \geq 0$ is the **regularization strength** (a hyperparameter), and the sum runs over the weights but conventionally not the bias b. With $\lambda = 0$ we recover the original model; as λ grows, the weights are pulled toward zero and the fitted function becomes smoother; too large a λ underfits. For linear regression, this is **ridge regression**, introduced by Hoerl and Kennard to stabilize least-squares estimates when features are strongly correlated [20].

The gradient-descent update shows what the penalty does:

$$w_j \leftarrow w_j \left(1 - \alpha \frac{\lambda}{m}\right) - \alpha \frac{\partial J}{\partial w_j}.$$

Before each ordinary step, every weight is multiplied by a number slightly less than 1. This is why L2 regularization is also called **weight decay** in neural-network training [3].

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(0)
x = rng.uniform(0, 1, 25)
y = np.sin(2 * np.pi * x) + rng.normal(0, 0.25, 25)      # true curve + noise
X = x[:, None]

# 60% train, 20% validation, 20% test
X_tmp, X_te, y_tmp, y_te = train_test_split(X, y, test_size=0.2, random_state=0)
X_tr, X_va, y_tr, y_va = train_test_split(X_tmp, y_tmp, test_size=0.25, random_state=0)

def fit(degree, lam=0.0):
    reg = Ridge(alpha=lam) if lam > 0 else LinearRegression()   # sklearn calls lambda
    ↪  "alpha"
    model = make_pipeline(PolynomialFeatures(degree), StandardScaler(), reg)
    return model.fit(X_tr, y_tr)

for degree, lam in [(1, 0), (3, 0), (15, 0), (15, 1e-2)]:
    model = fit(degree, lam)
    tr = mean_squared_error(y_tr, model.predict(X_tr))
    va = mean_squared_error(y_va, model.predict(X_va))
    print(f"degree {degree:2d}, lambda {lam:g}: train MSE {tr:.3f}, val MSE {va:.3g}")
```

```
best = fit(15, 1e-2)                                    # chosen on validation only
print("test MSE:", round(mean_squared_error(y_te, best.predict(X_te)), 3))
```

The output tells the whole story: degree 1 has train/validation MSE of 0.144/0.292 (underfit), degree 3 has 0.031/0.052 (good), and unregularized degree 15 has a training MSE of 0.003 but a validation MSE in the millions, because some validation points fall in wide gaps between training points (for example between $x \approx 0.27$ and $x \approx 0.61$), where the polynomial swings wildly. Adding a mild L2 penalty ($\lambda = 0.01$) to the same degree-15 model gives 0.041/0.035, and its test MSE is 0.075.

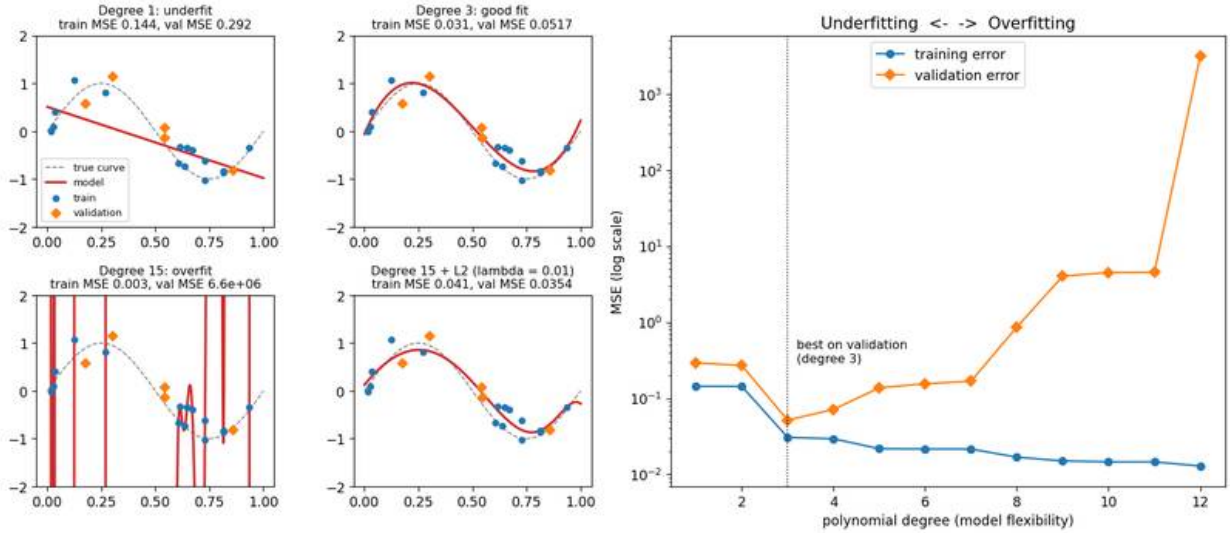


Figure 8 — Underfitting, a good fit, overfitting, and overfitting tamed by L2 regularization (training points in blue, validation points in orange). Right: as the degree grows, training error keeps falling while validation error falls and then rises; the bottom of the validation curve is the model to choose.

Evaluating a model

Plain version. A single number never tells the whole story. Choose a metric that matches what a mistake costs, and always compare against a trivial baseline.

Regression metrics

For regression, the most common metric is the MSE, $\frac{1}{m}\sum_i(\hat{y}^{(i)} - y^{(i)})^2$, here without the $\frac{1}{2}$ that was only a training convenience. Its square root, the **RMSE**, is in the same units as y and is easier to interpret ("off by about 3 points on average"). The **mean absolute error**, $\frac{1}{m}\sum_i |\hat{y}^{(i)} - y^{(i)}|$, is less sensitive to a few large errors [5].

Classification metrics

For classification, **accuracy** is the fraction of correct predictions. It is easy to understand and easy to be fooled by. In the digits data, about 10% of the images are 9s. A "model" that always answers "not a 9" is 90% accurate and completely useless.

The **confusion matrix** counts the four outcomes for a binary problem, with "positive" meaning the class we care about:

	predicted positive	predicted negative
actually positive	true positive (TP)	false negative (FN)
actually negative	false positive (FP)	true negative (TN)

From it,

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}.$$

Precision answers "when the model says positive, how often is it right?" **Recall** answers "of all the real positives, how many did the model find?" They trade off: lowering the 0.5 decision threshold of logistic regression finds more positives (higher recall) but raises more false alarms (lower precision). Which matters more depends on the cost of each mistake: a cancer screen should not miss tumors (high recall), while a spam filter should not hide real mail (high precision). The **F1 score**, the harmonic mean of precision and recall, combines both into one number [5].

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

X, y = load_digits(return_X_y=True)
y = (y == 9).astype(int)                  # "is it a 9?": only about 10% positives
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

lazy = np.zeros_like(y_te)                # always answers "not a 9"
model = LogisticRegression(max_iter=5000).fit(X_tr, y_tr)
pred = model.predict(X_te)

print("lazy accuracy :", round(accuracy_score(y_te, lazy), 3), " recall:",
 ↪  recall_score(y_te, lazy))
print("model accuracy:", round(accuracy_score(y_te, pred), 3),
      " precision:", round(precision_score(y_te, pred), 3),
      " recall:", round(recall_score(y_te, pred), 3))
print(confusion_matrix(y_te, pred))       # rows: true 0/1, columns: predicted 0/1
# lazy accuracy : 0.9   recall: 0.0
# model accuracy: 0.974  precision: 0.87  recall: 0.87
```

The lazy baseline reaches 90% accuracy with zero recall. The real model's 97.4% accuracy sounds only a little better, but its recall of 0.87 shows that it actually finds most of the 9s. Always report the baseline, and always look past accuracy when classes are imbalanced. The same metrics reappear, with more care about thresholds and multiple classes, in the DIP chapter on pattern classification.

Modern view

The mechanics in this tutorial (a parameterized model, a cost, gradient-based optimization, and held-out evaluation) have not changed in decades, and every system discussed below still uses them. What has changed is *where the features come from*, *where the labels come from*, and *how large the models are*. A handful of reviews map these shifts well.

Machine learning as a field. Jordan and Mitchell's review in *Science* [7] frames machine learning around a single question, how to build computers that improve automatically through experience, and places it at the intersection of computer science and statistics, at the core of artificial intelligence and data science. They attribute its rapid spread across science, medicine, manufacturing, education and finance to three things arriving together: new learning algorithms and theory, an explosion in the amount of available data, and cheap computation. That framing is a useful antidote to hype: the method is only as good as the data and the experience it is given.

From feature engineering to representation learning. For a long time, the success of a classical model depended mostly on hand-designed features, the "count the inked columns" step of Figure 1 done by experts: edge histograms, texture descriptors, keypoint descriptors (see IP 12). Bengio, Courville and Vincent [21] argue that this dependence is a weakness, because the right representation is what makes a task easy or hard, and propose learning the representation itself from data. Their review covers the tools for this (probabilistic models, autoencoders, manifold learning and deep networks) and the general priors that make a representation good, such as disentangling the separate explanatory factors behind the data. In the language of this tutorial, a deep network is still "features followed by logistic regression", except that the features are produced by earlier layers whose weights are learned by the same gradient descent. That is the subject of ML 02.

Self-supervised learning. Learning representations needs data, and labels are the expensive part. Jing and Tian's survey of self-supervised visual feature learning [22] sorts the label-free "pretext tasks" into four families: generation-based (for example, reconstructing or colorizing images), context-based (exploiting spatial or temporal structure, such as the arrangement of image patches), free-semantic-label-based (using labels that come automatically, for example from rendering engines or classical methods), and cross-modal (checking whether two signals, such as video and audio, belong together). It also sets out the evaluation protocol that is still standard: judge a representation by how well it transfers to downstream tasks such as classification, detection and segmentation. The later survey by Gui et al. [23] reorganizes the field around context-based methods, contrastive learning (with negative examples, self-distillation, or feature decorrelation), generative methods dominated by masked image modeling, and hybrids of contrastive and generative objectives, reflecting the shift toward masked prediction after masked autoen-

coders [12]. For practitioners, Balestriero et al.'s *Cookbook of Self-Supervised Learning* [24] groups modern methods into a deep metric learning family, a self-distillation family, a canonical correlation analysis family and masked image modeling, and is candid about how much the outcome depends on details such as data augmentation and the small "projector" network used during training.

Foundation models. The report by Bommasani et al. [10] named the resulting paradigm: models trained on broad data at scale, usually self-supervised, then adapted to many downstream tasks. It highlights two consequences. **Emergence**: capabilities such as following instructions from a few examples in the prompt [11] appear with scale rather than being designed in. **Homogenization**: many applications are now adapted from the same few base models, which gives great leverage but means that a defect in the base model is inherited by everything built on it. The report also stresses how limited our understanding is of how these models work and when they fail.

Reinforcement learning from human feedback. RL has moved from games to the final stage of training language models [13], [14]. Casper et al. [25] survey what can go wrong. They follow the three steps of RLHF (collecting human feedback, fitting a reward model, and optimizing the policy) and catalog problems at each: feedback from people is noisy, inconsistent and can be manipulated; a reward model is only an imperfect proxy for what people want; and a policy optimized hard against that proxy can exploit its errors. They separate problems that better methods within RLHF could solve from fundamental limitations that would require a different approach, and they argue for transparency and for combining RLHF with other safety measures rather than relying on it alone.

Optimization at scale. Bottou, Curtis and Nocedal [18] explain why the humble stochastic gradient method, and not more sophisticated optimizers from classical numerical analysis, became the workhorse of machine learning: when data is plentiful, a cheap noisy step beats an expensive exact one. Adaptive methods such as Adam [19] add per-parameter step sizes on top. Every model mentioned in this section, however large, is trained with a descendant of the mini-batch loop in this tutorial.

Open problems. Despite the scale, several questions from this beginner's tutorial remain the hard ones. *Generalization*: very large models fit their training data almost perfectly yet still generalize, which does not match the simple overfitting picture of Figure 8, and why this happens is not fully understood. *Evaluation*: when a model is trained on a large fraction of the public internet, it is hard to guarantee that the test set was not in the training data, so a clean train/test separation, the foundation of honest evaluation, is difficult to verify. *Objectives*: the cost function is the only thing the model optimizes, and RLHF shows how hard it is to write a cost that captures what people actually want. And *data*: what a model learns is bounded by the data and labels it sees, with all their gaps and biases.

For computer vision specifically, the classical route (hand-made features plus a simple classifier) is developed in the DIP chapter on pattern classification, and the learned route continues in ML 02 and Deep Learning for Image Processing.

Key takeaways

- Machine learning replaces hand-written rules with a flexible model whose parameters are fitted to examples; it shines when the rule is hard to state but examples are easy to get.
- The paradigms differ in feedback: supervised (right answers; regression for numbers, classification for categories), unsupervised (no answers; clustering, dimensionality reduction, anomaly detection), self-supervised (answers made from the data itself), and reinforcement learning (rewards for actions). KNN is supervised; k-means is clustering.
- Keep the model $f_{w,b}(x)$ and the cost $J(w,b)$ apart: the model maps inputs to predictions; the cost maps parameters to an error. Training means minimizing J.
- Gradient descent repeats $w \leftarrow w - \alpha\, \partial J/\partial w$. At a minimum the *derivative* is zero. The learning rate must be small enough to avoid overshooting and large enough to make progress; mini-batches make each step cheap.
- Scale features before gradient descent, using statistics from the training set only.
- Logistic regression applies a sigmoid to a linear score and minimizes cross-entropy; its decision boundary is $\mathbf{w} \cdot \mathbf{x} + b = 0$, and its gradient has the same form as linear regression's.
- Judge models on data they did not train on: choose with the validation set, report once on the test set, regularize (L2) when the model overfits, and look beyond accuracy when classes are imbalanced.

Exercises

1. **A derivative by hand.** For the one-feature MSE cost $J(w,b) = \frac{1}{2m}\sum_i (wx^{(i)} + b - y^{(i)})^2$, derive $\partial J/\partial w$ and $\partial J/\partial b$. Then, for the three points $(1,2)$, $(2,4)$, $(3,6)$ starting at $w = 0, b = 0$, compute one gradient-descent step with $\alpha = 0.1$.

   Hint

   The chain rule on each square gives $2(wx^{(i)}+b-y^{(i)})\cdot x^{(i)}$, and the 2 cancels the $\frac{1}{2}$, giving the formulas in the gradient-descent section. At $w = b = 0$ the errors are $-2, -4, -6$. So $\partial J/\partial w = (-2\cdot 1 - 4\cdot 2 - 6\cdot 3)/3 = -28/3$ and $\partial J/\partial b = -12/3 = -4$. After one step, $w = 0.1 \cdot 28/3 \approx 0.933$ and $b = 0.4$.

2. **Find the divergence threshold.** For $J(w) = \frac{1}{2m}\sum_i (wx^{(i)} - y^{(i)})^2$ (no bias), show that one gradient step multiplies the distance to the optimum w^* by $1 - \alpha s$, where $s = \frac{1}{m}\sum_i (x^{(i)})^2$. For which α does gradient descent converge? Check your answer with the `gd` function from the feature-scaling section.

   Hint

   $\partial J/\partial w = s\,(w - w^*)$ because J is a parabola with curvature s and minimum at w^*. So $w_{\text{new}} - w^* = (1-\alpha s)(w - w^*)$. It converges when $|1-\alpha s| < 1$, that is $0 < \alpha < 2/s$; at $\alpha = 1/s$ it lands on the minimum in one step. With several features, s is replaced by the largest eigenvalue of $\frac{1}{m}X^\top X$, which is why one huge-scale feature forces a tiny α.

3. **Regression for classification.** Generate 1-D data with `make_classification(n_features=1, n_informative=1, n_redundant=0, n_clusters_per_class=1)`, then add five extra examples of class 1 with very large x. Fit both `LinearRegression` (thresholding at 0.5) and `LogisticRegression`. Compare their accuracy before and after adding the extra points, and explain the difference.

   Hint

   The extra points are easy to classify, but the squared error still wants the line to pass near them. The line's slope drops, its 0.5 crossing moves toward class 1, and some ordinary class-1 points near the boundary become misclassified. The log loss of a correctly classified, far-away point is already close to 0, so logistic regression barely moves.

4. **Scaling the digits.** Train the from-scratch logistic regression of this tutorial on "9 versus not 9" with the digits data, once on raw pixels (0 to 16) and once on pixels divided by 16. Use the same number of steps and find the largest learning rate that works for each. Does scaling change the final accuracy, the speed, or both?

   Hint

   All pixels share one unit here, so the effect is milder than in Figure 6, but the raw inputs are 16 times larger, which multiplies the gradient's size and the landscape's curvature. The safe learning rate on raw pixels is roughly $16^2 = 256$ times smaller. With properly tuned learning rates, both reach similar accuracy; the scaled version is simply easier to tune.

5. **Choosing lambda.** Using the polynomial example, fix degree 15 and try $\lambda \in \{10^{-6}, 10^{-4}, 10^{-2}, 1, 100\}$. Plot training and validation MSE against λ on a log axis. Which end underfits, which overfits, and which λ would you pick? Why must you not use the test set to make this choice?

   Hint

   Tiny λ behaves like no regularization (overfitting: low training error, high validation error). Huge λ flattens the curve toward a constant (underfitting: both errors high). Pick the λ with the lowest validation error. If the test set guided the choice, it would no longer be an unbiased estimate of performance on new data.

6. **Which paradigm?** Classify each problem as supervised (regression or classification), unsupervised, self-supervised or reinforcement learning, and name the target or reward: (a) estimating the remaining battery life of a phone from usage logs with recorded outcomes; (b) grouping a million unlabeled product photos for browsing; (c) pre-training an image model by predicting the missing quarter of each photo; (d) teaching a robot arm to stack blocks with a score given only when the stack stands; (e) flagging unusual readings in a factory sensor stream with no labeled failures.

Hint

(a) Supervised regression; target: remaining hours. (b) Unsupervised clustering. (c) Self-supervised; the target is the hidden pixels, made from the data itself. (d) Reinforcement learning; reward: whether the stack stands, delayed until the end. (e) Unsupervised anomaly detection.

Continue through ML/DL

If the mathematics is new, take linear algebra, probability and information theory, and numerical computation before neural networks.

The rest of the track connects image filtering and CNNs with recurrent/recursive networks, transformers and representation learning. It then builds from factor models and autoencoders to VAEs, GANs, normalizing flows, diffusion/flow matching, autoregressive models and reusable foundation models. Finish with evaluation and deployment. For the statistical decision and inference viewpoint, follow Pattern Recognition; for task-specific systems, follow Deep Learning for Computer Vision.

References

1. Shyandram, "Introduction to Machine Learning," Medium post, 2024; updated 2026. link
2. A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM Journal of Research and Development*, vol. 3, no. 3, pp. 210–229, 1959. doi
3. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. book site
4. A. Ng, E. Shyu, A. Bagul and G. Ladwig, "Machine Learning Specialization," DeepLearning.AI and Stanford Online, Coursera. course page
5. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. link
6. E. Alpaydin and C. Kaynak, "Optical Recognition of Handwritten Digits," UCI Machine Learning Repository, 1998. doi
7. M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015. doi
8. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. book page
9. Y. LeCun and I. Misra, "Self-supervised learning: The dark matter of intelligence," Meta AI blog, 2021. link
10. R. Bommasani, D. A. Hudson, E. Adeli et al., "On the opportunities and risks of foundation models," arXiv:2108.07258, 2021. arXiv
11. T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 33, 2020. link
12. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), pp. 15979–15988, 2022. arXiv

13. L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright et al., “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 35, 2022. link
14. DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *Nature*, vol. 645, pp. 633–638, 2025. arXiv
15. A. K. Jain, “Data clustering: 50 years beyond K-means,” *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010. doi
16. I. T. Jolliffe and J. Cadima, “Principal component analysis: A review and recent developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, 20150202, 2016. doi
17. V. Chandola, A. Banerjee and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, 2009. doi
18. L. Bottou, F. E. Curtis and J. Nocedal, “Optimization methods for large-scale machine learning,” *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. doi
19. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2015. arXiv
20. A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970. doi
21. Y. Bengio, A. Courville and P. Vincent, “Representation learning: A review and new perspectives,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798–1828, 2013. doi
22. L. Jing and Y. Tian, “Self-supervised visual feature learning with deep neural networks: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 11, pp. 4037–4058, 2021. doi
23. J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo and D. Tao, “A survey on self-supervised learning: Algorithms, applications, and future trends,” arXiv:2301.05712, 2023. arXiv
24. R. Balestriero, M. Ibrahim, V. Sobal, A. Morcos, S. Shekhar et al., “A cookbook of self-supervised learning,” arXiv:2304.12210, 2023. arXiv
25. S. Casper, X. Davies, C. Shi et al., “Open problems and fundamental limitations of reinforcement learning from human feedback,” *Transactions on Machine Learning Research*, 2023. TMLR ·arXiv

ML 01A • Linear Algebra for Learning

Vectors, matrices and tensors, norms, eigendecomposition, the SVD, the pseudo-inverse, trace and determinant, and a step-by-step derivation of PCA, with tested NumPy code.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-01a-linear-algebra/

What you'll learn

- How scalars, vectors, matrices and tensors describe images, batches and learned transformations, and how to keep their shapes straight.
- Four ways to read a matrix product, and what identity, inverse, span, rank and null space mean for a system of equations.
- How norms measure size, how special matrices (diagonal, symmetric, orthogonal) simplify computation, and how dot products define similarity.
- What eigendecomposition and the singular value decomposition (SVD) reveal about a matrix, and why the SVD gives the best low-rank approximation.
- How the Moore–Penrose pseudo-inverse solves least-squares and under-determined problems, and why conditioning and ridge regularization matter.
- How the trace and determinant work, and how to derive principal component analysis (PCA) from scratch using only these tools.

The big picture

Almost every operation inside a neural network is a linear map followed by a simple nonlinear function. A fully connected layer multiplies by a weight matrix. A convolution is a structured matrix product. Attention computes dot products between vectors and then mixes vectors with a matrix of weights. Linear algebra is the language for all of these. It tells us what a map keeps, what it throws away, what it stretches, and how to compute it without losing accuracy.

> **Explain like I'm five**
>
> A vector is a list of measurements, like the height, weight and age of one person. A matrix is a machine that takes one list and mixes it into a new list. Some machines just turn the list around (rotation), some make some parts bigger and others smaller (stretching), and some squash parts flat so the information is gone forever. Linear algebra is the instruction manual for these machines: it tells you which way each machine stretches, by how much, and whether you can undo it.

This chapter follows the "Applied Math" part of Goodfellow, Bengio and Courville's *Deep Learning* [1] and the linear algebra chapters of *Mathematics for Machine Learning* [2]. Strang's textbook [3] is the gentlest full course if you want more practice, and Golub and Van Loan [4] and Trefethen and

Bau [5] are the standard references for how these computations are done reliably on a computer. Read this chapter before ML 02 if matrix notation is unfamiliar. The companion chapters are ML 01B ·Probability and Information Theory and ML 01C ·Numerical Computation.

The chapter has a single thread. We start with objects and shapes, move to products and solving equations, then measure size with norms. Eigendecomposition and the SVD then explain the "personality" of a matrix. The pseudo-inverse, trace and determinant are tools built on top of them. Finally, we use every one of these tools to derive PCA, the classic method for compressing data, exactly as it is derived in [1].

Scalars, vectors, matrices and tensors

Plain version. Every number in machine learning lives in an array with named axes. Name every axis before you multiply anything.

A **scalar** is a single number, written in italics: $s \in \mathbb{R}$. A **vector** is an ordered list of n numbers, written in bold lowercase or plain lowercase in this chapter: $x \in \mathbb{R}^n$, with entries $x_1, \dots, x_n$. By convention a vector is a column. A **matrix** is a 2-D grid of numbers: $A \in \mathbb{R}^{m \times n}$ has m rows and n columns, and A_{ij} is the entry in row i and column j. A **tensor** is an array with any number of axes.

These objects map directly onto data:

- A grayscale image of height H and width W is a matrix in $\mathbb{R}^{H \times W}$. Flattened, it becomes a vector $x \in \mathbb{R}^{HW}$. (See Images as Arrays.)
- A dataset of n examples with d features each, stored one example per row, is a **design matrix** $X \in \mathbb{R}^{n \times d}$.
- A fully connected layer with weights $W \in \mathbb{R}^{d \times m}$ and bias $b \in \mathbb{R}^m$ maps the whole batch at once: $Y = XW + \mathbf{1}b^\top \in \mathbb{R}^{n \times m}$, where $\mathbf{1} \in \mathbb{R}^n$ is a vector of ones.
- An image batch in PyTorch is usually a 4-D tensor of shape $B \times C \times H \times W$ (batch, channels, height, width). A batch of token embeddings in a Transformer is usually $B \times N \times D$ (batch, sequence length, embedding width).

The **transpose** $A^\top$ swaps rows and columns: $(A^\top)_{ij} = A_{ji}$. It satisfies $(A^\top)^\top = A$ and $(AB)^\top = B^\top A^\top$. Note the reversed order.

Broadcasting

Writing $\mathbf{1}b^\top$ is mathematically honest but tedious, so libraries let you add a vector to every row of a matrix directly. This is called **broadcasting**: when two arrays have different shapes, axes of length 1 (or missing leading axes) are stretched to match. Broadcasting is convenient and dangerous,

because a wrong shape often still runs. The most common bug is subtracting a column vector of shape $(n, 1)$ from a flat vector of shape $(n,)$, which silently produces an $n \times n$ matrix instead of n residuals.

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(32, 64))      # batch of 32 examples, 64 features
W = rng.normal(size=(64, 10))      # linear map 64 -> 10
b = rng.normal(size=10)            # one bias per output
Y = X @ W + b                      # b broadcasts over the 32 rows
assert Y.shape == (32, 10)

# A classic broadcasting bug: (32,) - (32, 1) silently becomes (32, 32)
y_true = rng.normal(size=32)
y_pred = rng.normal(size=(32, 1))
print("buggy residual shape:", (y_true - y_pred).shape)
print("fixed residual shape:", (y_true - y_pred[:, 0]).shape)

# Four equivalent views of the same matrix product
A, B = rng.normal(size=(3, 4)), rng.normal(size=(4, 2))
entry = np.array([[A[i] @ B[:, j] for j in range(2)] for i in range(3)])
cols = np.stack([A @ B[:, j] for j in range(2)], axis=1)
outer = sum(np.outer(A[:, k], B[k]) for k in range(4))
assert np.allclose(A @ B, entry) and np.allclose(A @ B, cols) and np.allclose(A @ B, outer)
print("all views agree")
```

The output shows `(32, 32)` for the buggy line and `(32,)` for the fixed one. A mean over a 32×32 array still returns a single number, so a loss computed this way trains without errors while optimizing the wrong thing. Put `assert` statements on shapes in any code you care about.

Multiplying matrices and vectors

Plain version. A matrix product chains two linear machines into one. Each output number is a dot product between a row of the first matrix and a column of the second.

For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, the **matrix product** $C = AB \in \mathbb{R}^{m \times p}$ is

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

Here i indexes rows of A (and of C), j indexes columns of B (and of C), and k runs over the shared inner dimension n. The inner dimensions must match; that is the first thing to check when a shape error appears.

The same product can be read in four ways, and each one is useful:

1. **Entry view.** C_{ij} is the dot product of row i of A with column j of B. This is how attention scores $QK^\top$ are read: every query is compared with every key.
2. **Column view.** Column j of C is A times column j of B. Each column of C is a combination of the columns of A.
3. **Row view.** Row i of C is row i of A times B. In $Y = XW$, each output example is its input row pushed through the same weights.
4. **Outer-product view.** $C = \sum_k a_{:k} b_{k:}$, a sum of n rank-one matrices, each formed from column k of A and row k of B. This view explains low-rank approximation and LoRA later in the chapter.

The code above checks that all four views give the same answer.

Matrix multiplication is **associative**, $(AB)C = A(BC)$, and **distributive**, $A(B + C) = AB + AC$, but **not commutative**: $AB \neq BA$ in general, and often one of the two products is not even defined. The **elementwise** (Hadamard) product $A \odot B$ multiplies matching entries instead and needs equal shapes. In NumPy and PyTorch, `A @ B` is the matrix product and `A * B` is elementwise. The **dot product** of two vectors $x, y \in \mathbb{R}^n$ is the special case $x^\top y = \sum_i x_i y_i$, and it is symmetric: $x^\top y = y^\top x$.

Identity, inverse and systems of equations

A system of m linear equations in n unknowns can be written compactly as

$$Ax = b, \qquad A \in \mathbb{R}^{m \times n},\ x \in \mathbb{R}^n,\ b \in \mathbb{R}^m.$$

Each row of A holds the coefficients of one equation, x holds the unknowns, and b holds the right-hand sides.

The **identity matrix** I_n has ones on the diagonal and zeros elsewhere; it leaves every vector unchanged, $I_n x = x$. A square matrix A is **invertible** if there is a matrix A^{-1} with $A^{-1}A = AA^{-1} = I$. Then the system has exactly one solution, $x = A^{-1}b$.

That formula is excellent for derivations and poor for computation. Explicitly forming A^{-1} costs more and loses more accuracy than solving the system directly with an LU, Cholesky or QR factorization [4], [5]. In NumPy, write `np.linalg.solve(A, b)`, not `np.linalg.inv(A) @ b`. In this chapter, A^{-1} is a thinking tool.

Linear dependence, span and rank

Plain version. Rank counts how many genuinely different directions a matrix can produce. Repeated or redundant columns add nothing new.

A **linear combination** of vectors $v^{(1)}, \dots, v^{(k)}$ is $\sum_i c_i v^{(i)}$ for scalars c_i. The **span** of a set of vectors is the set of all their linear combinations. Because $Ax = \sum_j x_j a_{:j}$, the vector Ax is a linear combination of the columns $a_{:j}$ of A. So $Ax = b$ has a solution exactly when b lies in the span of the columns, called the **column space** (or range) of A.

A set of vectors is **linearly independent** if no vector in it is a linear combination of the others; equivalently, $\sum_i c_i v^{(i)} = 0$ only when every $c_i = 0$. The **rank** of A is the number of linearly independent columns, which always equals the number of linearly independent rows. For $A \in \mathbb{R}^{m \times n}$, $\operatorname{rank}(A) \le \min(m, n)$; if equality holds, A has **full rank**.

The **null space** of A is the set of vectors z with $Az = 0$. Those are the input directions that A destroys. The **rank–nullity theorem** says

$$\operatorname{rank}(A) + \dim \operatorname{null}(A) = n,$$

where n is the number of columns and dim is the number of independent directions in the null space. This one equation explains the three regimes of $Ax = b$:

Shape and rank	What happens	Typical ML case
square, full rank	exactly one solution	a well-posed change of coordinates
tall ($m > n$), full column rank	usually no exact solution; solve in the least-squares sense	regression with more examples than features
wide ($m < n$) or rank-deficient	infinitely many solutions (if any)	more parameters than constraints, as in deep networks

For a square matrix, "invertible", "full rank", "columns linearly independent" and "null space contains only zero" are all the same statement. A square matrix that fails them is called **singular**.

Norms: measuring size

Plain version. A norm turns a vector into one non-negative number that says how big it is. Different norms disagree about what "big" means.

A **norm** is any function f with $f(x) = 0 \Rightarrow x = 0$, $f(x + y) \le f(x) + f(y)$ (the triangle inequality), and $f(\alpha x) = |\alpha| f(x)$ for every scalar α. The most common family is the L^p norm,

$$\|x\|_p = \Big(\sum_i |x_i|^p \Big)^{1/p}, \qquad p \ge 1.$$

- $\|x\|_2$, the **Euclidean** norm, is ordinary length; we often write $\|x\|$. Its square $\|x\|_2^2 = x^\top x$ is easier to differentiate and appears in mean-squared-error losses and weight decay.

- $\|x\|_1 = \sum_i |x_i|$ grows at the same rate everywhere, even near zero. That makes it useful when the difference between exactly zero and slightly nonzero matters, which is why L^1 penalties encourage sparse solutions.
- $\|x\|_\infty = \max_i |x_i|$, the **max norm**, is the size of the largest entry. Adversarial-robustness work often bounds perturbations in this norm.
- The count of nonzero entries is sometimes called the "L^0 norm", but it is not a norm: scaling x by 2 does not change it.

The dot product connects to the Euclidean norm through angles: $x^\top y = \|x\|_2 \|y\|_2 \cos\theta$, where θ is the angle between x and y. The **cosine similarity**

$$\cos(x, y) = \frac{x^\top y}{\|x\|_2 \|y\|_2}$$

is defined for nonzero vectors and ignores scale. That is useful for some embeddings (contrastive learning normalizes features for this reason) and wrong when magnitude carries meaning, such as image brightness.

Matrix norms

For matrices we use two norms constantly. The **Frobenius norm** treats the matrix as one long vector,

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}.$$

The **spectral norm** (operator 2-norm) measures the largest factor by which A can stretch a unit vector,

$$\|A\|_2 = \max_{\|x\|_2 = 1} \|Ax\|_2 = \sigma_{\max}(A),$$

where $\sigma_{\max}(A)$ is the largest singular value, defined in the SVD section below. In terms of all singular values $\sigma_1 \geq \sigma_2 \geq \ldots$, we have $\|A\|_F^2 = \sum_i \sigma_i^2$ while $\|A\|_2 = \sigma_1$. So many small singular values can make the Frobenius norm large without changing the largest amplification. The spectral norm bounds how much a layer can magnify a perturbation of its input, which is why it appears in robustness and GAN-stability work [17].

```
import numpy as np

x = np.array([3.0, -4.0, 0.0, 1.0])
for p in [1, 2, np.inf]:
    print(f"L{p} norm:", np.linalg.norm(x, p))
print("L0 'norm' (count of nonzeros):", np.count_nonzero(x))

A = np.array([[3.0, 0.0], [4.0, 5.0]])
```

```
s = np.linalg.svd(A, compute_uv=False)
print("singular values:", s.round(4))
print("Frobenius:", np.linalg.norm(A, "fro"), "=", np.sqrt((s**2).sum()))
print("spectral :", np.linalg.norm(A, 2), "=", s[0])
assert np.isclose(np.linalg.norm(A, "fro")**2, np.trace(A.T @ A))
```

For $x = (3, -4, 0, 1)$ this prints $L^1 = 8$, $L^2 = \sqrt{26} \approx 5.10$, $L^\infty = 4$ and three nonzeros. For the 2×2 matrix the singular values are $3\sqrt{5} \approx 6.71$ and $\sqrt{5} \approx 2.24$, giving $\|A\|_F = \sqrt{50} \approx 7.07$ and $\|A\|_2 \approx 6.71$.

Special kinds of matrices and vectors

Plain version. Some matrices are much easier to work with than others. Learning to spot them saves computation and explains many design choices.

- A **diagonal** matrix $D = \operatorname{diag}(v)$ has nonzero entries only on its main diagonal. Multiplying by it scales each coordinate: $\operatorname{diag}(v)x = v \odot x$. Its inverse, when every $v_i \neq 0$, is $\operatorname{diag}(1/v_1, \dots, 1/v_n)$. Per-channel scaling in normalization layers is a diagonal matrix in disguise. Rectangular diagonal matrices also exist; they appear in the SVD.
- A **symmetric** matrix satisfies $A = A^\top$. Covariance matrices, Gram matrices $X^\top X$, Hessians of smooth functions and distance matrices are symmetric.
- A **unit vector** has $\|x\|_2 = 1$. Two vectors are **orthogonal** if $x^\top y = 0$; if they are also unit vectors, they are **orthonormal**.
- An **orthogonal matrix** Q is square with orthonormal rows and columns, so $Q^\top Q = QQ^\top = I$ and $Q^{-1} = Q^\top$. Its inverse is free to compute. It preserves lengths and angles, $\|Qx\|_2 = \|x\|_2$, so geometrically it is a rotation, a reflection, or both.

If $Q \in \mathbb{R}^{n \times k}$ has $k < n$ orthonormal columns, then $Q^\top Q = I_k$ but $QQ^\top \neq I_n$. The matrix $P = QQ^\top$ is the **orthogonal projection** onto the span of the columns of Q. The projection Px is the closest point to x in that subspace, and the residual $x - Px$ is perpendicular to every column of Q. Projecting twice changes nothing: $P^2 = P$. PCA, below, is exactly the search for the best such Q.

Eigendecomposition

Plain version. Most vectors change direction when a matrix acts on them. Eigenvectors are the special directions that only get stretched or shrunk, and the eigenvalue says by how much.

An **eigenvector** of a square matrix $A \in \mathbb{R}^{n\times n}$ is a nonzero vector v with

$$Av = \lambda v,$$

where the scalar λ is the corresponding **eigenvalue**. If v is an eigenvector, so is cv for any $c \neq 0$, so we usually normalize eigenvectors to unit length.

Suppose A has n linearly independent eigenvectors $v^{(1)}, \dots, v^{(n)}$ with eigenvalues $\lambda_1, \dots, \lambda_n$. Stack them as columns of V and put the eigenvalues in $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$. Then $AV = V\Lambda$, so

$$A = V\Lambda V^{-1}.$$

This is the **eigendecomposition**. It says: change to eigenvector coordinates (V^{-1}), scale each coordinate independently (Λ), and change back (V). Not every matrix has one. The shear $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ has only one independent eigenvector. A rotation by 90° has eigenvalues $\pm i$, which are complex, because no real direction is left unturned.

The symmetric case

Symmetric matrices are the friendly case, and they are the ones ML meets most. The **spectral theorem** says that every real symmetric matrix can be written as

$$A = Q\Lambda Q^{\top},$$

where Q is orthogonal (its columns are orthonormal eigenvectors) and Λ is real and diagonal [3], [4]. Because $Q^{-1} = Q^{\top}$, no inversion is needed. Geometrically, A stretches space by λ_i along the direction $q^{(i)}$, the i-th column of Q. The decomposition is unique up to the order of the eigenvalues and the choice of basis within any repeated eigenvalue; by convention we sort eigenvalues from largest to smallest.

For a symmetric A, the **quadratic form** $f(x) = x^{\top}Ax$ restricted to unit vectors is bounded by the extreme eigenvalues:

$$\lambda_{\min} \leq x^{\top}Ax \leq \lambda_{\max} \qquad \text{for } \|x\|_2 = 1,$$

with the maximum reached at the top eigenvector and the minimum at the bottom one. The ratio $x^{\top}Ax/x^{\top}x$ is called the **Rayleigh quotient**. This fact is the engine of the PCA derivation later.

Positive definite matrices

The signs of the eigenvalues classify symmetric matrices:

Name	Eigenvalues	Equivalent condition
positive definite (PD)	all $\lambda_i > 0$	$x^\top A x > 0$ for every $x \neq 0$
positive semidefinite (PSD)	all $\lambda_i \geq 0$	$x^\top A x \geq 0$ for every x
negative (semi)definite	all $\lambda_i < 0$ (or ≤ 0)	the reverse inequalities
indefinite	both signs present	$x^\top A x$ takes both signs

Every covariance matrix and every Gram matrix $X^\top X$ is PSD, because $v^\top X^\top X v = \|Xv\|_2^2 \geq 0$. A Gaussian density needs a PD covariance so that it can be inverted. The Hessian of a loss is PD at a strict local minimum and indefinite at a saddle point; ML 01C uses this to explain why optimization can be slow. A PD matrix has a **Cholesky factorization** $A = LL^\top$ with L lower triangular, which is the fastest stable way to solve systems with it and to sample from a Gaussian [4].

```
import numpy as np

rng = np.random.default_rng(1)
M = rng.normal(size=(4, 4))
S = M @ M.T + 0.1 * np.eye(4)        # symmetric positive definite
lam, Q = np.linalg.eigh(S)           # ascending eigenvalues, orthonormal Q
assert np.allclose(Q @ np.diag(lam) @ Q.T, S)
assert np.allclose(Q.T @ Q, np.eye(4))
print("eigenvalues:", lam.round(3), "all positive:", bool((lam > 0).all()))

# The quadratic form x^T S x is bounded by the extreme eigenvalues
x = rng.normal(size=4); x /= np.linalg.norm(x)
print("Rayleigh quotient in range:", lam[0] <= x @ S @ x <= lam[-1])

# A non-symmetric matrix can have complex eigenvalues
R = np.array([[0.0, -1.0], [1.0, 0.0]])  # 90-degree rotation
print("rotation eigenvalues:", np.linalg.eigvals(R))
```

Use `np.linalg.eigh` for symmetric matrices: it is faster, returns real eigenvalues in ascending order and orthonormal eigenvectors. `np.linalg.eig` handles the general case and may return complex numbers, as the rotation shows.

Singular value decomposition

Plain version. Every matrix, of any shape, does the same three things in order: it rotates, it stretches along perpendicular axes, and it rotates again. The SVD tells you those rotations and stretch factors.

Eigendecomposition needs a square matrix and may not exist. The **singular value decomposition** always exists, for every real matrix $A \in \mathbb{R}^{m \times n}$:

$$A = U \Sigma V^\top.$$

- $U \in \mathbb{R}^{m \times m}$ is orthogonal. Its columns are the **left singular vectors**, directions in the output space.
- $\Sigma \in \mathbb{R}^{m \times n}$ is rectangular diagonal. Its diagonal entries $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$ are the **singular values**.
- $V \in \mathbb{R}^{n \times n}$ is orthogonal. Its columns are the **right singular vectors**, directions in the input space.

Read from right to left, $Ax = U\Sigma V^\top x$ first expresses x in the V coordinates (a rotation or reflection), then scales coordinate i by σ_i, then places the result along the U directions. Figure 1 shows this for a 2×2 matrix: the unit circle becomes an ellipse whose semi-axes have lengths σ_1 and σ_2.

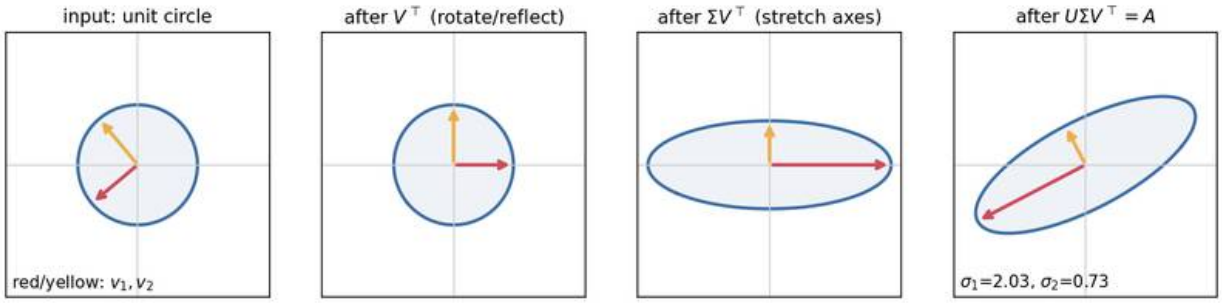


Figure 1 — The SVD as rotate, stretch, rotate. The right singular vectors v1 and v2 (red, yellow) are first rotated onto the axes by V-transpose, stretched by the singular values, then rotated by U. The final ellipse is the image of the unit circle under A.

The SVD and eigendecomposition are closely related. Since $A^\top A = V\Sigma^\top \Sigma V^\top$ and $AA^\top = U\Sigma\Sigma^\top U^\top$, the right singular vectors are eigenvectors of $A^\top A$, the left singular vectors are eigenvectors of $AA^\top$, and $\sigma_i = \sqrt{\lambda_i(A^\top A)}$. For a symmetric PSD matrix, the SVD and the eigendecomposition coincide.

The number of nonzero singular values equals the rank r. Keeping only those gives the **compact** (thin) SVD $A = U_r \Sigma_r V_r^\top = \sum_{i=1}^{r} \sigma_i u^{(i)} v^{(i)\top}$, a sum of r rank-one matrices. That is the outer-product view of a matrix product again.

The best low-rank approximation

Truncating that sum after k terms gives $A_k = \sum_{i=1}^{k} \sigma_i u^{(i)} v^{(i)\top}$. The **Eckart–Young theorem** [6] says that A_k is the best rank-k approximation of A in the Frobenius norm:

$$\min_{\operatorname{rank}(B) \leq k} \|A - B\|_F^2 = \|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2.$$

The same A_k is also optimal in the spectral norm, with error $\|A - A_k\|_2 = \sigma_{k+1}$ [4]. In words:

throw away the smallest singular values, and the energy you lose is exactly what you threw away. Storing A_k needs $k(m + n + 1)$ numbers instead of mn.

Figure 2 applies this to a 256 × 256 grayscale photograph. Natural images have rapidly decaying singular values, so a rank-50 approximation already keeps the main structure. The streaky artifacts at low rank come from the fact that each rank-one term is an outer product of a column pattern and a row pattern.

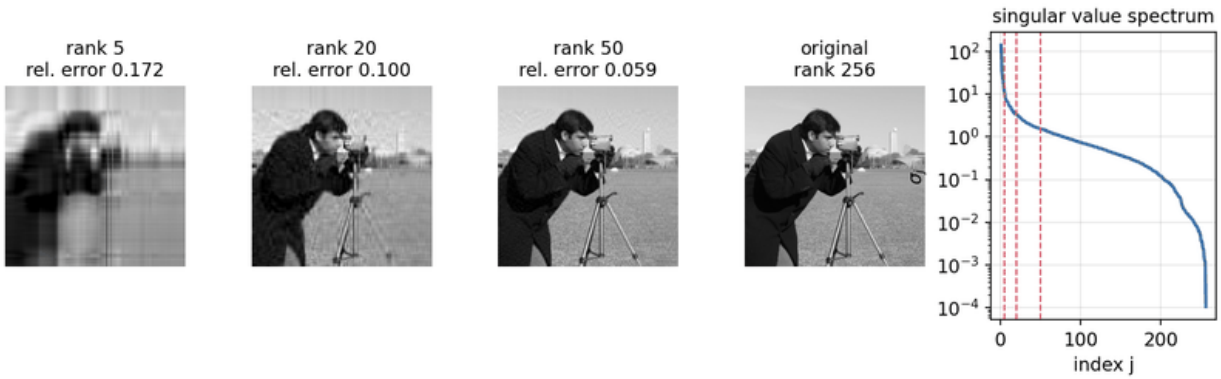


Figure 2 — Rank-k approximations of the 256×256 'camera' image from scikit-image, with the relative Frobenius error in each title. Right: singular values on a log scale; dashed lines mark $k = 5, 20, 50$. The error of each approximation is set by the singular values to the right of its line.

```
import numpy as np

rng = np.random.default_rng(0)
A = rng.normal(size=(12, 8))
U, s, Vt = np.linalg.svd(A, full_matrices=False)
assert np.allclose(U @ np.diag(s) @ Vt, A)
k = 3
Ak = (U[:, :k] * s[:k]) @ Vt[:k]
err_F = np.linalg.norm(A - Ak, "fro")**2
err_2 = np.linalg.norm(A - Ak, 2)
assert np.isclose(err_F, (s[k:]**2).sum()) and np.isclose(err_2, s[k])
print("rank-k squared Frobenius error:", round(err_F, 3))
print("rank-k spectral error = sigma_{k+1}:", round(err_2, 3), round(s[k], 3))
# Eigenvalues of A^T A are squared singular values of A
assert np.allclose(np.sort(np.linalg.eigvalsh(A.T @ A))[::-1], s**2)
print("storage: full", A.size, "numbers; rank-3 factors", k * (12 + 8 + 1))
```

`full_matrices=False` returns the thin SVD, which is what you almost always want. `(U[:, :k] * s[:k])` scales the columns of U by broadcasting instead of building a diagonal matrix. Both Eckart–Young identities hold to machine precision.

One caveat: the directions with the largest singular values carry the most energy, not necessarily the most useful information. For classification, a small-variance direction can be the one that separates the classes. PR 05 contrasts PCA with discriminant methods for exactly this reason.

The Moore–Penrose pseudo-inverse

Plain version. When a matrix has no inverse, the pseudo-inverse gives the next best thing: the answer that fits the data as closely as possible, and among equally good answers, the smallest one.

Only square, full-rank matrices have inverses. For any $A \in \mathbb{R}^{m\times n}$, the **Moore–Penrose pseudo-inverse** $A^+ \in \mathbb{R}^{n\times m}$ is the unique matrix satisfying the four Penrose conditions [7]:

$$AA^+A = A, \quad A^+AA^+ = A^+, \quad (AA^+)^\top = AA^+, \quad (A^+A)^\top = A^+A.$$

It is easiest to compute from the SVD. If $A = U\Sigma V^\top$, then

$$A^+ = V\Sigma^+U^\top,$$

where $\Sigma^+ \in \mathbb{R}^{n\times m}$ is formed by transposing Σ and replacing each nonzero singular value σ_i by $1/\sigma_i$ (zeros stay zero). Goodfellow et al. give the equivalent limit $A^+ = \lim_{\alpha\searrow 0}(A^\top A + \alpha I)^{-1}A^\top$ [1], which already hints at ridge regression.

The pseudo-inverse gives the right answer in every regime of the rank table:

- **Tall, full column rank** $(m > n)$. $Ax = b$ usually has no exact solution. Then $x = A^+b = (A^\top A)^{-1}A^\top b$ is the **least-squares** solution: it minimizes $\|Ax - b\|_2^2$.
- **Wide, full row rank** $(m < n)$. There are infinitely many solutions. Then $x = A^+b = A^\top(AA^\top)^{-1}b$ is the solution with the smallest $\|x\|_2$, the **minimum-norm** solution. Any other solution adds a null-space vector, which can only increase the norm.
- **Rank-deficient.** A^+b is both: among all least-squares minimizers, the one with the smallest norm.

The minimum-norm property is a small window into a modern question. Gradient descent on an under-determined least-squares problem, started from zero, converges to the minimum-norm solution, because every gradient $A^\top(Ax - b)$ lies in the row space of A. Over-parameterized networks are not linear, but this simple case is the usual starting point for thinking about which of many perfect fits a training algorithm picks.

Least squares, the normal equations and conditioning

For a regression design matrix $X \in \mathbb{R}^{n\times d}$ and targets $y \in \mathbb{R}^n$, least squares minimizes $\|Xw - y\|_2^2$.

Setting the gradient to zero gives the **normal equations**

$$X^\top Xw = X^\top y.$$

These are correct but numerically risky. The sensitivity of a linear problem is measured by the **condition number** $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$. Roughly, relative errors in the data can be amplified by up to a factor κ in the solution [5]. Forming $X^\top X$ squares the condition number, $\kappa(X^\top X) = \kappa(X)^2$, so a mildly ill-conditioned problem becomes a severely ill-conditioned one. Library least-squares solvers (`np.linalg.lstsq`, `torch.linalg.lstsq`) work with QR or SVD factorizations of X directly and avoid that squaring [4].

When two features are almost identical, $\sigma_{\min}$ is tiny, and the individual coefficients become extremely sensitive even when predictions are fine. **Ridge regression** [8] adds a penalty $\lambda\|w\|_2^2$ with $\lambda > 0$:

$$w_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y = \sum_i \frac{\sigma_i}{\sigma_i^2 + \lambda} \, (u^{(i)\top} y) \, v^{(i)}.$$

The SVD form on the right shows what ridge does. Plain least squares uses the factor $1/\sigma_i$, which explodes for small σ_i. Ridge uses $\sigma_i/(\sigma_i^2+\lambda)$, which stays bounded by $1/(2\sqrt{\lambda})$. Large-σ directions are nearly untouched; small-σ directions are suppressed. The price is bias. In neural networks the same penalty is called **weight decay**.

```
import numpy as np

rng = np.random.default_rng(0)
# Over-determined: more equations than unknowns -> least squares
x = np.linspace(-1, 1, 40)
X = np.c_[np.ones_like(x), x]
y = 2 + 3*x + rng.normal(0, .05, len(x))
w, residuals, rank, sv = np.linalg.lstsq(X, y, rcond=None)
print("intercept and slope:", w.round(3), "rank:", rank)
assert np.allclose(w, np.linalg.pinv(X) @ y)

# Under-determined: fewer equations than unknowns -> minimum-norm solution
A = rng.normal(size=(2, 5)); b = rng.normal(size=2)
w_min = np.linalg.pinv(A) @ b
null = np.linalg.svd(A)[2][-1]             # a direction with A @ null = 0
w_other = w_min + 0.7 * null
print("both solve Aw=b:", np.allclose(A @ w_min, b), np.allclose(A @ w_other, b))
print("norms:", np.linalg.norm(w_min).round(3), "<", np.linalg.norm(w_other).round(3))

# Near-collinear features: condition number and ridge
z = rng.normal(size=200)
F = np.c_[z, z + 1e-4 * rng.normal(size=200)]
t = z + 0.01 * rng.normal(size=200)
print("cond(F) = %.1e, cond(F^T F) = %.1e" % (np.linalg.cond(F), np.linalg.cond(F.T @ F)))
w_ls = np.linalg.lstsq(F, t, rcond=None)[0]
lam = 1e-2
w_ridge = np.linalg.solve(F.T @ F + lam * np.eye(2), F.T @ t)
print("least squares:", w_ls.round(2), " ridge:", w_ridge.round(3))
```

The line fit recovers intercept ≈ 2.00 and slope ≈ 3.01, and `pinv` agrees with `lstsq`. The underdetermined system has two exact solutions; the pseudo-inverse one has the smaller norm (0.885 versus 1.128). In the collinear case, $\kappa(F) \approx 2 \times 10^4$ while $\kappa(F^\top F) \approx 4 \times 10^8$, its square. Least squares returns wild coefficients $(-2.63, 3.63)$ whose sum is still about 1, so its predictions are fine. Ridge returns the stable split $(0.5, 0.5)$. The intercept column in the first example lets the line have a nonzero offset. In a real pipeline, fit any centering and scaling on the training set only, then apply it unchanged to validation and test data.

The trace operator

Plain version. The trace adds up the diagonal. It looks too simple to matter, but it lets you rewrite awkward sums as matrix expressions and move matrices around inside them.

The **trace** of a square matrix is

$$\operatorname{Tr}(A) = \sum_i A_{ii}.$$

Its useful properties are:

- **Frobenius norm.** $\|A\|_F = \sqrt{\operatorname{Tr}(AA^\top)} = \sqrt{\operatorname{Tr}(A^\top A)}$.
- **Transpose.** $\operatorname{Tr}(A) = \operatorname{Tr}(A^\top)$.
- **Cyclic property.** $\operatorname{Tr}(ABC) = \operatorname{Tr}(CAB) = \operatorname{Tr}(BCA)$ whenever the products are defined, even when the matrices are not square and the intermediate products have different shapes. In particular $\operatorname{Tr}(AB) = \operatorname{Tr}(BA)$ for $A \in \mathbb{R}^{m\times n}$, $B \in \mathbb{R}^{n\times m}$. You may cycle, but not arbitrarily reorder: $\operatorname{Tr}(ABC) \neq \operatorname{Tr}(ACB)$ in general.
- **Scalars.** A scalar is its own trace, $a = \operatorname{Tr}(a)$. This turns a quadratic form into a trace: $x^\top Ax = \operatorname{Tr}(x^\top Ax) = \operatorname{Tr}(Axx^\top)$.
- **Eigenvalues.** $\operatorname{Tr}(A) = \sum_i \lambda_i$. For a covariance matrix, that is the total variance, the sum of the variances along every principal direction.

The scalar trick is how many expectations are evaluated. For a random vector x with mean zero and covariance Σ, $\mathbb{E}[x^\top Ax] = \mathbb{E}[\operatorname{Tr}(Axx^\top)] = \operatorname{Tr}(A\Sigma)$, because trace and expectation are both linear.

The determinant

Plain version. The determinant of a square matrix says how much it scales area (in 2-D) or volume (in higher dimensions). If it is zero, the matrix flattens space and cannot be undone.

The **determinant** $\det(A)$ of a square matrix is a single number with these properties:

- $\det(A) = \prod_i \lambda_i$, the product of the eigenvalues.
- $|\det(A)|$ is the factor by which A scales volume. The unit cube maps to a parallelepiped of volume $|\det(A)|$.
- The sign records orientation. A negative determinant means A includes a reflection.
- $\det(A) = 0$ exactly when A is singular: some direction is collapsed to zero, so information is lost.
- $\det(AB) = \det(A)\det(B)$, $\det(A^\top) = \det(A)$, and $\det(A^{-1}) = 1/\det(A)$.

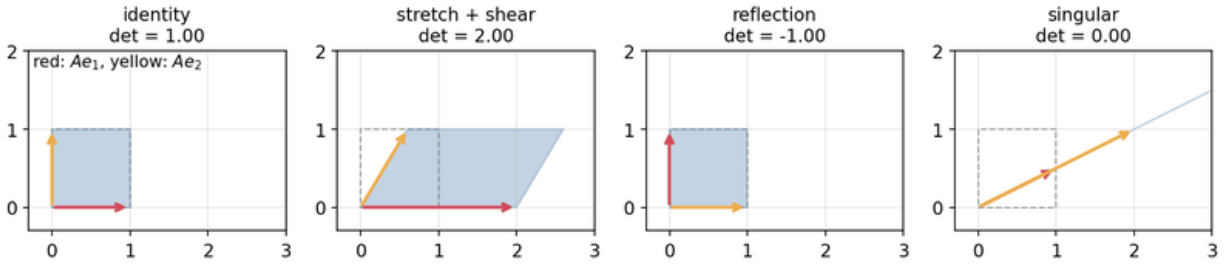


Figure 3 — The determinant as signed area. Dashed: the unit square. Shaded: its image, spanned by Ae1 (red) and Ae2 (yellow). A shear does not change area; a reflection flips the sign; a singular matrix collapses the square onto a line (det = 0).

Determinants appear in machine learning in two main places. The multivariate Gaussian density contains $\det(\Sigma)^{-1/2}$, the volume normalization of the covariance; ML 01B uses it. Normalizing flows (ML 06) need $\log|\det J|$ of a Jacobian matrix J to account for how an invertible network stretches probability mass. In both cases you need the logarithm, never the raw value: the determinant of a 500×500 diagonal matrix with entries 10^{-3} is 10^{-1500}, which underflows to zero in float64.

```
import numpy as np

rng = np.random.default_rng(2)
A, B, C = rng.normal(size=(3, 4)), rng.normal(size=(4, 5)), rng.normal(size=(5, 3))
print("cyclic trace:", np.isclose(np.trace(A @ B @ C), np.trace(C @ A @ B)),
      np.isclose(np.trace(A @ B @ C), np.trace(B @ C @ A)))

S = rng.normal(size=(4, 4))
lam = np.linalg.eigvals(S)
print("trace = sum of eigenvalues:", np.isclose(np.trace(S), lam.sum().real))
print("det = product of eigenvalues:", np.isclose(np.linalg.det(S), np.prod(lam).real))

# log-determinant of a covariance without overflow/underflow
Sigma = np.diag(np.full(500, 1e-3))
print("det underflows:", np.linalg.det(Sigma))
sign, logdet = np.linalg.slogdet(Sigma)
print("slogdet:", sign, round(logdet, 2))
L = np.linalg.cholesky(Sigma)
print("via Cholesky:", round(2 * np.log(np.diag(L)).sum(), 2))
```

The direct determinant prints `0.0`, while `slogdet` and the Cholesky route both give $\log\det =$

$500 \log 10^{-3} \approx -3453.88$. For a PD matrix, $\det(A) = \prod_i L_{ii}^2$ when $A = LL^\top$, so $\log\det A = 2\sum_i \log L_{ii}$.

Worked example: deriving principal component analysis

Plain version. PCA looks for the flat subspace that the data cloud lies closest to. Projecting onto it compresses each point to a few numbers while losing as little as possible.

PCA goes back to Pearson's "lines and planes of closest fit" [9] and Hotelling's analysis of "principal components" [10]; Jolliffe and Cadima give a modern review [11]. We derive it in the style of [1], using only the tools above.

Setup. We have n points $x^{(1)}, \dots, x^{(n)} \in \mathbb{R}^d$ and want to store each one with only $l < d$ numbers. First subtract the mean $\mu = \frac{1}{n}\sum_i x^{(i)}$ from every point, so we can assume the data are centered. Stack the centered points as rows of $X \in \mathbb{R}^{n \times d}$.

Step 1: choose the form of the code. We use a linear **decoder** $g(c) = Dc$ with $D \in \mathbb{R}^{d \times l}$. To make the problem well-posed we require the columns of D to be orthonormal, $D^\top D = I_l$. (Without this, we could shrink c and enlarge D forever.)

Step 2: find the best code for a fixed decoder. For a given x, choose c to minimize the reconstruction error:

$$c^* = \arg\min_c \|x - Dc\|_2^2 = \arg\min_c \left(-2x^\top Dc + c^\top D^\top Dc \right) = \arg\min_c \left(-2x^\top Dc + c^\top c \right).$$

We expanded the square, dropped $x^\top x$ (it does not depend on c) and used $D^\top D = I$. The gradient with respect to c is $-2D^\top x + 2c$. Setting it to zero gives the **encoder**

$$f(x) = D^\top x.$$

So encoding is a matrix–vector product, and the reconstruction is $r(x) = g(f(x)) = DD^\top x$, the orthogonal projection of x onto the span of D.

Step 3: find the best decoder. Now choose D to minimize the total reconstruction error over all points:

$$D^* = \arg\min_D \|X - XDD^\top\|_F^2 \quad \text{subject to } D^\top D = I_l.$$

Start with $l = 1$, so D is a single unit vector d. Write the Frobenius norm as a trace and expand:

$$\begin{aligned} \|X - Xdd^\top\|_F^2 &= \operatorname{Tr}\left((X - Xdd^\top)^\top (X - Xdd^\top)\right) \\ &= \operatorname{Tr}(X^\top X) - 2\operatorname{Tr}(X^\top Xdd^\top) + \operatorname{Tr}(dd^\top X^\top Xdd^\top) \\ &= \operatorname{Tr}(X^\top X) - 2\, d^\top X^\top Xd + d^\top X^\top Xd\, (d^\top d) \\ &= \operatorname{Tr}(X^\top X) - d^\top X^\top Xd. \end{aligned}$$

The cyclic property moved $d^\top$ to the front in the second and third terms, and $d^\top d = 1$ simplified the last line. The first term does not depend on d. So minimizing the error is the same as

$$d^* = \arg\max_{d} \; d^\top X^\top X d \quad \text{subject to } d^\top d = 1.$$

That is a Rayleigh quotient of the symmetric PSD matrix $X^\top X$. By the eigendecomposition section, it is maximized by the **eigenvector of $X^\top X$ with the largest eigenvalue**.

Step 4: more than one component. Repeating the argument for general l (for example, by induction, requiring each new direction to be orthogonal to the previous ones) gives the answer: the columns of D are the eigenvectors of $X^\top X$ for the l largest eigenvalues [1]. The minimum error equals the sum of the discarded eigenvalues.

Step 5: the variance view and the SVD link. Dividing by n turns $X^\top X$ into the sample covariance $S = \frac{1}{n} X^\top X$ (some texts use $n - 1$). The quantity $d^\top S d$ is the variance of the projected data Xd. So "minimize reconstruction error" and "maximize retained variance" select the same directions: Pearson's and Hotelling's views agree. Finally, if the centered data matrix has thin SVD $X = U \Sigma V^\top$, then $X^\top X = V \Sigma^2 V^\top$. The principal directions are the right singular vectors V, the eigenvalues are σ_i^2, and the codes are $XV_l = U_l \Sigma_l$. In practice PCA is computed with the SVD of X rather than by forming the covariance, for the conditioning reason we met in least squares.

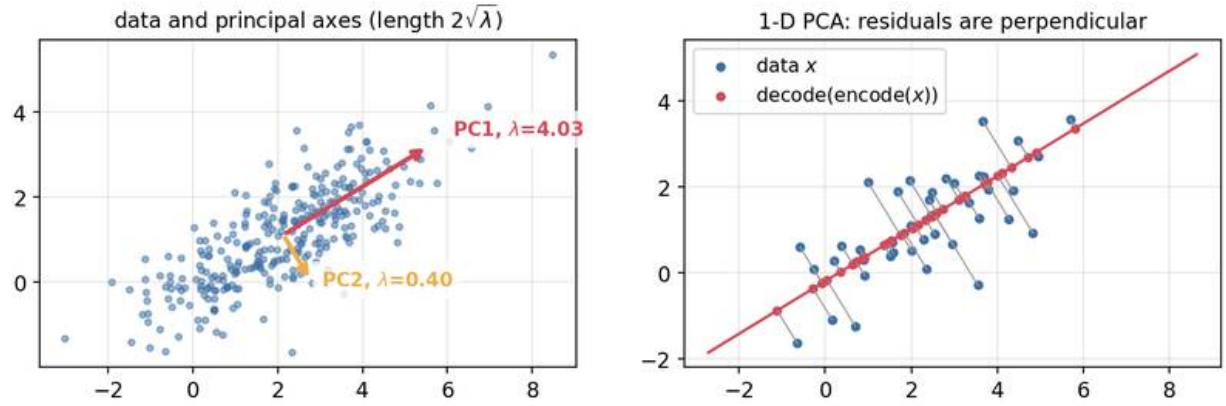


Figure 4 — PCA on 300 correlated 2-D points. Left: principal axes from the eigendecomposition of the sample covariance, drawn with length $2\sqrt{\lambda}$. Right: a 1-D PCA reconstruction $\mathrm{decode}(\mathrm{encode}(x))$; each gray residual meets the red line at a right angle, because the reconstruction is an orthogonal projection.

```
import numpy as np

rng = np.random.default_rng(0)
n, d, l = 500, 5, 2
Z = rng.normal(size=(n, l))
X = Z @ rng.normal(size=(l, d)) + 0.1 * rng.normal(size=(n, d)) + 3.0
```

```
mu = X.mean(axis=0)
Xc = X - mu                                  # centre first
S = Xc.T @ Xc / n                            # sample covariance (d x d)
lam, Q = np.linalg.eigh(S)
lam, Q = lam[::-1], Q[:, ::-1]               # descending
D = Q[:, :l]                                 # decoder matrix, orthonormal columns

codes = Xc @ D                               # encode: f(x) = D^T (x - mu)
X_hat = codes @ D.T + mu                     # decode: g(c) = D c + mu
mse = np.mean(np.sum((X - X_hat)**2, axis=1))
print("explained variance ratio:", (lam[:l].sum() / lam.sum()).round(4))
print("reconstruction MSE:", mse.round(5), " sum of discarded eigenvalues:",
 ↪  lam[l:].sum().round(5))
assert np.isclose(mse, lam[l:].sum())

# Same subspace from the SVD of the centred data matrix
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
assert np.allclose(s[:l]**2 / n, lam[:l])
assert np.allclose(np.abs(Vt[:l] @ D), np.eye(l), atol=1e-8)
print("SVD and eigendecomposition agree")
```

The synthetic data are 5-D points that really live near a 2-D plane, plus small noise. Two components explain 99.36% of the variance, and the average squared reconstruction error equals the sum of the three discarded eigenvalues, exactly as Step 4 predicts. The SVD route finds the same directions (up to sign, which is arbitrary for eigenvectors).

Three practical notes. Always center the data, or the first "component" just points at the mean. Standardize features if they have incomparable units, because PCA is not scale-invariant. And fit the mean and directions on training data only. PCA is the linear ancestor of autoencoders; ML 05A develops the probabilistic version and its nonlinear successors.

Matrix calculus you will meet

Plain version. Training means following gradients, and most gradients in ML are a handful of matrix formulas used over and over.

For a scalar function $f(x)$ of a vector $x \in \mathbb{R}^n$, the **gradient** $\nabla_x f \in \mathbb{R}^n$ collects the partial derivatives $\partial f / \partial x_i$. For a vector function $f : \mathbb{R}^n \to \mathbb{R}^m$, the **Jacobian** $J \in \mathbb{R}^{m \times n}$ has entries $J_{ij} = \partial f_i / \partial x_j$. The rules you will use most are:

Function	Gradient	Note
$a^\top x$	a	linear function
$x^\top A x$	$(A + A^\top)x$	becomes $2Ax$ when A is symmetric
$\frac{1}{2}\lVert Xw - y \rVert_2^2$	$X^\top(Xw - y)$	least-squares loss
$\lVert x \rVert_2^2$	$2x$	weight decay
$\mathrm{Tr}(AB)$ w.r.t. A	$B^\top$	used with the trace tricks above

Each can be verified by writing the expression as a sum over indices and differentiating term by term [2]. The **chain rule** for vector functions multiplies Jacobians: if $y = g(x)$ and $L = f(y)$ is a scalar, then $\nabla_x L = J_g^\top \nabla_y L$. Backpropagation applies this rule layer by layer, multiplying transposed local Jacobians by the incoming gradient without ever building the full Jacobians, which would be enormous. ML 02 builds this up in detail, and ML 01C adds the Hessian, the matrix of second derivatives.

Setting the least-squares gradient to zero recovers the normal equations from the previous section. The Hessian of that loss is $X^\top X$, which is PSD, so the loss is convex and every stationary point is a global minimum. A quick finite-difference check confirms the formula:

```
import numpy as np

rng = np.random.default_rng(3)
X, y = rng.normal(size=(30, 4)), rng.normal(size=30)
w = rng.normal(size=4)
L = lambda w: 0.5 * np.sum((X @ w - y)**2)
grad = X.T @ (X @ w - y)                    # analytic gradient
eps = 1e-6
num = np.array([(L(w + eps*e) - L(w - eps*e)) / (2*eps) for e in np.eye(4)])
print("max gradient mismatch:", np.abs(grad - num).max())
# Setting the gradient to zero gives the normal equations
w_star = np.linalg.lstsq(X, y, rcond=None)[0]
print("gradient at solution ~ 0:", np.abs(X.T @ (X @ w_star - y)).max() < 1e-10)
```

The analytic and numerical gradients agree to about 10^{-8}, the size expected from finite-difference rounding.

Modern view

The definitions in this chapter are a century old, but the way they are used in machine learning keeps changing. Four lines of work are worth knowing about.

Randomized numerical linear algebra. Exact SVDs of large matrices are expensive, roughly $O(mn \min(m, n))$ operations. Halko, Martinsson and Tropp's SIAM Review survey [12] made a simple idea standard: multiply A by a thin random matrix Ω to "sketch" its range, orthonormalize the result to get a basis Q, and compute an exact SVD of the small matrix $Q^\top A$. With a little oversampling and a few power iterations, the result is accurate for matrices whose spectrum decays. Martinsson and Tropp's *Acta Numerica* survey [13] covers the theory and a wide range of algorithms: norm estimation, random embeddings, regression, low-rank approximation, Nyström approximation of PSD matrices, streaming algorithms and kernel matrices from machine learning. Murray et al. [14] (LAPACK Working Note 299, 2023) take the software view. They note that RandNLA methods have entered MATLAB, the NAG Library, NVIDIA's cuSOLVER and scikit-learn, and they argue for standard "RandBLAS" and "RandLAPACK" libraries. scikit-learn's randomized PCA solver is a direct descendant. The core algorithm fits in a few lines:

```
import numpy as np

def randomized_svd(A, k, oversample=10, power_iters=2, seed=0):
    rng = np.random.default_rng(seed)
    Omega = rng.normal(size=(A.shape[1], k + oversample))  # random test matrix
    Y = A @ Omega                                          # sample the range of A
    for _ in range(power_iters):                           # sharpen a slow spectrum
        Y = A @ (A.T @ Y)
    Q, _ = np.linalg.qr(Y)                                 # orthonormal basis
    B = Q.T @ A                                            # small (k+p) x n matrix
    Ub, s, Vt = np.linalg.svd(B, full_matrices=False)
    return (Q @ Ub)[:, :k], s[:k], Vt[:k]

rng = np.random.default_rng(1)
m, n, r = 2000, 500, 20
A = rng.normal(size=(m, r)) @ rng.normal(size=(r, n)) + 0.01 * rng.normal(size=(m, n))
U, s, Vt = randomized_svd(A, k=r)
s_exact = np.linalg.svd(A, compute_uv=False)[:r]
print("max relative error in top singular values:", np.max(np.abs(s - s_exact) / s_exact))
rel = np.linalg.norm(A - (U * s) @ Vt) / np.linalg.norm(A)
print("relative reconstruction error:", round(rel, 5))
```

On this nearly rank-20 matrix the top 20 singular values match the exact ones to about 10^{-15} relative error, and the rank-20 reconstruction error (about 0.2%) is set by the added noise. The power iterations are what make it work when the spectrum decays slowly. For a matrix with a flat spectrum, no rank-k method can do well, randomized or not.

Low-rank structure in large models. The outer-product view of matrix products has become a practical tool for adapting large networks. LoRA [15] freezes a pretrained weight $W_0 \in \mathbb{R}^{d\times k}$ and learns an update $\Delta W = BA$ with $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$ and $r \ll \min(d,k)$. Only $r(d+k)$ numbers are trained instead of dk; for $d = k = 4096$ and $r = 8$, that is 65,536 instead of about 16.8 million per matrix. B starts at zero, so training begins exactly at the pretrained model. GaLore [16] (ICML 2024) keeps all weights trainable but projects the *gradients* onto a low-rank subspace, which shrinks the optimizer's memory. Its authors argue that restricting the weight update itself to a low-rank subspace, as LoRA does, can underperform full-rank training. Both rely on the Eckart–Young intuition that much of a matrix's useful content lives in a few singular directions. Whether that holds for a given task is an empirical question, not a theorem.

Controlling the spectral norm. A network's sensitivity to input perturbations is bounded by the product of its layers' spectral norms (for 1-Lipschitz activations such as ReLU). Spectral normalization [17] divides each weight matrix by an estimate of its largest singular value, obtained cheaply with one step of power iteration per training step. It was introduced to stabilize the discriminator in GANs (ML 06), and the same idea appears wherever a Lipschitz bound is wanted.

Beyond matrices. Convolution kernels and video features are naturally tensors with three or more axes. Kolda and Bader's SIAM Review survey [18] is the standard introduction to tensor

decompositions. It covers the CP decomposition (a sum of rank-one tensors, generalizing the outer-product view) and the Tucker decomposition (a small core tensor multiplied by a matrix along each mode, generalizing the SVD). Unlike matrices, computing the rank of a tensor is hard in general, and best low-rank approximations need not exist. The clean Eckart–Young picture does not carry over. These decompositions are used to compress layers and to analyze multi-way data.

Open questions. Why do trained weight matrices often have rapidly decaying spectra, and how does that relate to generalization? How low can the rank of an adaptation be before quality drops, and does that depend on the task or the model? How should randomized algorithms be combined with low-precision hardware, where rounding errors are much larger than in float64? None of these have settled answers. The tools in this chapter are what you need to read the papers that study them. For broader textbook treatments that connect linear algebra to deep learning, see [1], [2] and [19].

Key takeaways

- Name every axis. Most silent bugs are shape or broadcasting errors, and `assert` statements catch them.
- A matrix product has four readings (entries, columns, rows, sums of outer products); choose the one that makes the problem clear.
- Rank and null space decide whether $Ax = b$ has zero, one or infinitely many solutions. Never form A^{-1} in code; use a solver.
- Symmetric matrices have real eigenvalues and orthonormal eigenvectors; the signs of the eigenvalues say whether a quadratic form is a bowl, a saddle or a dome.
- The SVD exists for every matrix: rotate, stretch, rotate. Truncating it gives the best low-rank approximation, with error equal to the discarded singular values.
- The pseudo-inverse gives least-squares and minimum-norm solutions. Small singular values amplify noise; the condition number measures how much, and ridge regularization tames it.
- Trace and determinant summarize eigenvalues (sum and product). Use the cyclic trace property in derivations and log-determinants in code.
- PCA is a projection onto the top eigenvectors of the covariance, equivalently the top right singular vectors of the centered data. Minimizing reconstruction error and maximizing variance give the same answer.

Exercises

1. **Shapes and bias.** A batch has shape 32×64 and a weight matrix has shape 64×10. What shape should the bias have, and what goes wrong if you use a bias of shape $(32, 1)$ instead?

   Hint

A bias of shape $(10,)$ broadcasts over the 32 rows: one offset per output unit, shared by every example. A $(32, 1)$ bias broadcasts across the 10 columns, giving one offset per *example*. The code runs, but the "bias" now depends on the position of an example in the batch, which is meaningless and changes with batch size.

2. **Projection.** Let $Q \in \mathbb{R}^{n \times k}$ have orthonormal columns and $P = QQ^\top$. Show that $P^2 = P$, $P^\top = P$, and that $x - Px$ is orthogonal to every column of Q. What are the eigenvalues of P?

Hint

$P^2 = Q(Q^\top Q)Q^\top = QIQ^\top = P$. For orthogonality compute $Q^\top(x - Px) = Q^\top x - Q^\top QQ^\top x = 0$. From $P^2 = P$, any eigenvalue satisfies $\lambda^2 = \lambda$, so $\lambda \in \{0, 1\}$: eigenvalue 1 with multiplicity k (the subspace) and 0 with multiplicity $n - k$ (its orthogonal complement). Hence $\mathrm{Tr}(P) = k$.

3. **SVD by hand.** Find the singular values of $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$ by computing the eigenvalues of $A^\top A$. Check that $\|A\|_F^2 = \sigma_1^2 + \sigma_2^2$ and $|\det A| = \sigma_1 \sigma_2$.

Hint

$A^\top A = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$ has eigenvalues 45 and 5, so $\sigma_1 = 3\sqrt{5}$, $\sigma_2 = \sqrt{5}$. Then $\sigma_1^2 + \sigma_2^2 = 50 = 9 + 16 + 25$, and $\sigma_1 \sigma_2 = 15 = |\det A| = |3 \cdot 5 - 0 \cdot 4|$. In general $|\det A| = \prod_i \sigma_i$ for a square matrix, because $|\det U| = |\det V| = 1$.

4. **Conditioning and ridge.** Starting from the least-squares code, replace the second feature by an exact copy of the first. What does `np.linalg.lstsq` return, and why is it the minimum-norm solution? Then derive the SVD form of the ridge solution and show that each factor $\sigma/(\sigma^2 + \lambda)$ is at most $1/(2\sqrt{\lambda})$.

Hint

With identical columns, the rank is 1 and the solution set is a line $w_1 + w_2 = c$. `lstsq` uses the SVD and returns the pseudo-inverse solution, which splits the weight equally, $w_1 = w_2 = c/2$, because that point on the line is closest to the origin. For the bound, maximize $g(\sigma) = \sigma/(\sigma^2 + \lambda)$: $g'(\sigma) = 0$ at $\sigma = \sqrt{\lambda}$, giving $g = 1/(2\sqrt{\lambda})$.

5. **PCA error.** Using the derivation in this chapter, show that the total reconstruction error of l-component PCA equals $\sum_{i>l} \lambda_i(X^\top X)$, and that the fraction of variance retained is $\sum_{i \le l} \lambda_i / \sum_i \lambda_i$. Then run PCA on `sklearn.datasets.load_digits()` and find the smallest l that keeps 90% of the variance.

Hint

Write $\|X - XDD^\top\|_F^2 = \mathrm{Tr}(X^\top X) - \mathrm{Tr}(D^\top X^\top X D)$ (use $D^\top D = I$ and the cyclic property). With D the top-l eigenvectors, the second trace is $\sum_{i \le l} \lambda_i$, and $\mathrm{Tr}(X^\top X) = \sum_i \lambda_i$. For the digits, center the 1797×64 matrix, compute its singular values, and take the cumulative sum of σ_i^2. Report your own number rather than trusting a quoted one.

6. **Determinants and volume.** A 2×2 matrix maps the unit square to a parallelogram with vertices 0, $(2,1)$, $(1,3)$ and $(3,4)$. Write down a matrix that does this, compute its determinant, and explain why a normalizing flow must track $\log|\det J|$ for every layer.

Hint

Put the edge vectors in the columns: $A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$, with $\det A = 5$, so areas grow by a factor of 5. If a map stretches volume by 5, probability density must shrink by the same factor to keep total probability equal to 1. Stacking layers multiplies these factors, so their logarithms add, which is why flows sum log-determinants layer by layer.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 2 “Linear Algebra”. Book
2. M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Book
3. G. Strang, *Introduction to Linear Algebra*, 6th ed., Wellesley-Cambridge Press, 2023. Book site
4. G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013. DOI
5. L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*, SIAM, 1997. DOI
6. C. Eckart and G. Young, “The Approximation of One Matrix by Another of Lower Rank,” *Psychometrika*, vol. 1, pp. 211–218, 1936. DOI
7. R. Penrose, “A Generalized Inverse for Matrices,” *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 51, pp. 406–413, 1955. DOI
8. A. E. Hoerl and R. W. Kennard, “Ridge Regression: Biased Estimation for Nonorthogonal Problems,” *Technometrics*, vol. 12, pp. 55–67, 1970. DOI
9. K. Pearson, “On Lines and Planes of Closest Fit to Systems of Points in Space,” *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 2, pp. 559–572, 1901. DOI
10. H. Hotelling, “Analysis of a Complex of Statistical Variables into Principal Components,” *Journal of Educational Psychology*, vol. 24, pp. 417–441, 1933. DOI
11. I. T. Jolliffe and J. Cadima, “Principal Component Analysis: A Review and Recent Developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, 20150202, 2016. DOI
12. N. Halko, P.-G. Martinsson and J. A. Tropp, “Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions,” *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011. DOI ·arXiv

13. P.-G. Martinsson and J. A. Tropp, “Randomized Numerical Linear Algebra: Foundations and Algorithms,” *Acta Numerica*, vol. 29, pp. 403–572, 2020. DOI ·arXiv
14. R. Murray, J. Demmel, M. W. Mahoney, N. B. Erichson, M. Melnichenko, O. A. Malik, L. Grigori, P. Luszczek, M. Dereziński, M. E. Lopes, T. Liang, H. Luo and J. Dongarra, “Randomized Numerical Linear Algebra: A Perspective on the Field With an Eye to Software,” LAPACK Working Note 299, 2023. arXiv
15. E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2022. arXiv
16. J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar and Y. Tian, “GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection,” *ICML*, 2024. arXiv
17. T. Miyato, T. Kataoka, M. Koyama and Y. Yoshida, “Spectral Normalization for Generative Adversarial Networks,” *ICLR*, 2018. arXiv
18. T. G. Kolda and B. W. Bader, “Tensor Decompositions and Applications,” *SIAM Review*, vol. 51, no. 3, pp. 455–500, 2009. DOI
19. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book

ML 01B • Probability and Information Theory

Random variables, common distributions, Bayes' rule, expectation and covariance, likelihood, entropy, cross-entropy, KL divergence and structured models, connected to how neural networks are trained and evaluated.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-01b-probability-information/

What you'll learn

- Why machine learning needs probability, and the two common ways of reading a probability (frequency and degree of belief).
- How random variables, probability mass functions and densities work, and how marginal, conditional and joint probabilities relate through the sum rule, the product rule and the chain rule.
- How expectation, variance, covariance and correlation summarize a distribution, and why zero covariance does not mean independence.
- The distributions you will meet constantly in deep learning — Bernoulli, categorical, Gaussian, exponential, Laplace, Dirac, empirical and mixtures — plus the sigmoid and softplus functions that parameterize them.
- How Bayes' rule, the change-of-variables formula, maximum likelihood and MAP estimation work, with tested NumPy code.
- What entropy, cross-entropy, KL divergence and mutual information measure, why the classification loss is a cross-entropy, and how structured (graphical) models factor a large joint distribution.

The big picture

A neural network never sees the whole world. It sees a finite training set, measured by imperfect sensors, labelled by busy people, and then it is asked about images it has never seen. Every step of that story involves **uncertainty**. Probability theory is the mathematics of uncertainty: it tells us how to describe what we do not know, how to combine evidence, and how to make predictions that admit doubt. Information theory, its close relative, measures *how much* uncertainty there is and how costly it is to use the wrong distribution. Together they explain almost every loss function in this site's ML and DL tracks.

> **Explain like I'm five**
>
> Imagine a bag of marbles you cannot see into. **Probability** is your best guess about what you will pull out next: "probably red, maybe blue, almost never green." Every time you pull a marble, you learn a little more about the bag and update your guess — that updating is **Bayes' rule**. **Entropy** is how surprised you expect to be: a bag of all-red marbles never surprises you; a bag with ten colours in

equal amounts surprises you a lot. A machine learning model is a very fancy guesser, and training means making its guesses less surprised by the real marbles.

This chapter follows the "Applied Math" part of *Deep Learning* by Goodfellow, Bengio and Courville [1], with additional material from Bishop and Bishop [2], Murphy [3] and *Mathematics for Machine Learning* [4]. It sits between ML 01A ·Linear Algebra and ML 01C ·Numerical Computation. If you are heading to the Pattern Recognition track, PR 01 ·Bayes Decisions builds directly on the Bayes-rule section here.

Why probability?

Plain version. Computer science usually deals with certain facts: a bit is 0 or 1, a program either halts or does not. Machine learning cannot work that way because the data, the world and the model are all imperfect.

Goodfellow et al. [1] list three sources of uncertainty:

1. **Inherent stochasticity.** Some systems are genuinely random — photon arrival at a camera sensor (shot noise), the roll of a die, or the exact moment a pedestrian steps off a curb.
2. **Incomplete observability.** Even a deterministic system looks random if we cannot see all of its variables. A single photo of a car does not show whether its brake lights are about to turn on.
3. **Incomplete modelling.** A model must discard information to be useful. A classifier that sees 32×32 thumbnails cannot recover details that were lost when the image was downsampled, so its predictions about fine-grained classes must remain uncertain.

It is often more practical to use a simple rule with uncertainty ("most birds fly") than a complex rule that is always true ("birds fly, except penguins, ostriches, injured birds, ...").

Two readings of a probability

A probability can be read in two ways, and both obey the same mathematical rules.

- **Frequentist probability** describes long-run frequencies of repeatable events: "if we toss this coin many times, about 70% of tosses land heads".
- **Bayesian probability** describes a *degree of belief* about something that might not be repeatable: "given this X-ray, I am 40% confident the patient has the disease".

If degrees of belief are required to satisfy a few common-sense consistency properties, they turn out to obey exactly the same rules as frequencies, which is why one calculus serves both readings [1], [3]. In practice, deep learning mixes the two: the data are treated as samples from a distribution (frequentist), while the model's output is often read as a belief about the label (Bayesian).

Random variables and probability distributions

Plain version. A random variable is a quantity whose value is not known in advance. A probability distribution says how likely each value is.

We write random variables in roman letters (for example x in [1]); in this tutorial we use X for the variable and x for one of its values. A random variable can be **discrete** (a finite or countable set of values, such as a class label in $\{1, \dots, K\}$) or **continuous** (real values, such as a pixel intensity treated as a real number).

Probability mass functions

A discrete variable has a **probability mass function** (PMF) $P(X = x)$, often shortened to $P(x)$. It must satisfy

$$0 \le P(x) \le 1 \text{ for every } x, \qquad \sum_{x} P(x) = 1.$$

Here the sum runs over every possible value x of X. The first condition says no event is more than certain; the second says that *something* always happens. A joint PMF $P(X = x, Y = y)$ gives the probability that both events happen together.

Probability density functions

A continuous variable has a **probability density function** (PDF) $p(x)$, which must satisfy

$$p(x) \ge 0, \qquad \int p(x)\, dx = 1.$$

A density is *not* a probability. The probability that X falls in an interval $[a, b]$ is the area under the density:

$$P(a \le X \le b) = \int_a^b p(x)\, dx.$$

The probability of any single exact value is zero. Because only areas must be at most one, a density can exceed one: a narrow Gaussian with standard deviation 0.25 peaks at about 1.6 (Figure 1, top right). A common mistake is to report "probability 1.6" for a density value.

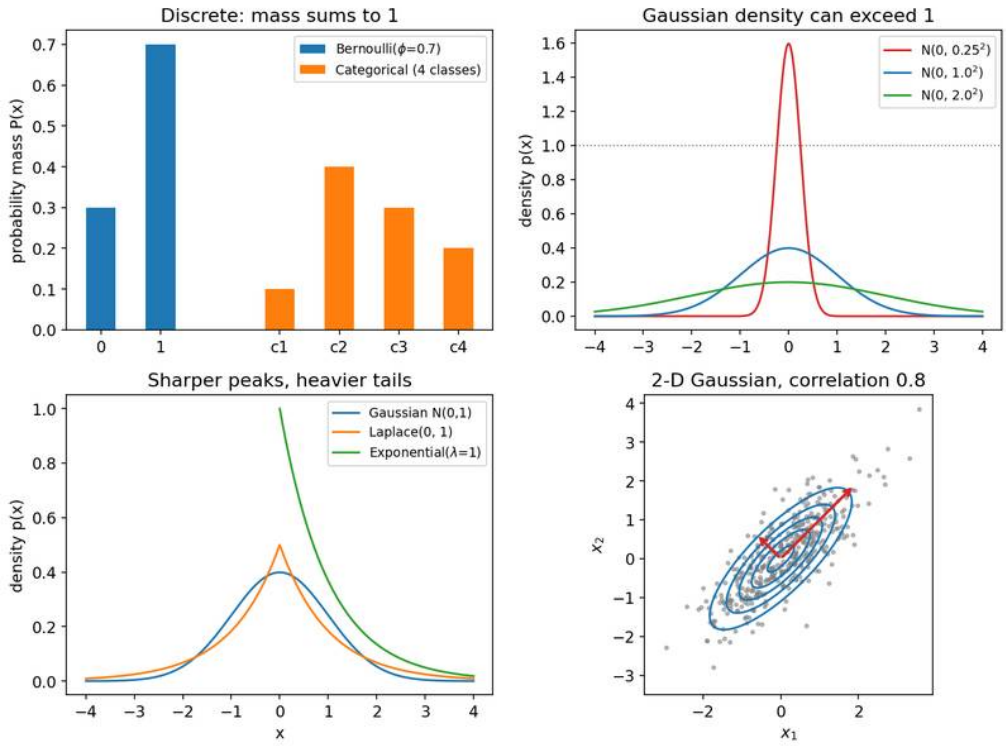


Figure 1 — Common distributions. Top left: discrete masses sum to one. Top right: a density can exceed one because only its area must equal one. Bottom left: the Laplace density has a sharp peak and heavier tails than the Gaussian; the exponential lives on $x \geq 0$. Bottom right: a 2-D Gaussian with correlation 0.8; red arrows show the eigenvectors of its covariance, scaled by two standard deviations.

Marginal probability

Plain version. If you know how two things vary together, you can recover how one of them varies on its own by "summing out" the other.

Given a joint distribution over X and Y, the **marginal** distribution of X is

$$P(x) = \sum_{y} P(x, y) \quad \text{(discrete)}, \qquad p(x) = \int p(x, y)\, dy \quad \text{(continuous)}.$$

This is the **sum rule**. If the joint is written as a table with rows for x and columns for y, the marginal of x is the row sums — historically written in the margin of the page, hence the name.

Conditional probability

Plain version. A conditional probability answers "how likely is this, *given that* I already know that?"

$$P(y \mid x) = \frac{P(x, y)}{P(x)}, \qquad P(x) > 0.$$

$P(y \mid x)$ is the probability of $Y = y$ after we have observed $X = x$. It is only defined when the condition can actually happen. Rearranging gives the **product rule** $P(x, y) = P(y \mid x)P(x)$.

Do not confuse conditioning with intervention. "The probability that the street is wet given that people carry umbrellas" is high, but *forcing* people to carry umbrellas will not wet the street. Questions about interventions belong to causal inference, which needs more than a joint distribution.

The chain rule of probability

Applying the product rule repeatedly factorizes any joint distribution over n variables:

$$P(x^{(1)}, \dots, x^{(n)}) = P(x^{(1)}) \prod_{i=2}^{n} P(x^{(i)} \mid x^{(1)}, \dots, x^{(i-1)}).$$

Here $x^{(i)}$ is the value of the i-th variable. This identity is *exact* — no assumption is involved. It is the backbone of autoregressive models: a language model or an autoregressive image model predicts one token or pixel at a time given all previous ones (ML 08, NLP 01). The modelling choice lies in *how* each conditional is parameterized, not in the factorization itself.

Independence and conditional independence

Two variables are **independent**, written $X \perp Y$, if

$$p(x, y) = p(x)\, p(y) \quad \text{for all } x, y.$$

They are **conditionally independent** given Z, written $X \perp Y \mid Z$, if

$$p(x, y \mid z) = p(x \mid z)\, p(y \mid z) \quad \text{for all } x, y, z.$$

Ice-cream sales and sunburn are dependent, but roughly independent once you condition on the weather. Conditional independence is the main tool for making high-dimensional models tractable — naive Bayes assumes the features are independent given the class; a hidden Markov model assumes the future is independent of the past given the present state. These are *assumptions* [3]: they should be justified by the problem, not adopted only because they make the algebra easy.

Code: a joint table, marginals and conditionals

The following example builds a small joint table for weather and umbrella use, computes marginals and conditionals, and tests independence. It also shows a case where two variables are clearly dependent yet have (almost) zero covariance.

```
import numpy as np

# Joint table P(weather, umbrella): rows = sunny/rainy, columns = no/yes
P = np.array([[0.56, 0.14],
              [0.06, 0.24]])
assert np.isclose(P.sum(), 1)

p_weather = P.sum(axis=1)            # marginal over umbrella
p_umbrella = P.sum(axis=0)           # marginal over weather
p_umb_given_w = P / p_weather[:, None]   # each row sums to 1
print("P(weather):", p_weather)
print("P(umbrella):", p_umbrella)
print("P(umbrella | weather):\n", p_umb_given_w)
print("independent?", np.allclose(P, np.outer(p_weather, p_umbrella)))

# Zero covariance does not imply independence
rng = np.random.default_rng(0)
x = rng.uniform(-1, 1, 200_000)
y = x**2                              # y is a deterministic function of x
print("cov(x, y) ~", np.cov(x, y)[0, 1].round(4))
```

Output: the marginals are $[0.7, 0.3]$ and $[0.62, 0.38]$; the conditional table is $[[0.8, 0.2], [0.2, 0.8]]$; the variables are not independent; and the sample covariance of x and x^2 is about -0.0007, essentially zero, even though y is completely determined by x.

Expectation, variance and covariance

Plain version. The expectation is the long-run average; the variance is how far values typically spread from that average; the covariance says whether two quantities tend to go up and down together.

The **expectation** of a function f under a distribution is

$$\mathbb{E}_{x\sim P}[f(x)] = \sum_{x} P(x) f(x), \qquad \mathbb{E}_{x\sim p}[f(x)] = \int p(x) f(x)\, dx.$$

The subscript $x \sim P$ names the distribution we average over; we drop it when it is obvious. Expectation is **linear**: $\mathbb{E}[\alpha f(x) + \beta g(x)] = \alpha \mathbb{E}[f(x)] + \beta \mathbb{E}[g(x)]$ for constants α, β. Almost every training loss is an expectation over the data distribution, estimated by an average over a mini-batch.

The **variance** measures spread around the mean $\mu = \mathbb{E}[f(x)]$:

$$\mathrm{Var}(f(x)) = \mathbb{E}[(f(x) - \mu)^2] = \mathbb{E}[f(x)^2] - \mu^2.$$

Its square root is the **standard deviation**, which has the same units as $f(x)$.

The **covariance** of two quantities measures how they vary together:

$$\mathrm{Cov}(f(x), g(y)) = \mathbb{E}[(f(x) - \mathbb{E}[f(x)])\,(g(y) - \mathbb{E}[g(y)])].$$

A positive covariance means both tend to be above (or below) their means at the same time; a negative covariance means one tends to be high when the other is low. Its magnitude depends on the units, so we often normalize it into the **correlation**

$$\rho = \frac{\mathrm{Cov}(X, Y)}{\sqrt{\mathrm{Var}(X)\,\mathrm{Var}(Y)}} \in [-1, 1].$$

Independence implies zero covariance, but **zero covariance does not imply independence**. Covariance measures only *linear* dependence. In the code above, $y = x^2$ with x uniform on $[-1, 1]$: knowing x tells you y exactly, but the covariance is zero because the relationship is symmetric and nonlinear. Mutual information, introduced later, captures every kind of dependence.

The covariance matrix

For a random vector $\mathbf{x} \in \mathbb{R}^n$ with mean $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x}]$, the **covariance matrix** is

$$\Sigma = \mathrm{Cov}(\mathbf{x}) = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top], \qquad \Sigma_{ij} = \mathrm{Cov}(x_i, x_j).$$

Its diagonal holds the variances $\Sigma_{ii} = \mathrm{Var}(x_i)$. Σ is symmetric and positive semidefinite, so (by ML 01A) it has real nonnegative eigenvalues and orthogonal eigenvectors. The eigenvectors are the principal axes of the data cloud and the eigenvalues are the variances along them — the red arrows in Figure 1 — which is exactly what PCA computes. For an affine map $\mathbf{y} = A\mathbf{x} + \mathbf{b}$, $\mathbb{E}[\mathbf{y}] = A\boldsymbol{\mu} + \mathbf{b}$ and $\mathrm{Cov}(\mathbf{y}) = A\Sigma A^\top$.

Common probability distributions

Plain version. A handful of distributions appear over and over. Choosing one is choosing an assumption about your data or your target.

Bernoulli

A single binary variable $X \in \{0, 1\}$ with parameter $\phi \in [0, 1]$:

$$P(X=1)=\phi, \quad P(X=0)=1-\phi, \quad P(X=x)=\phi^{x}(1-\phi)^{1-x}.$$

Its mean is ϕ and its variance $\phi(1-\phi)$. A binary classifier with a sigmoid output predicts the parameter ϕ of a Bernoulli distribution over the label.

Categorical (multinoulli)

A single discrete variable with K states, parameterized by a probability vector $\mathbf{p} \in [0,1]^K$ with $\sum_k p_k = 1$:

$$P(X=k)=p_k.$$

If the label is encoded one-hot as $\mathbf{y} \in \{0,1\}^K$, then $P(\mathbf{y}) = \prod_k p_k^{y_k}$. A softmax classifier outputs exactly such a vector. (The *multinomial* distribution counts outcomes over several draws; for one draw it reduces to the categorical, which Goodfellow et al. call "multinoulli" [1].)

Gaussian (normal)

The most widely used continuous distribution:

$$\mathcal{N}(x;\mu,\sigma^2)=\frac{1}{\sqrt{2\pi\sigma^2}}\exp\Big(-\frac{(x-\mu)^2}{2\sigma^2}\Big).$$

μ is the mean (and the peak), σ^2 the variance. Sometimes we use the **precision** $\beta = 1/\sigma^2$ instead, which is cheaper to evaluate when the variance changes often.

Two reasons make the Gaussian the default. The **central limit theorem** says that the sum of many independent variables with finite variance tends towards a Gaussian, so measurement noise built from many small effects is often near-Gaussian. And among all distributions with a given mean and variance on the real line, the Gaussian has the **maximum entropy**: it builds in the least extra structure.

In n dimensions,

$$\mathcal{N}(\mathbf{x};\boldsymbol{\mu},\Sigma)=\frac{1}{\sqrt{(2\pi)^n \det\Sigma}}\exp\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^{\top}\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\Big),$$

where $\boldsymbol{\mu} \in \mathbb{R}^n$ is the mean vector and $\Sigma \in \mathbb{R}^{n\times n}$ is a symmetric positive-definite covariance matrix. The **precision matrix** $\Lambda = \Sigma^{-1}$ has a neat meaning: $\Lambda_{ij} = 0$ exactly when x_i and x_j are conditionally independent given all other coordinates. An **isotropic** Gaussian has $\Sigma = \sigma^2 I$ — the noise distribution used by diffusion models (ML 07).

A fundamental link to the losses you will use: if a regression network predicts the mean of a Gaussian with fixed variance, maximizing the likelihood is the same as minimizing the **mean squared error**.

Exponential and Laplace

Sometimes we want a sharp point at a specific location.

$$p(x;\lambda) = \lambda\,\mathbf{1}_{x\geq 0}\,e^{-\lambda x}, \qquad \text{Laplace}(x;\mu,\gamma) = \frac{1}{2\gamma}\exp\Big(-\frac{|x-\mu|}{\gamma}\Big).$$

$\lambda > 0$ is the rate, $\mathbf{1}_{x\geq 0}$ is the indicator that is 1 when $x \geq 0$ and 0 otherwise, μ is the location and $\gamma > 0$ the scale. Compared with the Gaussian, the Laplace density has a sharper peak and heavier tails (Figure 1, bottom left). Predicting the location of a Laplace distribution by maximum likelihood is equivalent to minimizing the **absolute error** (L_1 loss), which is why L_1 is more tolerant of outliers than L_2; a Laplace *prior* on weights gives L_1 regularization (sparsity).

Dirac and empirical distributions

The **Dirac delta** $\delta(x-\mu)$ puts all of its mass at the point μ: it is zero everywhere else yet integrates to one. It is a generalized function, defined by what it does inside integrals: $\int \delta(x-\mu)f(x)\,dx = f(\mu)$.

A training set $\{\mathbf{x}^{(1)},\dots,\mathbf{x}^{(m)}\}$ defines the **empirical distribution**

$$\hat{p}(\mathbf{x}) = \frac{1}{m}\sum_{i=1}^{m}\delta(\mathbf{x}-\mathbf{x}^{(i)}),$$

which puts mass $1/m$ on each of the m examples. Training a model by averaging a loss over the dataset is the same as taking an expectation under $\hat{p}$. It is also the distribution that maximizes the likelihood of the training data — which is precisely why minimizing training loss alone overfits.

Mixtures and latent variables

Plain version. A mixture first picks a "type" at random, then draws a sample from the distribution belonging to that type.

$$P(\mathbf{x}) = \sum_{k=1}^{K} P(z=k)\,P(\mathbf{x}\mid z=k).$$

z is a categorical **latent variable** — a variable we never observe directly — chosen with probabilities $\pi_k = P(z=k)$, and $P(\mathbf{x}\mid z=k)$ is the k-th component. A **Gaussian mixture model** (GMM) uses Gaussian components $\mathcal{N}(\mathbf{x};\boldsymbol{\mu}_k,\Sigma_k)$; with enough components it can approximate any smooth density to any desired nonzero error [1]. Given an observation $\mathbf{x}$, Bayes' rule gives the **posterior responsibility** $P(z=k\mid\mathbf{x})$: how likely it is that component k generated that point. Fitting a GMM with expectation–maximization is the subject of PR 07.

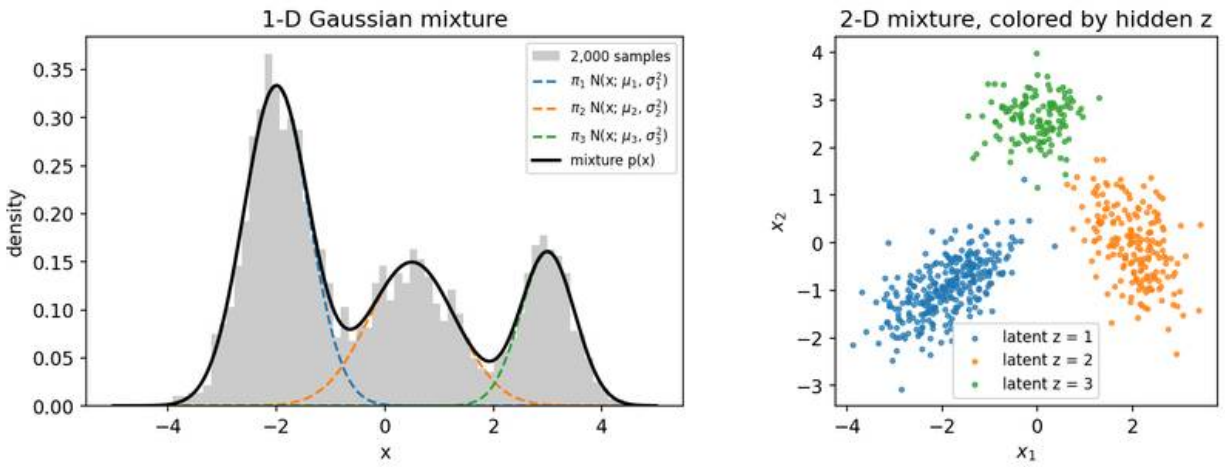


Figure 2 — A Gaussian mixture. Left: the black density is the weighted sum of three components; the grey histogram is 2,000 samples drawn by first picking a component and then sampling from it. Right: in two dimensions, colour shows the hidden component z, which a real dataset would not reveal.

Latent variables are central to modern generative models. A VAE (ML 06) is, informally, a mixture with a *continuous* latent $\mathbf{z}$ and a neural network for $p(\mathbf{x} \mid \mathbf{z})$ [8]; the marginal $p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})\, d\mathbf{z}$ is then intractable and must be approximated.

```
import numpy as np

rng = np.random.default_rng(0)
pi = np.array([0.5, 0.3, 0.2])      # mixing weights P(z = k)
mu = np.array([-2.0, 0.5, 3.0])     # component means
sd = np.array([0.6, 0.8, 0.5])      # component standard deviations

# Ancestral sampling: first the hidden component z, then x given z
z = rng.choice(3, size=100_000, p=pi)
x = rng.normal(mu[z], sd[z])

def log_normal(x, m, s):
    return -0.5*((x - m)/s)**2 - np.log(s) - 0.5*np.log(2*np.pi)

def mixture_logpdf(x):
    # log sum_k pi_k N(x; mu_k, sd_k^2), computed with log-sum-exp
    a = np.log(pi) + log_normal(x[:, None], mu, sd)
    m = a.max(axis=1, keepdims=True)
    return (m + np.log(np.exp(a - m).sum(axis=1, keepdims=True))).ravel()

grid = np.linspace(-6, 6, 4001)
print("density integrates to",
 ↪  (np.exp(mixture_logpdf(grid)).sum()*(grid[1]-grid[0])).round(4))
print("sample mean", x.mean().round(3), "vs exact", (pi*mu).sum().round(3))
# Posterior responsibility P(z | x) for one observation
a = np.log(pi) + log_normal(np.array([0.0])[:, None], mu, sd)
resp = np.exp(a - a.max()); resp /= resp.sum()
```

```
print("P(z | x=0):", resp.round(3))
```

The density integrates to 1.0, the sample mean (-0.255) matches the exact mean $\sum_k \pi_k \mu_k = -0.25$, and the point $x = 0$ is assigned to the middle component with probability 0.99. The log-sum-exp trick used here is explained in ML 01C.

Useful functions: sigmoid and softplus

Plain version. The sigmoid squashes any real number into a probability between 0 and 1; the softplus is a smooth version of $\max(0, x)$ that is always positive.

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \qquad \zeta(x) = \log\left(1 + e^{x}\right).$$

The **logistic sigmoid** σ produces the parameter ϕ of a Bernoulli distribution; it **saturates** (becomes flat) for large $|x|$, which makes gradients small there. The **softplus** ζ produces a positive number, such as a standard deviation σ or a precision β; its name comes from being a softened "positive part" $x^+ = \max(0, x)$.

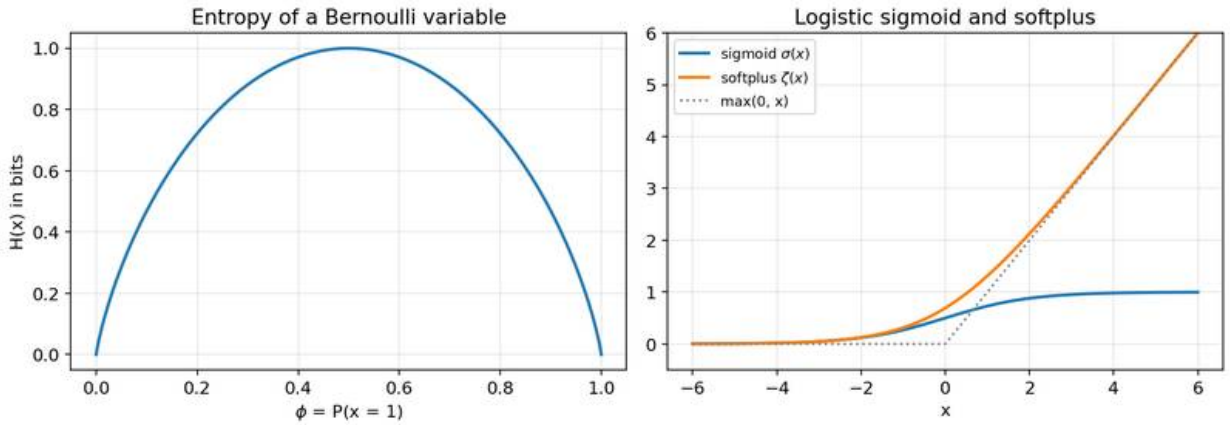


Figure 3 — Left: entropy of a Bernoulli variable, maximal (1 bit) at $\varphi = 0.5$ and zero when the outcome is certain. Right: the logistic sigmoid $\sigma(x)$ and the softplus $\zeta(x)$, a smooth version of $\max(0, x)$.

These identities are worth memorizing, because they appear every time a loss is simplified [1]:

$$1 - \sigma(x) = \sigma(-x), \qquad \frac{d}{dx}\sigma(x) = \sigma(x)(1 - \sigma(x)),$$

$$\log \sigma(x) = -\zeta(-x), \qquad \frac{d}{dx}\zeta(x) = \sigma(x),$$

$$\sigma^{-1}(p) = \log \frac{p}{1-p}, \qquad \zeta(x) - \zeta(-x) = x.$$

σ^{-1} is the **logit** (log-odds) function. The identity $\log \sigma(x) = -\zeta(-x)$ shows why binary cross-entropy should be computed from logits: $-\log \sigma(x) = \zeta(-x)$, and softplus can be evaluated without overflow.

```
import numpy as np

def sigmoid(x):
    return 1/(1 + np.exp(-x))

def softplus(x):
    return np.logaddexp(0, x)          # log(1 + exp(x)) without overflow

x = np.linspace(-30, 30, 13)
assert np.allclose(1 - sigmoid(x), sigmoid(-x))
assert np.allclose(np.log(sigmoid(x)), -softplus(-x))
assert np.allclose(softplus(x) - softplus(-x), x)
h = 1e-6
assert np.allclose((softplus(x+h) - softplus(x-h))/(2*h), sigmoid(x), atol=1e-6)
p = sigmoid(x[4:9])
assert np.allclose(np.log(p/(1-p)), x[4:9])   # logit inverts sigmoid
print("naive log(1+exp(1000)):", np.log1p(np.exp(np.float64(1000))))
print("stable softplus(1000):", softplus(1000.0))
```

All identities pass. The naive expression returns `inf` (with an overflow warning) while `np.logaddexp` returns 1000.0.

Bayes' rule

Plain version. Bayes' rule turns "how likely is the evidence if the hypothesis is true?" into "how likely is the hypothesis now that I have seen the evidence?" — and it insists that you account for how common the hypothesis was in the first place.

From the product rule written both ways, $P(a, b) = P(a \mid b)P(b) = P(b \mid a)P(a)$, so

$$P(a \mid b) = \frac{P(b \mid a)\, P(a)}{P(b)}, \qquad P(b) = \sum_{a'} P(b \mid a')\, P(a').$$

- $P(a)$ is the **prior**: how plausible a is before seeing the evidence.
- $P(b \mid a)$ is the **likelihood** of the evidence b under hypothesis a.
- $P(a \mid b)$ is the **posterior**: the updated plausibility.
- $P(b)$ is the **evidence** (or marginal likelihood), the normalizer obtained by summing over all hypotheses a'.

A worked example: the base-rate effect

Consider a hypothetical defect detector with **sensitivity** $P(+ \mid \text{defect}) = 0.9$, **false-positive rate** $P(+ \mid \text{no defect}) = 0.05$, and a **prevalence** (base rate) $P(\text{defect}) = 0.01$. The probability that an item flagged as positive really is defective is

$$P(\text{defect} \mid +) = \frac{0.9 \cdot 0.01}{0.9 \cdot 0.01 + 0.05 \cdot 0.99} \approx 0.154.$$

Only about 15% of alarms are real, even though the detector catches 90% of defects. Counting makes this obvious: among 10,000 items, 100 are defective and 90 of them raise an alarm, but 9,900 are fine and 495 of them also raise an alarm (Figure 4, right). The same detector at 10% prevalence gives $0.09/(0.09 + 0.045) = 2/3$. This is a mathematical illustration, not a performance claim about any real system. Its lesson carries over to every deployed classifier: **sensitivity and false-positive rate alone do not determine how trustworthy a positive prediction is; the deployment prevalence matters.** ML 10 returns to this when discussing precision under distribution shift.

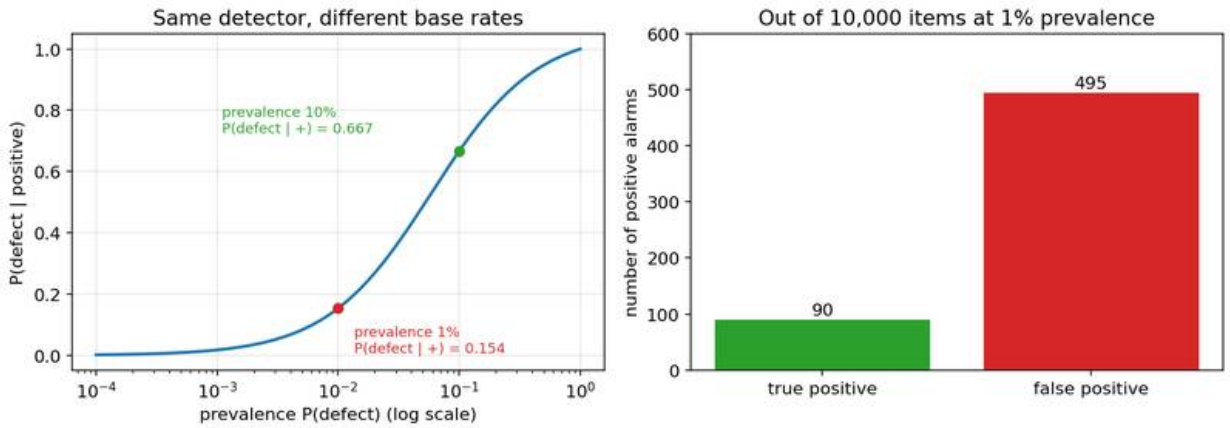


Figure 4 — The base-rate effect for a hypothetical detector with sensitivity 0.9 and false-positive rate 0.05. Left: the posterior $P(\text{defect} \mid +)$ depends strongly on prevalence. Right: at 1% prevalence, false alarms outnumber true alarms more than five to one.

```
import numpy as np

def posterior_positive(prevalence, sensitivity=0.9, false_pos=0.05):
    evidence = sensitivity*prevalence + false_pos*(1 - prevalence)   # P(+)
    return sensitivity*prevalence / evidence                          # P(defect | +)

for prev in [0.001, 0.01, 0.1, 0.5]:
    print(f"prevalence {prev:>5}: P(defect | +) = {posterior_positive(prev):.3f}")

# Simulation check with 1,000,000 synthetic items
```

```
rng = np.random.default_rng(0)
defect = rng.random(1_000_000) < 0.01
alarm = np.where(defect, rng.random(defect.size) < 0.9, rng.random(defect.size) < 0.05)
print("simulated:", defect[alarm].mean().round(3))
```

The formula gives 0.018, 0.154, 0.667 and 0.947 for the four prevalences, and a one-million-item simulation reproduces 0.154.

Bayes' rule also underlies the **Bayes classifier**: predicting the class k with the largest posterior $P(y = k \mid \mathbf{x}) \propto p(\mathbf{x} \mid y = k)P(y = k)$ minimizes the expected error rate. PR 01 develops this decision theory in depth.

Change of variables for densities

Plain version. If you stretch a rubber sheet, the ink on it gets thinner. When a random variable is transformed, its density must be rescaled by how much the transformation stretches space.

A tempting mistake: if $y = g(x)$ and x has density p_x, then $p_y(y) = p_x(g^{-1}(y))$. This is wrong, because it does not integrate to one. The correct rule for an invertible, differentiable g in one dimension is

$$p_y(y) = p_x(g^{-1}(y)) \left| \frac{d\, g^{-1}(y)}{dy} \right|.$$

The probability in a small interval must be preserved: $|p_y(y)\,dy| = |p_x(x)\,dx|$. In n dimensions, with $\mathbf{y} = g(\mathbf{x})$,

$$p_x(\mathbf{x}) = p_y(g(\mathbf{x})) \left| \det \frac{\partial g(\mathbf{x})}{\partial \mathbf{x}} \right|,$$

where $\partial g / \partial \mathbf{x}$ is the $n \times n$ **Jacobian matrix** of g (entry (i, j) is $\partial y_i / \partial x_j$) and $|\det(\cdot)|$ is the absolute value of its determinant — the factor by which g scales a small volume (see ML 01A for determinants). This formula is the entire foundation of **normalizing flows**, which stack invertible networks with cheap Jacobian determinants to obtain an exact likelihood [9].

```
import numpy as np

rng = np.random.default_rng(0)
x = rng.normal(0, 1, 500_000)        # x ~ N(0, 1)
y = np.exp(x)                        # y = g(x), invertible on (0, inf)

def p_x(x):
    return np.exp(-0.5*x**2)/np.sqrt(2*np.pi)

def p_y(y):
```

```
    # p_y(y) = p_x(g^{-1}(y)) * |d g^{-1}(y) / dy| = p_x(log y) / y
    return p_x(np.log(y))/y

counts, edges = np.histogram(y, bins=np.linspace(0.05, 4, 80), density=False)
centers = 0.5*(edges[:-1] + edges[1:])
empirical = counts/(y.size*np.diff(edges))
print("max |histogram - formula|:", np.abs(empirical - p_y(centers)).max().round(3))
print("wrong formula (no Jacobian) error:", np.abs(empirical -
 ↪  p_x(np.log(centers))).max().round(3))
```

With the Jacobian factor $1/y$ the histogram matches the formula to within 0.011 (sampling noise); without it the error is 0.46.

Likelihood, maximum likelihood and MAP

Plain version. Hold the observed data fixed and ask which parameter values make that data most plausible. If you also have prior knowledge about the parameters, weigh that in too.

Suppose a model $p_{\boldsymbol{\theta}}(\mathbf{x})$ has parameters $\boldsymbol{\theta}$, and we observe a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ of independent and identically distributed (i.i.d.) examples. The **likelihood** is

$$L(\boldsymbol{\theta}) = p_{\boldsymbol{\theta}}(\mathcal{D}) = \prod_{i=1}^{m} p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}).$$

As a function of $\boldsymbol{\theta}$, this is *not* a probability distribution over parameters: it need not integrate to one over $\boldsymbol{\theta}$. The **maximum likelihood estimate** (MLE) is

$$\boldsymbol{\theta}_{\mathrm{ML}} = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{m} \log p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = \arg\min_{\boldsymbol{\theta}} \; \mathbb{E}_{\mathbf{x}\sim\hat{p}}[\,-\log p_{\boldsymbol{\theta}}(\mathbf{x})\,].$$

We take logarithms because a product of many numbers below one underflows, while a sum of logs is numerically safe and has the same maximizer. The right-hand form says MLE **minimizes the average negative log-likelihood** (NLL) under the empirical distribution $\hat{p}$. For a classifier $p_{\boldsymbol{\theta}}(y \mid \mathbf{x})$ the same idea gives the conditional NLL $-\log p_{\boldsymbol{\theta}}(y \mid \mathbf{x})$, which is the cross-entropy loss we meet below.

Two classic results: for a Bernoulli coin with k heads in n flips, $\phi_{\mathrm{ML}} = k/n$; for a Gaussian, μ_{ML} is the sample mean and $\sigma^2_{\mathrm{ML}} = \frac{1}{m}\sum_i (x^{(i)} - \mu_{\mathrm{ML}})^2$ — a slightly *biased* estimate (dividing by $m-1$ removes the bias).

MAP estimation

Bayesian inference treats $\boldsymbol{\theta}$ itself as a random variable with a **prior** $p(\boldsymbol{\theta})$ and computes the **posterior**

$$p(\boldsymbol{\theta} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})}{p(\mathcal{D})} \;\propto\; p(\mathcal{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}).$$

The full posterior is often intractable, so a common shortcut is its mode, the **maximum a posteriori** (MAP) estimate:

$$\boldsymbol{\theta}_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} \left[\log p(\mathcal{D} \mid \boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) \right].$$

With a Gaussian prior $p(\boldsymbol{\theta}) = \mathcal{N}(\mathbf{0}, \frac{1}{\lambda} I)$, the term $-\log p(\boldsymbol{\theta})$ equals $\frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2$ plus a constant: **weight decay is MAP with a Gaussian prior**. A Laplace prior gives L_1 regularization. This links probabilistic assumptions to the regularizers in ML 02. But a MAP point estimate is still a single parameter vector; it does not tell you how uncertain the parameters are. Approximating the whole posterior is the topic of PR 08 and of variational inference [10].

```
import numpy as np

rng = np.random.default_rng(0)
flips = rng.random(10) < 0.7          # 10 coin flips, true P(heads) = 0.7
k, n = flips.sum(), flips.size

phi = np.linspace(1e-3, 1 - 1e-3, 999)
log_lik = k*np.log(phi) + (n - k)*np.log(1 - phi)
a, b = 5.0, 5.0                        # Beta(5, 5) prior, centered on 0.5
log_prior = (a - 1)*np.log(phi) + (b - 1)*np.log(1 - phi)

print(f"{k} heads in {n} flips")
print("MLE  grid:", phi[log_lik.argmax()].round(3), " closed form:", k/n)
print("MAP  grid:", phi[(log_lik + log_prior).argmax()].round(3),
      " closed form:", round((k + a - 1)/(n + a + b - 2), 3))

# Gaussian MLE: sample mean and the 1/m (biased) variance
x = rng.normal(2.0, 3.0, 50)
print("Gaussian MLE mean, var:", x.mean().round(3), x.var().round(3),
      " unbiased var:", x.var(ddof=1).round(3))
```

With this seed, 6 heads appear in 10 flips. The MLE is 0.6; the Beta$(5, 5)$ prior pulls the MAP estimate towards 0.5, giving 0.556, matching the closed form $(k + a - 1)/(n + a + b - 2)$. With more data the likelihood dominates and the two estimates converge. The Gaussian MLE variance (7.71) is smaller than the unbiased estimate (7.87), as expected.

Information theory

Plain version. Information theory measures surprise. A rare event carries a lot of information when it happens; a certain event carries none. Averaging surprise gives entropy, and the extra surprise from using the wrong distribution gives KL divergence.

Claude Shannon founded the field to study how many bits are needed to send messages over a noisy channel [5]; the textbook treatment is Cover and Thomas [7]. For machine learning, the useful idea is to quantify how different two distributions are.

Self-information and entropy

The **self-information** of an outcome x is

$$I(x) = -\log P(x).$$

With the natural logarithm the unit is the **nat**; with $\log_2$ it is the **bit** (one nat $= 1/\ln 2 \approx 1.443$ bits). Certain events ($P = 1$) carry zero information; unlikely events carry a lot; and information from independent events adds, because logs turn products into sums.

The **Shannon entropy** is the expected self-information:

$$H(P) = \mathbb{E}_{x\sim P}[I(x)] = -\sum_x P(x)\log P(x).$$

We use the convention $0\log 0 = 0$, justified by the limit $p\log p \to 0$ as $p \to 0$. Entropy is the minimum average number of bits (or nats) needed to encode outcomes drawn from P. It is zero for a deterministic outcome and maximal, $\log K$, for a uniform distribution over K outcomes. For a Bernoulli variable with parameter ϕ, $H = -\phi\log\phi - (1-\phi)\log(1-\phi)$, which peaks at one bit when $\phi = 0.5$ (Figure 3, left).

For continuous variables, $h(p) = -\int p(x)\log p(x)\,dx$ is the **differential entropy**. Treat it carefully: it can be negative (a narrow uniform distribution on $[0, \frac{1}{2}]$ has $h = -\log 2$) and it changes when you rescale the coordinates. It is not a limiting version of the discrete count of bits.

KL divergence

The **Kullback–Leibler divergence** [6] measures how different a distribution Q is from a reference P:

$$D_{\mathrm{KL}}(P\,\|\,Q) = \mathbb{E}_{x\sim P}\Big[\log\frac{P(x)}{Q(x)}\Big] = \sum_x P(x)\log\frac{P(x)}{Q(x)}.$$

In coding terms, it is the *extra* number of nats needed to encode samples from P when the code was designed for Q. Its key properties:

- **Nonnegative**: $D_{\mathrm{KL}}(P\|Q) \geq 0$, with equality if and only if $P = Q$ (for discrete variables; "almost everywhere" for continuous ones). This follows from Jensen's inequality, because $-\log$ is convex.
- **Infinite under unsupported mass**: if $Q(x) = 0$ for some x with $P(x) > 0$, the divergence is infinite.
- **Asymmetric**: in general $D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$, and it does not satisfy the triangle inequality. KL is therefore *not* a distance metric, even though it is often described as "the distance between distributions".

Forward versus reverse KL

The asymmetry has visible consequences when the approximating family is too simple. Figure 5 fits a single Gaussian q to a target p with two well-separated modes.

- Minimizing **forward KL** $D_{\mathrm{KL}}(p\|q)$ heavily penalizes q for being small where p has mass. The best Gaussian therefore **covers both modes**, placing its mean between them ($\mu = 0$, $\sigma \approx 2.62$) and putting a lot of mass in the empty valley. For Gaussian q, the forward-KL optimum simply matches the mean and variance of p.
- Minimizing **reverse KL** $D_{\mathrm{KL}}(q\|p)$ heavily penalizes q for being large where p is small. The best Gaussian therefore **locks onto one mode** ($\mu \approx 2.49$, $\sigma \approx 0.81$) and ignores the other.

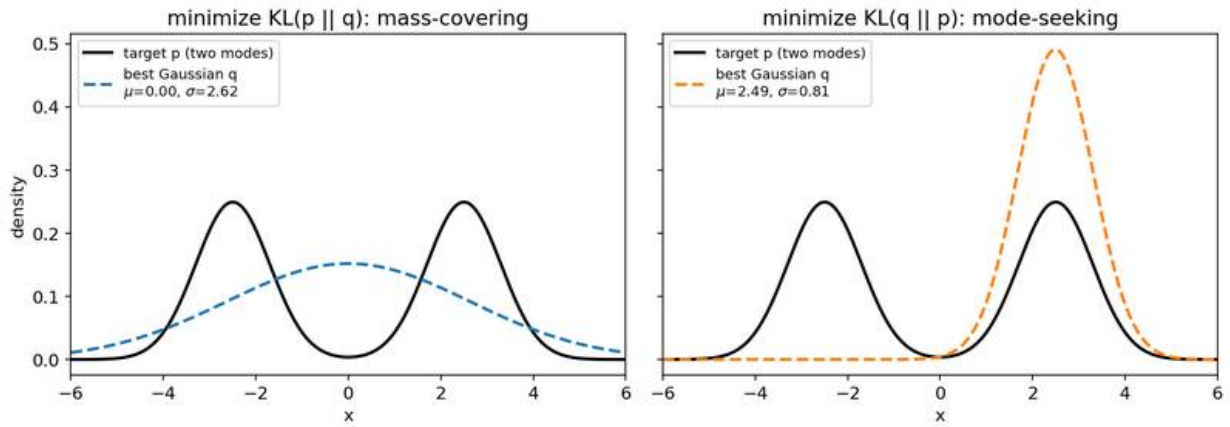


Figure 5 — KL is asymmetric. Fitting one Gaussian q to a two-mode target p: forward $KL(p\|q)$ gives a mass-covering fit centred between the modes; reverse $KL(q\|p)$ gives a mode-seeking fit on one peak (the other peak gives an equally good mirror solution).

Which direction you minimize is a design choice. **Maximum likelihood minimizes forward KL** from the data distribution to the model, so likelihood-trained models (autoregressive models, flows, VAEs) tend to spread probability over all the data, sometimes producing blurry samples.

Variational inference usually minimizes reverse KL from the approximation to the true posterior [10], and so tends to underestimate posterior uncertainty.

Cross-entropy

The **cross-entropy** of Q relative to P is

$$H(P, Q) = -\mathbb{E}_{x\sim P}[\log Q(x)] = H(P) + D_{\mathrm{KL}}(P \,\|\, Q).$$

Because $H(P)$ does not depend on Q, minimizing cross-entropy with respect to Q is the same as minimizing forward KL. In classification, P is the empirical label distribution — a one-hot vector for each example — and Q is the model's softmax output $\hat{\mathbf{y}}$. For one example with true class y,

$$H(\mathbf{e}_y, \hat{\mathbf{y}}) = -\sum_k [\mathbf{e}_y]_k \log \hat{y}_k = -\log \hat{y}_y,$$

where $\mathbf{e}_y$ is the one-hot vector for class y. Averaged over a dataset, this is exactly the conditional negative log-likelihood. So "cross-entropy loss", "log loss" and "negative log-likelihood of a categorical model" are three names for the same objective. Any loss consisting of a negative log-likelihood is a cross-entropy between the empirical distribution and the model — which is why the mean squared error is also "the cross-entropy" for a Gaussian model [1].

Mutual information

Mutual information measures how much knowing one variable reduces uncertainty about another:

$$I(X; Y) = D_{\mathrm{KL}}(p(x, y) \,\|\, p(x)p(y)) = H(X) - H(X \mid Y) = H(X) + H(Y) - H(X, Y).$$

Here $H(X \mid Y) = -\mathbb{E}_{x,y}[\log p(x \mid y)]$ is the conditional entropy and $H(X, Y)$ the joint entropy. $I(X; Y) \geq 0$, with equality if and only if X and Y are independent — unlike covariance, it detects any form of dependence. It is symmetric in X and Y.

Code: information quantities

```
import numpy as np

p = np.array([.7, .2, .1])
q = np.array([.5, .3, .2])
entropy = -(p*np.log(p)).sum()
cross_entropy = -(p*np.log(q)).sum()
```

```
kl = (p*np.log(p/q)).sum()
reverse_kl = (q*np.log(q/p)).sum()
assert np.allclose(cross_entropy - entropy, kl) and kl >= 0
print("H(p), H(p,q), KL(p||q), KL(q||p) in nats:",
      np.round([entropy, cross_entropy, kl, reverse_kl], 4))
print("H(p) in bits:", (entropy/np.log(2)).round(4))

# Mutual information from a joint table (same weather/umbrella table)
P = np.array([[0.56, 0.14],
              [0.06, 0.24]])
px, py = P.sum(1), P.sum(0)
mi = (P*np.log(P/np.outer(px, py))).sum()
H = lambda t: -(t[t > 0]*np.log(t[t > 0])).sum()
assert np.isclose(mi, H(px) + H(py) - H(P.ravel()))
print("I(weather; umbrella) =", mi.round(4), "nats")

# Cross-entropy loss on one-hot labels equals mean -log q(y)
probs = np.array([[0.7, 0.2, 0.1], [0.1, 0.8, 0.1], [0.3, 0.3, 0.4]])
labels = np.array([0, 1, 2])
print("mean cross-entropy:", (-np.log(probs[np.arange(3), labels])).mean().round(4))
```

Output: $H(p) = 0.8018$, $H(p,q) = 0.8869$, $D_{\mathrm{KL}}(p\|q) = 0.0851$ and $D_{\mathrm{KL}}(q\|p) = 0.0920$ nats — the two directions differ. $H(p)$ is 1.1568 bits. The weather–umbrella mutual information is 0.1637 nats, and both formulas for it agree. The mean cross-entropy of the three predictions is 0.4987 nats. In real training code, compute the cross-entropy from logits with a fused function rather than from probabilities (see ML 01C).

Structured probabilistic models

Plain version. A joint distribution over many variables is far too big to write down as a table. Structured models draw a graph saying which variables directly influence which, and then the joint splits into small pieces.

A joint table over 100 binary variables would need $2^{100} - 1$ numbers. **Structured probabilistic models**, also called **graphical models**, exploit conditional independence to factorize the joint into small factors [1], [3]. There are two main kinds.

Directed models (Bayesian networks) use a directed acyclic graph $\mathcal{G}$. Each variable depends only on its parents $Pa_{\mathcal{G}}(x_i)$:

$$p(\mathbf{x}) = \prod_i p(x_i \mid Pa_{\mathcal{G}}(x_i)).$$

For example, with $a \to b \to c$ and $a \to c$ we have $p(a,b,c) = p(a)\,p(b \mid a)\,p(c \mid a,b)$; deleting the edge $a \to c$ asserts $c \perp a \mid b$ and shortens the last factor to $p(c \mid b)$. Directed graphs are natural for

generative stories ("first choose the class, then the pixels"), and a mixture model is the simplest example: $z \to \mathbf{x}$.

Undirected models (Markov random fields) use an undirected graph. Each **clique** $C^{(i)}$ (a set of mutually connected nodes) has a nonnegative **factor** $\phi^{(i)}$, and

$$p(\mathbf{x}) = \frac{1}{Z} \prod_i \phi^{(i)}(C^{(i)}), \qquad Z = \sum_{\mathbf{x}} \prod_i \phi^{(i)}(C^{(i)}).$$

The **partition function** Z normalizes the product; computing it is often the hard part. Undirected models suit symmetric interactions, like neighbouring pixels in an image prior or a conditional random field for segmentation.

A graph is a *description* of a distribution, not a different kind of distribution: any distribution can be written either way, although one may be much more compact. Deep learning often uses these ideas implicitly — an autoregressive model is a fully connected directed model, and a VAE is a two-node directed model $\mathbf{z} \to \mathbf{x}$ whose conditional is a neural network [8].

Modern view

The probability and information theory in this chapter is classical. What has changed in the deep-learning era is *how* it is used and what goes wrong in practice. This section surveys the review literature and recent directions; it summarizes their framing rather than reproducing benchmark numbers.

Calibration: are predicted probabilities honest?

A classifier is **calibrated** if, among all inputs for which it predicts confidence 0.8, about 80% are correct. Guo et al. [11] (ICML 2017) found that modern deep networks are often **overconfident**, and identified depth, width, weight decay and batch normalization as factors that affect calibration. Their main recommendation, **temperature scaling** — dividing all logits by a single scalar T fitted on a validation set — is a one-parameter variant of Platt scaling and remains a strong baseline because it changes confidence without changing the predicted class. Wang's survey [12] organizes calibration methods into four families: post-hoc calibration, regularization during training, uncertainty estimation, and compositions of these; its revised version also covers calibration of large language models. The probabilistic lesson: a softmax output is a model of $P(y \mid \mathbf{x})$, and like any model it can be wrong in its uncertainty, not only in its argmax.

Uncertainty quantification

Two major reviews map the field of uncertainty in deep learning. Abdar et al. [13] (*Information Fusion*, 2021) review Bayesian approximations, ensembles and their applications across many domains. Gawlikowski et al. [14] (*Artificial Intelligence Review*, 2023) distinguish **reducible model (epistemic) uncertainty** from **irreducible data (aleatoric) uncertainty** — roughly the third and first sources of uncertainty from the start of this chapter — and compare deterministic single networks, Bayesian methods, ensembles and test-time augmentation, together with calibration measures.

Two primary studies are especially influential. **Deep ensembles** [15] train several networks from different random initializations and average their predictive distributions; this simple, parallel method was competitive with approximate Bayesian neural networks and expressed higher uncertainty on out-of-distribution inputs. Ovadia et al. [16] (NeurIPS 2019) benchmarked uncertainty methods under **dataset shift** and found that post-hoc calibration on in-distribution validation data did not carry over well, while methods that marginalize over models (such as ensembles) held up better.

A complementary, assumption-light approach is **conformal prediction**. Angelopoulos and Bates [17] give an accessible introduction: using a held-out calibration set, conformal methods turn any pre-trained model's scores into prediction *sets* that contain the true label with a user-chosen probability, assuming the calibration and test data are exchangeable. It does not require the model's probabilities to be calibrated, which makes it attractive for vision and language systems that are used as black boxes.

Mutual information in representation learning

Mutual information became a popular objective for self-supervised learning (ML 05). Contrastive Predictive Coding [18] introduced the **InfoNCE** loss, which classifies a positive pair among negatives and can be shown to maximize a lower bound on mutual information. Poole et al. [19] (ICML 2019) unified many variational MI bounds and showed that they degrade when the true MI is large, trading high bias against high variance. Tschannen et al. [20] (ICLR 2020) argued that the success of these methods "cannot be attributed to the properties of MI alone": the encoder architecture and the parameterization of the estimator matter at least as much. The modern consensus is to treat InfoNCE-style objectives as well-behaved contrastive losses with an information-theoretic motivation, not as precise MI estimators.

Divergences and latent variables in generative models

Generative modelling is applied probability. VAEs [8] maximize an evidence lower bound that combines a reconstruction likelihood with a KL term; Blei et al. [10] review variational inference,

which approximates intractable posteriors by minimizing (reverse) KL within a tractable family. Normalizing flows [9] are built entirely on the change-of-variables formula. Diffusion and flow-matching models (ML 07) combine Gaussian noise, conditional expectations and likelihood bounds. The choice of divergence (forward KL, reverse KL, or something else) shapes whether a model covers all the data or concentrates on its modes — the same picture as Figure 5, at scale.

Open problems

- **Calibration under shift.** Calibration that holds on a validation set can fail on new cameras, hospitals or prompts [16]; detecting when this happens is an open question.
- **Separating uncertainty types.** Decomposing predictive uncertainty into epistemic and aleatoric parts is conceptually clear but hard to do reliably for large networks [14].
- **Estimating information in high dimensions.** Variational MI bounds have fundamental bias–variance limits [19], so information-theoretic claims about deep representations need care.
- **Uncertainty for foundation models.** Applying calibration and conformal methods to large vision–language and language models, whose outputs are open-ended, is an active research area [12], [17].

Key takeaways

- Probability describes uncertainty from randomness, partial observation and simplified models; frequentist and Bayesian readings share the same rules.
- A PMF assigns probabilities; a PDF assigns densities, which can exceed one. Probabilities of continuous events are areas.
- The sum rule (marginalization), product rule (conditioning) and chain rule are exact; independence and conditional independence are modelling assumptions.
- Covariance measures linear co-variation only — zero covariance does not imply independence; mutual information detects any dependence.
- Choosing a distribution is choosing a loss: Bernoulli/categorical give cross-entropy, Gaussian gives squared error, Laplace gives absolute error; Gaussian and Laplace priors give L_2 and L_1 regularization.
- Bayes' rule combines evidence with base rates; the same detector can be trustworthy at one prevalence and misleading at another.
- Transforming a random variable rescales its density by the absolute Jacobian determinant.
- Cross-entropy equals entropy plus KL; minimizing it is maximum likelihood. KL is asymmetric: forward KL covers modes, reverse KL seeks one.

Exercises

1. **Units and certainty.** (a) Convert a loss of 1 nat into bits. (b) Compute the entropy, in bits, of a fair coin, a certain outcome, and a fair 8-sided die.

   Hint

   (a) Divide by $\ln 2$: about 1.443 bits. (b) 1 bit, 0 bits and $\log_2 8 = 3$ bits. A uniform distribution over K outcomes has entropy $\log K$, the maximum for K outcomes.

2. **Prevalence shift.** Using the detector in the Bayes section, find the prevalence at which a positive result is exactly a coin flip, $P(\text{defect} \mid +) = 0.5$.

   Hint

   Set $0.9\pi = 0.05(1 - \pi)$, where π is the prevalence. Then $\pi = 0.05/0.95 \approx 0.053$, about 5.3%. Below that, most alarms are false.

3. **Dependent but uncorrelated.** Let X take values $-1, 0, 1$ with equal probability and $Y = X^2$. Show that $\mathrm{Cov}(X, Y) = 0$ and compute $I(X; Y)$ in bits.

   Hint

   $\mathbb{E}[X] = 0$ and $\mathbb{E}[XY] = \mathbb{E}[X^3] = 0$, so the covariance is zero. Because Y is a function of X, $H(Y \mid X) = 0$ and $I(X; Y) = H(Y)$. $Y = 1$ with probability $2/3$ and $Y = 0$ with probability $1/3$, so $I \approx 0.918$ bits.

4. **Gaussian MLE equals squared error.** Show that maximizing $\prod_i \mathcal{N}(y^{(i)}; f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), \sigma^2)$ over $\boldsymbol{\theta}$ (with σ fixed) is equivalent to minimizing the mean squared error. What changes if σ is also predicted by the network?

   Hint

   The negative log-likelihood is $\sum_i \frac{(y^{(i)} - f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))^2}{2\sigma^2} + m \log \sigma + \text{const}$. With σ fixed only the squared error depends on $\boldsymbol{\theta}$. If the network also predicts $\sigma(\mathbf{x})$ (for example through a softplus), each example's error is weighted by $1/\sigma^2$ and a $\log \sigma$ penalty prevents the variance from growing without bound — a heteroscedastic regression model.

5. **Change of variables.** If $X \sim \mathcal{U}(0, 1)$ (uniform) and $Y = -\frac{1}{\lambda} \log X$ with $\lambda > 0$, what is the density of Y? Check your answer by sampling in NumPy.

   Hint

   $g^{-1}(y) = e^{-\lambda y}$ and $|dg^{-1}/dy| = \lambda e^{-\lambda y}$, while $p_X = 1$ on $(0, 1)$. So $p_Y(y) = \lambda e^{-\lambda y}$ for $y \geq 0$: the exponential distribution. This is the inverse-transform sampling trick.

6. **KL direction.** For $p = (0.5, 0.5, 0)$ and $q = (0.4, 0.4, 0.2)$, compute $D_{\mathrm{KL}}(p\|q)$ and $D_{\mathrm{KL}}(q\|p)$. Which one is infinite, and what does that tell you about training a model by maximum likelihood?

Hint

$D_{\mathrm{KL}}(p\|q) = \log(0.5/0.4) \approx 0.223$ nats (the third term is $0 \log 0 = 0$). $D_{\mathrm{KL}}(q\|p) = \infty$ because q puts mass where p has none. Maximum likelihood minimizes $D_{\mathrm{KL}}(p_{\mathrm{data}}\|q_{\mathrm{model}})$, which becomes infinite if the model assigns zero probability to any observed example — so likelihood-trained models must spread some probability everywhere the data might be.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 3 "Probability and Information Theory". Book
2. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
3. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. Book
4. M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Book
5. C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948. DOI
6. S. Kullback and R. A. Leibler, "On Information and Sufficiency," *The Annals of Mathematical Statistics*, vol. 22, pp. 79–86, 1951. DOI
7. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006. DOI
8. D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *Proc. International Conference on Learning Representations (ICLR)*, 2014. arXiv
9. G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed and B. Lakshminarayanan, "Normalizing Flows for Probabilistic Modeling and Inference," *Journal of Machine Learning Research*, vol. 22, no. 57, pp. 1–64, 2021. arXiv
10. D. M. Blei, A. Kucukelbir and J. D. McAuliffe, "Variational Inference: A Review for Statisticians," *Journal of the American Statistical Association*, vol. 112, no. 518, pp. 859–877, 2017. DOI
11. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On Calibration of Modern Neural Networks," *ICML*, 2017. arXiv
12. C. Wang, "Calibration in Deep Learning: A Survey of the State-of-the-Art," arXiv:2308.01222, 2023 (revised 2025). arXiv
13. M. Abdar et al., "A Review of Uncertainty Quantification in Deep Learning: Techniques, Applications and Challenges," *Information Fusion*, vol. 76, pp. 243–297, 2021. DOI
14. J. Gawlikowski et al., "A Survey of Uncertainty in Deep Neural Networks," *Artificial Intelligence Review*, vol. 56, pp. 1513–1589, 2023. DOI
15. B. Lakshminarayanan, A. Pritzel and C. Blundell, "Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles," *NeurIPS (NIPS)*, 2017. arXiv
16. Y. Ovadia et al., "Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift," *NeurIPS*, 2019. arXiv

17. A. N. Angelopoulos and S. Bates, “A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification,” arXiv:2107.07511, 2021. arXiv
18. A. van den Oord, Y. Li and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” arXiv:1807.03748, 2018. arXiv
19. B. Poole, S. Ozair, A. van den Oord, A. A. Alemi and G. Tucker, “On Variational Bounds of Mutual Information,” *ICML*, 2019. arXiv
20. M. Tschannen, J. Djolonga, P. K. Rubenstein, S. Gelly and M. Lucic, “On Mutual Information Maximization for Representation Learning,” *ICLR*, 2020. arXiv

ML 01C • Numerical Computation and Gradient Checks

Floating point, overflow and underflow, stable softmax, conditioning, gradients, Jacobians and Hessians, Newton-type methods, KKT conditions and mixed precision: make the mathematical model survive actual computation.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-01c-numerical-computation/

What you'll learn

- How computers store real numbers, what machine epsilon means, and why float16, bfloat16, float32 and float64 behave differently.
- Why exact formulas overflow, underflow or lose digits, and how to compute softmax, log-sum-exp and logistic losses stably.
- What a condition number measures, and how ill-conditioning differs from an unstable algorithm.
- How gradients, Jacobians and Hessians describe a loss surface, why gradient descent zigzags, and what Newton and quasi-Newton methods change.
- How to handle constraints with projections, the generalized Lagrangian and the KKT conditions.
- How to check derivatives, debug non-finite values and use mixed precision safely.

The big picture

Machine learning is written in the language of real numbers, but computers do not have real numbers. They have a finite set of floating-point numbers, and every arithmetic operation rounds its result to one of them. Most of the time this rounding is invisible. Sometimes it is catastrophic: an exponential overflows to infinity, a tiny probability becomes exactly zero and its logarithm becomes minus infinity, or two nearly equal numbers are subtracted and almost all their significant digits disappear.

The second half of the story is optimization. Training a model means minimizing a function of millions or billions of parameters, and we can only do that numerically, by taking many small steps guided by derivatives. How fast and how reliably those steps make progress depends on the shape of the loss surface — its slopes and its curvatures — and on how we take the steps. This chapter follows the "Numerical Computation" chapter of Goodfellow, Bengio and Courville [1], adds the floating-point details that modern low-precision training relies on, and connects each idea to small NumPy experiments.

Explain like I'm five

> Imagine a calculator that can show only seven digits. If you ask it for one million plus 0.1, it shows one million: the 0.1 fell off the screen. If you then subtract one million, you get zero instead of 0.1. Good numerical code arranges its steps so the important digits never fall off the screen. Optimization is a different game: you are blindfolded on a hilly field and want to reach the lowest point. You can feel the slope under your feet (the gradient) and, with more effort, how the ground curves (the Hessian). Numerical computation is about walking down safely without your calculator lying to you.

Read ML 01B first if softmax and cross-entropy are new, and keep ML 01A nearby for eigenvalues and the singular value decomposition. ML 02 then uses everything here to train neural networks.

Floating-point numbers

Plain version. A floating-point number stores a sign, a few significant digits and a power of two. The *relative* spacing between neighbouring numbers is roughly constant, so a big number has a big absolute rounding error and a small number has a small one.

How a float is stored

The IEEE 754 standard [3] defines binary floating-point formats used by essentially all CPUs and GPUs. A normal (non-special) number is stored as

$$x = (-1)^s \times \left(1 + \sum_{i=1}^{t} b_i\, 2^{-i}\right) \times 2^{e},$$

where $s \in \{0,1\}$ is the sign bit, $b_1, \dots, b_t \in \{0,1\}$ are the t stored fraction bits of the significand (the leading 1 is implicit), and e is an integer exponent restricted to a fixed range. Two numbers matter most:

- **Machine epsilon** $\varepsilon_{\text{mach}} = 2^{-t}$ is the gap between 1 and the next larger representable number.
- **Unit roundoff** $u = \varepsilon_{\text{mach}}/2$ bounds the relative error of rounding to nearest: for any real x in the normal range, the stored value is $\mathrm{fl}(x) = x(1+\delta)$ with $|\delta| \le u$.

The standard also requires that each basic operation $(+, -, \times, \div, \sqrt{\cdot})$ is computed as if exactly and then rounded once: $\mathrm{fl}(a \circ b) = (a \circ b)(1+\delta)$, $|\delta| \le u$. This one rule is the foundation of rounding-error analysis [2], [4].

Below the smallest normal number the format uses **subnormal** numbers, which give up leading precision to fill the gap down to zero gradually. Special values complete the picture: $\pm\infty$ for results that are too large, and **NaN** ("not a number") for invalid operations such as $0/0$, $\infty - \infty$ or $\infty \times 0$. Once a NaN appears, almost every operation that touches it returns NaN, which is why a single bad value can poison a whole training run.

The four formats you will meet

The table below was computed with `numpy.finfo` (and `torch.finfo` for bfloat16, which NumPy lacks). "Bits" lists sign/exponent/fraction.

Format	Bits	$\varepsilon_{\text{mach}}$	Largest finite	Smallest normal	Smallest subnormal
float16 (IEEE half)	1/5/10	9.77×10^{-4}	65504	6.10×10^{-5}	5.96×10^{-8}
bfloat16	1/8/7	7.81×10^{-3}	3.39×10^{38}	1.18×10^{-38}	9.18×10^{-41}
float32 (single)	1/8/23	1.19×10^{-7}	3.40×10^{38}	1.18×10^{-38}	1.40×10^{-45}
float64 (double)	1/11/52	2.22×10^{-16}	1.80×10^{308}	2.23×10^{-308}	4.94×10^{-324}

Read the table in two directions. The **exponent bits** set the *range*: float16 overflows above 65504,

while bfloat16 keeps float32's 8 exponent bits and therefore its range. The **fraction bits** set the *precision*: float32 carries about 7 significant decimal digits, float64 about 16, float16 about 3, and bfloat16 only about 2. That trade-off — bfloat16's wide range with coarse precision versus float16's narrow range with finer precision — is the reason the two half-precision formats need different training recipes (see the mixed-precision section below and [5], [12]).

```
import numpy as np

for dtype in (np.float16, np.float32, np.float64):
    info = np.finfo(dtype)
    print(f"{info.dtype}: eps={float(info.eps):.3e}  max={float(info.max):.3e}  "
          f"tiny={float(info.tiny):.3e}  subnormal={float(info.smallest_subnormal):.3e}")

# Rounding: 1 + eps/2 rounds back to 1 in float32
e = np.finfo(np.float32).eps
print(np.float32(1) + np.float32(e/2) == np.float32(1))   # True
print(np.float32(1) + np.float32(e) == np.float32(1))     # False

# Addition is not associative
a, b, c = np.float32(1e8), np.float32(-1e8), np.float32(1.0)
print((a + b) + c, a + (b + c))                           # 1.0 0.0
```

The last line is worth pausing on. In exact arithmetic both orders give 1. In float32, $-10^8 + 1$ rounds back to -10^8 because the spacing between float32 numbers near 10^8 is 8, so the 1 is lost. Consequences in practice: summing the same numbers in a different order — for example on a different number of GPUs, or with a different reduction kernel — can change the result in the last digits. This is a major source of run-to-run nondeterminism, and it is not a bug in your code [13].

Overflow, underflow and cancellation

Plain version. Overflow means a result is too large to store; underflow means it is too small and becomes zero (or loses precision as a subnormal). Cancellation means subtracting nearly equal numbers, which keeps the rounding error but throws away the true signal.

Overflow and underflow

The exponential is the usual culprit in machine learning. Because $\exp(z)$ exceeds the largest float32 once $z > \ln(3.40 \times 10^{38}) \approx 88.7$, a logit of 100 overflows in float32 (and a logit of about 11.1 already overflows in float16, since $\ln 65504 \approx 11.09$). In the other direction, $\exp(-110) \approx 1.7 \times 10^{-48}$ is below the smallest float32 subnormal and becomes exactly 0. Neither event raises an error by default; they produce `inf` and `0`, and the next operation turns them into `nan` or $-\infty$.

```
import numpy as np

with np.errstate(over="ignore", under="ignore", invalid="ignore"):
    print(np.exp(np.float32(100.0)))        # inf: overflow
    print(np.exp(np.float32(-110.0)))       # 0.0: underflow
    big = np.exp(np.float32(100.0))
    print(big / big)                        # nan: inf / inf

# Catastrophic cancellation and its cure
x = 1e-10
print(np.log(1 + x), np.log1p(x))           # roundoff vs accurate
print(np.exp(x) - 1, np.expm1(x))

# Variance: the one-pass formula cancels badly when the mean is large
rng = np.random.default_rng(0)
data = (1e4 + rng.normal(0, 1, 100_000)).astype(np.float32)
one_pass = np.mean(data**2) - np.mean(data)**2
two_pass = np.mean((data - data.mean())**2)
print("one-pass:", one_pass, " two-pass:", two_pass)
```

On this machine the one-pass variance comes out as -8.0 — a negative variance — while the two-pass formula gives the correct value of about 1.0. The data have mean 10^4 and variance 1, so $\mathbb{E}[X^2] \approx 10^8$ and $(\mathbb{E}X)^2 \approx 10^8$; float32 can only resolve differences of about 8 at that magnitude, so the true difference of 1 is pure noise.

Why subtraction is the dangerous operation

Suppose $\hat{a} = a(1 + \delta_a)$ and $\hat{b} = b(1 + \delta_b)$ are stored with relative errors $|\delta_a|, |\delta_b| \le u$. Even if the subtraction itself is exact,

$$\frac{|(\hat{a} - \hat{b}) - (a - b)|}{|a - b|} = \frac{|a\delta_a - b\delta_b|}{|a - b|} \le u \, \frac{|a| + |b|}{|a - b|}.$$

Here a, b are the true values and u is the unit roundoff. The factor $(|a| + |b|)/|a - b|$ is huge when $a \approx b$: the subtraction does not create error, it *reveals* the error already present in the inputs by removing the digits they shared. The cures are algebraic: rewrite the formula so the subtraction never happens. Examples you will use:

- `log1p(x)` computes $\log(1 + x)$ and `expm1(x)` computes $e^x - 1$ accurately for small x, where the naive forms lose almost all digits (see the output above).
- Variance: subtract the mean first (two-pass), or use Welford's streaming update.
- Softmax and log-sum-exp: subtract the maximum first, as the next section shows.
- Distances: $\|x - y\|^2 = \|x\|^2 + \|y\|^2 - 2x^\top y$ is fast but cancels badly when $x \approx y$; clamp it at zero or compute $x - y$ directly when accuracy matters.

Stable softmax and log-sum-exp

Plain version. Subtract a shared constant before exponentiating, and compute logarithms of probabilities directly instead of first creating tiny probabilities and then taking their logarithm.

The softmax turns a vector of logits $z \in \mathbb{R}^K$ into probabilities:

$$\operatorname{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}, \qquad i = 1, \ldots, K.$$

It is invariant to adding the same constant c to every logit, because the factor $\exp(c)$ cancels between numerator and denominator. Choose $c = -m$ with $m = \max_i z_i$:

$$p_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}.$$

Now every exponent is ≤ 0, so nothing overflows, and the largest term is exactly $\exp(0) = 1$, so the denominator is at least 1 and cannot underflow to zero. The same trick stabilizes the **log-sum-exp** function,

$$\operatorname{LSE}(z) = \log \sum_j \exp(z_j) = m + \log \sum_j \exp(z_j - m), \qquad \log p_i = z_i - \operatorname{LSE}(z).$$

LSE is a smooth version of the maximum: $\max_i z_i \le \operatorname{LSE}(z) \le \max_i z_i + \log K$. Its gradient is exactly the softmax, $\partial \operatorname{LSE} / \partial z_i = p_i$.

For a target class y, the cross-entropy loss is $-\log p_y = \operatorname{LSE}(z) - z_y$, and its gradient with respect to the logits is the famously simple $p - e_y$, where e_y is the one-hot vector of the target. Computing `softmax` first and then `log` can still fail: if p_y underflows to 0, its logarithm is $-\infty$ even though $\operatorname{LSE}(z) - z_y$ is a perfectly ordinary finite number. This is why frameworks provide fused `log_softmax` and `cross_entropy` functions that take logits.

Code: logits that overflow naive exponentiation

This standalone NumPy example checks stable probabilities, log probabilities and invariance to shifting logits.

```
import numpy as np

def logsumexp(z):
    m = np.max(z)
    return m + np.log(np.exp(z-m).sum())
def logsoftmax(z):
    return z-logsumexp(z)

z = np.array([1000., 1001., 999.], dtype=np.float64)
logp = logsoftmax(z)
p = np.exp(logp)
assert np.allclose(p.sum(), 1)
assert np.allclose(logsoftmax(z+12345), logp)
print("probabilities:", p)
print("cross-entropy for class 1:", -logp[1])
with np.errstate(over="ignore", invalid="ignore"):
    naive = np.exp(z) / np.exp(z).sum()
print("naive softmax:", naive)
```

The stable version prints probabilities $[0.245, 0.665, 0.090]$ and a cross-entropy of 0.408 nats for class 1; the naive version prints `[nan nan nan]`, because $\exp(1000)$ overflows even in float64 (whose limit is $\exp(709.78)$).

Stable logistic (binary cross-entropy) loss

Binary classification has the same issue in a different form. With logit z, label $y \in \{0, 1\}$ and sigmoid $\sigma(z) = 1/(1 + e^{-z})$, the loss $-y \log \sigma(z) - (1 - y) \log(1 - \sigma(z))$ simplifies algebraically to

$$\ell(z, y) = \operatorname{softplus}(z) - yz, \qquad \operatorname{softplus}(z) = \log(1 + e^{z}) = \max(z, 0) + \log\bigl(1 + e^{-|z|}\bigr).$$

The right-hand form never exponentiates a positive number, and `log1p` keeps the small correction accurate. Prefer `binary_cross_entropy_with_logits` to a hand-built sigmoid followed by a logarithm.

```
import numpy as np

def bce_naive(z, y):
    s = 1 / (1 + np.exp(-z))
    return -(y*np.log(s) + (1-y)*np.log(1-s))

def bce_stable(z, y):
    # softplus(z) - y*z, with softplus(z) = max(z,0) + log1p(exp(-|z|))
    return np.maximum(z, 0) + np.log1p(np.exp(-np.abs(z))) - y*z
```

```
z = np.array([-800., -30., 0., 30., 800.])
y = np.array([1., 1., 0., 0., 0.])
with np.errstate(over="ignore", divide="ignore", invalid="ignore"):
    print("naive :", bce_naive(z, y))
print("stable:", bce_stable(z, y))
```

The naive version returns `inf` for the two confident mistakes ($z = \pm 800$) and 30.001 instead of 30.000 at $z = 30$, because $1 - \sigma(30)$ is computed by cancellation. The stable version returns 800, 30, $\log 2$, 30 and 800 — the correct values.

Poor conditioning

Plain version. Some problems are inherently sensitive: a tiny change in the input produces a large change in the answer, no matter how carefully you compute. The **condition number** measures that sensitivity.

Condition number of a linear system

Consider solving $Ax = b$ for a square invertible matrix A. If the data change to $b + \delta b$, the solution changes to $x + \delta x$ with

$$\frac{\|\delta x\|}{\|x\|} \le \kappa(A)\,\frac{\|\delta b\|}{\|b\|}, \qquad \kappa(A) = \|A\|\,\|A^{-1}\| = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} \ \text{(2-norm)}.$$

Here $\|\cdot\|$ is the Euclidean vector norm and its induced matrix norm, and $\sigma_{\max}, \sigma_{\min}$ are the largest and smallest singular values of A. For a symmetric positive-definite matrix, singular values equal eigenvalues and $\kappa = \lambda_{\max}/\lambda_{\min}$. Goodfellow et al. state the same idea for the function $f(x) = A^{-1}x$ with an eigendecomposition: its condition number is $\max_{i,j} |\lambda_i/\lambda_j|$ [1]. A practical rule of thumb: with $\kappa \approx 10^k$, expect to lose about k decimal digits of accuracy relative to the precision of your inputs [4].

```
import numpy as np

A = np.array([[1.0, 1.0],
              [1.0, 1.0001]])
print("condition number:", np.linalg.cond(A))

b = np.array([2.0, 2.0001])
x = np.linalg.solve(A, b)
db = np.array([0.0, 0.0001])                # tiny change in the data
x2 = np.linalg.solve(A, b + db)
rel_in = np.linalg.norm(db) / np.linalg.norm(b)
rel_out = np.linalg.norm(x2 - x) / np.linalg.norm(x)
```

```
print("x:", x, " x after perturbation:", x2)
print(f"relative change in b: {rel_in:.1e}, in x: {rel_out:.1e}, "
      f"amplification: {rel_out/rel_in:.0f}")
```

The two columns of A are almost identical, $\kappa(A) \approx 4 \times 10^4$, and a relative change of 3.5×10^{-5} in b moves the solution from $(1, 1)$ to $(0, 2)$ — a 100% change, an amplification of about 28,000, within the bound κ. Nothing was computed wrongly. The *problem* is ill-conditioned.

Ill-conditioned problem versus unstable algorithm

Keep two ideas apart:

- An **ill-conditioned problem** amplifies small changes in its input. No algorithm can fix that; you need better data, regularization (which changes the problem) or a reformulation.
- An **unstable algorithm** adds avoidable error to a problem that may be perfectly well conditioned. The naive softmax and the one-pass variance are unstable algorithms; changing the algorithm fixes them.

Least squares shows both. If features are nearly collinear, the coefficients are ill-conditioned (the problem). Solving the normal equations $X^\top X w = X^\top y$ makes things worse than necessary because $\kappa(X^\top X) = \kappa(X)^2$ (the algorithm); a QR- or SVD-based solver works with X directly. Ridge regression, which adds $\lambda \|w\|^2$, changes the problem to $(X^\top X + \lambda I)w = X^\top y$, whose condition number $(\sigma_{\max}^2 + \lambda)/(\sigma_{\min}^2 + \lambda)$ is smaller — a theme ML 01A develops further.

Gradient-based optimization

Plain version. To minimize a function, look at its slope and step downhill. The gradient points uphill most steeply, so step in the opposite direction, with a step size small enough that the slope you measured is still roughly valid.

Derivatives, gradients and directional derivatives

We minimize an **objective** (also called cost, loss or error) function $f : \mathbb{R}^n \to \mathbb{R}$. For one variable, the derivative $f'(x)$ gives the first-order approximation $f(x+\epsilon) \approx f(x) + \epsilon f'(x)$, so moving a small step against the sign of $f'(x)$ decreases f. For many variables, the **gradient** $\nabla_x f(x) \in \mathbb{R}^n$ collects the partial derivatives $\partial f / \partial x_i$. The **directional derivative** in a unit direction u is the slope along u:

$$\frac{\partial}{\partial \alpha} f(x + \alpha u)\Big|_{\alpha=0} = u^\top \nabla_x f(x) = \|\nabla_x f(x)\|_2 \cos\theta,$$

where θ is the angle between u and the gradient. It is most negative when $\theta = \pi$, that is, when u points along $-\nabla_x f(x)$. This is the **method of steepest descent**, or **gradient descent**:

$$x \leftarrow x - \eta \nabla_x f(x),$$

with **learning rate** (step size) $\eta > 0$. You can pick η as a small constant, try several values and keep the one with the lowest objective (a **line search**), or follow a schedule. Gradient descent converges when every element of the gradient is (close to) zero.

Critical points: minima, maxima and saddles

Points where $\nabla_x f(x) = 0$ are **critical** or **stationary points**. A **local minimum** is lower than all neighbouring points, a **local maximum** is higher, and a **saddle point** is neither: it is a minimum along some directions and a maximum along others. A **global minimum** has the lowest value anywhere; a function can have several global minima and many local minima that are not global. For deep networks we rarely find a true global minimum, and we do not need to. We settle for parameters with a low enough loss that generalize well. In high dimensions, saddle points rather than poor local minima are the more common critical points of many random-like functions, and they can slow gradient-based optimizers because the gradient becomes small near them [8]. Figure 1 shows the three kinds of critical point.

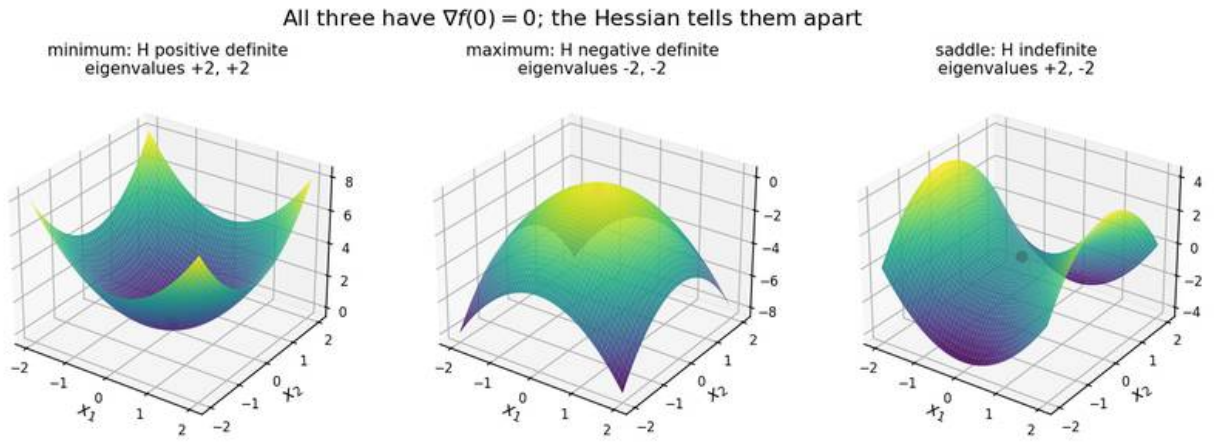


Figure 1 — Three functions with zero gradient at the origin. The Hessian's eigenvalues distinguish them: all positive (minimum), all negative (maximum), mixed signs (saddle).

Jacobian and Hessian matrices

Plain version. The Jacobian collects all first derivatives of a vector-valued function. The Hessian collects all second derivatives of a scalar function: it tells you how the slope itself changes, which is curvature.

Definitions

For a function $f : \mathbb{R}^m \to \mathbb{R}^n$, the **Jacobian** $J \in \mathbb{R}^{n \times m}$ has entries

$$J_{ij} = \frac{\partial f_i(x)}{\partial x_j}, \qquad i = 1, \dots, n,\ j = 1, \dots, m.$$

Row i is the gradient of output f_i. Backpropagation is repeated multiplication by Jacobians; reverse-mode automatic differentiation computes vector–Jacobian products $v^\top J$ without ever forming J, which is why a gradient costs only a small constant multiple of one function evaluation [11].

For a scalar function $f : \mathbb{R}^n \to \mathbb{R}$, the **Hessian** $H(f)(x) \in \mathbb{R}^{n \times n}$ is the Jacobian of the gradient:

$$H(f)(x)_{ij} = \frac{\partial^2 f(x)}{\partial x_i \, \partial x_j}.$$

Wherever the second partial derivatives are continuous, the order of differentiation does not matter, so $H_{ij} = H_{ji}$: the Hessian is symmetric. A real symmetric matrix has real eigenvalues $\lambda_1, \dots, \lambda_n$ and an orthonormal basis of eigenvectors $v_1, \dots, v_n$. The **second derivative in a unit direction** d is $d^\top H d$. If d is an eigenvector v_i, it equals λ_i; for other directions it is a weighted average $\sum_i \lambda_i (d^\top v_i)^2$ of the eigenvalues, with weights between 0 and 1 that sum to 1. So $\lambda_{\max}$ is the largest curvature in any direction and $\lambda_{\min}$ the smallest.

What a gradient step really does: second-order Taylor expansion

Write $g = \nabla_x f(x^{(0)})$ and $H = H(f)(x^{(0)})$. The second-order Taylor approximation around $x^{(0)}$ is

$$f(x) \approx f(x^{(0)}) + (x - x^{(0)})^\top g + \tfrac{1}{2}(x - x^{(0)})^\top H (x - x^{(0)}).$$

Substituting a gradient step $x = x^{(0)} - \eta g$ gives

$$f(x^{(0)} - \eta g) \approx f(x^{(0)}) - \eta\, g^\top g + \tfrac{1}{2}\eta^2\, g^\top H g.$$

The three terms are the current value, the improvement predicted by the slope, and a correction for curvature. If $g^\top H g \le 0$, the approximation keeps decreasing as η grows (until the approximation itself stops being valid). If $g^\top H g > 0$, the curvature term eventually wins, and the approximation is minimized at

$$\eta^* = \frac{g^\top g}{g^\top H g}.$$

In the worst case, g is aligned with the eigenvector of $\lambda_{\max}$ and $\eta^* = 1/\lambda_{\max}$. Large curvature forces small steps.

The second derivative test

At a critical point ($g = 0$) the Taylor expansion reduces to the quadratic term, so the Hessian decides what kind of point it is:

- H **positive definite** (all $\lambda_i > 0$): local minimum.
- H **negative definite** (all $\lambda_i < 0$): local maximum.
- H has at least one positive and at least one negative eigenvalue: **saddle point**.
- Otherwise (some $\lambda_i = 0$ and the rest of one sign) the test is inconclusive.

In one dimension a saddle cannot exist; in n dimensions it needs only one eigenvalue of the "wrong" sign, which is why saddles become so common as n grows [8].

Condition number of the Hessian: why gradient descent zigzags

On the quadratic $f(w) = \frac{1}{2} w^\top H w$ with H symmetric positive definite, gradient descent is linear: $w \leftarrow (I - \eta H) w$. Expanding w in the eigenvectors of H, the component along v_i is multiplied by $(1 - \eta\lambda_i)$ at every step. Therefore:

- The iteration is **stable** only if $|1 - \eta\lambda_i| < 1$ for every i, that is, $0 < \eta < 2/\lambda_{\max}$.
- The **slowest** direction is the flat one, contracting by $|1 - \eta\lambda_{\min}|$ per step.
- The best fixed step, $\eta = 2/(\lambda_{\max} + \lambda_{\min})$, contracts the error by $(\kappa - 1)/(\kappa + 1)$ per step, with $\kappa = \lambda_{\max}/\lambda_{\min}$ the condition number of the Hessian.

So a large condition number means one direction wants tiny steps (or it diverges) while another direction needs many steps to make progress. Gradient descent wastes its time bouncing across the steep walls of a narrow valley — the zigzag in Figure 2 — while creeping along the valley floor. Badly scaled input features produce exactly this situation, which is one reason standardization and normalization layers speed up training.

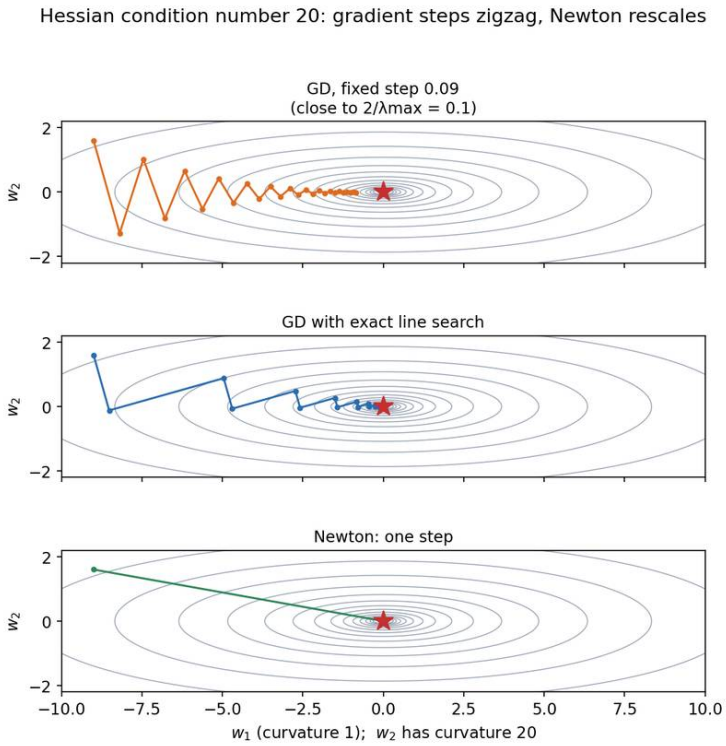


Figure 2 — Minimizing a quadratic with Hessian condition number 20. Gradient descent with a step close to the stability limit oscillates across the steep direction; exact line search makes successive steps orthogonal; Newton's method rescales by the inverse Hessian and reaches the minimum of a quadratic in one step.

Figure 3 shows the stability limit numerically. Below $2/\lambda_{\max}$ the loss decreases; just above it the steep component grows by a factor $|1 - \eta\lambda_{\max}| > 1$ every step and the loss explodes.

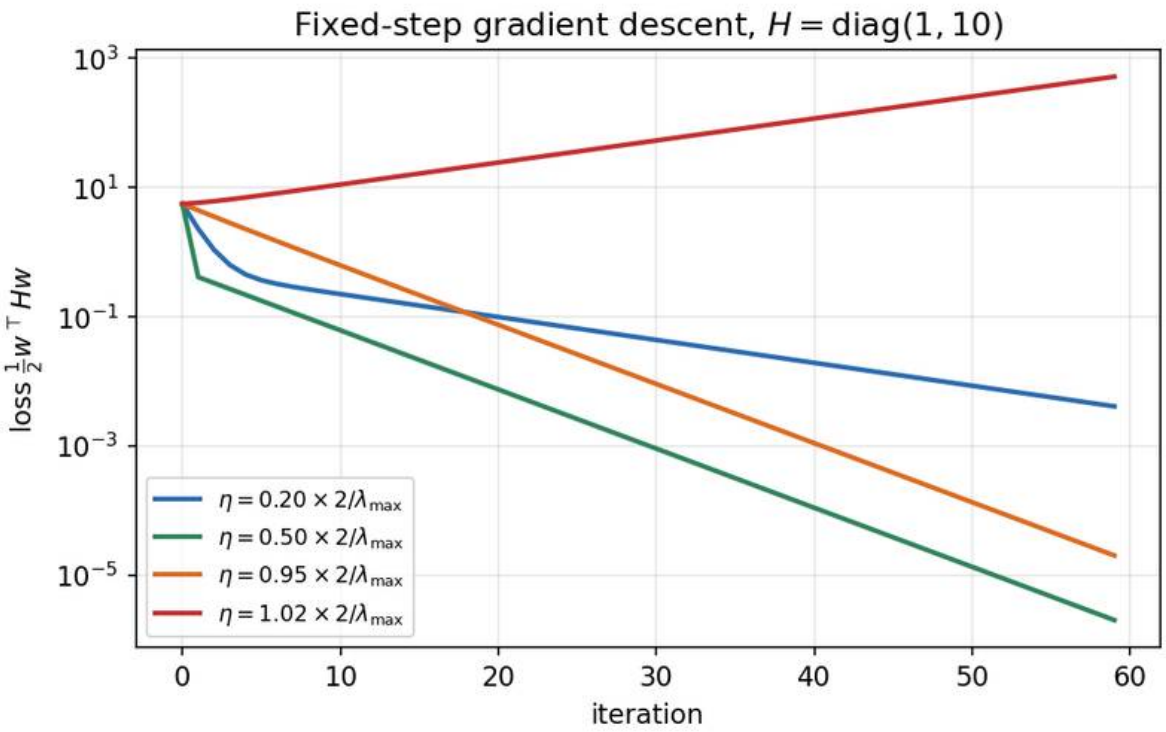


Figure 3 — Fixed-step gradient descent on a quadratic with eigenvalues 1 and 10. Step sizes below 2/λmax converge (fastest near 2/(λmax+λmin)); a step just 2% above the limit diverges.

```
import numpy as np

H = np.diag([1.0, 25.0])                    # Hessian: condition number 25
f = lambda w: 0.5 * w @ H @ w
grad = lambda w: H @ w

def run_gd(eta, steps=50):
    w = np.array([10.0, 1.0])
    for _ in range(steps):
        w = w - eta * grad(w)
    return f(w)

lam_max = np.linalg.eigvalsh(H).max()
for eta in (0.01, 0.07, 0.079, 0.081):       # stability limit 2/25 = 0.08
    print(f"eta={eta:<6} loss after 50 steps: {run_gd(eta):.3e}")

# Exact line search along -g for a quadratic: eps* = g'g / g'Hg
w = np.array([10.0, 1.0])
g = grad(w)
eps_star = (g @ g) / (g @ H @ g)
print("optimal step along the gradient:", round(eps_star, 4))

# One Newton step lands on the minimizer of a quadratic
w_newton = w - np.linalg.solve(H, g)
```

```
print("after one Newton step:", w_newton)
```

The output shows the whole story: $\eta = 0.01$ is stable but slow (loss 18.3 after 50 steps), $\eta = 0.07$ reaches 0.035, $\eta = 0.079$ is barely stable and the steep component still oscillates (loss about 1.0), and $\eta = 0.081$ diverges (loss 148). The optimal step along the first gradient is 0.046, and one Newton step lands exactly on $(0, 0)$.

Second-order methods

Plain version. Instead of following the slope alone, use the curvature as well: divide the step by how sharply the function curves in each direction, so steep directions get small steps and flat directions get large ones.

Newton's method

Setting the gradient of the second-order Taylor approximation to zero gives the **Newton step**

$$x^* = x^{(0)} - H(f)(x^{(0)})^{-1} \nabla_x f(x^{(0)}).$$

On a positive-definite quadratic this jumps to the minimum in one step (Figure 2, bottom). On a general function that is locally close to a positive-definite quadratic, iterating the update converges much faster than gradient descent — quadratically near the solution, under standard assumptions [6]. The cost is the Hessian: storing it takes $O(n^2)$ memory and solving with it $O(n^3)$ time, impossible for a network with 10^8 parameters.

There is also a more fundamental problem for non-convex objectives. Newton's method finds a point where the *gradient* vanishes, and it treats a saddle the same way as a minimum: near a saddle, the Newton step points *toward* it. In a landscape full of saddles, plain Newton is attracted to exactly the points we want to escape. Dauphin et al. proposed a "saddle-free Newton" method that rescales by the absolute values of the Hessian eigenvalues to avoid this [8]. Gradient descent, by contrast, is not attracted to saddles in the same way, although it can move slowly near them.

Quasi-Newton methods and L-BFGS

Quasi-Newton methods build an approximation $B_k \approx H$ (or of its inverse) from gradients alone. After a step $s_k = x_{k+1} - x_k$ with gradient change $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$, they require the **secant condition** $B_{k+1} s_k = y_k$, which says the model's curvature along the step matches the observed change in slope. BFGS is the most widely used update of this kind; **L-BFGS** (limited-memory BFGS) stores only the last m pairs (s_k, y_k) and applies the inverse-Hessian approximation implicitly in $O(mn)$ time [6], [10]. L-BFGS is excellent for smooth, deterministic, medium-sized

problems (logistic regression, style transfer on one image, fitting a few thousand parameters); its curvature estimates suffer when gradients are noisy mini-batch estimates.

Hessian-vector products

Many curvature-aware methods never need H itself, only products Hv. These can be computed exactly at roughly the cost of a couple of gradient evaluations, because $Hv = \nabla_x(\nabla_x f(x)^\top v)$ is the gradient of a scalar; Pearlmutter showed how to do this with automatic differentiation [9]. A finite-difference version, $Hv \approx [\nabla f(x + rv) - \nabla f(x - rv)]/(2r)$, is a useful check. Power iteration with Hessian-vector products estimates $\lambda_{\max}$, which is how researchers track the sharpness of the loss during training.

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(50, 4))
y = (rng.random(50) < 0.5).astype(float)

def grad(w):                                    # logistic regression, mean loss
    p = 1 / (1 + np.exp(-(X @ w)))
    return X.T @ (p - y) / len(y)

def hessian(w):
    p = 1 / (1 + np.exp(-(X @ w)))
    return (X.T * (p * (1 - p))) @ X / len(y)

w = rng.normal(size=4)
v = rng.normal(size=4)
r = 1e-5
hvp_fd = (grad(w + r*v) - grad(w - r*v)) / (2*r)   # never forms H
print("Hv (finite diff):", hvp_fd.round(6))
print("Hv (explicit)   :", (hessian(w) @ v).round(6))
print("eigenvalues of H:", np.linalg.eigvalsh(hessian(w)).round(4))
```

Both products agree to six decimals, and all Hessian eigenvalues are positive: logistic regression is convex, so its Hessian is positive semidefinite everywhere.

Lipschitz continuity and convexity

Deep-learning objectives come with few guarantees, but two properties explain much of the theory. A function is **Lipschitz continuous** with constant L if $|f(x) - f(y)| \le L\|x - y\|_2$ for all x, y: its rate of change is bounded. When the *gradient* is L-Lipschitz (the function is "L-smooth"), the curvature is bounded by L, and a gradient step with $\eta \le 1/L$ is guaranteed not to increase the function:

$$f(x - \eta\nabla f(x)) \le f(x) - \tfrac{\eta}{2}\|\nabla f(x)\|_2^2 \qquad (0 < \eta \le 1/L).$$

This is the general form of the "$\eta < 2/\lambda_{\max}$" rule above, and it is the starting point of most convergence proofs [7], [14].

A function is **convex** if its Hessian is positive semidefinite everywhere (for twice-differentiable functions). Convex functions have no saddles and no spurious local minima: every local minimum is global. Least squares, logistic regression and support vector machines are convex; deep networks are not. Convex optimization is the source of many tools, such as duality and the KKT conditions below, that deep learning borrows [7].

Constrained optimization

Plain version. Sometimes the answer must lie in an allowed region — weights with bounded norm, probabilities that sum to one, a physical quantity that must be positive. Either step and then project back into the region, or add a penalty for each constraint whose strength is chosen automatically.

Projected gradient descent

We want to minimize $f(x)$ over a feasible set $\mathbb{S} \subseteq \mathbb{R}^n$. When projecting onto $\mathbb{S}$ is easy, take a normal gradient step and then return to the nearest feasible point:

$$x \leftarrow \text{Proj}_{\mathbb{S}}\left(x - \eta \nabla f(x)\right), \qquad \text{Proj}_{\mathbb{S}}(v) = \arg\min_{x \in \mathbb{S}} \|x - v\|_2.$$

For the Euclidean ball $\{x : \|x\|_2 \leq r\}$ the projection is $v \cdot \min(1, r/\|v\|_2)$: rescale if too long. Max-norm constraints on weights, clipping parameters into a box, and projecting onto the probability simplex all follow this pattern. Another option is to reparameterize so the constraint holds automatically, for example $x = r\,(\cos\theta, \sin\theta)$ for a vector on a circle, or a softmax to produce probabilities.

The generalized Lagrangian

A more general approach describes the feasible set with m equality constraints $g^{(i)}(x) = 0$ and k inequality constraints $h^{(j)}(x) \leq 0$:

$$\mathbb{S} = \{x \mid g^{(i)}(x) = 0 \ \forall i,\ h^{(j)}(x) \leq 0 \ \forall j\}.$$

Introduce one **KKT multiplier** for each constraint, λ_i for equalities and α_j for inequalities, and define the **generalized Lagrangian**

$$L(x, \lambda, \alpha) = f(x) + \sum_{i=1}^{m} \lambda_i g^{(i)}(x) + \sum_{j=1}^{k} \alpha_j h^{(j)}(x).$$

The constrained problem is equivalent to the unconstrained min–max problem

$$\min_{x} \max_{\lambda} \max_{\alpha,\, \alpha \geq 0} L(x, \lambda, \alpha).$$

Why does this work? If x violates a constraint, the inner maximization can drive L to $+\infty$ by choosing a large multiplier, so infeasible points are never chosen. If x is feasible, the best the inner maximization can do is set the terms to zero (equalities are already 0; for $h^{(j)}(x) \leq 0$ the best non-negative α_j makes $\alpha_j h^{(j)}(x) = 0$), so the value is exactly $f(x)$ [1].

The KKT conditions

At a solution x^* (under mild regularity conditions called constraint qualifications), there exist multipliers satisfying the **Karush–Kuhn–Tucker (KKT) conditions** [1], [7]:

1. **Stationarity:** $\nabla_x L(x^*, \lambda^*, \alpha^*) = 0$.
2. **Primal feasibility:** all constraints hold at x^*.
3. **Dual feasibility:** $\alpha^* \geq 0$.
4. **Complementary slackness:** $\alpha_j^* \, h^{(j)}(x^*) = 0$ for every j.

Complementary slackness says each inequality is either **active** ($h^{(j)}(x^*) = 0$: the solution sits on the boundary and the constraint pushes back with $\alpha_j \geq 0$) or **inactive** ($h^{(j)}(x^*) < 0$: the constraint does not matter locally and $\alpha_j = 0$). Geometrically, stationarity says that at an active constraint the downhill direction $-\nabla f$ points straight out of the feasible set, parallel to ∇h; any feasible move would go uphill (Figure 4). For convex problems the KKT conditions are also sufficient for optimality [7].

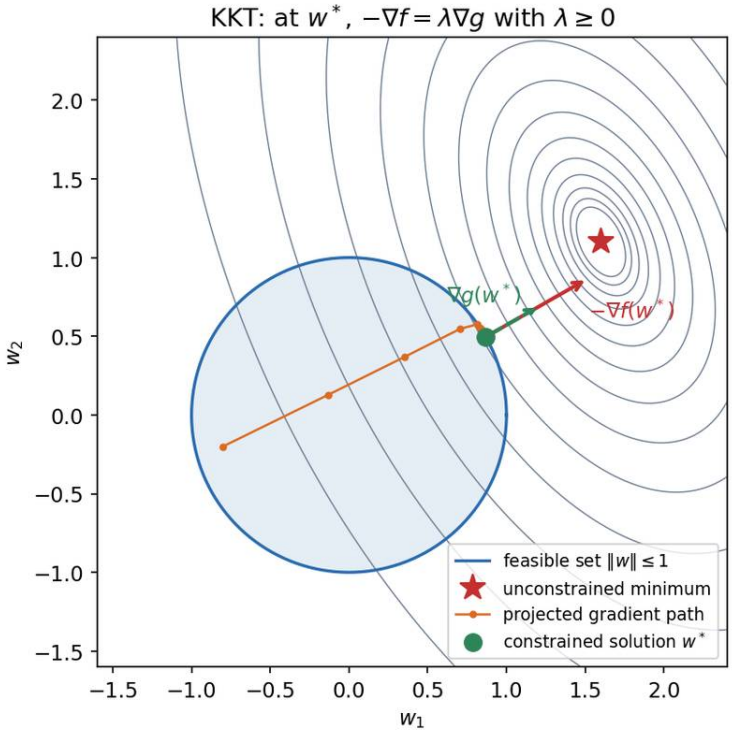


Figure 4 — Minimizing a quadratic over the unit disk. The unconstrained minimum (star) is infeasible. Projected gradient descent stops on the boundary at w, where $-\nabla f$ is parallel to the outward normal ∇g of the constraint: the KKT stationarity condition with a positive multiplier.*

Worked example: least squares with a norm constraint

Following the example in [1], minimize the squared error $f(w) = \|Aw - b\|_2^2$, with $A \in \mathbb{R}^{N \times n}$ and $b \in \mathbb{R}^N$, subject to $h(w) = w^\top w - r^2 \leq 0$. The Lagrangian is

$$L(w, \alpha) = \|Aw - b\|_2^2 + \alpha\,(w^\top w - r^2).$$

Stationarity gives $2A^\top(Aw - b) + 2\alpha w = 0$, so

$$w(\alpha) = (A^\top A + \alpha I)^{-1} A^\top b.$$

This is ridge regression with penalty α. If the ordinary least-squares solution $w(0)$ already satisfies $\|w\| \leq r$, the constraint is inactive, $\alpha^* = 0$, and we are done. Otherwise the constraint is active, and we increase α until $\|w(\alpha)\| = r$; because $\|w(\alpha)\|$ decreases monotonically in α, a bisection finds it. This is also how the inner maximization over α behaves: the multiplier grows while the constraint is violated and stops when it is just satisfied. The code solves the problem both ways and checks all four KKT conditions.

```
import numpy as np
```

```
# minimize ||Aw - b||^2  subject to  ||w||^2 <= r^2
rng = np.random.default_rng(1)
A = rng.normal(size=(30, 5))
b = rng.normal(size=30)
r = 0.2

def w_of(lam):                                  # stationarity of the Lagrangian
    return np.linalg.solve(A.T @ A + lam * np.eye(5), A.T @ b)

w_ls = w_of(0.0)
print("unconstrained norm:", np.linalg.norm(w_ls).round(3))

lo, hi = 0.0, 1e6                               # find lambda with ||w|| = r
for _ in range(200):
    mid = 0.5 * (lo + hi)
    if np.linalg.norm(w_of(mid)) > r:
        lo = mid
    else:
        hi = mid
lam = hi
w = w_of(lam)
print("lambda:", round(lam, 3), " ||w||:", np.linalg.norm(w).round(6))

# KKT checks
stationarity = 2 * A.T @ (A @ w - b) + 2 * lam * w
print("stationarity residual:", np.abs(stationarity).max())
print("primal feasible:", w @ w <= r**2 + 1e-9, " dual feasible:", lam >= 0)
print("complementary slackness:", lam * (w @ w - r**2))

# Projected gradient descent reaches the same point
L = 2 * np.linalg.eigvalsh(A.T @ A).max()     # Lipschitz constant of the gradient
v = np.zeros(5)
for _ in range(5000):
    v = v - (1 / L) * 2 * A.T @ (A @ v - b)
    n = np.linalg.norm(v)
    if n > r:
        v = v * (r / n)                         # projection onto the ball
print("projected GD matches KKT solution:", np.allclose(v, w, atol=1e-6))
```

The unconstrained solution has norm 0.219, larger than $r = 0.2$, so the constraint is active. Bisection finds $\alpha^* \approx 1.81$, the stationarity residual is about 10^{-15}, complementary slackness holds, and projected gradient descent with step $1/L$ converges to the same vector. Note how L, the Lipschitz constant of the gradient, sets a safe step size, exactly as in the previous section.

Checking derivatives

Plain version. Check that the gradient your code computes matches how the loss actually changes when you nudge the parameters.

A central finite difference estimates a partial derivative,

$$\frac{\partial f}{\partial x_j} \approx \frac{f(x + \epsilon e_j) - f(x - \epsilon e_j)}{2\epsilon},$$

where e_j is the j-th standard basis vector and $\epsilon > 0$ a small step. Its **truncation error** (from ignoring higher-order Taylor terms) is $O(\epsilon^2)$, better than the $O(\epsilon)$ of the forward difference $[f(x + \epsilon e_j) - f(x)]/\epsilon$. But the numerator subtracts two nearly equal numbers, so its **roundoff error** grows like $u\,|f|/\epsilon$. The total error is smallest at an intermediate step — roughly $\epsilon \approx u^{1/3}$ for central differences and $u^{1/2}$ for forward differences, when f and its derivatives have unit scale. Figure 5 shows the resulting V-shaped curves, and why float32 gradient checks are much less precise than float64 ones.

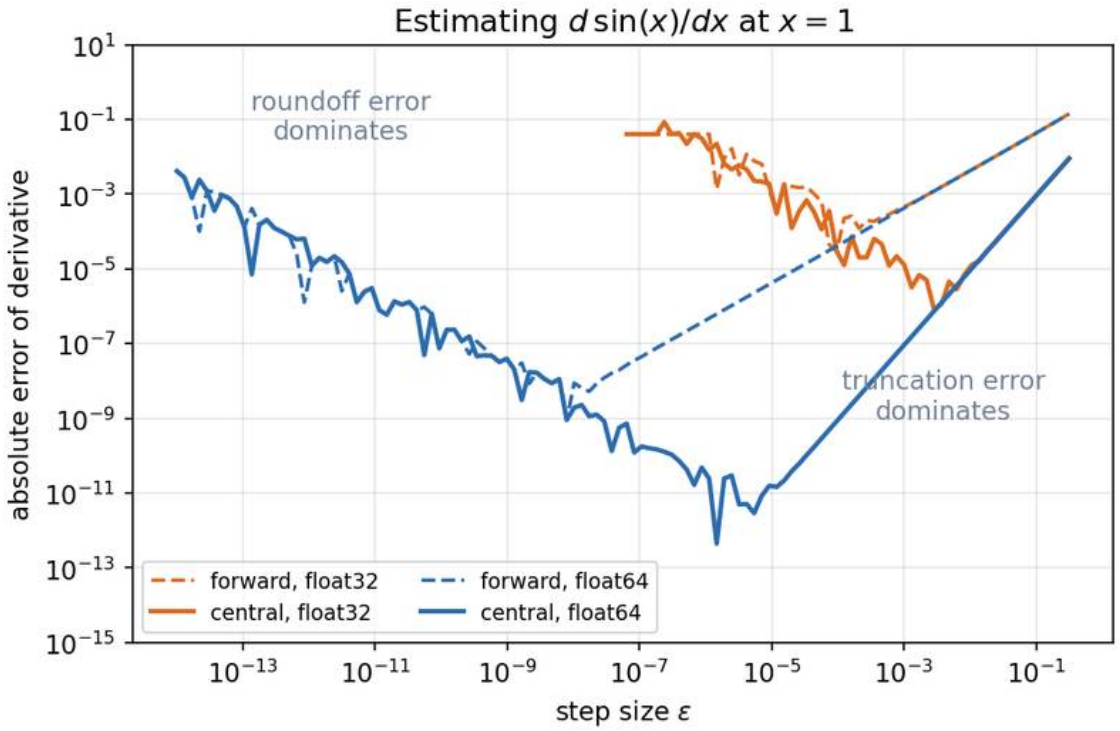


Figure 5 — Error of finite-difference derivatives of $\sin(x)$ at x=1. Right of each minimum, truncation error dominates (slope 1 for forward, 2 for central differences); left of it, roundoff dominates. Central differences bottom out around 1e−6 in float32 but around 1e−11 or lower in float64.

Practical rules: use float64 for checks; try several step sizes; compare with a relative error such as $|a - n|/\max(|a|, |n|, \text{tiny})$; and test at smooth points. ReLU exactly at zero is not differentiable, so there is no single derivative to match there. For many parameters, a **directional check** is cheaper: choose a random unit vector v and compare $\nabla f(w)^\top v$ with a finite difference along v — one pair of function evaluations instead of $2n$.

```
import numpy as np
```

```
rng = np.random.default_rng(0)
X = rng.normal(size=(20, 5))
y = rng.normal(size=20)
w = rng.normal(size=5)
v = rng.normal(size=5)
v /= np.linalg.norm(v)
def loss(w):
    return .5*np.mean((X@w-y)**2)
grad = X.T@(X@w-y)/len(y)
eps = 1e-5
numeric = (loss(w+eps*v)-loss(w-eps*v))/(2*eps)
analytic = grad@v
assert np.allclose(numeric, analytic, rtol=1e-6, atol=1e-8)
print("directional derivative:", numeric, analytic)
```

The two numbers agree to about nine significant digits. Remember what a passing check means. Automatic differentiation computes derivatives of the *recorded program*, not of the program you intended [11]. A detached tensor, a wrong reduction, accidental broadcasting or target leakage produces a perfectly differentiable but incorrect objective, and a gradient check will happily confirm its gradient. Check the loss values themselves on cases where you know the answer.

Mixed precision

Plain version. Doing most arithmetic in 16-bit numbers halves memory traffic and uses fast hardware units, but 16 bits are not enough for everything. Keep a few sensitive quantities in 32 bits.

Mixed-precision training, as introduced by Micikevicius et al. [12], combines three ideas:

1. Keep a **float32 master copy of the weights**. An update smaller than about 2^{-11} times the weight it is added to rounds away entirely in float16 (the same effect as $1 + \varepsilon_{\text{mach}}/2 = 1$ above), and many updates are that small.
2. Use **loss scaling** for float16: multiply the loss by a factor S before backpropagation so small gradients do not underflow below float16's smallest subnormal (6×10^{-8}), then divide the gradients by S before the update. Dynamic loss scaling reduces S when an overflow (inf or NaN) appears and skips that step.
3. **Accumulate** dot products and reductions in float32 even when inputs are float16.

bfloat16 has the same 8-bit exponent as float32, so it rarely needs loss scaling; Kalamkar et al. found bfloat16 training reached float32 results across a range of models without changing hyperparameters [5]. The price is precision: with $\varepsilon_{\text{mach}} \approx 0.0078$, bfloat16 cannot represent $1 + 0.003$ distinctly from 1, so long sums and weight updates still need float32 accumulation.

Loss scaling protects small float16 gradients; it cannot repair an invalid objective or an overflow elsewhere in the network (an attention logit or a normalization statistic, for example). Changing

precision can also change optimization trajectories and outputs, especially for reductions and long training runs. Validate on the target hardware; a smaller dtype does not guarantee that every operation becomes faster [13].

A debugging order for non-finite values

When a loss becomes `nan` or `inf`, resist the urge to clip everything. Find the *first* operation that produced a non-finite value, then ask which of this chapter's failure modes it is. A useful order:

1. **Inputs:** check shapes, ranges, dtypes and missing values. A single NaN pixel or an un-normalized 0–255 image changes everything downstream.
2. **Forward pass:** check that activations and the loss are finite. Look for `exp`, `log`, division, `sqrt` and normalization layers with near-zero variance.
3. **Loss:** use logits-based losses (`cross_entropy`, `binary_cross_entropy_with_logits`) rather than probabilities followed by `log`.
4. **Gradients:** run a gradient check on a tiny model; inspect per-layer gradient norms.
5. **Updates:** compare the update size with the weight size; a learning rate above the curvature limit $2/\lambda_{\max}$ causes oscillation and divergence even with perfect arithmetic.
6. **Precision:** reproduce in float32 (or float64) before adding mixed precision or compilation.

Overfit a tiny batch first to establish that the pipeline can learn. Gradient-norm clipping bounds the gradient passed to the optimizer; it does not directly bound an adaptive optimizer's update, and it does not make NaN gradients meaningful. ML 02 develops optimizers and normalization in more detail.

Modern view

The core ideas of this chapter are decades old, but they are at the centre of current research because models are larger, training runs are longer, and hardware keeps pushing precision downward.

Optimization theory for large-scale learning. Bottou, Curtis and Nocedal's SIAM Review survey [14] is the standard map of the field. It explains why stochastic gradient methods dominate large-scale machine learning (cheap noisy steps beat expensive exact ones when data are redundant), analyses their convergence under smoothness assumptions like the Lipschitz condition above, and organizes improvements into two families: *noise reduction* (larger or growing batches, variance reduction, iterate averaging) and *second-order* methods (Hessian-free Newton, Gauss–Newton, quasi-Newton, natural gradient and diagonal scaling). Nocedal and Wright [6] and Boyd and Vandenberghe [7] remain the reference textbooks for the deterministic and convex theory, and Baydin et al. [11] survey the automatic differentiation that makes gradients cheap.

What the loss surface actually looks like. Dauphin et al. [8] argued that saddle points, not poor local minima, are the main obstacle in high-dimensional non-convex optimization. Later, Cohen et al. [15] observed that full-batch gradient descent on neural networks typically operates at the "edge of stability": the largest Hessian eigenvalue rises until it hovers just above $2/\eta$ — precisely the classical stability limit derived above — while the loss keeps decreasing over long time scales, though not monotonically. The classical quadratic analysis predicts divergence there, so explaining this regime remains an open theoretical problem.

Curvature-aware optimizers. Full Newton is impossible at scale, but structured approximations are practical. K-FAC [16] approximates the Fisher information matrix (a positive-semidefinite curvature matrix closely related to the Gauss–Newton approximation of the Hessian) with Kronecker products of small per-layer matrices, so inverting it is cheap. Shampoo [17] keeps one preconditioning matrix per tensor dimension. More recently, Muon applies an approximate orthogonalization to each weight matrix's update; Liu et al. [18] report that, with suitable weight decay and per-parameter update scaling, it trains large language models with roughly twice the computational efficiency of their adaptive-optimizer baseline in their experiments. These methods all revisit the question this chapter asks with the Hessian: how should a step be rescaled direction by direction?

Lower precision. The format story continues from float16 with loss scaling [12] and bfloat16 [5] to 8-bit floating point. Micikevicius et al. [19] proposed two FP8 encodings, E4M3 (more precision) and E5M2 (more range), and showed training results comparable to 16-bit baselines across image and language models. The range–precision trade-off in the table above is exactly what these designs balance, and the IEEE 754 rules [3] are what they relax. Per-tensor or per-block scaling factors play the role that loss scaling played for float16.

Stable softmax at scale. The max-subtraction trick has a streaming version. Milakov and Gimelshein [20] showed that a softmax normalizer can be computed in one pass by keeping a running maximum and rescaling the running sum whenever the maximum changes. FlashAttention [21] builds on this idea: it computes exact attention block by block in fast on-chip memory, never materializing the full attention matrix, and FlashAttention-3 [22] extends it to FP8 with block quantization and "incoherent processing" to reduce quantization error. The core of the online trick fits in a few lines:

```
import numpy as np

def online_logsumexp(chunks):
    m, s = -np.inf, 0.0                 # running max and rescaled running sum
    for c in chunks:
        m_new = max(m, c.max())
        s = s * np.exp(m - m_new) + np.exp(c - m_new).sum()
        m = m_new
    return m + np.log(s)

rng = np.random.default_rng(0)
z = rng.normal(0, 50, size=10_000)
```

```
chunks = np.array_split(z, 37)          # seen one block at a time
m = z.max()
reference = m + np.log(np.exp(z - m).sum())
print(online_logsumexp(chunks), reference)
assert np.isclose(online_logsumexp(chunks), reference)
```

Both values print as 174.0918752579 ..., identical to the last digit shown, although the online version never sees all logits at once. ML 04 covers attention itself, and ML 10 covers quantization for deployment.

Open problems. Why gradient methods succeed at the edge of stability; how to make low-precision (8-bit and below) training as robust as float32 without hand-tuned scaling; how to make curvature-aware optimizers cheap and robust enough to replace first-order defaults; and how to obtain reproducible results when parallel reductions change summation order.

Key takeaways

- Floating-point numbers have a fixed relative precision ($\varepsilon_{\text{mach}}$) and a fixed range; exponent bits set range, fraction bits set precision.
- Overflow, underflow and cancellation are predictable. Fix them algebraically: subtract the max before `exp`, use `log1p`/`expm1`, use logits-based losses, subtract means before squaring.
- Conditioning is a property of the problem; stability is a property of the algorithm. A condition number of 10^k costs about k digits.
- The Hessian's eigenvalues give curvature. Gradient descent is stable only for $\eta < 2/\lambda_{\max}$ and slows down by the condition number $\lambda_{\max}/\lambda_{\min}$.
- Newton's method rescales by curvature and converges fast near minima, but it is expensive and attracted to saddles; quasi-Newton methods and Hessian-vector products are the practical compromises.
- Constraints can be handled by projection or by the generalized Lagrangian; the KKT conditions characterize the solution.
- Gradient checks need float64, sensible step sizes and smooth test points, and they only test the program you wrote.
- Mixed precision needs float32 master weights and accumulation, and loss scaling for float16; validate rather than assume identical behaviour.

Exercises

1. **Softmax shift.** Prove that softmax is invariant under $z_i \mapsto z_i + c$, and show that with $c = -\max_i z_i$ the denominator lies in $[1, K]$.

Hint

The shared factor $\exp(c)$ cancels between numerator and denominator. After the shift, every term $\exp(z_j - m)$ is in $(0, 1]$ and the largest equals 1, so the sum of K terms is between 1 and K. It can therefore neither overflow nor underflow to 0.

2. **Finite-difference sweep.** For $f(x) = e^x$ at $x = 0$, compute forward and central differences for $\epsilon = 10^{-1}, \ldots, 10^{-12}$ in float32 and float64. At which ϵ is each error smallest? Compare with $u^{1/2}$ and $u^{1/3}$.

Hint

For float64, $u \approx 1.1 \times 10^{-16}$, so $u^{1/2} \approx 10^{-8}$ (forward) and $u^{1/3} \approx 5 \times 10^{-6}$ (central). For float32, $u \approx 6 \times 10^{-8}$, giving about 2×10^{-4} and 4×10^{-3}. The observed minima should be within an order of magnitude of these, and the error at the minimum is roughly $u^{1/2}$ and $u^{2/3}$ respectively.

3. **Stability limit.** For $f(w) = \frac{1}{2} w^\top H w$ with $H = \mathrm{diag}(2, 50)$, find the largest stable fixed learning rate and the step size that minimizes the worst-case contraction factor. How many iterations does that best step need to reduce the error by a factor of 10^6?

Hint

Stability requires $\eta < 2/50 = 0.04$. The best step is $2/(\lambda_{\max} + \lambda_{\min}) = 2/52 \approx 0.0385$, with contraction $(\kappa - 1)/(\kappa + 1) = 24/26 \approx 0.923$. Solve $0.923^k \leq 10^{-6}$: $k \geq \ln 10^6 / (-\ln 0.923) \approx 173$ iterations. A Newton step would need one.

4. **Classify critical points.** Find and classify the critical points of $f(x, y) = x^3 - 3x + y^2$.

Hint

$\nabla f = (3x^2 - 3,\ 2y) = 0$ gives $(1, 0)$ and $(-1, 0)$. The Hessian is $\mathrm{diag}(6x,\ 2)$: at $(1, 0)$ it is $\mathrm{diag}(6, 2)$, positive definite, a local minimum; at $(-1, 0)$ it is $\mathrm{diag}(-6, 2)$, indefinite, a saddle. Neither is a global minimum because $f \to -\infty$ as $x \to -\infty$. Starting Newton's method near $(-1, 0)$ converges to the saddle.

5. **KKT on the simplex.** Minimize $f(x) = \frac{1}{2}\|x - c\|_2^2$ subject to $\sum_i x_i = 1$ and $x_i \geq 0$, for $c = (0.8,\ 0.6, -0.4)$. Write the Lagrangian and the KKT conditions, and find the solution.

Hint

$L = \frac{1}{2}\|x - c\|^2 + \lambda(\sum_i x_i - 1) - \sum_i \alpha_i x_i$ with $\alpha_i \geq 0$. Stationarity gives $x_i = c_i - \lambda + \alpha_i$; complementary slackness gives $x_i = \max(c_i - \lambda, 0)$. Guess that only the first two are positive: $(0.8 - \lambda) + (0.6 - \lambda) = 1$, so $\lambda = 0.2$ and $x = (0.6, 0.4, 0)$. Check: $c_3 - \lambda = -0.6 < 0$, so $x_3 = 0$ with $\alpha_3 = 0.6 \geq 0$. This is the Euclidean projection onto the probability simplex.

6. **A correct gradient of the wrong loss.** Change a mean reduction into a sum in the gradient-check example. Does the check still pass? Is autograd wrong? What else changes in training?

Hint

If you change both the loss and the analytic gradient consistently, the check passes: the derivative is correct for the new objective. Nothing is wrong with autograd. But the gradient is now N times larger, so the largest stable learning rate shrinks by N, and in a multi-term loss the balance between terms changes. A gradient check cannot tell you that you chose the wrong objective.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 4 (Numerical Computation) and ch. 8. Book
2. D. Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991. DOI
3. IEEE, "IEEE Standard for Floating-Point Arithmetic," IEEE Std 754-2019, 2019. DOI
4. N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002. DOI
5. D. Kalamkar et al., "A Study of BFLOAT16 for Deep Learning Training," arXiv:1905.12322, 2019. arXiv
6. J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006. DOI
7. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. Book
8. Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli and Y. Bengio, "Identifying and Attacking the Saddle Point Problem in High-Dimensional Non-Convex Optimization," *NeurIPS*, 2014. Paper
9. B. A. Pearlmutter, "Fast Exact Multiplication by the Hessian," *Neural Computation*, vol. 6, no. 1, pp. 147–160, 1994. DOI
10. D. C. Liu and J. Nocedal, "On the Limited Memory BFGS Method for Large Scale Optimization," *Mathematical Programming*, vol. 45, pp. 503–528, 1989. DOI
11. A. G. Baydin, B. A. Pearlmutter, A. A. Radul and J. M. Siskind, "Automatic Differentiation in Machine Learning: a Survey," *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018. arXiv
12. P. Micikevicius et al., "Mixed Precision Training," *ICLR*, 2018. arXiv
13. PyTorch documentation, "Numerical Accuracy." Documentation
14. L. Bottou, F. E. Curtis and J. Nocedal, "Optimization Methods for Large-Scale Machine Learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. DOI
15. J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter and A. Talwalkar, "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability," *ICLR*, 2021. arXiv
16. J. Martens and R. Grosse, "Optimizing Neural Networks with Kronecker-factored Approximate Curvature," *ICML*, 2015. Paper
17. V. Gupta, T. Koren and Y. Singer, "Shampoo: Preconditioned Stochastic Tensor Optimization," *ICML*, 2018. Paper
18. J. Liu, J. Su, X. Yao et al., "Muon is Scalable for LLM Training," arXiv:2502.16982, 2025. arXiv

19. P. Micikevicius et al., “FP8 Formats for Deep Learning,” arXiv:2209.05433, 2022. arXiv
20. M. Milakov and N. Gimelshein, “Online Normalizer Calculation for Softmax,” arXiv:1805.02867, 2018. arXiv
21. T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *NeurIPS*, 2022. Paper
22. J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani and T. Dao, “FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision,” *NeurIPS*, 2024. Paper

ML 02 • Neural Networks from Perceptron to Deep Networks

From a single perceptron to deep networks: activations, normalization, output layers and losses, backpropagation, optimizers, and the habits that make a network train and generalize.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-02-neural-networks/

What you'll learn

- What a perceptron is, how its learning rule works, and exactly how it differs from logistic regression.
- How stacking layers gives a multilayer perceptron (MLP), what the universal approximation theorem does and does not promise, and what "deep" really means.
- The activation functions in use today (sigmoid, tanh, softmax, ReLU, Leaky ReLU, GELU, SiLU/Swish, SwiGLU) and the normalization layers (BatchNorm, LayerNorm, RMSNorm), with their formulas and derivatives.
- How to pair the output layer with the right loss, and how backpropagation computes every gradient, derived by hand for a two-layer network and checked numerically in NumPy.
- How to train in practice: initialization, SGD, momentum, Adam and AdamW, learning-rate schedules, batch size, gradient clipping, and how to read a loss curve.
- How to diagnose underfitting and overfitting, the regularization tools that fix overfitting, and how residual connections lead to the Transformer.

The big picture

A neural network is a function with many adjustable knobs (its **parameters**). It takes an input, such as the pixels of an image or two coordinates of a point, and produces an output, such as a class label. Training means turning the knobs, a tiny amount at a time, so the outputs on known examples get closer to the right answers. Everything in this tutorial is about one of three questions: what shape the function has (the architecture), how to measure "closer" (the loss), and how to turn the knobs well (the optimization).

> **Explain like I'm five**
>
> Think of a team of people passing notes down a line. The first person looks at a picture and writes a few simple observations ("there is a curve here", "this part is bright"). The next person reads those notes and writes slightly smarter ones ("this looks like an eye"). The last person says "cat" or "dog". When the answer is wrong, a teacher walks back up the line telling each person how much their note

was to blame, and each one changes their habits a little. That walk backward is **backpropagation**, and doing it many thousands of times is **training**.

Why it matters: almost every modern system in vision, speech and language is built from the pieces in this tutorial. Convolutional networks, Transformers and diffusion models all use the same ingredients: linear layers, nonlinear activations, normalization, residual connections, a loss, backpropagation and an adaptive optimizer. Once these are clear, the deep-learning track is about arranging them for images.

About this tutorial

This tutorial is expanded from the author's earlier study notes on machine learning and neural networks [1], which are written in Traditional Chinese: 機器學習及類神經網路筆記. It assumes the ideas of ML 01: data, features, a loss, and gradient descent.

The perceptron

Plain version. A perceptron is the simplest artificial neuron. It multiplies each input by a weight, adds them up with a bias, and answers "yes" if the total is positive and "no" otherwise. It learns by nudging its weights only when it gets an example wrong.

The model

Rosenblatt introduced the perceptron in 1958 as a model of how a brain might store and recognize patterns [4]. For an input vector $\mathbf{x} \in \mathbb{R}^d$ it computes

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \mathbf{1}[z > 0],$$

where $\mathbf{w} \in \mathbb{R}^d$ is the weight vector, $b \in \mathbb{R}$ is the bias, z is the **pre-activation** (a weighted sum), $\mathbf{1}[\cdot]$ is the indicator that equals 1 when its condition is true and 0 otherwise, and $\hat{y} \in \{0, 1\}$ is the predicted class. The indicator is the **step function**. Geometrically, $\mathbf{w}^\top \mathbf{x} + b = 0$ is a hyperplane (a line in 2-D), and the perceptron labels points by which side of it they fall on.

The perceptron learning rule

Go through the training examples $(\mathbf{x}_i, y_i)$ one at a time. For each one, compute $\hat{y}_i$ and update

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y_i - \hat{y}_i)\,\mathbf{x}_i, \qquad b \leftarrow b + \eta\,(y_i - \hat{y}_i),$$

where $\eta > 0$ is the **learning rate** (step size) and $y_i \in \{0, 1\}$ is the true label. If the prediction is right, $y_i - \hat{y}_i = 0$ and nothing changes. If the perceptron said 0 but the answer was 1, the weights

move *toward* $\mathbf{x}_i$, which raises z for this input next time; the opposite mistake moves them away.

When the two classes can be separated by a hyperplane (the data are **linearly separable**), this rule stops making mistakes after a finite number of updates. When they cannot, it never settles.

```
import numpy as np

def perceptron_train(X, y, epochs=20, eta=1.0):
    """X: (N, d), y in {0, 1}. Returns w, b."""
    w, b = np.zeros(X.shape[1]), 0.0
    for _ in range(epochs):
        mistakes = 0
        for x_i, y_i in zip(X, y):
            y_hat = float(w @ x_i + b > 0)       # step function
            if y_hat != y_i:                     # update only on a mistake
                w += eta * (y_i - y_hat) * x_i
                b += eta * (y_i - y_hat)
                mistakes += 1
        if mistakes == 0:
            break
    return w, b

# AND is linearly separable; XOR is not
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
for name, y in [("AND", [0, 0, 0, 1]), ("XOR", [0, 1, 1, 0])]:
    w, b = perceptron_train(X, np.array(y, float))
    print(name, (X @ w + b > 0).astype(int))
# AND [0 0 0 1]
# XOR [1 1 0 0]   <- wrong: no single line can do it
```

Perceptron versus logistic regression

The two models are easy to confuse because they share the same first step, $z = \mathbf{w}^\top \mathbf{x} + b$. They are different methods:

	Perceptron	Logistic regression
Output function	step: $\hat{y} = \mathbf{1}[z > 0]$	sigmoid: $\hat{p} = \sigma(z) = \dfrac{1}{1+e^{-z}}$
Output meaning	a hard 0/1 decision	a probability $P(y = 1 \mid \mathbf{x})$
Training	perceptron rule, updates only on mistakes	minimize cross-entropy (maximum likelihood) with gradient descent
Non-separable data	never converges	converges to a well-defined best fit

The gradient of the logistic-regression loss for one example is $(\hat{p}_i - y_i)\,\mathbf{x}_i$, so its gradient-descent step is $\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y_i - \hat{p}_i)\,\mathbf{x}_i$. It looks almost identical to the perceptron rule, but $\hat{p}_i$ is a probability between 0 and 1, so *every* example contributes, and confidently correct ones contribute very little. That smoothness is what makes logistic regression trainable by gradient descent and what lets it

be stacked into larger differentiable networks. The step function, in contrast, has derivative zero almost everywhere, so gradients cannot flow through it.

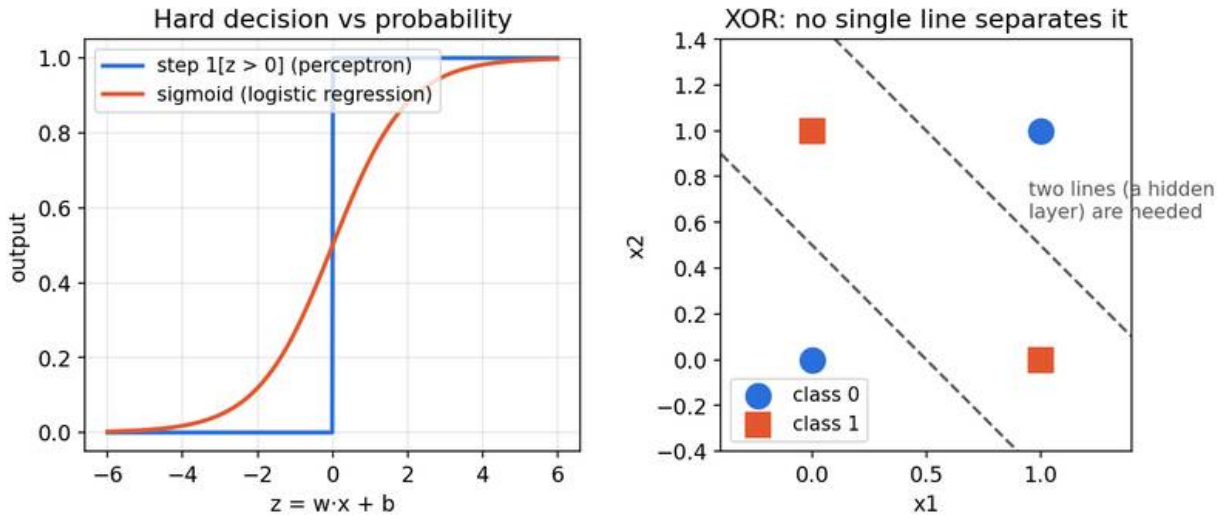


Figure 1 — Left: the perceptron's step output versus logistic regression's sigmoid probability. Right: XOR, the classic problem that no single linear boundary can solve; two boundaries (a hidden layer) are needed.

The multilayer perceptron (MLP)

Plain version. One neuron draws one straight line. Put many neurons side by side to get many lines, then feed their outputs into another layer that combines them. The combination can bend, so the network can carve out curved regions.

Stacking layers

An MLP with one hidden layer computes

$$\mathbf{h} = f(W_1\mathbf{x} + \mathbf{b}_1),$$
$$\hat{\mathbf{y}} = g(W_2\mathbf{h} + \mathbf{b}_2),$$

where $\mathbf{x} \in \mathbb{R}^d$ is the input, $W_1 \in \mathbb{R}^{m\times d}$ and $\mathbf{b}_1 \in \mathbb{R}^m$ are the first layer's weights and biases, m is the **width** (number of hidden units), f is a nonlinear **activation function** applied element-wise, $\mathbf{h} \in \mathbb{R}^m$ is the hidden representation, $W_2 \in \mathbb{R}^{K\times m}$ and $\mathbf{b}_2 \in \mathbb{R}^K$ map it to K outputs, and g is the output function chosen for the task (see below). Each hidden unit is one "logistic-regression-like" neuron; the name multilayer *perceptron* is historical, since modern MLPs use smooth activations, not the step function.

The nonlinearity f is essential. Without it,

$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2)$ is again a single linear map: any number of linear layers collapses to one, and the network can still only draw one hyperplane. For XOR, two hidden units can each draw one of the dashed lines in Figure 1, and the output unit then answers "between the two lines".

Universal approximation

Cybenko proved in 1989 that a network with **one** hidden layer of sigmoidal units can approximate any continuous function on a bounded box (the unit hypercube) to any desired accuracy, provided the hidden layer is wide enough [5]. Later work extended the result to other activations, including ReLU. The theorem is reassuring, but read it carefully:

- It is about **existence**. It says suitable weights exist; it says nothing about whether gradient descent will find them.
- It says nothing about **how wide** "wide enough" is. For some functions the required width grows extremely fast with the input dimension.
- It says nothing about **generalization**: fitting the training points well does not mean predicting new points well.

So the theorem explains why MLPs are flexible enough in principle. The rest of this tutorial is about the practical questions it leaves open.

Deep neural networks: depth and width

Plain version. "Deep" just means many layers stacked in sequence. There is no official number of layers at which a network becomes deep.

A **deep neural network** (DNN) is an MLP, or any other layered network, with several hidden layers: $\mathbf{h}_\ell = f(W_\ell\mathbf{h}_{\ell-1} + \mathbf{b}_\ell)$ for $\ell = 1, \dots, L$, with $\mathbf{h}_0 = \mathbf{x}$. There is no agreed threshold for "deep". A common informal usage calls any network with more than one hidden layer deep, while other texts count total layers or hidden layers differently, so "three layers" means different things in different books. What matters is the idea, not the count: a deep network builds its answer as a **composition** of many simple steps.

The two ways to add capacity are **width** (more units per layer) and **depth** (more layers). They are not equivalent. A wider layer adds more pieces that are combined once; a deeper network re-uses the features of earlier layers to build more abstract ones. The review by LeCun, Bengio and Hinton [2] explains the success of deep learning through exactly this point: each layer turns the representation from the previous layer into a slightly more abstract one, and composing many

such layers lets the network learn very intricate functions from raw data, with the features learned rather than hand-designed.

Historically, the 1990s were dominated by **shallow** methods. Support vector machines, a kernel method rather than a wide neural network, were the main tool, and deep networks were hard to train. From the mid-2000s onward, more data, GPUs and better training techniques made depth practical, and deep networks began to win benchmark after benchmark [2]. Depth alone is not the whole story, though: simply stacking more plain layers eventually makes training *worse*, a problem that residual connections solved (see the last main section).

Figure 2 shows width and depth at work on the two-moons dataset from scikit-learn. With two or eight hidden units in one layer, the boundary is made of only a few straight pieces and cannot follow the moons. Wider or deeper networks bend the boundary to fit; the widest and deepest one even wraps around single noisy points, a first hint of overfitting.

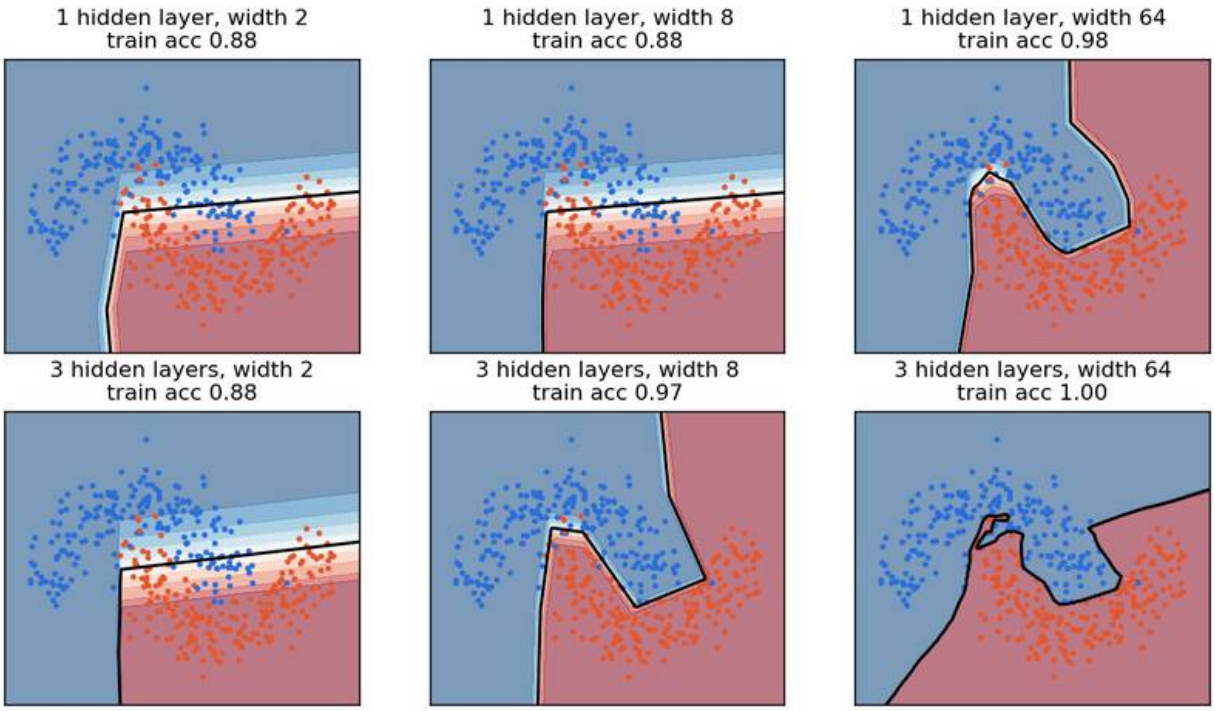


Figure 2 — Decision boundaries of ReLU MLPs on make_moons (400 points, noise 0.2) as width (columns) and depth (rows) change, each trained with Adam for 1,500 full-batch steps. The black curve is the 0.5 probability contour.

Activation functions

Plain version. The activation function is the "bend" applied after each weighted sum. Without it, the whole network would be a straight-line model. Different bends trade off smoothness, speed

and how well gradients pass through many layers.

Figure 3 plots the common choices and their derivatives. The derivative matters as much as the function, because backpropagation multiplies derivatives layer after layer (see the training section).

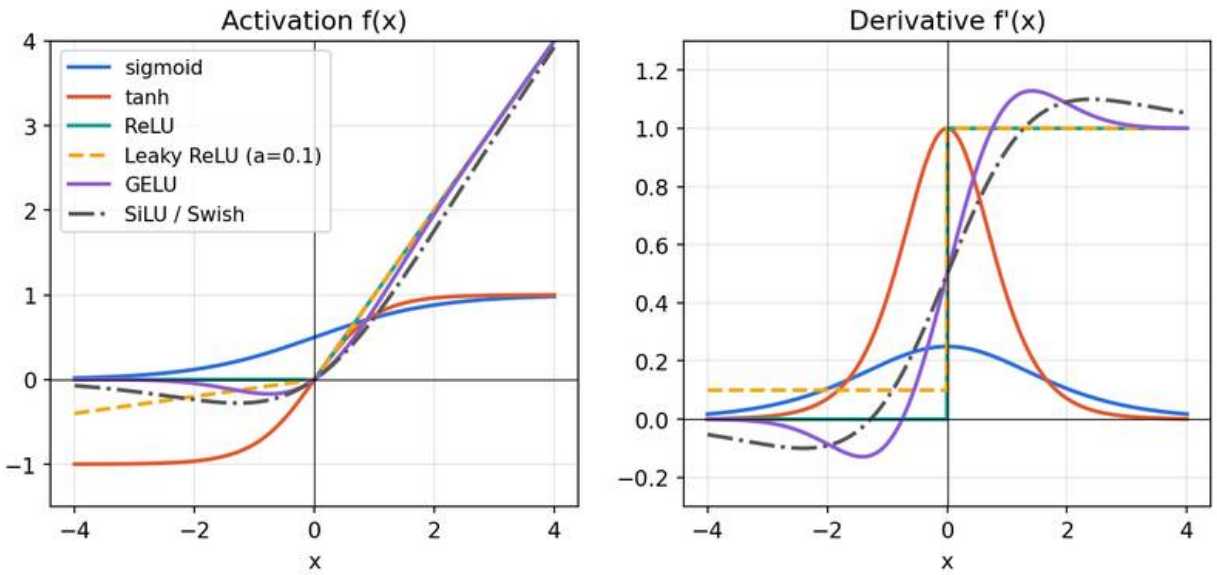


Figure 3 — Common activation functions (left) and their derivatives (right). Note how small the sigmoid's derivative is everywhere (at most 0.25) and how flat both sigmoid and tanh become for large inputs.

Sigmoid and tanh

$$\sigma(x) = \frac{1}{1+e^{-x}}, \quad \sigma'(x) = \sigma(x)(1-\sigma(x)); \qquad \tanh(x) = 2\sigma(2x) - 1, \quad \tanh'(x) = 1 - \tanh^2(x).$$

The sigmoid maps any real number to $(0, 1)$; tanh maps it to $(-1, 1)$ and is centered at zero. Both **saturate**: for large $|x|$ the curve is flat and the derivative is nearly zero, so almost no gradient passes through. The sigmoid's derivative is at most $1/4$ (at $x = 0$). These two facts made deep sigmoid networks hard to train, which Glorot and Bengio analyzed in detail [17]. Today sigmoid is used mainly at the **output** of binary classifiers and inside gates (as in SwiGLU below), and tanh in some recurrent networks.

Softmax

Softmax turns a vector of K real scores $\mathbf{z}$ (called **logits**) into a probability distribution:

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j),$$

where δ_{kj} is 1 if $k = j$ and 0 otherwise. Each p_k is positive and they sum to 1. Unlike the others in this section, softmax acts on a **whole vector**, not element by element, so it is used almost only in the output layer of multi-class classifiers (and inside attention), not as a hidden activation. In code, subtract $\max_j z_j$ before exponentiating; it does not change the result and avoids overflow.

ReLU and Leaky ReLU

$$\mathrm{ReLU}(x) = \max(0, x), \quad \mathrm{ReLU}'(x) = \mathbf{1}[x > 0]; \qquad \mathrm{LeakyReLU}_a(x) = \max(ax, x), \quad 0 < a < 1.$$

The rectified linear unit passes positive inputs unchanged and zeroes the rest. Its derivative is exactly 1 for all positive inputs, so gradients do not shrink as they pass through active units, and it is very cheap to compute. ReLU was one of the changes that made deep supervised networks trainable without unsupervised pre-training [2]. Its weakness is the **dying ReLU**: a unit whose input is negative for every example has zero gradient and stops learning. Leaky ReLU keeps a small slope a (for example 0.01) on the negative side so some gradient always flows. The activation survey by Dubey, Singh and Chaudhuri [7] groups these and many later variants into a ReLU-based family.

GELU and SiLU/Swish

These are smooth versions of ReLU that let small negative values through:

$$\mathrm{GELU}(x) = x\,\Phi(x), \qquad \mathrm{GELU}'(x) = \Phi(x) + x\,\varphi(x),$$

where Φ is the standard normal cumulative distribution function and φ its density [8]. Instead of gating by sign as ReLU does, GELU weights the input by how large it is: $\Phi(x)$ is the probability that a standard normal variable is below x. A common fast approximation is $0.5x(1 + \tanh[\sqrt{2/\pi}\,(x + 0.044715x^3)])$ [8]. GELU is the default activation in many Transformer models.

$$\mathrm{Swish}_\beta(x) = x\,\sigma(\beta x), \qquad \mathrm{SiLU}(x) = x\,\sigma(x), \qquad \mathrm{SiLU}'(x) = \sigma(x)(1 + x(1 - \sigma(x))).$$

The sigmoid-weighted linear unit (SiLU) was proposed by Elfwing, Uchibe and Doya for reinforcement learning [9]. Ramachandran, Zoph and Le found the same shape with a learnable or fixed β by automated search and called it Swish [10]; with $\beta = 1$, Swish is SiLU. GELU and SiLU look very similar (Figure 3). Both are non-monotonic: they dip slightly below zero for moderate negative inputs.

SwiGLU: a gated feed-forward block

A **gated linear unit** (GLU) multiplies two linear projections element-wise, one of them passed through a nonlinearity that acts like a gate. Shazeer tested several variants in the Transformer's feed-forward block [11]. The SwiGLU version, written in the paper's row-vector notation and without biases, is

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(\mathbf{x}) = \left(\mathrm{Swish}_1(\mathbf{x}W) \odot \mathbf{x}V\right) W_2,$$

where $\mathbf{x}$ is a row vector of size d_{model}, W and V are $d_{\text{model}} \times d_{\text{ff}}$ matrices, W_2 is $d_{\text{ff}} \times d_{\text{model}}$, and $\odot$ is element-wise multiplication. In Shazeer's experiments the GLU variants, SwiGLU and GEGLU among them, gave better results than the ReLU and GELU versions of the block. Because SwiGLU has three weight matrices instead of two, d_{ff} is reduced by a factor of $2/3$ to keep the parameter count equal [11]. LLaMA, for example, replaced ReLU with SwiGLU using a hidden size of $\frac{2}{3} \cdot 4d$ [12], and SwiGLU has become a common default in large language models.

Activation	Range	Smooth	Typical use today
sigmoid	$(0, 1)$	yes	binary / multi-label output, gates
tanh	$(-1, 1)$	yes	some recurrent networks
softmax	probability vector	yes	multi-class output, attention weights
ReLU / Leaky ReLU	$[0, \infty)$ / $\mathbb{R}$	no (kink at 0)	CNNs, MLPs, the safe default
GELU, SiLU	about $[-0.17, \infty)$, $[-0.28, \infty)$	yes	Transformers, modern CNNs
SwiGLU	$\mathbb{R}$	yes	Transformer feed-forward blocks

Normalization layers

Plain version. As training goes on, the numbers flowing through a network can drift to very large or very small scales, which makes learning unstable. A normalization layer rescales them to a standard size at each step, then lets the network choose the final scale it prefers.

All three common layers share one template. For a set of values $x_1, \ldots, x_n$,

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2, \qquad y_i = \gamma\,\frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta,$$

where μ and σ^2 are the mean and variance, ϵ is a small constant for numerical safety, and γ, β are a learnable scale and shift (so the layer can undo the normalization if that helps). The layers differ in **which values** form the set (Figure 4).

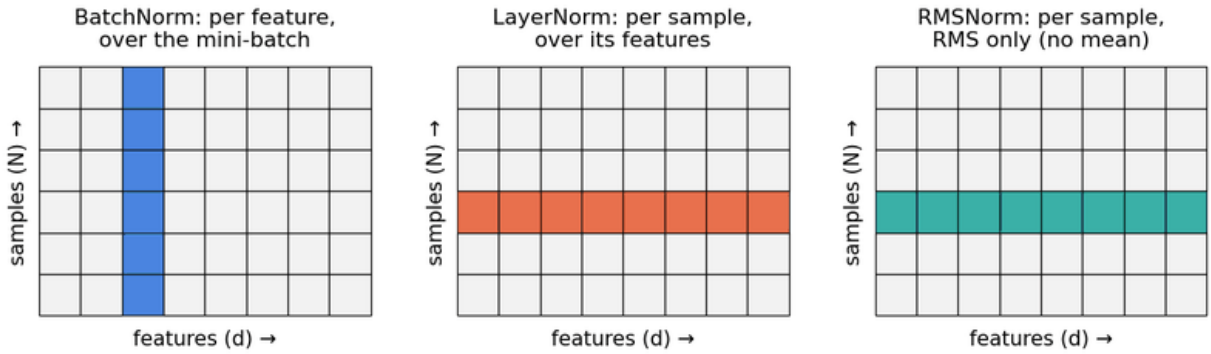


Figure 4 — Which entries share statistics. BatchNorm normalizes each feature over the mini-batch; LayerNorm and RMSNorm normalize each sample over its own features.

- **BatchNorm** (Ioffe and Szegedy [13]) computes μ, σ^2 for each feature (each channel, in a CNN) **across the mini-batch**. It speeds up training and became standard in CNNs. Because the statistics depend on the batch, it behaves differently in training and inference: at inference it uses running averages collected during training. It also degrades when batches are very small, since the batch statistics become noisy.
- **LayerNorm** (Ba, Kiros and Hinton [14]) computes μ, σ^2 **over the features of a single sample**. It does not depend on the batch, behaves identically at training and test time, and works naturally for sequences. It is the standard normalization in Transformers.
- **RMSNorm** (Zhang and Sennrich [15]) drops the mean subtraction and divides by the root mean square, $y_i = \gamma_i \, x_i / \sqrt{\frac{1}{n}\sum_j x_j^2 + \epsilon}$. The authors hypothesize that LayerNorm's re-centering is dispensable and report comparable quality with lower running time [15]. LLaMA uses RMSNorm and applies it to the **input** of each Transformer sub-layer ("pre-normalization") rather than to its output [12].

```
import torch

x = torch.randn(4, 8) * 5 + 3                       # 4 samples, 8 features
ln = torch.nn.LayerNorm(8, elementwise_affine=False)
bn = torch.nn.BatchNorm1d(8, affine=False)
rms = x / torch.sqrt(x.pow(2).mean(dim=1, keepdim=True) + 1e-6)

print(ln(x).mean(dim=1))     # ~0 for every sample (row)
print(bn(x).mean(dim=0))     # ~0 for every feature (column), in train mode
print(rms.pow(2).mean(dim=1))  # ~1 for every sample: unit RMS, mean not removed
```

Huang et al. [16] survey this family and decompose any normalization method into three choices: which values are pooled together (the *normalization area*), what operation is applied (standard-

izing, centering, scaling or whitening), and how the representation is recovered afterward (the learnable affine step). BatchNorm, LayerNorm and RMSNorm differ mainly in the first two.

Designing the output layer and choosing the loss

Plain version. The last layer must speak the language of the answer: any real number for a quantity, a probability for yes/no, a probability distribution for "which of K". Each kind of output has a natural loss, and the pairs below are the ones to use.

Task	Output layer	Output range	Loss
Regression	linear (no activation)	$\mathbb{R}$	mean squared error (MSE, L2) or mean absolute error (MAE, L1)
Binary classification	1 unit + sigmoid	$(0, 1)$	binary cross-entropy
Multi-label (several yes/no)	K units + sigmoid each	$(0, 1)^K$	binary cross-entropy per label
Multi-class (exactly one of K)	K units + softmax	probability vector	cross-entropy

A common mistake is to put a sigmoid on a regression output. The sigmoid's range is the open interval $(0, 1)$, and its job is to output a probability for binary or multi-label classification. A regression head is normally **linear**, so it can produce any real value. A sigmoid on a regression output is reasonable only when the target itself is known to lie between 0 and 1, such as a normalized pixel intensity or a fraction.

The losses, for one example, are

$$\begin{aligned} \text{MSE:} \quad & \ell = \tfrac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|^2, \\ \text{binary cross-entropy:} \quad & \ell = -[\, y \log \hat{p} + (1 - y) \log(1 - \hat{p}) \,], \\ \text{cross-entropy:} \quad & \ell = -\sum_{k=1}^{K} y_k \log p_k = -\log p_c, \end{aligned}$$

where $\mathbf{y}$ is the target, $\hat{p} = \sigma(z)$, $\mathbf{p} = \text{softmax}(\mathbf{z})$, y_k is the one-hot label and c is the correct class. (The factor $\frac{1}{2}$ in MSE is a convenience; PyTorch's `MSELoss` averages without it.) MAE, $\|\hat{\mathbf{y}} - \mathbf{y}\|_1$, is less sensitive to outliers than MSE. Each pair is a maximum-likelihood estimate: MSE corresponds to Gaussian noise on the target, binary cross-entropy to a Bernoulli label, cross-entropy to a categorical label [3].

The pairs are also chosen for a beautiful practical reason. In all three cases the gradient of the loss with respect to the **pre-activation** of the output layer is simply

$$\frac{\partial \ell}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y} \quad \text{(prediction minus target)},$$

with $\hat{\mathbf{y}}$ meaning the linear output, $\sigma(z)$ or $\text{softmax}(\mathbf{z})$ respectively. The saturating sigmoid or softmax derivative cancels out, so a confidently wrong prediction still produces a large gradient. (Exercise 3 asks you to verify this for softmax.) This is why frameworks fuse the last activation into the loss: PyTorch's `nn.CrossEntropyLoss` and `nn.BCEWithLogitsLoss` take **logits**, so the model itself must not end with a softmax or sigmoid during training.

Training I: the forward pass and backpropagation

Plain version. The forward pass computes the prediction and the loss, layer by layer from input to output. Backpropagation then walks back from the loss to the input, using the chain rule to find how much each weight contributed to the error. It reuses intermediate results, so all gradients cost about as much as one extra forward pass.

Backpropagation was popularized for training multi-layer networks by Rumelhart, Hinton and Williams [6]. It is nothing more than the chain rule of calculus, organized so that no work is repeated. Figure 5 shows the computation graph for the network we will differentiate.

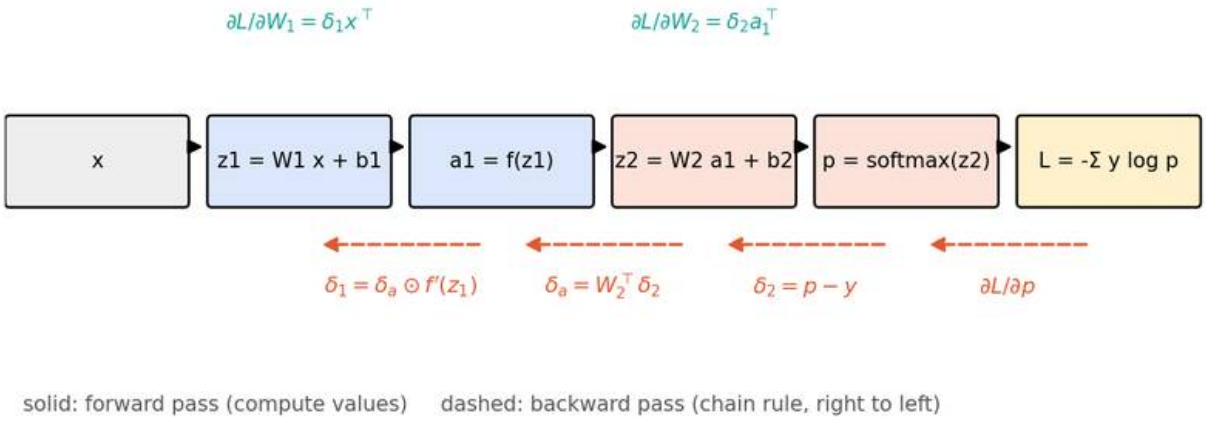


Figure 5 — Computation graph of a two-layer MLP with softmax cross-entropy. Solid arrows: forward pass. Dashed arrows: the backward pass, which carries the error signal δ from the loss back toward the input.

Derivation for a two-layer MLP

Take one example $(\mathbf{x}, \mathbf{y})$ with $\mathbf{x} \in \mathbb{R}^d$ and a one-hot label $\mathbf{y} \in \{0,1\}^K$. The forward pass is

$$\begin{aligned} \mathbf{z}_1 &= W_1\mathbf{x} + \mathbf{b}_1 \in \mathbb{R}^m, \quad \mathbf{a}_1 = f(\mathbf{z}_1) \in \mathbb{R}^m, \\ \mathbf{z}_2 &= W_2\mathbf{a}_1 + \mathbf{b}_2 \in \mathbb{R}^K, \quad \mathbf{p} = \text{softmax}(\mathbf{z}_2), \qquad L = -\textstyle\sum_k y_k \log p_k, \end{aligned}$$

with $W_1 \in \mathbb{R}^{m\times d}$, $W_2 \in \mathbb{R}^{K\times m}$, and f an element-wise activation. Define the **error signal** of a layer as the gradient of the loss with respect to its pre-activation, $\delta_\ell = \partial L/\partial \mathbf{z}_\ell$.

Step 1, output layer. From the previous section, softmax with cross-entropy gives

$$\delta_2 = \frac{\partial L}{\partial \mathbf{z}_2} = \mathbf{p} - \mathbf{y} \in \mathbb{R}^K.$$

Step 2, output weights. Since $z_{2,k} = \sum_j W_{2,kj}\, a_{1,j} + b_{2,k}$, we have $\partial z_{2,k}/\partial W_{2,kj} = a_{1,j}$, so

$$\frac{\partial L}{\partial W_2} = \delta_2\, \mathbf{a}_1^\top \in \mathbb{R}^{K\times m}, \qquad \frac{\partial L}{\partial \mathbf{b}_2} = \delta_2.$$

The gradient of a weight is (error at its output) times (activation at its input). The shapes match W_2, which is a useful check.

Step 3, back through the linear layer. Each hidden activation $a_{1,j}$ affects every output, so its gradient sums over them: $\partial L/\partial a_{1,j} = \sum_k \delta_{2,k} W_{2,kj}$, or in matrix form

$$\frac{\partial L}{\partial \mathbf{a}_1} = W_2^\top \delta_2 \in \mathbb{R}^m.$$

Step 4, back through the activation. Because f acts element by element, its Jacobian is diagonal, and the chain rule becomes an element-wise product:

$$\delta_1 = \frac{\partial L}{\partial \mathbf{z}_1} = (W_2^\top \delta_2) \odot f'(\mathbf{z}_1).$$

Step 5, first-layer weights. Exactly as in Step 2,

$$\frac{\partial L}{\partial W_1} = \delta_1\, \mathbf{x}^\top \in \mathbb{R}^{m\times d}, \qquad \frac{\partial L}{\partial \mathbf{b}_1} = \delta_1.$$

For a deeper network, Steps 3 to 5 simply repeat:
$\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$ and $\partial L/\partial W_\ell = \delta_\ell\, \mathbf{a}_{\ell-1}^\top$. For a mini-batch of N examples, the loss is the average, so each gradient is the average of the per-example gradients.

From scratch in NumPy, with a gradient check

The code below stores examples as **rows** (`X` has shape $N\times d$), which is the usual convention in code. Then `X @ W1` with `W1` of shape $d \times m$ plays the role of $W_1\mathbf{x}$ in the math, and every formula above appears transposed: for example `dW2 = A1.T @ dZ2` is $\delta_2\mathbf{a}_1^\top$ stacked over the batch. The **gradient check** compares one analytic derivative with a central finite difference, $(L(w+\epsilon) - L(w-\epsilon))/2\epsilon$. If

they agree to many digits, the backward pass is correct. Always do this when you write gradients by hand.

```
import numpy as np
from sklearn.datasets import make_moons

rng = np.random.default_rng(0)
X, y = make_moons(n_samples=200, noise=0.2, random_state=0)
N, d, h, K = X.shape[0], 2, 16, 2
Y = np.eye(K)[y]                                  # one-hot targets, (N, K)

# He-style initialization (std = sqrt(2 / fan_in))
W1 = rng.normal(0, np.sqrt(2 / d), (d, h)); b1 = np.zeros(h)
W2 = rng.normal(0, np.sqrt(2 / h), (h, K)); b2 = np.zeros(K)

def forward(X, W1, b1, W2, b2):
    Z1 = X @ W1 + b1                              # (N, h)
    A1 = np.maximum(Z1, 0)                        # ReLU
    Z2 = A1 @ W2 + b2                             # (N, K) logits
    Z2 = Z2 - Z2.max(axis=1, keepdims=True)       # numerical stability
    P = np.exp(Z2) / np.exp(Z2).sum(axis=1, keepdims=True)
    loss = -np.mean(np.sum(Y * np.log(P + 1e-12), axis=1))
    return Z1, A1, P, loss

def backward(X, Z1, A1, P, W2):
    dZ2 = (P - Y) / N                             # delta at the output
    dW2 = A1.T @ dZ2;  db2 = dZ2.sum(axis=0)
    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * (Z1 > 0)                          # ReLU'(z) = 1[z > 0]
    dW1 = X.T @ dZ1;   db1 = dZ1.sum(axis=0)
    return dW1, db1, dW2, db2

# Gradient check on one entry of W1 (central difference)
Z1, A1, P, loss = forward(X, W1, b1, W2, b2)
dW1, db1, dW2, db2 = backward(X, Z1, A1, P, W2)
eps = 1e-6; E = np.zeros_like(W1); E[0, 3] = eps
num = (forward(X, W1 + E, b1, W2, b2)[3] - forward(X, W1 - E, b1, W2, b2)[3]) / (2 * eps)
print(f"analytic {dW1[0, 3]:.8f}  numeric {num:.8f}")

# Plain gradient descent
lr = 0.5
for step in range(2000):
    Z1, A1, P, loss = forward(X, W1, b1, W2, b2)
    dW1, db1, dW2, db2 = backward(X, Z1, A1, P, W2)
    W1 -= lr * dW1; b1 -= lr * db1; W2 -= lr * dW2; b2 -= lr * db2
acc = (P.argmax(axis=1) == y).mean()
print(f"final loss {loss:.3f}, train accuracy {acc:.3f}")
# analytic -0.01941581  numeric -0.01941581
# final loss 0.063, train accuracy 0.970
```

In practice you never write the backward pass yourself: frameworks such as PyTorch record the forward computation and run this exact procedure automatically when you call `loss.backward`

`()`. Writing it once by hand is still the best way to understand what that call does, and why it fails when something in the graph has a zero derivative.

Training II: making optimization work

Plain version. Knowing the gradient is not enough. You also need a sensible starting point (initialization), a rule for turning gradients into steps (the optimizer), a step size that changes over time (the schedule), a choice of how many examples to look at per step (the batch size), and a guard against occasional huge steps (clipping).

Vanishing and exploding gradients

Unroll the recursion $\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$ across L layers and the gradient at the first layer contains a **product** of L weight matrices and L activation derivatives. If the typical factor is below 1, the product shrinks exponentially with depth and the early layers barely learn: **vanishing gradients**. With the sigmoid, every f' is at most 0.25, so ten layers can shrink the signal by a factor of up to $0.25^{10} \approx 10^{-6}$. If the typical factor is above 1, for example because the initial weights are too large, the product grows exponentially, updates become huge, and the loss turns into `inf` or `NaN`: **exploding gradients**.

The remedies, each covered below, attack the product from different sides:

- activations whose derivative is 1 over a wide range (ReLU and its smooth relatives);
- initialization that keeps signal size constant from layer to layer;
- normalization layers that re-standardize activations;
- **residual connections**, which give the gradient a path with factor 1 (the last main section);
- **gradient clipping**, the standard emergency brake for explosions.

Other practices sometimes listed, such as fine-tuning a pre-trained model or using a smaller network, make training easier overall, but they do not directly fix the gradient problem.

Weight initialization

If all weights start equal, every hidden unit computes the same thing and receives the same gradient, and they stay identical forever. So weights start random. The scale matters: we want the variance of activations (forward) and of gradients (backward) to stay roughly constant from layer to layer. For a linear layer with n_{in} inputs and n_{out} outputs:

- **Xavier (Glorot) initialization** [17] balances the forward and backward conditions for symmetric activations such as tanh: $\mathrm{Var}(W_{ij}) = \dfrac{2}{n_{\text{in}} + n_{\text{out}}}$, for example uniform on $[-\sqrt{6/(n_{\text{in}} + n_{\text{out}})}, \sqrt{6/(n_{\text{in}} + n_{\text{out}})}]$.
- **He (Kaiming) initialization** [18] accounts for ReLU zeroing about half its inputs, which halves the variance at every layer, by doubling the scale: $\mathrm{Var}(W_{ij}) = \dfrac{2}{n_{\text{in}}}$, for example $W_{ij} \sim \mathcal{N}(0, 2/n_{\text{in}})$.

Biases usually start at zero. He et al. showed that this initialization lets very deep ReLU networks be trained from scratch [18]. The NumPy example above uses it.

Optimizers

Every optimizer below is a variant of **stochastic gradient descent** (SGD): estimate the gradient on a small random mini-batch, take a step against it, repeat. Write $\mathbf{g}_t = \nabla_\theta L_{\text{batch}}(\theta_t)$ for the mini-batch gradient at step t, θ for all parameters and η for the learning rate.

SGD. $\theta_{t+1} = \theta_t - \eta\,\mathbf{g}_t$. Simple and memory-free, but it zigzags in narrow valleys and moves slowly along flat directions.

SGD with momentum. Keep a running velocity: $\mathbf{v}_{t+1} = \mu\,\mathbf{v}_t + \mathbf{g}_t$, $\theta_{t+1} = \theta_t - \eta\,\mathbf{v}_{t+1}$, with μ around 0.9. Like a heavy ball rolling downhill, it averages out the zigzag and builds speed along consistent directions.

Adam (Kingma and Ba [19]) keeps two running averages per parameter, of the gradient and of its square:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\,\mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\,\mathbf{g}_t^2,$$
$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \qquad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}, \qquad \theta_t = \theta_{t-1} - \eta\,\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon},$$

where squares, square roots and division are element-wise, $\beta_1 \approx 0.9$ and $\beta_2 \approx 0.999$ are decay rates, and ϵ is a small constant. The first average is momentum. Dividing by the root of the second gives every parameter its own step size, which is the idea of RMSprop that Adam builds on [19]: parameters with consistently large gradients take smaller steps, rarely updated ones take larger steps. The **bias correction** $1/(1-\beta^t)$ fixes the fact that both averages start at zero and would otherwise be too small in the first steps.

AdamW (Loshchilov and Hutter [20]). Weight decay shrinks weights toward zero at every step. For plain SGD this is the same as adding an L2 penalty $\frac{\lambda}{2}\|\theta\|^2$ to the loss. For Adam it is not: an L2 term enters $\mathbf{g}_t$ and is then divided by $\sqrt{\hat{\mathbf{v}}_t}$, so weights with large gradients are regularized *less*. AdamW **decouples** the decay from the adaptive step:

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda\, \theta_{t-1} \right).$$

The authors showed this improves Adam's generalization [20], and AdamW is now the most common optimizer for Transformers and large language models. In PyTorch, `torch.optim.Adam(..., weight_decay=λ)` is the coupled L2 version and `torch.optim.AdamW` is the decoupled one.

Newer optimizers (brief notes). Two recent alternatives are worth knowing by name. **Lion** was found by an automated program search [21]: it keeps only a momentum buffer (so it uses less memory than Adam) and updates every parameter by the **sign** of an interpolated momentum, so all updates have the same magnitude; it needs a smaller learning rate than AdamW. **Muon** [22] applies to the 2-D weight matrices of hidden layers: it takes the momentum update and approximately orthogonalizes it with a few Newton–Schulz iterations, while embeddings, output layers, biases and other scalar or vector parameters are still trained with AdamW. Liu et al. report that, with weight decay and per-parameter update scaling added, Muon reached about twice the compute efficiency of AdamW in their compute-optimal large-language-model training experiments [23]. These are active research topics; AdamW remains the safe default.

Learning-rate schedules

The learning rate is usually the most important hyperparameter. A rate that is fine early in training is often too large later, when the model needs small, careful steps, so the rate is varied over time (Figure 6).

- **Step decay**: multiply the rate by a factor such as 0.1 at fixed epochs. Traditional for CNN training with SGD.
- **Cosine annealing**: $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$, a smooth decay from $\eta_{\max}$ to $\eta_{\min}$ over T steps; SGDR [24] combined it with periodic warm restarts.
- **Warmup**: increase the rate linearly from near zero over the first few hundred or thousand steps. Early in training, Adam's second-moment estimates are noisy and the weights are far from any good region, and a full-size step can be destabilizing.

As a concrete recipe from a published large model, LLaMA was trained with AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$), weight decay 0.1, gradient clipping at 1.0, 2,000 warmup steps, and a cosine schedule ending at 10% of the peak rate [12].

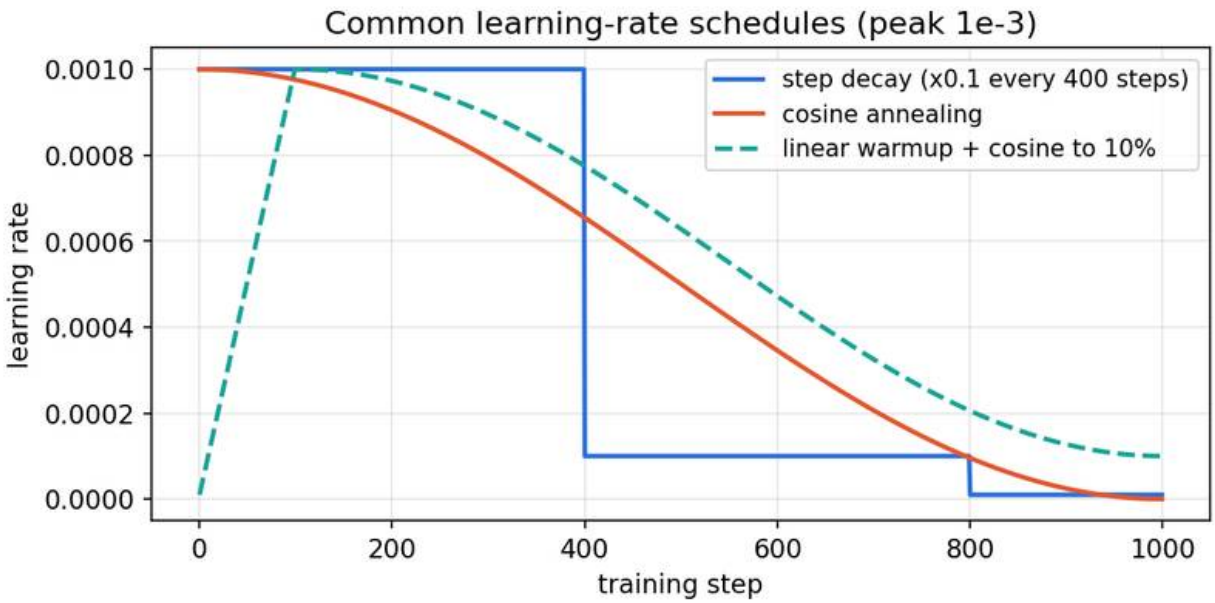


Figure 6 — Three common learning-rate schedules over 1,000 steps with a peak of 1e-3.

Mini-batches, iterations and epochs

A **mini-batch** (often just "batch") is the small random subset of B examples used for one gradient estimate. One **iteration** is one parameter update, using one mini-batch. One **epoch** is one full pass over the training set, which takes $\lceil N/B \rceil$ iterations for N examples. For example, $N = 50{,}000$ and $B = 128$ give 391 iterations per epoch.

The batch size B affects both speed and the result, and bigger is **not** always better:

- **Small batches** give noisy gradient estimates (the variance of the mini-batch mean falls roughly like $1/B$), so the loss curve is jumpy and each epoch takes many steps with poor hardware utilization.
- **Large batches** give stable gradients and run efficiently in parallel on GPUs, but each epoch has fewer updates, and they tend to generalize worse when other settings are kept fixed. Keskar et al. found that large-batch training tends to converge to **sharp minima**, where the loss rises steeply around the solution, while small-batch training, thanks to its gradient noise, tends to find **flat minima**, which generalize better [25].

In practice, choose the largest batch that fits comfortably in memory *and* still trains well, and retune the learning rate when you change B; the two interact strongly.

Gradient clipping

Even in a well-designed network, a rare batch can produce a huge gradient. **Gradient-norm clipping** rescales the whole gradient when its norm exceeds a threshold c:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right),$$

where $\|\mathbf{g}\|$ is the norm of all parameter gradients concatenated. It keeps the direction but limits the step size, which is the standard defense against exploding gradients, especially in recurrent networks and Transformers [3]. A threshold of 1.0 is a common choice.

Putting it together in PyTorch

The snippet trains a small MLP on `make_moons` with mini-batches, AdamW, a cosine schedule and gradient clipping. It runs on a CPU in seconds.

```
import torch, torch.nn as nn
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
X, y = make_moons(n_samples=1000, noise=0.25, random_state=0)
Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.3, random_state=0)
Xtr, Xva = torch.tensor(Xtr, dtype=torch.float32), torch.tensor(Xva, dtype=torch.float32)
ytr, yva = torch.tensor(ytr), torch.tensor(yva)

model = nn.Sequential(
    nn.Linear(2, 64), nn.ReLU(),
    nn.Linear(64, 64), nn.ReLU(),
    nn.Linear(64, 2),               # logits: no softmax here
)
loss_fn = nn.CrossEntropyLoss()     # applies log-softmax internally
opt = torch.optim.AdamW(model.parameters(), lr=1e-2, weight_decay=1e-2)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=200)

for epoch in range(200):
    model.train()
    perm = torch.randperm(len(Xtr))
    for i in range(0, len(Xtr), 64):                # mini-batches of 64
        idx = perm[i:i + 64]
        loss = loss_fn(model(Xtr[idx]), ytr[idx])
        opt.zero_grad()
        loss.backward()                              # backpropagation
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        opt.step()
    sched.step()

model.eval()
```

```
with torch.no_grad():
    acc = (model(Xva).argmax(dim=1) == yva).float().mean().item()
print(f"validation accuracy: {acc:.3f}")   # about 0.94
```

Note `model.train()` and `model.eval()`: they switch layers such as dropout and BatchNorm between their training and inference behavior. Forgetting `eval()` at test time is a classic bug.

Generalization: underfitting, overfitting and regularization

Plain version. The goal is not to do well on the training data; it is to do well on data the model has never seen. A model can fail in two opposite ways: it can be too weak to learn even the training data (underfitting), or it can memorize the training data, noise included, and fail on new data (overfitting).

Capacity and the two failure modes

A model's **capacity** is, informally, how complex a function it can fit; it grows with the number of parameters, depth and width, and training time. As capacity increases, training error keeps falling, while validation error first falls and then rises again once the model starts fitting noise.

The gap between the two is the **generalization gap** [3].

- **Underfitting**: the **training** error itself is high. The model, or the training procedure, cannot capture the pattern.
- **Overfitting**: the training error is low but the validation or test error is much higher.

What to do about underfitting, in order:

1. **Check the code and data first.** A bug, mismatched inputs and labels, wrong preprocessing, or a loss applied to the wrong tensor all look like underfitting.
2. **Increase capacity**: a wider or deeper model, or a better architecture.
3. **Improve optimization**: train longer, tune the learning rate and schedule, switch optimizer, check initialization and normalization.

Making the model **smaller** never fixes underfitting; it lowers capacity further. A smaller model can be a reasonable choice for speed or memory once accuracy is adequate, but that is an efficiency decision, not a cure.

What to do about overfitting: use more or more varied data, add regularization (below), reduce capacity, and stop early. It is also worth checking that the training and validation sets really come

from the same distribution; a mismatch between them looks like overfitting but needs a data fix, not a regularizer.

Reading a loss curve

Plot both the training and the validation loss against iterations or epochs. Four patterns cover most situations:

- **A. Both curves flat from the start.** If the loss is high, the model is not learning at all: suspect a bug, a far too small or large learning rate, or a model/data mismatch. If the loss is already low, the task may be very easy, or each logged point covers so many iterations that the curve's shape is hidden; log more often.
- **B. Training loss keeps falling, validation loss turns upward.** Classic overfitting. Keep the parameters from the point of lowest validation loss (early stopping) and add regularization.
- **C. Both curves fall and level off close together.** Either the model is near its best, which is common when training and validation data are very similar, or both are still high, which is underfitting: train longer or increase capacity.
- **D. Validation loss above training loss, but both still decreasing steadily.** The normal, healthy situation. Keep training and keep the checkpoint with the lowest validation loss.

Regularization tools

Figure 7 shows overfitting and two cures on purpose. A wide MLP (two hidden layers of 256 units) is trained on only 100 noisy two-moons points. Without regularization the training loss falls toward zero while the validation loss reaches its minimum early and then climbs steeply. Weight decay and dropout both slow the climb dramatically.

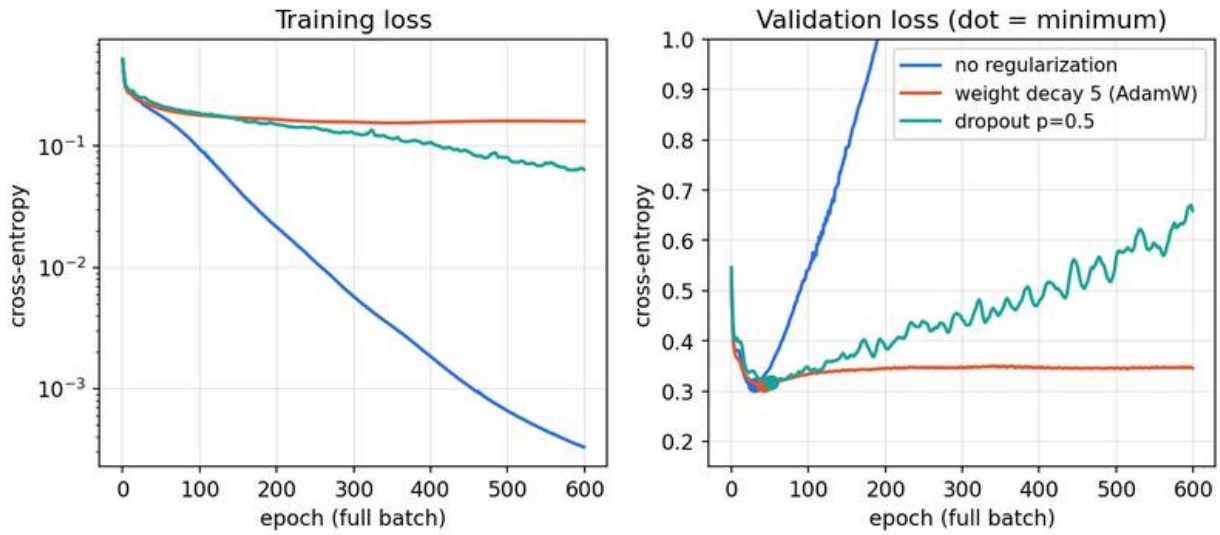


Figure 7 — Overfitting on 100 training points from make_moons (noise 0.35), evaluated on 2,000 validation points. Full-batch AdamW, learning rate 3e-3. Dots mark each run's minimum validation loss, which is where early stopping would keep the model.

Weight decay (L2 regularization). Penalize large weights by adding $\frac{\lambda}{2}\|\theta\|^2$ to the loss, or, with Adam, use decoupled decay (AdamW). Large weights let a network make sharp, wiggly boundaries around individual points; keeping them small favors smoother functions. In Figure 7, decoupled decay with $\lambda = 5$ keeps the validation loss almost flat after its minimum.

Dropout (Srivastava et al. [27]). During training, set each hidden unit to zero with probability p, independently for every example and step, and scale the survivors by $1/(1-p)$ so the expected activation is unchanged ("inverted dropout"). At test time, use all units with no scaling. Each step thus trains a different thinned sub-network; units cannot rely on specific partners and must learn features that are useful on their own, and the full network at test time behaves like an average of many thinned networks [27]. This is why `model.eval()` matters.

Data augmentation. Create new training examples by applying transformations that do not change the label: flips, crops, small rotations, color jitter and noise for images. It is often the single most effective regularizer in vision, because it teaches the invariances directly. It must respect the task: a horizontal flip is fine for cats, not for reading text.

Early stopping. Monitor the validation loss during training and keep the parameters from the epoch where it was lowest; stop when it has not improved for a set number of epochs (the "patience"). It costs nothing, needs no change to the model, and acts as a regularizer by limiting how far the parameters can travel from their initialization [3].

More data and better strategy. The other levers the original notes list are worth repeating: more training data is the most reliable cure for overfitting; a stronger architecture (deeper, or better

structured, so that it converges well) helps when the model underfits; and a better training strategy (optimizer, learning-rate schedule, batch size) helps both.

Residual connections and the road to Transformers

Plain version. If you stack many layers, each must pass the signal on intact as well as improve it, which turns out to be hard. A residual connection gives every block an express lane: the block only has to learn a *correction* that is added to its input.

He et al. observed that adding more plain layers to an already deep network made even the **training** error worse, which is an optimization problem rather than overfitting [28]. Their **residual block** computes

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x}),$$

where $\mathbf{x}$ is the block's input and F is a small stack of layers (in ResNet, convolutions with BatchNorm and ReLU). If the best thing a block can do is nothing, it only has to push F toward zero, which is easy. The backward pass shows why it trains well:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = I + \frac{\partial F}{\partial \mathbf{x}},$$

where I is the identity matrix. The identity term passes the gradient straight through every block, so the product over many blocks no longer has to shrink to zero. With residual connections, He et al. trained networks with up to 152 layers on ImageNet and won several ILSVRC 2015 tasks [28].

The **Transformer** (Vaswani et al. [29]) combines every idea in this tutorial. Each block has two sub-layers, multi-head self-attention (which uses softmax to mix information between positions) and a position-wise MLP, and each sub-layer is wrapped in a residual connection with LayerNorm. The original paper normalized after the addition; many recent models, LLaMA among them, normalize the sub-layer **input** instead (pre-normalization) and use RMSNorm and SwiGLU [12]:

$$\mathbf{h} = \mathbf{x} + \mathrm{Attention}\,(\,\mathrm{Norm}(\mathbf{x})), \qquad \mathbf{y} = \mathbf{h} + \mathrm{FFN}\,(\,\mathrm{Norm}(\mathbf{h})).$$

Since 2017 this design has spread from machine translation to language modeling, vision, speech and more, and it is the basis of today's large language models. The trained recipe is the one from this tutorial too: AdamW, warmup and cosine decay, gradient clipping, and weight decay [12]. The deep-learning track picks up from here with convolutional networks and Transformers for images, starting with image classification.

Modern view

The basic toolkit above has been stable for several years; research continues on why it works and on better versions of each piece. The following reviews are good guides, each from a different angle.

The big picture: LeCun, Bengio and Hinton [2] (Nature, 2015). This short review is still the clearest statement of what deep learning is: representation learning with many layers, where each layer transforms the previous representation into a more abstract one and the features are learned from data with backpropagation rather than engineered. It walks through supervised learning with SGD, the role of ReLU, convolutional networks for images, recurrent networks for sequences, and distributed representations, and it closes by predicting that unsupervised learning and the combination of representation learning with reasoning would matter more in the future. For a textbook treatment of every topic in this tutorial, including the maximum-likelihood view of loss functions, regularization and optimization, the book by Goodfellow, Bengio and Courville [3] remains the standard reference.

Optimization: Sun [26]. Sun's survey asks "when and why can a neural network be successfully trained?" and organizes the answer in three parts: problems with the gradient (explosion, vanishing) and their fixes through careful initialization and normalization; the practical algorithms (SGD, adaptive methods such as Adam, and distributed training) with their convergence theory; and the global picture of the loss landscape, including bad local minima, mode connectivity, the lottery-ticket hypothesis and the infinite-width limit. It is a good bridge from the practice above to theory.

Normalization: Huang et al. [16] (IEEE TPAMI, 2023). This survey gives a unified view of normalization methods from the perspective of optimization, and a taxonomy that splits any method into its normalization area, normalization operation and representation recovery. It covers the history from BatchNorm on, the analyses of why normalization speeds training and helps generalization, and applications in different domains. Read it to see BatchNorm, LayerNorm, group-wise and weight-based normalization as points in one design space.

Activation functions: Dubey, Singh and Chaudhuri [7] (Neurocomputing, 2022). This survey classifies activation functions into logistic sigmoid/tanh-based, ReLU-based, ELU-based and learning-based families, characterizes them by properties such as output range, monotonicity and smoothness, and benchmarks 18 of them across network types and data. Its practical message is that no single activation is best everywhere; the choice interacts with the architecture and the data.

Where things stand (2023–2026). Several convergences are visible in published model recipes:

- **Architecture.** The pre-normalized Transformer block with residual connections is the dominant architecture for language and increasingly for vision; RMSNorm and SwiGLU, as in

LLaMA [12], are common defaults in large language models.

- **Optimization.** AdamW [20] with linear warmup, a cosine (or similar) decay, and gradient clipping is the standard recipe. Optimizers that look beyond per-coordinate scaling are an active area: Lion [21] uses sign updates and less memory, and Muon [22] orthogonalizes matrix updates, with reported compute-efficiency gains at scale [23]. How robust such gains are across tasks and scales is still being studied, which is why AdamW remains the safe choice.
- **Generalization.** The sharp-versus-flat-minima view [25] is a useful intuition rather than a complete theory, and why heavily over-parameterized networks generalize as well as they do is still an open question; Sun's survey [26] summarizes the main lines of attack.

Open problems. A reliable theory of generalization for deep networks; principled rules for choosing the learning rate, batch size and schedule together, especially when scaling models up; understanding exactly how normalization, residual connections and adaptive optimizers interact; and making training stable at very large scale without the many small tricks that current recipes rely on.

Key takeaways

- A perceptron is a linear score followed by a step function and trained by a mistake-driven rule; logistic regression uses a sigmoid probability and cross-entropy. Similar structure, different methods, and only the smooth one can be stacked and trained by gradients.
- Hidden layers with nonlinear activations are what make networks more than linear models. One wide hidden layer can in principle approximate any continuous function, but that is an existence result; depth makes useful functions easier to represent and learn. There is no fixed layer count that defines "deep".
- ReLU-family activations (ReLU, Leaky ReLU, GELU, SiLU) keep gradients alive; sigmoid and softmax belong mostly at the output; SwiGLU is the modern feed-forward choice in Transformers. BatchNorm normalizes over the batch, LayerNorm and RMSNorm over the features of each sample.
- Match the head to the task: linear plus MSE for regression, sigmoid plus binary cross-entropy for binary or multi-label, softmax plus cross-entropy for multi-class. In each case the output gradient is prediction minus target.
- Backpropagation is the chain rule applied from the loss backward: $\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$ and $\partial L/\partial W_\ell = \delta_\ell \mathbf{a}_{\ell-1}^\top$. Check hand-written gradients numerically.
- Practical training: He or Xavier initialization, AdamW with warmup and a decaying schedule, gradient clipping, and a batch size chosen with the learning rate, not "as large as possible".
- Underfitting means high training error and calls for more capacity or better optimization, never a smaller model; overfitting calls for more data, augmentation, weight decay, dropout and early stopping.
- Residual connections make very deep networks trainable, and the Transformer is these ideas combined.

Exercises

1. **Perceptron by hand.** Run the perceptron rule on the AND data with $\eta = 1$, $\mathbf{w} = \mathbf{0}$, $b = 0$, visiting the points in the order $(0,0), (0,1), (1,0), (1,1)$. Write down $\mathbf{w}$ and b after each update until an epoch passes with no mistakes. Then explain in one sentence why the same procedure on XOR never stops.

   Hint

   With the strict test $z > 0$, the first point $(0,0)$ has $z = 0$, so it is predicted 0 and is correct. The first mistake is $(1,1)$, which gives $\mathbf{w} = (1,1)$, $b = 1$. Keep going: each mistake on a 0-labeled point subtracts $\mathbf{x}$ and 1, each mistake on the 1-labeled point adds them. With this order there are ten updates over five epochs, and the sixth epoch is error-free with $\mathbf{w} = (2,1)$, $b = -2$, that is, the line $2x_1 + x_2 = 2$. A different visiting order gives a different, equally valid line. For XOR, no $(\mathbf{w}, b)$ classifies all four points correctly, so some point is always wrong and an update always happens.

2. **How fast do sigmoid gradients vanish?** Show that $\sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 1/4$. Then consider a chain of 20 sigmoid layers with all weights equal to 1. What is the largest factor by which the gradient can be multiplied on its way back? Repeat the reasoning for ReLU with active units.

   Hint

   $s(1-s)$ with $s \in (0,1)$ is a downward parabola with maximum $1/4$ at $s = 1/2$. Twenty factors of at most $1/4$ give at most $4^{-20} \approx 9 \times 10^{-13}$. For an active ReLU unit, $f' = 1$, so the activations contribute no shrinkage at all; only the weights do, which is why initialization matters.

3. **The prediction-minus-target gradient.** Using $\partial p_k / \partial z_j = p_k(\delta_{kj} - p_j)$, prove that for $L = -\sum_k y_k \log p_k$ with a one-hot $\mathbf{y}$, $\partial L / \partial z_j = p_j - y_j$. Then show the same result for the sigmoid with binary cross-entropy.

   Hint

   $\partial L / \partial z_j = -\sum_k (y_k / p_k)\, p_k(\delta_{kj} - p_j) = -\sum_k y_k \delta_{kj} + p_j \sum_k y_k = -y_j + p_j$, since $\sum_k y_k = 1$. For the sigmoid, $\partial \ell / \partial \hat{p} = -y/\hat{p} + (1-y)/(1-\hat{p})$ and $\partial \hat{p} / \partial z = \hat{p}(1-\hat{p})$; multiply and simplify to $\hat{p} - y$.

4. **Extend the gradient check.** In the NumPy example, check every entry of `W2`, `b1` and `b2` (not just one entry of `W1`) against finite differences, and report the maximum relative error $|a-n| / \max(|a|, |n|, 10^{-12})$. Then deliberately introduce a bug, for example drop the `* (Z1 > 0)` factor, and see how large the error becomes.

   Hint

Loop over `np.ndindex(param.shape)`, perturb one entry by $\pm 10^{-6}$, and recompute the loss. A correct implementation gives relative errors around 10^{-7} or smaller (a few entries next to ReLU kinks can be larger). Removing the ReLU derivative makes errors of order 1 appear in `W1` and `b1`, while `W2` and `b2` stay correct, because the bug is only in the backward path below the hidden layer.

5. **Iterations, epochs and batch size.** A dataset has 60,000 training images. (a) How many iterations make one epoch with $B = 32$, and with $B = 1{,}024$? (b) If you train both for 20 epochs, how many parameter updates does each make? (c) Based on the batch-size discussion, give two reasons the $B = 1{,}024$ run might reach a worse validation accuracy with the same learning rate, and one change that could help.

Hint

(a) $\lceil 60000/32 \rceil = 1875$ and $\lceil 60000/1024 \rceil = 59$. (b) 37,500 versus 1,180 updates. (c) Far fewer updates in the same number of epochs, and less gradient noise, which the sharp-minima findings associate with worse generalization. Tuning the learning rate for the new batch size (typically larger, with warmup) and training for more epochs are the usual adjustments.

6. **Diagnose the curve.** A colleague's network has a training loss that drops quickly at first and then stays flat at a high value; the validation loss tracks it closely. They propose halving the number of hidden units "to make it converge faster". What is happening, what would you check first, and what would you suggest instead?

Hint

Both losses are high and close together: this is underfitting (pattern C with high values), not overfitting. First check for bugs: labels aligned with inputs, the loss applied to logits, data normalized, the learning rate sensible. Then increase capacity or improve optimization (wider or deeper model, a learning-rate schedule, longer training). Shrinking the model lowers capacity and will make underfitting worse.

References

1. Shyandram, "機器學習及類神經網路筆記 (Notes on machine learning and neural networks)," Medium post (in Traditional Chinese), 2024; updated 2026. link
2. Y. LeCun, Y. Bengio and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. doi
3. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. online book
4. F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, pp. 386–408, 1958. doi
5. G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, vol. 2, pp. 303–314, 1989. doi

6. D. E. Rumelhart, G. E. Hinton and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986. doi
7. S. R. Dubey, S. K. Singh and B. B. Chaudhuri, "Activation functions in deep learning: A comprehensive survey and benchmark," *Neurocomputing*, vol. 503, pp. 92–108, 2022. arXiv
8. D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," arXiv:1606.08415, 2016. arXiv
9. S. Elfwing, E. Uchibe and K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *Neural Networks*, vol. 107, pp. 3–11, 2018. arXiv
10. P. Ramachandran, B. Zoph and Q. V. Le, "Searching for activation functions," arXiv:1710.05941, 2017. arXiv
11. N. Shazeer, "GLU variants improve Transformer," arXiv:2002.05202, 2020. arXiv
12. H. Touvron, T. Lavril, G. Izacard et al., "LLaMA: Open and efficient foundation language models," arXiv:2302.13971, 2023. arXiv
13. S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. International Conference on Machine Learning* (*ICML*), PMLR 37, pp. 448–456, 2015. PMLR
14. J. L. Ba, J. R. Kiros and G. E. Hinton, "Layer normalization," arXiv:1607.06450, 2016. arXiv
15. B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2019. arXiv
16. L. Huang, J. Qin, Y. Zhou, F. Zhu, L. Liu and L. Shao, "Normalization techniques in training DNNs: Methodology, analysis and application," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 8, pp. 10173–10196, 2023. arXiv
17. X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. International Conference on Artificial Intelligence and Statistics* (*AISTATS*), PMLR 9, pp. 249–256, 2010. PMLR
18. K. He, X. Zhang, S. Ren and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE International Conference on Computer Vision* (*ICCV*), pp. 1026–1034, 2015. arXiv
19. D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations* (*ICLR*), 2015. arXiv
20. I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. International Conference on Learning Representations* (*ICLR*), 2019. arXiv
21. X. Chen, C. Liang, D. Huang, E. Real, K. Wang, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu and Q. V. Le, "Symbolic discovery of optimization algorithms," in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2023. arXiv
22. K. Jordan, "Muon: An optimizer for hidden layers in neural networks," Medium post, December 2024. link
23. J. Liu, J. Su, X. Yao et al., "Muon is scalable for LLM training," arXiv:2502.16982, 2025. arXiv
24. I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. International Conference on Learning Representations* (*ICLR*), 2017. arXiv
25. N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in *Proc. International Conference on Learning Representations* (*ICLR*), 2017. arXiv
26. R. Sun, "Optimization for deep learning: Theory and algorithms," arXiv:1912.08957, 2019.

arXiv
27. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014. JMLR
28. K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016. arXiv
29. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv

ML 03 • Convolution, Image Filtering and CNNs

From blur and Sobel kernels to learned filters: convolution versus correlation, border handling, padding, stride, dilation, pooling, receptive fields, backpropagation through a convolution, and the classic CNN architectures.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-03-convolutional-networks/

Notebook: https://github.com/Shyandram/cv-blog/blob/6b02ce5/notebooks/image-processing/convolution.ipynb

What you'll learn

- What a kernel is, how it slides over an image, and why the same operation can blur, sharpen or find edges.
- The exact difference between **correlation** and **convolution**, and which one NumPy, SciPy, OpenCV and PyTorch actually compute.
- How to write 2-D filtering from scratch in NumPy, reproduce it in OpenCV, SciPy and PyTorch, and handle image borders deliberately.
- How a convolutional layer generalizes a fixed filter: channels, weight sharing, padding, stride, dilation, 1×1 convolutions and pooling.
- How receptive fields grow with depth, and how gradients flow backward through a convolution.
- What the classic architectures (LeNet, AlexNet, VGG, GoogLeNet, ResNet, U-Net, MobileNet, EfficientNet, ConvNeXt) each contributed, and where CNNs stand in 2026.

The big picture

An image is a grid of numbers (see Images as arrays). Many of the most useful things we can do to it are **local**: whether a pixel sits on an edge, inside a smooth region or in a noisy patch depends mostly on its neighbors. **Convolution** is the operation that formalizes "look at a small neighborhood, combine it with fixed weights, and repeat everywhere." It is the workhorse of classical image processing, and it is also the building block of **convolutional neural networks** (**CNNs**), which learn their weights from data instead of having them designed by hand.

> **Explain like I'm five**
>
> Imagine you have a small cardboard window with nine holes, and behind each hole you write a number. You place the window on a picture, multiply each visible dot of color by the number behind its

hole, add everything up, and write the answer on a new sheet. Then you slide the window one step and do it again, until you have covered the whole picture. If all nine numbers are equal, the new picture is a blurry average. If the numbers on the left are negative and those on the right are positive, you get big answers only where the picture changes from dark to light: an edge finder. A CNN is a machine that slides thousands of such windows and **learns** which numbers to write behind the holes.

This tutorial has two halves. The first half (filtering) is the classical toolbox and needs only NumPy. The second half (CNNs) builds on ML 02, where you met layers, losses and backpropagation. If you want more depth on fixed filters afterward, IP 03 covers spatial filtering and IP 04 explains the frequency-domain view through the convolution theorem. For the deep-learning side, DLCV 02 studies modern classification backbones in detail, ML 03A turns to sequences, and ML 04 introduces attention, the main alternative to convolution. Standard textbook treatments are chapter 9 of Goodfellow et al. [1] and chapter 10 of Prince [10].

Part I • Filtering with a fixed kernel

The sliding weighted sum

Plain version. Every output pixel is a weighted sum of the input pixels around the same position. The small grid of weights is called a **kernel** (also *filter* or *mask*).

Precise version. Let I be a grayscale image, with $I(x, y)$ the intensity at column x and row y. Let K be a kernel of size $(2a + 1) \times (2b + 1)$, indexed from $-a$ to a vertically and $-b$ to b horizontally, so that $K(0, 0)$ is its center. The filtered image G is

$$G(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i, j)\, I(x + j,\ y + i).$$

Here i indexes kernel rows and j kernel columns, $G(x, y)$ is the output at the same position, and the double sum visits every tap of the kernel once. For a 3×3 kernel, $a = b = 1$ and each output pixel costs nine multiplications and eight additions.

Figure 1 works through one output value by hand. The orange window covers the top-left 3×3 neighborhood of the padded input. Multiply element-wise by the kernel and add: $1 \cdot 1 + 0 \cdot 2 + (-1) \cdot 0 + 2 \cdot 0 + 0 \cdot 1 + (-2) \cdot 3 + 1 \cdot 2 + 0 \cdot 0 + (-1) \cdot 1 = -4$. That number becomes G at the window's center. Sliding the window one step to the right and repeating fills in the next output.

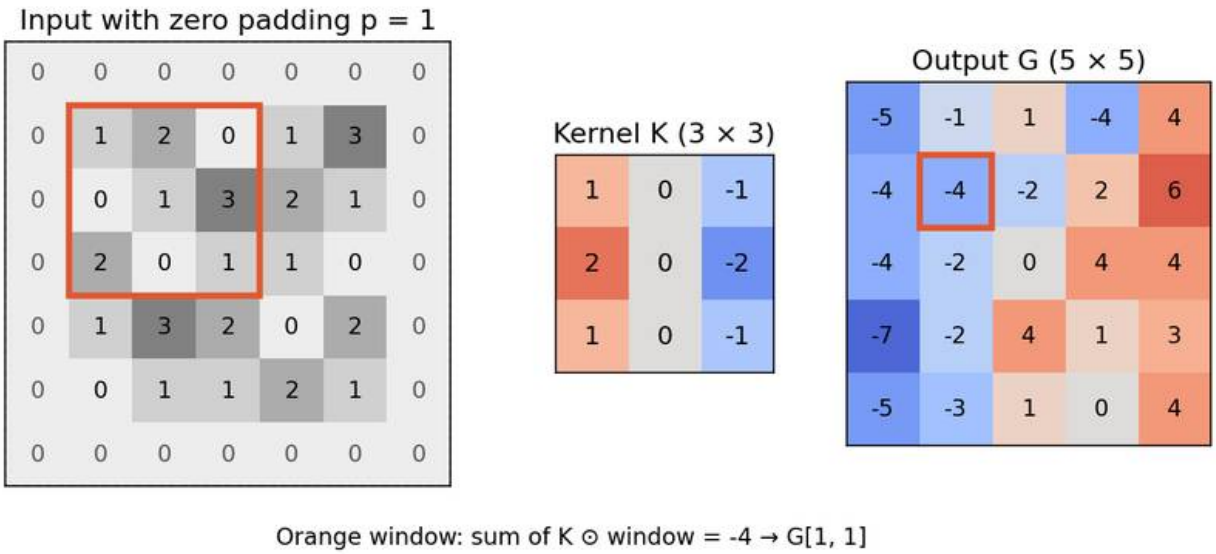


Figure 1 — One step of the sliding weighted sum. The orange 3 × 3 window, multiplied element-wise by K and summed, gives −4 at the matching output position. Zero padding (gray) lets the output keep the input size.

Two observations already matter for everything that follows. First, the same nine weights are used at every position: the operation is **shift-invariant** (also called *space-invariant*). Second, the operation is **linear**: filtering $\alpha I_1 + \beta I_2$ gives $\alpha G_1 + \beta G_2$. Linear shift-invariant (LSI) operations are exactly the ones a convolution can express, which is why convolution is so central in signal processing.

Correlation versus convolution

Plain version. The formula above is technically **cross-correlation**. True **convolution** does the same thing with the kernel rotated by 180 degrees. For symmetric kernels the two agree; for asymmetric ones they differ.

Precise version. Mathematical convolution, written $K * I$, is

$$(K * I)(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i, j)\, I(x - j,\ y - i) = \sum_{i,j} K(-i, -j)\, I(x + j,\ y + i).$$

The second form shows that convolution equals correlation with the flipped kernel $K(-i, -j)$. A clean way to see the difference is to filter a single bright pixel (an **impulse**). Correlation leaves a copy of the kernel rotated by 180 degrees; convolution leaves an exact copy of the kernel. That copy is the **impulse response**, and it is why signal processing prefers convolution: the output for

any image is the sum of shifted, scaled impulse responses.

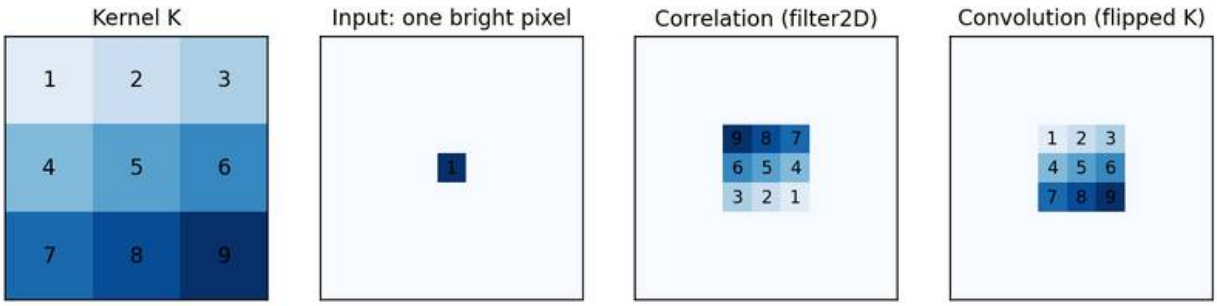


Figure 2 — Filtering an impulse. Correlation (what cv2.filter2D computes) stamps a 180°-rotated kernel; convolution stamps the kernel itself.

Convolution has algebraic properties that correlation lacks. It is **commutative** ($K * I = I * K$) and **associative** ($K_2 * (K_1 * I) = (K_2 * K_1) * I$), so two filters applied in a row equal one bigger filter. Applying a 3×3 box blur twice, for instance, is a single 5×5 kernel. Associativity also explains **separable** kernels: a $k \times k$ Gaussian equals a $k \times 1$ column filter followed by a $1 \times k$ row filter, cutting the cost per pixel from k^2 to $2k$ operations. The **convolution theorem** says convolution in space is multiplication in the Fourier domain; IP 04 develops that view.

Which libraries compute which?

Library call	Computes	Default border
`cv2.filter2D`	correlation [11]	reflect-101 (`BORDER_DEFAULT`)
`scipy.ndimage.correlate` / `convolve`	correlation / convolution	`reflect` (edge sample repeated)
`scipy.signal.convolve2d`	convolution	zero fill
`torch.nn.functional.conv2d`, `nn.Conv2d`	correlation [31]	zero padding

For hand-designed asymmetric kernels (Sobel, emboss, motion blur) the flip changes the sign or direction of the result, so flip deliberately when you need true convolution, for example with `cv2.flip(K, -1)`. For learned kernels the distinction disappears: a network that "should" use the flipped kernel simply learns it. That is why deep-learning frameworks call their correlation "convolution" without harm.

Blur, edges and sharpening

Plain version. Positive weights that sum to one average neighbors and blur. Weights that sum to zero cancel flat regions and keep only changes, so they find edges. Adding a bit of "the image minus its blur" back to the image sharpens it.

A **box blur** gives every neighbor equal weight; the 3×3 version has nine entries of $1/9$. A **Gaussian blur** samples

$$K(i,j) = \frac{1}{Z}\exp\!\left(-\frac{i^2+j^2}{2\sigma^2}\right),$$

where σ (in pixels) sets the blur width and Z normalizes the weights to sum to one. Nearby pixels count more than distant ones, which avoids the blocky artifacts of a box. A practical rule is to use a kernel radius of about 3σ.

The **sum of the weights** tells you what happens in a flat region where every pixel equals c: the output is $c\sum K$. A blur with sum one preserves brightness; a kernel with sum two doubles it (and saturates an 8-bit image); a kernel with sum zero outputs zero.

The **Sobel** kernels estimate horizontal and vertical derivatives while smoothing in the orthogonal direction:

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}\begin{bmatrix} -1 & 0 & 1 \end{bmatrix}, \qquad S_y = S_x^{\top}.$$

S_x responds to intensity changes along x, so it highlights **vertical** boundaries; S_y highlights **horizontal** ones. The outputs G_x, G_y are signed: dark-to-light and light-to-dark edges have opposite signs. The **gradient magnitude** $\sqrt{G_x^2+G_y^2}$ is a nonnegative edge strength, and $\operatorname{atan2}(G_y, G_x)$ is the edge orientation. The factorization shows that Sobel is separable: a $[1,2,1]^{\top}$ smoothing column times a $[-1,0,1]$ central difference.

The **Laplacian** kernel $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ approximates $\partial^2 I/\partial x^2 + \partial^2 I/\partial y^2$; it responds to thin lines and to both sides of an edge. **Unsharp masking** sharpens with

$$I_{\text{sharp}} = I + \lambda\,(I - G_\sigma * I),$$

where $G_\sigma * I$ is a Gaussian-blurred copy, $I - G_\sigma * I$ is the high-frequency detail, and $\lambda > 0$ controls strength. Large λ amplifies noise and creates bright and dark halos along edges.

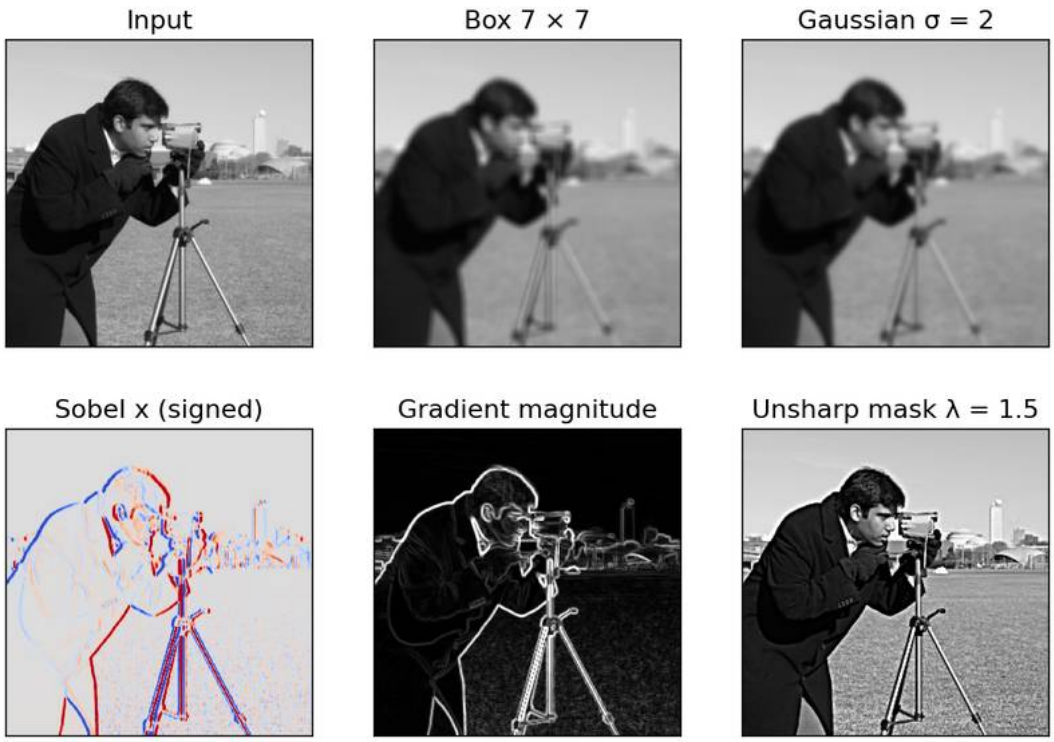


Figure 3 — The same image through six kernels. Box and Gaussian blur suppress fine detail; Sobel x is signed (red = dark-to-light, blue = light-to-dark); the gradient magnitude outlines objects; unsharp masking adds back amplified detail.

Try it: the kernel playground

Edit the kernel cells below and watch the output update. Start from Sobel x, then transpose it to detect horizontal edges instead of vertical ones. Make every weight of the blur preset larger and notice that flat regions brighten, exactly as the sum-of-weights rule predicts. Try the Laplacian and emboss presets and predict their kernel sums before you look.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

The slider below compares a synthetic test image (a disc, a square, a checkerboard and stripes) with a 5×5 Gaussian blur, $\sigma = 1.4$. The fine stripes and the checkerboard disappear first because a blur is a **low-pass filter**: it removes fast intensity variations while leaving slow ones.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Code: from scratch, then OpenCV, SciPy and PyTorch

The loop below is the whole algorithm. Instead of looping over every pixel (slow in Python), it loops over the kernel taps and adds one shifted copy of the padded image per tap, which NumPy vectorizes. The script then checks that four libraries agree once you match the definition (correlation versus convolution) and the border rule. It needs `numpy`, `scipy`, `opencv-python` and `torch`.

```
import numpy as np
import cv2
import torch
import torch.nn.functional as F
from scipy import ndimage, signal

def correlate2d(img, K, mode="edge"):
    """Cross-correlation of a 2-D float image with an odd-sized kernel.
    mode is any np.pad mode: "constant" (zeros), "edge", "reflect", "wrap"."""
    kh, kw = K.shape
    ph, pw = kh // 2, kw // 2
    padded = np.pad(img, ((ph, ph), (pw, pw)), mode=mode)
    H, W = img.shape
    out = np.zeros((H, W), dtype=np.float64)
    for i in range(kh):              # loop over kernel taps, not pixels:
        for j in range(kw):          # each step adds one shifted copy of the image
            out += K[i, j] * padded[i:i + H, j:j + W]
    return out

def convolve2d(img, K, mode="edge"):
    return correlate2d(img, K[::-1, ::-1], mode)     # convolution = correlation with
     ↪  flipped K

rng = np.random.default_rng(0)
img = rng.random((32, 40))
K = rng.standard_normal((3, 5))                      # deliberately asymmetric

ours_corr = correlate2d(img, K, "reflect")
ours_conv = convolve2d(img, K, "reflect")
# OpenCV: filter2D computes correlation; BORDER_REFLECT_101 is np.pad's "reflect"
cv_corr = cv2.filter2D(img, cv2.CV_64F, K, borderType=cv2.BORDER_REFLECT_101)
# SciPy: ndimage.correlate / ndimage.convolve with mode="mirror" (= reflect-101)
sp_corr = ndimage.correlate(img, K, mode="mirror")
sp_conv = ndimage.convolve(img, K, mode="mirror")
# PyTorch: conv2d is correlation; pad first with the same rule
t = torch.tensor(img)[None, None]
t = F.pad(t, (2, 2, 1, 1), mode="reflect")
pt_corr = F.conv2d(t, torch.tensor(K)[None, None])[0, 0].numpy()

for name, a in [("OpenCV", cv_corr), ("SciPy", sp_corr), ("PyTorch", pt_corr)]:
    print(f"{name:8s} correlation matches:", np.allclose(ours_corr, a))
print("SciPy    convolution matches:", np.allclose(ours_conv, sp_conv))
print("conv == corr for this kernel?", np.allclose(ours_conv, ours_corr))
# zero padding: our convolution equals signal.convolve2d with mode="same"
same = signal.convolve2d(img, K, mode="same", boundary="fill")
```

```
print("zero-padded 'same' convolution matches:", np.allclose(convolve2d(img, K, "constant"),
 ↪  same))
```

All checks print `True` except `conv == corr`, which is `False` because the random kernel is asymmetric. Note the naming trap: NumPy's `"reflect"` padding (`dcb|abcd|cba`, edge sample not repeated) is OpenCV's `BORDER_REFLECT_101` and SciPy's `"mirror"`, while SciPy's `"reflect"` repeats the edge sample (`dcba|abcd|dcba`).

The second snippet shows the two bugs that cause most beginner frustration with real images: integer saturation and the kernel-sum rule.

```
import numpy as np
import cv2
from skimage import data

img_u8 = data.camera()                                  # 512 x 512 uint8 test image
img = img_u8.astype(np.float32) / 255.0
sobel_x = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], np.float32)

gx = cv2.filter2D(img, -1, sobel_x)                     # float in, float out: negatives kept
gy = cv2.filter2D(img, -1, sobel_x.T)
mag = np.hypot(gx, gy)
print("float Sobel x range:", gx.min().round(3), gx.max().round(3))

gx_u8 = cv2.filter2D(img_u8, -1, sobel_x)               # uint8 in, uint8 out: saturated
print("uint8 Sobel x range:", gx_u8.min(), gx_u8.max(),
      "| pixels clipped to 0:", f"{(gx_u8 == 0).mean():.0%}")

blur = cv2.GaussianBlur(img, (5, 5), 1.4)               # 5 x 5 Gaussian, sigma 1.4
print("mean brightness kept by blur:", np.isclose(img.mean(), blur.mean(), atol=1e-3))
K2 = 2 * np.ones((3, 3), np.float32) / 9                # weights sum to 2
print("mean after sum-2 kernel:", round(float(cv2.filter2D(img, -1, K2).mean() /
 ↪  img.mean()), 3))
```

In floating point the Sobel response ranges from about −3.4 to 3.3. With `uint8` input and output, every negative response is clipped to zero (more than half the pixels here), so all light-to-dark edges vanish. Always filter in `float32` and convert back only for display.

Border handling

Plain version. Near the edge of the image, part of the kernel hangs outside. You must decide what values to pretend are there.

The common choices are: **zero** (constant) padding, which pretends the world outside is black; **replicate**, which repeats the edge pixel; **reflect**, which mirrors the image at its border; and **wrap**, which treats the image as periodic, as the discrete Fourier transform implicitly does. OpenCV's

default, `BORDER_REFLECT_101`, mirrors without repeating the edge pixel [11]. PyTorch's `nn.Conv2d` uses zero padding unless you set `padding_mode` to `'reflect'`, `'replicate'` or `'circular'` [31].

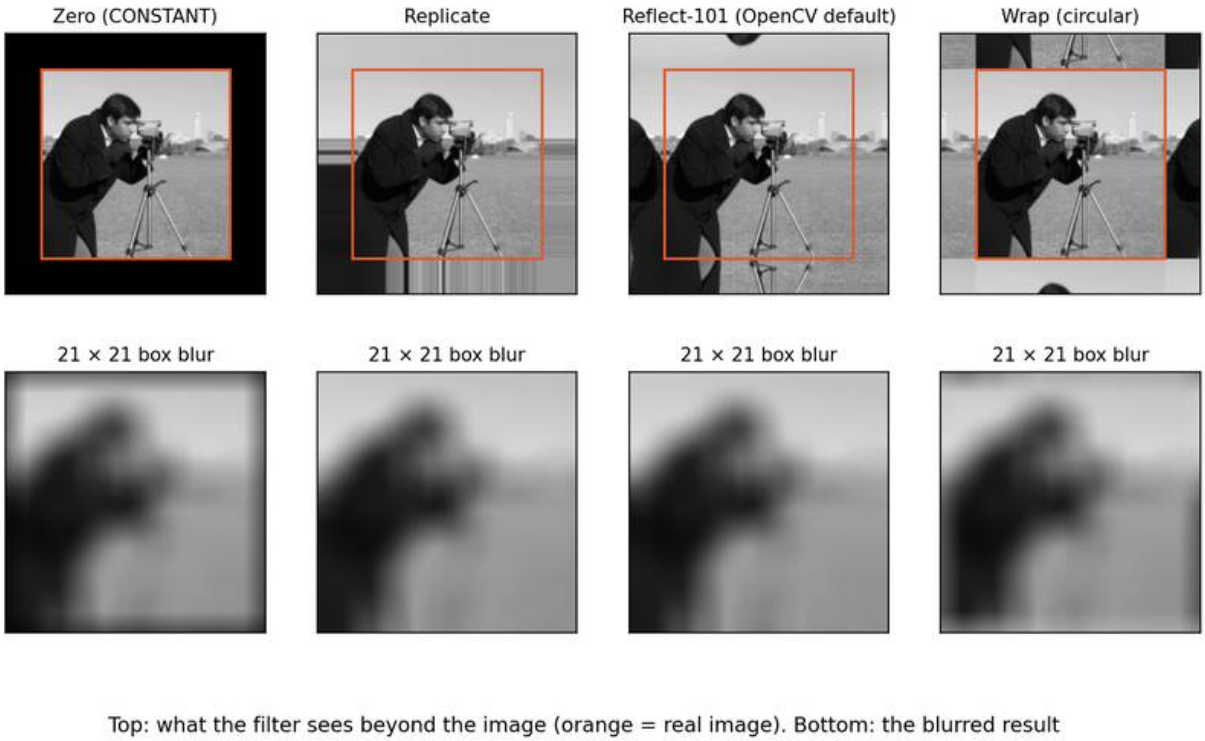


Figure 4 — Four border rules and their effect on a large 21 × 21 box blur. Zero padding darkens every border; wrap blends opposite edges; replicate and reflect keep flat borders flat.

Zero padding darkens the borders of a blurred image because black pixels are averaged in. Wrap mixes the top of the image into the bottom. Replicate and reflect are usually the least visible choices for photographs. With small kernels the difference touches only a thin frame, but in a deep CNN every layer adds its own frame: after many zero-padded layers, a large fraction of the feature map has "seen" the artificial border, which lets networks infer absolute position from padding.

Part II • Convolutional neural networks

From a fixed filter to a learned layer

Plain version. A convolutional layer is a bank of filters whose weights are learned by gradient descent. Each filter looks at all input channels at once and produces one output channel, called a **feature map**.

Precise version. Let the input be $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ (channels, height, width) and the weights $K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$ with bias $b \in \mathbb{R}^{C_{\text{out}}}$. A stride-one layer with zero padding p computes

$$Y_{o,u,v} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} K_{o,c,i,j}\, X_{c,\, u+i-p,\, v+j-p}.$$

o indexes output channels (filters), c input channels, (u, v) output positions and (i, j) kernel taps; positions outside the image read the padding value. A nonlinearity such as ReLU is then applied element-wise. An RGB filter has three $k \times k$ slices, one per color channel, and their responses are **summed** into a single output number per position: a filter does not produce three separate images.

Two structural assumptions are built into this formula:

- **Local connectivity**: each output depends only on a $k \times k$ neighborhood, not on the whole image.
- **Weight sharing** (parameter sharing): the same K is used at every position (u, v).

Together they are the CNN's **inductive bias**, an assumption about which functions are likely. Its payoff is efficiency. A layer with $C_{\text{in}} = 3$, $C_{\text{out}} = 32$ and $k = 3$ has $32 \times (3 \cdot 9 + 1) = 896$ parameters, whatever the image size. A fully connected layer from a flattened $224 \times 224 \times 3$ image to just 32 numbers needs $150{,}528 \times 32 \approx 4.8$ million weights, and it would have to learn the same edge detector separately at every position.

Goodfellow et al. [1] summarize the motivation as sparse interactions, parameter sharing and equivariance. Figure 5 shows what a tiny CNN learns in its first layer when trained on the 8×8 digits dataset used in the classifier code later on. The learned 3×3 filters look like oriented derivative and blob detectors: the network rediscovered Sobel-like kernels because they are useful, not because anyone told it to. Zeiler and Fergus [30] showed the same phenomenon at scale by visualizing the filters and feature maps of large ImageNet networks.

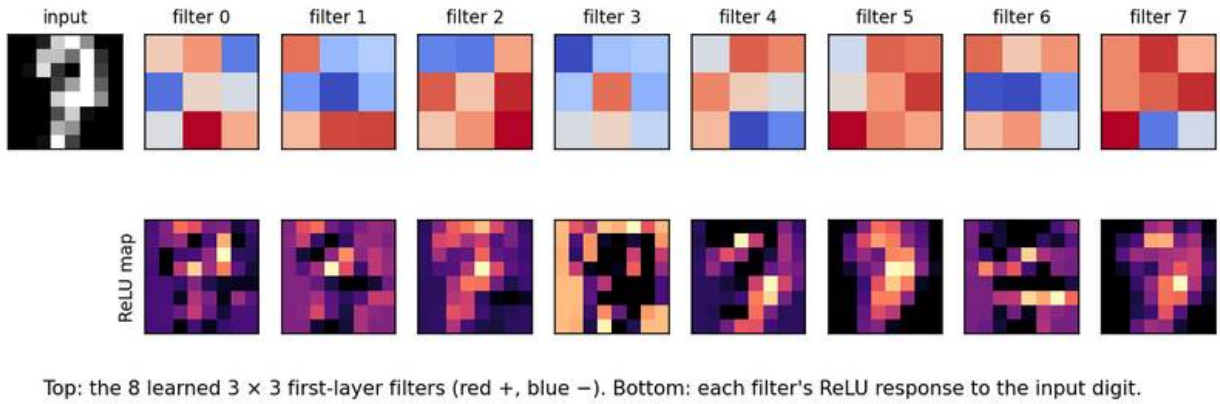


Figure 5 — First-layer filters learned on 8×8 digits (top) and their ReLU responses to one test digit (bottom). Many look like oriented edge or stroke detectors.

Padding, stride and dilation

Plain version. Padding controls what happens at the border and whether the output keeps its size. Stride skips positions to shrink the output. Dilation spreads the kernel taps apart to see farther without adding weights.

For input size n (height or width), kernel size k, padding p on each side, stride s and dilation d, the output size is [12]

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

$d(k-1)+1$ is the **effective kernel size**: dilation d inserts $d-1$ gaps between taps. Three padding conventions have names: **valid** ($p = 0$, output shrinks by $k-1$), **same** ($p = (k-1)/2$ for odd k and $s = 1$, output keeps its size) and **full** ($p = k-1$, every partial overlap counted, output grows by $k-1$). Stride two roughly halves each spatial dimension and is a common alternative to pooling for downsampling.

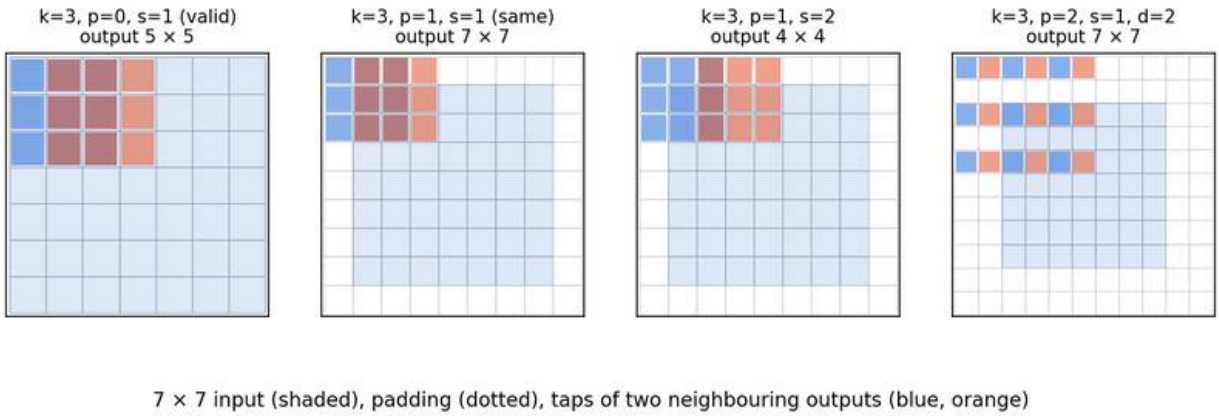


Figure 6 — Taps of two neighboring outputs (blue and orange; overlap appears brown) under four settings. Stride moves the window by s; dilation spaces the taps d apart.

Dilated (atrous) convolutions were popularized for dense prediction by Yu and Koltun [18]: stacking layers with dilations 1, 2, 4, ... grows the field of view exponentially while keeping full resolution. A single dilated layer sees only a sparse lattice of inputs, so dilations should be mixed with care to avoid "gridding" artifacts. A **transposed convolution** runs the forward map of a strided convolution backward to upsample; it reappears in the backpropagation section below. The convolution arithmetic guide by Dumoulin and Visin [12] has diagrams for every combination of these settings.

The snippet checks the formula against PyTorch, counts parameters, shows that a 1×1 convolution is a per-pixel linear layer, and measures translation equivariance:

```
import torch
from torch import nn

def out_size(n, k, p=0, s=1, d=1):
    return (n + 2 * p - d * (k - 1) - 1) // s + 1

x = torch.randn(1, 3, 64, 64)                          # one RGB 64 x 64 image
for k, p, s, d in [(3, 1, 1, 1), (5, 2, 2, 1), (3, 2, 1, 2), (1, 0, 1, 1)]:
    conv = nn.Conv2d(3, 16, k, stride=s, padding=p, dilation=d)
    y = conv(x)
    n_params = sum(q.numel() for q in conv.parameters())
    assert y.shape[-1] == out_size(64, k, p, s, d)
    assert n_params == 16 * (3 * k * k + 1)
    print(f"k={k} p={p} s={s} d={d}: output {tuple(y.shape[1:])}, parameters {n_params}")

# A 1x1 convolution is the same linear layer applied at every pixel
conv1 = nn.Conv2d(16, 4, kernel_size=1)
lin = nn.Linear(16, 4)
lin.weight.data = conv1.weight.data[:, :, 0, 0].clone()
lin.bias.data = conv1.bias.data.clone()
z = torch.randn(2, 16, 8, 8)
same = torch.allclose(conv1(z), lin(z.permute(0, 2, 3, 1)).permute(0, 3, 1, 2), atol=1e-6)
print("1x1 conv == per-pixel Linear:", same)

# Translation equivariance away from borders (stride 1, no padding effects in the centre)
conv = nn.Conv2d(1, 4, 3, padding=1, bias=False)
img = torch.zeros(1, 1, 32, 32); img[..., 10:14, 8:12] = 1
shift = lambda t: torch.roll(t, shifts=(3, 5), dims=(-2, -1))
with torch.no_grad():
    a, b = conv(shift(img)), shift(conv(img))
print("shift then conv == conv then shift:", torch.allclose(a, b, atol=1e-6))
conv2 = nn.Conv2d(1, 4, 3, padding=1, stride=2, bias=False)
with torch.no_grad():
    y0, y1 = conv2(img), conv2(torch.roll(img, 1, dims=-1))   # shift by one pixel
print("stride 2, 1-pixel shift: max output change =", round((y0 - y1).abs().max().item(), 3))
```

It prints output shapes $16 \times 64 \times 64$, $16 \times 32 \times 32$, $16 \times 64 \times 64$ and $16 \times 64 \times 64$ with 448, 1216, 448 and 64 parameters, confirms the 1×1 and equivariance checks, and shows that a one-pixel shift changes a stride-two output by a clearly nonzero amount; the exact value depends on the randomly initialized weights.

Channels and 1×1 convolutions

Plain version. Channels are the "what" dimension of a feature map; height and width are the "where". A 1×1 convolution mixes channels at each pixel without looking at neighbors.

A 1×1 convolution with C_{in} inputs and C_{out} outputs multiplies the channel vector at every pixel by the same $C_{\text{out}} \times C_{\text{in}}$ matrix, as the snippet verified. Lin et al.'s Network in Network [17] used

this idea as a tiny per-pixel MLP and introduced **global average pooling** as a classifier head. GoogLeNet [16] used 1×1 layers to reduce channels before expensive 3×3 and 5×5 branches, and ResNet's bottleneck block [5] wraps a 3×3 layer between a 1×1 reduction and a 1×1 expansion. In every case the motivation is the same: spatial filtering costs k^2 times more than channel mixing, so do the cheap part with 1×1 layers and keep the expensive part narrow.

Pooling and downsampling

Plain version. Pooling summarizes each small block of a feature map by a single number, usually the maximum or the average. It shrinks the map and makes the representation less sensitive to small shifts.

Max pooling with a 2×2 window and stride two keeps the largest value in each block: "was this feature present anywhere here?". **Average pooling** keeps the mean. **Global average pooling** averages each channel over all positions, producing one number per channel regardless of input size, which is why modern classifiers can accept different image sizes. Pooling has no learnable parameters. Its gradient is simple: max pooling routes the whole upstream gradient to the winning input of each window; average pooling spreads it equally over the window.

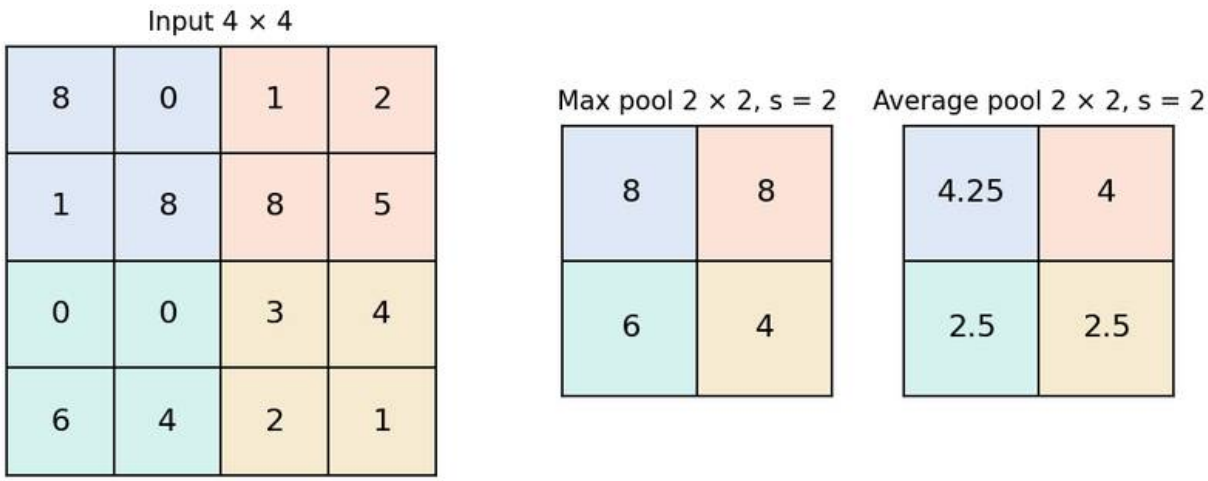


Figure 7 — Max and average pooling with a 2×2 window and stride 2. Each colored block becomes one number.

Downsampling, by pooling or by strided convolution, saves computation and enlarges later receptive fields, but it is also **subsampling** without a proper low-pass filter. High frequencies alias, and the output can change noticeably when the input moves by a single pixel, as the snippet above measured. Zhang [20] showed that inserting a small blur before subsampling ("anti-aliased" or

"blur pooling") makes standard CNNs measurably more shift-invariant: a neat case of a classical signal-processing rule fixing a deep network.

Equivariance is not invariance

Plain version. Shift the input, and a convolutional feature map shifts with it. That is not the same as the final prediction staying unchanged.

Let T_Δ shift an image by Δ pixels. Away from the borders, a stride-one convolution is **translation-equivariant**: $f(T_\Delta X) = T_\Delta f(X)$. A classifier wants **invariance**: $g(T_\Delta X) = g(X)$. CNNs get approximate invariance by stacking equivariant layers and then pooling, especially global average pooling, which discards position. Padding, strides, cropping and the finite image break exact equivariance. Rotation and scale equivariance are **not** built in; data augmentation or specially designed group-equivariant layers must supply them.

The distinction matters in practice. A detector must be equivariant: moving a person should move the predicted box. A classifier should be invariant: moving the person should not change the label. The same backbone serves both, with different heads.

Receptive field

Plain version. Early layers see small pieces of the image; deeper layers combine those pieces and see larger regions.

The **receptive field** of a unit is the set of input pixels that can influence it. Let r_ℓ be its width after layer ℓ, and j_ℓ the **jump**, the distance in input pixels between adjacent units of layer ℓ. Starting from $r_0 = j_0 = 1$, a layer with kernel size k_ℓ, dilation d_ℓ and stride s_ℓ gives

$$r_\ell = r_{\ell-1} + (d_\ell(k_\ell - 1))\, j_{\ell-1}, \qquad j_\ell = s_\ell\, j_{\ell-1}.$$

Three stacked stride-one 3×3 layers see 7×7 pixels, the same area as one 7×7 layer but with $3 \cdot 9C^2 = 27C^2$ instead of $49C^2$ weights and three nonlinearities instead of one; this was the central design argument of VGG [15]. A stride-two layer doubles the jump, so every later layer grows the field twice as fast.

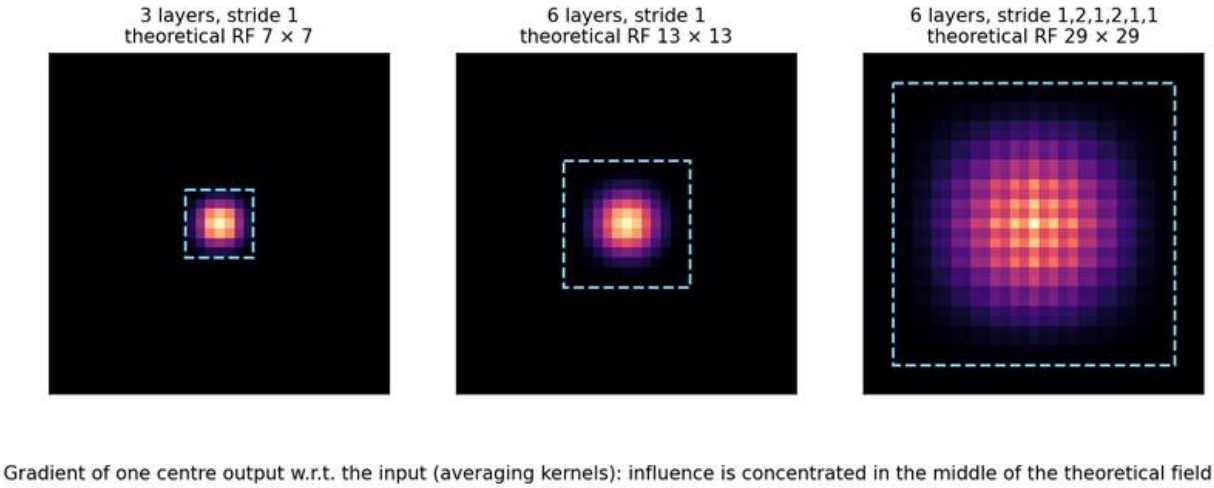


Figure 8 — Theoretical receptive field (dashed) versus actual influence, measured as the input gradient of one central output through stacks of 3 × 3 averaging layers. Influence is Gaussian-like and concentrated in the center; strides add a checkerboard pattern.

The theoretical field says which pixels **could** matter. Luo et al. [3] measured the **effective receptive field** via input gradients, as in Figure 8, and found that it is roughly Gaussian and occupies only a fraction of the theoretical field, growing like the square root of depth for uniform layers. Two practical lessons follow: a network may need more context than its theoretical field suggests, and downsampling, dilation and large kernels are ways to enlarge the effective field without unbounded depth.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Backpropagation through a convolution

Plain version. A convolution is linear, so its backward pass is also a convolution: the gradient with respect to the kernel correlates the input with the upstream gradient, and the gradient with respect to the input convolves the upstream gradient with the kernel.

Precise version. Take one channel, no padding, stride one: $Y_{u,v} = \sum_{i,j} K_{i,j} X_{u+i,v+j}$. Let L be the loss and $G_{u,v} = \partial L / \partial Y_{u,v}$ the **upstream gradient** delivered by backpropagation, as in ML 02. Each weight $K_{i,j}$ is used at every output position, so by the chain rule its gradient sums over them:

$$\frac{\partial L}{\partial K_{i,j}} = \sum_{u,v} G_{u,v}\, X_{u+i,v+j}.$$

That is a valid correlation of the input X with G, which is why weight sharing does not complicate training: shared weights simply accumulate gradients from every location. Each input pixel $X_{m,n}$ contributes to every output whose window covers it, at tap $(i, j) = (m - u, n - v)$:

$$\frac{\partial L}{\partial X_{m,n}} = \sum_{i,j} G_{m-i,\,n-j}\, K_{i,j},$$

with G taken as zero outside its range. This is a **full convolution** of G with K, equivalently a correlation of the zero-padded G with the flipped kernel. It is exactly what a **transposed convolution** computes, which is how transposed convolutions got their name. Bias gradients are just $\sum_{u,v} G_{u,v}$ per output channel. With multiple channels, the input gradient also sums over output channels, and the kernel gradient also sums over the batch.

```
import torch
import torch.nn.functional as F

torch.manual_seed(0)
X = torch.randn(1, 1, 6, 7, requires_grad=True)     # input
K = torch.randn(1, 1, 3, 3, requires_grad=True)     # kernel
Y = F.conv2d(X, K)                                   # 'valid' correlation: 4 x 5
G = torch.randn_like(Y)                              # pretend upstream gradient dL/dY
(Y * G).sum().backward()                             # so that dL/dY = G exactly

# 1) dL/dK: correlate the input with the upstream gradient
dK = F.conv2d(X.detach(), G)
# 2) dL/dX: 'full' convolution of G with K = correlation of zero-padded G with flipped K
dX = F.conv2d(F.pad(G, (2, 2, 2, 2)), K.detach().flip(-1, -2))
# 3) the same thing is what a transposed convolution computes
dX_t = F.conv_transpose2d(G, K.detach())

print("dL/dK matches autograd:", torch.allclose(dK, K.grad, atol=1e-5))
print("dL/dX matches autograd:", torch.allclose(dX, X.grad, atol=1e-5))
print("conv_transpose2d gives the same:", torch.allclose(dX_t, X.grad, atol=1e-5))
```

All three lines print `True`. The trick `(Y * G).sum().backward()` makes autograd use a chosen upstream gradient G, a handy way to test any hand-derived backward pass.

Classic architectures in brief

Plain version. Every famous CNN is a stack of convolutions, nonlinearities and downsampling. What changed over the years was depth, how to keep deep stacks trainable, and how to spend computation wisely.

Model (year)	Key idea	What it taught
LeNet-5 (1998) [13]	conv → subsample → conv → subsample → fully connected, trained end to end	Learned convolutional features beat hand-designed ones for digits
AlexNet (2012) [14]	Deeper and wider, ReLU, dropout, data augmentation, GPU training	Large labeled data plus GPUs made CNNs win ImageNet
VGG (2014) [15]	Only 3×3 convolutions, 16–19 layers	Depth with small kernels is simple and effective
GoogLeNet / Inception (2014) [16]	Parallel 1×1, 3×3, 5×5 branches with 1×1 reductions	Multi-scale features at controlled cost
ResNet (2015) [5]	Residual connections $y = x + F(x)$, BatchNorm	Very deep networks (100+ layers) become trainable
U-Net (2015) [4]	Encoder–decoder with skip connections at each resolution	Dense, pixel-aligned outputs from few images
MobileNetV2 (2018) [7]	Depthwise-separable convolutions, inverted residuals, linear bottlenecks	Accurate models for phones
EfficientNet (2019) [8]	Compound scaling of depth, width and resolution	Scale dimensions together, not one at a time
ConvNeXt (2022) [9]	7×7 depthwise kernels, LayerNorm, inverted bottlenecks, transformer-era training	A modernized pure CNN competes with vision transformers

The pattern from LeNet to ResNet is a steady increase in depth. AlexNet [14] showed that a deep CNN trained on ImageNet with GPUs, ReLU and dropout could beat traditional pipelines by a wide margin. VGG [15] argued for uniform stacks of 3×3 layers, as computed in the receptive-field section. GoogLeNet [16] made width cheap with 1×1 reductions. Then depth hit an optimization wall: plain networks with more layers had **higher training error**, which is not overfitting but a failure to optimize. ResNet [5] fixed it with identity shortcuts. DLCV 02 follows this line further, through training recipes, transfer learning and evaluation.

Residual blocks and normalization

Plain version. Instead of asking a block to compute the whole output, ask it to compute a correction to its input.

A residual block computes $y = x + F_\theta(x)$, where F_θ is typically two or three convolutions with normalization and ReLU. Its Jacobian is $\partial y/\partial x = I + \partial F_\theta/\partial x$: even if F_θ contributes little, the identity path carries the signal forward and the gradient backward. If $F_\theta \approx 0$ is the best a block can do, it can learn that easily, so adding blocks no longer hurts optimization. When the shape changes (more channels, stride two), the shortcut becomes a projection $P(x)$, usually a strided 1×1 convolution.

Batch normalization [19] standardizes each channel using the batch mean and variance during training, then applies a learned scale and shift; at evaluation it uses running averages. It makes training faster and less sensitive to initialization, but it misbehaves with very small or correlated

batches, where **GroupNorm** [6] (statistics over groups of channels within each sample) is more reliable. Always call `model.eval()` before validation so BatchNorm uses its running statistics.

```
import torch
from torch import nn

count = lambda m: sum(p.numel() for p in m.parameters())

C = 64
dense = nn.Conv2d(C, C, 3, padding=1, bias=False)
separable = nn.Sequential(
    nn.Conv2d(C, C, 3, padding=1, groups=C, bias=False),   # depthwise: one 3x3 per channel
    nn.Conv2d(C, C, 1, bias=False))                        # pointwise: mix channels
print("dense:", count(dense), "| depthwise+pointwise:", count(separable),
      "| ratio:", round(count(separable) / count(dense), 4), "vs 1/C + 1/9 =",
      round(1 / C + 1 / 9, 4))

class BasicBlock(nn.Module):
    """ResNet-style block: y = relu(x + F(x)), with a 1x1 projection when shapes change."""
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.f = nn.Sequential(
            nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False), nn.BatchNorm2d(c_out),
            ↪  nn.ReLU(),
            nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False), nn.BatchNorm2d(c_out))
        self.proj = (nn.Identity() if stride == 1 and c_in == c_out else
                     nn.Sequential(nn.Conv2d(c_in, c_out, 1, stride, bias=False),
                                   nn.BatchNorm2d(c_out)))
    def forward(self, x):
        return torch.relu(self.proj(x) + self.f(x))

x = torch.randn(2, 64, 32, 32)
print("same shape :", tuple(BasicBlock(64, 64)(x).shape))
print("downsample :", tuple(BasicBlock(64, 128, stride=2)(x).shape))
```

The output reports 36,864 weights for the dense layer against 4,672 for the separable pair, a ratio of 0.1267 that matches the formula below, and block outputs of shape $64 \times 32 \times 32$ and $128 \times 16 \times 16$.

Efficient CNNs separate spatial filtering from channel mixing

Plain version. Looking around (spatial filtering) and combining features (channel mixing) need not happen in one expensive operation.

A dense $k \times k$ convolution has $k^2 C_{\text{in}} C_{\text{out}}$ weights. A **depthwise** convolution filters each channel separately ($k^2 C_{\text{in}}$ weights, `groups=C` in PyTorch); a following **pointwise** 1×1 convolution mixes channels ($C_{\text{in}} C_{\text{out}}$ weights). With $C_{\text{in}} = C_{\text{out}} = C$,

$$\frac{\text{depthwise} + \text{pointwise}}{\text{dense}} = \frac{k^2C + C^2}{k^2C^2} = \frac{1}{C} + \frac{1}{k^2}.$$

For $k = 3$ and large C this approaches $1/9$.

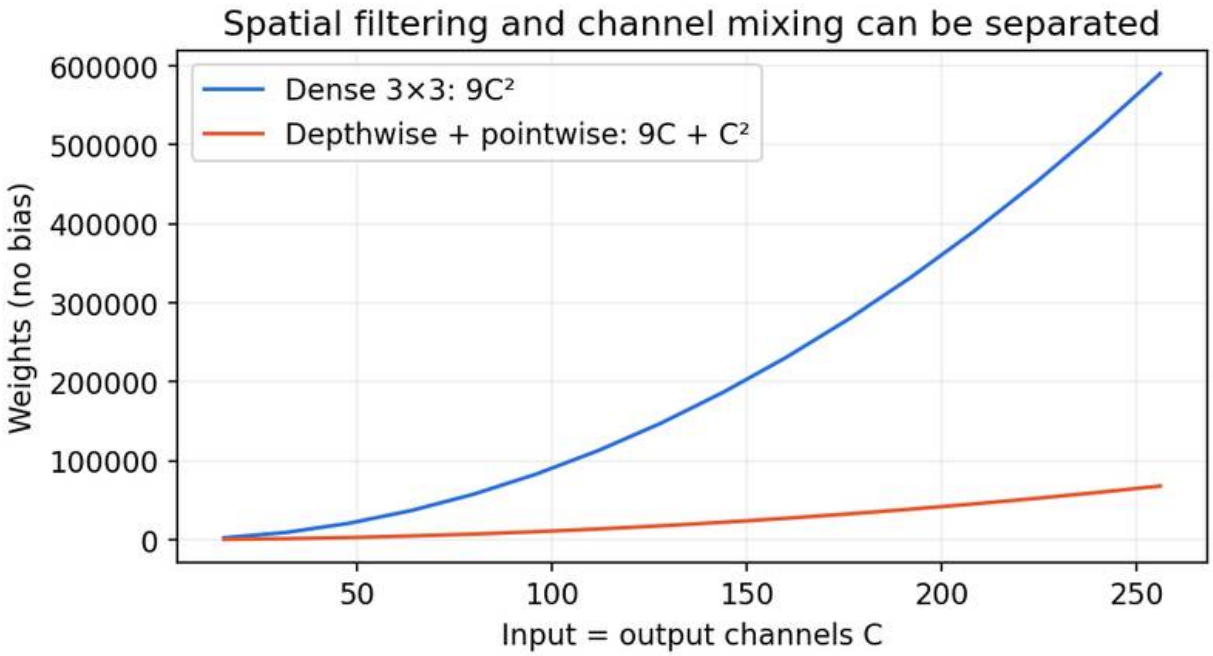


Figure 9 — Analytical weight counts for equal input and output channels. Fewer weights do not automatically mean lower latency: depthwise layers have low arithmetic intensity and may run slowly on some hardware.

MobileNetV2 [7] builds blocks that expand channels with a 1×1 layer, filter with a depthwise 3×3, then project back with a linear (no ReLU) 1×1 bottleneck, with a residual connection between the narrow ends. EfficientNet [8] scales depth, width and input resolution together with a single compound coefficient. ConvNeXt [9] started from a ResNet and applied, one at a time, design choices associated with vision transformers: patchify stem, 7×7 depthwise kernels, inverted bottlenecks, fewer activations, LayerNorm, and longer training with heavy augmentation. The result matched Swin Transformers of similar size, evidence that much of the transformer advantage came from training recipes and macro-design rather than from attention alone.

Code: a complete small-image classifier

This example needs `torch`, `numpy` and `scikit-learn`. It uses the bundled 8×8 digits dataset, so no download or GPU is needed. A stratified split gives separate training, validation and test sets: validation picks the best epoch, and the test set is evaluated once at the end. It is a teaching baseline, not an ImageNet recipe.

```
import copy
import numpy as np
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)
d = load_digits()
X = torch.tensor(d.images[:, None] / 16.0, dtype=torch.float32)
y = torch.tensor(d.target, dtype=torch.long)
train, test = train_test_split(np.arange(len(y)), test_size=.2,
                               stratify=d.target, random_state=0)
train, val = train_test_split(train, test_size=.2,
                              stratify=d.target[train], random_state=1)
model = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),                       # B × 32 × 4 × 4
    nn.Flatten(), nn.Linear(32 * 4 * 4, 10),
)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
best_loss, best_state = float("inf"), None
for epoch in range(12):
    model.train()
    order = torch.tensor(train)[torch.randperm(len(train))]
    for idx in order.split(64):
        loss = nn.functional.cross_entropy(model(X[idx]), y[idx])
        opt.zero_grad()
        loss.backward()
        opt.step()
    model.eval()
    with torch.no_grad():
        val_loss = nn.functional.cross_entropy(model(X[val]), y[val]).item()
    if val_loss < best_loss:
        best_loss = val_loss
        best_state = copy.deepcopy(model.state_dict())
model.load_state_dict(best_state)
model.eval()
with torch.no_grad():
    accuracy = (model(X[test]).argmax(1) == y[test]).float().mean().item()
print("parameters:", sum(p.numel() for p in model.parameters()))
print("best validation loss:", round(best_loss, 4))
print("held-out accuracy:", round(accuracy, 4))
```

On our CPU run it reports 9,930 parameters and about 96% held-out accuracy (exact numbers vary slightly across PyTorch versions). Cross-entropy receives **logits**, with no softmax in front. The head flattens the $32 \times 4 \times 4$ map rather than global-average-pooling it, because on tiny digits the position of strokes carries information; for variable-size images, adaptive or global pooling

gives a fixed-size head.

Inspect the errors, not just the score. Which digit pairs get confused? Does shifting every test digit by one pixel hurt? Compare against a parameter-matched MLP and a classical SVM, as in IP 13, before crediting every gain to convolution.

Modern view

Survey literature. Several reviews map the CNN landscape, each with a different emphasis. Gu et al. [23] (*Pattern Recognition*, 2018) organize CNN advances by component: convolution variants, pooling, activations, loss functions, regularization and optimization, followed by applications. Khan et al. [22] (*Artificial Intelligence Review*, 2020) propose a taxonomy of architectures by their main innovation: spatial exploitation (kernel sizes), depth, multi-path designs (shortcuts, Inception branches), width, feature-map exploitation (channel attention), and attention-based designs. Li et al. [21] (*IEEE TNNLS*, 2022) cover the history from LeNet onward, compare one-, two- and multi-dimensional convolutions, and survey lightweight networks and applications with rules of thumb for choosing hyperparameters. Alzubaidi et al. [2] (*Journal of Big Data*, 2021) give a broad, application-oriented review of deep learning centered on CNN architectures, with discussion of data scarcity, transfer learning and hardware. These surveys predate ConvNeXt, so read them as maps of the 2012–2020 era.

CNNs after transformers (2022–2026). Vision transformers (ML 04) prompted a re-examination of convolution, and the main finding is that the gap was smaller than early comparisons suggested:

- **Modernized CNNs.** ConvNeXt [9] showed that a carefully modernized ResNet matches Swin. ConvNeXt V2 [24] (CVPR 2023) added masked-autoencoder pretraining with sparse convolutions and a global response normalization layer, bringing self-supervised pretraining (see ML 05) to pure CNNs.
- **Large kernels.** RepLKNet [26] (CVPR 2022) used depthwise kernels as large as 31×31, with re-parameterized small kernels to ease optimization, to enlarge the effective receptive field. UniRepLKNet [27] (CVPR 2024) extended large-kernel design to audio, video, point clouds and time series.
- **Dynamic and deformable convolution.** InternImage [25] (CVPR 2023) scaled deformable convolution, whose sampling locations are predicted from the input, into a large foundation backbone for detection and segmentation.
- **Scale.** Smith et al. [29] (2023) pretrained NFNet CNNs on the large JFT-4B image dataset and found that their performance followed similar compute scaling trends to vision transformers under comparable budgets, arguing that compute and data, more than architecture, dominate at scale.
- **Edge devices.** MobileNetV4 [28] (ECCV 2024) designed blocks and a neural architecture

search for good latency across CPUs, GPUs, DSPs and accelerators, an acknowledgment that parameter counts alone do not predict speed.

Where CNNs fit in 2026. Convolution remains a default for latency-constrained and on-device inference, for dense prediction at high resolution (restoration, segmentation, medical imaging, see DLCV 04), for small datasets where the locality bias helps, and as the stem or early stages of hybrid CNN–transformer models. Large vision foundation models are mostly transformer-based, but they frequently contain convolutions in their patch embeddings, upsampling decoders and diffusion U-Nets.

Open problems. Exact shift equivariance under downsampling (aliasing [20]); building rotation and scale symmetry in without heavy cost; understanding what the effective receptive field really needs to be for a task [3]; and fair comparison across architectures, which requires matching data, augmentation, resolution, schedule and compute before attributing a difference to convolution versus attention.

Key takeaways

- A kernel slides over the image and computes a weighted sum at every position. Libraries usually compute **correlation**; true convolution flips the kernel, which matters only for asymmetric, hand-designed kernels.
- The sum of a kernel's weights sets its response to flat regions: sum 1 blurs while keeping brightness; sum 0 detects changes. Filter in floating point, and choose the border rule on purpose.
- A convolutional layer is a bank of learned filters with **local connectivity** and **weight sharing**. Its parameter count, $C_{\text{out}}(C_{\text{in}}k^2 + 1)$, does not depend on image size.
- Padding, stride and dilation set the output size, $\lfloor (n + 2p - d(k-1) - 1)/s \rfloor + 1$. 1×1 convolutions mix channels; pooling and strides downsample but can alias.
- Convolution is shift-**equivariant**; invariance comes from pooling. Receptive fields grow with depth and stride, but the effective field is much smaller than the theoretical one.
- Backpropagation through a convolution is itself a convolution: kernel gradients correlate input with upstream gradient, and input gradients are a transposed convolution.
- Architecture history is a story of depth (LeNet → VGG → ResNet), efficient channel use (1×1, depthwise-separable) and better training. Modernized CNNs remain competitive with transformers when budgets are matched.

Exercises

1. **Design and verify.** Write a 3×3 kernel that detects only horizontal edges, and another that detects only diagonal edges running from top-left to bottom-right. Verify both in the kernel

playground or with `cv2.filter2D` on a synthetic image containing a horizontal, a vertical and a diagonal line.

Hint

A horizontal edge is a change along y, so use $S_y = S_x^\top$ (rows $-1,-2,-1$ / $0,0,0$ / $1,2,1$). For a diagonal edge, put the derivative across the diagonal, for example $\begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix}$. Its weights sum to zero, so flat regions give zero.

2. **Kernel sums.** Why should a blur kernel's weights sum to one? Predict, then check, what happens to a mid-gray `uint8` image filtered with a 3×3 kernel whose weights all equal $2/9$, and what changes if you filter in `float32` instead.

Hint

A flat region of value c maps to $c \sum K$. With sum two, a gray of 128 becomes 256, which saturates at 255 in `uint8`; every brighter-than-mid-gray region turns white. In `float32` the values double without clipping, and you can rescale afterward.

3. **Composition.** Show that applying a 3×3 box blur twice equals one 5×5 kernel, and write that kernel. Verify numerically away from the borders, and explain why the equality can fail at the borders.

Hint

By associativity, the combined kernel is the box convolved with itself: the outer product of $[1, 2, 3, 2, 1]$ with itself, divided by 81. At the border, the first pass already invented padding values, and the second pass pads that result; a single 5×5 pass pads the original image instead, so the two differ in a frame of width two.

4. **Layer arithmetic.** For a 64×64 RGB input and a layer with 16 filters, $k = 5$, $p = 2$, $s = 2$: give the output shape and the number of parameters. Then compute (r, j) after three 3×3 layers with strides 1, 2, 1, and compare with all strides equal to one.

Hint

$\lfloor (64 + 4 - 4 - 1)/2 \rfloor + 1 = 32$, so the output is $16 \times 32 \times 32$ with $16(3 \cdot 25 + 1) = 1216$ parameters. Receptive field: $(3, 1) \to (5, 2) \to (9, 2)$. All strides one gives $(7, 1)$.

5. **Gradient by hand.** For a 1×4 input $X = [x_1, x_2, x_3, x_4]$ and a kernel $K = [k_1, k_2]$ with valid correlation, write Y, then derive $\partial L/\partial K$ and $\partial L/\partial X$ in terms of $G = \partial L/\partial Y$. Check your formulas with the autograd trick from the backpropagation snippet.

Hint

$Y = [k_1x_1 + k_2x_2,\ k_1x_2 + k_2x_3,\ k_1x_3 + k_2x_4]$. Then $\partial L/\partial k_1 = G_1x_1 + G_2x_2 + G_3x_3$ and $\partial L/\partial k_2 = G_1x_2 + G_2x_3 + G_3x_4$ (a correlation of X with G). $\partial L/\partial X = [G_1k_1,\ G_1k_2 + G_2k_1,\ G_2k_2 + G_3k_1,\ G_3k_2]$, the full convolution of G with K.

6. **Ablation on digits.** In the classifier, replace `Flatten` + `Linear(512, 10)` by global average pooling + `Linear(32, 10)`. Compare parameter counts, validation loss and test accuracy, then evaluate both models on test digits shifted by one pixel. Explain the trade-off you observe.

Hint

`nn.AdaptiveAvgPool2d(1)` then `nn.Flatten()`. Pooling removes most head parameters and position information, which can help shift robustness but hurt on 8×8 digits where stroke position matters. Keep everything else fixed, select on validation only, and report both accuracy and parameter count.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 9. Book
2. L. Alzubaidi et al., "Review of deep learning: concepts, CNN architectures, challenges, applications, future directions," *Journal of Big Data*, 2021. Review
3. W. Luo, Y. Li, R. Urtasun and R. Zemel, "Understanding the Effective Receptive Field in Deep Convolutional Neural Networks," NeurIPS, 2016. Paper
4. O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI, 2015. Paper
5. K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, 2016. Paper
6. Y. Wu and K. He, "Group Normalization," ECCV, 2018. Paper
7. M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018. Paper
8. M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML, 2019. Paper
9. Z. Liu et al., "A ConvNet for the 2020s," CVPR, 2022. Paper
10. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023, ch. 10. Book and notebooks
11. OpenCV, "Image Filtering" (`filter2D`, border types), documentation, version 4.x. Documentation
12. V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," arXiv:1603.07285, 2016. arXiv
13. Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, 1998. DOI
14. A. Krizhevsky, I. Sutskever and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," NeurIPS, 2012. Paper
15. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," ICLR, 2015. arXiv

16. C. Szegedy et al., “Going Deeper with Convolutions,” CVPR, 2015. arXiv
17. M. Lin, Q. Chen and S. Yan, “Network In Network,” ICLR, 2014. arXiv
18. F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” ICLR, 2016. arXiv
19. S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015. arXiv
20. R. Zhang, “Making Convolutional Networks Shift-Invariant Again,” ICML, 2019. arXiv
21. Z. Li, F. Liu, W. Yang, S. Peng and J. Zhou, “A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects,” *IEEE Transactions on Neural Networks and Learning Systems*, 2022. DOI
22. A. Khan, A. Sohail, U. Zahoora and A. S. Qureshi, “A Survey of the Recent Architectures of Deep Convolutional Neural Networks,” *Artificial Intelligence Review*, 2020. arXiv
23. J. Gu et al., “Recent Advances in Convolutional Neural Networks,” *Pattern Recognition*, 2018. arXiv
24. S. Woo et al., “ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders,” CVPR, 2023. arXiv
25. W. Wang et al., “InternImage: Exploring Large-Scale Vision Foundation Models with Deformable Convolutions,” CVPR, 2023. arXiv
26. X. Ding et al., “Scaling Up Your Kernels to 31x31: Revisiting Large Kernel Design in CNNs” (RepLKNet), CVPR, 2022. arXiv
27. X. Ding et al., “UniRepLKNet: A Universal Perception Large-Kernel ConvNet for Audio, Video, Point Cloud, Time-Series and Image Recognition,” CVPR, 2024. arXiv
28. D. Qin et al., “MobileNetV4: Universal Models for the Mobile Ecosystem,” ECCV, 2024. arXiv
29. S. L. Smith, A. Brock, L. Berrada and S. De, “ConvNets Match Vision Transformers at Scale,” arXiv:2310.16764, 2023. arXiv
30. M. D. Zeiler and R. Fergus, “Visualizing and Understanding Convolutional Networks,” ECCV, 2014. arXiv
31. PyTorch, `torch.nn.Conv2d` documentation. Documentation

ML 03A • Recurrent and Recursive Networks

Learn from sequences and trees: shared recurrent state, backpropagation through time, vanishing and exploding gradients, LSTM and GRU gates, bidirectional and encoder–decoder models, attention as the bridge to Transformers, recursive networks, and their uses in vision.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-03a-recurrent-recursive-networks/

What you'll learn

- How a recurrent network reuses one set of weights at every time step, so it can read sequences of any length.
- How backpropagation through time (BPTT) trains it, and why gradients over long spans vanish or explode.
- How LSTM and GRU gates build a protected memory path, written out equation by equation and checked against PyTorch.
- How bidirectional, stacked and encoder–decoder (sequence-to-sequence) models are assembled, and how attention removed the encoder bottleneck and led to the Transformer.
- How recursive networks compose representations over trees instead of chains.
- Where recurrence is used in vision (captioning, video, spatiotemporal forecasting) and how Mamba and Mamba-2 use selective state spaces, parallel scans and fixed-size streaming memory.

The big picture

ML 03 used one assumption about images: nearby pixels are related, and the same pattern can appear anywhere, so the same filter is reused across **space**. Sequences invite the same trick across **time**. A video is a sequence of frames, a caption is a sequence of words, a pen stroke is a sequence of positions. A **recurrent neural network** (**RNN**) reads such a sequence one element at a time, carries a summary called the **hidden state** from step to step, and uses the same weights at every step [1]. A **recursive neural network** generalizes the idea from a chain to a tree: it combines parts into wholes by repeatedly applying one shared composition rule.

> **Explain like I'm five**
>
> When you listen to a story, you do not remember every word. You keep a little picture in your head of what has happened so far, and each new sentence updates that picture. A recurrent network does exactly this: it has a small notebook (the hidden state), reads one word or one video frame at a time, and rewrites the notebook after each one using the same rules every time. An LSTM is a notebook with

an eraser and a pen that the network learns to use carefully, so important notes from the beginning are not smudged away by the end. A recursive network is like building with LEGO: snap two small pieces into a bigger piece, then snap that into an even bigger piece, using the same kind of snap each time.

Why study RNNs when Transformers dominate large-scale language and vision? Three reasons. First, every concept in modern sequence modeling — teacher forcing, encoder–decoder, attention, masking, causal generation — was developed on RNNs, and the Transformer is easiest to understand as the answer to RNN limitations. Second, recurrence is efficient for **streaming**: a constant-size state per step suits robots, sensors and on-device video. Third, recurrence is back: modern state-space models such as Mamba are recurrences with clever parameterizations, and ML 04 applies them to images.

A recurrent state

Plain version. At every step, combine the current input with the previous state to make a new state, using the same weights each time. Read outputs off the state whenever you need them.

Precise version. Let the input sequence be $x_1, \dots, x_T$ with $x_t \in \mathbb{R}^d$, and let the hidden state be $h_t \in \mathbb{R}^m$ with a fixed starting state h_0 (usually zeros). The simple (Elman [9]) RNN is

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \qquad \hat{y}_t = g\big(W_y h_t + c\big).$$

$W_x \in \mathbb{R}^{m \times d}$ maps the input into the state space, $W_h \in \mathbb{R}^{m \times m}$ is the **recurrent** weight matrix that carries information from the previous step, b and c are biases, W_y maps the state to outputs, and g is an output function such as softmax. Crucially, none of these parameters depends on t. That is **parameter sharing across time**, the temporal version of a CNN's weight sharing: the model has the same number of parameters whether the sequence has 5 or 5,000 steps, and a pattern learned at step 3 can be recognized at step 300.

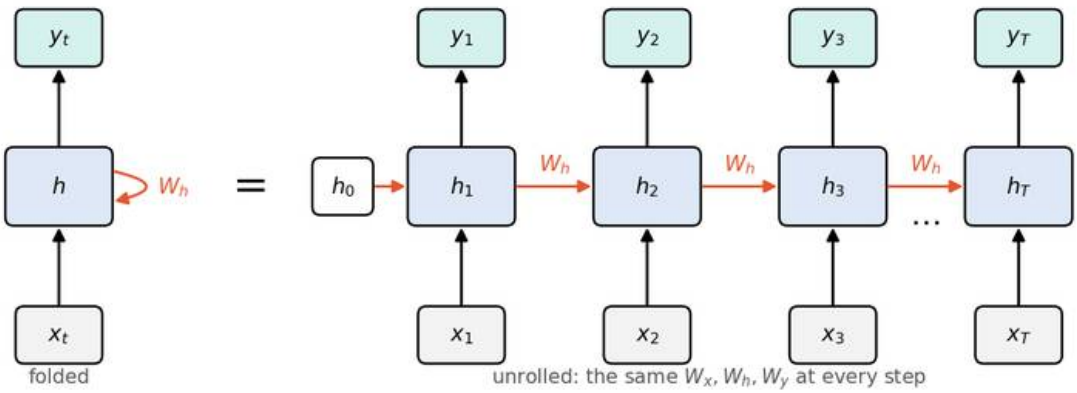


Figure 1 — Folded and unrolled views of an RNN. Unrolling turns the loop into a deep feed-forward network whose layers all share the same weights.

Unrolling (Figure 1) is the key mental move. Writing out the loop for T steps produces an ordinary feed-forward network that is T layers deep, in which every layer has the same weights. Everything you learned about backpropagation in ML 02 applies to this unrolled graph.

The same recurrence supports several input–output patterns:

Pattern	Example	Output read from
Many-to-one	Classify a video clip; sentiment of a sentence	The last state h_T (or a pooled state)
Many-to-many, aligned	Label every video frame; tag every word	Every h_t
One-to-many	Generate a caption from one image vector	A decoder that feeds its own outputs back in
Many-to-many, unaligned	Translate a sentence; describe a video	An **encoder–decoder** (covered below)

An RNN that predicts x_{t+1} from $x_{\le t}$ is an **autoregressive** model, and sampling from it generates new sequences; but an RNN used as an encoder is not automatically a generator. The head and the loss decide the task.

Code: an RNN and its backward pass in NumPy

The snippet below implements the forward pass and backpropagation through time by hand for a loss $L = v^\top h_T$, then checks one gradient entry against a finite difference. Notice the line `dWh += ⋯`: because W_h is used at every step, its gradient is the **sum** of contributions from all steps.

```
import numpy as np

rng = np.random.default_rng(0)
d, m, T = 3, 5, 7                                  # input size, state size, length
Wx = rng.normal(0, 0.5, (m, d))
Wh = rng.normal(0, 0.5, (m, m))
b = np.zeros(m)

def rnn_forward(xs, h0):
    hs = [h0]
    for x in xs:                                    # the SAME Wx, Wh, b at every step
        hs.append(np.tanh(Wx @ x + Wh @ hs[-1] + b))
    return hs

def rnn_backward(xs, hs, dhT):
    """BPTT for the loss L = dhT . h_T; returns dL/dWh and dL/dx_t for every t."""
    dWh, dxs, dh = np.zeros_like(Wh), [None] * len(xs), dhT
    for t in reversed(range(len(xs))):
```

```
        da = dh * (1 - hs[t + 1] ** 2)              # through tanh
        dWh += np.outer(da, hs[t])                  # shared weight: gradients ADD up
        dxs[t] = Wx.T @ da
        dh = Wh.T @ da                              # pass the gradient one step back
    return dWh, dxs

xs = rng.normal(size=(T, d))
hs = rnn_forward(xs, np.zeros(m))
dhT = rng.normal(size=m)
dWh, dxs = rnn_backward(xs, hs, dhT)

# finite-difference check of one entry of dL/dWh
eps, (i, j) = 1e-6, (2, 3)
Wh[i, j] += eps; up = dhT @ rnn_forward(xs, np.zeros(m))[-1]
Wh[i, j] -= 2 * eps; down = dhT @ rnn_forward(xs, np.zeros(m))[-1]
Wh[i, j] += eps
print("BPTT vs finite difference:", np.isclose(dWh[i, j], (up - down) / (2 * eps)))
print("|dL/dx_t| from first to last step:", np.round([np.linalg.norm(g) for g in dxs], 3))
```

It prints `True`, and the input-gradient norms are $[0.009, 0.015, 0.006, 0.029, 0.383, 0.212, 0.433]$: even in a seven-step sequence, the early inputs receive gradients roughly ten to fifty times smaller than the late ones. That is the vanishing-gradient problem in miniature.

Backpropagation through time

Plain version. Unroll the network, run ordinary backpropagation through the unrolled graph, and add up the gradients for each shared weight.

Precise version. Write the pre-activation as $a_t = W_x x_t + W_h h_{t-1} + b$, so $h_t = \tanh(a_t)$. Suppose the loss is a sum over steps, $L = \sum_t L_t(h_t)$. Working backward from $t = T$, the total gradient arriving at h_t is

$$\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + \left(\frac{\partial h_{t+1}}{\partial h_t}\right)^{\top} \frac{\partial L}{\partial h_{t+1}}, \qquad \frac{\partial h_{t+1}}{\partial h_t} = \operatorname{diag}(1 - h_{t+1}^2)\, W_h.$$

The first term is the local loss at step t; the second is everything later that h_t influenced, passed back one step. With $\delta_t = (1 - h_t^2) \odot \partial L / \partial h_t$ (the gradient at the pre-activation), the parameter gradients are sums over time:

$$\frac{\partial L}{\partial W_h} = \sum_{t=1}^{T} \delta_t\, h_{t-1}^{\top}, \qquad \frac{\partial L}{\partial W_x} = \sum_{t=1}^{T} \delta_t\, x_t^{\top}, \qquad \frac{\partial L}{\partial b} = \sum_{t=1}^{T} \delta_t.$$

This algorithm, **backpropagation through time** (**BPTT**), was laid out clearly by Werbos [8]. Its cost is linear in T, but so is its memory: every h_t must be stored for the backward pass.

Truncated BPTT

For a very long stream (an hour of video, a continuous sensor log) storing all states is impossible.

Truncated BPTT processes the stream in windows of k steps. It carries the hidden state forward

from one window to the next, so the model still *sees* the whole past, but it **detaches** the state from the graph at each window boundary, so gradients flow back at most k steps. The model can therefore use long-range information only if it learns to preserve it from the gradient signal within windows.

```
import torch
from torch import nn

torch.manual_seed(0)
rnn, head = nn.GRU(1, 16, batch_first=True), nn.Linear(16, 1)
params = list(rnn.parameters()) + list(head.parameters())
opt = torch.optim.Adam(params, lr=1e-2)
t = torch.arange(0, 2000) * 0.05
stream = torch.sin(t) + 0.1 * torch.randn_like(t)   # one long noisy signal
x, y = stream[:-1, None][None], stream[1:, None][None]   # predict the next sample

h, window = None, 50
for start in range(0, x.shape[1] - window, window):
    out, h = rnn(x[:, start:start + window], h)
    loss = nn.functional.mse_loss(head(out), y[:, start:start + window])
    opt.zero_grad()
    loss.backward()                       # gradients flow back at most `window` steps
    nn.utils.clip_grad_norm_(params, 1.0)
    opt.step()
    h = h.detach()                        # keep the state, cut the graph
print("final window MSE:", round(loss.item(), 4), "(noise variance is 0.01)")
```

After a single pass over the stream, the final-window mean squared error is about 0.014, close to the 0.01 floor set by the added noise. Forgetting `h.detach()` makes the second window try to backpropagate into a graph that was already freed, and PyTorch raises an error; resetting `h = None` at each window instead would throw the context away.

Vanishing and exploding gradients

Plain version. The gradient that travels from step t back to step s is multiplied by roughly the same matrix at every step. Repeated multiplication either shrinks it toward zero or blows it up, so plain RNNs struggle to learn dependencies more than a few dozen steps long.

Precise version. Chaining the one-step Jacobians gives

$$\frac{\partial h_t}{\partial h_s} = \prod_{k=s+1}^{t} \frac{\partial h_k}{\partial h_{k-1}} = \prod_{k=s+1}^{t} \operatorname{diag}(1 - h_k^2)\, W_h,$$

an ordered product of $t - s$ matrices (matrices do not commute, so the order matters). Because $|\tanh'| \le 1$,

$$\left\| \frac{\partial h_t}{\partial h_s} \right\| \le \left(\sigma_{\max}(W_h)\right)^{t-s},$$

where $\sigma_{\max}$ is the largest singular value. If $\sigma_{\max}(W_h) < 1$, the gradient from distant steps **vanishes** exponentially. If W_h has large singular values and the units are not saturated, the gradient can **explode**. Bengio, Simard and Frasconi [6] showed in 1994 that this is not a bug of a particular training method: a system that robustly stores information in the presence of noise must be in a regime where gradients of long-range dependencies shrink. Pascanu, Mikolov and Bengio [7] revisited the analysis with dynamical-systems tools and gave a practical remedy for explosions.

The intuition is clearest for a scalar linear RNN, $h_t = \lambda h_{t-1} + x_t$: then $\partial h_t / \partial h_s = \lambda^{t-s}$, which is $0.9^{80} \approx 2 \times 10^{-4}$ for $\lambda = 0.9$ and $1.1^{80} \approx 2 \times 10^{3}$ for $\lambda = 1.1$. Figure 2 shows the same behavior measured in real networks.

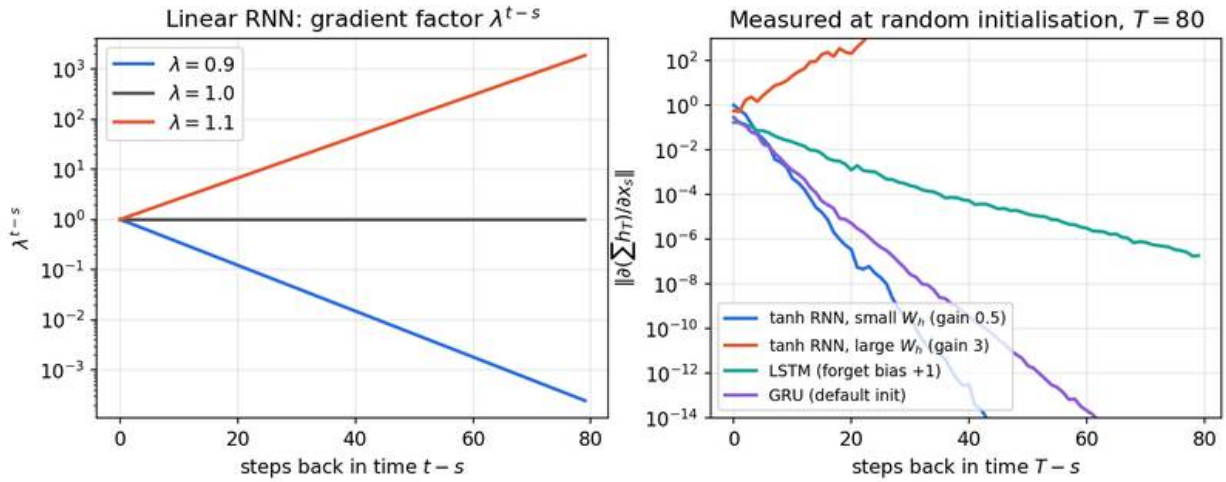


Figure 2 — Left: the scalar factor λ^(t−s). Right: gradient of the final state with respect to each input, measured at random initialization ($T = 80$). The plain tanh RNN vanishes or explodes; the LSTM's additive memory path decays far more slowly.

Remedies. Explosions are handled by **gradient clipping** [7]: if the gradient norm $\|g\|$ exceeds a threshold τ, rescale it to $g \leftarrow \tau\, g / \|g\|$. This keeps the update direction and caps its length. PyTorch's `clip_grad_norm_` does exactly this. Vanishing gradients are harder: they require a different architecture, one whose state update is close to the identity so that gradients can pass through many steps. That architecture is the LSTM.

Long short-term memory (LSTM)

Plain version. Give the network a separate memory lane, the **cell state**, that is updated by addition rather than by repeated matrix multiplication, and let learned **gates** decide what to erase, what to write and what to reveal.

Precise version. With $[x_t, h_{t-1}]$ denoting concatenation and σ the logistic sigmoid, a standard LSTM step is

$$
\begin{aligned}
f_t &= \sigma\big(W_f[x_t, h_{t-1}] + b_f\big) && \text{forget gate} \\
i_t &= \sigma(W_i[x_t, h_{t-1}] + b_i) && \text{input gate} \\
o_t &= \sigma(W_o[x_t, h_{t-1}] + b_o) && \text{output gate} \\
\tilde{c}_t &= \tanh(W_c[x_t, h_{t-1}] + b_c) && \text{candidate memory} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{cell update} \\
h_t &= o_t \odot \tanh(c_t) && \text{exposed state}
\end{aligned}
$$

Each gate is a vector of numbers between 0 and 1, and $\odot$ is element-wise multiplication, so each gate acts like a bank of dimmer switches, one per memory unit. The forget gate f_t decides how much of the old memory to keep, the input gate i_t how much of the candidate $\tilde{c}_t$ to write, and the output gate o_t how much of the memory to reveal as h_t.

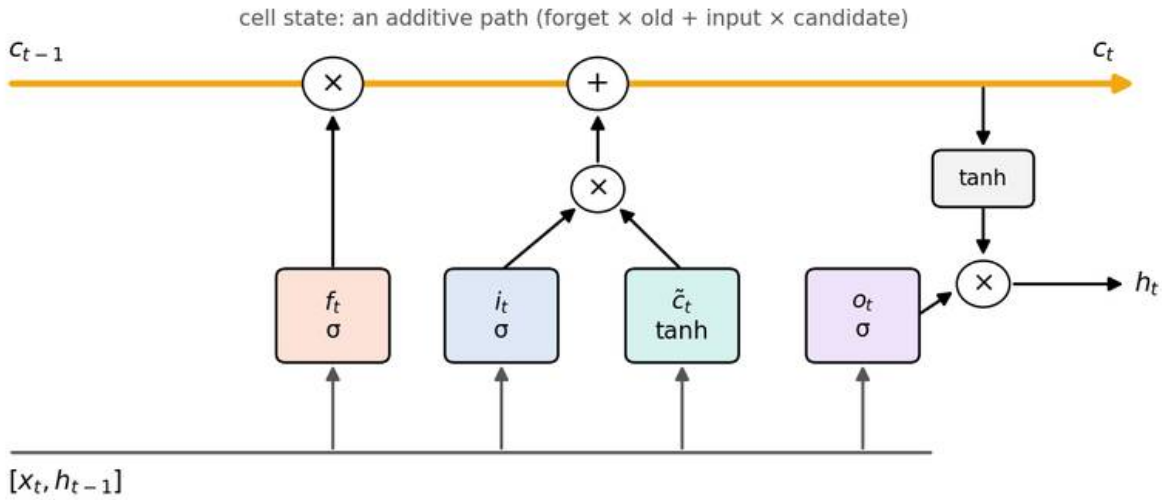


Figure 3 — Inside an LSTM cell. The cell state (gold) changes only by an element-wise scaling and an addition, which gives gradients a protected path through time.

Why it works. Along the cell-state path, $\partial c_t / \partial c_{t-1} = \mathrm{diag}(f_t)$ (ignoring the smaller indirect paths through the gates). There is no recurring weight matrix and no squashing nonlinearity on this path. If the forget gate stays near one, the gradient passes through many steps almost unchanged.

That is why the LSTM curve in Figure 2 decays so slowly. It is also why a common initialization sets the forget-gate bias to a positive value such as +1, so the network starts out remembering. This is not a guarantee of unlimited memory: the forget gate can and does learn to close, and long-range credit assignment still gets harder with distance.

History and attribution. Hochreiter and Schmidhuber introduced the LSTM in 1997 [2] with a constant-error "carousel": the cell state passed on unchanged and there was **no forget gate**. Gers, Schmidhuber and Cummins added the forget gate in 2000 [5] so that the cell could learn to reset its memory, for example between independent segments of a continuous stream. The equations above, with the forget gate, are the version used in practice and in every major framework. Greff et al. [19] compared eight LSTM variants on speech, handwriting and music tasks and found that none significantly beat the standard architecture, while the forget gate and the output activation were its most critical components.

Parameters. Each of the four gate blocks has an $m \times (d + m)$ weight matrix and a bias, so an LSTM layer has about $4m(d + m + 1)$ parameters, four times a simple RNN with the same state size. (PyTorch keeps separate input and recurrent biases, giving $4m(d + m + 2)$.)

Code: an LSTM step from scratch, checked against PyTorch

PyTorch stores the four gate blocks stacked in the order input, forget, cell candidate (g), output. Implementing the step by hand and comparing with `nn.LSTMCell` is the best way to be sure you read the equations correctly:

```
import torch
from torch import nn

torch.manual_seed(0)
d, m = 4, 6
cell = nn.LSTMCell(d, m)                # PyTorch stacks gate weights in order i, f, g, o

def lstm_step(x, h, c):
    z = x @ cell.weight_ih.T + cell.bias_ih + h @ cell.weight_hh.T + cell.bias_hh
    i, f, g, o = z.chunk(4, dim=-1)
    i, f, o = torch.sigmoid(i), torch.sigmoid(f), torch.sigmoid(o)
    g = torch.tanh(g)                   # candidate memory, often written c~_t
    c = f * c + i * g                   # additive cell update
    h = o * torch.tanh(c)
    return h, c

x = torch.randn(2, d)
h0, c0 = torch.randn(2, m), torch.randn(2, m)
h_ref, c_ref = cell(x, (h0, c0))
h_ours, c_ours = lstm_step(x, h0, c0)
print("matches nn.LSTMCell:", torch.allclose(h_ours, h_ref, atol=1e-6),
      torch.allclose(c_ours, c_ref, atol=1e-6))
```

```
print("parameters:", sum(p.numel() for p in cell.parameters()), "= 4 * m * (d + m + 2) =",
      4 * m * (d + m + 2))
```

It prints `True True` and `288 = ⋯ = 288`.

Gated recurrent unit (GRU)

Plain version. A GRU is a slimmer LSTM: it merges the cell and hidden states into one, and uses two gates instead of three.

Precise version. Following Cho et al. [3] and PyTorch's `nn.GRU`,

$$
\begin{aligned}
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) && \text{reset gate} \\
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) && \text{update gate} \\
\tilde{h}_t &= \tanh(W x_t + r_t \odot (U h_{t-1}) + b) && \text{candidate} \\
h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t && \text{interpolation}
\end{aligned}
$$

The reset gate r_t decides how much of the old state to use when forming the candidate; the update gate z_t interpolates between keeping the old state ($z_t \approx 1$) and replacing it ($z_t \approx 0$). The interpolation plays the role of the LSTM's additive cell path. Two caveats are worth knowing. Some papers and textbooks swap the roles of z_t and $1 - z_t$; the models are equivalent, but the gate means the opposite. And the original paper applies the reset gate before the recurrent matrix ($U(r_t \odot h_{t-1})$), whereas PyTorch and cuDNN apply it after, as written above. Always check the framework's documentation before reading meaning into a gate.

GRUs have three gate blocks instead of four, so about 25% fewer parameters than an LSTM of the same width. Neither consistently wins across tasks; the GRU is a reasonable default when data or compute is limited.

Code: a recurrent classifier that generalizes to fresh sequences

This complete CPU example learns to classify the sign of a sequence's sum, which requires accumulating information over all twelve steps. A fresh test set checks that the model did not merely memorize its training sequences.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
rnn = nn.GRU(input_size=1, hidden_size=24, batch_first=True)
head = nn.Linear(24, 2)
```

```
opt = torch.optim.Adam(list(rnn.parameters())+list(head.parameters()), lr=.003)
def batch(n):
    x = torch.randn(n, 12, 1)
    y = (x.sum(dim=(1, 2)) > 0).long()
    return x, y
for step in range(400):
    x, y = batch(64)
    _, h = rnn(x)
    loss = F.cross_entropy(head(h[-1]), y)
    opt.zero_grad(); loss.backward()
    nn.utils.clip_grad_norm_(list(rnn.parameters())+list(head.parameters()), 1.)
    opt.step()
rnn.eval(); head.eval()
with torch.no_grad():
    x, y = batch(512)
    _, h = rnn(x)
    acc = (head(h[-1]).argmax(1) == y).float().mean()
    print("fresh-sequence accuracy:", acc.item())
```

It reaches about 99% accuracy on fresh length-12 sequences. The returned hidden tensor has shape `layers × batch × hidden` even with `batch_first=True`, which affects only the input and output sequence layout. Testing on longer sequences asks a different question, length generalization, and a model can succeed at length 12 and fail at length 100 (Exercise 2).

Deep, bidirectional and streaming RNNs

Stacking. A **deep RNN** feeds the sequence of hidden states of one recurrent layer into another (`num_layers` in PyTorch). Lower layers can capture short-range patterns and higher layers longer ones. Two to four layers are typical; deeper stacks usually need residual connections between layers.

Bidirectional. Schuster and Paliwal [10] proposed running one RNN forward in time and another backward, then concatenating their states, $h_t = [\vec{h}_t; \overleftarrow{h}_t]$. Each position then summarizes both its past and its future. This is ideal for **offline** tasks where the whole input is available (tagging a recorded video, encoding a sentence), but it is **not causal**: it cannot be used for a live prediction that must be made before future frames arrive. Reporting offline bidirectional accuracy for an online application overstates what the deployed system can know.

Variable lengths. Batches contain sequences of different lengths, padded to a common length. Without care, the padding leaks into states and losses. Use per-sequence lengths (`pack_padded_sequence` in PyTorch) or masks so that padded steps neither update the state nor contribute to the loss.

Streaming. For a live stream, keep the state between calls and reset it only between indepen-

dent streams. Accidentally carrying state from one user, document or video into the next leaks information and produces irreproducible results.

Encoder–decoder models and teacher forcing

Plain version. To turn one sequence into another of a different length (a sentence into its translation, a video into a description), use one RNN to read the input into a summary and a second RNN to write the output from that summary, one token at a time.

Precise version. In the **sequence-to-sequence** (seq2seq) model of Sutskever, Vinyals and Le [11] and the closely related encoder–decoder of Cho et al. [3], an **encoder** RNN reads $x_1, \dots, x_T$ and its final state becomes a context vector $c = h_T$. A **decoder** RNN, initialized from c, models

$$p(y_1, \dots, y_U \mid x) = \prod_{u=1}^{U} p(y_u \mid y_{<u}, c), \qquad s_u = \mathrm{RNN}(s_{u-1}, y_{u-1}),$$

where s_u is the decoder state and $p(y_u \mid \cdot)$ is a softmax over the vocabulary. Generation stops when the decoder emits an end-of-sequence token, so input and output lengths are independent.

Teacher forcing. During training, the decoder input at step u is the **ground-truth** previous token y_{u-1}, not the model's own prediction. This makes all steps trainable in parallel against the known target and greatly stabilizes learning. At test time, however, the model must consume its own predictions, which may contain errors it never saw during training. This train–test mismatch is called **exposure bias**. Scheduled sampling [25] gradually replaces ground-truth inputs with model samples during training to reduce it. Decoding usually uses greedy or beam search rather than sampling.

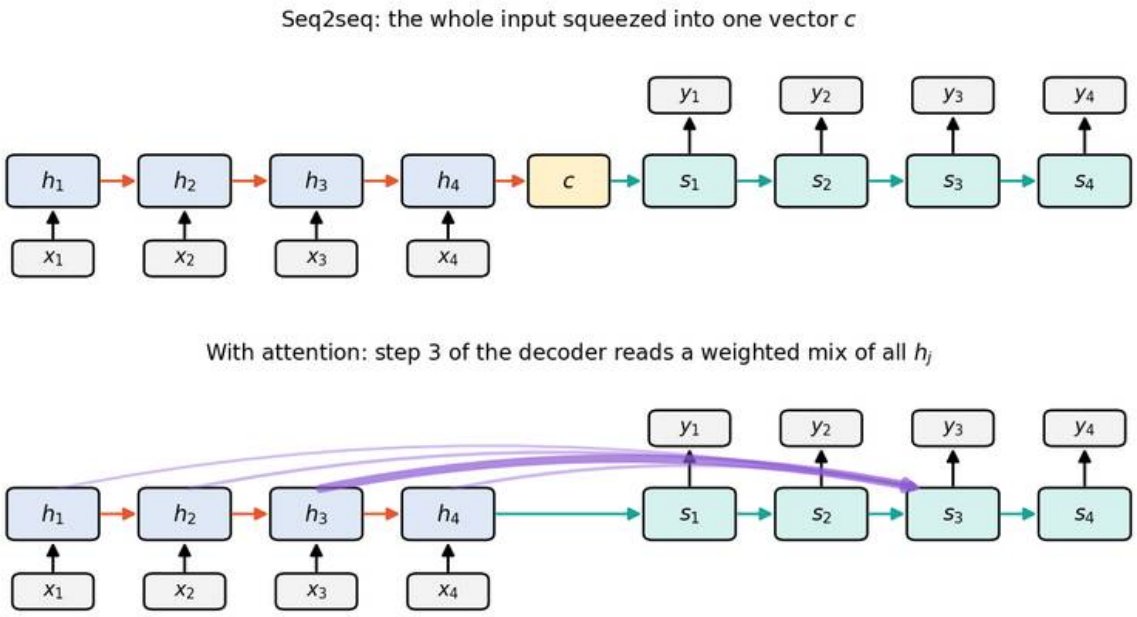


Figure 4 — Top: plain seq2seq squeezes the entire input into one vector c. Bottom: with attention, every decoder step reads a learned, weighted mix of all encoder states.

The weakness is visible in Figure 4 (top): everything the decoder knows about the input must pass through one fixed-size vector c, no matter how long the input. Translation quality of plain seq2seq models dropped on long sentences. Sutskever et al. found that simply **reversing the source sentence** helped, because it shortened the distance between corresponding words, a hint that the bottleneck and long paths were the problem [11].

Attention: the bridge to Transformers

Plain version. Instead of forcing the decoder to rely on one summary vector, let it look back at **all** encoder states at every step and decide which ones are relevant right now.

Precise version. Bahdanau, Cho and Bengio [12] kept every encoder state $h_1, \dots, h_T$ (from a bidirectional encoder) and, at each decoder step u, computed

$$e_{u,j} = v^\top \tanh\big(W s_{u-1} + U h_j\big), \qquad \alpha_{u,j} = \frac{\exp(e_{u,j})}{\sum_{k=1}^{T} \exp(e_{u,k})}, \qquad c_u = \sum_{j=1}^{T} \alpha_{u,j}\, h_j .$$

$e_{u,j}$ is a learned **alignment score** between the decoder's current need (s_{u-1}) and encoder position j; W, U and v are learned parameters. The softmax turns scores into weights $\alpha_{u,j}$ that are positive and sum to one, and the **context vector** c_u is their weighted average of encoder states. The decoder

then uses c_u together with its own state. This form is called **additive attention**. Because c_u changes at every step and has direct access to every input position, the path length between any input and any output becomes one step, regardless of the sequence length.

```
import torch
from torch import nn

class AdditiveAttention(nn.Module):
    """Bahdanau-style scores e_j = v^T tanh(W s + U h_j), then softmax over j."""
    def __init__(self, d_dec, d_enc, d_att):
        super().__init__()
        self.W = nn.Linear(d_dec, d_att, bias=False)
        self.U = nn.Linear(d_enc, d_att, bias=False)
        self.v = nn.Linear(d_att, 1, bias=False)

    def forward(self, s, H, mask=None):
        # s: B x d_dec (decoder state), H: B x T x d_enc (all encoder states)
        e = self.v(torch.tanh(self.W(s)[:, None] + self.U(H))).squeeze(-1)   # B x T
        if mask is not None:                        # True where the input is padding
            e = e.masked_fill(mask, float("-inf"))
        alpha = e.softmax(dim=-1)                   # attention weights, rows sum to 1
        context = (alpha[:, :, None] * H).sum(1)    # B x d_enc weighted average
        return context, alpha

torch.manual_seed(0)
att = AdditiveAttention(d_dec=16, d_enc=32, d_att=8)
H = torch.randn(2, 5, 32)
s = torch.randn(2, 16)
pad = torch.tensor([[False] * 5, [False, False, False, True, True]])   # 2nd sequence has
 ↪  length 3
ctx, alpha = att(s, H, pad)
print("context:", tuple(ctx.shape), "| weights sum to 1:",
      torch.allclose(alpha.sum(-1), torch.ones(2)))
print("weights on padding:", alpha[1, 3:].tolist())
```

It prints a context of shape `(2, 32)`, confirms that weights sum to one, and shows exactly zero weight on padded positions: masking with $-\infty$ before the softmax is the standard way to make attention ignore padding (and, in ML 04, the future).

Attention weights are also interpretable as a soft **alignment**. Figure 5 shows a small GRU encoder–decoder with this attention trained for under a minute on CPU to reverse random digit sequences (about 96% per-token accuracy with teacher forcing on fresh sequences). Without being told, it learned to look at input position $T + 1 - u$ when writing output u: the anti-diagonal.

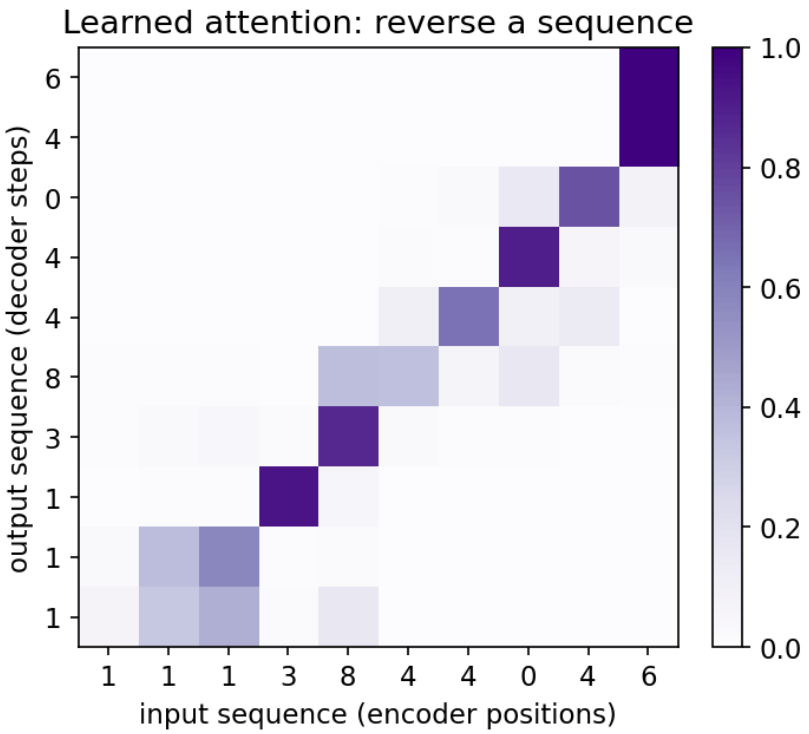


Figure 5 — Attention weights of a toy encoder–decoder trained to reverse a sequence (generated by the figure script). Each output step concentrates on the mirrored input position.

From here to Transformers. Attention fixed the bottleneck, but the encoder and decoder were still recurrent, so computation over a sequence of length T required T sequential steps that cannot be parallelized during training. Vaswani et al. [28] asked what happens if recurrence is removed entirely and **attention is used inside the encoder and decoder as well** (self-attention), with positional encodings to restore order. The answer, the Transformer, trains in parallel across positions and became the dominant architecture for language and, with ViT, for vision. ML 04 picks up the story with scaled dot-product attention, which replaces the additive score with $q^\top k / \sqrt{d_k}$.

Recurrent networks in vision

Image captioning. Show and Tell [13] (Vinyals et al.) is a direct image-to-sequence model: a CNN (ML 03) encodes the image into a vector that initializes an LSTM language model, which then generates the caption word by word, trained with teacher forcing on image–caption pairs. Show, Attend and Tell [14] (Xu et al.) kept the CNN's spatial feature grid instead of a single vector and let the LSTM attend over image regions at each word, producing attention maps that often highlight the object being named. This was one of the first widely seen demonstrations of visual attention.

Video understanding. Long-term Recurrent Convolutional Networks (LRCN) [15] (Donahue et al.) apply a CNN to each frame and an LSTM across frames, trained end to end, for activity recognition, image description and video description. The CNN handles space, the RNN handles time: a common pattern for any per-frame feature sequence, such as pose tracks or per-frame detections in object tracking.

Spatiotemporal forecasting. When the state itself should be an image, fully connected gates waste the spatial structure. The **ConvLSTM** of Shi et al. [16] replaces the matrix products in the LSTM gates with convolutions, $i_t = \sigma(W_{xi} * X_t + W_{hi} * H_{t-1} + \dots)$, so inputs, states and gates are all 3-D tensors (channels × height × width). It was introduced for precipitation nowcasting from radar maps and is widely used for video prediction and other forecasting on grids.

What changed. For captioning and video classification at scale, Transformers and vision–language models (see ML 09) have largely replaced LSTMs, because they train in parallel and scale better with data. Recurrent and convolutional-recurrent models remain common where frames arrive one at a time and memory must stay constant: online video processing, tracking, robotics and low-power devices.

Recursive networks: composing over trees

Plain version. A recursive network applies one shared combination rule to merge two child representations into a parent, following a tree, until it reaches the root.

Precise version. For a binary tree whose leaves carry vectors (word embeddings, region features), each internal node computes

$$h_{\text{parent}} = \tanh\!\left(W\,[h_{\text{left}}; h_{\text{right}}] + b\right),$$

with the **same** $W \in \mathbb{R}^{D \times 2D}$ and b at every node. The root vector represents the whole structure and can be classified; every node can also have its own label. Training uses **backpropagation through structure**: the same idea as BPTT, but along the tree's edges. A chain of length n is the special case of a completely unbalanced tree; a balanced tree reduces the path from any leaf to the root from $n-1$ to about $\log_2 n$ steps (Figure 6), which shortens the route for gradients.

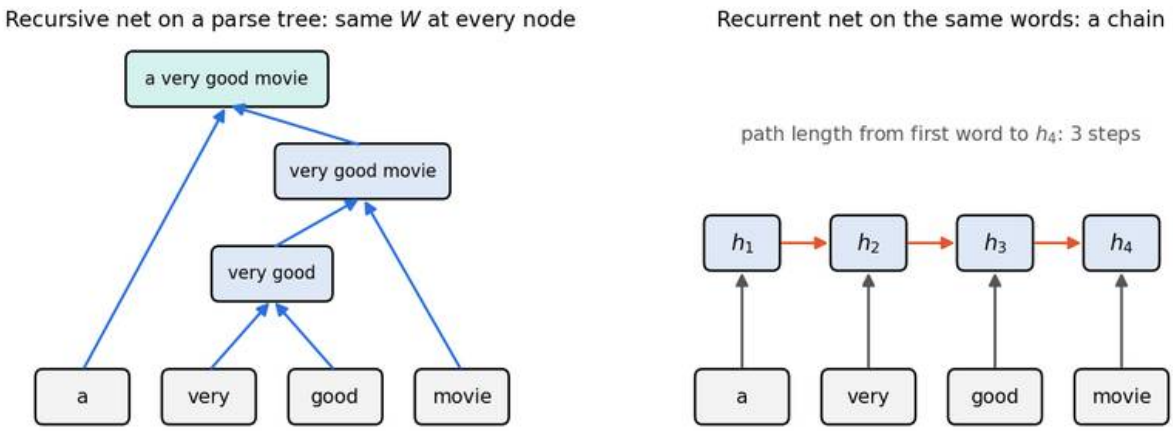


Figure 6 — A recursive network follows a tree (left); a recurrent network follows a chain (right). Both share weights, but the tree mirrors the structure of the input.

Socher et al. [18] used recursive networks in a vision setting first: they segmented an image into superpixels, then learned both to merge adjacent regions into a scene parse tree and to compute region representations, using the same method to parse sentences. Their later Recursive Neural Tensor Network and Stanford Sentiment Treebank [17] showed that composing along parse trees captures phenomena such as negation ("not very good"). Tai, Socher and Manning's **Tree-LSTM** [4] added LSTM-style gates to tree composition, with one forget gate per child, so a parent can selectively keep information from each subtree.

```
import torch
from torch import nn

torch.manual_seed(0)
D = 8
vocab = {"a": 0, "very": 1, "good": 2, "movie": 3}
emb = nn.Embedding(len(vocab), D)
compose = nn.Linear(2 * D, D)            # ONE weight matrix shared by every internal node
classify = nn.Linear(D, 2)               # e.g. negative / positive sentiment

def encode(tree):
    """tree is a word (leaf) or a pair (left, right)."""
    if isinstance(tree, str):
        return emb(torch.tensor(vocab[tree]))
    left, right = tree
    return torch.tanh(compose(torch.cat([encode(left), encode(right)])))

tree = ("a", (("very", "good"), "movie"))          # (a ((very good) movie))
root = encode(tree)
loss = nn.functional.cross_entropy(classify(root)[None], torch.tensor([1]))
loss.backward()                                     # backpropagation through structure
print("root vector:", tuple(root.shape), "| loss:", round(loss.item(), 4))
print("gradient reached the shared W:", compose.weight.grad.abs().sum().item() > 0)
swapped = encode((("movie", ("good", "very")), "a"))
```

```
print("swapping children changes the root:", not torch.allclose(root, swapped))
```

The recursion is ordinary Python, and autograd builds the tree-shaped graph on the fly. Swapping children changes the result because concatenation gives the left and right slots different roles.

The main practical limitation is the tree itself: someone has to provide it (a parser, a segmentation hierarchy), and batching variable trees efficiently is awkward. Learning structure and representation jointly is possible but harder. In current practice, self-attention can learn soft, input-dependent groupings without an explicit tree, which is one reason recursive networks are now uncommon; the idea of hierarchical composition lives on in hierarchical vision backbones and scene graphs.

Mamba: selective state spaces and scan

A useful memory should hold a relevant observation through distractions and overwrite it when the situation changes. Mamba [21] makes the memory update depend on the current input, while keeping that update affine in the previous state. This distinction lets us retain a recurrent interpretation and compute whole sequences with a parallel prefix scan.

From continuous dynamics to selective updates

Start with a small continuous-time memory. A stable transition lets stored information decay; an input projection writes new information; an output projection reads it:

$$\frac{dh(\tau)}{d\tau} = Ah(\tau) + Bu(\tau), \qquad y(\tau) = Ch(\tau).$$

Here τ is continuous time, $u(\tau) \in \mathbb{R}$ is one input channel, $h(\tau) \in \mathbb{R}^N$ is its N-dimensional state, $A \in \mathbb{R}^{N\times N}$ is the transition matrix, $B \in \mathbb{R}^{N\times 1}$ writes the input, $C \in \mathbb{R}^{1\times N}$ reads the state, and $y(\tau) \in \mathbb{R}$ is the output. A full neural block mixes multiple channels with learned projections and includes additional local convolution and gating [21]. This equation isolates its memory mechanism.

Hold the input and its write matrix constant during each interval. Exact zero-order-hold discretization gives

$$\bar{A}_t = \exp(\Delta_t A), \qquad \bar{B}_t = \left(\int_0^{\Delta_t} \exp(sA)\, ds\right) B_t,$$
$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t u_t, \qquad y_t = C_t h_t.$$

Here t indexes discrete tokens, $\Delta_t > 0$ is the interval length, s is an integration variable, u_t is the held input, B_t, C_t are the current write/read matrices, and bars mark discrete coefficients. The integral remains valid when A is singular; an inverse-based formula needs additional assumptions. A diagonal A makes these operations elementwise.

Selectivity means predicting Δ_t, B_t, C_t from the current token. For example, a learned affine projection followed by softplus$(z) = \log(1 + e^z)$ supplies a positive interval. The underlying learned A can remain fixed: $\bar{A}_t$ nevertheless changes through Δ_t. With negative diagonal entries, a larger interval reduces the fraction of old state retained. Selective writes and reads also determine which content enters and leaves memory. The resulting map is nonlinear in its inputs even though each state update is affine. Figure 7 isolates the retention effect with synthetic coefficients; it does not measure a trained model.

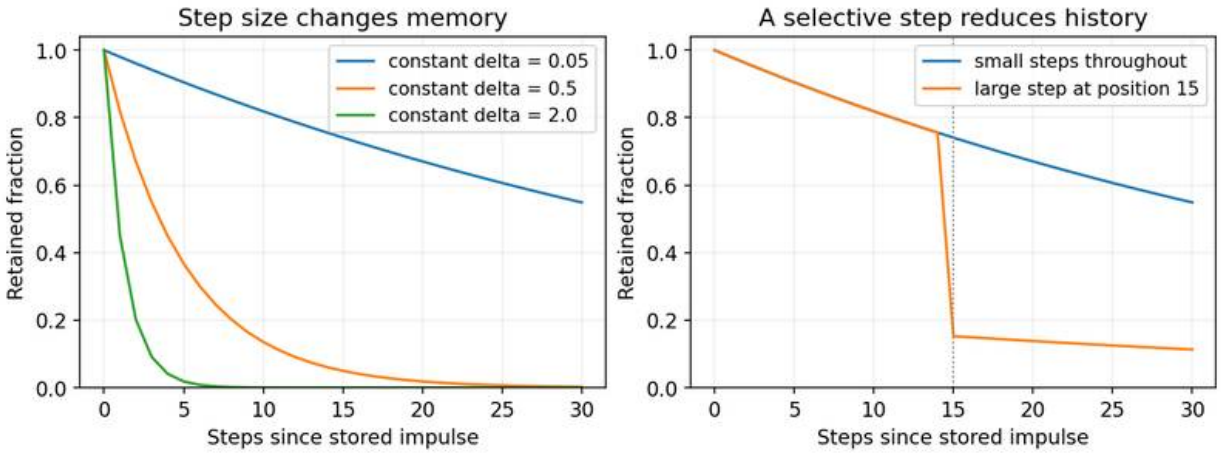


Figure 7 — For a scalar transition $A = -0.4$, retained history is the product of exp$(-0.4\ delta)$. A large selected interval sharply reduces old memory; small intervals preserve it longer.

Why this recurrence can be scanned

Once input-dependent coefficients are computed, composing two affine updates requires only another affine update:

$$(a_2, b_2) \circ (a_1, b_1) = (a_2 a_1,\ a_2 b_1 + b_2).$$

Here pair (a_i, b_i) represents $h \mapsto a_i h + b_i$, $i = 1, 2$ identifies consecutive updates, and $\circ$ means applying the first update and then the second. Composition is associative; for a diagonal transition, products are elementwise. A prefix scan can therefore regroup the operations into a tree while preserving their chronological effect. Ordinary LSTM/GRU gates depend on the previous hidden state, so their coefficients cannot generally be prepared this way before the state is known.

The following standalone CPU example uses exact diagonal discretization and hand-chosen selective coefficients. It checks causality and state continuity across chunks. It is a teaching recurrence, not a pretrained Mamba implementation or a speed benchmark.

```
import numpy as np

rng = np.random.default_rng(44)
u = rng.normal(size=24)
```

```
A = -np.array([0.1, 0.8])  # two stable diagonal entries

def selective_scan(values, state=None):
    h = np.zeros(2) if state is None else state.copy()
    outputs = []
    for x in values:
        delta = np.logaddexp(0.0, 0.5 * x)
        B = np.array([1.0, np.tanh(x)])
        C = np.array([1.0, 0.5 + 0.1 * np.tanh(x)])
        a = np.exp(delta * A)
        b = np.expm1(delta * A) / A * B * x
        h = a * h + b
        outputs.append(C @ h)
    return np.array(outputs), h

y, final_state = selective_scan(u)
changed = u.copy()
changed[12:] += 10.0
changed_y, _ = selective_scan(changed)
assert np.allclose(y[:12], changed_y[:12])
left, boundary = selective_scan(u[:12])
right, resumed_state = selective_scan(u[12:], boundary)
assert np.allclose(y, np.r_[left, right])
assert np.allclose(final_state, resumed_state)
reset_right, _ = selective_scan(u[12:])
assert not np.allclose(right, reset_right)
print('causal prefix and carried-state chunk checks passed')
```

The output confirms both checks. Resetting the state at a batch boundary changes the computation. Conversely, sharing state between independent examples leaks information. A streaming implementation must track which sequence owns each state and reset it at genuine sequence boundaries.

Mamba-2 and the attention connection

Mamba-2 [29] changes both the inner sequence mixer and the surrounding block. Its structured state-space duality uses a scalar-times-identity transition within each head and organizes computation into chunks whose dense matrix multiplications suit accelerator hardware. It also restructures parameter projections and normalization. This is more specific than adding attention to the original Mamba block.

With a zero initial state, the scalar-transition recurrence can be expanded as

$$y_t = \sum_{j=1}^{t} \left[C_t \left(\prod_{k=j+1}^{t} a_k \right) B_j \right] u_j.$$

Here j indexes earlier inputs, k indexes intervening transitions, a_k is the scalar discrete transition multiplying the identity, B_j is the discrete write column, C_t is the read row, and an empty product equals one. The bracketed quantity is a structured causal mixing coefficient. This exposes an

attention-like matrix view without requiring ordinary softmax attention or storing that matrix. Hu et al. [30] (ICML 2026) extend the duality to diagonal transitions and clarify why it does not extend directly to standard softmax attention.

At fixed width and state dimension, recurrent inference carries a fixed-size state, whereas ordinary cached causal attention retains keys and values whose storage grows with context length. Compression saves memory but puts pressure on exact recall of many unrelated details. Linear sequence scaling also does not imply constant training memory: activations, gradients and recomputation policies matter. Evaluate retrieval quality, context length and measured device latency together. The image-specific ordering problem appears next in ML 04.

Modern view

Survey literature. Goodfellow et al.'s chapter 10 [1] remains the clearest unified treatment of recurrent and recursive networks, echo-state ideas, and long-term dependencies. Lipton, Berkowitz and Elkan [20] wrote a critical review of RNNs for sequence learning in 2015: a self-contained, historically ordered account of three decades of work leading to LSTMs and bidirectional RNNs, with conflicting notation reconciled and applications to captioning, translation and handwriting recognition. Greff et al. [19] is less a survey than a large empirical study, but it is the reference for which LSTM components matter. Two recent reviews by Tiezzi et al. reframe recurrence for the Transformer era: an extensive survey [26] places Transformer–RNN hybrids, attention variants with recurrent form, and deep state-space models (a special class of linear RNNs) under the single umbrella of recurrence, and highlights research opportunities in learning online from potentially infinite streams rather than fixed-length sequences; and a shorter perspective in *Nature Machine Intelligence* [27] revisits recurrent processing at the crossroad of Transformers and state-space models.

Recurrence returns (2023–2026). Transformers process all pairs of positions, which costs memory and computation quadratic in length. A family of models revisits recurrence to get linear-time inference with a constant-size state:

- **Linear recurrent units.** Orvieto et al. [24] (ICML 2023) showed that a carefully parameterized **linear** recurrence with diagonal complex-valued state matrices, stable initialization and normalization matches deep state-space models on long-range benchmarks, and that the nonlinearity can live outside the recurrence.
- **Selective state spaces.** Mamba [21] makes the state-space parameters functions of the input, so the model can choose what to store or ignore, as LSTM gates do, while a hardware-aware parallel scan keeps training efficient. Vision variants and the debate about whether they are necessary for images are covered in ML 04.
- **Revived LSTMs and GRUs.** xLSTM [22] (NeurIPS 2024) adds exponential gating and a matrix-valued memory to the LSTM and scales it to language-model sizes. Feng et al. [23] (2024) remove the hidden-state dependence from the gates of LSTMs and GRUs ("minLSTM", "min-

GRU"), which allows a parallel scan during training, and report results competitive with recent sequence models.

The shared lesson is that the **gated, additive state update** invented for the LSTM was right; what limited classical RNNs was mostly sequential training. Making the recurrence linear in the state (so it can be computed with a parallel scan) recovers parallel training while keeping cheap, streaming inference.

Open problems. Length generalization beyond the training horizon; credit assignment over very long streams without full BPTT; how much a fixed-size state can remember compared with attention's growing cache, a real limitation for tasks that need exact recall of distant details; and choosing a scan order for 2-D images and 3-D video, where no single natural sequence order exists.

Key takeaways

- An RNN applies one shared update $h_t = f(x_t, h_{t-1})$ at every step, so it handles any sequence length with a fixed number of parameters. Unrolled, it is a deep network with tied weights.
- BPTT is ordinary backpropagation on the unrolled graph; gradients of shared weights are summed over time. Truncated BPTT carries state forward but limits how far gradients flow back.
- Long-range gradients are products of Jacobians and tend to vanish or explode. Clipping fixes explosions; architectures with near-identity, additive state paths fix vanishing.
- LSTM gates (forget, input, output) and GRU gates (reset, update) control a protected memory path. The forget gate was a 2000 addition to the 1997 LSTM. Check the framework's gate conventions.
- Bidirectional encoders use future context and are not causal; streaming models must manage state carefully.
- Seq2seq maps sequences to sequences with teacher forcing; attention removed its single-vector bottleneck, and self-attention without recurrence became the Transformer.
- Recursive networks compose along trees with one shared rule. In vision, recurrence powered captioning, video models and ConvLSTM forecasting, and gated linear recurrences are back as state-space models.
- Mamba predicts write, read and interval coefficients from inputs while keeping updates affine in state; Mamba-2 uses structured duality for efficient chunk computation. Compression saves cache memory but limits exact recall.

Exercises

1. **Causality.** Your team reports 92% frame-level accuracy for an action-recognition model with a bidirectional LSTM and proposes deploying it for live alerts. What is wrong with this plan, and how would you evaluate the deployable alternative fairly? Run the selective-scan example, then add a backward scan: which causality check should fail?

   Hint

   The backward LSTM at frame t has read frames after t, which a live system has not seen yet. Retrain a unidirectional (causal) model, or allow a fixed look-ahead delay and evaluate with exactly that delay. Report accuracy as a function of latency. A backward scan reads the altered future, so the combined prefix generally changes even when the forward prefix does not.

2. **Length generalization.** Train the GRU sum classifier at length 12, then evaluate on fresh sequences of length 6, 24, 100 and 400 without retraining. Plot accuracy versus length. Then repeat with an LSTM and with training on mixed lengths 6–24.

   Hint

   Generate fresh data per length. The sum's scale grows like $\sqrt{T}$, so the state may saturate on long inputs even if the logic is right. Mixed-length training usually helps more than switching cell type. Success at length 12 says nothing by itself about length 400.

3. **Gradient bound.** For the scalar RNN $h_t = \tanh(wh_{t-1} + x_t)$, show that $|\partial h_t / \partial h_s| \leq |w|^{t-s}$. For what values of w can the gradient explode, and why does saturation of tanh work against explosion?

   Hint

   Each factor is $w(1 - h_k^2)$ with $0 \leq 1 - h_k^2 \leq 1$. Explosion requires $|w| > 1$, but a large $|w|$ also drives h_k toward ± 1, which makes $1 - h_k^2$ small. The gradient explodes only on trajectories that stay in the non-saturated region, which is why explosions are bursty in practice.

4. **LSTM memory.** In the LSTM equations, suppose $f_t = 1$ and $i_t = 0$ for all t from s to t. What is c_t in terms of c_s, and what is $\partial c_t / \partial c_s$ along the cell path? What would happen if the forget gate were fixed at 1 forever, as in the original 1997 design, on a never-ending input stream?

   Hint

   $c_t = c_s$ and $\partial c_t / \partial c_s = I$: perfect memory and perfect gradient flow. With $f_t \equiv 1$ and nonzero writes, the cell state accumulates without bound over an endless stream, which is exactly the problem Gers et al. solved by letting the network learn to forget.

5. **Attention shapes and masks.** In the additive-attention code, a batch has encoder states of shape $8 \times 40 \times 256$ (with some sequences padded) and a decoder state of shape 8×512. Give the shapes of e, α and the context, and explain what would go wrong if padded positions were masked with 0 instead of $-\infty$.

   Hint

   e and α are 8×40; the context is 8×256. A score of 0 is an ordinary score: $e^0 = 1$ would still give padding positive weight, and the context would mix in meaningless padded states.

6. **Recursive versus recurrent.** For a balanced binary tree over $n = 1024$ leaves, how many composition steps lie between a leaf and the root? Compare with a left-to-right RNN over the same 1024 inputs, and relate the answer to the vanishing-gradient bound.

   Hint

   $\log_2 1024 = 10$ steps versus up to 1023 for the chain. The gradient bound decays like $(\text{factor})^{\text{path length}}$, so a shorter path from each input to the output helps optimization. Attention takes this to the extreme: path length one.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 10. Book
2. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997. DOI
3. K. Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014. arXiv
4. K. S. Tai, R. Socher and C. D. Manning, "Improved Semantic Representations From Tree-Structured Long Short-Term Memory Networks," ACL, 2015. arXiv
5. F. A. Gers, J. Schmidhuber and F. Cummins, "Learning to Forget: Continual Prediction with LSTM," *Neural Computation*, 2000. DOI
6. Y. Bengio, P. Simard and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, 1994. DOI
7. R. Pascanu, T. Mikolov and Y. Bengio, "On the difficulty of training Recurrent Neural Networks," ICML, 2013. arXiv
8. P. J. Werbos, "Backpropagation through time: what it does and how to do it," *Proceedings of the IEEE*, 1990. DOI
9. J. L. Elman, "Finding Structure in Time," *Cognitive Science*, 1990. DOI
10. M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, 1997. DOI
11. I. Sutskever, O. Vinyals and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," NeurIPS, 2014. arXiv
12. D. Bahdanau, K. Cho and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR, 2015. arXiv

13. O. Vinyals, A. Toshev, S. Bengio and D. Erhan, “Show and Tell: A Neural Image Caption Generator,” CVPR, 2015. arXiv
14. K. Xu et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” ICML, 2015. arXiv
15. J. Donahue et al., “Long-term Recurrent Convolutional Networks for Visual Recognition and Description,” CVPR, 2015. arXiv
16. X. Shi et al., “Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting,” NeurIPS, 2015. arXiv
17. R. Socher et al., “Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank,” EMNLP, 2013. Paper
18. R. Socher, C. C.-Y. Lin, A. Y. Ng and C. D. Manning, “Parsing Natural Scenes and Natural Language with Recursive Neural Networks,” ICML, 2011. Paper
19. K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink and J. Schmidhuber, “LSTM: A Search Space Odyssey,” *IEEE Transactions on Neural Networks and Learning Systems*, 2017. arXiv
20. Z. C. Lipton, J. Berkowitz and C. Elkan, “A Critical Review of Recurrent Neural Networks for Sequence Learning,” arXiv:1506.00019, 2015. arXiv
21. A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” COLM, 2024. arXiv
22. M. Beck et al., “xLSTM: Extended Long Short-Term Memory,” NeurIPS, 2024. Paper
23. L. Feng et al., “Were RNNs All We Needed?,” arXiv:2410.01201, 2024. arXiv
24. A. Orvieto et al., “Resurrecting Recurrent Neural Networks for Long Sequences,” ICML, 2023. arXiv
25. S. Bengio, O. Vinyals, N. Jaitly and N. Shazeer, “Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks,” NeurIPS, 2015. arXiv
26. M. Tiezzi, M. Casoni, A. Betti, T. Guidi, M. Gori and S. Melacci, “On the Resurgence of Recurrent Models for Long Sequences: Survey and Research Opportunities in the Transformer Era,” arXiv:2402.08132, 2024. arXiv
27. M. Tiezzi, M. Casoni, A. Betti, T. Guidi, M. Gori and S. Melacci, “Back to recurrent processing at the crossroad of transformers and state-space models,” *Nature Machine Intelligence*, 2025. DOI
28. A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. arXiv
29. T. Dao and A. Gu, “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality,” ICML, PMLR 235, 2024. Paper
30. J. Y.-C. Hu, X. Zhang, A. Elsheikh, W. Wu and H. Liu, “On Structured State-Space Duality,” ICML, PMLR 306, 2026. Paper

ML 04 • Attention and Vision Transformers

Build self-attention from queries, keys and values; add multiple heads, positional encodings and the Transformer block; turn images into patch tokens with ViT; then study DeiT, Swin, hybrid CNN–Transformer models, efficient attention and state-space alternatives.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-04-vision-transformers/

What you'll learn

- How self-attention computes an input-dependent weighted average, why queries, keys and values play different roles, and why the scores are divided by $\sqrt{d_k}$.
- How multi-head attention, causal masks and cross-attention are built, with code checked against PyTorch.
- Why attention needs positional information, and how sinusoidal, learned, relative and rotary encodings provide it.
- How the Transformer block combines attention, an MLP, LayerNorm and residual connections.
- How the Vision Transformer (ViT) turns an image into patch tokens, why it needs so much data, and how DeiT, Swin and hybrid CNN–Transformer designs respond.
- What makes attention expensive, what FlashAttention, windows and token merging change, and how Mamba and Mamba-2 compare with attention, and how image scan ordering affects global context and directional bias.

The big picture

A CNN (ML 03) moves information through fixed local neighborhoods: a pixel learns about a distant pixel only after many layers. **Attention** lets every location decide, based on its current content, which other locations to read from, in a single step. To recognize a bird, a patch containing its head can directly consult a distant patch containing its wing.

> **Explain like I'm five**
>
> Imagine a classroom where each child holds one piece of a jigsaw puzzle. Each child asks a question about their piece ("I have a blue edge, who has sky?"), and every other child holds up a card describing their own piece. Each child listens most closely to the classmates whose cards best match the question, and updates their idea of what their piece shows. Then everyone does it again with the improved ideas. Attention is the listening rule. A Transformer repeats many rounds of listening, with a little thinking time in between.

The attention idea came from machine translation with recurrent networks: ML 03A shows how it removed the single-vector bottleneck of encoder–decoder RNNs. The Transformer of Vaswani

et al. [1] went further and removed recurrence entirely, building both encoder and decoder from attention and small per-token networks. The Vision Transformer (ViT) of Dosovitskiy et al. [2] applied the Transformer encoder almost unchanged to sequences of image patches. Bishop and Bishop [10] give a modern textbook treatment, and the surveys discussed in the Modern view [3], [11], [12] map the many vision variants. Afterwards, DLCV 02 compares CNN and Transformer backbones for classification in practice, and ML 05 shows how such backbones are pretrained without labels.

Self-attention: queries, keys and values

Plain version. Each token produces three vectors: a **query** ("what am I looking for?"), a **key** ("what do I contain?") and a **value** ("what will I pass on if you listen to me?"). A token's output is the average of all values, weighted by how well its query matches each key.

Precise version. Stack N tokens as the rows of $X \in \mathbb{R}^{N\times D}$, where D is the model width. Three learned matrices project them:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

with $W_Q, W_K \in \mathbb{R}^{D\times d_k}$ and $W_V \in \mathbb{R}^{D\times d_v}$. **Scaled dot-product attention** [1] is

$$\text{Attention}(Q,K,V) = \text{softmax}_{\text{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V = AV.$$

$QK^\top$ is an $N\times N$ matrix of **scores**: entry (i,j) is the dot product of query i with key j, a measure of how relevant token j is to token i. The row-wise softmax turns each row into nonnegative weights that sum to one, giving the **attention matrix** A. The output $Y = AV \in \mathbb{R}^{N\times d_v}$ has one row per token: row i is $\sum_j A_{ij}v_j$, a weighted average of value vectors.

Why three different projections? If queries and keys were the same vector, every token would match itself most strongly, and relevance would be forced to be symmetric. Separate W_Q and W_K let "what I seek" differ from "what I offer" (an eye patch can seek a nose without being one), and a separate W_V lets "what I offer" differ from "what I pass on".

Why divide by $\sqrt{d_k}$? If the components of a query and a key are independent with mean zero and variance one, their dot product $q\cdot k = \sum_{l=1}^{d_k} q_l k_l$ has mean zero and variance d_k. Without scaling, larger heads produce larger scores, the softmax saturates (almost all weight on one token), and its gradients become tiny. Dividing by $\sqrt{d_k}$ restores unit variance. It is a scale convention, not a normalization of each query to unit length.

Content, not position, decides who talks to whom. Unlike a convolution, whose weights are

tied to relative positions, attention weights are computed from the data at run time. A related consequence: self-attention is **permutation-equivariant**. Shuffle the input tokens, and the outputs are shuffled in the same way, because nothing in the formula refers to token order. The snippet checks both the shapes and this symmetry.

```
import torch

torch.manual_seed(0)
X = torch.randn(5, 8)                    # N=5, D=8
Wq, Wk, Wv = [torch.randn(8, 4) for _ in range(3)]
def attention(x):
    q, k, v = x @ Wq, x @ Wk, x @ Wv
    a = (q @ k.T / q.shape[-1]**.5).softmax(dim=-1)
    return a @ v, a
y, a = attention(X)
perm = torch.tensor([3, 0, 4, 1, 2])
yp, _ = attention(X[perm])
assert torch.allclose(a.sum(-1), torch.ones(5), atol=1e-6)
assert torch.allclose(yp, y[perm], atol=1e-5)
print("output shape:", tuple(y.shape))
```

This symmetry suits sets, but an image is not a set of patches: a face with its eyes below its mouth is not a face. Positional information, introduced below, breaks the symmetry.

Multi-head attention

Plain version. Run several attention operations in parallel, each with its own smaller projections, so different heads can track different relationships (nearby texture, the same object, symmetric parts). Then concatenate their outputs and mix them.

Precise version. With h heads, head r has its own $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)} \in \mathbb{R}^{D\times d_k}$, usually with $d_k = d_v = D/h$:

$$\text{head}_r = \text{Attention}\big(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)}\big), \qquad \text{MHA}(X) = [\,\text{head}_1, \dots, \text{head}_h\,]\,W_O,$$

where the brackets concatenate the h outputs of width D/h back to width D, and $W_O \in \mathbb{R}^{D\times D}$ mixes them. Because each head has width D/h, multi-head attention costs about the same as one full-width head, and its projections have $4D^2$ weights in total (plus biases). Heads are not guaranteed to learn human-readable roles; some are redundant and can be pruned.

The implementation below splits the projections into heads, computes the scores per head, concatenates and projects, and matches `nn.MultiheadAttention` exactly. It also shows a **causal mask**: setting forbidden scores to $-\infty$ before the softmax gives them exactly zero weight.

```
import torch
from torch import nn

torch.manual_seed(0)
B, N, D, h = 2, 6, 16, 4
mha = nn.MultiheadAttention(D, num_heads=h, batch_first=True, bias=False)
X = torch.randn(B, N, D)

def multi_head(x):
    Wq, Wk, Wv = mha.in_proj_weight.chunk(3)          # each D x D (rows = outputs)
    q, k, v = x @ Wq.T, x @ Wk.T, x @ Wv.T              # B x N x D
    split = lambda t: t.view(B, N, h, D // h).transpose(1, 2)   # B x h x N x d_k
    q, k, v = split(q), split(k), split(v)
    scores = q @ k.transpose(-2, -1) / (D // h) ** 0.5  # B x h x N x N
    A = scores.softmax(dim=-1)
    heads = A @ v                                       # B x h x N x d_k
    concat = heads.transpose(1, 2).reshape(B, N, D)     # put the heads side by side
    return concat @ mha.out_proj.weight.T, A

ours, A = multi_head(X)
ref, _ = mha(X, X, X)
print("matches nn.MultiheadAttention:", torch.allclose(ours, ref, atol=1e-5))
print("attention tensor:", tuple(A.shape), "| parameters:",
      sum(p.numel() for p in mha.parameters()), "= 4 * D^2 =", 4 * D * D)

# Causal mask: query i may only read keys j <= i
causal = torch.triu(torch.ones(N, N, dtype=torch.bool), diagonal=1)   # True = forbidden
out_c, w_c = mha(X, X, X, attn_mask=causal)
print("no weight on the future:", torch.all(w_c[:, causal] == 0).item())
```

It prints `True`, an attention tensor of shape `(2, 4, 6, 6)` (batch, heads, queries, keys), `1024 = 4 * D^2 = 1024`, and `True` for the causal check. In production code, call `torch.nn.functional.scaled_dot_product_attention`, which computes the same thing and dispatches to fused, memory-efficient kernels when the device, data type and shapes allow [7].

Three ways to connect tokens

Plain version. Which tokens a query is allowed to read is part of the model's design, expressed as a mask on the score matrix.

Mechanism	A query reads	Typical use
Bidirectional self-attention	All tokens of the same sequence	ViT and other image encoders
Causal (masked) self-attention	Itself and earlier positions only	Next-token generation, autoregressive image models
Cross-attention	Tokens from a different sequence	A decoder reading an encoder; image tokens reading text

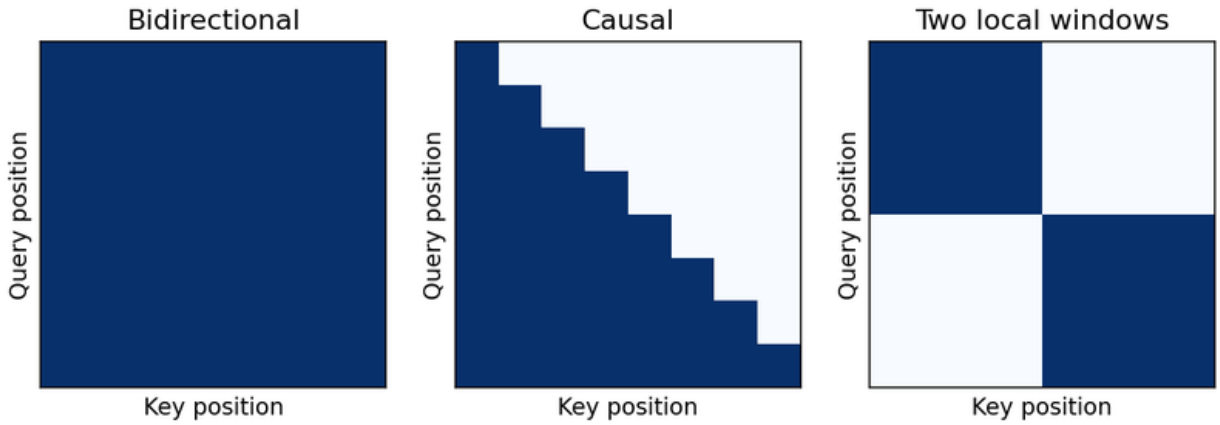


Figure 1 — Attention masks: blue entries are allowed, white entries are forbidden. Causal generation also shifts its inputs by one position so a target cannot simply be copied.

In **cross-attention**, queries come from one sequence X and keys and values from another sequence Z: $Q = XW_Q$, $K = ZW_K$, $V = ZW_V$. The score matrix has shape $N_X \times N_Z$, and the output has one row per query. This is how the original Transformer decoder reads its encoder, how image captioners read image features, and how text-to-image diffusion models read a text prompt (ML 07) without being autoregressive over image patches.

Attention weights are part of a computation, not automatically an explanation. A large weight can multiply an uninformative value, and the residual and MLP paths also shape the output.

Positional information

Plain version. Because attention ignores order, we must tell the model where each token is. The simplest way is to add a position-specific vector to each token before the first layer; more refined ways inject relative position directly into the attention scores.

Sinusoidal encodings. The original Transformer [1] adds a fixed vector $p_{pos} \in \mathbb{R}^D$ to the token at position pos:

$$p_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/D}}\right), \qquad p_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/D}}\right), \qquad i = 0, \dots, \frac{D}{2} - 1.$$

Each pair of dimensions $(2i, 2i+1)$ is a point rotating around a circle at angular frequency $\omega_i = 10000^{-2i/D}$, from fast (one radian per position) to very slow. Two properties follow from the angle-addition formulas. First, for any fixed offset k, $p_{pos+k} = p_{pos} R_k$ for a block-diagonal rotation R_k that does not depend on pos; a linear layer can therefore learn to "look k positions ahead". Second, $p_m \cdot p_n = \sum_i \cos(\omega_i (m - n))$ depends only on the offset $m - n$ (Figure 2, middle).

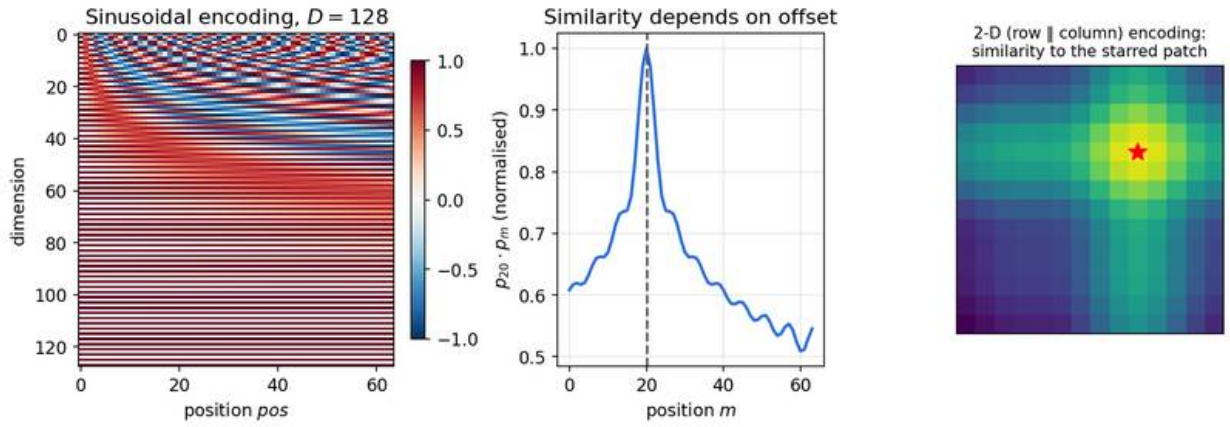


Figure 2 — Sinusoidal positional encodings. Left: each dimension pair oscillates at its own frequency. Middle: similarity between encodings depends on the offset. Right: concatenating row and column encodings gives a 2-D encoding for a 14 × 14 patch grid.

```
import numpy as np

def sinusoidal(n_pos, D):
    pos = np.arange(n_pos)[:, None]
    omega = 1.0 / 10000 ** (np.arange(0, D, 2) / D)    # one frequency per sin/cos pair
    pe = np.zeros((n_pos, D))
    pe[:, 0::2] = np.sin(pos * omega)
    pe[:, 1::2] = np.cos(pos * omega)
    return pe, omega

pe, omega = sinusoidal(100, 16)
k = 7                                                  # a fixed offset
# block-diagonal rotation that maps p_pos to p_{pos+k}, the same for every pos
R = np.zeros((16, 16))
for i, w in enumerate(omega):
    c, s = np.cos(w * k), np.sin(w * k)
    R[2 * i:2 * i + 2, 2 * i:2 * i + 2] = [[c, -s], [s, c]]
print("p[pos+k] == p[pos] @ R for all pos:", np.allclose(pe[:-k] @ R, pe[k:]))
print("dot products depend only on offset:",
      np.isclose(pe[10] @ pe[17], pe[50] @ pe[57]))
```

Both checks print `True`.

Other schemes.

- **Learned absolute embeddings**: a trainable vector per position, as in ViT [2]. ViT's 1-D learned embeddings, indexed by raster order, worked about as well as explicitly 2-D ones; after training, embeddings of patches in the same row or column became similar, so the model learned the grid by itself. At a new resolution, the grid of embeddings is resized by 2-D interpolation before fine-tuning.

- **Relative position representations** (Shaw et al. [17]) add a learned term that depends on the offset $j - i$ to the score between tokens i and j. Swin [5] adds a learned **relative position bias** table to each window's scores.
- **Rotary position embedding (RoPE)** (Su et al. [16]) rotates each query and key by an angle proportional to its position, using the same rotations as the sinusoidal code above. The score $q_m^\top k_n$ then depends on the positions only through $m - n$. RoPE is standard in large language models and increasingly used in vision with separate rotations for the two image axes.
- **No explicit encoding**: convolutions inside the network, as in CvT [19], or zero padding at borders can leak enough position information on their own.

The Transformer block

Plain version. A block alternates two jobs. Attention lets tokens exchange information; a small MLP then processes each token on its own. Residual connections and normalization keep the stack stable.

Precise version. The **pre-norm** block used by ViT and most modern models is

$$U = X + \mathrm{MHA}(\mathrm{LN}(X)), \qquad Y = U + \mathrm{MLP}(\mathrm{LN}(U)), \qquad \mathrm{MLP}(u) = W_2\,\mathrm{GELU}(W_1 u + b_1) + b_2.$$

LN is **layer normalization** [14]: it standardizes each token's D features to zero mean and unit variance, then applies a learned scale and shift; unlike BatchNorm, it does not depend on other samples in the batch. The MLP expands each token from D to typically $4D$ hidden units, applies the GELU nonlinearity [26] ($\mathrm{GELU}(z) = z\,\Phi(z)$, where Φ is the standard normal CDF), and projects back. The residual additions mean each sub-layer only learns a correction, exactly as in ResNet.

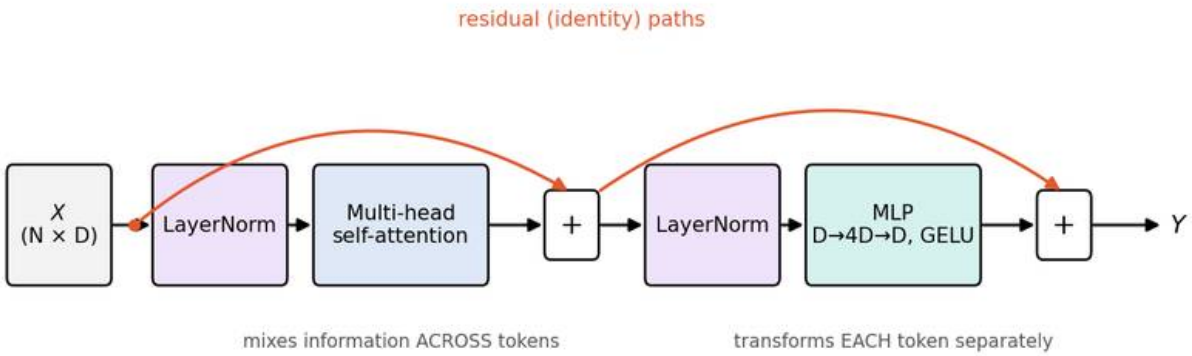


Figure 3 — A pre-norm Transformer block. Attention mixes information across tokens; the MLP transforms each token separately; residual paths (orange) carry the input around each branch.

The original Transformer placed LayerNorm **after** each residual addition (post-norm). Xiong et al. [15] showed that with post-norm the gradients near the output are large at initialization, which is why it needs a learning-rate warm-up, whereas pre-norm keeps gradients well behaved and can train without one. So "a Transformer" is not fully specified until you say where the normalization goes. Counting weights per block: $4D^2$ for attention and $8D^2$ for the MLP, so about $12D^2$, two thirds of it in the MLP.

The Vision Transformer (ViT)

Plain version. Cut the image into square patches, turn each patch into a vector, add position information, and feed the sequence to a standard Transformer encoder. Read the answer from a special extra token.

Patchify. For an image with $H \times W$ pixels and C channels, and patch size P dividing both H and W, there are $N = HW/P^2$ patches. Flatten each into $x_i^{\text{patch}} \in \mathbb{R}^{P^2C}$ and project linearly:

$$z_0 = [\, x_{\text{cls}};\ x_1^{\text{patch}}E;\ \ldots;\ x_N^{\text{patch}}E \,] + E_{\text{pos}},$$

where $E \in \mathbb{R}^{P^2C \times D}$ is the learned **patch embedding**, $x_{\text{cls}} \in \mathbb{R}^D$ is a learned **class token** prepended to the sequence, and $E_{\text{pos}} \in \mathbb{R}^{(N+1)\times D}$ holds learned position embeddings. After L blocks, a LayerNorm and a linear head applied to the class token's output give the logits. Averaging all patch outputs (mean pooling) is an equally common alternative to the class token.

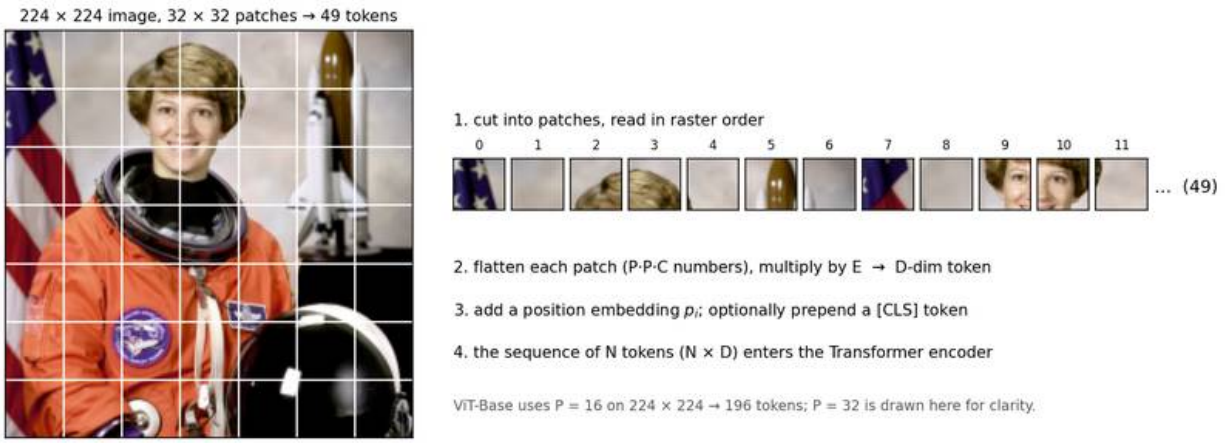


Figure 4 — Patchify. A 32-pixel grid is drawn for readability; ViT-Base uses 16 × 16 patches, which turn a 224 × 224 image into 196 tokens.

A linear map applied to non-overlapping $P \times P$ patches is exactly a convolution with kernel size and stride both equal to P, which is how implementations write it:

```
import torch
from torch import nn

torch.manual_seed(0)
B, C, H, W, P, D = 2, 3, 32, 32, 8, 48
x = torch.randn(B, C, H, W)

# 1) explicit: cut into P x P patches, flatten each, multiply by E
patches = x.unfold(2, P, P).unfold(3, P, P)                 # B, C, H/P, W/P, P, P
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(B, -1, C * P * P)   # B, N, P*P*C
E = torch.randn(C * P * P, D) * 0.02
tokens_explicit = patches @ E                                # B, N, D

# 2) the same linear map written as a convolution with kernel = stride = P
conv = nn.Conv2d(C, D, kernel_size=P, stride=P, bias=False)
conv.weight.data = E.T.reshape(D, C, P, P)
tokens_conv = conv(x).flatten(2).transpose(1, 2)             # B, N, D

print("tokens:", tuple(tokens_conv.shape), "| identical:",
      torch.allclose(tokens_explicit, tokens_conv, atol=1e-5))
```

It prints 16 tokens of width 48 and `True`.

Sizes. The standard configurations are ViT-Base (12 blocks, $D = 768$, 12 heads, about 86 million parameters), ViT-Large (24 blocks, $D = 1024$, 16 heads) and ViT-Huge (32 blocks, $D = 1280$, 16 heads) [2]. "ViT-B/16" means Base with 16×16 patches. **Patch size is the main compute knob**: halving P quadruples N and multiplies the attention-score count by sixteen. Small patches keep fine detail; large patches are cheap but lose anything finer than the patch, which no later layer can recover.

The data lesson. Dosovitskiy et al. [2] found that ViT trained only on ImageNet-1k (about 1.3 million images) was somewhat worse than comparable ResNets, but when pretrained on ImageNet-21k (14 million) or JFT-300M (300 million) and then fine-tuned, it matched or exceeded them at lower pretraining cost. Their explanation is **inductive bias**: a CNN assumes locality and translation equivariance in every layer, whereas a ViT is local only within the patch embedding and must **learn** spatial relationships from data. Raghu et al. [18] compared the internal representations of ViTs and ResNets and found that ViTs have more uniform representations across layers, that their lower layers attend both locally and globally, and that learning local attention in early layers requires large-scale pretraining.

Code: a tiny ViT trained on digits

This complete example trains a small ViT on the same 8×8 digits as the CNN in ML 03, with 2×2 patches (16 tokens), a class token, learned positions and four pre-norm blocks. It runs on CPU in about half a minute.

```
import copy
import numpy as np
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)

class TinyViT(nn.Module):
    def __init__(self, img=8, patch=2, chans=1, D=64, depth=4, heads=4, classes=10):
        super().__init__()
        N = (img // patch) ** 2
        self.patch = nn.Conv2d(chans, D, kernel_size=patch, stride=patch)   # patchify + E
        self.cls = nn.Parameter(torch.zeros(1, 1, D))                       # [CLS] token
        self.pos = nn.Parameter(torch.randn(1, N + 1, D) * .02)             # learned
         ↪  positions
        layer = nn.TransformerEncoderLayer(D, heads, dim_feedforward=4 * D, dropout=0.1,
                                           activation="gelu", batch_first=True,
                                           norm_first=True)                 # pre-norm block
        self.blocks = nn.TransformerEncoder(layer, depth, enable_nested_tensor=False)
        self.norm, self.head = nn.LayerNorm(D), nn.Linear(D, classes)

    def forward(self, x):
        t = self.patch(x).flatten(2).transpose(1, 2)                         # B, N, D
        t = torch.cat([self.cls.expand(len(t), -1, -1), t], dim=1) + self.pos
        return self.head(self.norm(self.blocks(t))[:, 0])                    # read [CLS]

d = load_digits()
X = torch.tensor(d.images[:, None] / 16.0, dtype=torch.float32)
y = torch.tensor(d.target)
train, test = train_test_split(np.arange(len(y)), test_size=.2, stratify=d.target,
                               random_state=0)
train, val = train_test_split(train, test_size=.2, stratify=d.target[train], random_state=1)
model = TinyViT()
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.05)
best = (float("inf"), None)
for epoch in range(30):
    model.train()
    for idx in torch.tensor(train)[torch.randperm(len(train))].split(64):
        loss = nn.functional.cross_entropy(model(X[idx]), y[idx])
        opt.zero_grad(); loss.backward(); opt.step()
    model.eval()
    with torch.no_grad():
        v = nn.functional.cross_entropy(model(X[val]), y[val]).item()
    if v < best[0]:
        best = (v, copy.deepcopy(model.state_dict()))
model.load_state_dict(best[1]); model.eval()
with torch.no_grad():
    acc = (model(X[test]).argmax(1) == y[test]).float().mean().item()
print("parameters:", sum(p.numel() for p in model.parameters()))
```

```
print("best validation loss:", round(best[0], 4), "| held-out accuracy:", round(acc, 4))
```

On our run it uses about 202,000 parameters and reaches about 95% held-out accuracy, while the CNN in ML 03 reached about 96% with fewer than 10,000 parameters and the same split. On a small dataset, the CNN's built-in locality is worth a factor of twenty in parameters. This is the data lesson in miniature, not a verdict on large-scale performance.

Training recipes: DeiT and beyond

Plain version. Much of the early gap between ViTs and CNNs on ImageNet-sized data turned out to be about training, not architecture.

DeiT (Touvron et al. [4]) trained ViTs competitively **on ImageNet-1k alone** using strong data augmentation (RandAugment, Mixup, CutMix, random erasing), heavy regularization (stochastic depth, label smoothing) and long schedules. It also introduced a **distillation token**: an extra learned token, like the class token, whose output is trained to match the predictions of a pre-trained CNN teacher. The CNN's inductive bias is thus transferred through its predictions. DeiT III [27] later showed that a simpler, carefully tuned supervised recipe (a lightweight "3-Augment" scheme, simple cropping and a binary cross-entropy loss) makes plain ViTs stronger still. Steiner et al. [25] ran a large study of data, augmentation and regularization ("AugReg") and found that the right combination of augmentation, regularization and compute can match models trained on roughly ten times more data.

The general lesson, repeated in ML 03's discussion of ConvNeXt: compare architectures only under matched data, augmentation, resolution, schedule and compute.

Swin and hierarchical Transformers

Plain version. Global attention over thousands of tokens is expensive, and plain ViT keeps one coarse resolution throughout, which is awkward for detection and segmentation. Swin restricts attention to local windows, shifts the windows between layers so information still spreads, and merges patches to build a CNN-like pyramid.

Windows. Swin [5] partitions the token grid into non-overlapping windows of $M \times M$ tokens (typically $M = 7$) and computes self-attention inside each window only. For an $h \times w$ token grid with C channels, the paper gives the costs

$$\Omega(\text{global MSA}) = 4hwC^2 + 2(hw)^2C, \qquad \Omega(\text{window MSA}) = 4hwC^2 + 2M^2hwC,$$

where $4hwC^2$ counts the four projections and the second term counts the score and weighting products. Global cost is quadratic in the number of tokens hw; window cost is linear for fixed M.

Shifted windows. With fixed windows, tokens in different windows would never interact. Swin alternates regular windows with windows displaced by $\lfloor M/2 \rfloor$ tokens in each direction. A token near a window boundary at layer ℓ shares a window with tokens from neighboring windows at layer $\ell + 1$, so information crosses boundaries (Figure 5). An efficient implementation cyclically shifts the feature map and uses a mask so that tokens wrapped from opposite edges do not attend to each other.

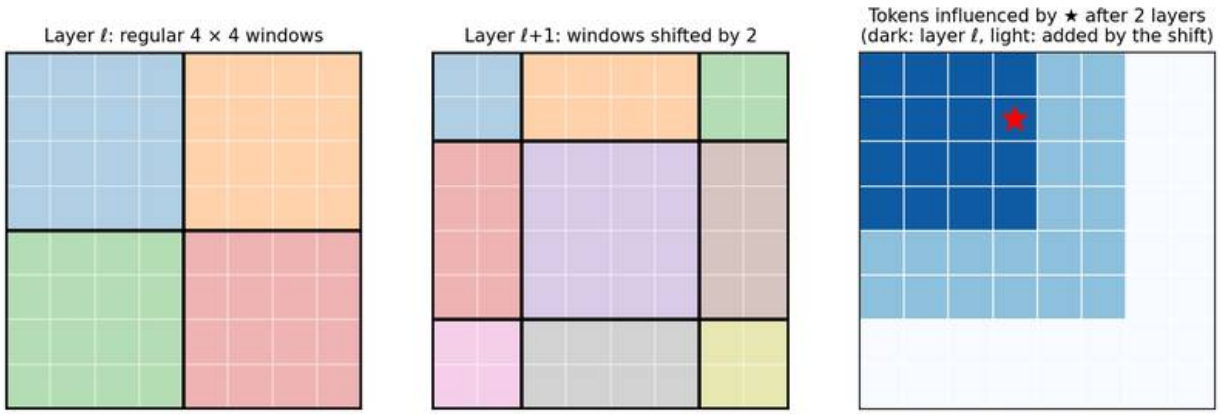


Figure 5 — Swin's regular and shifted windows on an 8 × 8 token grid. After two layers, information from the starred token reaches the dark window (layer ℓ) plus the light band added by the shifted windows (layer ℓ+1).

Hierarchy. Between stages, **patch merging** concatenates each 2 × 2 group of neighboring tokens and projects them linearly, halving the resolution and doubling the channels, as a strided convolution would. The four stages produce feature maps at 1/4, 1/8, 1/16 and 1/32 of the input resolution, the same pyramid a ResNet produces, so Swin can replace a CNN backbone in detection and segmentation frameworks. Each window also adds a learned **relative position bias** to its scores. Swin V2 [32] scaled the design up with residual post-normalization, scaled cosine attention and a continuous, log-spaced position bias that transfers better across window sizes and resolutions.

Hybrid CNN–Transformer models

Plain version. Convolutions are good at local detail with little data; attention is good at long-range interaction. Hybrids use each where it helps.

- **Convolutional stem.** The original ViT paper already tested a "hybrid" in which the patches come from a ResNet feature map instead of raw pixels [2]; it helped at smaller compute budgets.
- **Convolutions inside attention.** CvT [19] uses a convolutional token embedding at each stage

and depthwise-convolutional projections for queries, keys and values, which builds in locality and lets it drop explicit position embeddings.

- **Stacking by stage.** CoAtNet [20] places depthwise-convolution stages early, where resolution is high and local patterns dominate, and relative-attention stages later; it reports strong results across data scales because the convolutional stages supply inductive bias when data is limited.
- **Mobile hybrids.** MobileViT [28] inserts small Transformer blocks into a MobileNet-style network to add global context at mobile cost.
- **The reverse direction.** ConvNeXt (see ML 03) imported Transformer-era design choices into a pure CNN and matched Swin.

Cost and efficient attention

Plain version. Global attention compares every token with every other one, so doubling the image side multiplies the work by sixteen. Efficient methods either make exact attention faster, restrict which tokens interact, or use fewer tokens.

For N tokens of width D, the score matrix costs about N^2D multiply–adds per layer and the projections and MLP about $12ND^2$. A naive implementation stores an $N \times N$ matrix per head. Doubling the image height and width at a fixed patch size multiplies N by four and the pairwise scores by sixteen.

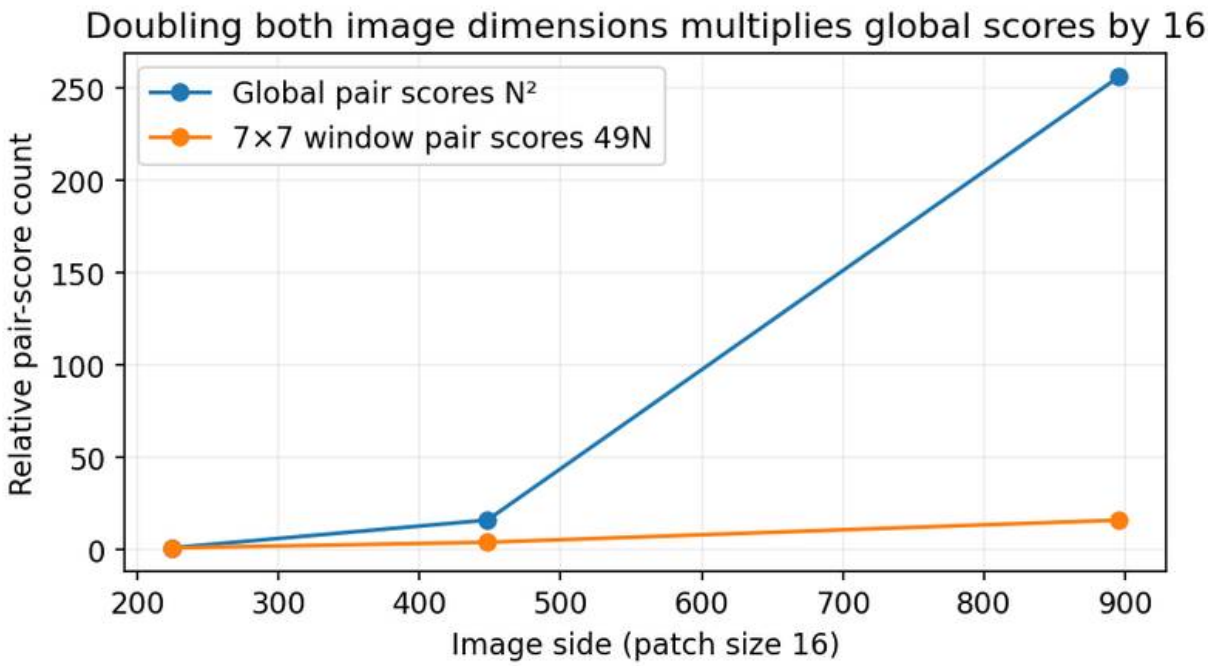


Figure 6 — Pair-score scaling, not a measured runtime. Projections, MLPs, memory traffic and kernel

quality also determine latency.

The main families:

- **Exact but faster.** FlashAttention [6] computes **exact** attention by tiling the computation so that blocks of queries, keys and values stay in fast on-chip memory, and never writes the full $N \times N$ matrix to slow GPU memory; FlashAttention-2 [29] improves the work partitioning further. The arithmetic is still quadratic; the memory is linear and wall-clock time drops substantially. PyTorch's `scaled_dot_product_attention` selects such kernels automatically when it can [7].
- **Restricted connectivity.** Windowed (Swin) or other sparse patterns reduce the number of pairs, at the price of needing several layers for global communication.
- **Fewer tokens.** Token Merging (ToMe) [21] gradually merges similar tokens inside a pretrained ViT using a fast matching step, increasing throughput without retraining. Larger patches, token pruning and pooling stages do the same more bluntly.
- **Smaller models.** Papa et al.'s survey [13] organizes efficient ViTs into compact architectures, pruning, knowledge distillation and quantization, and proposes an "Efficient Error Rate" metric to compare models across parameters, bits, FLOPs and size.

State-space models and hybrids

Plain version. Instead of comparing every pair of tokens, carry a compact state through the sequence, as a recurrent network does (ML 03A). The cost grows linearly with the number of tokens.

A simplified state-space recurrence over a token sequence is

$$h_t = \bar{A}_t\, h_{t-1} + \bar{B}_t\, x_t, \qquad y_t = C_t\, h_t,$$

where h_t is a hidden state of fixed size, and $\bar{A}_t$, $\bar{B}_t$, C_t are (discretized) state-space matrices. In **selective** models such as Mamba, the write/read coefficients and step sizes depend on input x_t; the underlying continuous transition can remain fixed while its discretization varies. Input selection and an affine state update permit a parallel scan [33]. An image has no natural one-dimensional order, so vision variants must choose **scan orders**: Vision Mamba (Vim) [30] scans the patch sequence forward and backward with position embeddings, and VMamba [31] scans along four directions over the 2-D grid. The Visual Mamba survey [8] reviews over 200 papers and identifies scanning as the critical design decision for vision.

A useful counterpoint is MambaOut [9] (CVPR 2025). Its authors argue that Mamba suits tasks with long sequences and causal (autoregressive) structure, and build MambaOut models by removing the state-space component while keeping the gated convolutional blocks. Their models outperform visual Mamba models on ImageNet classification, suggesting that Mamba is unnecessary there, while for detection and segmentation, which involve long token sequences, they find state-space models may still be worth exploring. The broader point is methodological: isolate the mechanism (attention, scan, convolution) from gating, normalization, training recipe and

parameter budget before crediting a result to it.

An image scan is an inductive bias

Mamba [33] supplies an efficient sequence operator; it does not choose an image geometry for us. Vim [30] uses a patch sequence, position embeddings and forward/backward processing. VMamba [31] instead places a four-route selective scan inside a hierarchical visual backbone. Neither label alone specifies receptive-field strength, resolution behavior or runtime. The state dimensions, routes, local operators and downsampling stages all matter.

For a fixed scan order, two adjacent pixels in the image may be many steps apart in the sequence. Conversely, the last pixel of one raster row and the first pixel of the next row become consecutive despite being far apart spatially. A local convolution can supply nearby evidence before the scan carries context farther. Multiple routes offer additional paths, but do not restore every geometric symmetry automatically. Position embeddings can also encode useful location while making transfer to a different grid a separate design problem.

A mathematical way to expose the choice is

$$Y^{(r)} = P_r^{-1} \mathcal{S}_r(P_r X), \qquad Y = \frac{1}{m} \sum_{r=1}^{m} Y^{(r)}.$$

Here X contains flattened image tokens, r indexes one of m routes, P_r permutes tokens into that route, $\mathcal{S}_r$ applies its sequence recurrence, P_r^{-1} restores image coordinates, and Y averages the resulting features. This is a conceptual route-fusion model. A learned vision block can use different projections or fusion rules; averaging is chosen here to keep the experiment interpretable.

CPU experiment: measure directional bias

The next example uses a single scalar state with the same decay on every route. It compares forward/backward raster scans with four raster/column routes, then rotates the input by 90 degrees, processes it and rotates the output back. Everything uses a synthetic 6 × 6 impulse; no weights or images are downloaded. This isolates ordering rather than reproducing a trained Vim or VMamba.

```
import numpy as np

def scan_image(image, four=False):
    ids = np.arange(image.size).reshape(image.shape)
    row, col = ids.ravel(), ids.T.ravel()
    orders = [row, row[::-1]]
    if four:
        orders += [col, col[::-1]]
    outputs = []
    for order in orders:
        h = 0.0
        out = np.zeros(image.size)
```

```
        for i in order:
            h = 0.8 * h + 0.2 * image.ravel()[i]
            out[i] = h
        outputs.append(out.reshape(image.shape))
    return np.mean(outputs, axis=0)

image = np.zeros((6, 6))
image[1, 4] = 1.0
errors = []
for four in [False, True]:
    original = scan_image(image, four=four)
    rotated_back = np.rot90(scan_image(np.rot90(image), four=four), -1)
    error = np.max(np.abs(original - rotated_back))
    errors.append(error)
    print('four routes:', four, 'rotation discrepancy:', round(error, 6))
assert 0.0 < errors[1] < errors[0]
```

The maximum discrepancies are `0.063622` for two routes and `0.026738` for four. More routes reduce directional bias in this example, but the remaining discrepancy is nonzero. Rotating a raster path reverses one traversal axis; that path need not belong to the original four-route collection. Even shared coefficients and averaging therefore do not guarantee rotation equivariance. Learned directional projections, position information and local operations introduce further choices. Test the property you need instead of inferring it from "2-D scan."

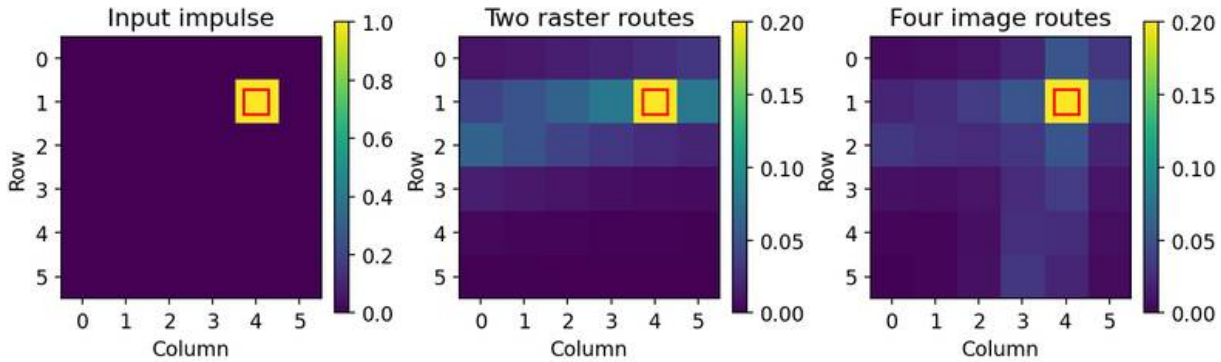


Figure 7 — The red outline marks the input impulse. Two raster routes spread context unevenly; adding column routes changes the influence map. These synthetic responses use the exact recurrence in the example, not learned attention maps or trained VMamba activations.

The recurrence can have a global receptive field because information travels through the entire scan. This says a dependence is possible, not that its influence remains strong at every distance. With scalar decay 0.8, the carried contribution decays by another factor of 0.8 per intervening step. Selective transitions can change this retention, as derived in ML 03A, but training must still learn which information to preserve. Fine boundaries and small objects can be lost through state compression or downsampling even when distant context is available.

What Mamba-2 changes, and how to evaluate it

Mamba-2 [34] reorganizes the selective sequence mixer around structured state-space duality and efficient chunk computation. It changes the transition structure and block parameter projections, rather than making an arbitrary image patch sequence equivalent to a Transformer. The causal structured coefficient matrix has an attention-like interpretation, but standard softmax attention retains different expressive and computational properties. An image application must still specify directions and fusion. The duality therefore answers a computation question; it does not remove the geometry question.

A fair vision comparison should separate three experiments: replace only the mixer inside a shared block; compare complete backbones under a matched training budget; and measure the deployed models on the target hardware. Keep data, augmentation, token resolution and downstream heads controlled. Record state size as well as parameter count, and measure latency and peak memory across image resolutions and batch sizes. A linear token-count term can coexist with expensive projections, route permutations and small-kernel overheads. GPU throughput claims do not follow from the Python loop above.

For dense tasks, inspect errors near boundaries, on small structures and across aspect ratios; for video, distinguish an offline bidirectional encoder from causal live processing. Include rotation/crop sensitivity and longer-grid evaluations rather than only in-distribution classification. MambaOut [9] supplies an important mixer-removal ablation. Its results support a task-dependent comparison, not a theorem that scans are unnecessary for every image problem. Xu et al. [8] organize visual Mamba work by backbones, scan mechanisms and image/video/point-cloud/multimodal applications. Their survey remains a 2024 preprint on the verified record. Its taxonomy is useful for selecting controlled experiments; the number of proposed variants does not establish a universal winner.

Modern view

Survey literature. Khan et al. [3] (*ACM Computing Surveys*, 2022) start from the foundations (self-attention, large-scale pretraining, bidirectional encoding) and then survey Transformers across recognition, generative modeling, multimodal tasks such as visual question answering, video, low-level vision and 3-D analysis, closing with open problems. Han et al. [11] (*IEEE TPAMI*, 2023) organize vision Transformers by task (backbones, high- and mid-level vision such as detection and segmentation, low-level vision, video), and also cover efficient Transformer methods. Liu et al. [12] (*IEEE TNNLS*, 2024) review over a hundred visual Transformers for classification, detection and segmentation on images, point clouds and vision–language data, and suggest three future directions. For efficiency, Papa et al. [13] (*IEEE TPAMI*, 2024) is the most systematic. For state-space vision models, see Xu et al. [8]. Bishop and Bishop [10] provide the textbook derivations.

State of the art, 2023–2026. The plain ViT has become the default **foundation** backbone. ViT-22B [24] (ICML 2023) scaled a ViT to 22 billion parameters, with architectural changes (such

as normalizing queries and keys) needed for stable training at that size. DINOv2 [23] (TMLR 2024) trained ViTs with self-supervision on a large curated image collection and produced general-purpose features that work frozen across classification, segmentation and depth estimation (ML 05). Darcet et al. [22] (ICLR 2024) found that large ViTs repurpose uninformative background patches as internal scratch space, producing high-norm artifact tokens in attention maps, and that adding a few extra **register tokens** removes the artifacts and improves dense predictions. Vision–language foundation models (ML 09) and diffusion Transformers (ML 07) build on the same block. At the same time, hierarchical and hybrid designs remain common for dense, high-resolution tasks, modernized CNNs remain competitive under matched budgets (ML 03), and state-space and linear-attention models continue to be explored for very long token sequences such as video.

Open problems. Quadratic cost at high resolution and long video; the large data and compute appetite of plain ViTs; position encodings that generalize across resolutions and aspect ratios; interpreting attention without over-reading it; and fair, compute-matched comparisons between attention, convolution and state-space mechanisms.

Key takeaways

- Self-attention is a learned, input-dependent weighted average: $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$. The $\sqrt{d_k}$ keeps scores at unit variance so the softmax does not saturate.
- Multi-head attention runs h narrower heads in parallel at about the cost of one; masks (causal, window) and cross-attention control who reads whom.
- Attention is permutation-equivariant, so position must be injected: sinusoidal, learned absolute, relative bias or rotary encodings.
- A pre-norm Transformer block is $X + \mathrm{MHA}(\mathrm{LN}(X))$ then $U + \mathrm{MLP}(\mathrm{LN}(U))$; attention mixes tokens and the MLP transforms each token.
- ViT = patchify (a stride-P convolution) + position embeddings + class token + Transformer encoder. It has weak inductive bias and needs large pretraining or strong recipes (DeiT, AugReg).
- Swin brings locality and hierarchy back with shifted windows and patch merging; hybrids such as CvT, CoAtNet and MobileViT combine convolution and attention.
- FlashAttention makes exact attention faster without changing its quadratic arithmetic; windows, token merging and state-space scans change what is computed. Compare mechanisms only under matched budgets.
- Vision Mamba and VMamba adapt selective state spaces to image sequences. Multiple routes change influence paths; they do not automatically guarantee rotation equivariance or preserve every distant detail.

Exercises

1. **Token budget.** Calculate the number of tokens and attention-score entries per head for a 512×512 image with patches of size 16 and 8. By what factor does the score count grow when the patch size halves?

   Hint

   $N = (512/16)^2 = 1024$ and $(512/8)^2 = 4096$ tokens; $N^2 = 1{,}048{,}576$ and $16{,}777{,}216$ scores. Halving the patch size multiplies N by 4 and the scores by 16.

2. **Scaling check.** Draw random queries and keys with independent standard-normal components for $d_k = 4, 64, 1024$. Measure the variance of $q \cdot k$ with and without division by $\sqrt{d_k}$, and the largest softmax weight over 100 keys in each case. What happens to the softmax without scaling?

   Hint

   The variance of the raw dot product is close to d_k; after scaling it is close to 1. Without scaling, at $d_k = 1024$ the softmax puts nearly all weight on a single key (it becomes almost one-hot), so gradients through the other keys vanish.

3. **Remove position.** In the TinyViT code, set the position embeddings to zero, drop the class token and use mean pooling, then permute the patch tokens before the blocks and compare the logits in evaluation mode. Explain the result, and what it implies for an image classifier without positional information.

   Hint

   The logits agree up to floating-point error, because every block is permutation-equivariant and mean pooling is permutation-invariant. Use dropout off (`eval()`) and permute tokens, not pixels inside a patch. Such a model cannot tell a face from a scrambled face built from the same patches.

4. **Cross-attention shapes.** In a text-to-image model, 256 image tokens of width 320 read 77 text tokens of width 768 with 8 heads of width 40. Give the shapes of W_Q, W_K, W_V, one head's score matrix, and the output.

   Hint

   $W_Q \in \mathbb{R}^{320\times320}$ (8 heads × 40), $W_K, W_V \in \mathbb{R}^{768\times320}$. Each head's scores are 256×77. The output follows the queries: 256×320 after the output projection. Key and value lengths must match each other, not the query length.

5. **Swin arithmetic.** Using the cost formulas, compare global and window attention for a 56×56 token grid with $C = 96$ and $M = 7$. Which term dominates in each case?

Hint

$hw = 3136$. Global: $4 \cdot 3136 \cdot 96^2 \approx 1.16 \times 10^8$ plus $2 \cdot 3136^2 \cdot 96 \approx 1.89 \times 10^9$, so the quadratic term dominates. Window: the same 1.16×10^8 plus $2 \cdot 49 \cdot 3136 \cdot 96 \approx 2.95 \times 10^7$, so the projections dominate. The window version is about 14 times cheaper here.

6. **Fair comparison.** You have 2,000 labeled images and access to pretrained CNN, ViT and visual Mamba backbones. Design an experiment that tells you which backbone to deploy, and list what must be held fixed. Include a mixer-removal ablation and a resolution/rotation test.

Hint

Compare linear probes on frozen features first, then fine-tuning with matched budgets and tuned learning rates per model on a validation split. Record pretraining data, input resolution, parameters, FLOPs and measured latency on the target device. A pretrained ViT against a CNN trained from scratch tests pretraining, not architecture. For the scan backbone, record state size and test whether benefits survive removing the mixer while retaining gating and local convolution. Keep the same preprocessing in the resolution and rotation tests.

References

1. A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017. arXiv
2. A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," ICLR, 2021. arXiv
3. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan and M. Shah, "Transformers in Vision: A Survey," *ACM Computing Surveys*, 2022. arXiv
4. H. Touvron et al., "Training data-efficient image transformers & distillation through attention," ICML, 2021. arXiv
5. Z. Liu et al., "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," ICCV, 2021. arXiv
6. T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," NeurIPS, 2022. arXiv
7. PyTorch, `scaled_dot_product_attention` and `TransformerEncoderLayer` documentation. Attention, Encoder layer
8. R. Xu et al., "Visual Mamba: A Survey and New Outlooks," arXiv:2404.18861, 2024. arXiv
9. W. Yu and X. Wang, "MambaOut: Do We Really Need Mamba for Vision?," CVPR, 2025. Paper
10. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024, ch. 12. Book
11. K. Han et al., "A Survey on Vision Transformer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023. DOI

12. Y. Liu et al., "A Survey of Visual Transformers," *IEEE Transactions on Neural Networks and Learning Systems*, 2024. DOI
13. L. Papa, P. Russo, I. Amerini and L. Zhou, "A Survey on Efficient Vision Transformers: Algorithms, Techniques, and Performance Benchmarking," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. DOI
14. J. L. Ba, J. R. Kiros and G. E. Hinton, "Layer Normalization," arXiv:1607.06450, 2016. arXiv
15. R. Xiong et al., "On Layer Normalization in the Transformer Architecture," ICML, 2020. arXiv
16. J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *Neurocomputing*, 2024. DOI
17. P. Shaw, J. Uszkoreit and A. Vaswani, "Self-Attention with Relative Position Representations," NAACL, 2018. arXiv
18. M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang and A. Dosovitskiy, "Do Vision Transformers See Like Convolutional Neural Networks?," NeurIPS, 2021. arXiv
19. H. Wu et al., "CvT: Introducing Convolutions to Vision Transformers," ICCV, 2021. arXiv
20. Z. Dai, H. Liu, Q. V. Le and M. Tan, "CoAtNet: Marrying Convolution and Attention for All Data Sizes," NeurIPS, 2021. arXiv
21. D. Bolya et al., "Token Merging: Your ViT But Faster," ICLR, 2023. arXiv
22. T. Darcet, M. Oquab, J. Mairal and P. Bojanowski, "Vision Transformers Need Registers," ICLR, 2024. arXiv
23. M. Oquab et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. arXiv
24. M. Dehghani et al., "Scaling Vision Transformers to 22 Billion Parameters," ICML, 2023. arXiv
25. A. Steiner et al., "How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers," *Transactions on Machine Learning Research*, 2022. arXiv
26. D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv:1606.08415, 2016. arXiv
27. H. Touvron, M. Cord and H. Jégou, "DeiT III: Revenge of the ViT," ECCV, 2022. arXiv
28. S. Mehta and M. Rastegari, "MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer," ICLR, 2022. arXiv
29. T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," ICLR, 2024. arXiv
30. L. Zhu et al., "Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model," ICML, 2024. arXiv
31. Y. Liu et al., "VMamba: Visual State Space Model," NeurIPS, 2024. arXiv
32. Z. Liu et al., "Swin Transformer V2: Scaling Up Capacity and Resolution," CVPR, 2022. arXiv
33. A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," COLM, 2024. Author version
34. T. Dao and A. Gu, "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," ICML, PMLR 235, 2024. Paper

ML 05 • Representation Learning and Self-Supervision

What a representation is, why distributed features transfer, and how SimCLR, MoCo, BYOL, SimSiam, DINO, MAE, JEPA and CLIP learn them without labels, with honest linear-probe and fine-tuning evaluation.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-05-representation-learning/

What you'll learn

- What a **representation** is, why **distributed** representations generalize better than one-hot codes, and which invariances a good representation should (and should not) have.
- How **transfer learning** works: pretrain an encoder once, then reuse it through k-NN, a linear probe or fine-tuning.
- How the main **self-supervised learning** families build a training signal from unlabeled data: contrastive (SimCLR, MoCo, with the InfoNCE loss), teacher–student without negatives (BYOL, SimSiam, DINO), redundancy reduction (Barlow Twins, VICReg), masked modeling (BEiT, MAE) and feature prediction (I-JEPA, V-JEPA 2).
- Why "make two views agree" can **collapse**, and how alignment and uniformity make that precise.
- How **CLIP** uses captions as supervision and enables zero-shot classification.
- How to evaluate a representation honestly, including why fine-tuning can underperform a linear probe and what LP-FT does about it.

The big picture

Suppose you have a million unlabeled images but only 500 labeled examples for the task you care about. Training a large network from scratch on those 500 examples ignores almost all the available data. **Representation learning** splits the job in two. First, learn an encoder $h = f_\theta(x)$ from a broad signal that does not need your labels. Second, train a small task-specific head on top of h using the few labels you have.

> **Explain like I'm five**
>
> Before a child learns the names of every bird, they already notice wings, beaks, colors and how things move. Later, a parent points at three robins and two crows, and that is enough: the child can tell robins from crows from then on. The "noticing" was learned for free, just by looking at the world. A

representation is that reusable way of noticing things, and self-supervised learning is the "just looking" part.

Why it matters: almost every modern vision system starts from a pretrained encoder. Image classifiers, detectors, segmentation models, image–text models and the image encoders inside generative models are built on representations learned this way. The quality of that starting point often matters more than the details of the task head.

This chapter starts with what a representation is and why transfer works, then walks through the self-supervised objectives that dominate practice, and ends with how to measure whether a representation is any good. It assumes you know neural networks (ML 02), CNNs (ML 03) and vision transformers (ML 04). Linear factor models and autoencoders, the classical route to representations, have their own chapter: ML 05A.

What is a representation?

Plain version. A representation is the list of numbers a model uses to describe an input. A good one makes the things you care about easy to read off, and hides the things you do not care about.

Precise version. An encoder is a function $f_\theta : \mathcal{X} \to \mathbb{R}^D$ from inputs x (here, images) to feature vectors $h = f_\theta(x)$, where θ are the learnable parameters and D is the feature dimension. A task head g maps h to a prediction, so the full model is $g \circ f_\theta$. We call h a good representation for a task when a *simple* head, such as a linear classifier, can solve the task from h using *few* labeled examples. Bengio, Courville and Vincent's review [1] is the classic statement of this view; it lists the "priors" that make representations useful, such as smoothness, multiple explanatory factors, a hierarchy of increasingly abstract features, and shared factors across tasks.

Local versus distributed representations

The simplest representation of a category is **one-hot**: one unit per concept, with exactly one unit on. It is easy to read but it cannot generalize. Knowing about "red car" says nothing about "red truck", because they share no active units. A **distributed** representation describes each input with many features that are each shared by many inputs: "red", "blue", "car", "truck". Now "red truck" reuses the "red" unit learned from red cars.

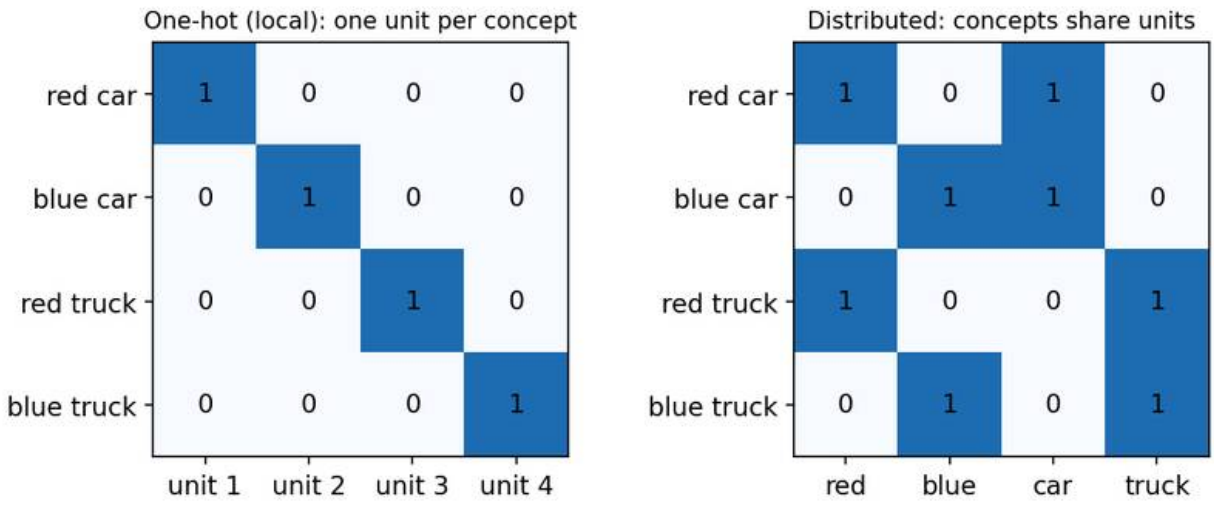


Figure 1 — Local (one-hot) versus distributed codes for four concepts. With distributed codes, a new combination such as a red truck reuses features learned from other examples.

The counting argument is striking. With D binary features, a one-hot code distinguishes D concepts, while a distributed code can in principle distinguish 2^D configurations [1]. Deep networks learn distributed codes automatically: each hidden unit responds to many inputs, and each input activates many units. Most of the practical value of representation learning comes from this sharing.

Invariance and equivariance

A useful representation keeps differences relevant to the task and suppresses differences that distract from it.

- A feature is **invariant** to a transformation T if $f(T(x)) = f(x)$: shifting a cat by two pixels should not change "this is a cat".
- A feature is **equivariant** if it changes in a predictable way, $f(T(x)) = T'(f(x))$: shifting the cat should shift its segmentation mask by the same amount.

"Useful" always depends on the downstream task. Color invariance helps object recognition but hurts fruit-ripeness estimation. Rotation invariance helps for textures but confuses 6 and 9. Dense prediction (segmentation, depth) needs spatially equivariant patch features, not a single invariant vector per image. We will see that self-supervised objectives choose their invariances mostly through the **data augmentations** they use, so this choice is a modeling decision, not a detail.

Learning distances: metric learning

PCA, introduced in ML 01 and developed in ML 05A, keeps the directions of highest variance. But high variance may come from lighting rather than object identity. Supervised learning shapes features around labels; **metric learning** shapes distances around pairs that should match. For an anchor a, a positive p (same identity) and a negative n (different identity), the triplet loss is

$$\mathcal{L}_{\text{triplet}} = \max\left(0,\ \|h_a - h_p\|^2 - \|h_a - h_n\|^2 + m\right),$$

where h_a, h_p, h_n are the three feature vectors and $m > 0$ is a margin. The loss is zero once the positive is closer than the negative by at least m. An easy negative gives zero loss and teaches nothing, so choosing informative ("hard") negatives matters. Contrastive self-supervised learning, below, is metric learning in which the positives are created by augmentation instead of by labels.

Transfer learning: pretrain once, reuse many times

Plain version. Train a network on a big task, keep its feature extractor, and reuse it for a small task. You can freeze the features and only train a new head, or let the whole network keep learning.

Precise version. Pretraining finds encoder parameters $\theta^\star$ on a source dataset or objective. Transfer to a target task with labeled set $\{(x_i, y_i)\}_{i=1}^n$ uses one of three protocols:

1. **k-nearest neighbors (k-NN):** no training. Classify a test feature $f_{\theta^\star}(x)$ by the labels of its k most similar training features.
2. **Linear probe:** keep $\theta^\star$ frozen and fit a linear classifier $\hat{y} = \text{softmax}(Wh + b)$ on $h = f_{\theta^\star}(x)$, where W and b are the only trained parameters.
3. **Fine-tuning:** start from $\theta^\star$ and update the encoder together with the head, usually with a small learning rate.

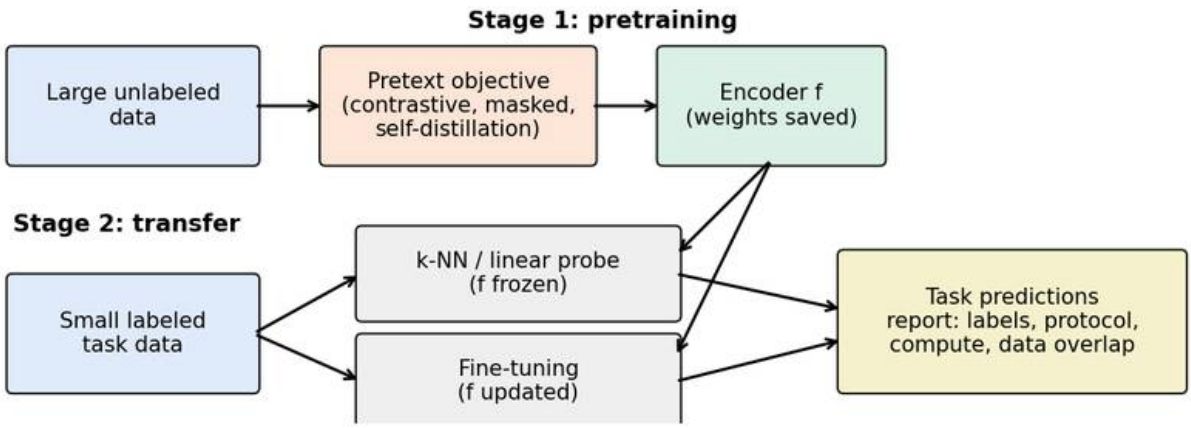


Figure 2 — The two-stage pattern of representation learning. Pretraining is expensive and done once; transfer is cheap and done per task.

Why does transfer work at all? Yosinski et al. [2] measured it layer by layer on ImageNet-trained CNNs. First-layer features (edge and color detectors much like Gabor filters) were general: they transferred well between tasks. Features became progressively more specific to the source task in higher layers, and transferability decreased as the source and target tasks became more different. They also found that initializing from transferred features and fine-tuning improved generalization even on the target task, compared with training from random initialization.

For years, the default pretraining source was supervised ImageNet classification. Labels are expensive, though, and a label like "dog" tells the network nothing about the dog's pose, the grass or the lighting. Self-supervised learning removes the label bottleneck by making the data supervise itself.

Self-supervised learning: let the data make the labels

Plain version. Invent a task whose answer you can compute from the data itself: "are these two crops from the same photo?", "what was under this mask?", "which caption goes with this image?". Solving such a **pretext task** forces the network to learn useful features.

Precise version. A self-supervised objective is a loss $\mathcal{L}(\theta; x)$ computed from unlabeled inputs alone (or from naturally paired data such as image–caption pairs). The surveys by Liu et al. [3] and Gui et al. [4] and the practical "cookbook" by Balestriero et al. [5] organize the field into the families summarized here:

Family	Training signal	Representative methods	Main risk
Contrastive	Pull positive pairs together, push other samples apart	SimCLR, MoCo	Needs many negatives; false negatives
Self-distillation (no negatives)	Student matches a teacher on another view	BYOL, SimSiam, DINO	Collapse without asymmetry or centering
Redundancy reduction	Agreement plus decorrelated, non-constant feature dimensions	Barlow Twins, VICReg	Extra hyperparameters for the regularizers
Masked modeling	Predict hidden pixels or tokens	BEiT, MAE	Features may focus on low-level detail
Feature prediction	Predict features of hidden regions	I-JEPA, V-JEPA 2	Needs anti-collapse design like distillation
Language supervision	Match images with their captions	CLIP, SigLIP	Inherits web-data biases and noise

The families are learning objectives, not architectures: each can train a CNN or a transformer. Modern systems often combine several of them. The sections below take them in turn.

Contrastive learning and the InfoNCE loss

Plain version. Make two augmented versions of the same image agree, and make them disagree with every other image in the batch. It is a multiple-choice question: "out of these N candidates, which one is my partner?"

InfoNCE

Van den Oord, Li and Vinyals introduced the **InfoNCE** loss in Contrastive Predictive Coding (CPC) [6]. Take a query q, one positive key k^+ that "belongs" to it, and $N-1$ negative keys k_1^-, $\dots, k_{N-1}^-$. With a similarity score $s(\cdot,\cdot)$ and a temperature $\tau > 0$,

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp\left(s(q,k^+)/\tau\right)}{\exp\left(s(q,k^+)/\tau\right) + \sum_{j=1}^{N-1} \exp\left(s(q,k_j^-)/\tau\right)}.$$

This is exactly softmax cross-entropy for an N-way classification problem whose correct class is the positive. Two consequences follow.

- **A lower bound on mutual information.** CPC shows that $I(q;k^+) \geq \log N - \mathcal{L}_{\text{InfoNCE}}$, where I is mutual information. Minimizing the loss raises this lower bound, and more negatives (larger N) allow a higher bound. In practice this bound is loose, and the quality of the features depends heavily on the encoder and augmentations, so read it as motivation rather than as the explanation of why the features are good.
- **The temperature controls hardness.** Dividing by a small τ sharpens the softmax, so the loss concentrates on the negatives most similar to the query (the hard negatives).

SimCLR

SimCLR [7] is InfoNCE in its simplest form. Take a batch of B images and produce two augmented views of each (random crop and resize, color distortion, blur), giving $2B$ views. A backbone f maps each view to h, a small MLP **projection head** g maps h to $z = g(h)$, and the similarity is cosine similarity $s(z_i, z_j) = z_i^\top z_j / (\|z_i\|\|z_j\|)$. For view i with partner j,

$$\ell_{i,j} = -\log \frac{\exp(z_i^\top z_j/\tau)}{\sum_{k \neq i} \exp(z_i^\top z_k/\tau)},$$

where the z are ℓ_2-normalized and the sum runs over all $2B-1$ other views. Every other image in the batch is a negative, so no memory bank is needed, but the method benefits from large batches. The total loss averages $\ell_{i,j}$ over both orderings of every positive pair; the paper calls this the NT-Xent loss (normalized temperature-scaled cross-entropy).

Three findings from the SimCLR paper are worth remembering [7]. The *composition* of augmentations matters a lot, with random cropping plus color distortion being especially important. The projection head helps: features *before* the head (h) transfer better than those after it (z), so the head is discarded after pretraining. And contrastive learning benefits from larger batches and longer training than supervised learning.

Code: check the loss and its collapse value

This standalone PyTorch example uses synthetic paired vectors to verify the loss, the self-mask and the gradients. It does not train an encoder.

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
B, D = 8, 16
base = torch.randn(B, D)
views = torch.cat([base + .1*torch.randn_like(base),
                   base + .1*torch.randn_like(base)]).requires_grad_()
z = F.normalize(views, dim=-1)
logits = z @ z.T / .2
self_mask = torch.eye(2*B, dtype=torch.bool)
logits = logits.masked_fill(self_mask, float("-inf"))
positive = (torch.arange(2*B) + B) % (2*B)
loss = F.cross_entropy(logits, positive)
loss.backward()
assert torch.isfinite(views.grad).all()
print("paired loss:", round(loss.item(), 4))

collapsed = F.normalize(torch.ones(2*B, D), dim=-1)
scores = (collapsed @ collapsed.T / .2).masked_fill(self_mask, float("-inf"))
collapsed_loss = F.cross_entropy(scores, positive)
print("collapsed loss:", round(collapsed_loss.item(), 4))
print("expected:", round(torch.log(torch.tensor(2*B-1.)).item(), 4))
```

It prints a paired loss of 0.1423 and a collapsed loss of 2.7081, equal to $\log 15$. If every representation is identical, the positive gets probability $1/(2B-1)$ and the loss is $\log(2B-1)$, the worst value a uniform guess can get. Contrastive discrimination therefore makes complete collapse a bad solution, although optimization and negative selection can still be difficult.

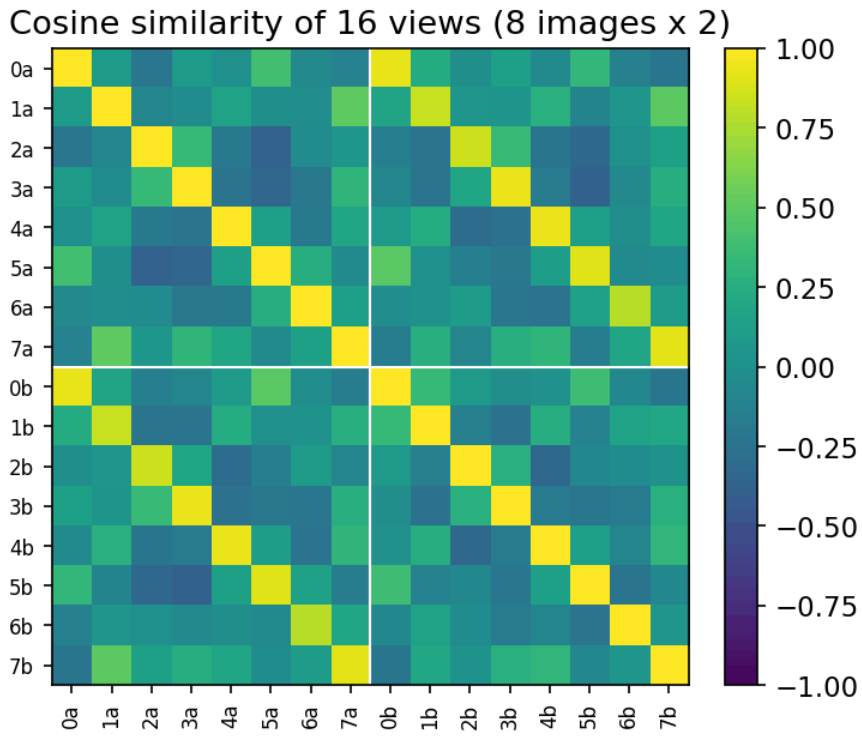


Figure 3 — Cosine similarities between 16 views (8 digit images, two augmentations each) after training the small SimCLR encoder used later in this chapter. The bright off-diagonal stripes are the positive pairs; InfoNCE pushes them up and everything else down.

MoCo: a queue of negatives and a momentum encoder

InfoNCE prefers many negatives, but SimCLR gets them only from the current batch. **Momentum Contrast (MoCo)** [8] decouples the number of negatives from the batch size with two ideas.

1. **A queue.** Keys encoded in recent batches are stored in a first-in, first-out queue of length K and reused as negatives. The query sees K negatives no matter how small the batch is.
2. **A momentum encoder.** Keys in the queue were produced by slightly older networks. If the key encoder changed quickly, old and new keys would be inconsistent. MoCo therefore computes keys with a separate encoder whose parameters θ_k follow the query encoder's parameters θ_q slowly:

$$\theta_k \leftarrow m\,\theta_k + (1-m)\,\theta_q,$$

where $m \in [0,1)$ is the momentum, close to 1 (the paper's default is 0.999). Only θ_q receives gradients. This **exponential moving average (EMA)** teacher idea reappears in BYOL, DINO and

JEPA.

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
B, D, K = 8, 16, 32          # batch, embedding size, queue length

# --- MoCo-style InfoNCE: one positive key per query, negatives from a queue
q = F.normalize(torch.randn(B, D), dim=1)                 # queries (online encoder)
k = F.normalize(q + 0.1 * torch.randn(B, D), dim=1)       # keys (momentum encoder)
queue = F.normalize(torch.randn(K, D), dim=1)             # keys from past batches
l_pos = (q * k).sum(1, keepdim=True)                      # B x 1
l_neg = q @ queue.T                                       # B x K
logits = torch.cat([l_pos, l_neg], dim=1) / 0.07
loss_moco = F.cross_entropy(logits, torch.zeros(B, dtype=torch.long))
queue = torch.cat([k, queue])[:K]                         # enqueue new, drop oldest
print("MoCo logits:", tuple(logits.shape), "loss:", round(loss_moco.item(), 3))

# momentum (EMA) update of the key encoder's parameters
theta_q, theta_k, m = torch.randn(5), torch.zeros(5), 0.999
theta_k = m * theta_k + (1 - m) * theta_q
print("key params moved by:", round((theta_k.norm() / theta_q.norm()).item(), 4))
```

Each query gets a $1 + K$-way classification problem whose correct answer is always column 0, hence the all-zero targets. The output shows logits of shape `(8, 33)`, and the EMA step moves the key encoder only 0.1% of the way toward the query encoder (from zero, after one step). In a real implementation, q and k come from the two encoders applied to two augmentations of the same images.

Alignment and uniformity

Wang and Isola [9] gave a clean geometric reading of what contrastive losses do on the unit sphere. With normalized features, the loss asymptotically optimizes two properties:

$$\mathcal{L}_{\text{align}} = \mathbb{E}_{(x,x^+)} \|f(x) - f(x^+)\|_2^2, \qquad \mathcal{L}_{\text{unif}} = \log \mathbb{E}_{x,y}\, e^{-t\|f(x)-f(y)\|_2^2},$$

where (x, x^+) is a positive pair, x, y are independent samples, and $t > 0$ (commonly $t = 2$).

Alignment is low when positive pairs map to nearby points. **Uniformity** is low when features spread out over the sphere, so they keep as much information about the input as possible. A collapsed encoder has perfect alignment (0) and the worst uniformity (0); a useful encoder has both small.

```
import torch
from torch.nn import functional as F

def alignment(z1, z2):           # mean squared distance between positive pairs
```

```
    return (z1 - z2).square().sum(1).mean()

def uniformity(z, t=2.0):         # log of mean Gaussian potential over all pairs
    return torch.pdist(z).square().mul(-t).exp().mean().log()

torch.manual_seed(0)
n, D = 512, 16
collapsed = F.normalize(torch.ones(n, D), dim=1)
spread = F.normalize(torch.randn(n, D), dim=1)
noisy_view = F.normalize(spread + 0.1 * torch.randn(n, D), dim=1)
print("collapsed: align", alignment(collapsed, collapsed).item(),
      "uniform", round(uniformity(collapsed).item(), 3))
print("spread:    align", round(alignment(spread, noisy_view).item(), 3),
      "uniform", round(uniformity(spread).item(), 3))
```

The collapsed set scores alignment 0.0 and uniformity 0.0; the spread set scores about 0.138 and −3.51. Figure 4 tracks the same two quantities while training the digit encoder below. At step 0 the random network maps every view to almost the same point, so alignment is near zero only because everything has collapsed. Training first spreads the features out (uniformity drops sharply), then gradually pulls positives back together.

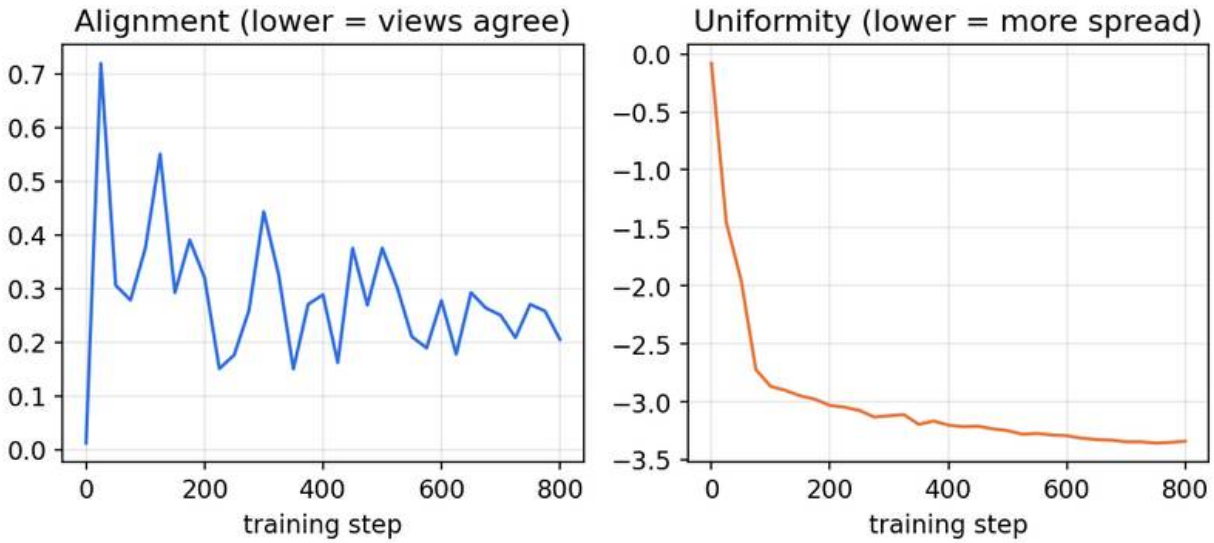


Figure 4 — Alignment and uniformity of the projection features during SimCLR training on 8×8 digits. Low alignment alone is meaningless (a constant encoder has it); the useful regime is low alignment together with low uniformity.

A worked example: SimCLR on 8×8 digits

Plain version. Train a tiny SimCLR encoder on scikit-learn's digits without using any labels, then check whether its features make a nearest-neighbor classifier more robust.

The augmentation here is a random shift of up to one pixel, a random brightness change and pixel noise. We therefore expect the learned features to be roughly invariant to small shifts, something raw pixels are not. The evaluation uses the same labeled training set for every feature type and tests on clean digits and on digits shifted by up to one pixel.

```
import copy
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(1)
d = load_digits()
Xtr, Xte, ytr, yte = train_test_split(d.data / 16., d.target, test_size=.3,
                                      random_state=0, stratify=d.target)
Xtr_t = torch.tensor(Xtr, dtype=torch.float32)

def augment(x):                       # the augmentations define the invariances
    img = F.pad(x.view(-1, 1, 8, 8), (1, 1, 1, 1))
    i, j = np.random.randint(0, 3, size=2)              # shift by up to 1 px
    img = img[:, :, i:i + 8, j:j + 8]
    img = img * (0.7 + 0.6 * torch.rand(len(img), 1, 1, 1))  # brightness
    return (img + 0.1 * torch.randn_like(img)).reshape(len(x), 64)  # noise

backbone = nn.Sequential(nn.Unflatten(1, (1, 8, 8)),
                         nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(),
                         nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
                         nn.AdaptiveAvgPool2d(2), nn.Flatten())  # h: 256-d
random_backbone = copy.deepcopy(backbone)                    # baseline
head = nn.Sequential(nn.ReLU(), nn.Linear(256, 64))          # projection g
opt = torch.optim.Adam([*backbone.parameters(), *head.parameters()], lr=1e-3)

def nt_xent(z1, z2, tau=0.2):
    z = F.normalize(torch.cat([z1, z2]), dim=-1)
    n = len(z1)
    logits = (z @ z.T / tau).fill_diagonal_(float("-inf"))
    return F.cross_entropy(logits, (torch.arange(2 * n) + n) % (2 * n))

for step in range(800):                                  # about a minute on CPU
    x = Xtr_t[torch.randint(len(Xtr_t), (256,))]
    loss = nt_xent(head(backbone(augment(x))), head(backbone(augment(x))))
    opt.zero_grad(); loss.backward(); opt.step()

def shift(x, dy, dx):                    # move one 8x8 image, fill with zeros
    img = np.zeros((10, 10)); img[1:9, 1:9] = x.reshape(8, 8)
    return img[1 - dy:9 - dy, 1 - dx:9 - dx].ravel()

moves = np.random.default_rng(0).integers(-1, 2, size=(len(Xte), 2))
Xte_shift = np.stack([shift(x, *m) for x, m in zip(Xte, moves)])
```

```
enc = lambda net: (lambda A: net(torch.tensor(A, dtype=torch.float32)).detach().numpy())
pca = PCA(32, random_state=0).fit(Xtr)
for name, f in [("raw pixels", lambda A: A), ("PCA-32", pca.transform),
                ("random CNN", enc(random_backbone)), ("SimCLR CNN", enc(backbone))]:
    knn = KNeighborsClassifier(5, metric="cosine").fit(f(Xtr), ytr)
    print(f"{name:11s} clean {knn.score(f(Xte), yte):.3f}  "
          f"shifted {knn.score(f(Xte_shift), yte):.3f}")
```

On our machine this prints:

```
raw pixels  clean 0.978  shifted 0.524
PCA-32      clean 0.978  shifted 0.507
random CNN  clean 0.974  shifted 0.524
SimCLR CNN  clean 0.987  shifted 0.919
```

Exact numbers vary slightly across PyTorch versions and hardware. The pattern is the lesson. On clean digits, raw pixels are already excellent: this dataset is easy, and a fancy encoder adds little. On shifted digits, raw pixels, PCA and an untrained CNN of the same architecture all drop to about 50%, while the self-supervised features keep most of their accuracy. The network learned the invariance we asked for through the augmentations, and nothing more: had we tested on rotated digits, which the augmentations never showed, we should not expect the same robustness.

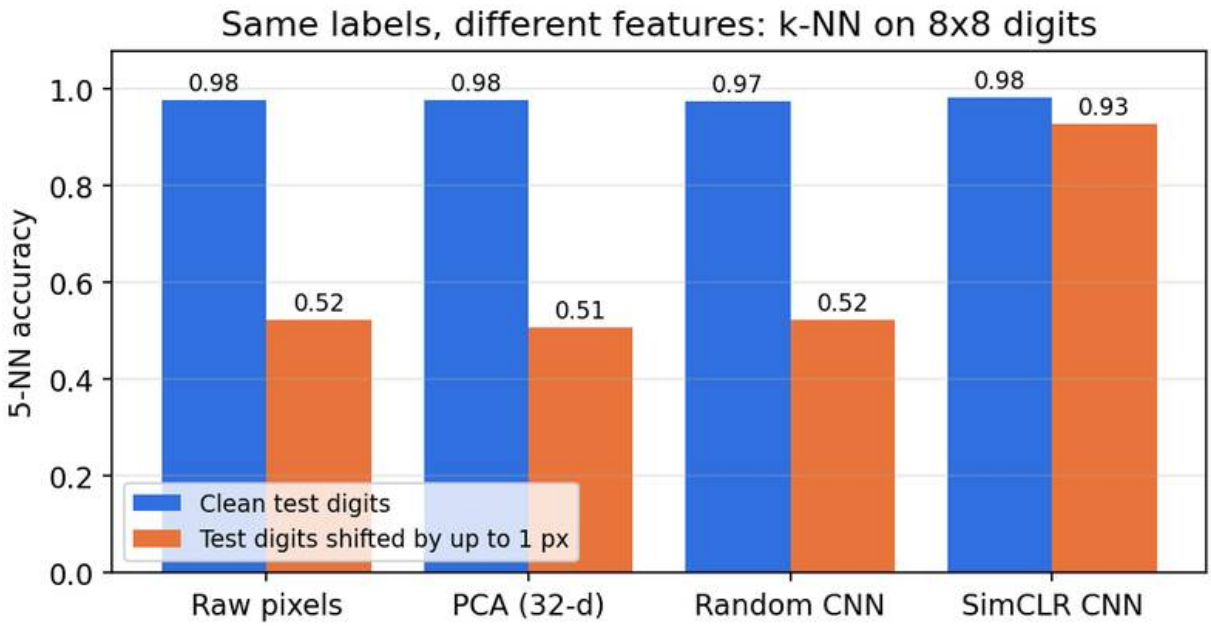


Figure 5 — Same labels, same classifier, different features. The figure script uses a separate random stream, so its numbers differ from the printed output by up to 0.01. The random CNN rules out the explanation that convolution alone produces the robustness.

Without negatives: why matching alone can collapse

Plain version. If the only instruction is "make the two outputs agree", returning the same constant for every image solves it perfectly. Methods without negatives need some other mechanism to stop this.

Minimizing $\|f_\theta(v_1) - f_\theta(v_2)\|^2$ over two views v_1, v_2 has the trivial solution $f_\theta(x) = c$ for a constant c. Agreement alone does not require the features to carry any information. **Complete collapse** maps everything to one point; **dimensional collapse** is subtler: the features vary, but only along a few directions, wasting most of the D dimensions.

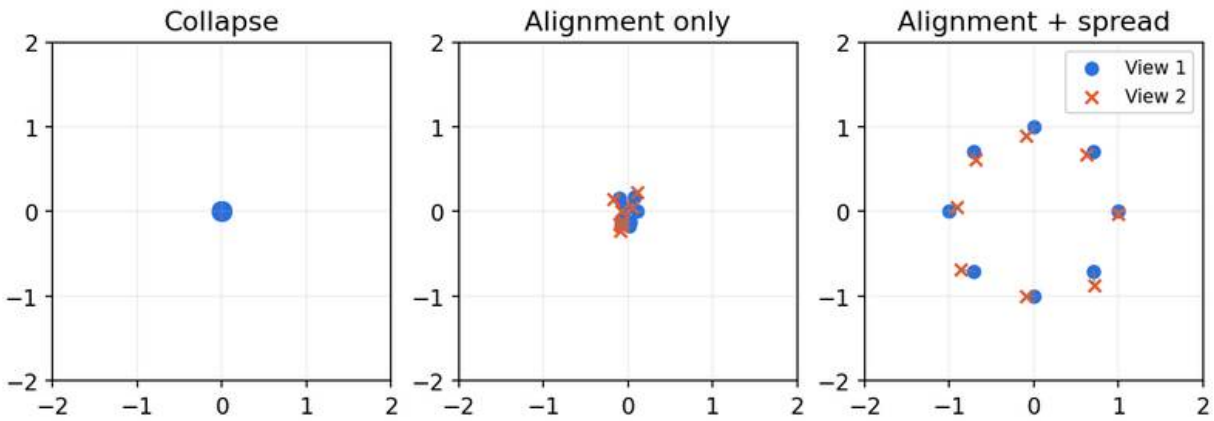


Figure 6 — Schematic of collapse, alignment without spread, and alignment with spread. The points are constructed illustrations, not embeddings learned by a published model.

BYOL: an online network chases a slow target

BYOL [10] uses two networks. The **online** network (encoder, projector and an extra **predictor** q_ϕ) is trained to predict the **target** network's projection of a different view of the same image. The target is not trained by gradients; it is an EMA of the online weights, as in MoCo:

$$\theta_{\text{target}} \leftarrow m\,\theta_{\text{target}} + (1-m)\,\theta_{\text{online}}.$$

The loss is the squared distance between the normalized prediction and the normalized target projection, symmetrized over the two views. The target output is treated as a constant (**stop-gradient**). There are no negatives, yet BYOL does not collapse in practice. The asymmetry (predictor on one side, stop-gradient and EMA on the other) and the full training recipe matter; EMA alone does not prove collapse impossible.

SimSiam: stop-gradient is the key ingredient

Chen and He's SimSiam [11] stripped BYOL down further: one shared encoder for both views, a predictor h on one branch, **no** momentum encoder, **no** negatives and no large batches. With $p_1 = h(f(v_1))$ and $z_2 = f(v_2)$,

$$\mathcal{L} = \tfrac{1}{2}\,\mathcal{D}(p_1, \mathrm{sg}(z_2)) + \tfrac{1}{2}\,\mathcal{D}(p_2, \mathrm{sg}(z_1)), \qquad \mathcal{D}(p, z) = -\frac{p^\top z}{\|p\|\,\|z\|},$$

where sg is stop-gradient and $\mathcal{D}$ is negative cosine similarity. The paper's experiments show that removing the stop-gradient makes the model collapse quickly, while the predictor is also needed. The authors interpret the method as an alternating, expectation-maximization-like optimization. The practical lesson: very small architectural asymmetries decide whether these methods learn or collapse.

DINO: self-distillation with centering and sharpening

DINO [12] turns the agreement problem into classification with soft labels. Teacher and student both output a probability vector over K "prototype" dimensions via a temperature-scaled softmax. The student, seeing a view x', is trained to match the teacher's distribution on another view x:

$$\mathcal{L} = -\sum_{k=1}^{K} P_t(x)_k \,\log P_s(x')_k, \qquad P_t(x) = \mathrm{softmax}\Big(\frac{g_t(x) - c}{\tau_t}\Big),$$

where g_t is the teacher network (an EMA of the student), c is a running mean of teacher outputs (**centering**) and τ_t is a low teacher temperature (**sharpening**). Centering stops one dimension from dominating; sharpening stops the output from becoming uniform. Each alone collapses in a different way; together they balance. DINO also uses **multi-crop**: the teacher sees only large global crops, while the student also sees small local crops, so the student must infer global content from local evidence. With vision transformers, DINO's attention maps were notable for highlighting object boundaries without any segmentation labels.

Barlow Twins and VICReg: regularize the statistics directly

A different route is to forbid collapse explicitly in the loss. **Barlow Twins** [13] computes the $D \times D$ cross-correlation matrix C between the (batch-standardized) embeddings of the two views and pushes it toward the identity:

$$\mathcal{L}_{\mathrm{BT}} = \sum_i (1 - C_{ii})^2 + \lambda \sum_i \sum_{j \neq i} C_{ij}^2,$$

where the first term makes each dimension agree across views and the second, weighted by $\lambda > 0$, decorrelates different dimensions so they do not carry redundant information. **VICReg** [14]

splits the same idea into three named terms: **invariance** (mean squared distance between the two embeddings), **variance** (a hinge loss keeping the standard deviation of every dimension above a threshold, which directly forbids constant outputs) and **covariance** (penalizing off-diagonal covariance entries). "No negatives" does not mean "no anti-collapse design"; these methods simply put the design in the loss instead of in the architecture.

Masked modeling: learn by filling gaps

Plain version. Hide part of the input and predict what is missing. This is how BERT learned language, and it works for images too, once you choose the right amount of hiding.

BEiT [15] brought BERT-style pretraining to vision transformers. An image is split into patches; some are masked, and the network predicts discrete **visual tokens** for the masked positions, where the tokens come from a separately trained image tokenizer.

MAE [16] predicts raw pixels instead. It masks a large fraction of the patches, sends only the visible ones through the encoder, and reconstructs the missing pixels with a lightweight decoder. With the set of masked patch indices $\mathcal{M}$,

$$\mathcal{L}_{\text{mask}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{x}_i - x_i\|^2,$$

where x_i is the true pixel vector of patch i and $\hat{x}_i$ is the prediction. MAE's design choices are deliberate. A high masking ratio (75% by default) makes the task hard enough that the network cannot simply interpolate from neighbors. The encoder only processes visible patches, which cuts pretraining cost. Normalizing each target patch by its own mean and variance improves the features. After pretraining, the decoder is discarded.

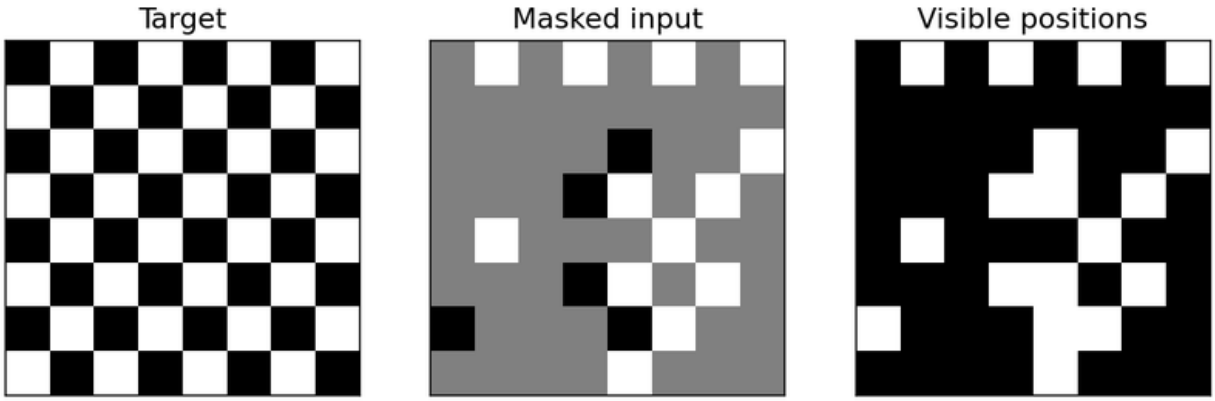


Figure 7 — Masked prediction creates its own targets from the input. The checkerboard illustrates the masking operation, not semantic understanding.

Reconstruction is not the same as generation. A model that predicts masked pixels from visible ones is not trained to sample complete images from scratch. Low reconstruction loss does not guarantee the best semantic features either: background texture can dominate pixel error. Masked-modeling features are typically weaker than contrastive or distillation features under a linear probe but strong after fine-tuning, which is one reason the evaluation protocol must be stated.

Predicting features instead of pixels: JEPA

Plain version. Instead of painting in every uncertain pixel, predict a *description* of the hidden region.

I-JEPA [17] (Image Joint-Embedding Predictive Architecture) encodes a visible **context** block and predicts the encoder features of several **target** blocks from it. The target encoder is an EMA of the context encoder, and a predictor receives the context features plus the positions of the targets. A simplified objective is

$$\mathcal{L}_{\text{pred}} = \left\| g_\phi(f_\theta(x_{\text{context}}), \text{positions}) - \text{sg}\left(f_{\bar{\theta}}(x_{\text{target}})\right) \right\|^2,$$

where f_θ is the context encoder, $f_{\bar{\theta}}$ the EMA target encoder, g_ϕ the predictor and sg stop-gradient. The formula summarizes the idea; the masking strategy (large, semantic-scale target blocks) and the architecture are part of the method, not optional details.

V-JEPA 2 [18] extends feature prediction to video at scale. Its report separates action-free pretraining on video from an action-conditioned model, trained afterward on a small amount of robot data, that is used for planning. Predicting future latent states, recognizing actions and generating photorealistic frames are different capabilities, and a "world model" claim should say which one it makes.

Language as supervision: CLIP

Plain version. The internet is full of images with captions. Train an image encoder and a text encoder so that each image lands next to its own caption. Afterwards you can classify images by writing the class names as sentences.

CLIP [19] trains on image–text pairs collected from the web. In a batch of N pairs, the image encoder gives normalized embeddings $u_1, \dots, u_N$ and the text encoder gives $v_1, \dots, v_N$. The $N \times N$ similarity matrix $S_{ij} = u_i^\top v_j / \tau$, with a learned temperature τ, has the correct pairs on its diagonal. The loss is the average of two InfoNCE losses, one over rows (which caption fits this image?) and one over columns (which image fits this caption?):

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\Big[\frac{1}{N}\sum_i -\log \frac{e^{S_{ii}}}{\sum_j e^{S_{ij}}} + \frac{1}{N}\sum_j -\log \frac{e^{S_{jj}}}{\sum_i e^{S_{ij}}}\Big].$$

Zero-shot classification follows directly. Embed one prompt per class, such as "a photo of a dog", and assign an image to the class whose prompt embedding is most similar. No task-specific labels are needed, although prompt wording affects accuracy.

SigLIP [20] replaces the softmax with an independent **sigmoid** loss on every image–text pair: each entry of the matrix becomes a yes/no question, "do these match?". This removes the need to normalize over the whole batch, which simplifies large-scale distributed training.

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
D = 16

# --- CLIP-style symmetric loss on N matched image-text pairs
N = 6
img = F.normalize(torch.randn(N, D), dim=1)
txt = F.normalize(img + 0.3 * torch.randn(N, D), dim=1)   # caption i matches image i
logit_scale = torch.tensor(1 / 0.07)
sim = logit_scale * img @ txt.T                           # N x N, diagonal = matches
labels = torch.arange(N)
loss_clip = (F.cross_entropy(sim, labels) + F.cross_entropy(sim.T, labels)) / 2
print("CLIP loss:", round(loss_clip.item(), 3))

# zero-shot classification: embed one prompt per class, pick the most similar
class_prompts = txt[:3]                                   # stand-ins for "a photo of a
 ↪  {class}"
pred = (img @ class_prompts.T).argmax(1)
print("zero-shot predictions for images 0-2:", pred[:3].tolist())

# SigLIP-style alternative: an independent yes/no decision for every pair
t, b = 10.0, -10.0                                        # learnable in practice
signs = 2 * torch.eye(N) - 1                              # +1 on matches, -1 elsewhere
loss_sig = -F.logsigmoid(signs * (t * img @ txt.T + b)).sum() / N
print("sigmoid loss:", round(loss_sig.item(), 3))
```

The synthetic captions are noisy copies of the image embeddings, so the CLIP loss is small (0.054) and the zero-shot step recovers the right "classes" `[0, 1, 2]`. In real CLIP the two encoders are a vision transformer or CNN and a text transformer, and the hard part is the data: what the captions describe, and what they leave out, determines what the representation knows. The sigmoid loss value (3.473) is not comparable to the softmax loss; it uses different units and a bias term b.

DINOv2 and DINOv3: features as a product

Plain version. With enough curated data and compute, pretraining can produce an encoder that you freeze and reuse for many different tasks, from classification to depth estimation.

DINOv2 [21] combines image-level self-distillation (as in DINO) with a patch-level masked objective, and pays close attention to **data curation**: building a large, diverse, deduplicated pretraining set from uncurated images by retrieval against curated ones. The result is a family of frozen encoders whose global and patch features work well with simple heads. DINOv3 [22], released in 2025, scales this further and adds **Gram anchoring** to counter the degradation of dense (patch) features during long training. Rather than only matching each feature independently, Gram anchoring constrains the matrix of similarities among patch features toward that of an earlier model. The report also distills a very large teacher into smaller models for deployment.

The practical question is whether frozen global or dense features solve **your** task. A model strong at semantic retrieval may miss a small scratch on a manufactured part. Increasing the input resolution and using patch features can help, but it changes the compute budget and the comparison protocol.

Evaluate transfer honestly

Plain version. A representation has earned its name only when it helps on data and tasks that were not used to build or choose it. How you test it changes the answer.

Protocol	What is trained	What it measures
k-NN	Nothing; similarity to labeled reference features	Local structure of the representation
Linear probe	Linear head; encoder frozen	Information accessible through a simple readout
Fine-tuning	Head plus some or all encoder layers	Adaptability under a stated label and compute budget
Dense transfer	Spatial head on patch maps	Suitability for pixel or region outputs

Linear probing versus fine-tuning

Fine-tuning has more freedom, so it usually wins when the target data is plentiful and similar to the pretraining data. It is not always better. Kumar et al. [23] showed theoretically and empirically that full fine-tuning can **distort** good pretrained features: while a randomly initialized head is still wrong, its large gradients flow into the encoder and change features in directions that suit the training distribution but hurt **out-of-distribution** (OOD) inputs. A linear probe cannot distort the features. Their remedy, **LP-FT**, is two-step: first train the linear head with the encoder frozen, then fine-tune everything starting from that head.

We reproduced the qualitative effect on the digits example. The encoder is the SimCLR CNN from above; the labeled training digits are clean; the test digits are shifted, a mild distribution shift. For each label budget, we average three random draws of the labeled examples.

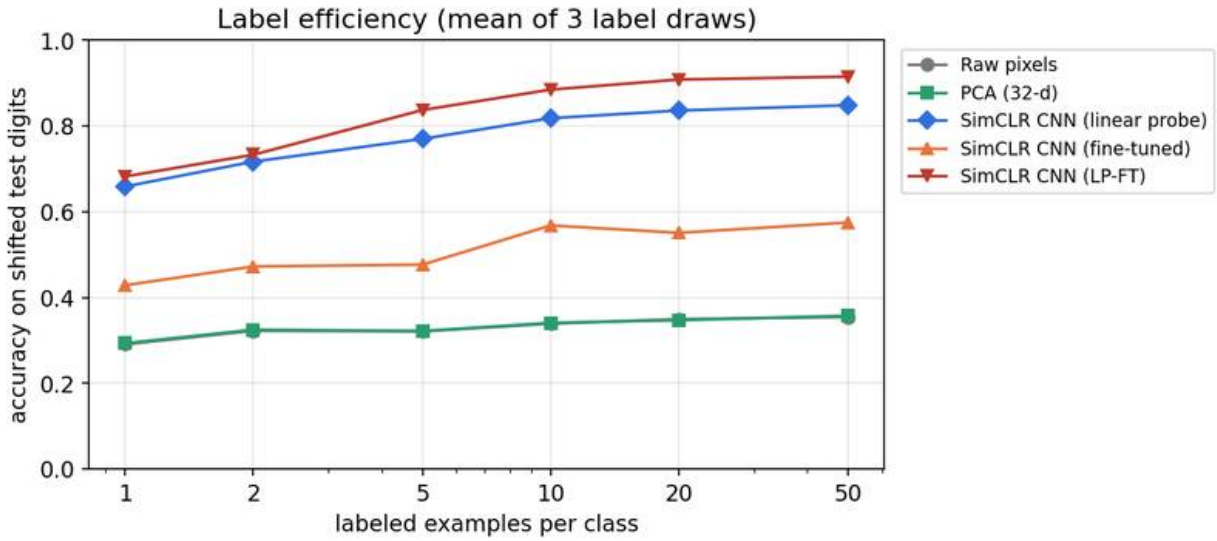


Figure 8 — Label efficiency on shifted test digits. Self-supervised features are far more label-efficient than pixels or PCA. With one fixed recipe, plain fine-tuning from a random head hurt the shifted-test accuracy, while LP-FT improved on the probe. A toy result: with a tuned learning rate or more labels, plain fine-tuning can do better.

Read Figure 8 for its shape, not its exact values. The frozen SSL features with a linear head need only a handful of labels per class to beat raw pixels with 50 labels per class. Plain fine-tuning with a randomly initialized head did worse than the frozen probe here, and LP-FT did better than both, the same ordering Kumar et al. report under distribution shift. Fine-tuning needs its own hyperparameter search, and the head initialization matters.

Rules against leakage

- Normalize features consistently for similarity comparisons (cosine k-NN needs normalized vectors).
- Fit scalers, PCA and classifier hyperparameters on training and validation data only.
- A frozen encoder still has a training history: overlap between pretraining data and evaluation data can inflate results, especially for web-scale models.
- Report the number of labels, backbone, checkpoint, input resolution, augmentations, head, tuning budget and protocol. Choosing the best of many encoders on the test set is test leakage, even if none of them is fine-tuned.

Modern view

Surveys and review papers. Four reviews map the field from complementary angles.

- **Bengio, Courville and Vincent (TPAMI 2013) [1]** frame the whole enterprise. They ask what makes one representation better than another and answer with a list of generic priors: distributed codes, depth and abstraction, disentangling of explanatory factors, manifolds and natural clustering. They review probabilistic models, autoencoders and manifold learning. It predates modern SSL but remains the best statement of *why* we learn representations.
- **Liu et al. (TKDE 2023) [3]** classify self-supervised methods into **generative** (autoencoding and autoregressive reconstruction), **contrastive** (context–instance and instance–instance) and **generative-contrastive** (adversarial) groups across vision, language and graphs, and discuss why contrastive objectives were winning on vision benchmarks at the time.
- **Gui et al. (TPAMI 2024) [4]** survey algorithms more recently, covering contrastive, self-distillation, redundancy-reduction and masked image modeling methods, and discuss applications and open problems such as theory, the choice of pretext task and multimodal SSL.
- **Balestriero et al. (2023) [5]** is a practitioner's guide written by many of the field's authors. It groups methods into a deep-metric-learning family (SimCLR and relatives), a self-distillation family (BYOL, SimSiam, DINO), a canonical-correlation-analysis family (VICReg, Barlow Twins) and masked image modeling, and collects the "tricks" (augmentations, projectors, EMA schedules, batch-size effects) that the papers often bury. Prince's textbook [24] gives a gentle, figure-rich introduction to the same ideas with notebooks.

State of practice (2023–2026). The trends visible in the papers cited above are:

- **Combined objectives.** Leading image encoders mix image-level distillation with patch-level masked prediction (DINOv2 [21], DINOv3 [22]) rather than relying on one pretext task.
- **Data as a first-class design choice.** DINOv2's curation pipeline and the web-scale pair collection behind CLIP [19] and SigLIP [20] show that what goes into pretraining matters as much as the loss.
- **Dense features.** Segmentation, depth and correspondence need good patch features, which motivated objectives like Gram anchoring [22].
- **Prediction in latent space and video.** I-JEPA [17] and V-JEPA 2 [18] predict features instead of pixels, and extend to video and action-conditioned prediction.
- **Language-aligned encoders as the default backbone** for multimodal models (see ML 09).

Open problems. We still lack a predictive theory of which pretext task suits which downstream task; collapse is prevented by recipes more than by guarantees; evaluation depends heavily on protocol and on overlap between pretraining and test data; and web-scale pretraining inherits biases and gaps from its data. Our reading of [3], [4], [5], [21], [22] is that data and target design

deserve as much attention as architecture. That is an interpretation of the literature, not a theorem that one objective wins everywhere.

The next chapters change the question: instead of *describing* an image, how can a model describe and **sample** its whole distribution? See ML 05A for autoencoders and ML 06 for generative models.

Key takeaways

- A representation is useful when a simple head can solve the task from it with few labels. Distributed codes generalize because features are shared across examples.
- The invariances you want depend on the task. In self-supervised learning, the augmentations choose them, as the shifted-digits example showed.
- Contrastive methods (SimCLR, MoCo) are InfoNCE classification: find your positive among negatives. Collapse costs them $\log(2B-1)$.
- Methods without negatives (BYOL, SimSiam, DINO) avoid collapse through asymmetry: predictors, stop-gradient, EMA teachers, centering and sharpening. Barlow Twins and VICReg put the anti-collapse term in the loss.
- Masked modeling (BEiT, MAE) reconstructs hidden content; JEPA predicts hidden *features*. CLIP aligns images with captions and enables zero-shot classification.
- k-NN, linear probes and fine-tuning answer different questions. Fine-tuning can distort features under distribution shift; LP-FT is a simple fix.

Exercises

1. **Temperature and collapse.** In the first code block, change the temperature from 0.2 to 0.05 and to 1.0. Does the collapsed loss change? Does the paired loss?

   Hint

   The collapsed loss stays $\log 15$: all non-self logits are equal, so dividing them by any τ leaves the softmax uniform. The paired loss changes, because temperature rescales the *differences* between scores, and with real differences a smaller τ gives a sharper softmax.

2. **Augmentations are assumptions.** You want features for (a) counting cells in microscopy, (b) recognizing bird species, (c) reading street-sign text. For each, name one standard SimCLR augmentation you would remove and why.

   Hint

   (a) Random cropping can change the number of cells in view, so the "same" label is no longer the same. (b) Strong color jitter can remove plumage color, a key species cue. (c) Horizontal flips turn text into mirror text. Decide what must be preserved before choosing invariances.

3. **MAE compute.** An image has 196 patches and MAE masks 75%. How many patches enter the encoder? By what factor does the number of pairwise attention scores in the encoder shrink?

Hint

49 visible patches enter the encoder. Attention scores scale with the square of the token count: $(49/196)^2 = 1/16$. The decoder still processes all 196 positions, but it is small.

4. **Break the digits experiment.** Modify the SimCLR digits code so that `augment` only adds noise (no shift). Retrain and compare the shifted-test k-NN accuracy with the original. What does this tell you?

Hint

Replace the random `i, j` with `i, j = 1, 1`. On our machine the shifted-test accuracy fell from 0.919 to 0.743, while clean accuracy stayed at 0.981. Without the shift augmentation, nothing in the objective asks for shift invariance; the robustness that remains comes from what a trained convolution-and-pooling encoder learns anyway (the untrained one in the main example had none). The augmentations, not the word "self-supervised", decide most of what is learned.

5. **SimSiam ablation, on paper.** In SimSiam, suppose you remove the stop-gradient so both branches receive gradients. Explain in two sentences why a constant output becomes an easy minimizer, and why the predictor alone is not enough to prevent it.

Hint

With gradients on both sides, the loss $-\cos(p_1, z_2)$ is minimized whenever p_1 and z_2 point the same way, which a constant encoder plus a predictor that outputs that same constant direction achieves for every input. The stop-gradient changes the optimization dynamics so that the target is held fixed during each step; the SimSiam paper reports rapid collapse without it.

6. **Fair probe.** Design an experiment comparing a frozen pretrained encoder with PCA plus logistic regression at 10, 50 and 100 labels per class. List three things you must hold fixed and two things you must report.

Hint

Hold fixed: the labeled subsets (same examples for both), the held-out test split, and the procedure for choosing regularization (on validation data only). Report: the pretrained encoder's training data (and any overlap with your test set) and the number of random label draws with their spread.

References

1. Y. Bengio, A. Courville and P. Vincent, "Representation Learning: A Review and New Perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, 2013. Paper ·arXiv
2. J. Yosinski, J. Clune, Y. Bengio and H. Lipson, "How transferable are features in deep neural networks?" NeurIPS, 2014. Paper
3. X. Liu et al., "Self-supervised Learning: Generative or Contrastive," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, 2023 (online 2021). Paper ·arXiv
4. J. Gui et al., "A Survey on Self-Supervised Learning: Algorithms, Applications, and Future Trends," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. Paper ·arXiv
5. R. Balestriero et al., "A Cookbook of Self-Supervised Learning," arXiv:2304.12210, 2023. Paper
6. A. van den Oord, Y. Li and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," arXiv:1807.03748, 2018. Paper
7. T. Chen, S. Kornblith, M. Norouzi and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," ICML, 2020. Paper
8. K. He, H. Fan, Y. Wu, S. Xie and R. Girshick, "Momentum Contrast for Unsupervised Visual Representation Learning," CVPR, 2020. Paper
9. T. Wang and P. Isola, "Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere," ICML, 2020. Paper
10. J.-B. Grill et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," NeurIPS, 2020. Paper
11. X. Chen and K. He, "Exploring Simple Siamese Representation Learning," CVPR, 2021. Paper
12. M. Caron et al., "Emerging Properties in Self-Supervised Vision Transformers," ICCV, 2021. Paper
13. J. Zbontar, L. Jing, I. Misra, Y. LeCun and S. Deny, "Barlow Twins: Self-Supervised Learning via Redundancy Reduction," ICML, 2021. Paper
14. A. Bardes, J. Ponce and Y. LeCun, "VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning," ICLR, 2022. Paper
15. H. Bao, L. Dong, S. Piao and F. Wei, "BEiT: BERT Pre-Training of Image Transformers," ICLR, 2022. Paper
16. K. He et al., "Masked Autoencoders Are Scalable Vision Learners," CVPR, 2022. Paper
17. M. Assran et al., "Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture," CVPR, 2023. Paper
18. M. Assran et al., "V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning," technical report, 2025. Report
19. A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," ICML, 2021. Paper
20. X. Zhai, B. Mustafa, A. Kolesnikov and L. Beyer, "Sigmoid Loss for Language Image Pre-Training," ICCV, 2023. Paper
21. M. Oquab et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. Paper
22. O. Siméoni et al., "DINOv3," technical report, 2025. Report
23. A. Kumar, A. Raghunathan, R. Jones, T. Ma and P. Liang, "Fine-Tuning can Distort Pretrained

Features and Underperform Out-of-Distribution," ICLR, 2022. Paper
24. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book and notebooks

ML 05A • Linear Factor Models and Autoencoders

PCA, probabilistic PCA, factor analysis, ICA, sparse coding, NMF and autoencoders (undercomplete, sparse, denoising, contractive): how compression, noise assumptions and the manifold view lead to VAEs.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-05a-factor-models-autoencoders/

What you'll learn

- How a small number of hidden **factors** can explain many correlated measurements, and why that idea underlies both classical statistics and modern deep generative models.
- How PCA, probabilistic PCA and factor analysis differ, and what each one assumes about noise.
- How independent component analysis (ICA), sparse coding and nonnegative matrix factorization (NMF) pick *different* factors by adding different assumptions.
- How linear and nonlinear autoencoders relate to these models, and how sparse, denoising and contractive penalties change what an autoencoder learns.
- What the **manifold view** says about high-dimensional data, and why it leads naturally from autoencoders to variational autoencoders (VAEs).

The big picture

An 8×8 digit image has 64 pixel values, and a phone photo has millions. Yet the meaningful variation in a collection of such images depends on far fewer underlying quantities: which digit it is, the stroke thickness, the slant, the position, the lighting. A **factor model** tries to explain each observation through a handful of hidden (latent) coordinates. An **autoencoder** is a neural network that learns to squeeze an input into such coordinates and expand it back again.

> **Explain like I'm five**
>
> Imagine describing a friend's face over the phone. You don't read out every freckle; you say "round face, short hair, big smile, light from the left." Those few words are factors: they let the listener rebuild a good picture. A factor model looks for the words that explain the most about many faces at once. An autoencoder is a pair of helpers: one who turns a picture into a few words, and one who turns the words back into a picture. They practise together until the rebuilt pictures look right.

Why spend a whole chapter on this? Because almost every idea in modern representation learning and generative modeling has an ancestor here. Dimensionality reduction for visualization, denoising, compression, anomaly detection, the "tokenizers" that turn images into compact codes

for diffusion models, and the probabilistic machinery of VAEs all start from the same question: *what small set of causes could have produced these observations?* The progression from linear factor models to autoencoders follows Goodfellow, Bengio and Courville [1, ch. 13–14] and Murphy [2]; Bengio, Courville and Vincent's review [3] gives the broader motivation for why learned representations matter.

One template, many models

Every method in this chapter fits the same template. There is an **encoder** that maps an observation $x \in \mathbb{R}^d$ to a code $z \in \mathbb{R}^k$, and a **decoder** that maps a code back to a reconstruction $\hat{x}$. The models differ in three choices:

Choice	Options in this chapter
Shape of encoder/decoder	linear (PCA, factor analysis, ICA, NMF, sparse coding) or nonlinear (deep autoencoders)
Size of the code	**undercomplete** ($k < d$: the bottleneck forces compression) or **overcomplete** ($k \geq d$: something else must stop the model from copying)
Extra assumption or penalty	Gaussian noise (PPCA, FA), independence and non-Gaussianity (ICA), sparsity (sparse coding, sparse autoencoders), nonnegativity (NMF), robustness to corruption (denoising), insensitivity to small input changes (contractive)

Keep this table in mind. When two methods disagree, it is almost always because they made a different choice in one of these rows, not because one of them is "wrong."

PCA: the best linear reconstruction subspace

Plain version. Find a few orthogonal directions such that, if you keep only the coordinates of each example along those directions, you lose as little as possible.

Let $X \in \mathbb{R}^{n \times d}$ hold n centered examples as rows (the column means have been subtracted). PCA chooses a matrix $W \in \mathbb{R}^{d \times k}$ with orthonormal columns that minimizes the squared reconstruction error

$$\min_{W} \ \|X - XWW^{\top}\|_F^2 \qquad \text{subject to} \qquad W^{\top}W = I_k.$$

Here $\|\cdot\|_F$ is the Frobenius norm (the square root of the sum of squared entries), $Z = XW \in \mathbb{R}^{n \times k}$ are the **codes** (also called scores), and $\hat{X} = ZW^{\top}$ is the reconstruction. The encoder is $x \mapsto W^{\top}x$ and the decoder is $z \mapsto Wz$: PCA is already an autoencoder, just a linear one with tied weights.

Where the solution comes from

Two equivalent views lead to the same answer.

Maximum variance. Take a single unit vector w. The variance of the projected data Xw is $w^\top Sw$, where $S = \frac{1}{n-1}X^\top X$ is the sample covariance. Maximize $w^\top Sw$ subject to $w^\top w = 1$ with a Lagrange multiplier λ:

$$\nabla_w(w^\top Sw - \lambda(w^\top w - 1)) = 0 \quad \Longrightarrow \quad Sw = \lambda w.$$

So w must be an eigenvector of S, and the variance it captures is its eigenvalue λ. The best direction is the eigenvector with the largest eigenvalue; the next best direction, constrained to be orthogonal to the first, is the second eigenvector, and so on.

Minimum reconstruction error. Because W has orthonormal columns, the error splits into "total variance minus kept variance." Minimizing the error is therefore the same as maximizing the kept variance, and the residual equals the sum of the discarded eigenvalues (times $n-1$). The Eckart–Young theorem states the same result in matrix language: truncating the singular value decomposition $X = U\Sigma V^\top$ to its k largest singular values gives the best rank-k approximation of X in Frobenius norm. The PCA directions are the first k right singular vectors (columns of V), and the covariance eigenvalues are $\sigma_j^2/(n-1)$.

Code: PCA from the SVD

The snippet fits PCA on the training split of scikit-learn's bundled digits, evaluates reconstruction on held-out images, and checks the result against scikit-learn's own implementation.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)

mean = train.mean(axis=0)
U, s, Vt = np.linalg.svd(train - mean, full_matrices=False)
var = s**2 / (len(train) - 1)              # eigenvalues of the covariance
ratio = np.cumsum(var) / var.sum()
for k in (2, 10, 20):
    W = Vt[:k].T                            # d x k, orthonormal columns
    rec = (test - mean) @ W @ W.T + mean
    print(f"k={k:2d}  explained={ratio[k-1]:.3f}  "
          f"test MSE={np.mean((test - rec)**2):.4f}")

sk = PCA(n_components=10).fit(train)
print("matches sklearn:", np.allclose(np.abs(sk.components_), np.abs(Vt[:10])))
```

Output:

```
k= 2  explained=0.291  test MSE=0.0546
k=10  explained=0.740  test MSE=0.0202
k=20  explained=0.896  test MSE=0.0084
matches sklearn: True
```

The `np.abs` in the last line is not a hack: each principal direction is only defined up to sign, because w and $-w$ span the same line. Note that the mean is estimated on the training split only and reused for the test split; estimating it on test data would leak information.

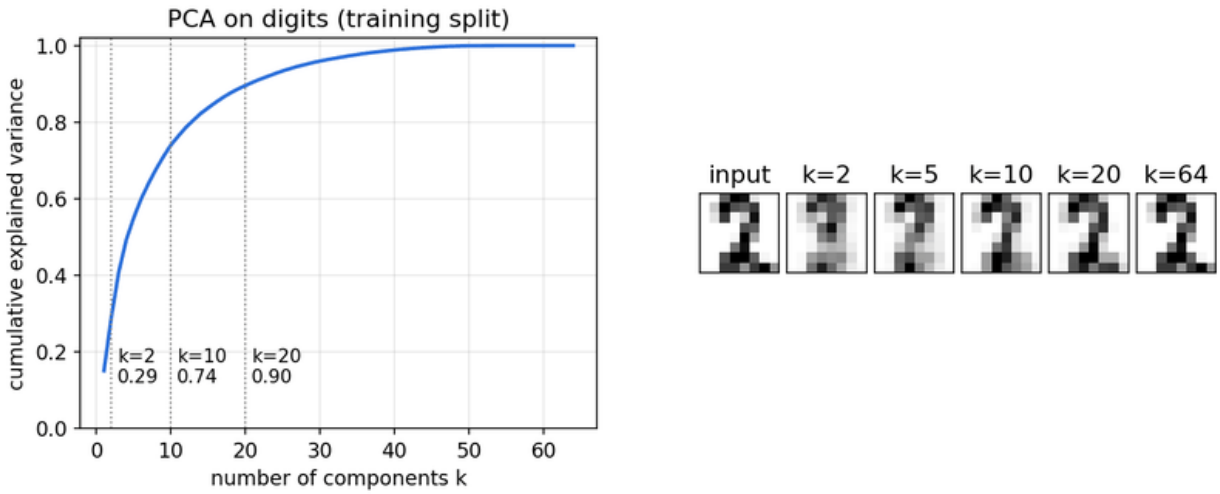


Figure 1 — PCA on the bundled digits. Left: cumulative explained variance (fit on the training split). Right: one held-out digit reconstructed from k components; k = 64 reproduces the input exactly.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

What PCA does not promise

Three cautions follow directly from the objective.

- **Centering and scaling change the answer.** Without centering, the first "component" mostly points at the mean. Standardizing every feature to unit variance gives each pixel equal weight, so raw-pixel PCA and standardized PCA solve different problems.
- **Variance is not relevance.** The direction of largest variance can be lighting or background rather than the class you care about. PCA is unsupervised; it maximizes retained variance, not class separation (that is the job of methods such as linear discriminant analysis).

- **PCA is linear.** If the data lie on a curved surface, a flat subspace may need many dimensions to describe what is really a low-dimensional shape. We return to this in the manifold section.

Probabilistic PCA and factor analysis

Plain version. Instead of just finding a subspace, write down a story for how the data were generated: draw a few hidden factors, mix them linearly, then add noise. Then fit that story by maximum likelihood.

Both models share the generative equation

$$x = \mu + Wz + \epsilon, \qquad z \sim \mathcal{N}(0, I_k),$$

where $\mu \in \mathbb{R}^d$ is the mean, $W \in \mathbb{R}^{d \times k}$ is the **loading matrix** (how each factor shows up in each measured coordinate), z is the vector of latent factors, and ϵ is noise independent of z. They differ only in the noise.

Probabilistic PCA (PPCA)

In PPCA [4], the noise is isotropic, $\epsilon \sim \mathcal{N}(0, \sigma^2 I_d)$: every coordinate gets the same noise variance. Integrating out z gives a Gaussian marginal

$$x \sim \mathcal{N}(\mu,\, C), \qquad C = WW^\top + \sigma^2 I_d.$$

The covariance has a clear interpretation: a low-rank part $WW^\top$ that the factors explain, plus a constant "floor" σ^2 that they do not. Tipping and Bishop showed that the maximum-likelihood solution is available in closed form [4]:

$$W_{\mathrm{ML}} = U_k(\Lambda_k - \sigma^2 I)^{1/2} R, \qquad \sigma^2_{\mathrm{ML}} = \frac{1}{d-k} \sum_{j=k+1}^{d} \lambda_j,$$

where U_k holds the top k eigenvectors of the sample covariance, Λ_k their eigenvalues $\lambda_1, \dots, \lambda_k$, and R is any $k \times k$ orthogonal matrix. In words: the loadings span exactly the PCA subspace, and the noise variance is the average of the variance PCA throws away. PPCA therefore *is* PCA, plus a likelihood. That likelihood is useful: you can compare models with different k on held-out data, handle missing values with expectation–maximization (EM), and build mixtures of PPCA models.

Given an observation, the posterior over its factors is Gaussian with mean

$$\mathbb{E}[z \mid x] = M^{-1} W^\top (x - \mu), \qquad M = W^\top W + \sigma^2 I_k.$$

This is the probabilistic encoder. As $\sigma^2 \to 0$ it approaches the ordinary PCA projection; for $\sigma^2 > 0$ it shrinks codes toward zero, because part of what we see is attributed to noise.

Factor analysis (FA)

Factor analysis keeps the same structure but allows a diagonal noise covariance, $\epsilon \sim \mathcal{N}(0, \Psi)$ with $\Psi = \text{diag}(\psi_1, \dots, \psi_d)$. Each observed coordinate has its own residual noise level ψ_i, while the latent factors explain the **shared** covariance between coordinates. The marginal is $x \sim \mathcal{N}(\mu, WW^\top + \Psi)$, and the posterior mean of the factors is $\mathbb{E}[z \mid x] = W^\top (WW^\top + \Psi)^{-1}(x - \mu)$.

There is no closed-form solution, so FA is fitted with EM or direct likelihood optimization. The extra flexibility matters when measurements come from sensors with unequal noise, or when some pixels are almost constant (such as the always-blank border pixels of the digits). PCA happily spends components on any high-variance coordinate; FA can instead explain an individually noisy coordinate as "noise," and reserve its factors for correlations.

The rotation problem

Both models share an ambiguity. For any orthogonal R, replacing W by WR and z by $R^\top z$ leaves $WW^\top$, and hence the distribution of x, unchanged, because $R^\top z$ is still standard normal. The *subspace* is identified; the individual axes are not. That is why the factor-analysis literature uses rotations such as varimax to pick more interpretable axes, and why a factor is not automatically a physical cause. To pin down individual factors we must assume something beyond Gaussianity, which is precisely what ICA does.

Independent component analysis (ICA)

Plain version. Several people talk at once in a room, and several microphones record different mixtures of their voices. ICA tries to recover each voice by finding mixtures that are as statistically independent, and as non-Gaussian, as possible.

The noise-free ICA model is

$$x = As, \qquad s = (s_1, \dots, s_k) \text{ with independent components,}$$

where $x \in \mathbb{R}^d$ is the observed mixture, $A \in \mathbb{R}^{d \times k}$ is an unknown **mixing matrix**, and the sources s_j are independent and **non-Gaussian**. The goal is an unmixing matrix $B \approx A^{-1}$ (or a pseudo-inverse) so that $\hat{s} = Bx$ recovers the sources [5].

Why non-Gaussianity is the key

The rotation problem from the previous section explains why ICA needs non-Gaussian sources.

First **whiten** the data: transform x so that its covariance is the identity (PCA followed by rescaling each component to unit variance does this). After whitening, the remaining unknown is just a

rotation. If the sources were Gaussian, every rotation of independent unit Gaussians would again be independent unit Gaussians, so no rotation could be preferred. ICA's identifiability theorem states the converse: if **at most one** source is Gaussian, the sources can be recovered up to three harmless ambiguities [5]:

- **order** (we cannot know which source is "first"),
- **sign**, and
- **scale** (a louder source with a quieter mixing column looks the same).

The central limit theorem gives the intuition for how to find the right rotation: a sum of independent variables tends to be *more* Gaussian than its parts. So the projections of x that are *least* Gaussian are the ones most likely to be single sources rather than mixtures. ICA algorithms therefore maximize a measure of non-Gaussianity. A principled one is **negentropy**,

$$J(y) = H(y_{\text{gauss}}) - H(y) \;\geq 0,$$

where H is differential entropy and y_{gauss} is a Gaussian with the same variance as y. Among all distributions with a given variance, the Gaussian has the largest entropy, so $J(y) = 0$ exactly for Gaussian y. Because entropy is hard to estimate, the **FastICA** algorithm approximates negentropy with contrast functions such as $G(u) = \log\cosh u$, and finds each unmixing direction with a fast fixed-point iteration [5]. Maximizing non-Gaussianity is closely related to minimizing the mutual information between the estimated components and to maximum-likelihood estimation of the ICA model.

Code: unmix three signals

The toy problem uses a sine wave, a square wave and a sawtooth, all non-Gaussian, mixed by a fixed matrix.

```
import numpy as np
from sklearn.decomposition import FastICA, PCA

rng = np.random.default_rng(0)
t = np.linspace(0, 8, 2000)
S = np.c_[np.sin(2*t),                  # smooth sine
          np.sign(np.sin(3*t)),         # square wave
          2*(t % 1) - 1]                # sawtooth
S += 0.05 * rng.normal(size=S.shape)
A = np.array([[1.0, 1.0, 1.0],
              [0.5, 2.0, 1.0],
              [1.5, 1.0, 2.0]])         # unknown mixing matrix
X = S @ A.T                             # three "microphones"

def best_match(est):
    C = np.abs(np.corrcoef(S.T, est.T)[:3, 3:])
```

```
    return np.round(C.max(axis=1), 3)

ica = FastICA(n_components=3, whiten="unit-variance", random_state=0)
print("ICA |corr| with each source:", best_match(ica.fit_transform(X)))
print("PCA |corr| with each source:", best_match(PCA(3).fit_transform(X)))
```

Output:

```
ICA |corr| with each source: [0.997 0.999 0.997]
PCA |corr| with each source: [0.689 0.811 0.735]
```

We compare with absolute correlation and take the best match because ICA returns the sources in arbitrary order, sign and scale. PCA only decorrelates; it finds orthogonal directions of high variance, which here are still mixtures.

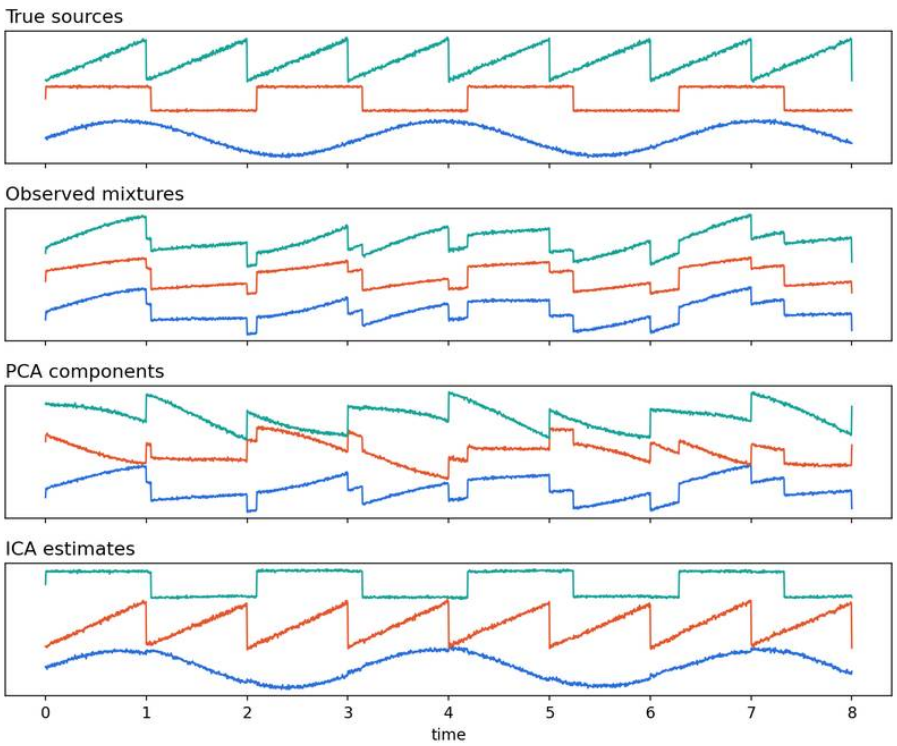


Figure 2 — Blind source separation. PCA components remain mixtures; FastICA recovers the sine, square and sawtooth waves up to order, sign and scale.

ICA is widely used for separating artifacts in EEG and other sensor recordings. Its assumptions also mark its limits: it needs genuinely independent, non-Gaussian sources and a mixing process that is close to linear and instantaneous. On image patches, ICA tends to learn localized, oriented edge-like filters, which brings us to a closely related idea.

Sparse coding

Plain version. Keep a large dictionary of small patterns ("atoms") and describe each input using only a few of them at a time.

Sparse coding learns a dictionary $D = [d_1, \dots, d_m] \in \mathbb{R}^{d \times m}$ and, for each example x_i, a sparse code $\alpha_i \in \mathbb{R}^m$ by solving

$$\min_{D,\,\{\alpha_i\}} \sum_{i=1}^{n} \left(\tfrac{1}{2} \|x_i - D\alpha_i\|_2^2 + \lambda \|\alpha_i\|_1 \right) \qquad \text{subject to} \qquad \|d_j\|_2 \le 1.$$

The first term asks for accurate reconstruction; the ℓ_1 norm $\|\alpha\|_1 = \sum_j |\alpha_j|$ pushes most code entries to exactly zero; $\lambda > 0$ sets the trade-off. The norm constraint on atoms stops the model from cheating by making atoms huge and codes tiny. The dictionary is often **overcomplete** ($m > d$): there are more atoms than pixel dimensions, but only a few are active for any one input.

Olshausen and Field [6] trained this kind of model on patches of natural images and found that the learned atoms were localized, oriented and band-pass, much like the receptive fields of simple cells in the primary visual cortex. The result was influential because it suggested that sparsity alone, without any labels, can explain a known property of biological vision.

Two practical points distinguish sparse coding from the autoencoders later in this chapter:

- **Inference is an optimization.** Given D, finding α for a new input means solving a lasso problem. There is no fixed encoder network; the "encoder" is an algorithm. Training alternates between updating the codes (with D fixed) and updating the dictionary (with codes fixed).
- **Sparsity is a prior.** The ℓ_1 penalty corresponds to a Laplace prior on the codes, so sparse coding can be read as maximum a posteriori inference in a linear factor model with a sparse, non-Gaussian prior, which is the same family that ICA belongs to.

Code: sparsity versus fidelity

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import DictionaryLearning
from sklearn.model_selection import train_test_split

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)
for alpha in (0.1, 1.0):
    dl = DictionaryLearning(n_components=32, alpha=alpha, max_iter=30,
                            transform_algorithm="lasso_lars",
                            transform_alpha=alpha, random_state=0)
    dl.fit(train[:600])
    codes = dl.transform(test)
    rec = codes @ dl.components_
```

```
    active = np.mean(np.abs(codes) > 1e-8, axis=1).mean()
    print(f"alpha={alpha}: active atoms per image={active*32:.1f}/32, "
          f"test MSE={np.mean((test - rec)**2):.4f}")
```

Output (about half a minute on a laptop CPU):

```
alpha=0.1: active atoms per image=17.9/32, test MSE=0.0075
alpha=1.0: active atoms per image=3.0/32, test MSE=0.0401
```

Raising λ (here `alpha`) buys a much sparser description, about three active atoms per image instead of eighteen, at the price of higher reconstruction error. Neither setting is "correct": a compression codec, a denoiser and an interpretability tool would each choose differently. Note that this dictionary does not subtract a mean, so its error is not directly comparable to the PCA numbers above.

Nonnegative matrix factorization (NMF)

Plain version. Describe each example as an *additive* mixture of nonnegative parts: you may add strokes, but you may never subtract one.

For data $X \geq 0$ (pixel intensities, word counts, spectra), NMF finds $Z \geq 0$ and $H \geq 0$ that minimize a reconstruction criterion such as

$$\min_{Z \geq 0,\, H \geq 0} \|X - ZH\|_F^2,$$

where the rows of $H \in \mathbb{R}^{k \times d}$ are nonnegative parts and the rows of $Z \in \mathbb{R}^{n \times k}$ are nonnegative mixing weights. Because nothing can cancel, the parts tend to look like pieces of the object rather than the positive-and-negative "ghost" patterns of PCA. Lee and Seung [7] demonstrated this parts-based behavior on faces and text and popularized simple multiplicative update rules for fitting it.

Two cautions. First, parts-based results are a tendency, not a guarantee: on some datasets the factors are not clean parts. Second, the objective is nonconvex in Z and H jointly, so initialization matters and different runs can give different factors. Scikit-learn's `init="nndsvda"` uses an SVD-based start to make results more stable.

Code: compare factor reconstructions

This CPU example fits each model on training digits only and evaluates reconstruction on held-out images. It compares different model assumptions; it is not a claim that one method wins every task.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA, FactorAnalysis, NMF

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)
models = {
    "PCA": PCA(n_components=12),
    "factor analysis": FactorAnalysis(n_components=12, random_state=0),
    "NMF": NMF(n_components=12, init="nndsvda", max_iter=1000,
               random_state=0),
}
for name, model in models.items():
    model.fit(train)
    z = model.transform(test)
    if name == "factor analysis":
        reconstructed = z @ model.components_ + model.mean_
    else:
        reconstructed = model.inverse_transform(z)
    mse = np.mean((test-reconstructed)**2)
    print(name, "held-out reconstruction MSE:", round(mse, 5))
```

Output:

```
PCA held-out reconstruction MSE: 0.01652
factor analysis held-out reconstruction MSE: 0.02501
NMF held-out reconstruction MSE: 0.02207
```

PCA wins on squared reconstruction error, as it must: among all rank-12 linear reconstructions, the PCA subspace is optimal for this criterion (up to the train/test split). That does not make it the best *model*. The factor-analysis reconstruction above is a posterior-mean reconstruction that deliberately attributes part of each pixel to noise, and FA's real output, a full probability model with per-pixel noise, is not summarized by MSE at all. NMF trades some error for nonnegative, additive parts.

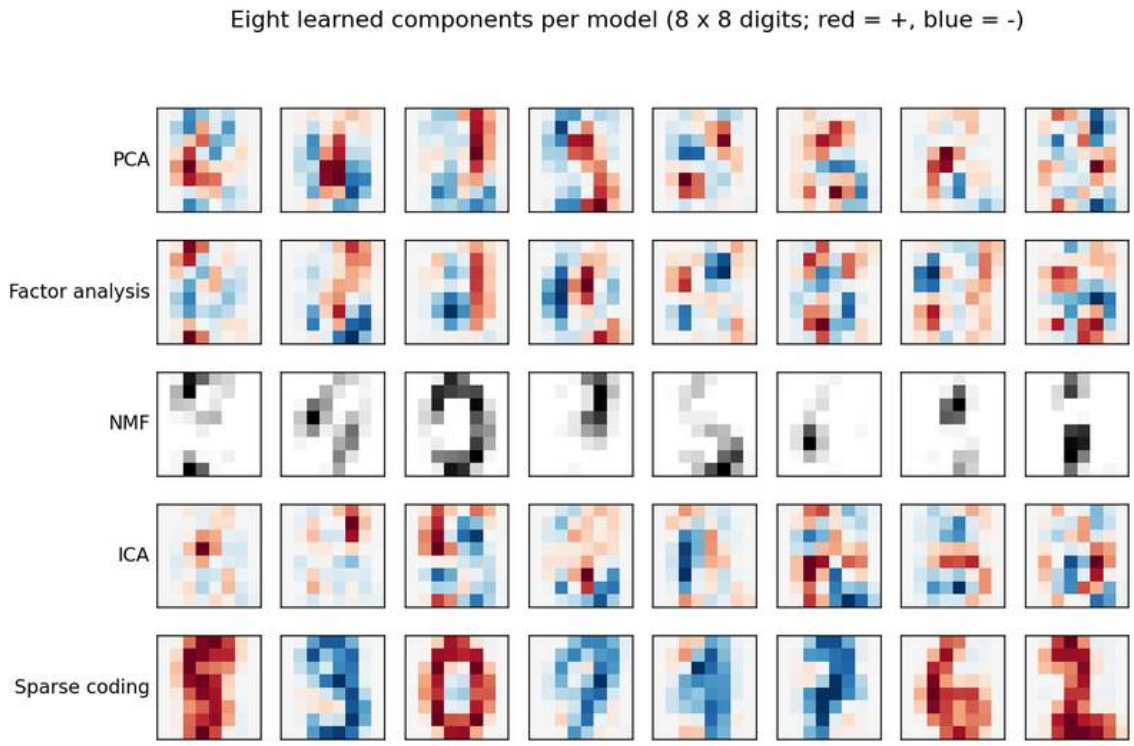


Figure 3 — Eight components from each linear factor model on the same digits. PCA and FA give signed, global patterns; NMF gives nonnegative strokes; ICA gives signed but more localized patterns; sparse-coding atoms resemble prototypes that are combined a few at a time.

Linear autoencoders and PCA

Plain version. If you train a neural network with no nonlinearities to compress and reconstruct its input, it ends up doing PCA, though possibly in rotated or stretched coordinates.

A linear autoencoder with centered input computes

$$z = W_e x, \qquad \hat{x} = W_d z, \qquad \mathcal{L}(W_e, W_d) = \frac{1}{n} \sum_i \|x_i - W_d W_e x_i\|^2,$$

with encoder $W_e \in \mathbb{R}^{k \times d}$ and decoder $W_d \in \mathbb{R}^{d \times k}$, $k < d$. The product $W_d W_e$ has rank at most k, so by Eckart–Young the best possible product is the projection onto the top-k principal subspace. Baldi and Hornik [8] proved more: for this squared-error linear network, every local minimum is a global one, and at the global minimum the network projects onto the principal subspace. The individual weight matrices, however, are not unique. For any invertible $k \times k$ matrix G, replacing W_e by GW_e and W_d by $W_d G^{-1}$ gives the same product, so the code coordinates can be any invertible linear transform of the PCA scores.

Code: a linear autoencoder rediscovers the PCA subspace

```
import numpy as np, torch
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from scipy.linalg import subspace_angles

torch.manual_seed(0)
X = load_digits().data / 16.
X = X - X.mean(axis=0)
Xt = torch.tensor(X, dtype=torch.float32)
k = 5
enc = torch.nn.Linear(64, k, bias=False)
dec = torch.nn.Linear(k, 64, bias=False)
opt = torch.optim.Adam([*enc.parameters(), *dec.parameters()], lr=1e-2)
for step in range(3000):
    loss = (dec(enc(Xt)) - Xt).square().mean()
    opt.zero_grad(); loss.backward(); opt.step()

Wd = dec.weight.detach().numpy()            # 64 x k decoder columns
P = PCA(n_components=k).fit(X).components_.T
angles = np.degrees(subspace_angles(Wd, P))
print("largest angle to PCA subspace (deg):", round(angles.max(), 3))
print("decoder columns orthonormal?",
      np.allclose(Wd.T @ Wd, np.eye(k), atol=1e-2))
```

Output:

```
largest angle to PCA subspace (deg): 0.003
decoder columns orthonormal? False
```

The subspace angle (the largest angle between the two k-dimensional subspaces) is essentially zero: the network found the PCA subspace. Yet its decoder columns are not orthonormal, so they are not the principal components themselves. This is the coordinate ambiguity in action, and it is the first hint of a theme that recurs through the chapter: reconstruction alone pins down *what is kept*, not *how it is labeled*.

Nonlinear autoencoders

Plain version. Replace the linear maps with neural networks, so the code can describe curved structure that no flat subspace can capture.

A general autoencoder has an encoder f_ϕ and a decoder g_θ, both neural networks:

$$z = f_\phi(x), \qquad \hat{x} = g_\theta(z), \qquad \min_{\phi,\theta} \frac{1}{n} \sum_i \ell(x_i,\, g_\theta(f_\phi(x_i))),$$

where ℓ is a reconstruction loss, such as squared error for real values or binary cross-entropy for values in $[0, 1]$. The choice of ℓ is itself an assumption: squared error corresponds to a Gaussian likelihood around $\hat{x}$, cross-entropy to a Bernoulli likelihood.

Hinton and Salakhutdinov [9] showed in 2006 that deep autoencoders, initialized with greedy layer-wise pretraining and then fine-tuned end to end, could find low-dimensional codes that reconstructed images much better than PCA with the same number of dimensions. Today, better initializations, normalization and optimizers make the pretraining step unnecessary, but the core lesson stands: nonlinear encoders can compress much more aggressively than linear ones when the data are curved.

Undercomplete is not enough, overcomplete needs a rule

An **undercomplete** autoencoder ($k < d$) is forced to compress. That is useful, but not a full guarantee of good features. A sufficiently powerful encoder and decoder could, in principle, map each training example to its own arbitrary code (say, its index) and memorize the decoding, achieving low training error without learning any reusable structure [1, ch. 14]. An **overcomplete** autoencoder ($k \geq d$) can simply learn the identity function. In both cases, what makes the representation useful is not the bottleneck by itself but the *inductive bias*: limited capacity, the architecture (convolutions, for instance), and explicit penalties. The next section covers the three classic penalties, which together are called **regularized autoencoders**.

Regularized autoencoders: sparse, denoising and contractive

Each penalty answers the question "what should the code ignore?" in a different way.

Sparse autoencoders

Plain version. Allow many code units, but require that only a few of them be active for any given input.

The loss adds a sparsity penalty on the code $h = f_\phi(x)$:

$$\mathcal{L} = \ell(x,\, g_\theta(h)) + \lambda\, \Omega(h), \qquad \Omega(h) = \sum_j |h_j|.$$

Alternatives penalize the deviation of each unit's average activation from a small target rate, or keep only the k largest activations (a "top-k" or k-sparse autoencoder). A sparse autoencoder is the amortized cousin of sparse coding: the encoder network predicts the sparse code in one forward pass, instead of solving a lasso problem for every input. Goodfellow et al. interpret the penalty as an approximate prior over codes rather than an ordinary weight regularizer [1, ch. 14].

As the Modern view explains, sparse autoencoders have recently become a key tool for interpreting the internals of large neural networks.

Denoising autoencoders

Plain version. Show the network a damaged input and ask it to rebuild the clean one. To succeed, it must learn what clean data look like.

A **denoising autoencoder (DAE)** draws a corrupted copy $\tilde{x} \sim C(\tilde{x} \mid x)$ and minimizes

$$\mathcal{L} = \mathbb{E}_x \, \mathbb{E}_{\tilde{x} \sim C(\cdot \mid x)} [\, \ell(x, \, g_\theta(f_\phi(\tilde{x}))) \,],$$

where C is a chosen corruption process: additive Gaussian noise, masking noise (setting a random fraction of inputs to zero), or salt-and-pepper noise [10], [11]. Vincent et al. introduced the idea in 2008 [10] and showed in 2010 that stacking DAE layers gave useful features for classifiers [11]. Because the input differs from the target, copying is no longer a solution, even for an overcomplete code.

The corruption defines what the model learns to be robust to. A DAE trained on Gaussian noise has no reason to fill in missing patches, and one trained with masking noise learns to complete missing parts. Modern masked image modeling, such as the masked autoencoder (MAE) in ML 05, is a direct descendant of masking-noise DAEs.

There is also a deeper geometric reading. For small Gaussian corruption, the vector $g(f(\tilde{x})) - \tilde{x}$ points from a corrupted point back toward regions of high data density; it approximately estimates the **score** $\nabla_x \log p(x)$, the gradient of the log-density [1, ch. 14]. This connection is one of the roots of score-based and diffusion generative models, covered in ML 07.

Code: a denoising autoencoder on digits

This standalone PyTorch example trains a small nonlinear DAE on training digits and evaluates clean reconstruction from noisy held-out input.

```
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); torch.set_num_threads(1)
train, test = train_test_split(load_digits().data/16., test_size=.2, random_state=0)
X = torch.tensor(train, dtype=torch.float32)
Xt = torch.tensor(test, dtype=torch.float32)
encoder = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 12))
decoder = nn.Sequential(nn.Linear(12, 32), nn.ReLU(), nn.Linear(32, 64), nn.Sigmoid())
model = nn.Sequential(encoder, decoder)
opt = torch.optim.Adam(model.parameters(), lr=.003)
for step in range(2000):
```

```
    clean = X[torch.randint(len(X), (64,))]
    noisy = (clean + .2*torch.randn_like(clean)).clamp(0, 1)
    loss = (model(noisy)-clean).square().mean()
    opt.zero_grad(); loss.backward(); opt.step()
model.eval()
with torch.no_grad():
    noisy = (Xt + .2*torch.randn_like(Xt)).clamp(0, 1)
    print("noisy-input MSE:", (noisy-Xt).square().mean().item())
    print("reconstructed MSE:", (model(noisy)-Xt).square().mean().item())
```

Output (about 0.0248 for the noisy input and 0.0197 after the DAE; the last digits depend on the PyTorch build):

```
noisy-input MSE: 0.024781296029686928
reconstructed MSE: 0.01968955807387829
```

The improvement is real but modest, and Figure 4 shows why: with a 12-dimensional code, the DAE removes the speckle but also smooths away some stroke detail. Denoising and faithful compression pull in different directions; a wider code or a convolutional architecture would shift the balance.

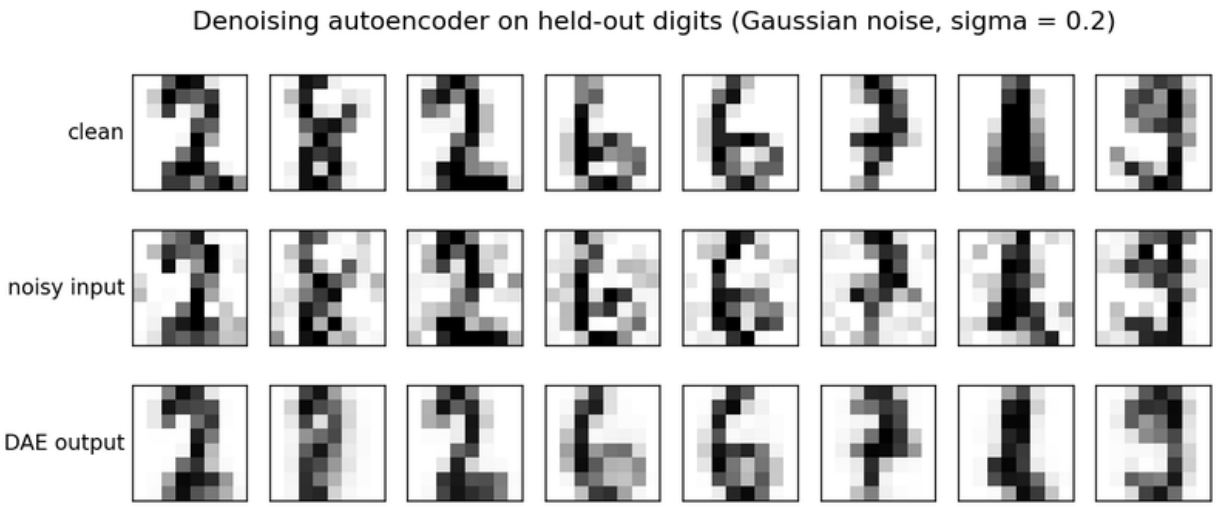


Figure 4 — Denoising autoencoder outputs on held-out digits. The background speckle is removed; some fine stroke detail is lost through the 12-unit bottleneck.

Contractive autoencoders

Plain version. Penalize the encoder whenever a tiny change in the input causes a big change in the code. The code should only react to changes that matter.

Rifai et al. [12] add the squared Frobenius norm of the encoder's Jacobian:

$$\mathcal{L} = \ell(x,\, g_\theta(f_\phi(x))) + \lambda \left\| \frac{\partial f_\phi(x)}{\partial x} \right\|_F^2, \qquad \left\| \frac{\partial f_\phi(x)}{\partial x} \right\|_F^2 = \sum_{j,i} \left(\frac{\partial h_j}{\partial x_i} \right)^2 .$$

Here $h = f_\phi(x)$ and the Jacobian entry $\partial h_j / \partial x_i$ measures how much code unit j moves when input coordinate i moves. For a single sigmoid layer $h = \sigma(Wx + b)$, the entry is $h_j(1 - h_j)W_{ji}$, so the penalty is cheap to compute:

$$\left\| \frac{\partial h}{\partial x} \right\|_F^2 = \sum_j \left(h_j(1 - h_j) \right)^2 \sum_i W_{ji}^2 .$$

The reconstruction term wants the code to change with the input (otherwise it could not reconstruct different inputs), while the penalty wants it to change as little as possible. The compromise is a code that is sensitive only along the few directions in which the training data actually vary. A DAE achieves a related effect implicitly: for small Gaussian noise, denoising training penalizes the sensitivity of the whole reconstruction function [1, ch. 14], whereas the CAE penalizes only the encoder, explicitly.

Code: the contractive penalty in action

```
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

train, test = train_test_split(load_digits().data/16., test_size=.2, random_state=0)
X = torch.tensor(train, dtype=torch.float32)
Xt = torch.tensor(test, dtype=torch.float32)

def fit(lam):
    torch.manual_seed(0)
    W = nn.Linear(64, 16)                    # encoder h = sigmoid(Wx + b)
    dec = nn.Linear(16, 64)
    opt = torch.optim.Adam([*W.parameters(), *dec.parameters()], lr=3e-3)
    for step in range(1500):
        x = X[torch.randint(len(X), (64,))]
        h = torch.sigmoid(W(x))
        rec = (dec(h) - x).square().sum(1).mean()
        # ||J||_F^2 = sum_j (h_j(1-h_j))^2 * ||W_j||^2 for a sigmoid layer
        jac = ((h*(1-h))**2 * W.weight.square().sum(1)).sum(1).mean()
        loss = rec + lam*jac
        opt.zero_grad(); loss.backward(); opt.step()
    with torch.no_grad():
        h = torch.sigmoid(W(Xt))
        mse = (dec(h) - Xt).square().mean().item()
        jac = ((h*(1-h))**2 * W.weight.square().sum(1)).sum(1).mean().item()
    return mse, jac
```

```
for lam in (0.0, 0.5):
    mse, jac = fit(lam)
    print(f"lambda={lam}: test MSE={mse:.4f}, mean ||J||_F^2={jac:.2f}")
```

Output:

```
lambda=0.0: test MSE=0.0141, mean ||J||_F^2=4.79
lambda=0.5: test MSE=0.0194, mean ||J||_F^2=0.70
```

The penalty cuts the encoder's average sensitivity by a factor of almost seven, at the cost of a somewhat higher reconstruction error. Whether that trade is worthwhile depends on the downstream use: a smoother code often helps nearest-neighbor retrieval or classification with few labels, while a codec would prefer the unpenalized model.

The manifold view

Plain version. Real data do not fill their high-dimensional space. They cluster near a thin, curved surface, like a sheet of paper rolled up inside a room. Good representations describe where you are *on the sheet*.

The **manifold hypothesis** says that natural data, such as images, concentrate near a low-dimensional manifold embedded in the high-dimensional input space [3]. A random 64-pixel array almost never looks like a digit; the set of plausible digits is a tiny, structured region.

Locally, around any data point, the manifold looks like a flat plane, the **tangent space**, spanned by the few directions you can move while staying realistic (thicker stroke, slight slant, small shift). Directions orthogonal to the tangent space lead off the manifold into noise.

This view gives a single lens for everything in the chapter:

- **PCA** fits one global flat sheet. It works when the manifold is nearly flat and fails when it is curved.
- **Mixtures of PPCA or FA** fit many local flat sheets.
- **Nonlinear autoencoders** learn a curved chart of the manifold: the encoder gives coordinates on the sheet, and the decoder maps them back into the room.
- **Denoising** teaches the model to move points that fell off the manifold back onto it.
- **Contractive** penalties make the encoder insensitive to directions orthogonal to the manifold, while reconstruction keeps it sensitive along the tangent directions [1, ch. 14], [12].

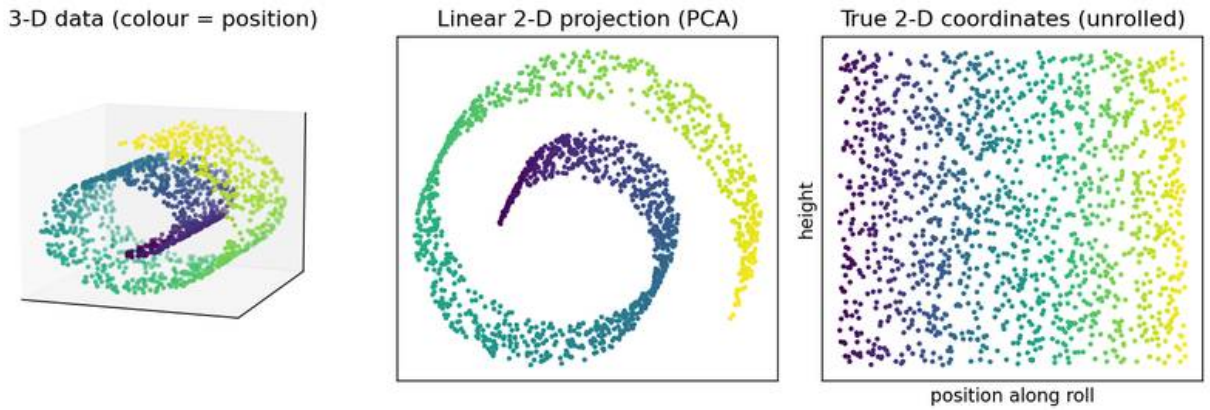


Figure 5 — The Swiss roll is a two-dimensional sheet rolled up in three dimensions. A linear projection (PCA) flattens it into a spiral that does not preserve position along the sheet; the true intrinsic coordinates form a flat rectangle.

Figure 5 makes the limitation of linear methods concrete. The Swiss roll has two intrinsic dimensions, position along the roll and height, but no two-dimensional linear projection can unroll it: the PCA view keeps the spiral, so points far apart along the sheet are not laid out in order. A nonlinear encoder can, in principle, learn the unrolled coordinates. In practice, whether it does depends on capacity, regularization and the amount of data; manifold learning in high dimensions remains hard.

From autoencoders to VAEs

Plain version. An ordinary autoencoder learns to compress and decompress, but it gives no rule for *inventing* new codes. A VAE adds that rule.

Suppose a trained autoencoder maps every training digit to a code in $\mathbb{R}^{12}$. To generate a new digit, we would like to pick a random code and decode it. But which random code? The training codes may occupy a strange, clumpy region, and a random point between them may decode to garbage. Nothing in the reconstruction loss asked the code space to be smooth, filled, or shaped like any known distribution. Excellent reconstruction therefore does not imply good generation.

The **variational autoencoder** [13] fixes this by returning to the probabilistic factor-model story of PPCA and FA, but with a neural decoder:

$$z \sim p(z) = \mathcal{N}(0, I), \qquad x \sim p_\theta(x \mid z),$$

where $p_\theta(x \mid z)$ is a distribution whose parameters are produced by a decoder network. Because the true posterior $p_\theta(z \mid x)$ is now intractable, a second network, the encoder, outputs an approximate posterior $q_\phi(z \mid x)$, typically a diagonal Gaussian. Training maximizes the **evidence lower bound (ELBO)** on $\log p_\theta(x)$:

$$\log p_\theta(x) \;\geq\; \underbrace{\mathbb{E}_{q_\phi(z|x)}[\,\log p_\theta(x \mid z)\,]}_{\text{reconstruction}} \;-\; \underbrace{D_{\mathrm{KL}}(q_\phi(z \mid x)\,\|\,p(z))}_{\text{keep codes near the prior}}.$$

Read the two terms against everything above:

- The **reconstruction** term is exactly an autoencoder loss, now with noise injected into the code. With a Gaussian decoder it becomes squared error, with a Bernoulli decoder binary cross-entropy.
- The **KL** term is a regularizer, like the sparse and contractive penalties, but chosen so that codes from all training examples together fill the prior. That is what makes sampling $z \sim \mathcal{N}(0, I)$ meaningful.
- With a **linear** Gaussian decoder, the model *is* PPCA. Its exact posterior is Gaussian and can be matched by a linear Gaussian encoder, so the bound becomes tight and the optimum coincides with the PPCA maximum-likelihood solution. The VAE is the nonlinear generalization of the factor models we began with.

Kingma and Welling's introduction [14] develops this connection at length. Weighting the KL term by a factor $\beta > 1$ gives the **β-VAE** [15], which was proposed to encourage disentangled factors at the cost of reconstruction quality; note that with $\beta \neq 1$ the objective is no longer the ELBO. The full derivation of the ELBO, the reparameterization trick and failure modes such as posterior collapse are developed in ML 06.

What reconstruction error can and cannot tell you

Before leaving autoencoders, a practical warning. Reconstruction error is often used as an anomaly score: an autoencoder trained on normal data should reconstruct normal inputs well and unusual ones poorly. This can work, but it fails when the autoencoder generalizes too well and reconstructs unusual inputs too, or when anomalies differ only in fine detail that the bottleneck discards anyway. Similarly, a code that reconstructs well may be poor for classification, and a code that classifies well may discard the pixel detail a restoration task needs. Evaluate compression, denoising, downstream usefulness and generation separately.

Modern view

Plain version. The classical models of this chapter did not disappear. They became components: compression stages for generative models, pretraining objectives for vision, and microscopes for looking inside large networks.

Surveys worth reading

- **Bengio, Courville and Vincent (2013)** [3] is the standard review of representation learning. It frames good representations through a list of general priors (smoothness, multiple explanatory factors, a hierarchy of concepts, shared factors across tasks, the manifold hypothesis, sparsity and others), and connects autoencoders, probabilistic models and manifold learning under that lens. It is a good map for *why* the penalties in this chapter exist.
- **Bank, Koenigstein and Giryes (2020)** [16] is a compact survey chapter on autoencoders. It walks through regularized variants (sparse, denoising, contractive), VAEs and disentangled variants, applications (generative modeling, classification, clustering, anomaly detection, recommendation and dimensionality reduction) and hybrids with adversarial training, which makes it a useful companion to this tutorial.
- **Kingma and Welling (2019)** [14] is the authors' own extended introduction to VAEs. It covers the ELBO, the reparameterization trick, richer posteriors and hierarchical models, and places VAEs among other deep latent-variable models.
- **Goodfellow, Bengio and Courville** [1, ch. 13–14] and **Murphy** [2] remain the clearest textbook treatments of linear factor models, the DAE–score connection and the manifold interpretation.

Disentanglement: a sober result

A recurring hope has been that unsupervised factor models would recover *the* true underlying factors, one per code unit ("disentanglement"). Locatello et al. [17] showed theoretically that unsupervised learning of disentangled representations is fundamentally impossible without inductive biases on both models and data, which mirrors the rotation ambiguity of factor analysis at a much larger scale. In a large reproducible study they also found that well-disentangled models could not be identified without supervision, and that more disentanglement did not reliably reduce the sample complexity of downstream tasks. The practical lesson echoes ICA: identifiability always comes from extra assumptions, so state them.

Autoencoders as tokenizers

The biggest modern role of autoencoders is as the first stage of large generative models. **VQ-VAE** [18] replaces the continuous code with a learned codebook of discrete vectors, so images become grids of discrete tokens that an autoregressive model can predict (see ML 08). **Latent diffusion** [19] trains an autoencoder for perceptual compression and then runs the diffusion model in its latent space instead of pixel space, which greatly reduces compute. In both cases, the autoencoder's reconstruction quality sets a ceiling: the generator cannot restore detail the encoder discarded. Designing these tokenizers, balancing compression, reconstruction and how easy the latent space is for the generator to model, is an active research area as of 2026.

Sparse autoencoders for interpretability (2023–2026)

Sparse autoencoders have found a new and very active application: **mechanistic interpretability** of large language and vision models. The motivating observation is that individual neurons are often polysemantic, responding to several unrelated concepts. Cunningham et al. [20] trained overcomplete sparse autoencoders to reconstruct a language model's internal activations, and found sparsely activating features that automated measures rated more interpretable and monosemantic than the neurons or other decompositions. Gao et al. [21] used k-sparse (top-k) autoencoders to control sparsity directly, reported clean scaling laws in autoencoder size and sparsity, proposed new metrics for feature quality, and trained a 16-million-latent autoencoder on GPT-4 activations. Conceptually this is sparse coding (Olshausen and Field's idea) applied to a network's activations instead of image patches.

Open problems

- **Evaluation.** Reconstruction error, sparsity and downstream accuracy disagree, and there is no agreed measure of whether learned features are the "right" ones, either for disentanglement or for interpretability.
- **Identifiability.** When can learned factors be trusted to correspond to real causes? Nonlinear ICA and causal representation learning study which extra assumptions (time structure, auxiliary variables, interventions) make this possible.
- **Tokenizer design.** How much to compress, whether codes should be discrete or continuous, and how to make latents both reconstructive and easy to model remain open engineering and theory questions.

Key takeaways

- All models here share one template: an encoder to a code, a decoder back, and an assumption that decides which code to prefer.
- PCA minimizes linear reconstruction error; its directions are covariance eigenvectors, found with the SVD of centered data.
- PPCA adds isotropic Gaussian noise and recovers the PCA subspace by maximum likelihood; factor analysis allows a separate noise level per coordinate. Both identify the subspace but not the axes.
- ICA resolves the axes by assuming independent, non-Gaussian sources, up to order, sign and scale; sparse coding and NMF pick factors by sparsity and nonnegativity.
- A linear autoencoder recovers the PCA subspace, not unique PCA weights. Nonlinear autoencoders need a bottleneck and/or a penalty (sparse, denoising, contractive) to learn useful structure.

- The manifold view unifies these: good codes move along the data's low-dimensional surface and ignore directions off it.
- Reconstruction does not provide a sampler. The VAE adds a prior and an approximate posterior, and with a linear decoder it reduces to PPCA.

Exercises

1. **Noise assumptions.** You record 20 sensors, five of which are much noisier than the rest, but all respond to the same few underlying causes. Would you start with PPCA or factor analysis? What would PCA tend to do with the noisy sensors?

   Hint

   Factor analysis: its diagonal Ψ can assign large residual variance to the five noisy sensors without spending factors on them. PCA (and PPCA, with a single shared σ^2) may devote components to the high-variance noisy sensors, because high variance is all PCA sees.

2. **Latent ambiguity.** For an invertible $k \times k$ matrix G, show that replacing W_e by GW_e and W_d by $W_d G^{-1}$ leaves a linear autoencoder's reconstruction unchanged. Which matrices G also preserve the PPCA likelihood?

   Hint

   The product is $W_d G^{-1} G W_e = W_d W_e$. For PPCA, the likelihood depends on $WW^\top$; replacing W by WG gives $WGG^\top W^\top$, which is unchanged when $GG^\top = I$, that is, when G is orthogonal.

3. **Why ICA fails on Gaussians.** Generate two independent standard Gaussian sources, mix them with any invertible 2×2 matrix, and run FastICA several times with different random seeds. What do you observe, and why?

   Hint

   After whitening, every rotation of two independent unit Gaussians is again two independent unit Gaussians, so there is no "right" rotation to find. Different seeds give different, equally valid unmixing directions, and the correlation with the true sources varies from run to run.

4. **Sparse codes versus PCA codes.** Using the sparse-coding snippet, compare the number of nonzero code entries per test image with PCA codes of the same dimension (32). Then plot the test MSE of sparse coding as `alpha` varies over 0.05, 0.1, 0.3, 1.0.

   Hint

   PCA codes are dense: essentially all 32 entries are nonzero. As `alpha` grows, the number of active atoms drops and the error rises, tracing a sparsity–fidelity curve. Remember that the dictionary was not given a mean term, so compare curves rather than single numbers with PCA.

5. **Denoising mismatch.** Train the DAE from the tutorial on Gaussian noise, then test it on inputs where a random 3×3 block is set to zero. Then train a second DAE on that masking corruption and test both on both corruptions.

Hint

These are different corruption processes. The Gaussian-trained model has never been asked to fill in missing structure, so it typically leaves holes; the masking-trained model learns completion. Report a 2×2 table (train corruption × test corruption) rather than inferring robustness to one from success on the other.

6. **Contractive penalty by hand.** For a single sigmoid unit $h = \sigma(w^\top x + b)$, derive $\|\partial h / \partial x\|^2$. When is it largest, and what does that say about where a contractive autoencoder allows its code to be sensitive?

Hint

$\partial h / \partial x = h(1-h)w$, so the squared norm is $h^2(1-h)^2\|w\|^2$. It is largest at $h = 1/2$ (the steep middle of the sigmoid) and nearly zero when the unit is saturated. The penalty pushes units to be saturated for most inputs, keeping them in the sensitive regime only where the data require variation.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 13–14. Book
2. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. Book
3. Y. Bengio, A. Courville and P. Vincent, "Representation Learning: A Review and New Perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, 2013. Paper
4. M. E. Tipping and C. M. Bishop, "Probabilistic Principal Component Analysis," *Journal of the Royal Statistical Society: Series B*, vol. 61, no. 3, 1999. Paper
5. A. Hyvärinen and E. Oja, "Independent component analysis: algorithms and applications," *Neural Networks*, vol. 13, no. 4–5, 2000. Paper
6. B. A. Olshausen and D. J. Field, "Emergence of simple-cell receptive field properties by learning a sparse code for natural images," *Nature*, vol. 381, 1996. Paper
7. D. D. Lee and H. S. Seung, "Learning the parts of objects by non-negative matrix factorization," *Nature*, vol. 401, 1999. Paper
8. P. Baldi and K. Hornik, "Neural networks and principal component analysis: Learning from examples without local minima," *Neural Networks*, vol. 2, no. 1, 1989. Paper
9. G. E. Hinton and R. R. Salakhutdinov, "Reducing the Dimensionality of Data with Neural Networks," *Science*, vol. 313, 2006. Paper
10. P. Vincent, H. Larochelle, Y. Bengio and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," ICML, 2008. Paper

11. P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio and P.-A. Manzagol, “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” *JMLR*, vol. 11, 2010. Paper
12. S. Rifai, P. Vincent, X. Muller, X. Glorot and Y. Bengio, “Contractive Auto-Encoders: Explicit Invariance During Feature Extraction,” ICML, 2011. Paper
13. D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” ICLR, 2014. Paper
14. D. P. Kingma and M. Welling, “An Introduction to Variational Autoencoders,” *Foundations and Trends in Machine Learning*, vol. 12, no. 4, 2019. Paper
15. I. Higgins et al., “β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework,” ICLR, 2017. Paper
16. D. Bank, N. Koenigstein and R. Giryes, “Autoencoders,” survey chapter, arXiv:2003.05991, 2020. Survey
17. F. Locatello et al., “Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations,” ICML, 2019. Paper
18. A. van den Oord, O. Vinyals and K. Kavukcuoglu, “Neural Discrete Representation Learning,” NeurIPS, 2017. Paper
19. R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” CVPR, 2022. Paper
20. H. Cunningham, A. Ewart, L. Riggs, R. Huben and L. Sharkey, “Sparse Autoencoders Find Highly Interpretable Features in Language Models,” arXiv:2309.08600, 2023. Paper
21. L. Gao et al., “Scaling and evaluating sparse autoencoders,” arXiv:2406.04093, 2024. Paper

ML 06 • Generative Models: Autoencoders, VAEs, GANs and Flows

Learn distributions rather than labels: likelihood-based versus implicit models, the ELBO and reparameterization, adversarial training and Wasserstein GANs, normalizing flows, and how generative models are evaluated.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-06-generative-models/

What you'll learn

- How modeling a distribution $p(x)$ differs from predicting a label, and why maximum likelihood is the same as minimizing a KL divergence.
- The two big families: **likelihood-based** models that write down (or bound) $\log p(x)$, and **implicit** models that only provide a sampler.
- How a VAE turns an intractable integral into a trainable lower bound (the ELBO), and why the reparameterization trick makes it differentiable.
- How GANs learn through a discriminator, what the original objective secretly minimizes, why mode collapse happens, and what Wasserstein GANs change.
- How normalizing flows compute exact likelihoods with the change-of-variables formula, and where energy-based, autoregressive and diffusion models fit.
- How to evaluate generators with likelihood, Inception Score, FID and precision/recall — and what each of these cannot tell you.

The big picture

A classifier asks, "What is in this image?" A **generative model** asks, "What kinds of images could occur?" If it learns a useful distribution, we can draw new samples, fill missing regions, compress data, detect unusual inputs, or condition generation on text, a class label or another image.

> **Explain like I'm five**
>
> A child who recognizes drawings of houses can say "house." A child who understands how houses are drawn can invent a new one with a door, windows and a roof. Generative modeling tries to learn the recipe behind the examples, so it can cook up new ones — and a good cook makes a different, but still tasty, dish every time instead of the same one over and over.

Learning a distribution is harder than learning a label. A label is one number; an image has thousands of pixels whose values depend on one another in complicated ways. Every family

in this chapter is a different answer to the same engineering question: how do you represent a distribution over very high-dimensional data so that you can both **train** it from examples and **sample** from it afterwards?

This chapter uses the probabilistic viewpoint developed in Bishop and Bishop [1] and Prince [2]. The comparative review by Bond-Taylor et al. [3] organizes the same families along the trade-offs we will meet — training speed, sample quality, diversity, and whether a likelihood is available — and the flow review by Papamakarios et al. [4] is particularly helpful for distinguishing a learned distribution from a convenient sampling procedure. Diffusion and autoregressive models get their own chapters (ML 07 and ML 08) because their training and sampling deserve careful treatment; here we place them on the map.

Modeling a distribution

Plain version. Fit a probability model so that the observed examples look likely under it, then use the model to produce new examples that look like they came from the same source.

Maximum likelihood

Suppose the data $x_1, \dots, x_n$ are independent draws from an unknown distribution p_{data}, and p_θ is a model with parameters θ. Maximum likelihood minimizes the average **negative log-likelihood (NLL)**:

$$\mathcal{L}_{\text{NLL}}(\theta) = -\frac{1}{n}\sum_{i=1}^{n} \log p_\theta(x_i).$$

Here n is the number of training examples and $\log p_\theta(x_i)$ is the log-density the model assigns to example i.

Why is this a sensible goal? As n grows, the average approaches an expectation over the data distribution, and

$$\mathbb{E}_{x\sim p_{\text{data}}}[-\log p_\theta(x)] = D_{\text{KL}}(p_{\text{data}} \,\|\, p_\theta) + H(p_{\text{data}}).$$

$D_{\text{KL}}(p\|q) = \mathbb{E}_p[\log p - \log q]$ is the Kullback–Leibler divergence, which is zero only when the two distributions agree, and $H(p_{\text{data}})$ is the entropy of the data, which does not depend on θ. So **maximum likelihood is the same as minimizing the forward KL divergence** from data to model.

The direction of the KL matters. The forward KL heavily punishes a model that assigns near-zero density where real data lives, so likelihood training is "mass-covering": it prefers to spread probability over every mode, even if that means putting some probability between modes. Keep this in mind; we will see that GANs behave differently.

Densities, discrete pixels and bits per dimension

For continuous values, $p_\theta(x)$ is a **density**, not the probability of observing exactly x; a density can exceed 1. Real images, however, store integers such as 0–255. Fitting a continuous density directly to integers is ill-posed: the model can place infinitely tall spikes on the integer grid and drive the likelihood to infinity. The standard fix is **dequantization** — add uniform noise in $[0, 1)$ to each integer value before fitting a continuous density. Theis et al. [5] show that the continuous model's log-likelihood on dequantized data is then a lower bound on the log-likelihood of a corresponding discrete model, which makes the numbers comparable across papers. Alternatively one uses a discrete likelihood directly (a softmax over 256 values, or a Bernoulli for binary pixels, as in our code below).

Image likelihoods are usually reported in **bits per dimension**:

$$\text{bpd} = \frac{\mathcal{L}_{\text{NLL}}}{D \ln 2},$$

where $\mathcal{L}_{\text{NLL}}$ is measured in nats per image and D is the number of pixel values (height × width × channels). Bits per dimension are the average number of bits a lossless compressor built on the model would need per subpixel, which is why likelihood and compression are two sides of the same coin.

Conditional and unconditional generation

A discriminative classifier models $p(y \mid x)$. A conditional generator models $p(x \mid c)$, where c can be text, a class, a low-resolution image or observed pixels. Bayes' rule links related quantities, but a classifier does not by itself provide an image sampler.

Conditioning also explains a familiar failure. Minimizing pixel MSE predicts the **conditional mean** $\mathbb{E}[x \mid c]$. If an ambiguous input has several valid outputs — a gray-scale photo whose shirt could be red or blue — the mean blends them into a brownish average. A generative model instead tries to represent the whole distribution $p(x \mid c)$ and sample one plausible answer at a time.

Two families: likelihood-based and implicit

Every model below needs two abilities: a way to **train** from data and a way to **sample**. They differ in whether they can evaluate $p_\theta(x)$.

- **Likelihood-based (explicit) models** write down a density, or a bound on it, and maximize it. Autoregressive models and normalizing flows give exact likelihoods; VAEs (and diffusion mod-

els, in their variational view) optimize a lower bound; energy-based models define a density only up to an unknown normalizing constant.

- **Implicit models** define a sampler — "push noise through a network" — without any formula for $p_\theta(x)$. GANs are the main example. Because there is no likelihood to maximize, they must be trained by comparing samples with data in some other way.

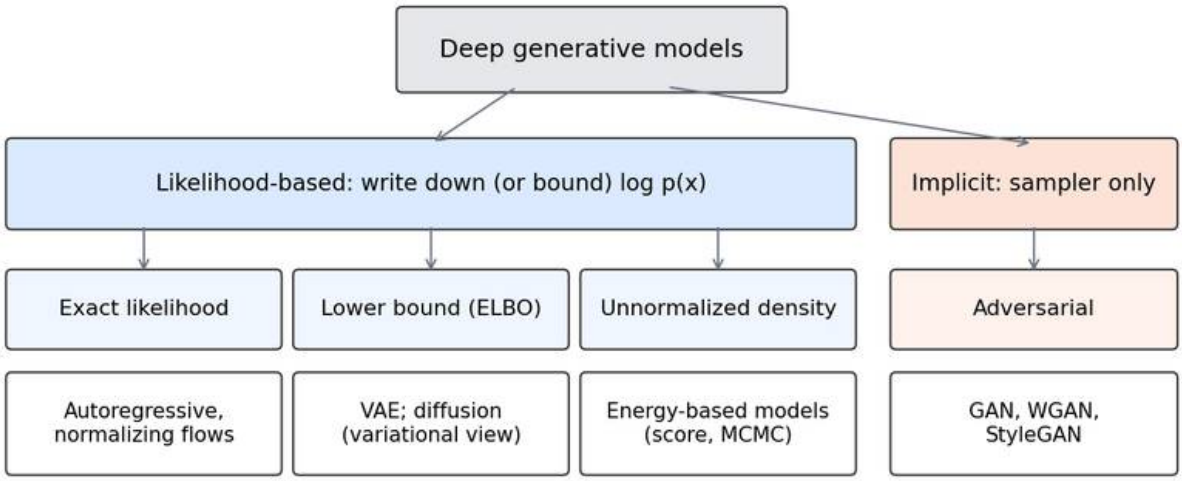


Figure 1 — A teaching taxonomy of generative models, following the explicit/implicit split used in comparative reviews [3]. Real systems often combine several branches.

Ordinary autoencoders: compression without a prior

Plain version. Encode an example into a compact code and decode it back. That is excellent for compression and representation learning, but it does not by itself tell you how to invent new codes.

An encoder $z = f_\phi(x)$ and decoder $\hat{x} = g_\theta(z)$ minimize a reconstruction loss such as $\|x - \hat{x}\|^2$. Autoencoders, their linear relatives (PCA, factor analysis) and their regularized variants are developed in ML 05A. The point for this chapter is narrow: an ordinary autoencoder places no requirement on **where** codes live. The training codes may occupy a thin, oddly shaped region of latent space. If you then sample $z \sim \mathcal{N}(0, I)$ and decode it, you may land in an empty region the decoder never saw, and get garbage. Excellent reconstruction therefore does not guarantee good unconditional generation.

A VAE fixes this by turning the autoencoder into a probability model with a prior over codes.

VAEs: a latent-variable probability model

Plain version. Imagine that each image was made by first picking a few hidden "settings" (pose, thickness, style) at random from a simple distribution, then rendering an image from those settings with a bit of noise. A VAE learns both the renderer (decoder) and a quick guesser (encoder) that reads an image and proposes which settings probably produced it.

The model

Choose a prior $p(z)$ over a latent vector $z \in \mathbb{R}^k$, usually a standard Gaussian $\mathcal{N}(0, I)$, and a decoder $p_\theta(x \mid z)$, a neural network that outputs the parameters of a distribution over images (Bernoulli probabilities for binary pixels, or Gaussian means). The model's density over images is the marginal

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz.$$

Sampling is easy: draw $z \sim p(z)$, then draw $x \sim p_\theta(x \mid z)$. Evaluating $p_\theta(x)$ is hard: the integral runs over every possible code, and the decoder is a nonlinear network, so there is no closed form. The true posterior $p_\theta(z \mid x) = p_\theta(x \mid z)p(z)/p_\theta(x)$ is intractable for the same reason.

Deriving the evidence lower bound

Kingma and Welling [6] introduce an **encoder** (or inference network) $q_\phi(z \mid x)$ that approximates the posterior. Here is the derivation step by step. Since $\log p_\theta(x)$ does not depend on z, we can take its expectation under any distribution over z without changing it:

$$\begin{aligned}
\log p_\theta(x) &= \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x)\big] \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{p_\theta(z \mid x)}\right] \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right] + \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\right] \\
&= \mathrm{ELBO}(x) + D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)).
\end{aligned}$$

The second line uses $p_\theta(x) = p_\theta(x,z)/p_\theta(z \mid x)$; the third multiplies and divides by $q_\phi(z \mid x)$ inside the log. Because a KL divergence is never negative, the first term is a lower bound on $\log p_\theta(x)$:

the **evidence lower bound (ELBO)**. The **gap** between the bound and the true log-likelihood is exactly the KL divergence between the approximate and true posteriors. A better encoder tightens the bound; a perfect one closes it.

Expanding $p_\theta(x,z) = p_\theta(x \mid z)p(z)$ gives the form used in code:

$$\mathrm{ELBO}(x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{reconstruction}} - \underbrace{D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p(z))}_{\text{stay close to the prior}}.$$

The first term rewards codes that let the decoder reproduce x; the second penalizes codes whose distribution wanders away from the prior. The same bound can be reached in one line with Jensen's inequality ($\log \mathbb{E} \geq \mathbb{E} \log$), but the decomposition above is more useful because it names the gap. Training maximizes the average ELBO over the data with respect to both θ and ϕ; equivalently, it minimizes the **negative ELBO**.

For a diagonal Gaussian encoder $q_\phi = \mathcal{N}(\mu, \mathrm{diag}(\sigma^2))$ and a standard Gaussian prior, the KL term has a closed form:

$$D_{\mathrm{KL}}(q_\phi \| p) = \frac{1}{2} \sum_{j=1}^{k} \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right),$$

where μ_j and σ_j^2 are the encoder's mean and variance for latent coordinate j. Each summand is zero exactly when $\mu_j = 0$ and $\sigma_j^2 = 1$.

Reparameterization

The reconstruction term is an expectation over $z \sim q_\phi(z \mid x)$, and we need its gradient with respect to ϕ. Sampling is not differentiable as written: a random draw is a black box. The **reparameterization trick** [6] moves the randomness into a fixed noise variable:

$$\epsilon \sim \mathcal{N}(0, I), \qquad z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon.$$

Here $\odot$ is elementwise multiplication. Now z is a deterministic, differentiable function of ϕ and an independent ϵ, so ordinary backpropagation gives low-variance gradient estimates through μ and σ. In code, the encoder predicts `logvar` $= \log \sigma^2$ and uses `exp(.5 * logvar)` as the standard deviation, which guarantees positivity without constraints.

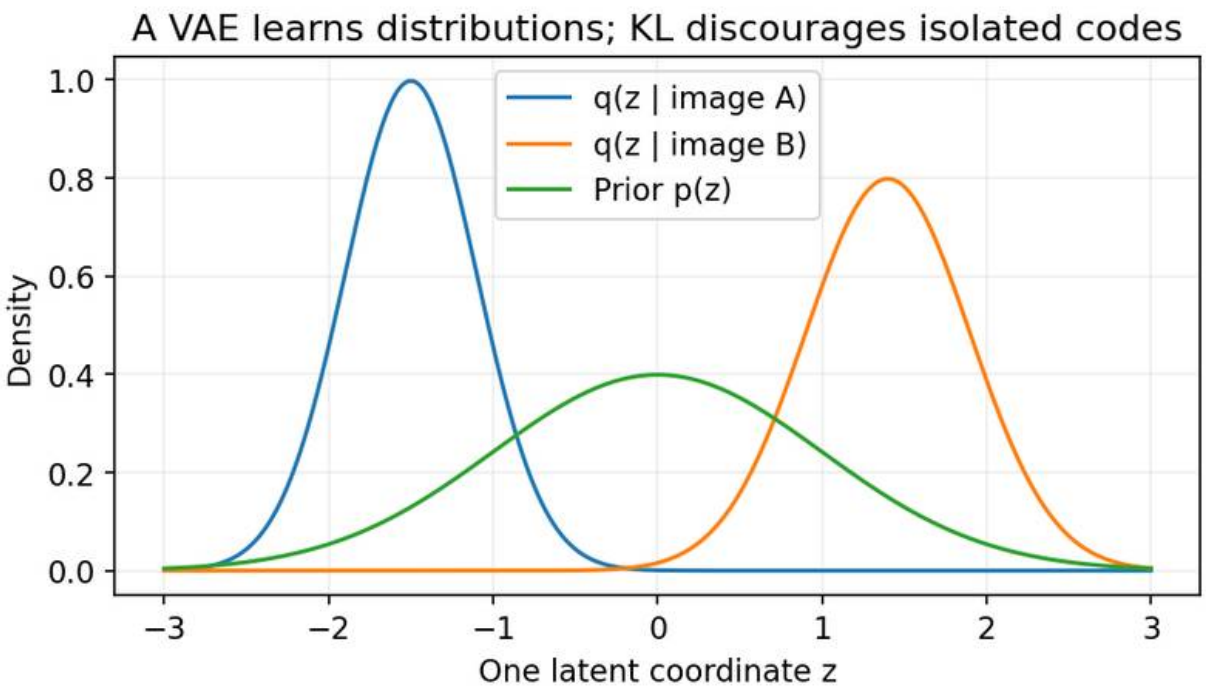


Figure 2 — A VAE represents uncertainty in its code and encourages compatibility with a shared prior. The curves are illustrative.

Code: a small VAE for binary digits

This complete CPU example uses scikit-learn's bundled 8 × 8 digits, binarized so that a Bernoulli decoder is a stated likelihood model. Reconstruction is summed over pixels and KL over latent dimensions, then both are averaged over examples; silently averaging only one term would change their balance.

```
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)
d = load_digits()
train, test = train_test_split(d.data, test_size=.2, random_state=0,
                               stratify=d.target)
X = torch.tensor((train > 8).astype("float32"))
Xt = torch.tensor((test > 8).astype("float32"))
enc = nn.Sequential(nn.Linear(64, 64), nn.ReLU(), nn.Linear(64, 16))
dec = nn.Sequential(nn.Linear(8, 64), nn.ReLU(), nn.Linear(64, 64))
opt = torch.optim.Adam(list(enc.parameters()) + list(dec.parameters()), lr=.002)
```

```
def terms(x):
    mu, logvar = enc(x).chunk(2, dim=-1)
    z = mu + (.5*logvar).exp() * torch.randn_like(mu)
    recon = F.binary_cross_entropy_with_logits(dec(z), x, reduction="none").sum(1)
    kl = .5*(mu.square() + logvar.exp() - logvar - 1).sum(1)
    return recon, kl

for step in range(400):
    x = X[torch.randint(len(X), (64,))]
    recon, kl = terms(x)
    loss = (recon + kl).mean()
    opt.zero_grad()
    loss.backward()
    opt.step()
enc.eval(); dec.eval()
with torch.no_grad():
    recon, kl = terms(Xt)
    samples = dec(torch.randn(4, 8)).sigmoid().reshape(4, 8, 8)
    print("test negative ELBO estimate:", round((recon + kl).mean().item(), 3))
    print("reconstruction / KL:", round(recon.mean().item(), 3),
          round(kl.mean().item(), 3))
    print("sample probability maps:", tuple(samples.shape))
```

On our machine it prints a test negative ELBO of about 20.5 nats per image, split into roughly 17.0 nats of reconstruction and 3.5 nats of KL. Since the bound is a lower bound on log-likelihood, 20.5 nats is an **upper bound** on the test NLL. The output maps are Bernoulli probabilities; to obtain binary samples, draw each pixel with those probabilities. The test estimate is stochastic, so average several latent draws for a more stable number. No test-based checkpoint selection happens here.

Figure 3 shows what a VAE with only a **two-dimensional** latent space learns on the same data (trained longer by the figure script). Even though labels are never used, digits of the same class land near each other, and decoding a regular grid of z values produces smooth transitions between digit shapes. This is the property an ordinary autoencoder lacks: every point near the prior's bulk decodes to something digit-like.

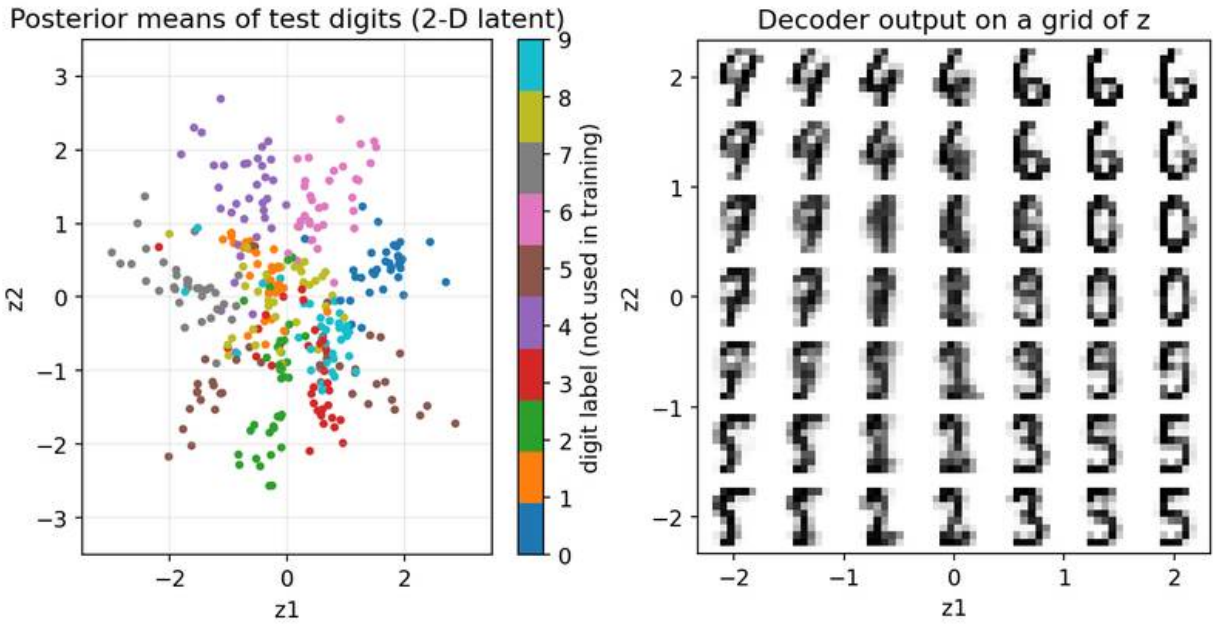


Figure 3 — A VAE with a 2-D latent space on binarized 8 × 8 digits. Left: encoder means for test images (labels used only for coloring). Right: decoder outputs on a grid of latent values.

Posterior collapse, β-VAE and deeper VAEs

If the decoder is very powerful (for example, an autoregressive decoder that can model pixels on its own), the cheapest way to raise the ELBO may be to ignore z entirely: set $q_\phi(z \mid x) = p(z)$ so the KL term is zero, and let the decoder do all the work. This is **posterior collapse**. Bowman et al. [7] observed it when training VAEs on text and proposed **KL annealing**: start with a small weight on the KL term and raise it to 1 during training, so the model learns to use the code before the penalty bites. Monitoring the KL per latent dimension is the simplest diagnostic — dimensions with KL near zero carry no information.

Higgins et al. [8] go the other way: the **β-VAE** multiplies the KL term by $\beta > 1$, pressuring the model toward a more factorized, "disentangled" code at the cost of reconstruction. Be precise about what this means: with $\beta \neq 1$ the objective is no longer the ELBO, so the value is not a bound on $\log p_\theta(x)$, and Kingma and Welling's tutorial review [9] discusses this and many other variants. Finally, single-layer VAEs with factorized Gaussians tend to produce blurry images, partly because the Gaussian decoder averages over uncertainty. **Hierarchical VAEs** stack several layers of latent variables; NVAE [10] showed that a carefully engineered deep hierarchy with stabilizing regularization can produce sharp samples on natural images up to 256 × 256 pixels.

GANs: learning with an adversary

Plain version. Two networks play a game. A **generator** turns random noise into fake examples; a **discriminator** looks at an example and guesses whether it is real or fake. The generator wins by fooling the discriminator, which forces its outputs to look more and more like data.

The minimax game

Let G map noise $z \sim p(z)$ to samples $G(z)$, whose distribution we call p_g. Let $D(x) \in (0,1)$ be the discriminator's estimated probability that x is real. The original GAN objective of Goodfellow et al. [11] is

$$\min_G \max_D \; V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))].$$

The discriminator maximizes V: it is ordinary binary cross-entropy with real data labeled 1 and fakes labeled 0. The generator minimizes V: it wants $D(G(z))$ to be large.

What the game minimizes

For a fixed generator, write V as an integral over x: $V = \int [p_{\text{data}}(x)\log D(x) + p_g(x)\log(1 - D(x))]dx$. For each x separately, the function $a\log D + b\log(1-D)$ with $a, b > 0$ is maximized at $D = a/(a+b)$ (set the derivative $a/D - b/(1-D)$ to zero). So the **optimal discriminator** is

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

It is the probability that a point came from the data rather than the generator, if both were equally likely a priori. Substituting D^* back in and adding and subtracting $\log 4$ gives

$$V(D^*, G) = -\log 4 + 2\,\text{JSD}(p_{\text{data}} \,\|\, p_g),$$

where $\text{JSD}(p\|q) = \frac{1}{2}D_{\text{KL}}(p\|m) + \frac{1}{2}D_{\text{KL}}(q\|m)$ with $m = \frac{1}{2}(p+q)$ is the **Jensen–Shannon divergence**. So, with a perfect discriminator, the generator minimizes a symmetric divergence between data and model, and the global optimum is $p_g = p_{\text{data}}$, where $D^* = 1/2$ everywhere [11]. Notice that no likelihood was ever computed: the discriminator acts as a learned, adaptive test of "does this look like the data?"

The non-saturating loss and how to implement it

Early in training the generator is poor, $D(G(z)) \approx 0$, and $\log(1 - D(G(z)))$ is flat, so the generator's gradient vanishes. In practice the generator instead minimizes $-\mathbb{E}\log D(G(z))$, the **non-**

saturating loss proposed in the same paper [11], which has strong gradients precisely when the generator is losing. It has the same fixed point but different dynamics.

Updates alternate. When updating D, **detach** generated samples so no gradient reaches G; when updating G, let gradients flow through D into G but step only G's optimizer. Using logits with `binary_cross_entropy_with_logits` is numerically safer than applying a sigmoid and then a log.

Code: a GAN on eight Gaussians

The classic test bed for mode coverage is a ring of eight tight Gaussian clusters in 2-D. It is small enough to train on a CPU in under a minute, and you can **see** whether every mode is covered.

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
angles = torch.arange(8) * (2 * math.pi / 8)
centers = 2.0 * torch.stack([angles.cos(), angles.sin()], dim=1)

def sample_data(n):
    idx = torch.randint(8, (n,))
    return centers[idx] + 0.05 * torch.randn(n, 2)

def mlp(i, o):
    return nn.Sequential(nn.Linear(i, 128), nn.ReLU(),
                         nn.Linear(128, 128), nn.ReLU(), nn.Linear(128, o))

G, D = mlp(8, 2), mlp(2, 1)
opt_g = torch.optim.Adam(G.parameters(), lr=1e-3, betas=(.5, .999))
opt_d = torch.optim.Adam(D.parameters(), lr=1e-3, betas=(.5, .999))
for step in range(3000):
    real = sample_data(256)
    fake = G(torch.randn(256, 8))
    # Discriminator: logits, real -> 1, fake -> 0; detach so G is untouched.
    d_loss = (F.binary_cross_entropy_with_logits(D(real), torch.ones(256, 1))
              + F.binary_cross_entropy_with_logits(D(fake.detach()),
                                                   torch.zeros(256, 1)))
    opt_d.zero_grad(); d_loss.backward(); opt_d.step()
    # Generator: non-saturating loss -log D(G(z)).
    g_loss = F.binary_cross_entropy_with_logits(D(fake), torch.ones(256, 1))
    opt_g.zero_grad(); g_loss.backward(); opt_g.step()

with torch.no_grad():
    x = G(torch.randn(5000, 8))
    dist = torch.cdist(x, centers)
    nearest = dist.min(1)
```

```
    good = nearest.values < 0.15          # within 3 std of some mode
    counts = torch.bincount(nearest.indices[good], minlength=8)
print("fraction near a mode:", round(good.float().mean().item(), 3))
print("modes with >= 1% of samples:", int((counts >= 50).sum()))
print("samples per mode:", counts.tolist())
```

It prints:

```
fraction near a mode: 0.896
modes with >= 1% of samples: 7
samples per mode: [590, 14, 710, 276, 651, 731, 863, 646]
```

About 90% of samples land on a mode, but the mode at 45° receives only 14 of 5,000 samples and another is under-represented. That is **partial mode collapse** caught in the act. Figure 4 shows the generator's journey: an unstructured cloud early on, a ring that covers every mode but also the space between them, and finally sharper clusters with one mode nearly abandoned. Exact numbers depend on the seed and library version; the qualitative picture is typical.

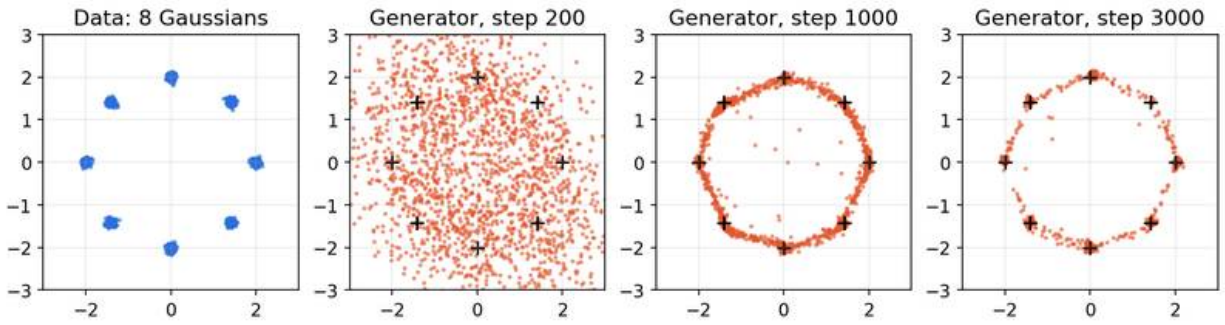


Figure 4 — A small GAN learning a ring of eight Gaussians (crosses mark the true modes). By step 3000 samples are sharp but one mode is nearly dropped: sharpness and coverage are separate achievements.

Mode collapse and why it happens

Mode collapse occurs when the generator produces too few kinds of output. The generator is rewarded for samples the current discriminator accepts, not for diversity. If it finds one region the discriminator likes, it can pile samples there; the discriminator then learns to reject that region, the generator hops to another, and the pair can chase each other around instead of converging.

Unlike maximum likelihood, nothing in the per-sample generator loss directly punishes **missing** a mode. Salimans et al. [12] proposed early remedies such as feature matching and minibatch discrimination, which let the discriminator look at statistics of a whole batch so that a lack of diversity becomes detectable.

A collection of sharp, convincing images can therefore still cover only a small part of the data distribution. Always measure coverage separately from quality — we return to this under evaluation.

Wasserstein GANs

When p_{data} and p_g live on thin, non-overlapping sets (as images do in pixel space), the JSD is stuck at its maximum $\log 2$ and provides no useful gradient about **how far apart** they are. Arjovsky et al. [13] proposed the **Wasserstein-1 (earth mover's) distance** instead: the minimum cost of moving probability mass from one distribution to the other. Its dual form, from the Kantorovich–Rubinstein theorem, is

$$W(p_{\text{data}}, p_g) = \sup_{\|f\|_L \le 1} \mathbb{E}_{x \sim p_{\text{data}}}[f(x)] - \mathbb{E}_{x \sim p_g}[f(x)],$$

where the supremum is over all 1-Lipschitz functions f (functions whose output changes by at most $\|x - x'\|$ when the input changes from x' to x). The network f is now called a **critic**: it outputs an unbounded score, not a probability. The Wasserstein distance shrinks smoothly as the generator's samples move toward the data, even when the supports do not overlap.

The hard part is enforcing the Lipschitz constraint. The original WGAN clips every critic weight to a small interval, which is crude and can make the critic too simple. **WGAN-GP** [14] instead adds a gradient penalty

$$\lambda \, \mathbb{E}_{\hat{x}}\big[(\|\nabla_{\hat{x}} f(\hat{x})\|_2 - 1)^2\big],$$

where $\hat{x}$ is drawn uniformly on straight lines between pairs of real and generated samples and λ is a weight (the paper uses 10). Two other widely used stabilizers act on the discriminator directly: **spectral normalization** [15] divides each weight matrix by its largest singular value, bounding the Lipschitz constant layer by layer, and the **R1 penalty** analyzed by Mescheder et al. [16] penalizes $\frac{\gamma}{2}\|\nabla_x D(x)\|^2$ on real data, which they show gives local convergence where unregularized GAN training can oscillate.

Architectures and where GANs are used now

StyleGAN [17] illustrates how architecture matters as much as the loss: it maps noise to an intermediate latent space and injects that "style" at every resolution, giving controllable, high-quality face synthesis. GANs sample with a single generator pass, which makes them fast at inference.

Adversarial losses also appear as **components** of other systems — for example, inside image tokenizers trained to reconstruct sharp detail. That does not make every model using such a tokenizer a GAN generator; the distribution may be modeled by something else entirely.

Normalizing flows: invertible transformations

Plain version. Start from simple noise, such as a standard Gaussian, and push it through a chain of invertible transformations. Because every step can be undone, you can run any image backwards to the noise that would have produced it — and keep exact track of how each step stretches or squeezes space, which gives an exact likelihood.

Change of variables

Let $z \sim p_0(z)$ be a simple base distribution and $x = f_\theta(z)$ an invertible, differentiable map with the same dimension. Then

$$\log p_\theta(x) = \log p_0(f_\theta^{-1}(x)) - \log \left| \det \frac{\partial f_\theta}{\partial z} \right|_{z=f_\theta^{-1}(x)} .$$

$\partial f_\theta / \partial z$ is the Jacobian matrix of the map, and its determinant measures how much the map scales a small volume around z. Probability mass is conserved, so if a region is stretched to twice its volume, its density must halve. In one dimension, $x = 2z$ doubles every interval and adds $-\log 2$ to the log density. A composition $f = f_K \circ \cdots \circ f_1$ simply adds up the log-determinants of the steps.

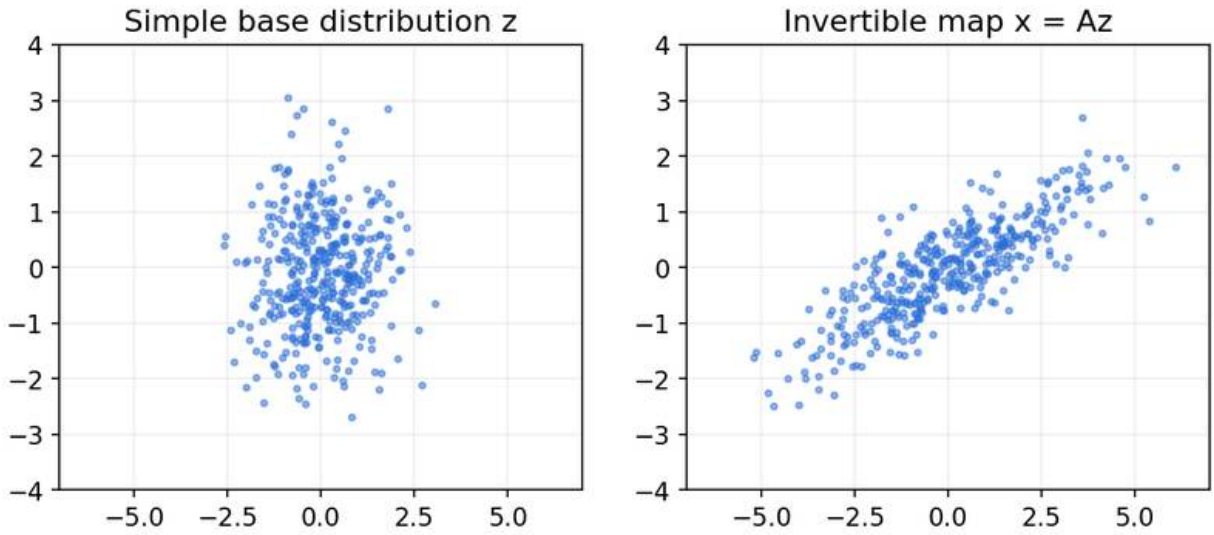


Figure 5 — Change of variables preserves total probability while changing density. This affine map has an analytically tractable determinant.

Coupling layers

The formula is exact, but a general $D \times D$ determinant costs $O(D^3)$ — hopeless for images. Flow layers are designed so that the Jacobian is **triangular**, whose determinant is just the product of the

diagonal. The affine coupling layer of RealNVP [18] splits x into two halves (x_1, x_2) and computes

$$y_1 = x_1, \qquad y_2 = x_2 \odot \exp\big(s(x_1)\big) + t(x_1),$$

where s and t are arbitrary neural networks. The Jacobian is block-triangular with the identity on the first block and $\operatorname{diag}(\exp s(x_1))$ on the second, so $\log|\det J| = \sum_j s_j(x_1)$. The inverse is just as cheap: $x_2 = (y_2 - t(y_1)) \odot \exp(-s(y_1))$. Crucially, s and t never need to be inverted, so they can be as complex as you like. Stacking layers with alternating splits lets every coordinate be transformed. Glow [19] added learned invertible 1×1 convolutions as a generalized channel permutation between coupling layers.

Code: verify a coupling layer's log-determinant

The snippet below builds one affine coupling layer and checks its cheap log-determinant against the full Jacobian computed by autograd.

```
import torch
from torch import nn

torch.manual_seed(0)

class AffineCoupling(nn.Module):
    """RealNVP-style layer: keep x1, scale-and-shift x2 using x1."""
    def __init__(self, dim, hidden=32):
        super().__init__()
        self.d = dim // 2
        self.net = nn.Sequential(nn.Linear(self.d, hidden), nn.Tanh(),
                                 nn.Linear(hidden, 2 * (dim - self.d)))

    def forward(self, x):
        x1, x2 = x[..., :self.d], x[..., self.d:]
        s, t = self.net(x1).chunk(2, dim=-1)
        s = torch.tanh(s)                       # keep scales moderate
        y2 = x2 * s.exp() + t
        return torch.cat([x1, y2], -1), s.sum(-1)   # output, log|det J|

    def inverse(self, y):
        y1, y2 = y[..., :self.d], y[..., self.d:]
        s, t = self.net(y1).chunk(2, dim=-1)
        s = torch.tanh(s)
        return torch.cat([y1, (y2 - t) * (-s).exp()], -1)

layer = AffineCoupling(4)
x = torch.randn(4)
y, logdet = layer(x)
J = torch.autograd.functional.jacobian(lambda v: layer(v)[0], x)
print("log|det J| from layer:   ", round(logdet.item(), 5))
print("log|det J| from autograd:", round(torch.linalg.slogdet(J)[1].item(), 5))
print("upper-right block is zero:", bool(torch.allclose(J[:2, 2:], torch.zeros(2, 2))))
print("inverse error:", (layer.inverse(y) - x).abs().max().item() < 1e-6)
```

```
# Log density of y under a standard-normal base after one layer:
base = torch.distributions.Normal(0., 1.)
z = layer.inverse(y)
log_px = base.log_prob(z).sum() - layer(z)[1]
print("log p(y):", round(log_px.item(), 4))
```

Both log-determinants print 0.14867, the upper-right Jacobian block is exactly zero (the triangular structure), and the inverse recovers the input. The last line is the change-of-variables formula in action. Training a flow simply maximizes this `log p` over data with a stack of such layers.

Strengths, limits and continuous flows

Flows give exact likelihoods, exact inversion (useful for editing and for mapping data to a well-behaved latent space) and one-pass sampling. The price is architectural: every layer must be invertible with a cheap determinant, and the latent must have the **same dimension** as the data, which wastes capacity on representing noise in all pixel directions. Rezende and Mohamed [20] introduced flows as a way to make the approximate posterior of a VAE more flexible, which shows that the same tool serves inference as well as generation.

Continuous normalizing flows replace a discrete stack by an ordinary differential equation $\mathrm{d}x/\mathrm{d}t = v_\theta(x, t)$. Neural ODEs [21] showed that the log-density then evolves through the **divergence** (trace of the Jacobian) of v_θ rather than a determinant, which removes the architectural restriction but makes likelihood evaluation require an ODE solve. **Flow matching** [22], covered in ML 07, trains such a vector field by simple regression instead of maximum likelihood. The word "flow" therefore refers to related transformations, but the training objective need not be likelihood.

Energy-based models

Plain version. Instead of describing how to generate an image, give every possible image a score — an "energy" — where low energy means "looks real." Probability is high wherever energy is low.

An energy-based model (EBM) defines

$$p_\theta(x) = \frac{\exp(-E_\theta(x))}{Z_\theta}, \qquad Z_\theta = \int \exp(-E_\theta(x))\, dx,$$

where E_θ is any network that outputs a scalar and Z_θ is the **partition function** that makes the density integrate to one. EBMs are extremely flexible — any architecture works — but Z_θ is intractable in high dimensions, so neither the likelihood nor exact samples are directly available.

Goodfellow, Bengio and Courville [23] cover the classical tools (the partition-function chapter and approximate inference), and Song and Kingma [24] survey modern training routes: maximum likelihood with MCMC samples (for example, Langevin dynamics), **score matching**, which fits $\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$ and thereby never needs Z_θ, and noise-contrastive methods. Score matching is the bridge to diffusion models.

Autoregressive models, briefly

Plain version. Generate an image one piece at a time, each piece conditioned on everything generated so far, like writing a sentence word by word.

The chain rule of probability is exact for any ordering of the D elements of x:

$$p_\theta(x) = \prod_{i=1}^{D} p_\theta(x_i \mid x_1, \ldots, x_{i-1}).$$

Each factor is an ordinary classification or regression problem, so training is simple, stable maximum likelihood with an exact likelihood. PixelRNN and PixelCNN [25] modeled raw pixels this way with a 256-way softmax per subpixel. The drawback is sequential sampling: D network evaluations for D elements. Modern systems instead predict compressed tokens, groups of tokens or whole scales at once. ML 08 develops this family in detail.

Diffusion models, briefly

Plain version. Teach a network to remove a little noise from a noisy image. Then start from pure noise and apply the denoiser many times, slowly sculpting an image.

A diffusion model defines a fixed forward process that gradually adds Gaussian noise to data until nothing but noise remains, and learns a reverse process that undoes it step by step. Ho et al. [26] showed that training reduces, up to weighting, to predicting the noise that was added — a simple regression — and that the objective is a reweighted variational bound, which is why the taxonomy in Figure 1 lists diffusion next to the VAE. The connection to score matching (see the energy-based models section) gives a second, equivalent view. The survey by Yang et al. [27] maps the many variants. Diffusion and flow matching now dominate high-quality image generation and are the subject of ML 07.

The family map

Family	Basic training signal	Sampling mechanism	Likelihood	Main qualification
VAE	Variational bound (ELBO)	Sample latent, then decoder (one pass)	Lower bound	Bound depends on approximate inference; samples can be blurry
GAN	Adversarial discrimination	Generator pass (one pass)	None	Unstable game; mode collapse
Normalizing flow	Change-of-variables likelihood	Invertible transform (one pass)	Exact	Invertibility and same-dimension constraints
Autoregressive	Conditional likelihoods	Sequential conditional sampling	Exact	Generation dependencies can be costly
Diffusion	Denoising/score objectives	Reverse noise process (many steps)	Bound or ODE-based	Quality depends on model and sampler
Flow matching	Velocity regression	Integrate learned vector field	Needs extra computation	Integration cost
Energy-based	Energy learning (MCMC, score matching)	Often iterative sampling	Unnormalized	Normalization and mixing can be hard

Bond-Taylor et al. [3] summarize these trade-offs as a triangle that is hard to satisfy all at once: fast sampling, high sample quality, and good mode coverage. GANs are fast and sharp but can miss modes; likelihood models cover modes but were historically slower or blurrier; diffusion models reach high quality and coverage at the cost of many sampling steps. This table is a teaching synthesis, not an exclusive taxonomy. Real models combine families: a VAE compresses an image, a transformer models its latent distribution, and a diffusion objective trains that transformer.

Evaluating generative models

Plain version. A generator can be good in different ways: it can assign high probability to real data, make individually convincing samples, cover all the variety in the data, or follow its conditioning. No single number captures all of these, so use several measures and know what each misses.

Theis et al. [5] made this point sharply: average log-likelihood, sample quality and older proxies such as Parzen-window estimates can be **largely independent** of one another in high dimensions. A model can have excellent likelihood and poor samples, or the reverse. Evaluate against the purpose you have in mind.

Likelihood

For likelihood-based models, held-out NLL (or bits per dimension) is principled and comparable when dequantization and preprocessing match. It cannot be computed for GANs. And a high likelihood need not mean reliable out-of-distribution detection: Nalisnick et al. [28] found that

flows, VAEs and PixelCNNs trained on CIFAR-10 can assign **higher** likelihood to the unrelated SVHN digit images, because likelihood is dominated by low-level statistics such as smoothness rather than semantic content.

Inception Score

Salimans et al. [12] proposed the **Inception Score (IS)**. Feed each generated image x to a pre-trained ImageNet classifier (Inception-v3) to get a label distribution $p(y \mid x)$, and let $p(y) = \mathbb{E}_x[p(y \mid x)]$ be the average over samples. Then

$$\mathrm{IS} = \exp\left(\mathbb{E}_{x \sim p_g} D_{\mathrm{KL}}(p(y \mid x) \,\|\, p(y))\right).$$

IS is high when each image is confidently classified ($p(y \mid x)$ peaked) and the set covers many classes ($p(y)$ spread out). It never looks at real data, so it cannot tell whether samples resemble **your** dataset, and it is meaningful only for ImageNet-like content.

Fréchet Inception Distance

The **Fréchet Inception Distance (FID)** of Heusel et al. [29] compares real and generated images in a feature space. Extract Inception-v3 features for both sets, fit a Gaussian to each — means μ_r, μ_g and covariances Σ_r, Σ_g — and compute the Fréchet distance between the Gaussians:

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\right).$$

Lower is better. FID uses real data and is sensitive to both quality and diversity, which made it the standard metric. But it is not an image-by-image truth measure: it depends on the feature extractor, image resizing, and especially the **number of samples** (it is biased for finite sets, so compare FIDs only with equal sample counts), and it folds quality and coverage into one number.

Precision and recall

To separate the two, Sajjadi et al. [30] defined **precision and recall for distributions**: precision asks what fraction of generated samples fall where real data lives (quality), recall asks what fraction of the real distribution the generator covers (diversity). Kynkäänniemi et al. [31] gave a simple, robust estimator that we implement below. Approximate each set's manifold by a union of balls around its points, with each ball's radius equal to the distance to the point's k-th nearest neighbor in the same set. Then

- **precision** = fraction of generated samples inside some real sample's ball;

- **recall** = fraction of real samples inside some generated sample's ball.

On images, the points are deep features rather than pixels. On our 2-D ring we can use coordinates directly:

```
import numpy as np

rng = np.random.default_rng(0)
angles = np.arange(8) * 2 * np.pi / 8
centers = 2 * np.stack([np.cos(angles), np.sin(angles)], 1)

def ring(n, modes):
    idx = rng.choice(modes, n)
    return centers[idx] + .05 * rng.standard_normal((n, 2))

def knn_radius(a, k):
    d = np.linalg.norm(a[:, None] - a[None], axis=-1)
    return np.sort(d, axis=1)[:, k]          # column 0 is the point itself

def coverage(query, ref, k=3):
    """Fraction of query points inside some reference point's k-NN ball."""
    r = knn_radius(ref, k)
    d = np.linalg.norm(query[:, None] - ref[None], axis=-1)
    return np.mean((d <= r[None]).any(axis=1))

real = ring(1000, range(8))
fakes = {
    "all 8 modes": ring(1000, range(8)),
    "only 2 modes": ring(1000, [0, 4]),
    "8 modes, blurry": ring(1000, range(8)) + .4 * rng.standard_normal((1000, 2)),
}
for name, fake in fakes.items():
    precision = coverage(fake, real)   # are fake samples on the real manifold?
    recall = coverage(real, fake)      # is the real manifold covered by fakes?
    print(f"{name:16s} precision={precision:.2f} recall={recall:.2f}")
```

```
all 8 modes      precision=0.97 recall=0.96
only 2 modes     precision=0.97 recall=0.26
8 modes, blurry  precision=0.07 recall=0.99
```

The mode-collapsed generator keeps high precision — each of its samples is perfect — but its recall drops to about 2/8. The blurry generator is the mirror image: it covers every mode but most samples are off the data. A single scalar such as FID would rank both as "worse" without saying **why**. Figure 6 shows the three cases.

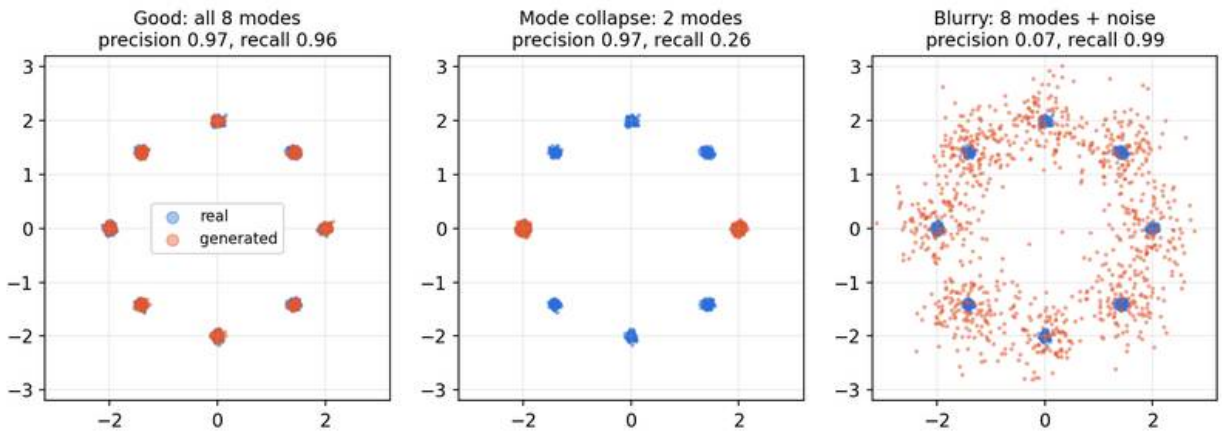


Figure 6 — k-nearest-neighbor precision and recall on a toy ring: mode collapse lowers recall, blur lowers precision.

Known limitations of current metrics

Evaluation is itself an active research area. Stein et al. [32] ran large human studies and found that no existing automatic metric strongly correlates with human judgments of realism; in particular, FID computed on Inception features under-rates the perceptual realism of diffusion models. They recommend richer self-supervised feature spaces (DINOv2 ViT-L/14) for metrics like FID and precision/recall. Jayasumana et al. [33] argue that FID's normality assumption and Inception features are poor fits for modern text-to-image models and propose **CMMD**, a maximum mean discrepancy computed on CLIP embeddings, which needs no Gaussian assumption. Practical advice: report several complementary metrics with their exact protocol (feature extractor, sample count, resolution, reference set), show uncurated samples, and run a human study when the claim is about perceived quality.

For conditional tasks, add a measure of **alignment** (does the image match the text or the low-resolution input?). For restoration, a beautiful generated texture may be unsupported by the observation; the distinction between plausible samples and measured evidence is central in DLCV 04.

Modern view

Surveys worth reading. Four reviews cover this chapter's ground from different angles.

- **Bond-Taylor et al. [3]** (IEEE TPAMI 2022) is the best single comparative review of VAEs, GANs, normalizing flows, energy-based and autoregressive models. It organizes each family by its core technique and its known pathologies, and compares them along training speed,

sampling speed, sample quality, diversity and likelihood, concluding that each family pays for its strengths with a weakness somewhere else, and that hybrids are the natural way forward.

- **Papamakarios et al. [4]** (JMLR 2021) is a unified treatment of normalizing flows: their expressive power, the design space of coupling, autoregressive, residual and continuous flows, and their uses for density estimation, inference and simulation.
- **Kingma and Welling [9]** (Foundations and Trends in ML 2019) is the authoritative tutorial-review of VAEs by their inventors: the derivation, the reparameterization estimator, tighter bounds, more flexible posteriors (including flows), hierarchical models and the β-weighted variants.
- **Yang et al. [27]** (ACM Computing Surveys 2023) surveys diffusion models — sampling acceleration, likelihood improvements, structured data, and the links to VAEs, GANs, flows and EBMs — and is the bridge to the next chapter.

The textbooks of Bishop and Bishop [1] and Prince [2] give coherent derivations of all the families, with Prince's notebooks providing runnable companions.

Where the field is (2023–2026). Diffusion and flow-matching models, usually operating in a learned latent space, now produce most state-of-the-art image and video generation. That makes the **autoencoder** a load-bearing component again: VQ-VAE [34] showed how to learn discrete latent codes that a separate prior can model, and latent diffusion [35] showed that running the generative model inside a compressed autoencoder latent space cuts cost dramatically while keeping detail. A tokenizer must balance compression, reconstruction and how easy its latent space is to model; evaluate its reconstruction first, because no downstream prior can restore detail the encoder discarded.

GANs have not disappeared. GigaGAN [36] scaled a GAN to text-to-image synthesis and emphasized what GANs still do best: single-pass sampling fast enough for interactive use, high-resolution upsampling, and a smooth, editable latent space. Huang et al. [37] revisited the training objective itself, deriving a regularized relativistic GAN loss with local convergence guarantees and showing that a minimalist modern baseline (R3GAN) without the traditional bag of tricks can surpass StyleGAN2 on standard benchmarks. Adversarial losses also live on inside other systems, for example to sharpen tokenizer reconstructions.

On evaluation, the 2023–2024 critiques of FID [32], [33] have moved the field toward richer feature extractors, separate precision/recall-style measures and human studies, but there is still no accepted single metric.

Open problems. (1) Reliable evaluation that agrees with humans and separates quality, diversity and memorization. (2) Detecting when a model reproduces training images rather than generalizing. (3) Calibrated likelihoods that are actually useful for out-of-distribution detection [28]. (4) The speed–quality–coverage trade-off: getting diffusion-level quality with GAN-level speed without losing modes. (5) Learning latent spaces that are simultaneously compressive, faithful and easy to model.

Key takeaways

- Generation requires a distribution or a defined sampling process. Maximum likelihood equals minimizing $D_{\mathrm{KL}}(p_{\mathrm{data}} \| p_\theta)$, which encourages covering every mode.
- Likelihood-based models (autoregressive, flows, VAEs, EBMs, diffusion's bound) write down or bound $\log p(x)$; implicit models (GANs) only provide a sampler.
- An ordinary autoencoder supplies reconstruction but no prior over codes, so decoding random codes need not give realistic samples.
- The ELBO equals $\log p_\theta(x)$ minus $D_{\mathrm{KL}}(q_\phi(z \mid x) \| p_\theta(z \mid x))$; reparameterization makes it trainable by backpropagation.
- With an optimal discriminator, the original GAN minimizes $2\,\mathrm{JSD} - \log 4$; the non-saturating loss, Wasserstein critics, gradient penalties, spectral normalization and R1 make the game trainable, but mode collapse must still be measured.
- Flows compute exact likelihoods via the change-of-variables formula, made cheap by triangular Jacobians in coupling layers.
- Likelihood, IS, FID and precision/recall measure different things; report several, with their exact protocol.

Exercises

1. **KL sanity check.** Substitute $\mu = 0$ and $\sigma^2 = 1$ into the Gaussian KL. Then compute the KL for $\mu = 1, \sigma^2 = 1$ in one dimension.

   Hint

   For $\mu = 0, \sigma^2 = 1$ every term is zero, because posterior and prior are the same distribution. For $\mu = 1$, $\sigma^2 = 1$ you get $\frac{1}{2}(1 + 1 - 0 - 1) = 0.5$ nats: shifting the mean by one standard deviation costs half a nat.

2. **Loss scaling.** Average the VAE's reconstruction across pixels rather than summing. What happens if KL is unchanged?

   Hint

   Reconstruction becomes 64 times smaller relative to KL. This is equivalent to a β-VAE with $\beta = 64$ on the summed loss: it effectively changes regularization (often toward posterior collapse), rather than merely changing reported units.

3. **Optimal discriminator.** At a point x where $p_{\mathrm{data}}(x) = 0.3$ and $p_g(x) = 0.1$, what is $D^*(x)$? What is D^* everywhere when $p_g = p_{\mathrm{data}}$, and what is the value of the game then?

Hint

$D^*(x) = 0.3/(0.3 + 0.1) = 0.75$. When the distributions match, $D^* = 1/2$ everywhere and $V = \log \frac{1}{2} + \log \frac{1}{2} = -\log 4$, consistent with JSD $= 0$.

4. **Affine flow.** For $x = Az + b$ with $z \sim \mathcal{N}(0, I)$, derive the density correction and explain why singular A invalidates it. Check your answer against the known Gaussian $\mathcal{N}(b, AA^\top)$.

Hint

Subtract $\log|\det A|$. Since $\det(AA^\top)^{1/2} = |\det A|$, this matches the normalizing constant of $\mathcal{N}(b, AA^\top)$. A singular map is not invertible and collapses volume, so the same-dimensional change-of-variables formula does not apply.

5. **Spot collapse.** Propose different checks for VAE posterior collapse and GAN mode collapse. Then modify the GAN code to use WGAN-GP (critic without sigmoid, gradient penalty with $\lambda = 10$, several critic steps per generator step) and compare the per-mode counts.

Hint

For a VAE, inspect the KL per latent dimension and whether decoded outputs change when you vary z. For a GAN, count samples per mode (as in our code), compute recall, and look for repeated samples at fixed conditions. They are different failures. For WGAN-GP, sample $\hat{x} = \alpha x_{\text{real}} + (1 - \alpha)x_{\text{fake}}$ with $\alpha \sim U(0, 1)$, call `torch.autograd.grad` with `create_graph=True`, and lower the learning rate if training oscillates. Compare across several seeds before concluding anything.

6. **Metric disagreement.** Using the precision/recall code, construct a fake set that has roughly the same precision and recall as the good generator but places half its samples in only four modes. What does this tell you about the limits of the metric?

Hint

Unequal mode weights can still give high recall, because recall only asks whether each real point is near **some** fake point, not whether mode frequencies match. Precision/recall measure support overlap, not density; pair them with a distribution-level measure such as FID or a per-mode histogram.

References

1. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
2. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book and notebooks
3. S. Bond-Taylor, A. Leach, Y. Long and C. G. Willcocks, "Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022. Paper

4. G. Papamakarios et al., “Normalizing Flows for Probabilistic Modeling and Inference,” *JMLR*, 2021. Review
5. L. Theis, A. van den Oord and M. Bethge, “A note on the evaluation of generative models,” ICLR, 2016. Paper
6. D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” ICLR, 2014. Paper
7. S. R. Bowman et al., “Generating Sentences from a Continuous Space,” CoNLL, 2016. Paper
8. I. Higgins et al., “β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework,” ICLR, 2017. Paper
9. D. P. Kingma and M. Welling, “An Introduction to Variational Autoencoders,” *Foundations and Trends in Machine Learning*, 2019. Review
10. A. Vahdat and J. Kautz, “NVAE: A Deep Hierarchical Variational Autoencoder,” NeurIPS, 2020. Paper
11. I. Goodfellow et al., “Generative Adversarial Nets,” NeurIPS, 2014. Paper
12. T. Salimans et al., “Improved Techniques for Training GANs,” NeurIPS, 2016. Paper
13. M. Arjovsky, S. Chintala and L. Bottou, “Wasserstein GAN,” ICML, 2017. Paper
14. I. Gulrajani et al., “Improved Training of Wasserstein GANs,” NeurIPS, 2017. Paper
15. T. Miyato et al., “Spectral Normalization for Generative Adversarial Networks,” ICLR, 2018. Paper
16. L. Mescheder, A. Geiger and S. Nowozin, “Which Training Methods for GANs do actually Converge?” ICML, 2018. Paper
17. T. Karras, S. Laine and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” CVPR, 2019. Paper
18. L. Dinh, J. Sohl-Dickstein and S. Bengio, “Density estimation using Real NVP,” ICLR, 2017. Paper
19. D. P. Kingma and P. Dhariwal, “Glow: Generative Flow with Invertible 1x1 Convolutions,” NeurIPS, 2018. Paper
20. D. J. Rezende and S. Mohamed, “Variational Inference with Normalizing Flows,” ICML, 2015. Paper
21. R. T. Q. Chen, Y. Rubanova, J. Bettencourt and D. Duvenaud, “Neural Ordinary Differential Equations,” NeurIPS, 2018. Paper
22. Y. Lipman et al., “Flow Matching for Generative Modeling,” ICLR, 2023. Paper
23. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 18–20. Book
24. Y. Song and D. P. Kingma, “How to Train Your Energy-Based Models,” arXiv, 2021. Paper
25. A. van den Oord, N. Kalchbrenner and K. Kavukcuoglu, “Pixel Recurrent Neural Networks,” ICML, 2016. Paper
26. J. Ho, A. Jain and P. Abbeel, “Denoising Diffusion Probabilistic Models,” NeurIPS, 2020. Paper
27. L. Yang et al., “Diffusion Models: A Comprehensive Survey of Methods and Applications,” *ACM Computing Surveys*, 2023. Survey
28. E. Nalisnick et al., “Do Deep Generative Models Know What They Don’t Know?” ICLR, 2019. Paper
29. M. Heusel et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” NeurIPS, 2017. Paper
30. M. S. M. Sajjadi, O. Bachem, M. Lucic, O. Bousquet and S. Gelly, “Assessing Generative Models via Precision and Recall,” NeurIPS, 2018. Paper

31. T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen and T. Aila, “Improved Precision and Recall Metric for Assessing Generative Models,” NeurIPS, 2019. Paper
32. G. Stein et al., “Exposing flaws of generative model evaluation metrics and their unfair treatment of diffusion models,” NeurIPS, 2023. Paper
33. S. Jayasumana et al., “Rethinking FID: Towards a Better Evaluation Metric for Image Generation,” CVPR, 2024. Paper
34. A. van den Oord, O. Vinyals and K. Kavukcuoglu, “Neural Discrete Representation Learning,” NeurIPS, 2017. Paper
35. R. Rombach et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” CVPR, 2022. Paper
36. M. Kang et al., “Scaling up GANs for Text-to-Image Synthesis,” CVPR, 2023. Paper
37. Y. Huang, A. Gokaslan, V. Kuleshov and J. Tompkin, “The GAN is dead; long live the GAN! A Modern GAN Baseline,” NeurIPS, 2024. Paper

ML 07 • Diffusion Models and Flow Matching

Derive noise, clean-image and velocity prediction, compare pixel and latent generation with JiT and representation autoencoders, and understand pretrained image/video diffusion and flow pipelines.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-07-diffusion-flow-matching/

What you'll learn

- How a known noise process turns "generate an image" into an ordinary supervised regression problem.
- How the simplified noise-prediction loss of DDPM falls out of a variational bound, step by step.
- How noise, clean-image and velocity predictions convert algebraically, why their losses and finite networks differ, and how score estimation and the SDE view connect them.
- How DDPM, DDIM and fast ODE solvers sample, and why the number of steps is a design choice.
- How classifier guidance and classifier-free guidance steer a sampler, and what they cost.
- How pixel and latent diffusion, DiT, JiT, representation autoencoders, flow matching and few-step generation fit together, and how pretrained image/video pipelines are evaluated.

The big picture

A diffusion model learns to undo controlled corruption. During training we take a real example, add a known amount of Gaussian noise, and ask a network to tell us what noise we added. Because we added the noise ourselves, the correct answer is always available: no labels, no adversary, just regression. During generation we start from pure noise and apply the learned "denoise a little" step many times until an example appears.

> **Explain like I'm five**
>
> Imagine slowly covering a drawing with pencil scribbles. You keep copies of the drawing at many scribble levels: a few lines, lots of lines, so many that the drawing is gone. A model practices one small job: look at a scribbled page, and guess which scribbles were added. After lots of practice you hand it a page that is *only* scribbles. It erases a little, looks again, erases a little more, and a brand-new drawing slowly appears — one that was never in the pile.

Why does this matter? Since 2021, diffusion-style models have produced many of the state-of-the-art results in image, video and audio synthesis, and the same machinery is used for inpainting, super-resolution, image editing, 3-D generation and scientific data. Diffusion also replaced two pain points of earlier families covered in ML 06: GANs need a fragile two-player game, and VAEs

tend to produce blurry samples. Diffusion training is a stable regression, and its samples are sharp because generation is spread over many small, easy steps.

The price is sampling cost: a classic sampler calls the network hundreds of times per image. Much of the research since 2021 — DDIM, fast ODE solvers, distillation, consistency models, flow matching, rectified flow — is about keeping the quality while cutting the number of network calls.

The chapter follows the organization of the major surveys. Yang et al. [1] group the field into efficient sampling, improved likelihood and data with special structure; Croitoru et al. [27] group it by formulation (DDPM, noise-conditioned score networks, SDEs); Lipman et al.'s flow-matching guide [2] and the monograph by Lai et al. [29] present diffusion and flows as one family of learned transport. Prince's textbook [14] is a gentle companion for the derivations. Throughout, we use **one explicit time convention per section** and say when it changes — mixing conventions is the most common source of sign errors in this area.

Forward diffusion has a closed form

Plain version. At each step, keep most of the image and add a little independent Gaussian noise. After enough steps, nothing of the image is left — only noise with a known distribution.

The idea goes back to Sohl-Dickstein et al. [15], who proposed destroying structure with a slow forward diffusion and learning to reverse it; DDPM [3] made it practical. Let $x_0 \in \mathbb{R}^d$ be a data example (an image flattened into d numbers, scaled to roughly $[-1, 1]$). Pick a **noise schedule** $\beta_1, \dots, \beta_T$ with $0 < \beta_t < 1$ and write $\alpha_t = 1 - \beta_t$. The forward (noising) process is a Markov chain

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\, x_{t-1},\ \beta_t I\big), \qquad t = 1, \dots, T,$$

where $\mathcal{N}(\mu, \Sigma)$ is a Gaussian with mean μ and covariance Σ, and I is the $d \times d$ identity. The factor $\sqrt{\alpha_t}$ shrinks the signal slightly so that the total variance stays bounded instead of growing; this is why this family is called **variance preserving (VP)**.

Deriving the one-shot formula

Write one step as $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t}\,\epsilon_t$ with $\epsilon_t \sim \mathcal{N}(0, I)$. Substituting the previous step,

$$\begin{aligned} x_t &= \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_{t-1}}\,\epsilon_{t-1}\right) + \sqrt{1-\alpha_t}\,\epsilon_t \\ &= \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \underbrace{\sqrt{\alpha_t(1-\alpha_{t-1})}\,\epsilon_{t-1} + \sqrt{1-\alpha_t}\,\epsilon_t}_{\text{sum of independent Gaussians}}. \end{aligned}$$

The two noise terms are independent zero-mean Gaussians, so their sum is Gaussian with variance $\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = 1 - \alpha_t\alpha_{t-1}$. Repeating the argument down to x_0 gives, with $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$,

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar\alpha_t}\,x_0,\ (1-\bar\alpha_t)I\big) \quad\Longleftrightarrow\quad x_t = \sqrt{\bar\alpha_t}\,x_0 + \sqrt{1-\bar\alpha_t}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

Here $\bar\alpha_t$ is the fraction of signal variance that survives to step t, and ϵ is a single standard Gaussian vector. This formula is what makes training cheap: we can jump straight to any noise level without simulating the earlier steps.

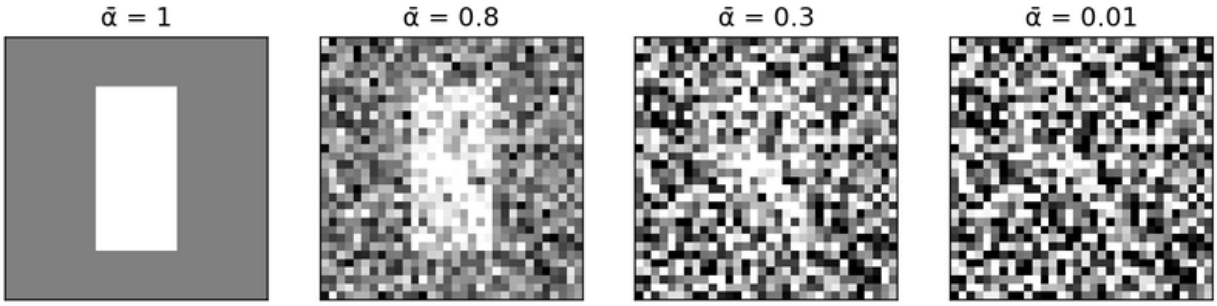


Figure 1 — The closed-form forward process at several signal levels. A shared noise realization makes the effect visible; training draws fresh noise for every example and every step.

Noise schedules and the signal-to-noise ratio

The **signal-to-noise ratio** at step t is

$$\mathrm{SNR}(t) = \frac{\bar\alpha_t}{1-\bar\alpha_t},$$

the ratio of signal variance to noise variance (for unit-variance data). It decreases monotonically from very large at $t = 1$ to nearly zero at $t = T$. Two requirements constrain the schedule: $\bar\alpha_T$ must be close to 0, so that x_T is practically a standard Gaussian (the sampler starts from $\mathcal{N}(0, I)$); and the steps in between should spend the training budget on noise levels where the network has something useful to learn.

DDPM [3] used $T = 1000$ and a **linear schedule** β_t from 10^{-4} to 0.02. Nichol and Dhariwal [19] observed that this destroys information quickly: for a large part of the chain the image is already nearly pure noise. Their **cosine schedule** defines $\bar\alpha_t = f(t)/f(0)$ with $f(t) = \cos^2\!\left(\frac{t/T+s}{1+s}\cdot\frac{\pi}{2}\right)$ and a small offset $s = 0.008$, and clips each β_t at 0.999. Figure 2 compares the two. Karras et al. [20] later argued that the schedule, the network's input/output scaling and the distribution of training noise levels are separate design choices that are best tuned independently.

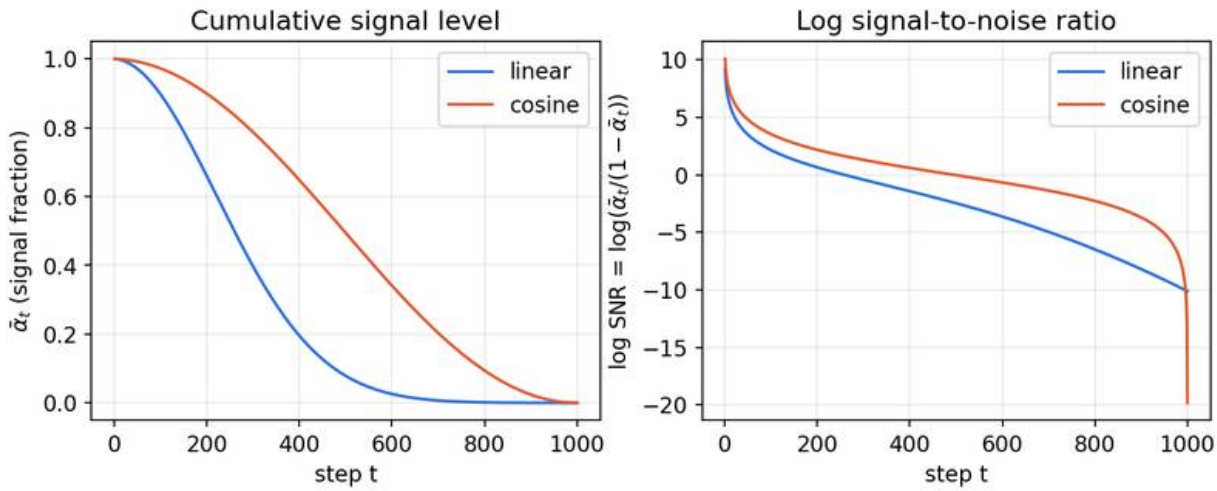


Figure 2 — Linear versus cosine schedule for $T = 1000$. The cosine schedule keeps more signal for longer and spreads the log-SNR more evenly over the steps.

The reverse process and the variational bound

Plain version. To generate, we need the reverse step: "given a noisy example, what did the slightly less noisy example look like?" We cannot compute this directly for real data, but we *can* compute it if we also know the clean example — and that turns out to give a simple training loss.

The model and the bound

The generative model is a reverse Markov chain that starts at $p(x_T) = \mathcal{N}(0, I)$ and uses learned Gaussian transitions

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t),\ \sigma_t^2 I\big)\,,$$

where μ_θ is a neural network (with parameters θ) and σ_t^2 is a variance we will choose. This is a latent-variable model with latents $x_{1:T}$, exactly like a deep VAE whose encoder q is fixed rather than learned (see ML 06). The same Jensen-inequality argument gives an evidence lower bound. Following [3], [15], the negative bound can be rearranged into a sum of KL divergences:

$$-\log p_\theta(x_0) \;\le\; \mathbb{E}_q\Big[\underbrace{D_{\mathrm{KL}}(q(x_T \mid x_0)\,\|\,p(x_T))}_{L_T} + \sum_{t=2}^{T} \underbrace{D_{\mathrm{KL}}(q(x_{t-1} \mid x_t, x_0)\,\|\,p_\theta(x_{t-1} \mid x_t))}_{L_{t-1}} \underbrace{-\log p_\theta(x_0 \mid x_1)}_{L_0}\Big].$$

D_{KL} is the Kullback–Leibler divergence. L_T has no learnable parameters (it is small when $\bar{\alpha}_T \approx 0$); L_0 is a decoder term for the last step; and each L_{t-1} compares the model's reverse step with the **forward posterior** $q(x_{t-1} \mid x_t, x_0)$ — the reverse step that would be exact if the clean example x_0 were known.

The forward posterior is Gaussian

By Bayes' rule, $q(x_{t-1} \mid x_t, x_0) \propto q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0)$. Both factors are Gaussian in x_{t-1}, so their product is Gaussian. Collecting the quadratic and linear terms in x_{t-1} (completing the square) gives

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(\tilde{\mu}_t(x_t, x_0),\ \tilde{\beta}_t I\big),$$

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\, x_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\, x_t, \qquad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t.$$

The posterior mean is a weighted average of the clean example and the current noisy one; the posterior variance $\tilde{\beta}_t$ is slightly smaller than β_t. You do not have to trust the algebra: the snippet below samples the forward chain and fits a regression of x_{t-1} on x_t.

```
import numpy as np

rng = np.random.default_rng(0)
beta = np.linspace(1e-4, 0.02, 1000)
alpha = 1 - beta
abar = np.cumprod(alpha)
t, x0, n = 300, 1.3, 400_000          # check q(x_{t-1} | x_t, x_0) at step t

# Sample the forward chain: x_{t-1} from q(x_{t-1}|x_0), then one more step.
x_prev = np.sqrt(abar[t-1])*x0 + np.sqrt(1-abar[t-1])*rng.normal(size=n)
x_t = np.sqrt(alpha[t])*x_prev + np.sqrt(beta[t])*rng.normal(size=n)

# Empirical regression of x_{t-1} on x_t (the posterior mean is linear in x_t).
slope, intercept = np.polyfit(x_t, x_prev, 1)
resid_var = np.var(x_prev - (slope*x_t + intercept))

# Closed-form posterior from the derivation.
c_x0 = np.sqrt(abar[t-1])*beta[t] / (1-abar[t])
c_xt = np.sqrt(alpha[t])*(1-abar[t-1]) / (1-abar[t])
var = beta[t]*(1-abar[t-1]) / (1-abar[t])
print(f"slope     {slope:.4f}  vs  {c_xt:.4f}")
print(f"intercept {intercept:.4f}  vs  {c_x0*x0:.4f}")
print(f"variance  {resid_var:.6f}  vs  {var:.6f}")
```

The output matches to three or four decimals (slope 0.9934 vs 0.9930, variance 0.006050 vs 0.006052); the small gaps are Monte Carlo error.

From the bound to the simplified loss

Now use the forward formula backwards: $x_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon)/\sqrt{\bar{\alpha}_t}$. Substituting into $\tilde{\mu}_t$ and simplifying with $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$,

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\epsilon\right).$$

At sampling time the network sees x_t but not ϵ. So DDPM parameterizes the model mean in exactly the same form, with the unknown noise replaced by a prediction $\epsilon_\theta(x_t, t)$:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right).$$

The KL between two Gaussians with the same fixed covariance $\sigma_t^2 I$ is a scaled squared distance between their means, so

$$L_{t-1} = \frac{1}{2\sigma_t^2} \|\tilde{\mu}_t - \mu_\theta\|^2 + C = \frac{\beta_t^2}{2\sigma_t^2 \, \alpha_t \, (1-\bar{\alpha}_t)} \|\epsilon - \epsilon_\theta(x_t, t)\|^2 + C,$$

where C does not depend on θ. Every term of the bound is therefore a **weighted noise-prediction error**. Ho et al. [3] found that dropping the weight, and sampling t uniformly, trains better:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,\, t\sim \mathcal{U}\{1,\dots,T\},\, \epsilon\sim\mathcal{N}(0,I)} \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon,\ t) \right\|^2.$$

Read it as an algorithm: pick a training example, pick a random step, pick random noise, build x_t in one line, and regress the noise. The time t is fed to the network (usually through a sinusoidal embedding) because the same pixel values mean different things at different noise levels.

Two points are easy to get wrong. First, $\mathcal{L}_{\text{simple}}$ is *not* the likelihood bound: removing the weights relatively down-weights the low-noise steps, which helps sample quality but hurts the reported log-likelihood. Improved DDPM [19] recovered good likelihoods by learning the variances σ_t^2 (as an interpolation between β_t and $\tilde{\beta}_t$) with a hybrid loss. Second, the same network also yields a **clean estimate**,

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}},$$

which every modern sampler uses internally. When $\bar{\alpha}_t$ is tiny, the division amplifies prediction errors, so $\hat{x}_0$ at very high noise is a blurry average, not a sample.

Noise, clean data, velocity and the score

Plain version. Predicting the noise, predicting the clean image and predicting "which direction makes this example more typical" are three ways to express the same knowledge. They differ in scaling, and scaling changes how the training loss weights different noise levels.

Three parameterizations

Because $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ is a fixed linear relation, a network can output any one of

- the noise ϵ (DDPM [3]),

- the clean example x_0,
- the **velocity** $v = \sqrt{\bar{\alpha}_t}\,\epsilon - \sqrt{1-\bar{\alpha}_t}\,x_0$, introduced by Salimans and Ho [22] for distillation, which stays well scaled at both ends of the schedule.

Given x_t and t, each converts to the others exactly. However, an unweighted MSE on ϵ, on x_0 or on v weights the noise levels differently, so the choice affects training. Karras et al. [20] made this explicit with input/output "preconditioning" coefficients that keep the network's inputs and targets at unit variance for every noise level. Note that this diffusion "velocity" is a reparameterization of signal and noise; it is related to, but not identical with, the transport velocity of flow matching later in this chapter.

Exact conversion does not make finite networks equivalent

For the variance-preserving path, write $a = \sqrt{\bar{\alpha}_t}$ and $b = \sqrt{1-\bar{\alpha}_t}$. Thus $a^2 + b^2 = 1$ and $x_t = ax_0 + b\epsilon$. The diffusion velocity $v_{\mathrm{VP}} = a\epsilon - bx_0$ completes a rotation:

$$x_0 = ax_t - bv_{\mathrm{VP}}, \qquad \epsilon = bx_t + av_{\mathrm{VP}},$$
$$\hat{x}_0^{(\epsilon)} = (x_t - b\hat{\epsilon})/a, \quad \hat{\epsilon}^{(x)} = (x_t - a\hat{x}_0)/b.$$

Here a, b are known schedule coefficients; v_{VP} is the target used in [22]; hats denote estimates; and superscripts identify the network's direct output. The last two conversions require nonzero divisors. Their instability near an endpoint is a numerical and statistical issue, even though the algebra is exact in the interior.

A noise error δ_ϵ becomes a clean-image error $-(b/a)\delta_\epsilon$. Therefore, for corresponding predictions at a fixed noise level,

$$\|\hat{x}_0 - x_0\|^2 = \frac{b^2}{a^2}\|\hat{\epsilon} - \epsilon\|^2 = \frac{1}{\mathrm{SNR}(t)}\|\hat{\epsilon} - \epsilon\|^2, \qquad \|\hat{x}_0^{(v)} - x_0\|^2 = b^2\|\hat{v}_{\mathrm{VP}} - v_{\mathrm{VP}}\|^2.$$

$\mathrm{SNR}(t) = a^2/b^2$ is the signal-to-noise ratio; the second identity follows from the rotation above. Equal unweighted losses consequently emphasize different times. Changing the target without recording time sampling and loss weighting confounds an experiment [20], [22].

There is also a capacity issue. If a network directly outputs a clean image, its final representation can discard nuisance noise. A network directly outputting noise may need to preserve more coordinates of the noisy input, or use a skip connection that supplies those coordinates. Converting one network's output afterward does not make its architecture identical to a network trained to output another target. This distinction becomes especially relevant for large pixel patches and high-dimensional latents; the JiT discussion below supplies recent evidence [42]. It does not imply that noise prediction generally fails: the earlier DDPM examples and dense image denoisers are counterexamples.

```
import numpy as np

rng = np.random.default_rng(7)
```

```
x0 = rng.normal(size=32)
eps = rng.normal(size=32)
a, b = np.sqrt(.2), np.sqrt(.8)
xt = a*x0 + b*eps
v_vp = a*eps - b*x0
assert np.allclose(a*xt - b*v_vp, x0)
assert np.allclose(b*xt + a*v_vp, eps)

# A deliberately imperfect noise estimate: compare equivalent losses.
eps_hat = eps + .03*rng.normal(size=32)
x_hat = (xt - b*eps_hat)/a
noise_mse = np.mean((eps_hat - eps)**2)
clean_mse = np.mean((x_hat - x0)**2)
assert np.allclose(clean_mse/noise_mse, b*b/(a*a))
print("Clean/noise MSE ratio:", round(clean_mse/noise_mse, 3))

# A straight flow uses a different path and a different velocity.
t = .35
zt = (1-t)*eps + t*x0
u = x0 - eps
x_from_flow = zt + (1-t)*u
u_from_clean = (x0 - zt)/(1-t)
assert np.allclose(x_from_flow, x0)
assert np.allclose(u_from_clean, u)
print("VP velocity equals straight-flow target:", np.allclose(v_vp, u))
```

The example prints a loss ratio of 4.0 and `False` for the two velocity targets. Both use the letter v in parts of the literature; a checkpoint's scheduler must identify the actual path, normalization and time direction.

> **Interactive demonstration** (**online**): open the online edition of this tutorial to use this demonstration.Open demonstration

What is a score, and why does denoising estimate it?

The **score** of a density p is $\nabla_x \log p(x)$: a vector field that points toward higher probability. If you know the score, you can sample with **Langevin dynamics**: repeatedly step a little along the score and add a little noise. Song and Ermon [17] trained "noise-conditional score networks" at many noise levels and sampled with annealed Langevin dynamics — a parallel route to diffusion that turned out to be the same idea.

The bridge is **denoising score matching** (Vincent [16]). For Gaussian corruption, the best possible noise predictor (in mean squared error) is a conditional expectation, $\epsilon^*(x_t, t) = \mathbb{E}[\epsilon \mid x_t]$, and **Tweedie's formula** [40] links it to the score of the noisy marginal p_t:

$$\nabla_{x_t} \log p_t(x_t) = -\frac{\mathbb{E}[\epsilon \mid x_t]}{\sqrt{1-\bar{\alpha}_t}} \quad \Longrightarrow \quad s_\theta(x_t, t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}},$$

with s_θ the learned score. The same formula in "clean" units reads $\mathbb{E}[x_0 \mid x_t] = (x_t + (1 - \bar{\alpha}_t)\nabla \log p_t(x_t))/\sqrt{\bar{\alpha}_t}$. Two warnings: this is the score of the *noisy marginal distribution* at level t, not the gradient for any single training image; and it holds for the optimal predictor — a trained network approximates it.

You can verify Tweedie's formula where everything is known: one-dimensional Gaussian data, for which the best noise predictor is linear in x_t.

```
import numpy as np

rng = np.random.default_rng(0)
s, abar, n = 0.7, 0.4, 500_000     # data x0 ~ N(0, s^2); one noise level
x0 = s*rng.normal(size=n)
eps = rng.normal(size=n)
xt = np.sqrt(abar)*x0 + np.sqrt(1-abar)*eps

# "Train" the best linear noise predictor eps_hat = w * x_t by least squares.
w = (xt @ eps) / (xt @ xt)
score_from_eps = -w / np.sqrt(1-abar)       # s(x_t) = -eps_hat / sqrt(1-abar)

# Exact: x_t ~ N(0, abar s^2 + 1 - abar), so grad log p_t(x) = -x / var.
exact = -1 / (abar*s**2 + 1 - abar)
print(f"score slope from noise predictor: {score_from_eps:.4f}")
print(f"exact score slope:                {exact:.4f}")
```

It prints -1.2560 versus -1.2563: a regression on noise recovers the score.

The SDE view

Song et al. [4] took the step count to infinity. With continuous time $t \in [0, 1]$ and a rate $\beta(t)$, the VP forward process becomes the **stochastic differential equation (SDE)**

$$dx = -\tfrac{1}{2}\beta(t)\, x\, dt + \sqrt{\beta(t)}\, dw,$$

where w is a standard Wiener process (Brownian motion). A classical result says that this process run backwards in time is again an SDE, and that the only unknown in it is the score:

$$dx = \Big[-\tfrac{1}{2}\beta(t)\, x - \beta(t)\, \nabla_x \log p_t(x) \Big] dt + \sqrt{\beta(t)}\, d\bar{w},$$

with dt negative and $\bar{w}$ a reverse-time Wiener process. Moreover, there is a deterministic **probability-flow ODE** whose solutions have the same marginal distributions p_t at every time:

$$\frac{dx}{dt} = -\tfrac{1}{2}\beta(t)\Big[x + \nabla_x \log p_t(x) \Big].$$

This unifies the field. DDPM's ancestral sampler is a discretization of the reverse SDE; NCSN [17] corresponds to a "variance exploding" SDE; and the ODE turns a diffusion model into a continuous normalizing flow, so exact log-likelihoods can be computed with the instantaneous change-of-variables formula [4]. The ODE also explains why deterministic samplers such as DDIM work, and why general-purpose ODE solvers can be applied.

Sampling is a reverse process

Plain version. A loss evaluated at one random noise level is not a generator. To generate, you must chain many reverse steps with a consistent schedule, starting from pure noise — and you can choose how many steps and how much fresh randomness to inject.

DDPM ancestral sampling

DDPM samples by following the learned reverse chain:

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z, \qquad z \sim \mathcal{N}(0, I), \qquad \sigma_t^2 \in \{\beta_t,\ \tilde{\beta}_t\},$$

with no noise added at the final step. Ho et al. [3] report similar results for $\sigma_t^2 = \beta_t$ and $\sigma_t^2 = \tilde{\beta}_t$; learned variances [19] help most when using fewer steps.

Code: DDPM on a two-dimensional distribution

This complete CPU example (about 10 seconds) uses two Gaussian clusters centered at $x = \pm 1.5$ with standard deviation 0.25. It contains every operation of an image model — schedule, random-time training, noise prediction, reverse mean — without an image-scale budget. With 100 steps and this schedule, almost no signal survives to $t = T$.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
T = 100
beta = torch.linspace(.001, .15, T)
alpha = 1-beta
abar = alpha.cumprod(0)
previous = torch.cat([torch.ones(1), abar[:-1]])
post_var = beta * (1-previous) / (1-abar)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
def data(n):
    side = torch.randint(2, (n, 1))*2-1
```

```
    return torch.cat([side*1.5, torch.zeros(n, 1)], 1) + .25*torch.randn(n, 2)
for step in range(3000):
    x0 = data(128)
    t = torch.randint(T, (len(x0),))
    eps = torch.randn_like(x0)
    a = abar[t, None]
    xt = a.sqrt()*x0 + (1-a).sqrt()*eps
    predicted = net(torch.cat([xt, t[:, None].float()/(T-1)], 1))
    loss = F.mse_loss(predicted, eps)
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()
with torch.no_grad():
    x = torch.randn(512, 2)
    for t in reversed(range(T)):
        time = torch.full((len(x), 1), t/(T-1))
        eps = net(torch.cat([x, time], 1))
        mean = (x-beta[t]/(1-abar[t]).sqrt()*eps)/alpha[t].sqrt()
        x = mean + post_var[t].sqrt()*torch.randn_like(x) if t else mean
    assert torch.isfinite(x).all()
    print("terminal signal:", round(abar[-1].item(), 6))
    left = x[x[:, 0] < 0]
    print("fraction in left cluster:", round((x[:, 0] < 0).float().mean().item(), 3))
    print("left cluster: mean x", round(left[:, 0].mean().item(), 2),
          "std", round(left[:, 0].std().item(), 2), "(data: -1.5 and 0.25)")
```

On our machine it prints a terminal signal of 0.000349, a left-cluster fraction of 0.441, and a left-cluster mean and standard deviation of -1.47 and 0.25. The cluster *shapes* are right; the 44/56 split shows the small network has not learned the mode weights exactly. This is why a single summary number is never enough: always plot samples, and check mode coverage, within-mode spread and outliers against fresh data.

DDIM: same training, different sampler

Song, Meng and Ermon [5] noticed that $\mathcal{L}_{\text{simple}}$ only depends on the marginals $q(x_t \mid x_0)$, not on the Markov chain that produced them. Many non-Markovian forward processes share those marginals, and each gives a different reverse sampler — all usable with the *same* trained ϵ_θ. The

DDIM update, for any decreasing subsequence of steps $\tau_1 > \tau_2 > \cdots$, is

$$x_{\tau_{i+1}} = \sqrt{\bar\alpha_{\tau_{i+1}}}\,\hat{x}_0 + \sqrt{1-\bar\alpha_{\tau_{i+1}}-\sigma^2}\,\epsilon_\theta(x_{\tau_i},\tau_i) + \sigma z, \qquad \sigma = \eta\sqrt{\frac{1-\bar\alpha_{\tau_{i+1}}}{1-\bar\alpha_{\tau_i}}\Big(1-\frac{\bar\alpha_{\tau_i}}{\bar\alpha_{\tau_{i+1}}}\Big)},$$

where $\hat{x}_0$ is the clean estimate at x_{τ_i} and $\eta \geq 0$ controls stochasticity. Read it as: "estimate the clean example, then re-noise it to the next, lower noise level using the predicted noise direction." With $\eta = 1$ and consecutive steps, this recovers DDPM's posterior variance; with $\eta = 0$ the sampler

is **deterministic** and is a discretization of the probability-flow ODE. Determinism has practical consequences: the same starting noise always gives the same output, so you can interpolate in

noise space or invert a real image to its noise (DDIM inversion), which is the basis of many editing methods.

The following standalone example repeats the same small training setup so it can run independently. When experimenting in one session, reuse the already trained `net`, `abar` and `T` instead:

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
T = 100
beta = torch.linspace(.001, .15, T)
alpha = 1-beta
abar = alpha.cumprod(0)
previous = torch.cat([torch.ones(1), abar[:-1]])
post_var = beta * (1-previous) / (1-abar)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
def data(n):
    side = torch.randint(2, (n, 1))*2-1
    return torch.cat([side*1.5, torch.zeros(n, 1)], 1) + .25*torch.randn(n, 2)
for step in range(3000):
    x0 = data(128)
    t = torch.randint(T, (len(x0),))
    eps = torch.randn_like(x0)
    a = abar[t, None]
    xt = a.sqrt()*x0 + (1-a).sqrt()*eps
    predicted = net(torch.cat([xt, t[:, None].float()/(T-1)], 1))
    loss = F.mse_loss(predicted, eps)
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()

def ddim_sample(n, steps, eta=0.0, seed=1):
    g = torch.Generator().manual_seed(seed)
    taus = torch.linspace(T-1, 0, steps).round().long()
    x = torch.randn(n, 2, generator=g)
    with torch.no_grad():
        for i, t in enumerate(taus):
            time = torch.full((n, 1), t.item()/(T-1))
            eps = net(torch.cat([x, time], 1))
            a_t = abar[t]
            a_prev = abar[taus[i+1]] if i+1 < steps else torch.tensor(1.)
            x0_hat = (x - (1-a_t).sqrt()*eps) / a_t.sqrt()
            sigma = eta*((1-a_prev)/(1-a_t)*(1-a_t/a_prev)).sqrt()
            x = (a_prev.sqrt()*x0_hat + (1-a_prev-sigma**2).sqrt()*eps
                 + sigma*torch.randn(n, 2, generator=g))
    return x

for steps in [5, 20, 100]:
    s = ddim_sample(2000, steps)
    left = s[s[:, 0] < 0]
```

```
    print(f"{steps:3d} steps: left fraction {(s[:, 0] < 0).float().mean():.3f}, "
          f"left-cluster mean x {left[:, 0].mean():.2f}, std {left[:, 0].std():.2f}")
```

The output shows the typical pattern. With 5 steps the left cluster is centered at −1.11 with standard deviation 0.39 — the samples are pulled toward the middle and smeared, because large steps follow a curved trajectory with straight lines. With 20 steps it is −1.37 and 0.30; with 100 steps −1.42 and 0.24, close to the data. No retraining was needed to change the step count.

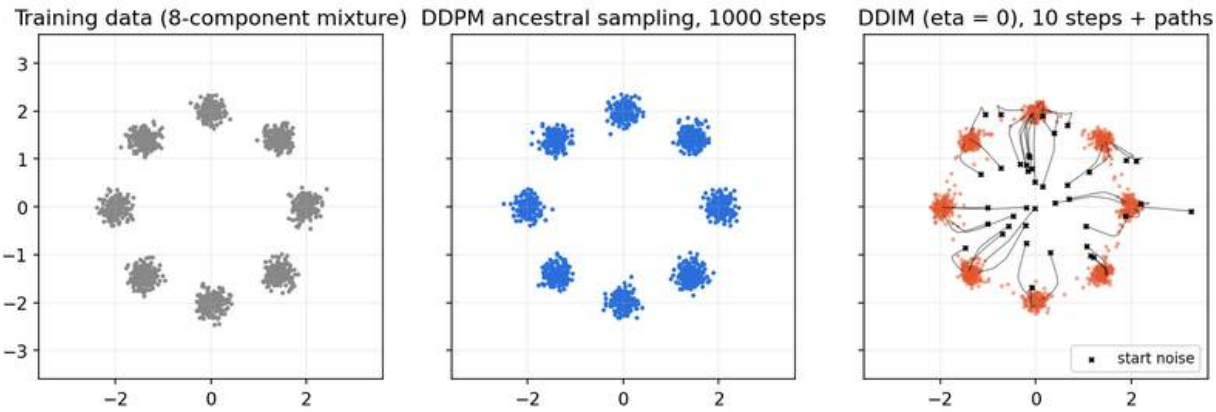


Figure 3 — Samplers driven by the exact denoiser of an 8-component mixture, so only the sampling rule differs. DDPM with 1000 steps reproduces the data; deterministic DDIM with 10 steps lands near the modes but with slightly wider, smeared clusters. Black lines are DDIM trajectories from starting noise (×).

Faster solvers

Once sampling is "solve an ODE," numerical analysis applies. Karras et al. (EDM) [20] used a second-order Heun solver with a carefully chosen spacing of noise levels and showed that sampler, schedule, preconditioning and training noise distribution can be tuned separately. DPM-Solver [21] exploits the semi-linear structure of the diffusion ODE — the linear part can be integrated exactly — to reach good quality in around 10 network evaluations. Stochastic samplers add a little fresh noise at each step; that costs more steps but can correct accumulated errors.

Whatever the solver, report the **number of function evaluations** (**NFE**): the number of network calls, which, rather than "steps," is the true sampling cost.

Steering generation: conditioning and guidance

Plain version. To generate "a cat" instead of "anything," give the network the condition. Guidance then turns a dial: follow the condition more strictly, at the cost of variety.

Conditioning

A conditional model learns $\epsilon_\theta(x_t, t, c)$ for a condition c. Class labels enter through embeddings added to the time embedding; text enters through **cross-attention** from image features to text-encoder tokens [7]; images (for super-resolution, inpainting, editing) are concatenated to the input channels or fed through extra encoders.

Classifier guidance

Bayes' rule for the noisy marginals gives $\nabla \log p_t(x_t \mid y) = \nabla \log p_t(x_t) + \nabla \log p_t(y \mid x_t)$. Dhariwal and Nichol [18] trained a separate classifier $p_\phi(y \mid x_t)$ on *noisy* images and added its gradient, scaled by w, to an unconditional model. In noise units,

$$\hat{\epsilon} = \epsilon_\theta(x_t, t) - w\sqrt{1-\bar{\alpha}_t}\,\nabla_{x_t} \log p_\phi(y \mid x_t).$$

A scale $w > 1$ samples roughly from $p_t(x_t)\,p_t(y \mid x_t)^w$, a sharpened distribution that trades diversity for fidelity. Together with architecture improvements, this was the work that showed diffusion models beating GANs on ImageNet [35] sample quality [18]. Its drawback is the extra classifier, which must be trained on noisy inputs for every noise level.

Classifier-free guidance (CFG)

Ho and Salimans [6] removed the classifier. During training, the condition is replaced by a "null" token for a random fraction of examples, so one network learns both $\epsilon_\theta(x_t, t, c)$ and $\epsilon_\theta(x_t, t, \emptyset)$. Bayes' rule says the classifier gradient equals the difference of the conditional and unconditional scores, so the implicit-classifier version of the formula above is

$$\epsilon_{\text{guided}} = \epsilon_\theta(x_t, t, \emptyset) + w(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \emptyset)).$$

In this convention $w = 0$ is unconditional, $w = 1$ is the ordinary conditional model, and $w > 1$ extrapolates past it. (Some papers write the same thing as $(1 + w')\epsilon_c - w'\epsilon_u$, so their "scale" is $w - 1$ in our notation; always check the convention before comparing numbers.)

The snippet below uses the exact noise predictor of a four-blob mixture, so any odd behaviour is due to guidance itself, not to a badly trained network. Class 1 is the two blobs with $x > 0$.

```
import numpy as np

# Data: four Gaussian blobs at the corners of a square; class c is the sign of x.
MU = np.array([[-1.5, -1.5], [-1.5, 1.5], [1.5, -1.5], [1.5, 1.5]])
SIG = 0.2
CLASS = np.array([0, 0, 1, 1])

def eps_exact(x, abar, comps):
    """Exact E[eps | x_t] for a Gaussian mixture restricted to `comps`."""
    m = np.sqrt(abar)*MU[comps]
    var = abar*SIG**2 + 1 - abar
```

```
    logw = -((x[:, None] - m[None])**2).sum(-1) / (2*var)
    w = np.exp(logw - logw.max(1, keepdims=True))
    w /= w.sum(1, keepdims=True)
    score = -(w[..., None]*(x[:, None] - m[None])).sum(1) / var
    return -np.sqrt(1-abar)*score

abar = np.cumprod(1 - np.linspace(1e-4, 0.02, 1000))
taus = np.linspace(999, 0, 50).round().astype(int)
everything, class1 = np.arange(4), np.where(CLASS == 1)[0]
print("data: mean distance to its mode", round(SIG*np.sqrt(np.pi/2), 3))
for w in [0, 1, 3, 8]:
    x = np.random.default_rng(0).normal(size=(4000, 2))
    for i, t in enumerate(taus):                    # deterministic DDIM
        e_u, e_c = eps_exact(x, abar[t], everything), eps_exact(x, abar[t], class1)
        e = e_u + w*(e_c - e_u)                     # classifier-free guidance
        x0 = (x - np.sqrt(1-abar[t])*e) / np.sqrt(abar[t])
        a_prev = abar[taus[i+1]] if i+1 < len(taus) else 1.0
        x = np.sqrt(a_prev)*x0 + np.sqrt(1-a_prev)*e
    dist = np.linalg.norm(x[:, None] - MU[None], axis=-1).min(1)
    print(f"w={w}: class-1 fraction {(x[:, 0] > 0).mean():.3f}, "
          f"mean distance to nearest mode {dist.mean():.3f}")
```

The data's mean distance to its mode is 0.251. With $w = 0$ about half the samples (0.490) are class 1; with $w = 1$ all of them are, at distance 0.225 (50 DDIM steps slightly under-disperse). At $w = 3$ the distance is 0.259 and at $w = 8$ it grows to 0.446: strong guidance does not just "make it more class 1" — it pushes samples *past* the data, a toy version of the over-saturated colours and distorted shapes seen with very high guidance in image models. Figure 4 shows the same effect on the 8-mode mixture.

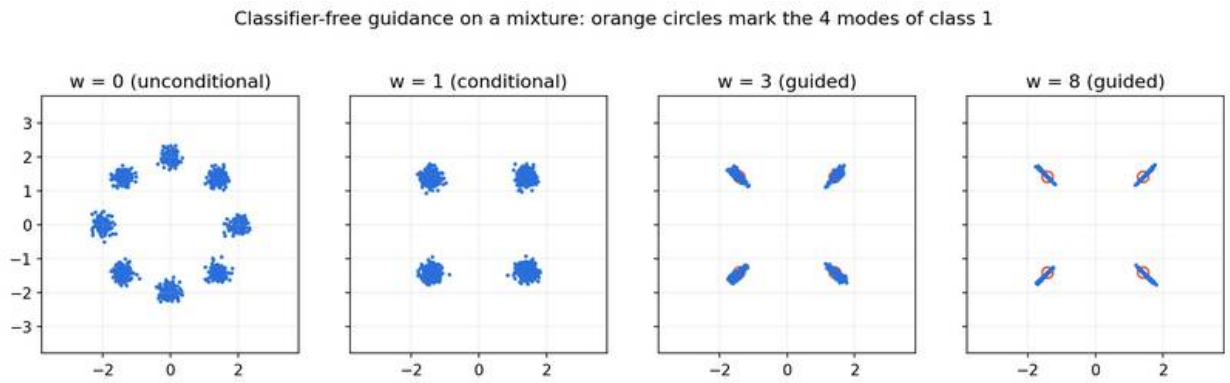


Figure 4 — Exact classifier-free guidance on an 8-mode mixture whose class 1 has 4 modes (orange circles). $w = 1$ already restricts samples to class 1; larger w shrinks clusters along some directions and stretches them along others — the samples no longer follow any data distribution.

Guidance has a compute cost too: each step needs a conditional *and* an unconditional prediction, so a 30-step guided sampler costs 60 network evaluations unless the two are fused. And w is a sampling knob, not a calibrated measure of how well the image matches the prompt.

Latent diffusion and diffusion transformers

Plain version. Pixels are expensive. Compress the image into a smaller grid of features with an autoencoder, run diffusion in that compressed space, then decode. Separately, the denoising network itself can be a transformer instead of a convolutional U-Net [37].

Latent diffusion

Rombach et al. [7] train an autoencoder first: an encoder $z = \mathcal{E}(x)$ maps an $H \times W \times 3$ image to an $h \times w \times c$ latent with a spatial downsampling factor $f = H/h$, and a decoder $\mathcal{D}$ maps back. The autoencoder is trained with reconstruction, perceptual and adversarial losses plus a light regularizer (a small KL term toward a Gaussian, or vector quantization) so that the latent space stays well behaved. Diffusion is then trained on z exactly as before, and generation ends with $\hat{x} = \mathcal{D}(z_0)$. With $f = 8$, a 512×512 image becomes a 64×64 latent: 64 times fewer spatial positions for every network call.

The trade-off is a reconstruction bottleneck: information discarded by the encoder cannot be faithfully recovered from that code, although the decoder may synthesize plausible detail. The latent's scale must also match what the diffusion model was trained on; implementations rescale latents to roughly unit variance. Text conditioning in [7] uses cross-attention layers in the U-Net, the design behind the Stable Diffusion family.

Diffusion transformers

DiT (Peebles and Xie [8]) replaces the U-Net with a Vision Transformer [41] (see ML 04) over latent patches. A latent of size $h \times w$ is cut into $p \times p$ patches, giving hw/p^2 tokens; smaller p means more tokens, more compute and better quality. The timestep and class label condition every block through **adaptive layer norm**: they predict the scale and shift of each LayerNorm, and in the "adaLN-Zero" variant also a gating factor initialized to zero, so each residual block starts as the identity. The paper's main finding is a scaling trend: across model sizes and patch sizes, more transformer compute (GFLOPs) gave consistently lower FID [8].

It helps to keep three choices separate. The **architecture** (U-Net or transformer) is the function approximator. The **objective** (noise, x_0, v, or flow velocity) is what it regresses. The **sampler** (DDPM, DDIM, ODE solver) is how predictions become an image. Stable Diffusion 3 [9] is an example of a new combination: a latent transformer with separate weights for the image and text token streams that attend jointly ("MM-DiT"), trained with a rectified-flow objective and a timestep distribution biased toward intermediate noise levels.

For video, the latent and the attention must cover time as well as space, so compression and attention patterns become the main engineering problem. Melnik et al.'s survey [31] reviews architectures, temporal-consistency mechanisms and evaluation for video diffusion. Per-frame quality

is not enough there: object permanence, motion plausibility and long-duration consistency need their own evaluation.

Pixel space, latent space and representation autoencoders

Plain version. The generator needs a place to do its repeated updates. That place can be the image itself, a compact reconstruction code, or features learned by a visual representation model. A latent stage buys a particular computational and statistical trade-off; it is not a prerequisite for generative modeling.

Why pixels become latents and then pixels again

Training images arrive as pixels, while the output must be viewable as pixels. Between these endpoints, an encoder can remove redundancy and produce a grid that is cheaper to update repeatedly [7]. Train the encoder and decoder on reconstruction first, then fit a generative distribution to encoded training images. These are two different tasks: reconstruction preserves a given example; generation learns which encoded examples are plausible.

The training and sampling routes are

$$\begin{aligned} \text{training: } & x \xrightarrow{\mathcal{E}} z_{\text{data}} \xrightarrow{\text{corrupt}} z_t \xrightarrow{\text{prediction loss}} \theta, \\ \text{generation: } & z_{\text{noise}} \xrightarrow{\text{learned sampler}} \hat{z} \xrightarrow{\mathcal{D}} \hat{x}, \\ \text{reconstruction check: } & x \xrightarrow{\mathcal{E}} \mathcal{E}(x) \xrightarrow{\mathcal{D}} \tilde{x}. \end{aligned}$$

Here x is a training image; $\mathcal{E}$ and $\mathcal{D}$ are the encoder and decoder; z_{data} is an encoded image; z_t is its corrupted state; θ parameterizes the generative predictor; $\hat{z}$ is a generated code; $\hat{x}$ is the generated image; and $\tilde{x}$ is a reconstruction. Pure text-to-image sampling does **not** encode an input image first. Editing and image-to-image generation can encode an actual input image and begin from a partially corrupted code.

The main saving occurs because the denoiser or flow predictor runs repeatedly, while encoding training data and decoding a sample occur much less often. A spatial factor f reduces grid positions by f^2, but the number of latent channels also matters. For an image with three channels and a latent with c channels, the scalar-count reduction is $3f^2/c$. It is incorrect to describe the grid reduction as the reduction in all floating-point values or total runtime.

For a transformer, token count and token width must also be distinguished. Full attention stores a matrix with a number of entries proportional to N^2, where N is the token count; feature projections depend on token width as well [41]. A patch size can reduce N in either pixels or latents. None of these counts alone predicts wall-clock speed because kernels, memory traffic, the decoder and conditioning contribute.

```
# Synthetic shape bookkeeping, not a benchmark or a specific checkpoint.
H = W = 512
channels, spatial_factor, patch = 4, 8, 2
pixel_values = H*W*3
h = H//spatial_factor
w = W//spatial_factor
latent_values = h*w*channels
pixel_tokens = (H//patch)*(W//patch)
latent_tokens = (h//patch)*(w//patch)
print("Spatial positions reduction:", (H*W)//(h*w))
print("Scalar values reduction:", pixel_values//latent_values)
print("Dense attention entries reduction:", pixel_tokens**2//latent_tokens**2)
assert pixel_values//latent_values == 48
assert pixel_tokens//latent_tokens == 64
assert pixel_tokens**2//latent_tokens**2 == 4096
```

This constructed example yields 64 times fewer spatial positions, 48 times fewer scalar values and 4096 times fewer attention entries **at the same numerical patch size in each coordinate space**. Changing patch sizes changes the comparison. A pixel transformer with much larger patches can have the same token count as a latent transformer.

The cost is information loss. If two images have the same code, a deterministic decoder cannot recover their different exact details. A decoder may invent plausible texture, but that does not restore information about the original image. The reconstruction check therefore deserves its own evaluation before generative training. Inspect fine lines, lettering and temporal changes, rather than relying only on average image error.

The following deliberately simple encoder averages blocks and the decoder repeats those averages. It demonstrates a bottleneck without pretending to reproduce a learned VAE or RAE.

```
import numpy as np

y, x = np.mgrid[:64, :64]
checker = ((x+y) % 2).astype(float)
smooth = x/63
image = .6*smooth + .4*checker
f = 4
latent = image.reshape(64//f, f, 64//f, f).mean(axis=(1, 3))
reconstruction = np.repeat(np.repeat(latent, f, axis=0), f, axis=1)
print("Synthetic reconstruction MSE:", round(np.mean((image-reconstruction)**2), 6))
assert np.mean((image-reconstruction)**2) > .04
# Recovering the exact latent does not recover the missing checkerboard.
assert np.allclose(reconstruction[0, :f], reconstruction[0, 0])
```

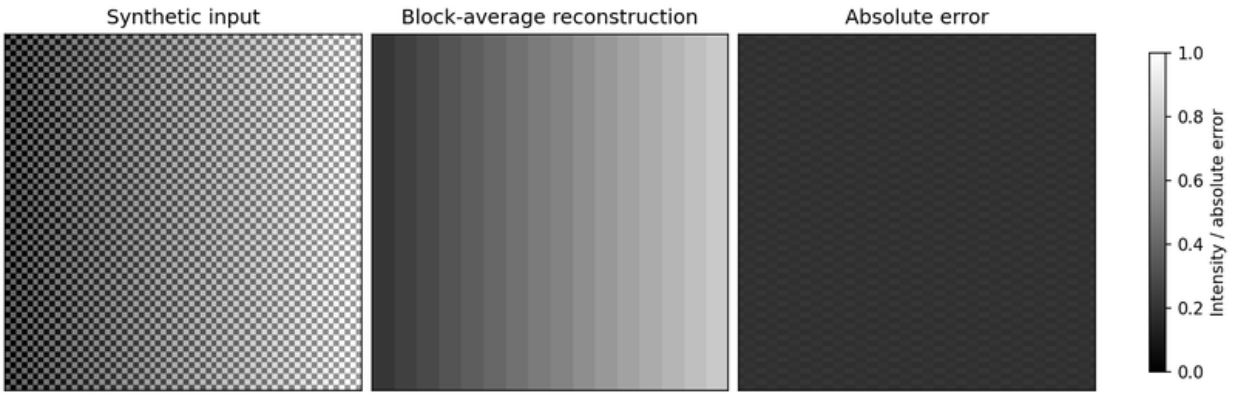


Figure 5 — A latent bottleneck can remove detail before the generative model is trained. This block-average toy is an explanation of information loss, not an evaluation of a learned autoencoder.

For editing, repeatedly encoding and decoding a result can accumulate distortion. Keep intermediate edits in the compatible latent space when the pipeline permits it, and inspect the eventual pixel output. A good unconditional generator may still be a poor reconstruction system. The claim that an autoencoder sets a ceiling concerns faithfulness to encoded information; it does not mean that a generative decoder can never synthesize new fine texture.

Reconstruction codes versus semantic representation codes

A VAE encoder is trained with a reconstruction objective and a distributional regularizer; its code is designed to support decoding and sampling [7]. A pretrained recognition encoder instead learns useful visual features. These goals can align, but recognition invariance may discard details needed for faithful reconstruction. The decoder and the retained feature structure determine whether the representation is a useful generative space.

Zheng, Ma, Tong and Xie's **Representation Autoencoders** (**RAEs**) [43] pair a frozen pretrained representation encoder with a trained decoder. Their ICLR 2026 study uses semantic visual features as the space for generative modeling. It examines high-dimensional token features, dimension-dependent noise scheduling, decoder robustness and a shallow wide diffusion head. This changes the representation being generated, rather than simply adding a feature-alignment loss to a conventional latent predictor.

Do not equate "latent" with "smaller in every dimension." A spatially sparse feature grid can have many channels per token. An RAE is also not automatically a variational model: a frozen feature extractor plus a decoder does not by itself define a Gaussian posterior, a KL regularizer or a likelihood bound. Keep reconstruction, representation quality and generative quality as separate measurements.

Tong et al. [44] extend this approach to freeform text-to-image generation in 2026. Their study compares RAE and VAE spaces at larger scales and examines decoder data composition. It finds that dimension-aware noise scheduling remains important while some small-scale architectural

and decoder modifications offer little benefit at the tested larger scales. This is a reason to re-test recipes when scale and data change, rather than treating an ImageNet result as a universal design rule.

For a new application, inspect whether the chosen encoder preserves the content that matters: fine text for documents, small defects for inspection, or frame-to-frame differences for video. Evaluate the decoder on held-out real features and on generated features. A decoder trained only on exact training codes can be sensitive to small off-distribution errors in a sampled code. Normalize features using the training recipe, and record the encoder version with the generative checkpoint.

JiT: direct clean-image prediction on pixels

Li and He's **Just image Transformers (JiT)** [42], published at CVPR 2026 after a 2025 preprint, revisit large-patch transformers operating directly on pixels. The network predicts the clean image; sampling derives a flow velocity from that prediction. Their controlled experiments support a manifold-based explanation: a bottleneck can retain structured clean-image information while making direct prediction of high-dimensional noise or velocity difficult. The reported setup uses no pretrained latent tokenizer or auxiliary representation loss. This is evidence for a particular architecture and training regime, not a proof that pixel-space models always outperform latent generators.

The crucial distinction is **direct network output versus the quantity used in the loss or solver**. For the straight path defined below, use $y_t = (1-t)\epsilon + tx$, with noise at $t = 0$ and clean data at $t = 1$. A network that directly predicts $\hat{x}_\theta(y_t, t)$ can supply

$$\hat{u}_\theta(y_t, t) = \frac{\hat{x}_\theta(y_t, t) - y_t}{1-t}, \qquad \|\hat{u}_\theta - (x-\epsilon)\|^2 = \frac{\|\hat{x}_\theta - x\|^2}{(1-t)^2}.$$

Here x is the clean image; ϵ is the noise draw; y_t is their interpolation; $\hat{x}_\theta$ is the direct prediction; and $\hat{u}_\theta$ is the corresponding transport velocity. The identity holds for $t < 1$ before numerical clipping. An x-output network can thus train with a velocity-space loss. Calling it "clean prediction" does not mean its loss is an unweighted clean-image MSE. JiT's released implementation makes this distinction explicit [42].

Near $t = 1$, division by $1 - t$ needs an endpoint convention and numerical safeguards. For the interior algebra, a fixed clean-image error is amplified increasingly as the endpoint approaches. During sampling, use the checkpoint's time distribution, noise scaling and solver recipe. Changing the direct output label alone does not reproduce the method.

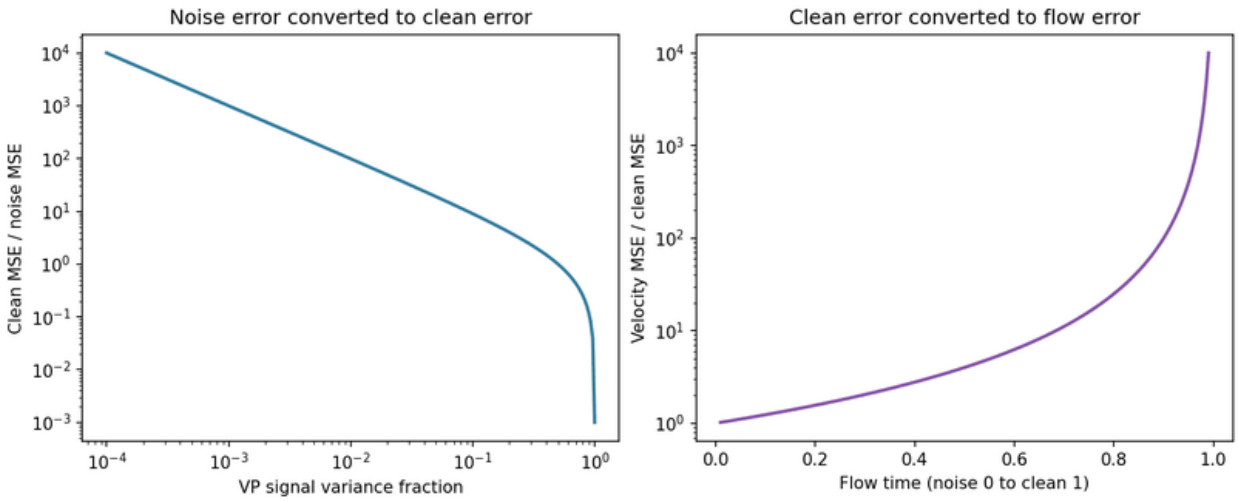


Figure 6 — Equivalent predictions use different units. The left curve shows noise-error amplification at low signal; the right shows velocity-error amplification near the clean endpoint. The curves are algebraic identities, not model performance measurements.

A useful comparison fixes training data, compute, resolution, token count and the sampler, then changes the representation or target one at a time. Compare a pixel transformer, a compact VAE-latent transformer and a representation-latent transformer on reconstruction needs, generated quality and total cost. Record encoder/decoder cost and any representation pretraining. A model that uses a pretrained visual encoder has inherited training even when its generative stage starts from scratch.

Flow matching: learn a transport velocity

Plain version. Forget noise schedules for a moment. Draw a noise point and a data point, connect them by a straight line, and train a network to predict the direction of travel at every point along such lines. To generate, drop a fresh noise point and follow the predicted directions.

Time convention change. In this section time runs from **noise at** $t = 0$ **to data at** $t = 1$, the opposite of the diffusion sections above (where large t meant more noise).

The conditional flow-matching objective

A continuous normalizing flow moves points with an ODE $dx_t/dt = v(x_t, t)$. If the velocity field v is right, it transports the noise distribution $p_0 = \mathcal{N}(0, I)$ to the data distribution p_1. Lipman et al. [10] (and, concurrently, Liu et al. [11] and Albergo and Vanden-Eijnden's stochastic interpolants [34]) showed how to train v without ever simulating the ODE.

Pair noise $z \sim \mathcal{N}(0, I)$ with a data example $x \sim p_1$ and define the straight **conditional path**

$$x_t = (1-t)\,z + t\,x, \qquad \frac{dx_t}{dt} = u_t(x_t \mid z, x) = x - z.$$

Every pair moves at constant speed in a straight line. The marginal velocity we actually want is the average of these conditional velocities over all pairs that pass through the point:

$$u_t(y) = \mathbb{E}[\,x - z \mid x_t = y\,].$$

We cannot compute this average, but we do not need to. Compare the "ideal" loss with the one we can compute:

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}_{t,x_t}\|v_\theta(x_t,t) - u_t(x_t)\|^2, \qquad \mathcal{L}_{\mathrm{CFM}} = \mathbb{E}_{t,z,x}\|v_\theta(x_t,t) - (x-z)\|^2,$$

with $t \sim \mathcal{U}[0,1]$. Expand both squares. The $\|v_\theta\|^2$ terms are identical. The cross terms are equal because, by the tower rule,

$\mathbb{E}[v_\theta(x_t,t)^\top (x-z)] = \mathbb{E}[v_\theta(x_t,t)^\top\, \mathbb{E}[x-z \mid x_t]] = \mathbb{E}[v_\theta^\top u_t(x_t)]$. The remaining terms do not involve θ. Therefore

$$\nabla_\theta \mathcal{L}_{\mathrm{CFM}} = \nabla_\theta \mathcal{L}_{\mathrm{FM}},$$

so regressing the per-pair velocity $x-z$ trains the marginal velocity field. This is the same trick as in diffusion: a target that is noisy for any single example has the right conditional mean. Generation solves $dx/dt = v_\theta(x,t)$ from $t=0$ to $t=1$ with any ODE solver.

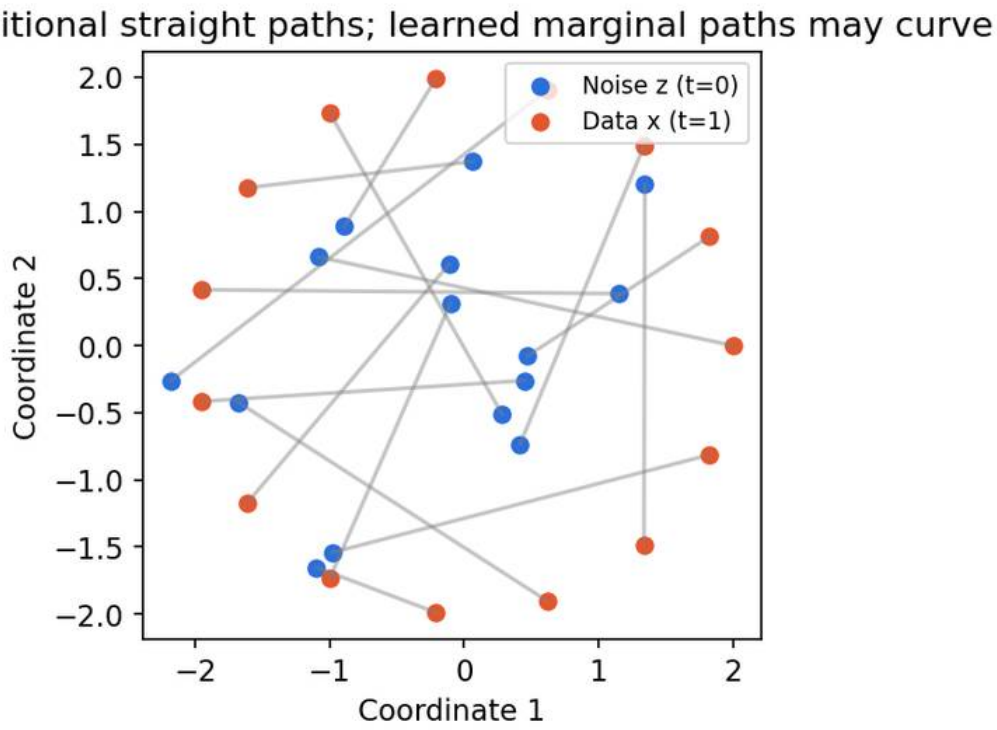


Figure 7 — Conditional training paths are straight. Paths of different pairs cross, so the marginal velocity at a crossing point is an average, and the learned trajectories can curve.

How flow matching relates to diffusion

The linear path is a Gaussian path: given x, $x_t \sim \mathcal{N}(t\,x,\ (1-t)^2 I)$. Writing $\hat{z} = \mathbb{E}[z \mid x_t]$ (a noise prediction) and using $x = (x_t - (1-t)z)/t$,

$$u_t(x_t) = \frac{x_t - \hat{z}}{t}, \qquad \nabla \log p_t(x_t) = -\frac{\hat{z}}{1-t}, \qquad \text{so} \qquad u_t(x_t) = \frac{x_t + (1-t)\,\nabla \log p_t(x_t)}{t}.$$

So a flow-matching velocity, a noise predictor and a score are again interchangeable for this path; what differs is the path (straight lines versus the VP curve), the weighting of noise levels in the loss, and the time convention. Diffusion's probability-flow ODE is itself a flow along a particular Gaussian path. Lipman et al.'s guide [2] and the stochastic-interpolant framework [34] treat both as special cases of one construction. The practical advice is the same as for parameterizations: the targets are related by derivation, not interchangeable by renaming.

Code: flow matching on two clusters

This standalone example learns the same two-cluster distribution as the DDPM snippet with linear conditional paths, then samples with Euler steps of size $1/N$.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
for step in range(3000):
    z = torch.randn(128, 2)
    side = torch.randint(2, (128, 1))*2-1
    x = torch.cat([1.5*side, torch.zeros(128, 1)], 1) + .25*torch.randn(128, 2)
    t = torch.rand(128, 1)
    xt = (1-t)*z+t*x
    loss = F.mse_loss(net(torch.cat([xt, t], 1)), x-z)
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()
def euler_sample(n, steps, seed=1):
    x = torch.randn(n, 2, generator=torch.Generator().manual_seed(seed))
    with torch.no_grad():
        for k in range(steps):
            t = torch.full((n, 1), k/steps)
            x = x + net(torch.cat([x, t], 1))/steps
    return x

for steps in [1, 4, 50]:
    s = euler_sample(2000, steps)
    assert torch.isfinite(s).all()
    left = s[s[:, 0] < 0]
```

```
    print(f"{steps:2d} Euler steps: left fraction {(s[:, 0] < 0).float().mean():.3f}, "
          f"left-cluster mean x {left[:, 0].mean():.2f}, std {left[:, 0].std():.2f}")
```

It prints, for 1, 4 and 50 steps, left-cluster means of -0.18, -1.20 and -1.45 with standard deviations 0.12, 0.42 and 0.29. One Euler step lands almost at the overall data mean: at $t = 0$ the marginal velocity is $\mathbb{E}[x] - z$, so a single full step sends *every* noise point to $\mathbb{E}[x]$. Straight conditional paths do not make the learned marginal flow straight. Figure 8 shows the same effect with the exact velocity of an 8-mode mixture.

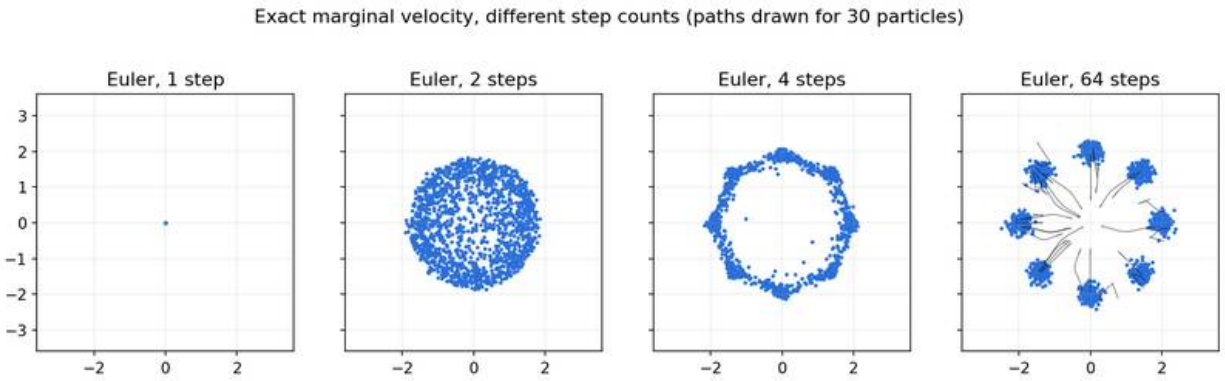


Figure 8 — Even with the exact marginal velocity, Euler sampling needs many steps: one step collapses every sample to the data mean, and the 64-step trajectories (black) bend toward the modes late in the path.

Rectified flow and reflow

Liu, Gong and Liu [11] use the same straight interpolation ("rectified flow") and add **reflow**: after training, generate pairs $(z, \hat{x})$ by solving the learned ODE from each z, then retrain on these *coupled* pairs instead of independent ones. Because the new pairs are already connected by the flow, their straight lines cross less, the learned trajectories become straighter, and fewer Euler steps are needed; a straight enough flow can then be distilled into a one-step model. Stable Diffusion 3 [9] showed that rectified-flow training of a large latent transformer works at text-to-image scale.

Pretrained image and video generators as complete pipelines

Plain version. A pretrained generator is a bundle of compatible components. Loading the denoiser weights alone does not establish the right representation, text conditioning, prediction target or sampling schedule. The image/video boundary also changes what a reconstruction must preserve.

Read the components before running the checkpoint

A practical pipeline combines a text encoder, an optional image/video encoder, a time-conditioned predictor, a sampler and an optional decoder. A separate refinement model may follow the first generator. Each component expects particular tensor shapes and scales. A scheduler declaring `v_prediction` may mean VP velocity; a flow scheduler may expect a derivative along a straight interpolation. Check the training path rather than guessing from a configuration string.

The following verified examples illustrate architectural differences. They are fixed research baselines, rather than a ranking of the latest product releases. The references include their papers or technical reports and official release documentation.

Baseline	Generation representation and predictor	What to inspect in the pretrained pipeline
SDXL [45]	Reconstruction latents with a convolutional denoiser	Text conditioning, latent scaling, optional base/refiner composition and compatible sampling configuration
Stable Diffusion 3 [9]	Latent rectified flow with a joint text–image transformer	Flow time convention, text streams, time sampling and autoencoder compatibility
FLUX.1 [46]	Latent rectified-flow transformer	Exact release variant, guidance behavior, text conditioning, autoencoder and release terms
JiT [42]	Class-conditioned clean-image prediction directly on pixels	Image normalization, patch/ resolution configuration, derived velocity and sampling recipe; this is not a freeform text-to-image checkpoint
HunyuanVideo [47]	Compressed video latents with a flow-matching transformer	Video autoencoder, image/video joint training, temporal dimensions and text-conditioning components
Wan2.1 [48]	Video diffusion transformer with a spatiotemporal autoencoder	Text-to-video versus image-to-video variant, causal video compression, temporal length and decode caching

SDXL [45] documents a larger latent image-generation architecture and optional refinement stage. Stable Diffusion 3 [9] illustrates how the same broad latent pipeline can instead use rectified flow and joint text/image attention. FLUX.1's official model card [46] describes a released rectified-flow transformer; its `.1-dev` variant is guidance-distilled. These facts do not make the checkpoints interchangeable. Ordinary two-pass CFG and a distilled guidance input can have different meanings and costs.

For image-to-image generation, encoding an input and adding partial noise trades input preservation against the ability to change it. The actual noise level is determined by the schedule, not merely by a UI label such as "strength." To compare edits, keep the same input and record the initial time, seed, prompt, guidance and reconstruction baseline. A reconstruction artifact can otherwise be mistaken for an edit requested by the text.

Video adds temporal compression and a consistency problem

Let a video have F frames, height H and width W. A hypothetical encoder compressing time by f_t and each spatial axis by f_s gives a rough latent grid of $(F/f_t) \times (H/f_s) \times (W/f_s)$ positions. Exact shapes depend on padding and special handling of the first frame. After patching, dense attention across all positions grows quadratically with the remaining token count; separately reducing time and space can therefore change cost substantially.

HunyuanVideo's technical report [47] combines a 3D video autoencoder with a flow-matching transformer and staged image/video training. Wan's report [48] explains a causal video VAE with special first-frame handling, spatial and temporal compression, and feature caching during chunked encoding/decoding. Causality in an autoencoder does not by itself imply that the entire video generator samples autoregressively; the generative attention and sampling procedure must be examined separately.

A frame can look excellent while a clip changes an object's identity, reverses motion unexpectedly or forgets an occluded object. Test reconstruction of held-out clips before generation, then evaluate appearance, motion, prompt correspondence and consistency across frames. Melnik et al. [31] provide the survey map for these video-specific issues. A per-image metric alone cannot diagnose temporal behavior.

A pretrained video generator also does not automatically become a reliable world model. Predicting plausible clips differs from predicting the consequences of a specified action. The world-model chapter develops this distinction through state, actions, rollouts and evaluation. Use the actual conditioning interface and evaluate the task the system claims to solve.

A reproducible first experiment

Start by recording a pipeline manifest: checkpoint revision, encoder/decoder revisions, text encoder, representation shape and scaling, prediction target, time direction, solver, evaluation count, guidance, dtype and seed. Then perform a reconstruction check if an autoencoder is present and generate a fixed small prompt/seed set. Compare one change at a time. Include decoder latency and peak memory when reporting the cost of a latent method.

This chapter's CPU examples test the underlying identities and failure modes without downloading weights. Full-resolution pretrained generation belongs in a separately provisioned experiment using the official release instructions [42], [45]–[48]. Treat an actual measured run, a paper's published result and a toy calculation as three different kinds of evidence.

Few-step and one-step generation

Plain version. If a sampler needs 50 network calls, train a second model that jumps along the trajectory in a few calls — or train one from scratch that is designed to jump.

The main families are:

- **Progressive distillation** [22]: a student learns to match two deterministic teacher steps with one step; repeating the procedure halves the step count each round.
- **Consistency models** [12]: learn a function $f_\theta(x_t, t)$ that maps *any* point on a probability-flow ODE trajectory to that trajectory's starting (clean) point. It must satisfy the boundary condition $f_\theta(x, \epsilon) = x$ at the smallest time ϵ, enforced by writing $f_\theta(x,t) = c_{\text{skip}}(t)\, x + c_{\text{out}}(t)\, F_\theta(x,t)$ with $c_{\text{skip}}(\epsilon) = 1$ and $c_{\text{out}}(\epsilon) = 0$. Training asks outputs at adjacent points of a trajectory to agree. In *consistency distillation*, adjacent points come from one ODE-solver step of a pretrained diffusion model; in *consistency training*, from the same data example noised at two levels, with no teacher. Sampling takes one step, or a few steps of "jump to clean, re-noise, jump again." Improved training recipes [32] simplified continuous-time consistency models and scaled them to 1.5 billion parameters on ImageNet 512 × 512 with two-step sampling.
- **Adversarial distillation**: ADD [33] combines score distillation from a large pretrained diffusion teacher with an adversarial loss so that a student produces sharp images in one to four steps.
- **Average-velocity models**: MeanFlow [13] learns the *average* velocity $u(x_t, r, t) = \frac{1}{t-r}\int_r^t v(x_s, s)\, ds$ over an interval instead of the instantaneous velocity v. Differentiating $(t-r)\,u = \int_r^t v\, ds$ with respect to t gives the identity $u = v - (t-r)\frac{d}{dt}u$, which supplies a training target without a teacher. One network call over the whole interval then generates a sample; the paper reports an FID of 3.43 with one evaluation on ImageNet 256 × 256, trained from scratch [13].

A small step count is a learned capability, not a free consequence of a nicer equation. When comparing, count NFE including guidance evaluations, the decoder's cost in latent models, resolution, quality *and* diversity — and remember that a one-step network can be larger and slower per call than a multi-step one.

Evaluating generative models

Plain version. "Does it look good?" hides at least three questions: are individual samples realistic (fidelity), does the set of samples cover the variety of the data (diversity), and does the sample match the condition (alignment)? No single number answers all three.

Fréchet Inception Distance

FID (Heusel et al. [23]) embeds real and generated images with an Inception-v3 network [36], fits a Gaussian to each set of features, and computes the Fréchet (2-Wasserstein) distance between the two Gaussians:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \operatorname{tr}\Big(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\Big),$$

where μ_r, Σ_r and μ_g, Σ_g are the feature mean and covariance of the real and generated sets and tr is the matrix trace. Lower is better. The snippet applies the formula to two-dimensional "features."

```
import numpy as np
from scipy import linalg

def frechet_distance(a, b):
    """Frechet distance between Gaussians fitted to feature sets a and b."""
    mu1, mu2 = a.mean(0), b.mean(0)
    c1, c2 = np.cov(a, rowvar=False), np.cov(b, rowvar=False)
    covmean = linalg.sqrtm(c1 @ c2).real
    return float(((mu1 - mu2)**2).sum() + np.trace(c1 + c2 - 2*covmean))

rng = np.random.default_rng(0)
centers = np.array([[-1.5, 0.], [1.5, 0.]])
real = centers[rng.integers(2, size=5000)] + .25*rng.normal(size=(5000, 2))
good = centers[rng.integers(2, size=5000)] + .25*rng.normal(size=(5000, 2))
one_mode = centers[[0]] + .25*rng.normal(size=(5000, 2))      # mode collapse
blurry = .6*rng.normal(size=(5000, 2))                         # mass in the gap
print("good     :", round(frechet_distance(real, good), 3))
print("one mode :", round(frechet_distance(real, one_mode), 3))
print("blurry   :", round(frechet_distance(real, blurry), 3))
small = [frechet_distance(real[:50], good[i*50:(i+1)*50]) for i in range(20)]
print("good, 50 samples: mean", round(np.mean(small), 3), "std", round(np.std(small), 3))
```

It prints about 0.001 for a correct sampler, 3.9 for a sampler that only produces the left mode, and 0.976 for a "blurry" sampler that puts its mass between the modes. The last line shows a practical trap: with 50 samples, even a *perfect* sampler scores 0.077 ± 0.073, because FID is biased upward for small sample sizes. Compare FIDs only with the same number of samples, the same reference set and the same feature extractor and preprocessing.

Beyond FID

- **Precision and recall** (Kynkäänniemi et al. [24]) separate fidelity from diversity: estimate the real and generated "manifolds" with k-nearest-neighbour balls in feature space; precision is the fraction of generated samples inside the real manifold, and recall the fraction of real samples inside the generated one. Our "one mode" sampler would have high precision and low recall.
- **Feature extractor matters.** Stein et al. [25] ran a large human study and found that no common metric correlated strongly with human judgments; in particular, the perceptual realism of diffusion models was not reflected in FID. They recommend features from DINOv2-ViT-L/14 [39] instead of Inception, and show that current metrics cannot separate memorization from underfitting or mode shrinkage.
- **CMMD** (Jayasumana et al. [26]) replaces Inception features with CLIP [38] embeddings and the Gaussian assumption with a maximum mean discrepancy using a Gaussian RBF kernel, giving an unbiased estimator that is more sample-efficient.

- **Likelihood.** Through the probability-flow ODE, diffusion models can report exact log-likelihoods (in bits per dimension) [4], [19]. Likelihood measures something different from visual quality and the two can disagree.
- **Conditional models** need alignment checks (does the image show what the prompt says?), usually by human rating or by vision–language models, and memorization checks against the training set.
- **Cost.** Report NFE, guidance evaluations, resolution and wall-clock time on stated hardware.

The evaluation chapter ML 10 returns to sample-size effects and honest comparisons.

Modern view

Surveys and reference texts. The literature is large; these reviews are good maps.

- Yang et al. [1] (*ACM Computing Surveys*, 2023) organize diffusion research into three improvement directions — efficient sampling, improved likelihood estimation, and data with special structure (manifolds, discrete data, invariances) — and also cover combinations with VAEs, GANs and flows and applications across domains.
- Croitoru et al. [27] (*IEEE TPAMI*, 2023) focus on vision. They identify three generic formulations — DDPMs, noise-conditioned score networks and SDEs — and categorize vision applications such as generation, super-resolution, inpainting, editing, translation and segmentation.
- Cao et al. [28] (*IEEE TKDE*, 2024) take a methods angle: fundamental formulation, algorithmic enhancements (sampling acceleration, objective design, conditioning) and applications from images to biology and healthcare.
- Lipman et al. [2] (2024) is a guide-with-code to flow matching that covers Gaussian and general probability paths, the link to diffusion and extensions to discrete and Riemannian data.
- Lai, Song, Kim, Mitsufuji and Ermon [29] (2025) is a book-length monograph presenting the variational, score-based and flow-based views as three routes to the same time-dependent velocity field, followed by guidance, fast solvers and flow-map (few-step) models.
- Shen et al. [30] (*TMLR*, 2025) survey efficiency at the algorithm level (training, sampling, compression), system level (hardware and serving) and framework level.
- Melnik et al. [31] (*TMLR*, 2024) survey video diffusion: architectures, temporal-consistency mechanisms, datasets and evaluation.

State of the art, 2023–2026. Several trends are visible across these reviews and the primary papers in this chapter. (1) *Transformers in latent space*: DiT-style backbones [8] replaced U-Nets in many large systems, and joint text–image transformers trained with rectified-flow objectives [9] became a standard text-to-image recipe. (2) *Flow matching as the default objective*: straight-path flow matching [10], [11] is now used alongside or instead of noise prediction, with diffusion understood as one choice of probability path [2], [29]. (3) *Fewer evaluations*: distillation [22], [33], consistency models [12], [32] and average-velocity models [13] bring sampling down to one to four network

calls with quality approaching multi-step samplers. (4) *Better evaluation*: the community is moving from Inception-based FID toward richer feature spaces, human studies and alignment metrics [25], [26]. (5) *Beyond images*: video, 3-D, audio and scientific data reuse the same machinery, with video's temporal consistency and cost the main open challenges [31].

Representation and prediction, 2025–2026. JiT [42] tests direct clean-image prediction on pixels; RAE [43] tests generation in frozen semantic feature spaces; its text-to-image follow-up [44] examines larger-scale behavior. These studies separate where the model operates from what the network directly predicts. The pretrained examples above [45]–[48] show why component compatibility and temporal reconstruction matter in actual image/video pipelines. Compare these choices under matched compute and data, including representation pretraining.

Open problems. Sampling cost at high resolution and long video duration; principled guidance that does not trade away diversity or produce over-saturation; evaluation metrics that agree with people and detect memorization; understanding generalization (why models produce new samples rather than copies of training data); controllability and editing with guarantees; and data provenance, consent and misuse, which are social rather than purely technical questions. The next chapter, ML 08, covers the main alternative route: autoregressive generation over pixels and tokens, and its hybrids with diffusion.

Key takeaways

- The forward process has a closed form $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$, so training reduces to "pick a random noise level and regress the noise."
- Every term of the variational bound is a weighted noise-prediction error; $\mathcal{L}_{\text{simple}}$ drops the weights, which improves samples but is not the likelihood.
- Noise, clean-image, VP velocity, score and flow velocity convert under a specified path; time direction, loss weighting, endpoint stability and network capacity matter. JiT directly outputs clean pixels while deriving a velocity for training and sampling.
- Sampling is solving a reverse SDE or ODE: DDPM, DDIM and higher-order solvers trade steps for accuracy with the same trained network; count NFE, not "steps."
- Guidance trades diversity for adherence and doubles the per-step cost for CFG; very large weights push samples off the data distribution.
- Pixels, reconstruction latents and RAE features are different generative spaces. Latent compression saves repeated updates but can lose information; pretrained image/video pipelines require compatible encoders, predictors, guidance and samplers.
- Conditional flow matching trains the marginal velocity by regressing per-pair straight-line velocities, but straight conditional paths do not make the learned flow straight — few-step generation needs reflow, distillation, consistency or average-velocity training.
- No single metric measures quality: FID depends on sample size and feature extractor; pair it with precision/recall, human or alignment studies and cost.

Exercises

1. **Recover the clean estimate.** Starting from $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$, derive $\hat{x}_0$ from a noise prediction. If ϵ_θ has an error δ, how large is the error in $\hat{x}_0$ at $\bar{\alpha}_t = 0.25$ and at $\bar{\alpha}_t = 10^{-4}$? Then derive the straight-flow velocity from a clean prediction and explain why JiT can output x while using a velocity loss.

   Hint

   $\hat{x}_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta)/\sqrt{\bar{\alpha}_t}$, so the error is $\sqrt{1-\bar{\alpha}_t}\,\delta/\sqrt{\bar{\alpha}_t}$: about $1.7\,\delta$ at $\bar{\alpha}_t = 0.25$ and about $100\,\delta$ at 10^{-4}. At high noise, $\hat{x}_0$ is an average over many plausible images, not one image.

2. **Posterior mean in two forms.** Substitute $x_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon)/\sqrt{\bar{\alpha}_t}$ into $\tilde{\mu}_t$ and show that it equals $\frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon)$. Then change `t` in the posterior snippet to 50 and 900 and check that the formula still matches.

   Hint

   Use $\bar{\alpha}_t = \alpha_t\bar{\alpha}_{t-1}$ and $\beta_t = 1 - \alpha_t$. The coefficient of x_t becomes $\frac{\beta_t + \alpha_t(1-\bar{\alpha}_{t-1})}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)} = \frac{1}{\sqrt{\alpha_t}}$ because the numerator equals $1 - \bar{\alpha}_t$.

3. **Count compute.** A latent text-to-image model samples with 30 DDIM steps and classifier-free guidance using separate conditional and unconditional passes, then runs the decoder once. How many denoiser evaluations are needed? How does the count change for a 4-step distilled model that has guidance "baked in"? For the synthetic shape example, change the latent channels to 16 and the pixel patch size to 16. Recompute scalar counts and attention entries. Which savings remain, and which depend on patching?

   Hint

   60 denoiser evaluations plus one decoder pass; batching both CFG branches reduces wall-clock time but not the work. A guidance-distilled 4-step model needs 4 evaluations plus the decoder. Compare quality and diversity, not just the counts.

4. **Time convention.** Diffusion sections use "large t = noisy"; the flow-matching section uses "$t = 0$ noise, $t = 1$ data." Reparameterize the flow with $s = 1 - t$. What is dx/ds, and what goes wrong if you only relabel the time input?

   Hint

   $dx/ds = -v(x, 1-s)$. If you feed s to a network trained on t without flipping the sign and the time input, you integrate in the wrong direction and move samples away from the data.

5. **Guidance sweep.** In the CFG snippet, add $w \in \{0.5, 1.5, 5\}$ and also record the fraction of class-1 samples that land closer to the wrong-class modes. Plot mean distance to the nearest mode against w. Where does guidance stop helping?

Hint

Already at $w = 1$ the exact conditional model puts every sample in class 1, so larger w cannot improve class accuracy here — it only distorts the clusters. With a learned, imperfect conditional model, moderate $w > 1$ can help adherence, which is why it is used in practice.

6. **Sampler ablation.** Hold the trained flow-matching network fixed. Compare Euler with 1, 2, 5, 20 and 100 steps against a second-order Heun step (predict with Euler, then average the velocities at both ends). Use the same starting noise and measure cluster means and spreads. Does the 2-step Heun sampler beat 4-step Euler at the same number of network calls?

Hint

Heun uses two network calls per step. Plot samples, not only numbers. One Euler step of a model trained for instantaneous velocity does not test a separately trained one-step model such as a consistency or MeanFlow model.

References

1. L. Yang, Z. Zhang, Y. Song, S. Hong, R. Xu, Y. Zhao, W. Zhang, B. Cui and M.-H. Yang, "Diffusion Models: A Comprehensive Survey of Methods and Applications," *ACM Computing Surveys*, 2023. arXiv
2. Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu and I. Gat, "Flow Matching Guide and Code," arXiv:2412.06264, 2024. arXiv
3. J. Ho, A. Jain and P. Abbeel, "Denoising Diffusion Probabilistic Models," *NeurIPS*, 2020. arXiv
4. Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon and B. Poole, "Score-Based Generative Modeling through Stochastic Differential Equations," *ICLR*, 2021. arXiv
5. J. Song, C. Meng and S. Ermon, "Denoising Diffusion Implicit Models," *ICLR*, 2021. arXiv
6. J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications; arXiv:2207.12598, 2022. arXiv
7. R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," *CVPR*, 2022. arXiv
8. W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," *ICCV*, 2023. arXiv
9. P. Esser, S. Kulal, A. Blattmann et al., "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis," *ICML*, PMLR 235, 2024. PMLR
10. Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel and M. Le, "Flow Matching for Generative Modeling," *ICLR*, 2023. arXiv
11. X. Liu, C. Gong and Q. Liu, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow," *ICLR*, 2023. arXiv

12. Y. Song, P. Dhariwal, M. Chen and I. Sutskever, “Consistency Models,” *ICML*, 2023. arXiv
13. Z. Geng, M. Deng, X. Bai, J. Z. Kolter and K. He, “Mean Flows for One-step Generative Modeling,” *NeurIPS*, 2025. arXiv
14. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. book site
15. J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan and S. Ganguli, “Deep Unsupervised Learning using Nonequilibrium Thermodynamics,” *ICML*, PMLR 37, 2015. PMLR
16. P. Vincent, “A Connection Between Score Matching and Denoising Autoencoders,” *Neural Computation*, vol. 23, no. 7, pp. 1661–1674, 2011. doi
17. Y. Song and S. Ermon, “Generative Modeling by Estimating Gradients of the Data Distribution,” *NeurIPS*, 2019. arXiv
18. P. Dhariwal and A. Nichol, “Diffusion Models Beat GANs on Image Synthesis,” *NeurIPS*, 2021. NeurIPS
19. A. Q. Nichol and P. Dhariwal, “Improved Denoising Diffusion Probabilistic Models,” *ICML*, PMLR 139, 2021. PMLR
20. T. Karras, M. Aittala, T. Aila and S. Laine, “Elucidating the Design Space of Diffusion-Based Generative Models,” *NeurIPS*, 2022. arXiv
21. C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li and J. Zhu, “DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps,” *NeurIPS*, 2022. arXiv
22. T. Salimans and J. Ho, “Progressive Distillation for Fast Sampling of Diffusion Models,” *ICLR*, 2022. arXiv
23. M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter, “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” *NeurIPS (NIPS)*, 2017. arXiv
24. T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen and T. Aila, “Improved Precision and Recall Metric for Assessing Generative Models,” *NeurIPS*, 2019. arXiv
25. G. Stein, J. C. Cresswell, R. Hosseinzadeh et al., “Exposing Flaws of Generative Model Evaluation Metrics and Their Unfair Treatment of Diffusion Models,” *NeurIPS*, 2023. arXiv
26. S. Jayasumana, S. Ramalingam, A. Veit, D. Glasner, A. Chakrabarti and S. Kumar, “Rethinking FID: Towards a Better Evaluation Metric for Image Generation,” *CVPR*, 2024. CVF
27. F.-A. Croitoru, V. Hondru, R. T. Ionescu and M. Shah, “Diffusion Models in Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023. doi
28. H. Cao, C. Tan, Z. Gao, Y. Xu, G. Chen, P.-A. Heng and S. Z. Li, “A Survey on Generative Diffusion Models,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 2814–2830, 2024. doi
29. C.-H. Lai, Y. Song, D. Kim, Y. Mitsufuji and S. Ermon, “The Principles of Diffusion Models,” monograph, arXiv:2510.21890, 2025. arXiv
30. H. Shen, J. Zhang, B. Xiong et al., “Efficient Diffusion Models: A Survey,” *Transactions on Machine Learning Research*, 2025. arXiv
31. A. Melnik, M. Ljubljanac, C. Lu, Q. Yan, W. Ren and H. Ritter, “Video Diffusion Models: A Survey,” *Transactions on Machine Learning Research*, 2024. arXiv
32. C. Lu and Y. Song, “Simplifying, Stabilizing and Scaling Continuous-Time Consistency Models,” *ICLR*, 2025. arXiv
33. A. Sauer, D. Lorenz, A. Blattmann and R. Rombach, “Adversarial Diffusion Distillation,” *ECCV*, 2024. arXiv
34. M. S. Albergo and E. Vanden-Eijnden, “Building Normalizing Flows with Stochastic Inter-

polants," *ICLR*, 2023. arXiv
35. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," *CVPR*, 2009, pp. 248–255. doi
36. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," *CVPR*, 2016, pp. 2818–2826. arXiv
37. O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *MICCAI*, LNCS 9351, pp. 234–241, 2015. arXiv
38. A. Radford, J. W. Kim, C. Hallacy et al., "Learning Transferable Visual Models From Natural Language Supervision," *ICML*, 2021. arXiv
39. M. Oquab, T. Darcet, T. Moutakanni et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. arXiv
40. B. Efron, "Tweedie's Formula and Selection Bias," *Journal of the American Statistical Association*, vol. 106, no. 496, pp. 1602–1614, 2011. doi
41. A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021. arXiv
42. T. Li and K. He, "Back to Basics: Let Denoising Generative Models Denoise," *CVPR*, 2026 (preprint 2025). paper, publication record, official JiT code.
43. B. Zheng, N. Ma, S. Tong and S. Xie, "Diffusion Transformers with Representation Autoencoders," *ICLR*, 2026 (preprint 2025). proceedings, manuscript.
44. S. Tong, B. Zheng, Z. Wang, B. Tang, N. Ma, E. Brown, J. Yang, R. Fergus, Y. LeCun and S. Xie, "Scaling Text-to-Image Diffusion Transformers with Representation Autoencoders," *CVPR Findings Track*, 2026. manuscript, author publication record.
45. D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna and R. Rombach, "SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis," *ICLR*, 2024. proceedings, official model card.
46. Black Forest Labs, "FLUX.1 [dev]," official pretrained-model card, 2024. model card.
47. W. Kong, Q. Tian, Z. Zhang et al., "HunyuanVideo: A Systematic Framework For Large Video Generative Models," technical report, arXiv:2412.03603, 2024. report, official release.
48. Team Wan, A. Wang, B. Ai, B. Wen et al., "Wan: Open and Advanced Large-Scale Video Generative Models," technical report, arXiv:2503.20314, 2025. report, official Wan2.1 release.

ML 08 • Autoregressive Vision: Pixels, Tokens and Scales

Factor an image distribution into conditionals: PixelCNN, VQ-VAE and VQGAN tokenizers, GPT-style token transformers, next-scale prediction, masked generative models, continuous tokens and how they compare with diffusion.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-08-autoregressive-vision/

What you'll learn

- How the probability chain rule turns "model a whole image" into a sequence of "predict the next piece" problems, and how to measure the result in bits per dimension.
- Why training can run in parallel even though sampling is sequential, and how causal masks and masked convolutions keep the answer hidden from its own predictor.
- How PixelRNN and PixelCNN model raw pixels, and why their sequence length eventually pushed the field toward learned tokens.
- How VQ-VAE and VQGAN tokenizers turn an image into a short grid of integers, including the codebook and commitment losses and the straight-through gradient.
- How GPT-style token transformers (Image GPT, Parti, LlamaGen), next-scale prediction (VAR), masked generative models (MaskGIT, Muse) and continuous-token models (MAR) change the unit of prediction.
- How sampling controls (temperature, top-k, top-p, classifier-free guidance, KV caching) work, and how autoregressive generators compare with diffusion models.

The big picture

An **autoregressive** (**AR**) generator builds a sample piece by piece. At each step it looks at everything it has produced so far, predicts a probability distribution for the next piece, draws from it, and appends the result. The "piece" can be a single pixel value, an integer token from a learned codebook, a vector of continuous features, or an entire map at the next resolution.

> **Explain like I'm five**
>
> Imagine drawing a comic one panel at a time. Each new panel must fit the panels already drawn. When you practice, you copy real comics, so you always know what the earlier panels look like. When you invent a new comic, you have to live with your own earlier panels, even the slightly wobbly ones. An autoregressive image model works the same way: practice on real images, then create new ones one piece at a time.

Why does this matter? Autoregression is the same recipe that powers large language models: next-token prediction with a transformer. Bringing images into that recipe lets one architecture

read and write text and pictures, reuse the engineering of language-model serving, and measure progress with an exact likelihood. The price is that images are not naturally sequences. Someone has to decide **what a token is** and **in which order tokens are produced**. Most of this chapter is about those two decisions.

Xiong et al.'s survey [1] organizes visual AR models into three families by their prediction unit: **pixel-based**, **token-based** and **scale-based**. We follow that map, then add masked (non-left-to-right) and continuous-token models, which blur the boundary with diffusion. The previous chapters on generative models and diffusion and flow matching provide the VAE and diffusion background used here; Prince [9] covers the general probabilistic material at textbook level.

The chain rule needs no independence assumption

Plain version. Any joint distribution can be written as a product of "next element given the earlier ones" distributions. That is a fact of probability, not a modeling assumption.

For an ordered sequence $x = (x_1, \dots, x_N)$,

$$p_\theta(x) = \prod_{i=1}^{N} p_\theta(x_i \mid x_{<i}), \qquad \mathcal{L}_{\mathrm{NLL}}(\theta) = -\sum_{i=1}^{N} \log p_\theta(x_i \mid x_{<i}).$$

Here x_i is the i-th element in the chosen order, $x_{<i} = (x_1, \dots, x_{i-1})$ is everything before it, θ are the network parameters, and $\mathcal{L}_{\mathrm{NLL}}$ is the negative log-likelihood of one training example (averaged over the dataset in practice).

The chain rule is exact for **any** order. Approximation enters only through the model: a finite network, a limited context window, or a restricted form for each conditional. For discrete values, each conditional is usually a softmax over the possible values; binary pixels can use a Bernoulli distribution; continuous features need a density or another conditional generator (we meet one in the MAR section).

An order is a computational choice, not a claim about physics. **Raster order** (left to right, top to bottom) is convenient but arbitrary for a two-dimensional image: the pixel below you is just as related as the pixel to your right. Different orders lead to different conditionals, and a finite model may find some orders easier to learn than others. Figure 1 shows three prediction units used in this chapter.

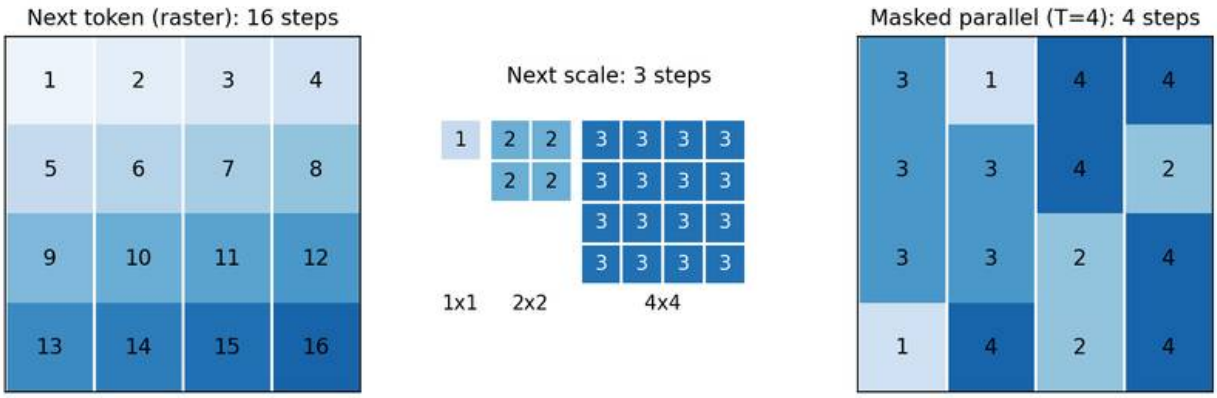


Figure 1 — The same 16 final tokens produced by three generation schemes. Numbers give the sequential step that produces each token: raster autoregression needs 16 steps, next-scale prediction 3 (one per map), and masked parallel decoding with $T = 4$ needs 4. Diagram only; no model is trained.

Measuring an AR model: bits per dimension

Because the chain rule gives an exact $\log p_\theta(x)$, AR models are usually compared by **negative log-likelihood in bits per dimension (bpd)**:

$$\text{bpd}(x) = \frac{-\log_2 p_\theta(x)}{D},$$

where D is the number of discrete values in the image (for example $D = 3 \cdot 32 \cdot 32$ for a 32×32 RGB image) and $\log_2$ converts nats to bits. A model that assigned every 8-bit value a uniform probability would score exactly 8 bpd; anything lower means the model has learned structure. Bpd is measured on held-out images, so it detects overfitting as well as underfitting.

Two cautions. First, bpd and visual quality can disagree: a model can spend most of its likelihood on imperceptible noise while still making odd global mistakes. Second, for a **token** model the likelihood is over codebook indices, not pixels. A lossy tokenizer throws information away, so a token-level likelihood is not an exact pixel-space likelihood and cannot be compared directly with pixel-model bpd.

Teacher forcing, causal masks and exposure bias

Plain version. During training the model always sees the real history; during sampling it sees its own history. A mask makes sure training never lets a position peek at its own answer.

With **teacher forcing**, every conditional in the training loss is computed using the ground-truth

previous elements. Since the whole training image is available, all N conditionals can be evaluated in a single forward pass, as long as position i is prevented from reading positions $\geq i$. A causal transformer enforces this with an attention mask; a PixelCNN enforces it with masked convolution kernels.

Two input conventions are common, and the mask must match the convention:

- In a **shifted-input** decoder, feed `[BOS, x1, x2, ...]` and predict `[x1, x2, x3, ...]`. A lower-triangular mask that *includes* the diagonal is correct, because the input at position i is the *previous* element.
- In a model that receives **unshifted** values, the dependency must be **strictly** earlier. Including the diagonal would let the network copy the value it is supposed to predict.

At inference, earlier sampled mistakes enter the context of later steps. This training/inference mismatch is often called **exposure bias**. It does not make maximum likelihood wrong (the factorization is still exact), but it matters in practice for imperfect models and long sequences: a small early error in the layout of an image can push later steps into contexts that never occurred in training.

Pixel models: PixelRNN and PixelCNN

Plain version. Treat each pixel's intensity as a 256-way classification problem whose input is all pixels above and to the left.

PixelRNN and PixelCNN were introduced together by van den Oord, Kalchbrenner and Kavukcuoglu [2]. Both model each 8-bit value with a **256-way softmax** rather than a Gaussian, so the distribution can be multimodal (a pixel can be "either black or white, but not grey"). Colour is handled by a fixed order inside each pixel, red then green then blue, so that green may depend on red and blue on both. PixelRNN uses two-dimensional LSTM layers (Row LSTM and Diagonal BiLSTM) whose dependency range is potentially unbounded; PixelCNN replaces them with stacked masked convolutions, which have a bounded receptive field but train much faster because all positions are computed in parallel.

Masked convolutions

A masked convolution multiplies its kernel by a binary mask that zeros the weights at "future" positions. In [2], **mask A** is used only in the first layer: it excludes the centre pixel, so the first layer never sees the value it predicts (and, for colour, only sees channels already predicted). **Mask B** is used in all later layers: it allows the centre position, because after the first layer the centre feature already summarizes only earlier pixels. Formally, for a $k \times k$ kernel with centre (c, c), $c = \lfloor k/2 \rfloor$,

$$M_{uv} = \begin{cases} 1 & u < c, \\ 1 & u = c \text{ and } v < c \text{ (A) or } v \le c \text{ (B)}, \\ 0 & \text{otherwise}, \end{cases}$$

where u indexes kernel rows and v kernel columns. The code below builds both masks, stacks five layers, and uses a gradient to find exactly which input pixels can influence the output at the centre of a 9×9 image.

```
import torch
from torch import nn
from torch.nn import functional as F

class MaskedConv2d(nn.Conv2d):
    """PixelCNN-style masked convolution for one channel (raster order)."""
    def __init__(self, cin, cout, k, mask_type):
        super().__init__(cin, cout, k, padding=k//2, bias=False)
        mask = torch.ones(k, k)
        c = k//2
        mask[c, c + (mask_type == "B"):] = 0   # centre (type A) and pixels to its right
        mask[c+1:] = 0                         # all rows below
        self.register_buffer("mask", mask)
    def forward(self, x):
        return F.conv2d(x, self.weight*self.mask, padding=self.padding)

torch.manual_seed(0)
layers = [MaskedConv2d(1, 8, 3, "A"), nn.Tanh()]
for _ in range(4):                             # 5 layers: reach of 5 pixels
    layers += [MaskedConv2d(8, 8, 3, "B"), nn.Tanh()]
net = nn.Sequential(*layers, MaskedConv2d(8, 1, 1, "B"))
print(net[0].mask.int())
x = torch.rand(1, 1, 9, 9, requires_grad=True)
net(x)[0, 0, 4, 4].backward()                  # logit for pixel (row 4, col 4)
dep = (x.grad[0, 0].abs() > 0).int()
print(dep)
later = torch.zeros(9, 9, dtype=torch.bool)
later[4, 4:] = True; later[5:] = True
assert dep[later].sum() == 0                   # never reads itself or later pixels
print("earlier pixels never seen (blind spot):", int(((dep == 0) & ~later).sum()))
```

The assertion passes: no pixel at or after the target influences its logit, so the model is a valid autoregressive factorization. But the printout reveals something else. Six pixels that come *earlier* in raster order, in a staircase above and to the right of the target, have zero gradient. This is the

blind spot of stacked masked convolutions (Figure 2): each layer's receptive field grows up and to the left, but the region up and to the right is never reached. The model is still correct, just less informed than it could be.

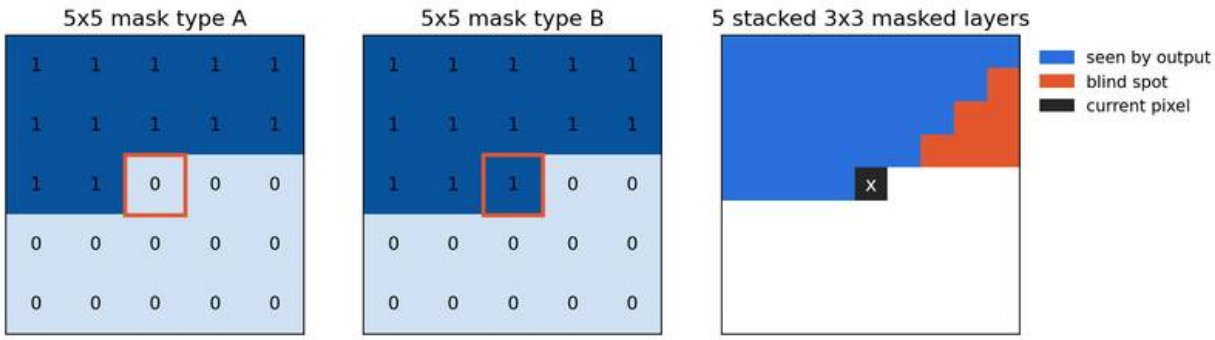


Figure 2 — Left and centre: mask A excludes the centre weight; mask B keeps it. Right: input pixels that influence the prediction at the dark cell after five stacked 3×3 masked layers, measured with a gradient. Orange cells come earlier in raster order but are never seen: the blind spot.

The Gated PixelCNN of van den Oord et al. [3] removed the blind spot by splitting the network into two stacks: a **vertical stack** that sees all rows above the current one without restriction and a **horizontal stack** that sees the current row up to the current pixel, combined after each layer. It also replaced ReLU with a gated unit $y = \tanh(W_f * x) \odot \sigma(W_g * x)$, where $*$ is convolution, σ the logistic sigmoid and $\odot$ elementwise multiplication, and showed how to condition the model on class labels or embeddings. With these changes PixelCNN matched PixelRNN's likelihood on ImageNet [23] at much lower cost [3].

Code: a minimal autoregressive model

Masked convolutions are one way to enforce causality; a transformer is not required either. The following complete CPU experiment generates 4×4 binary images whose rows all repeat the same four random bits. A single masked linear layer, whose strictly lower-triangular mask excludes the current pixel and all later ones, learns the conditionals directly.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
class BinaryAR(nn.Module):
    def __init__(self, n=16):
        super().__init__()
        self.weight = nn.Parameter(torch.zeros(n, n))
        self.bias = nn.Parameter(torch.zeros(n))
        self.register_buffer("mask", torch.tril(torch.ones(n, n), diagonal=-1))
    def forward(self, x):
        return F.linear(x, self.weight*self.mask, self.bias)

model = BinaryAR()
opt = torch.optim.Adam(model.parameters(), lr=.04)
```

```
for step in range(600):
    first_row = torch.randint(2, (128, 4)).float()
    x = first_row.repeat(1, 4)
    loss = F.binary_cross_entropy_with_logits(model(x), x)
    opt.zero_grad(); loss.backward(); opt.step()

with torch.no_grad():
    a = torch.rand(2, 16)
    b = a.clone(); b[:, 8:] = torch.rand(2, 8)
    assert torch.allclose(model(a)[:, :9], model(b)[:, :9])
    samples = torch.zeros(8, 16)
    for i in range(16):
        p = model(samples)[:, i].sigmoid()
        samples[:, i] = torch.bernoulli(p)
    rows = samples.reshape(-1, 4, 4)
    print("one generated image:\n", rows[0].int())
    print("repeat agreement:", (rows[:, 1:] == rows[:, :1]).float().mean().item())
```

The assertion checks causality: changing positions 8 onward cannot alter the logits for positions 0 through 8, because each of those outputs reads only earlier inputs. Training then drives the agreement between later rows and the first row to 1.0, while the first row stays random (the model must not memorize one particular row).

The true distribution has 16 equally likely images, so its entropy is $\log_2 16 = 4$ bits per image, or 0.25 bits per pixel. Later rows contain no new uncertainty. A model that treated all 16 pixels as independent fair coins would need 16 bits per image. The gap between 16 and 4 bits is exactly the structure an autoregressive model captures and an independent-pixel model cannot.

Why pixel models stopped scaling

A 256×256 RGB image has $3 \cdot 256 \cdot 256 = 196{,}608$ values, so pixel-level sampling needs about two hundred thousand sequential network evaluations. Most of the likelihood is also spent on fine, locally predictable detail, while the global layout that people notice is a small fraction of the bits. Image GPT [11] showed that a plain sequence transformer trained to predict pixels autoregressively on low-resolution images learns features that transfer well under linear probing, but the pixel route stays expensive at higher resolutions. The next step was to learn a shorter, more meaningful sequence.

Discrete visual tokens: VQ-VAE and VQGAN

Plain version. Train a separate network that compresses an image into a small grid of "words" from a learned dictionary, then do autoregression over the words instead of the pixels.

VQ-VAE

VQ-VAE [4] has an encoder that produces a feature vector $z_e(x) \in \mathbb{R}^d$ at each position of a downsampled grid, a **codebook** $\{e_1, \dots, e_K\} \subset \mathbb{R}^d$ of K learned vectors, and a decoder. Each feature is replaced by its nearest code:

$$k^* = \arg\min_k \|z_e(x) - e_k\|_2^2, \qquad z_q(x) = e_{k^*}.$$

The image is now represented by the integer grid of k^* values. Training minimizes

$$\mathcal{L}_{\text{VQ}} = \underbrace{\|x - D(z_q)\|^2}_{\text{reconstruction}} + \underbrace{\|\text{sg}[z_e] - e_{k^*}\|^2}_{\text{codebook}} + \beta \underbrace{\|z_e - \text{sg}[e_{k^*}]\|^2}_{\text{commitment}},$$

where D is the decoder, $\text{sg}[\cdot]$ is the **stop-gradient** operator (identity in the forward pass, zero gradient in the backward pass), and β is a small weight on the commitment term (the snippet below uses 0.25). The reconstruction term is written as squared error here; any reconstruction likelihood can take its place.

The three terms do three jobs. The **codebook** term moves the chosen code toward the encoder output, like a k-means update. The **commitment** term pulls the encoder toward its code so that encoder outputs do not drift far from the codebook. The `argmin` has no useful gradient, so the reconstruction gradient reaches the encoder through the **straight-through estimator**: in the forward pass use z_q, in the backward pass copy the gradient from z_q to z_e as if quantization were the identity. In code this is the one-liner `z_q = z_e + (e - z_e).detach()`.

The toy below trains a 2-D "encoder", codebook and decoder on four clusters, with eight codes available:

```
import torch
from torch import nn
from torch.nn import functional as F

def quantize(z_e, codebook, beta=0.25):
    """z_e: (n, d) encoder outputs; codebook: (K, d). Returns z_q with straight-through
     ↪  gradient."""
    dist = torch.cdist(z_e, codebook)            # (n, K) Euclidean distances
    idx = dist.argmin(1)                         # nearest code index per vector
    e = codebook[idx]
    codebook_loss = F.mse_loss(e, z_e.detach())  # moves codes toward encoder outputs
    commit_loss = F.mse_loss(z_e, e.detach())    # keeps the encoder near its code
    z_q = z_e + (e - z_e).detach()               # forward: e; backward: identity
    return z_q, idx, codebook_loss + beta*commit_loss

torch.manual_seed(0)
torch.set_num_threads(1)
centres = torch.tensor([[2., 0.], [-2., 0.], [0., 2.], [0., -2.]])
encoder = nn.Linear(2, 2)                        # toy "encoder" and "decoder"
decoder = nn.Linear(2, 2)
```

```
codebook = nn.Parameter(torch.randn(8, 2)*.1)    # K = 8 codes, 4 are enough
opt = torch.optim.Adam([*encoder.parameters(), *decoder.parameters(), codebook], lr=.03)
for step in range(800):
    x = centres[torch.randint(4, (256,))] + .1*torch.randn(256, 2)
    z_q, idx, vq_loss = quantize(encoder(x), codebook)
    loss = F.mse_loss(decoder(z_q), x) + vq_loss
    opt.zero_grad(); loss.backward(); opt.step()

with torch.no_grad():
    x = centres.repeat_interleave(100, 0) + .1*torch.randn(400, 2)
    z_q, idx, _ = quantize(encoder(x), codebook)
    print("reconstruction MSE:", round(F.mse_loss(decoder(z_q), x).item(), 4))
    print("codes used:", idx.unique().numel(), "of", len(codebook))
    print("codes per cluster:", [idx[i*100:(i+1)*100].unique().tolist() for i in range(4)])
z = torch.randn(3, 2, requires_grad=True)
z_q, _, _ = quantize(z, codebook.detach())
z_q.sum().backward()
print("straight-through gradient:", z.grad[0].tolist())
```

It prints a reconstruction MSE of about 0.0096, uses **4 of the 8 codes** (one per cluster), and shows a straight-through gradient of `[1.0, 1.0]`. Two lessons hide in those numbers. The reconstruction error is almost exactly the within-cluster noise variance $0.1^2 = 0.01$: quantization has kept *which cluster* and discarded *where inside the cluster*. And half the codebook is **dead**: codes that are never selected receive no codebook gradient and never move (Figure 3). In real tokenizers, low codebook usage wastes capacity and is a standard failure to monitor.

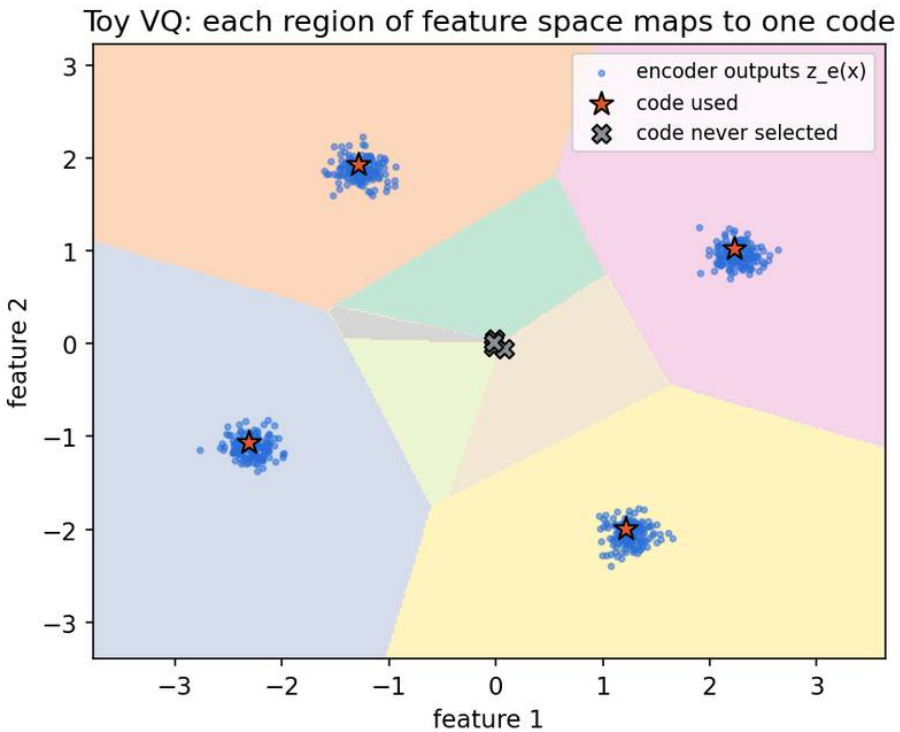


Figure 3 — Toy vector quantization. Each coloured region is the set of features mapped to one code. Four codes (stars) are used, one per cluster; four (crosses) were never selected and stayed near their initialization.

Two stages: a tokenizer, then a prior

A VQ-VAE alone is an autoencoder; it does not by itself say which token grids are likely. Generation therefore has **two stages** (Figure 4). Stage 1 trains the tokenizer. Stage 2 freezes it, converts every training image into its index grid, and trains an autoregressive model $p_\psi(k_1, \ldots, k_M)$ over the M indices. To sample, draw an index grid from the prior and decode it. In [4] the prior was a PixelCNN over codes; VQ-VAE-2 [10] used a hierarchy of two code maps (a coarse top level and a finer bottom level) with PixelCNN-style priors and produced diverse, high-fidelity ImageNet samples.

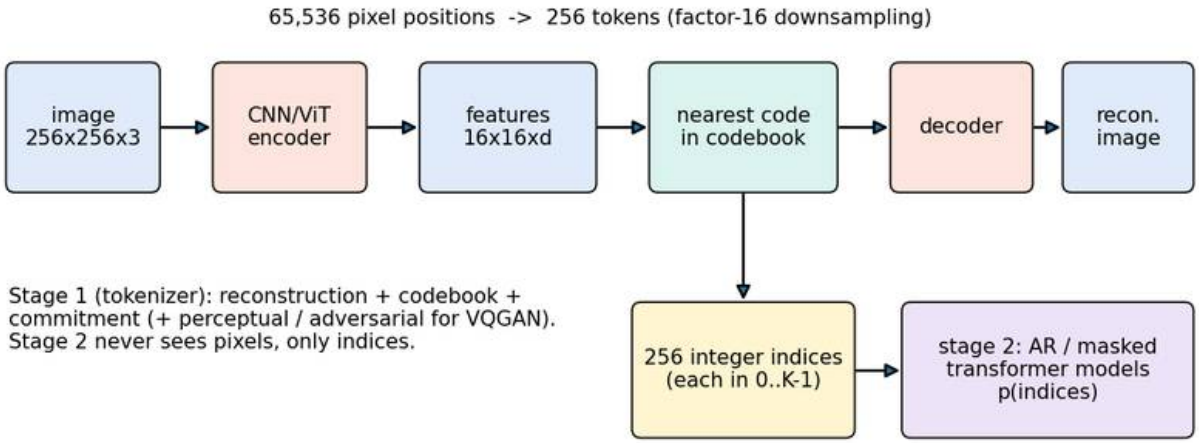


Figure 4 — Two-stage generation. The tokenizer (top row) is trained for reconstruction; the prior (bottom right) only ever sees integer indices. With factor-16 downsampling, a 256×256 image becomes 256 tokens instead of 65,536 pixel positions.

The arithmetic explains the appeal. If a tokenizer downsamples a 256×256 image by 16 in each direction, the prior sees $16 \times 16 = 256$ tokens rather than 65,536 pixel positions (or 196,608 colour values). Self-attention cost grows with the square of sequence length, so this is the difference between feasible and infeasible.

VQGAN: perceptual tokens

Esser, Rombach and Ommer's VQGAN [5] kept the two-stage design but trained the tokenizer with a **perceptual loss** and a **patch-based adversarial loss** in addition to reconstruction, and modelled the token grid with a transformer. The adversarial and perceptual terms let the decoder produce sharp, realistic texture from very few tokens, because the tokens no longer need to encode

every pixel exactly; they need to encode enough for the decoder to produce *a* plausible version. This is why VQGAN-style tokenizers became the default front end for token-based generation.

Note the division of labour: the **tokenizer** uses adversarial training, but the **prior** is an ordinary autoregressive model trained by maximum likelihood.

Codebook problems and alternatives

Real tokenizers face three recurring problems: dead codes (as in the toy), a gap between what the tokenizer reconstructs and what the prior can model, and loss of small details such as text and thin lines. Several designs respond to them:

- **ViT-VQGAN** [12] uses a Vision Transformer encoder/decoder, looks codes up in a low-dimensional **factorized** space and ℓ_2**-normalizes** codes and features, which improved codebook usage and reconstruction.
- **RQ-VAE** [15] uses **residual quantization**: each position is represented by a short stack of codes, each one quantizing what the previous ones missed. This allows a coarser spatial grid (shorter sequence) at similar fidelity.
- **FSQ** [16] (finite scalar quantization) projects to a handful of dimensions and rounds each to a few fixed levels. The implicit codebook is the product of the levels; there is no learned codebook to collapse and no commitment loss.

Compression always creates a bottleneck. Measure codebook usage, reconstruction quality and the generator's quality **separately**: a generator cannot produce detail its tokenizer cannot represent.

Autoregressive transformers over tokens

Plain version. Once an image is a sequence of integers, generating it is the same problem as generating text: a decoder-only transformer predicts the next token.

From pixels to tokens

Image GPT [11] applied a GPT-style transformer directly to sequences of low-resolution pixels.

Its main finding was about **representation learning**: features from a model trained only to predict the next pixel transfer well when a linear classifier is trained on top. Working on tokens instead of pixels shortens sequences enough to generate at higher resolution. The VQGAN transformer [5] is the classic example: a GPT-like model over VQGAN indices in raster order, which let a transformer synthesize high-resolution images that would be far too long as pixel sequences.

Text-to-image as translation: Parti

Parti [13] treats text-to-image generation like machine translation. A transformer **encoder** reads the text prompt; a transformer **decoder** autoregressively generates ViT-VQGAN image tokens [12] conditioned on it. The report scales this sequence-to-sequence model up to 20 billion parameters and introduces PartiPrompts, a benchmark of more than 1,600 English prompts spanning many categories of content.

Language-model architecture, unchanged: LlamaGen

LlamaGen [14] asks whether a vanilla language-model architecture, with no image-specific inductive biases, can generate images well when scaled. It uses a Llama-style [24] decoder (RMSNorm pre-normalization, SwiGLU feed-forward layers, 2-D rotary position embeddings) over the tokens of a factor-16 VQ tokenizer with $K = 16{,}384$ codes of dimension 8. The class label or projected text embedding is supplied as a **prefix** token, and generation proceeds in raster order. The paper releases class-conditional models from 111M to 3.1B parameters and reports that standard LLM serving software (vLLM) speeds up inference. Its broader message is architectural reuse: an image generator can share almost everything with a text model except the tokenizer.

Classifier-free guidance for autoregressive models

Classifier-free guidance (CFG) [18] was introduced for diffusion models, but it transfers directly to AR logits. During training, the condition c is randomly replaced by a learned **null** embedding so that one network learns both conditional and unconditional predictions. At each sampling step, compute both sets of logits and combine them:

$$\ell_g = \ell_u + s\,(\ell_c - \ell_u),$$

where ℓ_c are the next-token logits with the condition, ℓ_u the logits with the null condition, and s the guidance scale; $s = 1$ recovers the ordinary conditional model. This is the form LlamaGen uses [14]. Larger s usually improves prompt or class adherence and sample sharpness at the cost of diversity. It also doubles the per-step compute unless the two passes are batched, so step counts alone understate cost.

Next-scale prediction (VAR)

Plain version. Instead of predicting the next token in a fixed reading order, predict the next, sharper version of the whole image, conditioned on all coarser versions.

Raster order forces a strange dependency: the top-left of an image is generated without any knowledge of the bottom-right. VAR (Visual AutoRegressive modeling) [6] argues that this one-directional order conflicts with the two-directional correlations of image tokens and with spatial

locality, and replaces it with a **coarse-to-fine** order over token maps $r_1, \dots, r_K$ of increasing resolution:

$$p(r_1, \dots, r_K) = \prod_{k=1}^{K} p(r_k \mid r_1, \dots, r_{k-1}),$$

where r_k is an $h_k \times w_k$ grid of token indices, r_1 is 1×1 and r_K has the full latent resolution. All tokens of r_k are predicted **in parallel** given the coarser maps, so the number of sequential steps equals the number of scales, not the number of tokens.

The tokenizer is part of the method. VAR's multi-scale VQ encoder works **residually**: starting from the encoder feature map f, for $k = 1, \dots, K$ it downsamples f to $h_k \times w_k$, quantizes it to get r_k (with one codebook shared across scales), upsamples the looked-up codes back to full resolution, passes them through a small convolution ϕ_k, and subtracts the result from f before moving to the next scale [6]. Each scale therefore encodes what the coarser scales have not explained. Simply resizing ordinary token maps would not reproduce this.

During training, a **block-wise causal** attention mask lets every token of r_k attend to all tokens of $r_{\le k}$ but none of the finer scales; at inference a KV cache stores the coarser scales. VAR uses CFG as well. On class-conditional ImageNet 256×256 the paper reports that VAR improves its AR baseline's FID from 18.65 to 1.73, with faster inference, and that its models follow power-law scaling trends similar to those of language models [6]. VAR received a NeurIPS 2024 Best Paper award. As always, those numbers apply to the paper's data, resolution and training protocol.

Parallel prediction within a scale is itself an assumption: tokens in r_k are sampled independently given $r_{<k}$. The masked-model section shows what that assumption can cost on a toy problem.

Masked generative models (MaskGIT and Muse)

Plain version. Start with a grid of blanks. At each round, guess every blank at once, keep only the guesses you are most confident about, and blank out the rest again.

MaskGIT [8] uses the same VQ tokens as an AR model, but a **bidirectional** transformer that sees every position. Training is a masked-token objective: choose a mask ratio, replace that fraction of tokens with a special `[MASK]` token, and minimize cross-entropy on the masked positions,

$$\mathcal{L}_{\text{mask}} = -\mathbb{E}\Big[\sum_{i \in \mathcal{M}} \log p_\theta(y_i \mid y_{\bar{\mathcal{M}}}) \Big],$$

where y is the token grid, $\mathcal{M}$ is the set of masked positions and $y_{\bar{\mathcal{M}}}$ the unmasked tokens. The mask ratio is drawn as $\gamma(r)$ with $r \sim \mathcal{U}(0, 1)$ and a **mask schedule** γ that decreases from 1 to 0. Generation runs for T iterations. At iteration t the model predicts all masked tokens in parallel and samples them; each sample's probability serves as its **confidence**. The number of tokens that

remain masked after the iteration is

$$n_t = \lceil \gamma(t/T)\, N \rceil,$$

with N the total number of tokens, so the n_t least confident new tokens are re-masked and everything else is fixed. MaskGIT compares several schedules and selects the **cosine** schedule $\gamma(r) = \cos(\pi r/2)$, which fixes few tokens early (when the model knows little) and many late; it generates 256×256 images in 8 iterations with a 16×16 token grid and a 1,024-entry codebook [8]. Muse [17] scales the same idea to text-to-image generation, conditioning the masked-token transformer on text embeddings from a pre-trained large language model.

MaskGIT is **not** ordinary left-to-right chain-rule autoregression: the order is chosen on the fly by confidence, and several tokens are sampled in the same step. Those simultaneous tokens are drawn **independently** given the current context. The toy below shows when that matters. It trains a small bidirectional network on the repeated-row 4×4 images from earlier and decodes with the cosine schedule.

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
N, MASK = 16, 2                                  # 16 binary tokens; state 2 = [MASK]
gamma = lambda r: math.cos(math.pi*r/2)          # cosine schedule: fraction still masked
net = nn.Sequential(nn.Flatten(), nn.Linear(3*N, 128), nn.ReLU(), nn.Linear(128, N))

def data(n):                                     # 4x4 images whose rows repeat
    return torch.randint(2, (n, 4)).repeat(1, 4)

def logits(tokens):                              # sees every position: bidirectional
    return net(F.one_hot(tokens, 3).float())

opt = torch.optim.Adam(net.parameters(), lr=3e-3)
for step in range(1500):
    x = data(256)
    ratio = torch.tensor([gamma(r) for r in torch.rand(256).tolist()])
    n_mask = (ratio*N).ceil().clamp(min=1)
    masked = torch.rand(256, N).argsort(1) < n_mask[:, None]   # random positions
    inp = torch.where(masked, MASK, x)
    loss = F.binary_cross_entropy_with_logits(logits(inp)[masked], x[masked].float())
    opt.zero_grad(); loss.backward(); opt.step()

@torch.no_grad()
def generate(n, T=4):
    tokens = torch.full((n, N), MASK)
    for t in range(1, T+1):
        p = logits(tokens).sigmoid()
        sample = torch.bernoulli(p).long()
```

```
        conf = torch.where(sample == 1, p, 1-p)              # prob. of the drawn value
        still = tokens == MASK
        conf = torch.where(still, conf, torch.inf)           # never re-mask fixed tokens
        proposal = torch.where(still, sample, tokens)
        n_keep_masked = math.floor(gamma(t/T)*N)             # 0 after the last step
        order = conf.argsort(1)                              # least confident first
        remask = torch.zeros_like(still)
        remask.scatter_(1, order[:, :n_keep_masked], True)
        tokens = torch.where(remask, MASK, proposal)
    return tokens

for T in (1, 2, 4, 8, 16):
    rows = generate(1000, T).reshape(-1, 4, 4)
    agree = (rows[:, 1:] == rows[:, :1]).float().mean().item()
    valid = (rows == rows[:, :1]).all(2).all(1).float().mean().item()
    print(f"T={T:2d}  repeat agreement={agree:.3f}  valid images={valid:.3f}")
```

(The code uses `floor` rather than the paper's ceiling so that the count reaches exactly zero at $t = T$ in this tiny example.) The output is:

```
T= 1  repeat agreement=0.511  valid images=0.000
T= 2  repeat agreement=0.812  valid images=0.100
T= 4  repeat agreement=0.908  valid images=0.360
T= 8  repeat agreement=0.998  valid images=0.982
T=16  repeat agreement=1.000  valid images=0.999
```

With $T = 1$ every token is sampled independently from the all-mask context, so rows almost never agree. With more iterations, the first few tokens are fixed early and later ones can condition on them; by $T = 8$ nearly every sample is valid. On natural images the dependence between tokens decoded in the same round is weaker, which is why a handful of iterations works, but the trade-off between **fewer steps** and **more independence assumptions** is real (Figure 5).

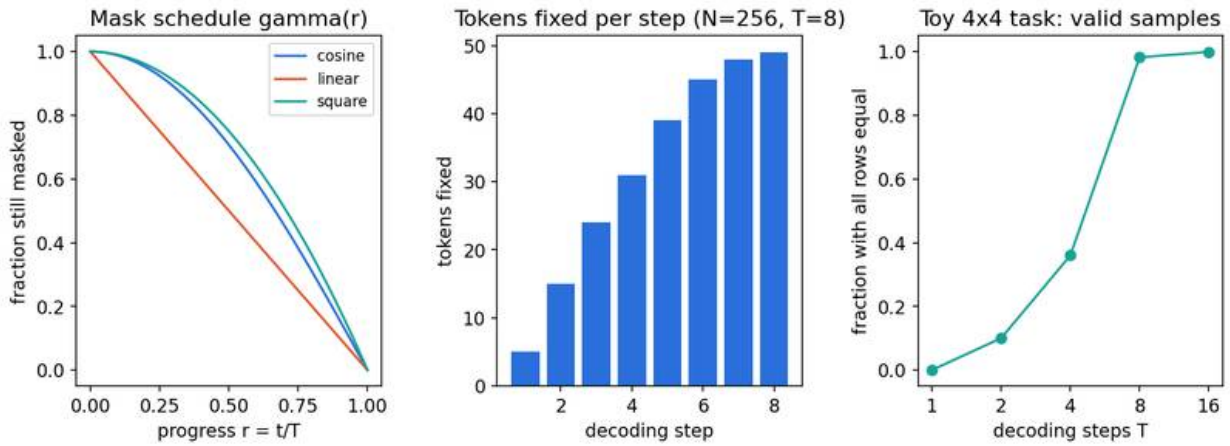


Figure 5 — Left: candidate mask schedules $\gamma(r)$. Centre: with the cosine schedule, 256 tokens and $T = 8$, few tokens are fixed early and many late. Right: on the toy repeated-row task, the fraction of valid samples grows with the number of decoding iterations because fewer tokens are sampled independently in the same step.

Continuous tokens and diffusion conditionals (MAR)

Plain version. Autoregression decides *what comes next*; a tiny diffusion model can decide *how to sample* it, so the tokens no longer need to be integers.

Vector quantization is lossy and awkward to train. Li et al. [7] observe that autoregression needs a way to model $p(x_i \mid x_{<i})$, not necessarily a categorical distribution. They let the AR (or masked AR) transformer output a conditioning vector z_i for each continuous-valued token x_i, and train a small MLP denoiser ϵ_θ with a **diffusion loss**:

$$\mathcal{L}(z_i, x_i) = \mathbb{E}_{\epsilon,t}\|\epsilon - \epsilon_\theta(x_{i,t} \mid t, z_i)\|^2, \qquad x_{i,t} = \sqrt{\bar{\alpha}_t}\, x_i + \sqrt{1-\bar{\alpha}_t}\, \epsilon,$$

where $\epsilon \sim \mathcal{N}(0, I)$, t is a diffusion time step and $\bar{\alpha}_t$ the noise schedule from ML 07. The gradient also flows into the transformer through z_i. To sample a token, run the reverse diffusion process of the small denoiser conditioned on z_i; a temperature scales the injected noise. The paper's main variant, **MAR** (masked autoregressive), uses a random order and predicts a set of tokens per step, combining the outer structure of MaskGIT with an inner continuous sampler [7].

Consequently, "AR versus diffusion" is not an either/or choice. A system can decide the order and grouping of tokens autoregressively and sample each group by diffusion. Likewise, a language decoder can autoregressively generate *text about* an image without generating any pixels; the unifying idea is factorization, not a particular output type.

Sampling, temperature and caching

Plain version. The trained model gives a distribution at every step; the sampler decides how adventurous to be when drawing from it, and the cache avoids recomputing the past.

For logits ℓ_k over K possible values, **temperature** T rescales the distribution:

$$p_T(k) = \frac{\exp(\ell_k / T)}{\sum_{j=1}^{K} \exp(\ell_j / T)}.$$

Lower T concentrates probability on the top values; larger T spreads it (Figure 6). **Top-k** sampling keeps only the k highest logits; **top-p** (nucleus) sampling keeps the smallest set of values whose probability mass reaches p. Both renormalize what remains.

```
import torch

def filtered_probs(logits, temperature=1.0, top_k=None, top_p=None):
    """Return the (renormalized) distribution actually sampled from."""
    logits = logits / temperature
    if top_k is not None:                                  # keep the k largest logits
        kth = logits.topk(top_k).values[..., -1, None]
        logits = logits.masked_fill(logits < kth, -torch.inf)
    if top_p is not None:                                  # smallest set with mass >= p
        sorted_logits, order = logits.sort(descending=True)
        cum = sorted_logits.softmax(-1).cumsum(-1)
        drop = cum - sorted_logits.softmax(-1) >= top_p  # mass before this token already >=
         ↪  p
        sorted_logits = sorted_logits.masked_fill(drop, -torch.inf)
        logits = torch.empty_like(logits).scatter(-1, order, sorted_logits)
    return logits.softmax(-1)

logits = torch.tensor([2.0, 1.0, 0.0, -1.0])
for name, kw in [("T=1", {}), ("T=0.5", {"temperature": .5}), ("T=2", {"temperature": 2.}),
                 ("top-k=2", {"top_k": 2}), ("top-p=0.8", {"top_p": .8})]:
    p = filtered_probs(logits, **kw)
    entropy = -(p[p > 0]*p[p > 0].log2()).sum().item()
    print(f"{name:10s}", [round(v, 3) for v in p.tolist()], f"entropy={entropy:.2f} bits")
```

The model's own distribution ($T = 1$) has 1.37 bits of entropy; $T = 0.5$ lowers it to 0.66 bits and $T = 2$ raises it to 1.80 bits. Top-$k = 2$ and top-$p = 0.8$ both keep the two most likely values here (the first value has mass 0.644, which is below 0.8, so the second is needed). Every one of these changes the distribution you sample from, so a sampler's visual quality should not be confused with the model's likelihood.

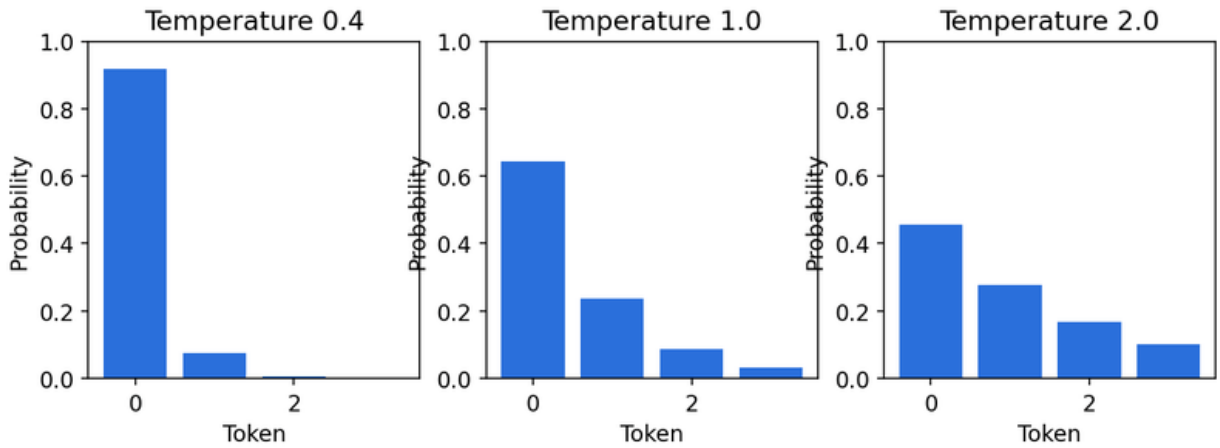


Figure 6 — Temperature changes diversity at each conditional. It cannot recover information that a tokenizer has already lost.

The KV cache

In a causal transformer, the keys and values of earlier tokens do not change when a new token is appended. A **KV cache** stores them, so each new step computes the query, key and value of the newest token only and attends to the cache:

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
d, n = 8, 6
Wq, Wk, Wv = (torch.randn(d, d) for _ in range(3))
x = torch.randn(n, d)                                  # embeddings of n tokens

# Full causal attention over the whole sequence (what training computes).
q, k, v = x@Wq, x@Wk, x@Wv
mask = torch.ones(n, n).tril().bool()
scores = (q@k.T/d**.5).masked_fill(~mask, -torch.inf)
full = scores.softmax(-1)@v

# Incremental decoding: each step computes q, k, v for the newest token only.
K_cache, V_cache, outputs = [], [], []
for i in range(n):
    K_cache.append(x[i]@Wk); V_cache.append(x[i]@Wv)
    K, V = torch.stack(K_cache), torch.stack(V_cache)
    att = F.softmax((x[i]@Wq)@K.T/d**.5, -1)        # attends to tokens 0..i
    outputs.append(att@V)
assert torch.allclose(full, torch.stack(outputs), atol=1e-5)
print("cached decoding matches full causal attention")
print("dot products per step:", [i+1 for i in range(n)], "total:", n*(n+1)//2)
```

The assertion confirms that cached decoding gives exactly the training-time causal outputs (for a single layer here; real models cache every layer and head). The cache removes recomputation but not the sequential dependency, and it grows linearly with sequence length. Step i still attends to i cached keys, so total attention work over a sequence of length n is $n(n+1)/2$, quadratic in n. This is why shorter sequences (better tokenizers) and fewer sequential steps (next-scale or masked decoding) matter so much.

For video, tokens can be ordered over space and time or grouped into chunks. Caching, compression and temporal ordering then dominate cost, and object consistency and long-duration drift need their own evaluation.

Autoregressive versus diffusion

Plain version. Both families turn one hard generation problem into many easier ones. AR splits the *image* into pieces; diffusion splits the *noise level* into steps.

Aspect	Autoregressive (token / scale)	Masked generative	Diffusion / flow (ML 07)
What is generated per step	One token (or one scale)	A confidence-chosen set of tokens	A refinement of the whole sample
Sequential steps	Tokens or scales	Fixed, typically few (T)	Solver steps (NFE)
Typical representation	Discrete codes (or continuous with a diffusion head [7])	Discrete codes	Continuous pixels or latents
Training objective	Exact NLL (cross-entropy)	Masked cross-entropy	Denoising / velocity regression (ELBO-related)
Likelihood	Exact over tokens	Not a single exact chain-rule likelihood	Bound or ODE-based estimate
Natural strengths	Shares architecture and serving with LLMs; mixes with text	Few steps; easy inpainting and editing	Strong image quality; flexible guidance
Typical weaknesses	Long sequential decoding; tokenizer bottleneck; error accumulation	Independence within a step	Many network evaluations unless distilled

Three points keep this table honest. First, the families mix: MAR [7] puts a diffusion model inside an AR loop, and latent diffusion also depends on a learned tokenizer-like autoencoder. Second, claims that one family "beats" the other (as in LlamaGen's title [14] or VAR's comparison with DiT [6]) hold for a stated benchmark, tokenizer, model size and guidance setting, not universally. Third, cost should be compared in network evaluations, guidance passes, sequence length and actual latency, not in "steps" alone.

Modern view

The field has moved from raw pixels to learned representations, from fixed raster order to flexible prediction units, and from separate image generators to models that read and write both text and images. Several surveys map this territory:

- **Xiong et al. [1]** (TMLR) is the most direct survey of visual autoregression. Its taxonomy by representation (pixel-, token- and scale-based models) is the one used in this chapter, and it covers applications in image, video, 3D and multimodal generation as well as embodied and medical settings.
- **Bond-Taylor et al. [19]** (TPAMI 2022) compare VAEs, GANs, normalizing flows, energy-based and autoregressive models side by side, including the trade-offs in run time, sample diversity and architectural constraints. It places AR models in the wider generative-model landscape: exact likelihood and stable training, paid for with slow sequential sampling.
- **Li et al. [20]** (TPAMI 2026) survey discrete tokenization for multimodal LLMs, organizing vector-quantization variants and their use in classical, single-modality LLM and multimodal LLM systems, and discussing codebook collapse and unstable gradient estimation.

- **Jia et al. [22]** survey discrete tokenizers across generation, comprehension, recommendation and retrieval, dissecting tokenizers into their sub-modules and mechanisms before organizing methods by application, and listing limitations and future directions.
- **Zhao et al. [21]** survey unified multimodal understanding-and-generation models, grouping them into diffusion-based, autoregressive-based and hybrid AR–diffusion designs, which is where most 2025–2026 activity sits.

State of the art (2023–2026). Three lines stand out in the verified literature. (i) *Scaling plain AR transformers over better tokenizers*: LlamaGen [14] shows that a language-model architecture with almost no image-specific design is competitive on class-conditional ImageNet when the tokenizer and training are good. (ii) *Changing the prediction unit*: VAR's next-scale prediction [6] and masked decoding in MaskGIT and Muse [8], [17] reduce the number of sequential steps; MAR [7] removes vector quantization by using a diffusion head per token. (iii) *Unified multimodal models*: the same next-token machinery is used to generate text and image tokens in one model, with tokenizers designed to serve both understanding and generation [20], [21]. Our interpretation is that choosing the **representation and prediction unit** now matters at least as much as choosing the backbone.

Open problems. The surveys repeatedly point to the following:

- *Tokenizer quality.* Reconstruction caps generation quality; small text, faces and fine geometry are the usual casualties. Better codebook usage and semantic structure without losing detail remain active topics [20], [22].
- *Discrete versus continuous tokens.* Discrete tokens fit LLM machinery; continuous tokens reconstruct better but need a continuous conditional model such as a diffusion head [7].
- *Speed.* Sequential decoding is latency-bound. Next-scale, masked and parallel decoding trade steps for independence assumptions.
- *Evaluation.* Token likelihood, FID-type metrics, prompt adherence and human preference measure different things; a lossy tokenizer's token likelihood is not a pixel likelihood.
- *Video and long contexts.* Token counts explode over time, and temporal consistency, object permanence and drift need dedicated evaluation [1].

Key takeaways

- Autoregression is a factorization, not a synonym for "transformer": the chain rule is exact for any order, and the model's capacity and context decide how good each conditional is.
- Training is parallel under teacher forcing; sampling is sequential. Shifted inputs and causal masks (attention masks or PixelCNN mask A/B) must be designed together.
- Pixel models are exact but long; learned tokenizers (VQ-VAE, VQGAN, RQ-VAE, FSQ) shorten the sequence at the price of a reconstruction bottleneck that the generator cannot exceed.
- Token transformers (Image GPT, VQGAN transformer, Parti, LlamaGen) reuse language-model machinery, including classifier-free guidance on logits and KV caching.

- Next-scale (VAR), masked (MaskGIT, Muse) and continuous-token (MAR) models change the **unit** of prediction; fewer sequential steps usually mean more conditional-independence assumptions.
- Temperature, top-k and top-p reshape the sampling distribution; they trade diversity for fidelity but cannot add information lost by the tokenizer.
- AR and diffusion are complementary decompositions and are increasingly combined; compare them on matched data, tokenizers, guidance and compute.

Exercises

1. **Leak the answer.** Change the `BinaryAR` mask to include the diagonal (`diagonal=0`). Explain why the training loss can fall almost to zero while sampling breaks.

   Hint

   Each output can now read its own target and simply copy it, so the training loss collapses. At sampling time the current position is still 0 (not yet drawn) when its logit is computed, so the learned "copy" rule produces nonsense. The causality assertion in the snippet would also fail.

2. **Bits per dimension.** For the repeated-row 4×4 binary data, compute the bits per pixel of (a) the true distribution, (b) a model that treats every pixel as an independent fair coin, and (c) a model that learns the first row perfectly but treats later rows as independent fair coins.

   Hint

   (a) 4 bits / 16 pixels = 0.25. (b) 16 / 16 = 1.0. (c) 4 + 12 = 16 bits per image, also 1.0 bits per pixel: learning the first row's marginal does not help, because its bits are already uniform. All gains come from modelling the dependence of later rows on the first.

3. **Tokenizer budget.** A 512×512 image is tokenized with factor-8 or factor-16 downsampling. Compute the sequence length in each case and the ratio of full self-attention costs. What might you lose with factor 16?

   Hint

   $64 \times 64 = 4096$ versus $32 \times 32 = 1024$ tokens. Attention cost scales with length squared, so the ratio is 16. With factor 16 each token must describe a 16×16 patch, so small text and thin structures are more likely to be lost unless the codebook or decoder compensates.

4. **Blind spot.** Modify the masked-convolution snippet to use 7 layers and an 11×11 input with the target at the centre. How many earlier pixels are never seen? Then sketch how a vertical stack plus a horizontal stack removes them.

Hint

The blind spot is the staircase above and to the right of the target that the up-left-growing receptive field never reaches; counting zero-gradient cells that are earlier in raster order gives 10. A vertical stack sees full rows above (no left/right restriction) and is fed into the horizontal stack, which handles the current row only, so every earlier pixel becomes reachable.

5. **Masked decoding steps.** Using $n_t = \lceil \gamma(t/T)N \rceil$ with the cosine schedule, $N = 256$ and $T = 8$, compute how many tokens are fixed at each step. Why does the schedule fix few tokens first?

Hint

Remaining masked counts are 252, 237, 213, 182, 143, 98, 50 and 0, so 4, 15, 24, 31, 39, 45, 48 and 50 tokens are fixed per step (the figure uses floor, which shifts counts by at most one). In exact arithmetic $\gamma(1) = 0$; in floating point $\cos(\pi/2)$ is a tiny positive number, so a literal ceiling leaves one token masked, another reason the snippet uses floor. Early on the model has little context, so its parallel guesses are the least reliable and the most likely to conflict; fixing only a few lets later steps condition on them.

6. **Classify the methods.** For PixelCNN, VQGAN transformer, Parti, LlamaGen, VAR, MaskGIT and MAR, give the prediction unit, the representation (pixels, discrete tokens, continuous tokens, multiscale maps), the conditional distribution type, and the number of sequential steps for a 16×16 token grid.

Hint

PixelCNN: pixels in raster order, 256-way softmax, one step per value. VQGAN transformer, Parti, LlamaGen: discrete tokens in raster order, softmax over the codebook, 256 steps. VAR: multiscale discrete maps, softmax per token with parallel tokens inside a scale, one step per scale. MaskGIT: discrete tokens, bidirectional masked prediction, T steps (8 in the paper). MAR: continuous tokens in random-order groups, a per-token diffusion head, as many steps as groups (each with its own inner diffusion steps).

References

1. J. Xiong, G. Liu, L. Huang et al., "Autoregressive Models in Vision: A Survey," *Transactions on Machine Learning Research* (*TMLR*), 2025 (arXiv:2411.05902, 2024). arXiv
2. A. van den Oord, N. Kalchbrenner and K. Kavukcuoglu, "Pixel Recurrent Neural Networks," in *Proc. ICML*, PMLR 48, 2016. PMLR
3. A. van den Oord, N. Kalchbrenner, O. Vinyals, L. Espeholt, A. Graves and K. Kavukcuoglu, "Conditional Image Generation with PixelCNN Decoders," in *NeurIPS*, 2016. NeurIPS
4. A. van den Oord, O. Vinyals and K. Kavukcuoglu, "Neural Discrete Representation Learning," in *NeurIPS*, 2017. NeurIPS

5. P. Esser, R. Rombach and B. Ommer, “Taming Transformers for High-Resolution Image Synthesis,” in *Proc. IEEE/CVF CVPR*, 2021. CVF
6. K. Tian, Y. Jiang, Z. Yuan, B. Peng and L. Wang, “Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction,” in *NeurIPS*, 2024 (Best Paper Award). arXiv
7. T. Li, Y. Tian, H. Li, M. Deng and K. He, “Autoregressive Image Generation without Vector Quantization,” in *NeurIPS*, 2024. arXiv
8. H. Chang, H. Zhang, L. Jiang, C. Liu and W. T. Freeman, “MaskGIT: Masked Generative Image Transformer,” in *Proc. IEEE/CVF CVPR*, 2022. CVF
9. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book
10. A. Razavi, A. van den Oord and O. Vinyals, “Generating Diverse High-Fidelity Images with VQ-VAE-2,” in *NeurIPS*, 2019. NeurIPS
11. M. Chen, A. Radford, R. Child, J. Wu, H. Jun, D. Luan and I. Sutskever, “Generative Pretraining From Pixels,” in *Proc. ICML*, PMLR 119, 2020. PMLR
12. J. Yu, X. Li, J. Y. Koh et al., “Vector-quantized Image Modeling with Improved VQGAN,” in *ICLR*, 2022. arXiv
13. J. Yu, Y. Xu, J. Y. Koh et al., “Scaling Autoregressive Models for Content-Rich Text-to-Image Generation,” *Transactions on Machine Learning Research*, 2022. arXiv
14. P. Sun, Y. Jiang, S. Chen, S. Zhang, B. Peng, P. Luo and Z. Yuan, “Autoregressive Model Beats Diffusion: Llama for Scalable Image Generation,” arXiv:2406.06525, 2024. arXiv
15. D. Lee, C. Kim, S. Kim, M. Cho and W.-S. Han, “Autoregressive Image Generation using Residual Quantization,” in *Proc. IEEE/CVF CVPR*, 2022. CVF
16. F. Mentzer, D. Minnen, E. Agustsson and M. Tschannen, “Finite Scalar Quantization: VQ-VAE Made Simple,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2024. arXiv
17. H. Chang, H. Zhang, J. Barber et al., “Muse: Text-To-Image Generation via Masked Generative Transformers,” in *Proc. ICML*, PMLR 202, 2023. PMLR
18. J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” arXiv:2207.12598, 2022 (short version at the NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications). arXiv
19. S. Bond-Taylor, A. Leach, Y. Long and C. G. Willcocks, “Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models,” *IEEE TPAMI*, vol. 44, no. 11, 2022. doi
20. J. Li, Y. Fu, J. Liu, L. Cao, W. Ji, M. Yang, I. King and M.-H. Yang, “Discrete Tokenization for Multimodal LLMs: A Comprehensive Survey,” *IEEE TPAMI*, 2026 (arXiv:2507.22920). doi
21. S. Zhao, X. Zhang, J. Guo et al., “Unified Multimodal Understanding and Generation Models: Advances, Challenges, and Opportunities,” arXiv:2505.02567, 2025; revised 2026. arXiv
22. J. Jia, J. Gao, B. Xue et al., “From Principles to Applications: A Comprehensive Survey of Discrete Tokenizers in Generation, Comprehension, Recommendation, and Information Retrieval,” arXiv:2502.12448, 2025. arXiv
23. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” *CVPR*, 2009, pp. 248–255. doi
24. H. Touvron, T. Lavril, G. Izacard et al., “LLaMA: Open and Efficient Foundation Language Models,” arXiv:2302.13971, 2023. arXiv

ML 09 • Foundation Models and Vision–Language Learning

What makes a model a foundation model: scaling laws, pre-training objectives, CLIP and SigLIP, DINOv2 features, promptable segmentation, multimodal language models, adaptation with linear probes and LoRA, and how to evaluate the risks.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-09-foundation-models/

What you'll learn

- What makes a model a foundation model, why the term describes training and reuse rather than an architecture, and why it need not generate text.
- What scaling laws say (a power law with a floor, compute-optimal allocation) and what they do not say.
- The main pre-training objectives — supervised, contrastive image–text, masked image modeling, self-distillation and generative — and the kind of features each tends to produce.
- How CLIP, SigLIP, DINOv2/DINOv3 and the Segment Anything family are used as reusable vision components.
- How multimodal large language models connect a vision encoder to a language model, and how LLaVA-style instruction tuning works.
- How to adapt a pretrained model — linear probe, fine-tuning, LP-FT, adapters, prompt tuning, LoRA, prompting — and how to evaluate the result honestly, including bias, hallucination and test-set contamination.

The big picture

For most of the history of computer vision, each task had its own model: one network for classifying flowers, another for detecting cars, another for segmenting tumors, each trained from scratch or from an ImageNet [43] classifier. A **foundation model** is trained once, on broad data and at large scale, and then **adapted** to many downstream tasks [1]. The word describes how the model is trained and reused, not a new layer type. A DINOv2 [15] or DINOv3 [16] image encoder, a CLIP-style image–text model [3], a promptable segmenter such as SAM [5], a multimodal language model and a broadly pretrained image generator can all play this role.

> **Explain like I'm five**
>
> Instead of teaching a new student from the beginning for every job, start with someone who has already looked at millions of pictures and read millions of captions. They already know what edges,

wheels, faces and dogs look like. For a new job you only need to give them a few examples, a short instruction, or a small new tool — not a whole new education.

Why does this matter? Three reasons. First, **labels are expensive** and pretraining moves most of the learning to data that needs no hand labels (captions, the images themselves). Second, **one backbone serves many tasks**, which concentrates engineering effort. Third, it changes the **interface**: instead of retraining, you can often specify a task with a text prompt, a click or a few examples. Bommasani et al. call the resulting concentration of methods *homogenization*, and warn that it also concentrates failure: a flaw in one shared backbone is inherited by every system built on it [1].

This chapter builds on ML 05 ·Representation Learning (what makes a feature reusable), ML 04 ·Vision Transformers (the usual backbone) and the generative chapters ML 07 and ML 08. The textbook treatment of transfer learning and representation in Bishop and Bishop [14] is a good companion. We follow the architectural map of the vision foundation-model survey by Awais et al. [39] and the multimodal survey by Caffagni et al. [2], with representative reports verified for this October 2026 edition. Examples illustrate families; they do not rank commercial products or claim that one checkpoint is best everywhere.

What makes a model a "foundation" model

Plain version. A foundation model is a large model pretrained on broad data with a generic objective, so that its internal representations or its interface can be reused for tasks it was never specifically trained on.

Bommasani et al. [1] coined the term in 2021 and emphasized two properties:

- **Emergence.** Capabilities arise from scale and the training objective rather than being designed explicitly. For example, CLIP was trained only to match images with captions, yet it can classify images into categories named in text [3]; DINO was trained without any labels, yet its attention maps outline objects [24].
- **Homogenization.** Many applications converge on a small number of pretrained backbones and recipes.

Be careful with "emergence". In this chapter it means only *a capability that was not a training target*. It does not mean an unpredictable or mysterious jump; whether a capability appears suddenly often depends on the metric used to measure it.

Formally, let f_θ be a model pretrained on an upstream dataset $\mathcal{D}_{\text{up}}$ with objective $\mathcal{L}_{\text{pre}}$:

$$\theta^\star = \arg\min_\theta \; \mathbb{E}_{x \sim \mathcal{D}_{\text{up}}}[\mathcal{L}_{\text{pre}}(f_\theta; x)].$$

A downstream task k supplies a (usually small) dataset $\mathcal{D}_k$ and a task loss ℓ_k. Adaptation chooses a small set of new or changed parameters ϕ_k and solves

$$\phi_k^\star = \arg\min_{\phi_k} \sum_{(x,y)\in\mathcal{D}_k} \ell_k(h_{\phi_k}(f_{\theta^\star}, x), y),$$

where h_{ϕ_k} is whatever we put on top of or inside the pretrained model: a linear head, an adapter, a LoRA update, a prompt, or nothing at all (pure prompting, $\phi_k = \emptyset$). The whole chapter is about the two lines above: what $\mathcal{L}_{\text{pre}}$ and $\mathcal{D}_{\text{up}}$ should be, and how small ϕ_k can be.

A foundation model does **not** have to produce text. A frozen image encoder whose features feed a hundred different classifiers is a foundation model. Conversely, a large model trained for one narrow task is not one, however big it is.

Scaling laws

Plain version. When you train bigger models on more data with more compute, the test loss usually falls smoothly and predictably — roughly as a straight line on a log–log plot — until it approaches a floor. That predictability is what lets labs plan large training runs from small ones.

Kaplan et al. [17] trained families of Transformer language models and reported that the loss scales as a power law in model size, dataset size and compute, with some trends spanning more than seven orders of magnitude, while width and depth mattered little within the tested range. A power law in model size N reads

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N},$$

where L is held-out cross-entropy loss, N the number of non-embedding parameters, N_c a fitted constant with the same units as N, and $\alpha_N > 0$ the fitted exponent. Taking logs gives $\log L = \alpha_N \log N_c - \alpha_N \log N$: a straight line with slope $-\alpha_N$.

Real curves bend because some loss can never be removed (the data itself is noisy or ambiguous). Hoffmann et al. [18] therefore fitted a form with an irreducible term:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta},$$

where D is the number of training tokens, E the irreducible loss, and A, B, α, β fitted constants. For dense Transformers a common approximation of training compute is $C \approx 6ND$ floating-point operations [17] (roughly two for the forward and four for the backward pass, per parameter per token). Fixing C and minimizing L over N (with $D = C/6N$) gives a **compute-optimal** model size. Hoffmann et al. trained over 400 language models and concluded that, for compute-optimal training, model size and training tokens should be scaled equally — every doubling of model size should come with a doubling of tokens [18]. That differed from Kaplan et al.'s earlier recommendation of very large models trained on relatively modest data [17]; the disagreement is a useful

reminder that fitted exponents depend on the experimental protocol (learning-rate schedules, which runs are included, what is counted as compute).

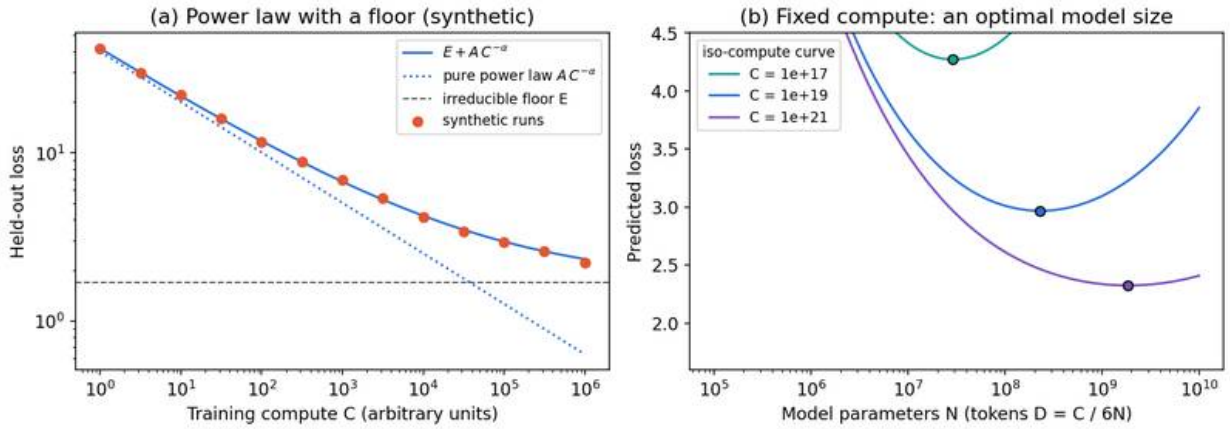


Figure 1 — (a) A power law with an irreducible floor E: a pure power law (dotted) keeps falling and over-promises at large scale. (b) With compute fixed, too small a model under-fits and too large a model sees too few tokens, so each budget has an optimal size. All constants are synthetic.

Vision has its own scaling studies. Zhai et al. [19] characterized the relationship between error rate, data and compute for Vision Transformers [21] trained on large labeled datasets, scaling to two-billion-parameter models. Cherti et al. [20] trained CLIP-style models on the public LAION dataset [34] and found power-law scaling for zero-shot classification, retrieval, linear probing and fine-tuning — but also that OpenAI's CLIP and OpenCLIP, with the same architectures and similar recipes, scale differently because their **training distributions** differ. Data is not a scalar.

Code: fit a scaling law and extrapolate

The snippet fits $L(C) = E + A\,C^{-\alpha}$ to the cheaper of 13 synthetic training runs and then predicts the expensive ones. Everything is synthetic; the point is the procedure.

```
import numpy as np
from scipy.optimize import curve_fit

rng = np.random.default_rng(0)
# Synthetic "runs": loss = E + A * C^(-alpha), with 2% multiplicative noise.
E_true, A_true, alpha_true = 1.7, 40.0, 0.3
compute = np.logspace(0, 6, 13)            # arbitrary compute units
loss = (E_true + A_true * compute**-alpha_true) * np.exp(.02 * rng.normal(size=13))

def law(c, E, A, alpha):
    return E + A * c**-alpha

# Fit only the cheaper runs, then extrapolate to the larger ones.
small = compute <= 1e4
```

```
(E, A, alpha), _ = curve_fit(law, compute[small], loss[small], p0=(1, 10, .5))
print(f"fit: E={E:.2f}  A={A:.1f}  alpha={alpha:.3f}")
for c, l in zip(compute[~small], loss[~small]):
    print(f"C={c:9.0f}  observed {l:.3f}  predicted {law(c, E, A, alpha):.3f}")

# Naive straight line in log-log space ignores the floor E.
slope = np.polyfit(np.log(compute[small]), np.log(loss[small]), 1)[0]
print(f"pure power-law slope without floor: {slope:.3f}")
```

Output: the fit recovers $E \approx 1.73$, $A \approx 40.0$, $\alpha \approx 0.300$; the extrapolated losses are within about 7% of the noisy observations; and a straight log–log line without the floor gives a slope of about -0.25, a wrong exponent. Two lessons transfer to real practice: always include the floor, and treat extrapolation over many orders of magnitude as a forecast with error bars.

What scaling laws do not tell you

- They predict **average loss** on the pretraining distribution, not accuracy on your task. Downstream metrics may improve smoothly, saturate, or move in steps.
- They hold **for a fixed recipe and data distribution** [20]. Better data filtering or a different objective changes the constants.
- They say nothing about **inference cost**. A smaller model trained on more data can be cheaper to deploy at the same loss, which is why many deployed models are trained beyond the compute-optimal point.
- Data is finite. If the high-quality data runs out, the B/D^{β} term stops shrinking however much compute you add.

Pre-training objectives: where does the target come from?

Plain version. Every pretraining method invents a prediction task. The difference is where the "answer" comes from: a human label, a caption, a hidden part of the same image, another network, or the next token.

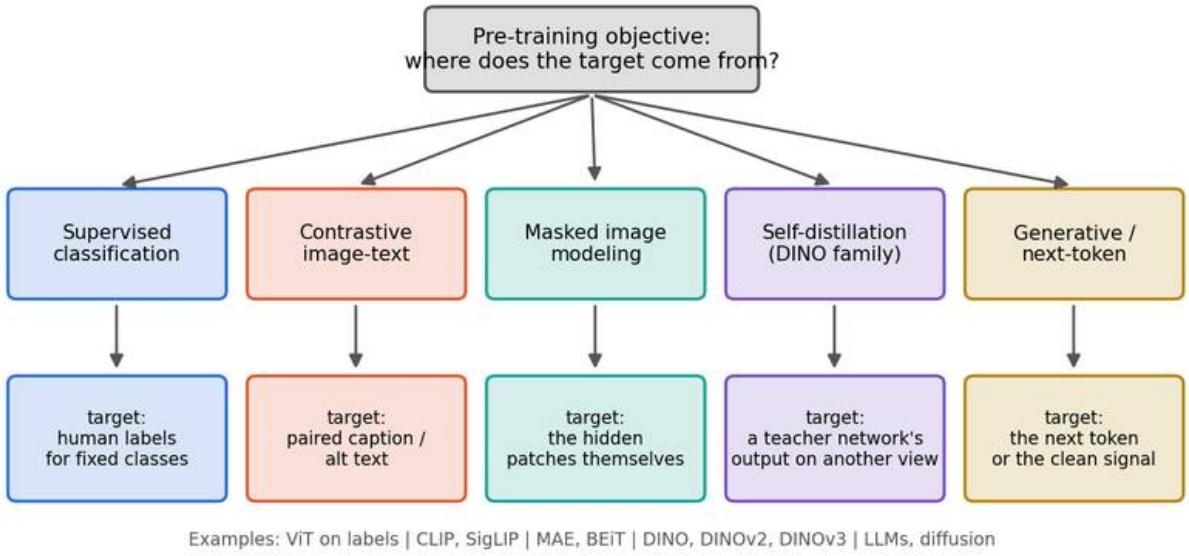


Figure 2 — Five families of pre-training objectives, organized by the source of the training target.

Objective	Target comes from	Representative models	Typical strength
Supervised classification	Human labels	ViT on large labeled sets [19], [21]	Strong global semantics for the label space
Contrastive image–text	Paired captions / alt text	CLIP [3], ALIGN [25], SigLIP [4]	Text-defined recognition, retrieval
Masked image modeling	Hidden patches of the same image	BEiT [23], MAE [22]	Strong after fine-tuning; local detail
Self-distillation	A teacher network's output on another view	DINO [24], DINOv2 [15], DINOv3 [16]	Frozen features, dense correspondence
Generative / next-token	Next token or clean signal	LLMs, diffusion (ML 07), AR (ML 08)	Generation; language interface

Supervised pre-training

Train a classifier with cross-entropy on a large labeled dataset, then drop the final layer. This was the ImageNet recipe for a decade and still works at scale [19], [21]. Its weakness is that the features are shaped by a fixed label set: distinctions that the labels do not require may be discarded.

Contrastive image–text learning

Captions come for free with web images, so CLIP [3] and ALIGN [25] train an image encoder and a text encoder together so that matching pairs score higher than mismatched ones within a batch. ALIGN showed that a large, noisy alt-text corpus with minimal filtering can still give strong representations [25]. The loss is detailed in the next section.

Masked image modeling

Hide most of an image and predict what is missing. BEiT [23] masks patches and predicts discrete visual tokens for them, borrowing the masked-language-modeling idea from BERT [44]. MAE [22] predicts the raw pixels directly and observes that masking a high proportion, e.g. 75% of patches, creates a nontrivial task; its encoder sees only the visible patches and a lightweight decoder reconstructs the rest, which makes training efficient. With $\mathcal{M}$ the set of masked patch indices, x_p the (normalized) pixels of patch p and $\hat{x}_p$ the reconstruction,

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{p \in \mathcal{M}} \|\hat{x}_p - x_p\|_2^2.$$

The loss is computed only on masked patches: predicting the visible ones would be trivial copying. Masked-modeling features are often excellent after fine-tuning but weaker as frozen features for linear probing than contrastive or self-distilled ones.

Self-distillation: the DINO family

DINO [24] — described by its authors as self-distillation with no labels — trains a **student** network g_s to match a **teacher** network g_t on different augmented views (crops) of the same image. Both output K-dimensional scores; probabilities are softmaxes with temperatures τ_s and τ_t:

$$P_s(x') = \text{softmax}\left(\frac{g_s(x')}{\tau_s}\right), \qquad P_t(x) = \text{softmax}\left(\frac{g_t(x) - c}{\tau_t}\right), \qquad \mathcal{L} = -\sum_{k=1}^{K} P_t(x)_k \log P_s(x')_k.$$

Here x and x' are two views, c is a running mean of teacher outputs (**centering**), and a small τ_t makes the teacher's distribution sharp (**sharpening**). Centering stops one dimension from dominating; sharpening stops the output from becoming uniform; together they avoid **collapse**, the trivial solution where every image maps to the same output. The teacher receives no gradients. Its weights are an exponential moving average (the momentum encoder) of the student's:

$$\theta_t \leftarrow \lambda\, \theta_t + (1 - \lambda)\, \theta_s, \qquad 0 < \lambda < 1.$$

DINO also uses multi-crop training (several small local crops for the student, global crops for the teacher), which forces local-to-global correspondence. Its authors report that the resulting ViT features contain explicit information about object segmentation that does not emerge as clearly with supervised ViTs or convnets, and that they work well as k-nearest-neighbor classifiers [24].

DINOv2 [15] scales this recipe and, crucially, curates the data: an automatic pipeline builds a dedicated, diverse, curated image dataset instead of using uncurated web images, a 1-billion-parameter ViT is trained, and it is distilled into smaller models. The authors report that these frozen features surpass OpenCLIP on most of their image-level and pixel-level benchmarks [15].

DINOv3 [16] scales data and model further and introduces **Gram anchoring**, a technique aimed at a known problem: dense (per-patch) feature maps degrading during long training schedules. It also adds post-hoc strategies for flexibility in resolution, model size and alignment with text.

Why dense features matter. A classifier needs one vector per image. Segmentation, depth estimation, correspondence and tracking need a good vector **per patch**. Self-distilled encoders are popular frozen backbones for these dense tasks precisely because their patch features stay semantically meaningful without fine-tuning.

Generative pre-training

Predicting the next token (language models, autoregressive image models in ML 08) or denoising (diffusion in ML 07) is also a pretraining objective. For language it produces the most flexible interface — instructions in natural language — which is why multimodal models attach vision to a pretrained language model rather than the other way round.

Image and text in a shared space: CLIP and SigLIP

Plain version. Train an image encoder and a text encoder so that corresponding pictures and captions point in similar directions. Then any sentence becomes a "class".

CLIP [3] normalizes image embeddings v_i and text embeddings t_j to unit length and forms scores $s_{ij} = v_i^\top t_j / \tau$ with a learned temperature τ. For a batch of B paired examples,

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\left[\frac{1}{B}\sum_i -\log \frac{\exp(s_{ii})}{\sum_j \exp(s_{ij})} + \frac{1}{B}\sum_j -\log \frac{\exp(s_{jj})}{\sum_i \exp(s_{ij})}\right].$$

Each image should select its matching caption among the B captions (first term, a row-wise softmax), and each caption its matching image (second term, column-wise). All other pairs in the batch act as negatives. Training uses paired text supervision, usually from existing captions or alt text; "zero-shot" downstream use does not mean the model was trained without supervision.

At inference, encode candidate descriptions such as "a photo of a dog" and compare an image with them. The text embeddings replace the class weights of a learned classifier:

$$\hat{y} = \arg\max_k v(x)^\top t(\text{prompt}_k).$$

The candidate set and wording affect the result; averaging the embeddings of several templates ("a photo of a k", "a close-up of a k") is a common way to reduce sensitivity. Zero-shot means no task-specific parameter training under the stated protocol; it does not guarantee that the classes or even the test images were absent from pretraining (see *Risks and evaluation*).

Code: paired loss and retrieval mechanics

These **synthetic embeddings** make the computation transparent. They are not outputs from a pretrained model, and their matches do not demonstrate real-world zero-shot accuracy.

```
import torch
from torch.nn import functional as F

images = F.normalize(torch.tensor([[1., .1, 0.], [.1, 1., 0.],
                                   [0., .1, 1.]]), dim=-1)
texts = F.normalize(torch.eye(3), dim=-1)
scores = images @ texts.T / .1
labels = torch.arange(3)
loss = (F.cross_entropy(scores, labels) +
        F.cross_entropy(scores.T, labels))/2
print("matching text:", scores.argmax(1).tolist())
print("paired loss:", round(loss.item(), 4))
ambiguous = F.normalize(torch.tensor([[.5, .5, .2]]), dim=-1)
print("cosines:", (ambiguous @ texts.T).tolist())
```

It prints `matching text: [0, 1, 2]`, a paired loss of about `0.0002`, and cosines of about `0.68 , 0.68, 0.27` for the ambiguous image: it is equally close to two descriptions, and no softmax can tell you which is right.

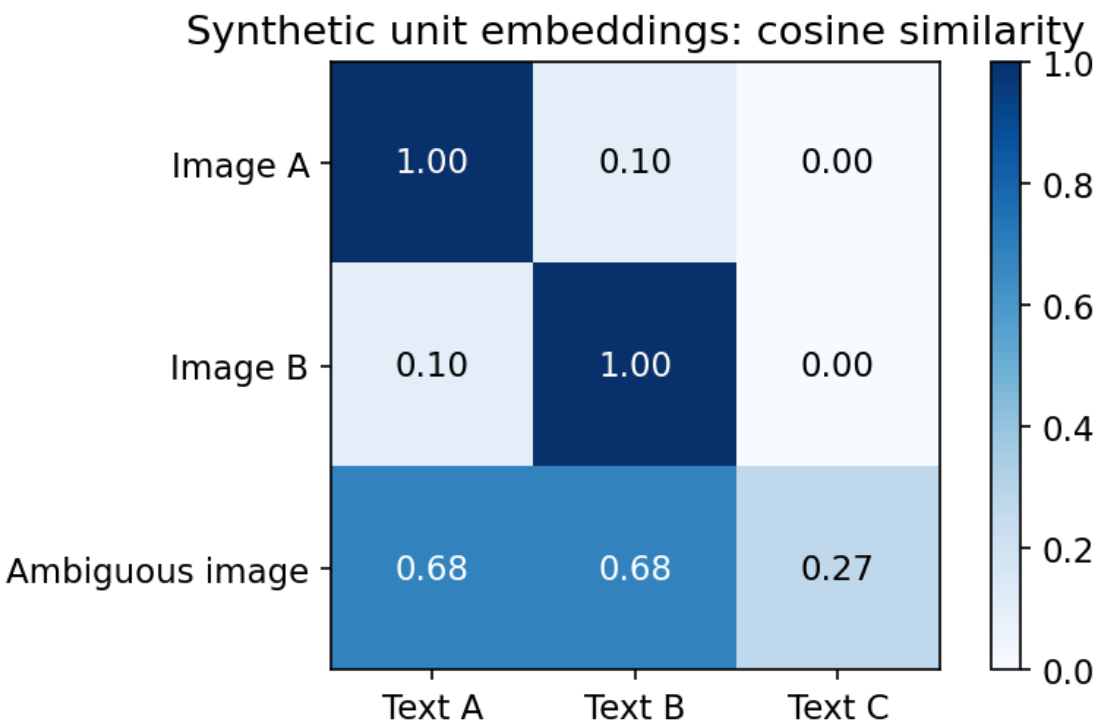


Figure 3 — Cosine similarity ranks candidates. It is not a calibrated probability that a concept is present.

SigLIP: a sigmoid instead of a softmax

SigLIP [4] replaces the batch-normalized softmax with an independent binary decision for every image–text pair. With labels $y_{ij} = +1$ for matching pairs and -1 otherwise, a learned scale a and bias b,

$$\mathcal{L}_{\text{SigLIP}} = -\frac{1}{B}\sum_{i=1}^{B}\sum_{j=1}^{B}\log\sigma(y_{ij}(a\,v_i^\top t_j + b)),$$

where $\sigma(u) = 1/(1+e^{-u})$, so each term equals $\log(1+\exp(-y_{ij}(a\,v_i^\top t_j + b)))$. Because no term needs a sum over the whole batch, the loss does not require a global normalization across devices, which simplifies large-batch training. Since there are B positives and $B^2 - B$ negatives, the bias b is initialized so that the initial predictions are strongly negative.

Shared embeddings support retrieval and open-vocabulary recognition, but fine-grained counting, negation ("a dog **without** a collar"), spatial relations and rare domain terms are known weak points. Evaluate those requirements directly rather than inferring them from a strong overall classification score.

Promptable segmentation: the Segment Anything family

Plain version. A prompt tells the model which region or concept you want, and the output is a mask.

SAM [5] combines a heavy image encoder (run once per image), a light prompt encoder and a light mask decoder, so many prompts can be answered quickly on the same image. Spatial prompts — points, boxes or rough masks — select objects without a fixed class vocabulary. Because a single click is ambiguous (a click on a shirt could mean the shirt or the whole person), SAM predicts several candidate masks with confidence scores. It was trained on SA-1B, over 1 billion masks on 11 million licensed and privacy-respecting images, collected with a model-in-the-loop data engine [5].

SAM 2 [6] extends promptable segmentation to video with a streaming memory: the model stores features of past frames and of the prompted object, and propagates the mask through time; corrective clicks on later frames update the memory.

SAM 3 [7], released in 2025 with a report revised in March 2026, introduces **promptable concept segmentation**: given a short noun phrase (e.g. "yellow school bus"), image exemplars, or both, it returns masks and identities for **all** matching instances. Its image-level detector and memory-based video tracker share one backbone, and a **presence head** decouples recognizing whether the

concept occurs at all from localizing its instances [7].

These interfaces supply different information: a spatial point says *where*, a noun phrase says *what*, an exemplar says *what it looks like*. Report prompt creation cost and whether prompts come from people, ground truth or an automatic detector. A "zero-shot segmentation" score obtained with ground-truth boxes as prompts hides substantial annotation information.

Multimodal large language models

Plain version. Turn visual features into "words" a language model can read, then teach it to follow visual instructions.

A common design has

$$\text{image/video} \xrightarrow{\text{vision encoder}} h \xrightarrow{\text{connector}} z \xrightarrow{\text{language decoder with text}} \text{answer}.$$

Here h is a grid of visual features (for a ViT, one vector per patch), z is a sequence of vectors in the language model's token-embedding space, and the decoder treats z like the embeddings of ordinary words.

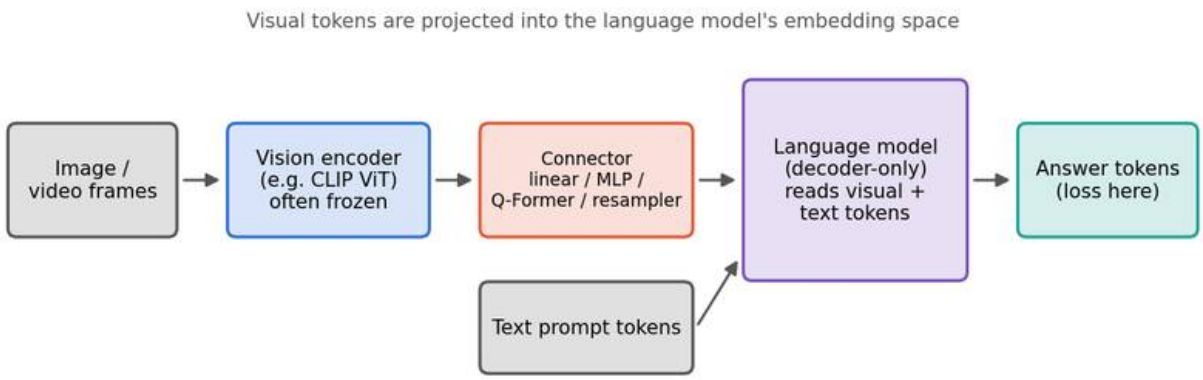


Figure 4 — The typical multimodal LLM pipeline. Visual features are converted into tokens in the language model's embedding space.

Connectors

- **Linear or MLP projection.** LLaVA [8] uses a single projection from CLIP features to the language model's embedding space; LLaVA-1.5 [28] reports that a simple MLP connector (with a CLIP ViT-L at 336-pixel resolution and academic VQA data) is surprisingly powerful and data-efficient. Every patch becomes one visual token.

- **Learned queries.** BLIP-2 [26] inserts a lightweight Querying Transformer (Q-Former) between a frozen image encoder and a frozen language model. A fixed set of learned query vectors cross-attends to the image features, so the language model always sees a small, fixed number of visual tokens. BLIP-2 trains the Q-Former in two stages: representation learning against the frozen image encoder, then generative learning against the frozen language model.
- **Resampling and cross-attention.** Flamingo [27] bridges frozen vision and language models and handles arbitrarily interleaved sequences of images and text, which enabled few-shot learning from examples written directly in the prompt.

The trade-off is token count versus detail: one token per patch keeps fine detail but makes long sequences (expensive for many images or video); a few learned queries are cheap but can lose small text.

Visual instruction tuning

LLaVA [8] used a language-only GPT-4 [45] to generate multimodal instruction-following data from image captions and boxes, then trained in two stages: first align the connector (feature alignment), then fine-tune connector and language model on the instructions. The loss is ordinary next-token cross-entropy, applied to the **answer** tokens only:

$$\mathcal{L} = -\sum_{i \in \mathcal{A}} \log p_\theta(w_i \mid z,\, w_{<i}),$$

where w_i are text tokens, $\mathcal{A}$ is the set of positions belonging to the assistant's answer, and z are the visual tokens. Instruction and prompt tokens are visible as context but masked out of the loss.

Qwen3-VL's 2025 report [9] illustrates the current direction: a family of dense and mixture-of-experts models with natively interleaved image, video and text contexts of up to 256K tokens. It is a representative report, not evidence that every task benefits from longer contexts or more generated reasoning.

What the diagram hides

Reading a document may require preserving small text through tiling or dynamic resolution. Video questions need timestamps and temporal order. These details change token counts and preprocessing; a generic "vision encoder + LLM" diagram is only the starting point. A fluent answer can also contain objects that are not in the image (**object hallucination**) or wrong counts. Distinguish object grounding, OCR, spatial reasoning, factual answering and stylistic instruction following — they need different evidence and metrics (see *Risks and evaluation*). For a survey of the whole design space, see Yin et al. [40] and Caffagni et al. [2].

Adapting a foundation model

Plain version. Use the least expensive adaptation that solves the target task, and check it on data that looks like deployment.

Strategy	Trainable components	Useful starting situation
Prompting / in-context examples	Nothing	The model's interface already fits the task
Frozen features + linear head (linear probe)	Small supervised readout	Few labels, good feature transfer
Prompt tuning / visual prompt tuning	A few learned input tokens	Many tasks share one frozen backbone
Adapters / LoRA	Small inserted or low-rank modules	Domain adaptation under memory limits
Partial fine-tuning	Selected blocks plus head	Some adaptation without full optimizer cost
Full fine-tuning (optionally LP-FT)	All model parameters	Enough data and compute; transfer alone insufficient

Linear probe

Freeze the encoder f, compute features once, and fit a linear classifier $\hat{y} = \text{softmax}(Wf(x) + b)$.

It is cheap, hard to overfit, and is also the standard **measurement** of representation quality: if a linear readout works, the information is linearly accessible in the features.

Code: why pretrained features help most with few labels

This toy experiment simulates the whole story. Observations are 300-dimensional and mix 8 **semantic** factors (which determine the class) with 8 large **nuisance** factors (think lighting and pose) plus noise. "Pretraining" sees 50 upstream classes with plenty of labels and learns an 8-dimensional projection (linear discriminant analysis, standing in for a deep encoder). Downstream, there are four **new** classes and only a few labels per class.

```
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(0)
D, k_sem, k_nui = 300, 8, 8
sem_map = rng.normal(size=(k_sem, D))   # semantic factors -> pixels
nui_map = rng.normal(size=(k_nui, D))   # nuisance factors (light, pose...)

def sample(centers, n):
    y = rng.integers(len(centers), size=n)
```

```
    z = centers[y] + 0.7 * rng.normal(size=(n, k_sem))
    u = 4.0 * rng.normal(size=(n, k_nui))          # large nuisance variation
    x = z @ sem_map + u @ nui_map + rng.normal(size=(n, D))
    return x, y

# 1) "Pre-training": many upstream classes with plenty of labels.
up_centers = rng.normal(size=(50, k_sem))
x_up, y_up = sample(up_centers, 10_000)
encoder = LinearDiscriminantAnalysis(n_components=k_sem).fit(x_up, y_up)

# 2) Downstream: four NEW classes, few labels, linear probe only.
new_centers = rng.normal(size=(4, k_sem))
x_te, y_te = sample(new_centers, 2_000)
f_te = encoder.transform(x_te)          # frozen features, computed once
for shots in [1, 5, 20, 100]:
    raw, feat = [], []
    for repeat in range(10):
        x_tr, y_tr = sample(new_centers, 4 * shots)
        raw.append(LogisticRegression(max_iter=3000)
                   .fit(x_tr, y_tr).score(x_te, y_te))
        feat.append(LogisticRegression(max_iter=3000)
                    .fit(encoder.transform(x_tr), y_tr).score(f_te, y_te))
    print(f"{shots:>3} labels/class | raw pixels {np.mean(raw):.3f}"
          f" | frozen features {np.mean(feat):.3f}")
```

On our machine it prints accuracies of roughly 0.29 vs 0.71 with one label per class, 0.62 vs 0.95 with five, 0.96 vs 0.99 with twenty and 0.98 vs 0.99 with a hundred (raw inputs vs frozen features; chance is 0.25). The pretrained projection already discards the nuisance directions, so a handful of labels suffice; with enough labels, the raw-input probe learns to ignore the nuisance itself and the gap closes. The upstream classes are different from the downstream ones — the features transfer because both share the same semantic factors. If the downstream task depended on a factor that pretraining threw away (here, the nuisance), frozen features would fail no matter how many labels you add. That is the failure mode to test for in practice.

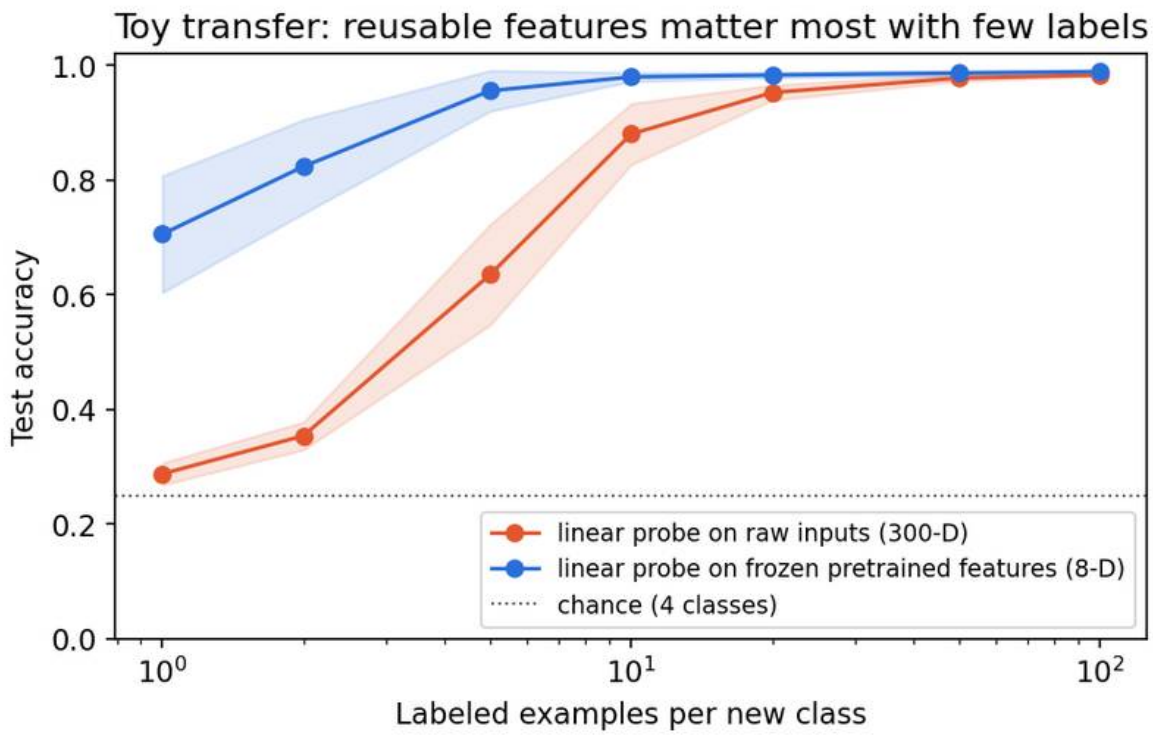


Figure 5 — The toy transfer experiment over more label budgets (mean ± one standard deviation over 10 draws). Reusable features matter most when labels are scarce.

Fine-tuning, LP-FT and robust fine-tuning

Full fine-tuning updates every weight and usually gives the best in-distribution accuracy when data suffices. But Kumar et al. [32] showed that it can distort good pretrained features: across their benchmarks, fine-tuning gave on average higher in-distribution but lower out-of-distribution accuracy than linear probing. Their explanation is that a randomly initialized head sends large, noisy gradients into the backbone early in training. **LP-FT** — first fit a linear probe, then fine-tune everything starting from that head — reduced this distortion and outperformed both alternatives in their experiments [32]. **WiSE-FT** [33] takes another route: linearly interpolate the weights of the zero-shot and fine-tuned models, $\theta_\gamma = (1-\gamma)\theta_{\text{zero-shot}} + \gamma\,\theta_{\text{ft}}$, which improved robustness under distribution shift for CLIP-style models while keeping strong target accuracy. Both are cheap to try and worth knowing before declaring that fine-tuning "hurt robustness".

Parameter-efficient fine-tuning (PEFT)

Adapters [29] insert small bottleneck MLPs inside each Transformer block and train only them, adding a few parameters per task while the original network stays fixed. **Prompt tuning** learns a few extra input tokens; **Visual Prompt Tuning (VPT)** [30] does this for ViTs, prepending learnable tokens to the patch sequence of a frozen backbone. Xu et al. [42] review the PEFT landscape with a taxonomy of additive, partial, reparameterized, hybrid and unified fine-tuning, plus experiments on parameter and memory efficiency.

LoRA [10] reparameterizes an update to a frozen weight $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ as

$$W = W_0 + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r},$$

with rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$ and a scaling constant α. Freeze W_0 and learn A, B. Trainable weights drop from $d_{\text{out}} d_{\text{in}}$ to $r(d_{\text{in}} + d_{\text{out}})$. Initialize $B = 0$ so the initial update is zero, and A randomly so learning can start. After training, BA can be merged into W_0, so inference costs nothing extra. QLoRA [31] stores the frozen base weights in 4-bit precision and backpropagates through them into LoRA adapters, cutting memory enough to fine-tune very large models on a single GPU.

Code: a low-rank update with a frozen base

This standalone example trains one adapter step on synthetic input. It shows the parameterization and initial behavior; it does not reproduce a full pretrained-model fine-tuning pipeline.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
class LoRALinear(nn.Module):
    def __init__(self, din, dout, rank=4, alpha=4):
        super().__init__()
        self.base = nn.Linear(din, dout, bias=False)
        self.base.weight.requires_grad_(False)
        self.A = nn.Parameter(torch.randn(rank, din) * .02)
        self.B = nn.Parameter(torch.zeros(dout, rank))
        self.scale = alpha/rank
    def forward(self, x):
        return self.base(x) + self.scale * F.linear(F.linear(x, self.A), self.B)

layer = LoRALinear(32, 16)
x = torch.randn(8, 32)
assert torch.allclose(layer(x), layer.base(x))
opt = torch.optim.Adam([layer.A, layer.B], lr=.01)
loss = F.mse_loss(layer(x), torch.randn(8, 16))
opt.zero_grad(); loss.backward(); opt.step()
```

```
assert layer.base.weight.grad is None
print("trainable:", sum(p.numel() for p in layer.parameters() if p.requires_grad))
```

It prints `trainable: 192`, which is $r(d_{\text{in}} + d_{\text{out}}) = 4 \times (32 + 16)$, against 512 for the full matrix. The first assertion checks that the zero-initialized B leaves the model unchanged at the start; the second, that the frozen base receives no gradient.

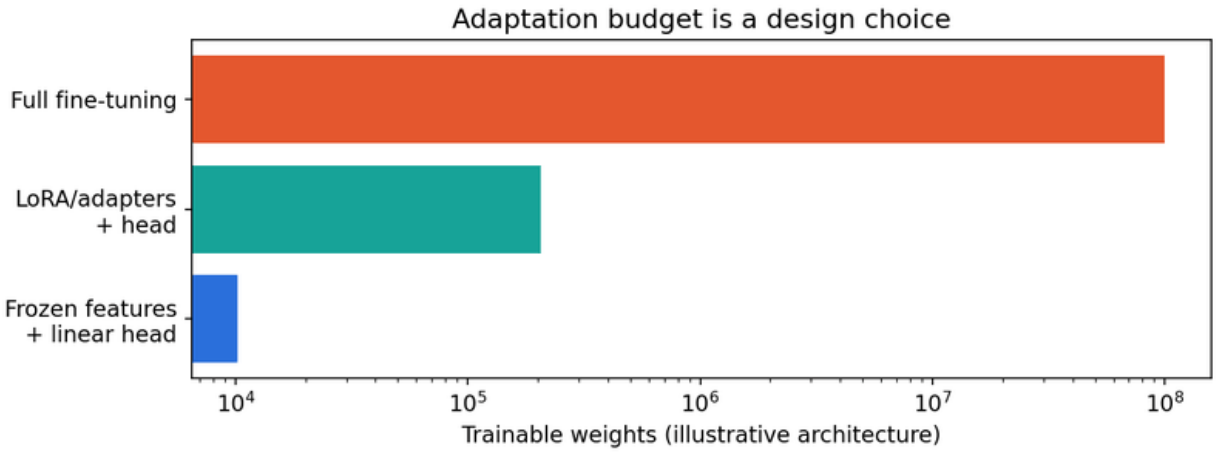


Figure 6 — Analytical trainable-weight budgets for an illustrative architecture (feature size 1024, 10 classes, rank 8 on 12 matrices, 100M parameters in total). Activation memory, frozen weight storage and optimizer state must be counted separately.

LoRA is a parameterization, not a guarantee against overfitting or forgetting. Adapters still require domain validation. Frozen BatchNorm statistics, image normalization, output-label mapping and checkpoint-specific preprocessing also need deliberate treatment.

Prompting and in-context learning

The cheapest adaptation changes no weights: write a better text prompt for CLIP, give SAM a box, or put a few solved examples into a multimodal model's context, as Flamingo did [27]. The vision–language survey by Zhang et al. [41] compares prompting, feature adaptation and fine-tuning across vision tasks. Prompting is fast to iterate but brittle: small wording changes can move results, so fix and report the prompt protocol.

Video, geometry and world models

Plain version. Beyond recognizing a frame, a model may predict change over time, infer three-dimensional structure, or support action planning.

V-JEPA 2 [11] studies self-supervised video models that predict in feature space rather than pixel space, with an action-conditioned extension used for planning. VGGT [12] predicts camera parameters, depth, point maps and tracks from one or many views in a single feed-forward pass. These illustrate two routes beyond classification: temporal state modeling and geometric inference.

A generated video that looks convincing need not obey physical constraints. A scene reconstruction can be useful without generating photorealistic pixels. Measure multi-view consistency, geometry, action-conditioned outcomes and temporal behavior according to the claimed capability. DLCV 07 ·World Models develops partial observability, learned dynamics and planning in more detail.

Risks and evaluation

Plain version. A model trained on a large slice of the internet inherits that slice's biases and gaps, may confidently describe things that are not there, and may already have seen your "test" set. Evaluation has to be designed around these facts.

Data provenance, bias and privacy

Web-scale image–text datasets are too large to inspect by hand. LAION-5B [34] made such a dataset public (5.85 billion CLIP-filtered pairs) and thereby enabled open replication such as OpenCLIP [20]; audits of its predecessor LAION-400M by Birhane et al. [35] documented explicit and stereotyping content in image–text pairs. Downstream systems inherit these associations. Licenses and consent also matter: training data, model weights and generated outputs may each carry different terms, and personal data (faces, names, medical images) raises privacy obligations that do not disappear because data was public. SAM's SA-1B is an example of a dataset whose description emphasizes licensed and privacy-respecting images [5]; record such properties for every model you deploy.

Hallucination

Multimodal LLMs can mention objects that are absent from the image. POPE [37] measures object hallucination with a polling approach: it asks yes/no questions about the presence of specific objects (including frequent and co-occurring distractors), which makes the measurement less dependent on the model's generation style than scoring free-form captions. A low POPE error does not cover counting, attributes or relations; test those separately.

Contamination and "zero-shot"

If benchmark images (or near-duplicates) are in the pretraining set, a "zero-shot" score partly measures memory. Mayilvahanan et al. [36] asked whether CLIP's generalization mainly stems

from high train–test similarity; after pruning LAION to remove images too similar to common out-of-distribution benchmarks and retraining, performance dropped on some benchmarks but remained strong overall. The lesson is two-sided: similarity explains part of the reported performance, not all of it — so measure it rather than assume either way. Practical checks: deduplicate the test set against available training data, report results on data collected after the model's data cutoff, and state the full zero-shot protocol (class names, templates, candidate set).

Benchmarks

Aggregate multimodal benchmarks such as MMMU [38] — 11.5K college-level questions from exams, quizzes and textbooks across six disciplines, with charts, diagrams, maps, tables and other image types — test knowledge and reasoning, but a single number mixes perception, OCR, domain knowledge and answer formatting. For a deployment, build a small, task-specific evaluation set with known failure cases, report variance over seeds or prompt wordings, and compare against a simple baseline (a linear probe on frozen features is often surprisingly strong). ML 10 · Evaluation and Deployment covers calibration, splits and deployment in depth.

Modern view

Surveys worth reading.

- **Bommasani et al. [1]** — the report that named foundation models. It covers capabilities (language, vision, robotics, reasoning), applications (law, health, education), technology (modeling, training, adaptation, evaluation, data) and societal impact. Its key takeaways — emergence, homogenization, and inherited failure — still frame the field.
- **Awais et al. [39]** — a computer-vision survey organizing foundation models by architecture design, training objective (contrastive versus generative), datasets, fine-tuning mechanisms and prompting, and listing open challenges in evaluation, real-world applicability, adversarial robustness and interpretability.
- **Zhang et al. [41]** — a TPAMI survey of vision–language models for vision tasks: network architectures, pre-training objectives, downstream tasks, datasets, transfer learning and knowledge distillation, with benchmarking of reviewed methods.
- **Yin et al. [40]** and **Caffagni et al. [2]** — surveys of multimodal large language models: architecture (encoder, connector, LLM), training strategy and data (pre-training, instruction tuning, alignment), evaluation, and hallucination.
- **Han et al. [13]** — a survey broadening the map to multimodal *generative* models, organized by generative category and technique.
- **Xu et al. [42]** — a critical review of parameter-efficient fine-tuning that sorts methods into additive, partial, reparameterized, hybrid and unified families and compares representative

ones experimentally.

State of the art, 2023–2026. Four trends stand out. (1) **Better frozen features**: DINOv2 [15] and DINOv3 [16] made self-supervised encoders strong enough that many dense tasks start from frozen patch features. (2) **Concept-level prompting**: segmentation moved from "click an object" (SAM [5]) to video memory (SAM 2 [6]) to "segment every instance of this phrase" (SAM 3 [7]). (3) **Long, interleaved multimodal contexts**: models such as Qwen3-VL [9] read many images, video and text together. (4) **Beyond recognition**: video prediction for planning (V-JEPA 2 [11]) and feed-forward geometry (VGGT [12]) treat foundation models as world and scene estimators.

Open problems.

- Grounded reliability: reducing hallucination and making models say "not visible" when appropriate [37].
- Honest evaluation under contamination and benchmark saturation [36].
- Data: curation as a scaling axis [15], [20], provenance, licensing and consent [34], [35].
- Efficient adaptation and inference: token budgets for high-resolution images and video; PEFT that preserves robustness [32], [33].
- Dense, 3D and temporal understanding that matches the semantic strength of image-level models.

For any new "foundation" model, record its input/output interface, training objective, data sources, supported adaptation, checkpoint license, preprocessing and task-level evaluation. This avoids treating model size or an open-ended chat interface as proof of general competence.

Key takeaways

- "Foundation" describes broad pretraining plus reuse, not a particular architecture; such models need not generate text.
- Scaling laws describe smooth, power-law-like loss improvements with a floor; compute-optimal training balances model size and data; the constants depend on data and recipe.
- The pretraining objective decides what the features are good at: contrastive for text-defined recognition, masked modeling for fine-tuning, self-distillation for frozen dense features, generative for flexible interfaces.
- CLIP-style alignment enables retrieval and text-defined classification; cosine similarity is not calibrated confidence.
- Spatial, conceptual and language prompts provide different kinds of information; report who made the prompts.
- Multimodal LLMs = vision encoder + connector + language model, trained with answer-only next-token loss; the connector sets a detail-versus-token-count trade-off.
- Start adaptation cheap (prompt, linear probe), then PEFT (LoRA, adapters), then full fine-

tuning; LP-FT and weight interpolation help preserve robustness.
- Evaluate for bias, hallucination and contamination explicitly; one aggregate benchmark number is not evidence of general competence.

Exercises

1. **Candidate dependence.** Add a fourth description to the synthetic CLIP example. Can the softmax "confidence" for an image change without changing the image?

   Hint

   Yes: the denominator of the softmax changes. Similarity to a fixed description is unchanged, but the normalized probability depends on the candidate set. Try adding a text vector close to the ambiguous image's embedding.

2. **Compute-optimal allocation.** Using the parametric law $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ and $C = 6ND$, show that the optimal model size grows as $N^{\star} \propto C^{\beta/(\alpha+\beta)}$. What does $\alpha = \beta$ imply?

   Hint

   Substitute $D = C/(6N)$, differentiate with respect to N and set to zero: $\alpha A N^{-\alpha-1} = \beta B (6N/C)^{\beta}/N$. Solving gives $N^{\alpha+\beta} \propto C^{\beta}$. With $\alpha = \beta$, $N^{\star} \propto C^{1/2}$ and therefore $D^{\star} \propto C^{1/2}$ too: scale parameters and tokens equally, as Hoffmann et al. concluded.

3. **Break the transfer.** In the linear-probe experiment, make the downstream class depend on the *nuisance* factors instead of the semantic ones. What happens to the frozen-feature probe as labels increase, and what does that say about choosing a foundation model?

   Hint

   Draw class centers for `u` instead of `z`. The frozen 8-D features were trained to discard the nuisance directions, so they stay near chance however many labels you add, while the raw-input probe improves. A backbone is only useful if its pretraining kept the information your task needs — check with a linear probe before investing in adaptation.

4. **LoRA budget.** A ViT-B block has four 768×768 attention projections and two MLP matrices of sizes 3072×768 and 768×3072. Count trainable weights for full fine-tuning of these six matrices and for rank-8 LoRA on all of them.

   Hint

   Full: $4{\cdot}768^2 + 2{\cdot}3072{\cdot}768 = 2{,}359{,}296 + 4{,}718{,}592 = 7{,}077{,}888$. LoRA: $8{\cdot}[4{\cdot}(768{+}768) + 2{\cdot}(3072{+}768)] = 8 \cdot (6144 + 7680) = 110{,}592$, about 1.6% of the full count.

5. **Prompt fairness.** Design a SAM evaluation that separates mask quality from prompt quality.

Hint

Fix the prompt protocol and count human interactions. Evaluate automatic proposals separately from masks produced with ideal ground-truth prompts, and report both. For SAM 3, compare noun-phrase prompts with exemplar prompts on the same images.

6. **Contamination audit.** You are given a new image–text model that reports high zero-shot accuracy on a public benchmark. List three checks you would run before trusting the number.

Hint

(a) Near-duplicate search of test images against any available training data (e.g. by embedding similarity); (b) evaluation on a freshly collected set from after the data cutoff; (c) sensitivity to the prompt templates and the candidate class list; optionally compare with a linear probe and a simple baseline.

References

1. R. Bommasani et al., "On the Opportunities and Risks of Foundation Models," Stanford CRFM report, arXiv:2108.07258, 2021; revised 2022. arXiv
2. D. Caffagni et al., "The Revolution of Multimodal Large Language Models: A Survey," *Findings of ACL*, 2024. arXiv
3. A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," *ICML*, 2021. arXiv
4. X. Zhai, B. Mustafa, A. Kolesnikov and L. Beyer, "Sigmoid Loss for Language Image Pre-Training," *ICCV*, 2023. arXiv
5. A. Kirillov et al., "Segment Anything," *ICCV*, 2023. arXiv
6. N. Ravi et al., "SAM 2: Segment Anything in Images and Videos," *ICLR*, 2025. arXiv
7. N. Carion et al., "SAM 3: Segment Anything with Concepts," arXiv:2511.16719, 2025; revised March 2026. arXiv
8. H. Liu, C. Li, Q. Wu and Y. J. Lee, "Visual Instruction Tuning," *NeurIPS*, 2023. arXiv
9. S. Bai et al., "Qwen3-VL Technical Report," arXiv:2511.21631, 2025. arXiv
10. E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR*, 2022. arXiv
11. M. Assran et al., "V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning," arXiv:2506.09985, 2025. arXiv
12. J. Wang et al., "VGGT: Visual Geometry Grounded Transformer," *CVPR*, 2025. arXiv
13. L. Han et al., "A Survey of Generative Categories and Techniques in Multimodal Generative Models," arXiv:2506.10016, 2025. arXiv
14. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
15. M. Oquab et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. arXiv

16. O. Siméoni et al., “DINOv3,” arXiv:2508.10104, 2025. arXiv
17. J. Kaplan et al., “Scaling Laws for Neural Language Models,” arXiv:2001.08361, 2020. arXiv
18. J. Hoffmann et al., “Training Compute-Optimal Large Language Models,” arXiv:2203.15556, 2022; published as “An empirical analysis of compute-optimal large language model training,” *NeurIPS*, 2022. arXiv
19. X. Zhai, A. Kolesnikov, N. Houlsby and L. Beyer, “Scaling Vision Transformers,” *CVPR*, 2022. arXiv
20. M. Cherti et al., “Reproducible Scaling Laws for Contrastive Language-Image Learning,” *CVPR*, 2023. arXiv
21. A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021. arXiv
22. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, “Masked Autoencoders Are Scalable Vision Learners,” *CVPR*, 2022. arXiv
23. H. Bao, L. Dong, S. Piao and F. Wei, “BEiT: BERT Pre-Training of Image Transformers,” *ICLR*, 2022. arXiv
24. M. Caron et al., “Emerging Properties in Self-Supervised Vision Transformers,” *ICCV*, 2021. arXiv
25. C. Jia et al., “Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision,” *ICML*, 2021. arXiv
26. J. Li, D. Li, S. Savarese and S. Hoi, “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” *ICML*, 2023. PMLR
27. J.-B. Alayrac et al., “Flamingo: a Visual Language Model for Few-Shot Learning,” *NeurIPS*, 2022. arXiv
28. H. Liu, C. Li, Y. Li and Y. J. Lee, “Improved Baselines with Visual Instruction Tuning,” *CVPR*, 2024. arXiv
29. N. Houlsby et al., “Parameter-Efficient Transfer Learning for NLP,” *ICML*, 2019. PMLR
30. M. Jia et al., “Visual Prompt Tuning,” *ECCV*, 2022. arXiv
31. T. Dettmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” *NeurIPS*, 2023. arXiv
32. A. Kumar, A. Raghunathan, R. Jones, T. Ma and P. Liang, “Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution,” *ICLR*, 2022. arXiv
33. M. Wortsman et al., “Robust Fine-Tuning of Zero-Shot Models,” *CVPR*, 2022. arXiv
34. C. Schuhmann et al., “LAION-5B: An Open Large-Scale Dataset for Training Next Generation Image-Text Models,” *NeurIPS Datasets and Benchmarks Track*, 2022. arXiv
35. A. Birhane, V. U. Prabhu and E. Kahembwe, “Multimodal Datasets: Misogyny, Pornography, and Malignant Stereotypes,” arXiv:2110.01963, 2021. arXiv
36. P. Mayilvahanan, T. Wiedemer, E. Rusak, M. Bethge and W. Brendel, “Does CLIP’s Generalization Performance Mainly Stem from High Train-Test Similarity?,” *ICLR*, 2024. arXiv
37. Y. Li et al., “Evaluating Object Hallucination in Large Vision-Language Models,” *EMNLP*, 2023. arXiv
38. X. Yue et al., “MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI,” *CVPR*, 2024. arXiv
39. M. Awais et al., “Foundational Models Defining a New Era in Vision: A Survey and Outlook,” arXiv:2307.13721, 2023. arXiv

40. S. Yin et al., “A Survey on Multimodal Large Language Models,” *National Science Review*, 2024. arXiv
41. J. Zhang, J. Huang, S. Jin and S. Lu, “Vision-Language Models for Vision Tasks: A Survey,” *IEEE TPAMI*, vol. 46, no. 8, 2024. arXiv
42. L. Xu et al., “Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment,” arXiv:2312.12148, 2023. arXiv
43. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” *CVPR*, 2009, pp. 248–255. doi
44. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL-HLT*, 2019, pp. 4171–4186. doi
45. OpenAI, “GPT-4 Technical Report,” arXiv:2303.08774, 2023. arXiv

ML 10 • Evaluation, Efficiency and Deployment

Split data without leakage, cross-validate, attach confidence intervals, check calibration and robustness under shift, measure FLOPs, memory and latency honestly, then export, compress, monitor and reproduce a model.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-10-evaluation-deployment/

What you'll learn

- How to build a baseline and a data split whose result actually supports the claim you want to make, and how leakage silently breaks that.
- How K-fold, grouped, time-ordered and nested cross-validation work, and when each one is the right tool.
- How to put an uncertainty on a test score: standard errors, the paired bootstrap and McNemar's test.
- How calibration differs from accuracy, how to measure it (Brier score, NLL, ECE) and how temperature scaling fixes part of it.
- What distribution shift is (covariate, label, concept), how robustness benchmarks probe it, and how to monitor it after deployment.
- How to count FLOPs, memory and latency correctly, and what export, quantization, pruning, distillation and inference runtimes such as ONNX Runtime and TensorRT really buy you.
- How to make an experiment reproducible and document it.

The big picture

Training a model is the middle of a project, not the end. Before anyone relies on it, you need to answer four questions. Does it work on the cases it will really see? How sure are we of that number? Does it know when it is likely to be wrong? Does it fit the device, latency and budget it has to run on? After it ships, a fifth question appears: is it still working?

> **Explain like I'm five**
>
> A student who memorized last year's exam gets full marks on last year's exam. That tells you nothing. A fair test uses new questions (a clean split). One test can be lucky, so you want to know how much the score would wobble on another day (a confidence interval). A good student also says "I'm not sure" on the questions they might get wrong (calibration). If the real exam is in a noisy hall, you check that too (robustness). If it is a timed exam, being right slowly is not enough (latency). And once the student starts a job, someone still checks their work now and then (monitoring).

This chapter closes the ML track. It gathers the practical methodology from Goodfellow et al. [1, ch. 11] and the statistical-learning view of James et al. [2], and adds the engineering steps that

turn a trained network into a dependable service. The task-specific metrics of vision (top-1, mAP, PSNR, FID and so on) are covered in DLCV 02 and DLCV 03; here we focus on what all tasks share.

Start with a meaningful baseline

Plain version. Before spending more compute, find out how well a simple method does on exactly the same split. The baseline tells you what the complicated model actually buys.

Good baselines for image tasks include: logistic regression on raw pixels or hand-designed features; a nearest-neighbor or SVM classifier; and a small linear head on frozen features from a pretrained model (see ML 09). Logistic regression learns a linear boundary; a support vector machine (SVM) maximizes a regularized margin; nearest neighbors compares examples directly; tree ensembles partition the feature space. The classical pattern-classification chapter, IP 13, develops these methods.

For reference, the binary linear SVM minimizes

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\max(0,\, 1 - y_i(w^\top x_i + b)), \qquad y_i \in \{-1, +1\},$$

where w and b are the weights and bias, x_i is the i-th feature vector, and $C > 0$ trades margin width against training errors. The term $\max(0, 1 - y_i f(x_i))$ is the **hinge loss**: it is zero for points on the correct side with margin at least 1 and grows linearly otherwise.

Code: baselines with preprocessing inside the pipeline

This example uses only bundled data. Model choice uses a validation split; the chosen model is evaluated on the test split once. Scaling and PCA are fitted inside each training pipeline, never on the full dataset.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, balanced_accuracy_score

X, y = load_digits(return_X_y=True)
Xa, Xt, ya, yt = train_test_split(X, y, test_size=.2, stratify=y, random_state=0)
Xtr, Xv, ytr, yv = train_test_split(Xa, ya, test_size=.25, stratify=ya, random_state=1)
models = {      # listed from simplest to most complex
    "linear": make_pipeline(StandardScaler(),
                            LogisticRegression(max_iter=2000)),
```

```
    "PCA + RBF SVM": make_pipeline(StandardScaler(), PCA(n_components=30),
                                   SVC(C=10, gamma="scale")),
}
scores = {}
for name, model in models.items():
    model.fit(Xtr, ytr)
    scores[name] = accuracy_score(yv, model.predict(Xv))
best = max(scores, key=scores.get)   # tie -> first listed, i.e. the simpler model
pred = models[best].predict(Xt)
print("validation:", scores)
print("selected:", best)
print("test accuracy:", accuracy_score(yt, pred))
print("test balanced accuracy:", balanced_accuracy_score(yt, pred))
```

On this split both candidates score exactly 0.9639 on validation (347 of 360 images). The rule "on a tie, prefer the simpler model" was fixed in advance, so the linear model is selected, and it scores 0.9639 accuracy and 0.9636 balanced accuracy on test. Two lessons hide in this tiny run. First, a tie-break rule is part of the protocol and should be written down before you look. Second, with 360 validation images a single image is worth 0.28 percentage points, so many apparent "wins" in small experiments are noise; the section on confidence intervals makes this precise.

Data splits and leakage

Plain version. The test set must be new in the same way that deployment data will be new. If test images share a patient, a video, a scene or a time period with training images, the test score measures memory, not generalization.

Choose the split unit from the claim

Images from one patient, one video, one production batch or one camera are correlated. Randomly splitting adjacent video frames puts almost the same picture in train and test. Ask what "new" means in deployment and split by that unit: by patient if the model will see new patients, by video if it will see new videos, by site if it will be installed in new hospitals, by time if it must predict the future.



Figure 1 — Choose the split unit according to the generalization claim. Group separation prevents near-duplicate context from crossing the split.

Stratification keeps class proportions similar across splits; it does not replace grouping. Deduplicate before splitting, and keep exact or near duplicates together. When the deployment population drifts over time, a time-based split (train on the past, test on the most recent period) is the honest choice.

The many faces of leakage

Leakage is any path by which information that would not be available at prediction time reaches the model or the model-selection process. Kapoor and Narayanan found leakage errors affecting 329 papers across 17 scientific fields that use ML, and propose a taxonomy of leakage types, among them missing train/test separation, preprocessing on the full dataset, duplicates, temporal leakage and illegitimate features [4]. The scikit-learn documentation lists the most common coding mistakes [3]. A practical checklist:

Leak	Example	Fix
Preprocessing on all data	Fit `StandardScaler`, PCA or a vocabulary on train + test	Put preprocessing inside a `Pipeline` fitted on the training fold only
Correlated units	Frames of one video, slices of one scan, crops of one photo	Split by group
Duplicates	The same image scraped twice	Hash or embed and deduplicate before splitting
Temporal	Using next week's data to predict this week	Time-ordered splits
Target proxy	A feature recorded *after* the outcome (e.g. "treatment given")	Audit each feature's timestamp
Test reuse	Tuning hyperparameters while watching the test score	Validation for decisions, test once

The last row is subtle. Every time you look at the test score and change something, the test set becomes a little more like a validation set. Public benchmarks suffer from this at community scale.

Cross-validation

Plain version. Instead of one train/validation split, rotate: split the data into K parts, train K times, each time holding out a different part, and average. You use every example for evaluation once and you see how much the score varies.

K-fold and its variants

Partition the indices into K disjoint folds $F_1, \dots, F_K$. For each k, train $\hat{f}^{(-k)}$ on all folds except F_k and compute its error on F_k. The cross-validation estimate is

$$\widehat{\mathrm{Err}}_{\mathrm{CV}} = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{|F_k|} \sum_{i \in F_k} \ell(y_i, \hat{f}^{(-k)}(x_i)),$$

where ℓ is the per-example loss (for example 0–1 error) and $|F_k|$ is the fold size. Common choices are $K = 5$ or $K = 10$ [2]. The variants differ only in how folds are formed:

- **Stratified K-fold** keeps class proportions equal across folds; useful when classes are imbalanced.
- **Group K-fold** assigns each group (patient, video, site) to exactly one fold, so no group appears in both training and held-out data.
- **Time-series split** always trains on earlier data and validates on later data.
- **Leave-one-group-out** is group K-fold with one group per fold; useful with few groups.

The fold scores are not independent (training sets overlap), so the standard deviation across folds understates the true uncertainty. Treat it as a rough indication of stability, not as a formal confidence interval.

Code: what random K-fold does to grouped data

We simulate 40 patients with 25 images each. Every patient's images share a "fingerprint" (patient-specific appearance), and the label is fixed per patient. The real class signal is weak.

```
import numpy as np
from sklearn.model_selection import KFold, GroupKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
```

```
rng = np.random.default_rng(0)
n_groups, per_group = 40, 25                    # e.g. 40 patients, 25 images each
groups = np.repeat(np.arange(n_groups), per_group)
y = np.repeat(rng.integers(0, 2, n_groups), per_group)   # label fixed per patient
identity = rng.normal(size=(n_groups, 20))[groups]       # patient "fingerprint"
signal = 0.3 * y[:, None] * np.ones((1, 20))             # weak real class signal
X = identity + signal + 0.3 * rng.normal(size=(len(y), 20))

model = RandomForestClassifier(n_estimators=200, random_state=0)
random_cv = cross_val_score(model, X, y, cv=KFold(5, shuffle=True, random_state=0))
group_cv = cross_val_score(model, X, y, cv=GroupKFold(5), groups=groups)
print(f"random K-fold accuracy: {random_cv.mean():.3f} +/- {random_cv.std():.3f}")
print(f"group  K-fold accuracy: {group_cv.mean():.3f} +/- {group_cv.std():.3f}")
```

Output:

```
random K-fold accuracy: 0.999 +/- 0.002
group  K-fold accuracy: 0.657 +/- 0.158
```

The random split reports near-perfect accuracy because the forest recognizes each patient's fingerprint and copies that patient's label. Group K-fold, which matches the claim "works on new patients", reports 0.657 with large variation across folds. Neither number is wrong as arithmetic; only one answers the question.

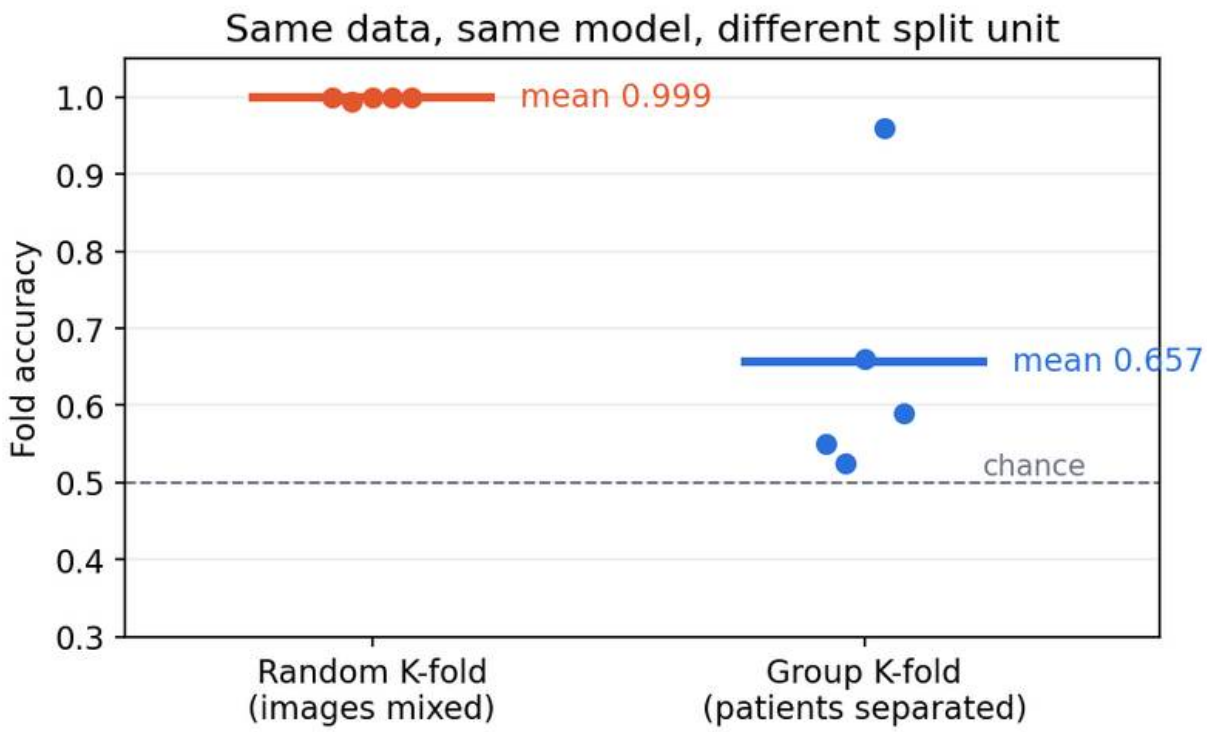


Figure 2 — Output of the code above, one dot per fold. The split unit, not the model, causes the 34-point gap.

Nested cross-validation

If you use cross-validation both to **choose** hyperparameters and to **report** performance, the reported number is optimistically biased: you picked the configuration that happened to look best on those folds. Varma and Simon demonstrated this with "null" data in which the two classes were statistically identical: after tuning an SVM by cross-validation, the CV error estimate fell below 30% on 38% of the simulated datasets, while the tuned classifiers were no better than chance on independent test data [5]; Cawley and Talbot showed that over-fitting the model-selection criterion can be as large as the differences between learning algorithms [6].

Nested CV fixes this with two loops. The outer loop holds out a fold for evaluation. Inside each outer training set, an inner CV selects hyperparameters. The outer fold is touched only to score the configuration the inner loop chose. In scikit-learn this is simply a `GridSearchCV` placed inside `cross_val_score`. The result estimates the performance of the whole *procedure* (including tuning), which is what you will run when you retrain.

For deep networks a full nested CV is often too expensive. The usual compromise is a single fixed train/validation/test split, a stated tuning budget for every method compared, and a test set that is opened once.

Metrics: a recap

Plain version. Decide what an error costs before choosing a number. One scalar rarely tells the whole story.

Task	Useful measurements	Common trap
Classification	Per-class precision/recall, macro-F1, balanced accuracy, AUROC/AUPRC	Accuracy hides rare failures; PR curves depend on prevalence
Detection/segmentation	AP, IoU/Dice, boundary errors	Comparing different IoU thresholds or protocols
Retrieval	Recall@k, mAP under a defined relevance set	Gallery duplicates inflate matches
Restoration	PSNR/SSIM, perceptual metrics, consistency with the observation	Plausible texture may not be true detail
Generation	FID/KID, coverage, conditional alignment, human study	Feature realism does not establish prompt fidelity
Video/geometry	Temporal consistency, task accuracy, geometric error	Good individual frames can hide inconsistent structure

The definitions and pitfalls of classification metrics (confusion matrix, top-k, ROC/PR curves) are worked through in DLCV 02, and those of detection (IoU, AP, COCO mAP) in DLCV 03. For

generative models, FID compares Gaussian fits (means and covariances) of feature distributions; KID instead uses a kernel maximum-mean-discrepancy estimate that is unbiased, which makes it better behaved with small sample counts [7]. Neither is a universal measure of human preference. Always state the feature extractor, reference set, sample count, resolution and sampling settings.

Two classical facts are worth remembering. With 1% positives, an always-negative classifier gets 99% accuracy and zero recall, so accuracy alone is meaningless for rare events. And precision depends on prevalence: the same detector has lower precision in a population where the event is rarer, while its recall and false-positive rate do not change.

Statistical significance and confidence intervals

Plain version. A test score is computed from a finite sample, so it would come out a little different on another sample. Report a range, not just a point, and compare two models on the *same* examples.

Standard error of an accuracy

If the n test examples are independent and the model's true accuracy is p, the number of correct predictions is binomial, and the observed accuracy $\hat{p}$ has standard error

$$\mathrm{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}, \qquad 95\%\ \text{interval} \approx \hat{p} \pm 1.96\,\mathrm{SE}.$$

Here $\hat{p}$ is the fraction correct and 1.96 is the 97.5th percentile of the standard normal distribution. For $\hat{p} = 0.972$ and $n = 540$, $\mathrm{SE} \approx 0.0071$, so the interval is about ± 1.4 percentage points. This normal approximation is poor when $\hat{p}$ is very close to 0 or 1 or n is small; Wilson or exact binomial intervals are better there. If test examples are grouped (several images per patient), the effective sample size is closer to the number of groups, and the interval must be computed by resampling groups.

The bootstrap

The **bootstrap** estimates the sampling distribution of any statistic by resampling the observed data with replacement [9]. Draw B resamples of size n, recompute the statistic on each, and take the 2.5th and 97.5th percentiles as an approximate 95% interval (the *percentile* interval). It works for metrics with no simple formula, such as macro-F1, AUROC or mAP.

To compare two models, use a **paired** bootstrap: resample example indices once, and evaluate both models on the same resample. Because the models usually fail on many of the same hard examples, pairing removes that shared variance and gives a much tighter interval for the difference than two separate intervals would.

McNemar's test

For two classifiers evaluated on the same test set, only the examples where they disagree carry information about which is better. Let n_{01} count examples model A gets right and B gets wrong, and n_{10} the reverse. Under the null hypothesis that both models have the same error rate, each discordant example is equally likely to fall either way, so $n_{10} \sim \text{Binomial}(n_{01} + n_{10}, \frac{1}{2})$. The exact McNemar test computes a two-sided binomial p-value from that. Dietterich compared five tests for comparing learning algorithms and found that, for algorithms that can be run only once (typical for expensive models), McNemar's test was the only one with acceptable type I error; when ten runs are affordable he recommended the 5×2 cross-validated t test [8].

Code: paired bootstrap and McNemar

```
import numpy as np
from scipy.stats import binomtest
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
a = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr)
b = make_pipeline(StandardScaler(), SVC(C=10, gamma="scale")).fit(Xtr, ytr)
ca, cb = a.predict(Xte) == yte, b.predict(Xte) == yte    # per-example correctness

# 1) Percentile bootstrap: resample test examples with replacement.
rng = np.random.default_rng(0)
idx = rng.integers(0, len(yte), size=(10_000, len(yte)))
diff = cb[idx].mean(axis=1) - ca[idx].mean(axis=1)       # paired: same resample
lo, hi = np.percentile(diff, [2.5, 97.5])
print(f"accuracy A={ca.mean():.4f}  B={cb.mean():.4f}  n={len(yte)}")
print(f"B - A = {cb.mean() - ca.mean():+.4f}, 95% bootstrap CI [{lo:+.4f}, {hi:+.4f}]")

# 2) Exact McNemar test: only the discordant examples carry information.
n01 = int(np.sum(ca & ~cb))      # A right, B wrong
n10 = int(np.sum(~ca & cb))      # A wrong, B right
p = binomtest(n10, n01 + n10, 0.5).pvalue
print(f"discordant pairs: A-only={n01}, B-only={n10}, exact McNemar p={p:.4f}")
```

Output:

```
accuracy A=0.9722  B=0.9852  n=540
B - A = +0.0130, 95% bootstrap CI [-0.0019, +0.0278]
discordant pairs: A-only=5, B-only=12, exact McNemar p=0.1435
```

The SVM is 1.3 points better, but the interval includes zero and McNemar's p-value is 0.14. Only 17 of 540 images separate the two models, and 12 versus 5 is not unusual under a fair coin. With this test set, the honest conclusion is "B is probably at least as good; the evidence that it is better is weak."

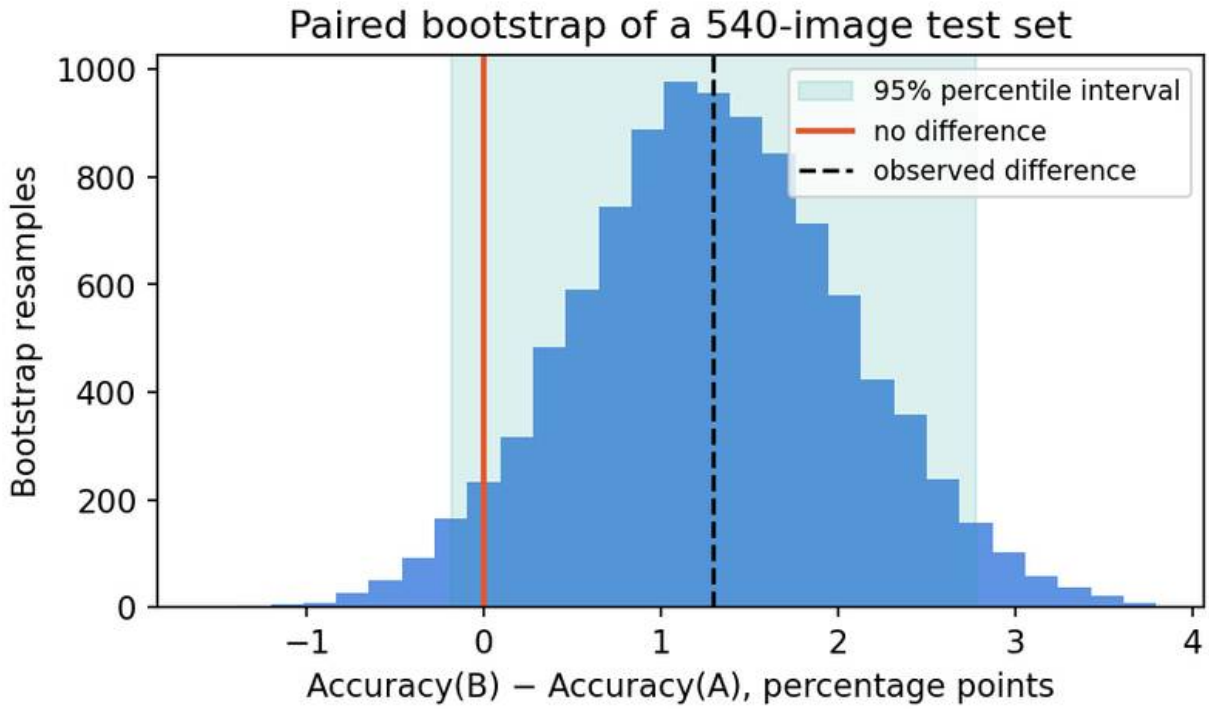


Figure 3 — The paired bootstrap distribution of Accuracy(B) − Accuracy(A) from the code above. The 95% interval crosses zero.

Other sources of variance

The test sample is not the only source of randomness. For a neural network, the random seed changes initialization, data order and augmentation; the data split itself is random; and some GPU kernels are nondeterministic. Bouthillier et al. modeled the whole benchmarking process and found that variance from data sampling, parameter initialization and hyperparameter choice markedly affects conclusions; they recommend randomizing as many of these sources as possible across repeated runs [10]. Practical rules:

- Run each configuration with several seeds (3–5 is common) and report mean and spread.
- Compare methods with paired tests on the same examples and splits.
- If you compare many models or many datasets, some will "win" by chance; adjust for multiple comparisons or confirm on a fresh set.
- Report effect sizes with intervals. A statistically significant difference can be practically irrelevant, and a large practical difference can be statistically unproven on a small set.

Calibration: knowing when you might be wrong

Plain version. A model's confidence should match how often it is right. Among all predictions made with 80% confidence, about 80% should be correct. Accuracy and calibration are separate properties.

Measuring probability quality

Let p_{ik} be the predicted probability of class k for example i, and $y_{ik} \in \{0, 1\}$ the one-hot label. Two **proper scoring rules** (scores minimized in expectation by the true probabilities) are:

$$\mathrm{NLL} = -\frac{1}{n}\sum_{i=1}^{n} \log p_{i,y_i}, \qquad \mathrm{Brier} = \frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} (p_{ik} - y_{ik})^2,$$

where y_i is the true class of example i and K is the number of classes. Some libraries divide the Brier score by K; say which convention you use.

The **expected calibration error** (ECE) groups predictions into M bins $B_1, \dots, B_M$ by their top-class confidence and compares average confidence with accuracy in each bin:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|,$$

where $\mathrm{acc}(B_m)$ is the fraction correct in bin m and $\mathrm{conf}(B_m)$ the mean confidence there. A **reliability diagram** plots acc against conf per bin; a perfectly calibrated model lies on the diagonal. ECE is not a proper scoring rule and depends on the bin count and the bin scheme, so report it alongside NLL or Brier and a reliability diagram.

Temperature scaling

Guo et al. observed that modern deep networks, which are much larger than those of earlier decades, are often overconfident, and that a single scalar fix works surprisingly well [11]. **Temperature scaling** divides the logits ℓ by $T > 0$ before the softmax:

$$\hat{p}_k = \frac{\exp(\ell_k / T)}{\sum_j \exp(\ell_j / T)}, \qquad T^\star = \arg\min_{T>0} \mathrm{NLL}_{\mathrm{cal}}(T).$$

$T > 1$ softens overconfident predictions; $T < 1$ sharpens underconfident ones. Because dividing all logits by the same positive number never changes which is largest, the predicted class (and so accuracy) is unchanged. T must be fitted on a held-out calibration set, never on the test set. Later work by Minderer et al. found that the most recent image classifiers, notably non-convolutional

ones such as Vision Transformers, are among the best calibrated, and that older trends such as calibration decaying under distribution shift are less pronounced for them [12].

Code: temperature scaling and what shift does to it

```
import numpy as np
from scipy.optimize import minimize_scalar
from scipy.special import log_softmax, softmax
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

X, y = load_digits(return_X_y=True)
X = X / 16.0
Xa, Xte, ya, yte = train_test_split(X, y, test_size=0.4, stratify=y, random_state=0)
Xtr, Xcal, ytr, ycal = train_test_split(Xa, ya, test_size=0.3, stratify=ya, random_state=0)
net = MLPClassifier(hidden_layer_sizes=(128,), alpha=1e-6, max_iter=300,
                    random_state=0).fit(Xtr, ytr)

def logits(X):                        # log-probabilities = logits up to a constant
    return np.log(np.clip(net.predict_proba(X), 1e-12, 1.0))

def ece(prob, y, bins=10):
    conf, pred = prob.max(1), prob.argmax(1)
    edges = np.linspace(0, 1, bins + 1)
    total = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            total += m.mean() * abs((pred[m] == y[m]).mean() - conf[m].mean())
    return total

def nll(T, z, y):
    return -log_softmax(z / T, axis=1)[np.arange(len(y)), y].mean()

T = minimize_scalar(nll, bounds=(0.05, 20), method="bounded",
                    args=(logits(Xcal), ycal)).x
print(f"fitted temperature T = {T:.2f}")
noise = np.random.default_rng(0).normal(size=Xte.shape)
for sigma in [0.0, 0.3, 0.6]:         # Gaussian pixel noise = a simple covariate shift
    Xs = np.clip(Xte + sigma * noise, 0, 1)
    z = logits(Xs)
    p_raw, p_ts = softmax(z, axis=1), softmax(z / T, axis=1)
    print(f"noise {sigma:.1f}: acc={(p_raw.argmax(1) == yte).mean():.3f}  "
          f"ECE raw={ece(p_raw, yte):.3f}  ECE scaled={ece(p_ts, yte):.3f}  "
          f"NLL raw={nll(1, z, yte):.3f}  NLL scaled={nll(T, z, yte):.3f}")
```

Output (scikit-learn may print a convergence warning, which is harmless here):

```
fitted temperature T = 1.19
noise 0.0: acc=0.972  ECE raw=0.007  ECE scaled=0.015  NLL raw=0.108  NLL scaled=0.110
```

```
noise 0.3: acc=0.794  ECE raw=0.088  ECE scaled=0.063  NLL raw=0.693  NLL scaled=0.639
noise 0.6: acc=0.435  ECE raw=0.380  ECE scaled=0.347  NLL raw=2.998  NLL scaled=2.598
```

Three observations. On clean data this small network is already well calibrated, and fitting T on 324 calibration images slightly *worsens* ECE: a fix fitted on a small set can add noise. Under pixel noise the model becomes badly overconfident: at $\sigma = 0.6$ it is right 43.5% of the time while its confidence suggests far more. Temperature scaling fitted on clean data helps only a little under shift. This matches the large study by Ovadia et al., who found that the calibration of many uncertainty methods deteriorates as shift increases [16].

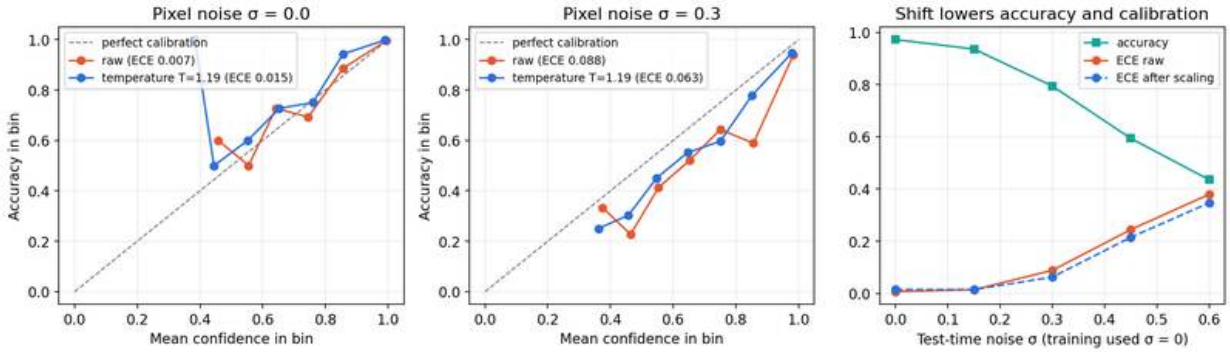


Figure 4 — Reliability diagrams and a shift sweep for the network above (same noise pattern, scaled by σ). Points below the diagonal mean overconfidence.

Robustness and distribution shift

Plain version. The world after deployment is rarely identical to the training set. Cameras change, lighting changes, populations change. A robust model degrades gracefully; a brittle one fails silently.

Three kinds of shift

Write the joint distribution as $p(x, y) = p(y \mid x)\,p(x) = p(x \mid y)\,p(y)$, with x the input and y the label. Shifts are named by what changes between training (p_{tr}) and deployment (p_{te}):

- **Covariate shift:** $p_{\text{te}}(x) \neq p_{\text{tr}}(x)$ while $p(y \mid x)$ is unchanged. New camera, new lighting, new scanner.
- **Label (prior) shift:** $p_{\text{te}}(y) \neq p_{\text{tr}}(y)$ while $p(x \mid y)$ is unchanged. A disease becomes more common during an outbreak.
- **Concept shift:** $p(y \mid x)$ itself changes. The definition of "spam" evolves.

Under pure covariate shift, the expected test loss can be rewritten as a weighted training loss, $\mathbb{E}_{\text{te}}[\ell] = \mathbb{E}_{\text{tr}}[w(x)\,\ell]$ with importance weight $w(x) = p_{\text{te}}(x)/p_{\text{tr}}(x)$. These weights are hard to estimate for images, but the identity explains why shift matters most where test inputs fall in regions with little training data. Under label shift, Lipton et al. showed that a fixed black-box classifier's confusion matrix on held-out training data, combined with its prediction frequencies on unlabeled test data, is enough to estimate the new class priors and re-weight the model [17].

Benchmarks that probe shift

- **ImageNet-C** applies 15 corruption types (noise, blur, weather, digital) at several severities to ImageNet validation images, and **ImageNet-P** adds perturbation sequences; together they measure robustness to common corruptions rather than adversarial attacks [13]. Our pixel-noise sweep is a miniature version.
- **ImageNetV2** re-created the ImageNet test set by following the original collection process as closely as possible. Accuracy dropped by 11–14% across many models, yet gains on the original test set translated into even larger gains on the new one, so the drop reflects slightly harder images rather than over-fitting to the reused test set [14]. Even a careful re-collection is a shift.
- **WILDS** gathers ten datasets with naturally occurring shifts, including tumor tissue from different hospitals, camera traps at new locations and satellite images from new countries, and reports a large gap between in-distribution and out-of-distribution performance that existing mitigation methods did not close [15].

The practical lesson: always add at least one *shifted* evaluation set built from the change you expect in deployment (new site, new device, later time), and report in-distribution and shifted results side by side.

Efficiency: FLOPs, memory and latency

Plain version. "Fast" and "small" each have several meanings. A model with fewer operations is not automatically quicker, and a model with fewer parameters does not automatically use less memory. Measure on the target device.

Counting operations

One **multiply–accumulate** (MAC) is a multiplication plus an addition, so 1 MAC $\approx$ 2 floating-point operations (FLOPs). For a fully connected layer with d_{in} inputs and d_{out} outputs, and a convolution with C_{in} input channels, C_{out} output channels, a $k \times k$ kernel and an $H' \times W'$ output:

$$\text{FLOPs}_{\text{linear}} \approx 2\, d_{\text{in}} d_{\text{out}}, \qquad \text{FLOPs}_{\text{conv}} \approx 2\, C_{\text{in}} k^2 C_{\text{out}} H' W'$$

per example (bias and activation costs are small). Libraries are inconsistent about whether they report MACs or FLOPs, so state which.

Why FLOPs do not equal speed

Dehghani et al. showed that common cost indicators (parameter count, FLOPs, throughput, latency) can disagree with each other and argued that reporting only one of them gives a misleading picture of efficiency [18]. The main reasons:

- **Memory traffic.** A layer whose weights must be read from memory for very little arithmetic is limited by memory bandwidth, not compute. A linear layer applied to a single example reads $4d_{\text{in}}d_{\text{out}}$ bytes of fp32 weights to perform $2d_{\text{in}}d_{\text{out}}$ FLOPs: 0.5 FLOPs per byte, far too little to keep a modern accelerator busy. Batching raises this ratio (called *arithmetic intensity*).
- **Parallelism.** Depthwise convolutions and many small layers have few FLOPs but use the hardware poorly.
- **Kernel support.** An operation without an optimized kernel on the target runtime may be slow regardless of its FLOP count.

Memory accounting

At inference, memory holds weights (parameters × bytes per value), activations of the layers currently alive, workspace for kernels, and for autoregressive Transformers a **KV cache** that grows with context length and batch size. During training add gradients and optimizer states: Adam keeps two extra values per parameter, so an fp32 model with Adam needs roughly 4× its weight memory before counting activations.

A latency protocol

Wall-clock latency depends on hardware, runtime, batch size, input resolution, numeric precision and thread settings, and the first calls are slower because of one-off initialization. A sound protocol:

1. Fix and report device, runtime version, precision, batch size, input size and thread count.
2. Run warm-up iterations and discard them.
3. Time many runs and report percentiles (p50 and p90 or p99), not just the mean: users experience the tail.
4. On GPUs, synchronize the device before stopping the timer, because kernels are launched asynchronously.
5. Measure end to end (pre-processing, transfers, decoding) as well as the network alone.

A **Pareto frontier** contains the candidates for which no other candidate is at least as good on every axis (accuracy, latency, memory) and strictly better on one. Presenting the frontier makes trade-offs visible without inventing a single "best model".

Deployment: export, compression and runtimes

Plain version. To ship a model, you freeze it into a portable file, often make it smaller or cheaper, and run it with an inference engine designed for the target hardware. Each step can change the outputs, so each step needs re-evaluation.

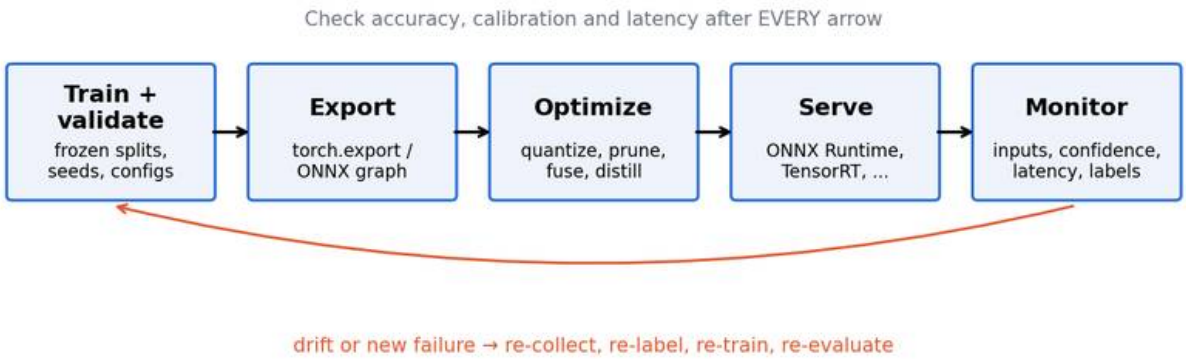


Figure 5 — A deployment loop. Accuracy, calibration and latency should be re-checked after every step, because export, compression and runtimes all change numerics.

Export and inference runtimes

Training code is written in a flexible, Python-driven framework. Deployment prefers a static description of the computation:

- `torch.export` traces a PyTorch module ahead of time into an `ExportedProgram`, a graph of tensor operations that can be serialized and executed with different inputs without the original Python class [19].
- **ONNX** (Open Neural Network Exchange) is an open standard file format for ML models, so a model trained in one framework can be run by tools from others [20].
- **ONNX Runtime** is a cross-platform inference engine for ONNX models. It partitions the graph into subgraphs and hands each to a hardware-specific *execution provider* (for example CUDA, TensorRT, OpenVINO or CoreML) [21].
- **NVIDIA TensorRT** is an SDK that compiles models (from frameworks or ONNX) into optimized engines for NVIDIA GPUs, using layer fusion and reduced precisions such as FP16, BF16, FP8 and INT8 [22].

These runtimes gain speed by fusing operations, choosing fast kernels for the exact shapes and lowering precision. The same model can therefore give slightly different outputs on different runtimes. Compare exported and original outputs on a validation batch and re-measure accuracy after conversion.

Quantization

Quantization stores and computes with fewer bits. In the common **affine (asymmetric) uniform** scheme, a real value x is mapped to an integer q and back:

$$q = \mathrm{clip}\Big(\mathrm{round}\big(\tfrac{x}{s}\big) + z,\ 0,\ 2^b - 1\Big), \qquad \hat{x} = s\,(q - z),$$

where b is the bit width, $s > 0$ the **scale** (the step between representable values), z the integer **zero point** chosen so that the real value 0 is represented exactly, and $\hat{x}$ the dequantized approximation. For a range $[x_{\min}, x_{\max}]$ that includes 0, $s = (x_{\max} - x_{\min})/(2^b - 1)$ and $z = \mathrm{round}(-x_{\min}/s)$. The *symmetric* variant fixes $z = 0$ and uses a signed range. The integer-arithmetic formulation of Jacob et al. showed how to run whole networks with 8-bit integer matrix multiplication and how to train for it [23].

Error comes from two sources: **rounding** inside the range (at most $s/2$ per value) and **clipping** outside it. Both depend on the **granularity** of the scale: one scale per tensor, per output channel, or per small group of weights. Nagel et al. give a practical guide covering these choices, *post-training quantization* (PTQ: calibrate ranges on a small representative dataset after training) and *quantization-aware training* (QAT: simulate rounding during training so the weights adapt) [24]. Gholami et al. survey the wider design space [25].

```
import numpy as np

def quantize(x, bits=8, axis=None):
    """Asymmetric (affine) uniform quantization, per tensor or per channel."""
    qmin, qmax = 0, 2**bits - 1
    lo = x.min(axis=axis, keepdims=True) if axis is not None else x.min()
    hi = x.max(axis=axis, keepdims=True) if axis is not None else x.max()
    lo, hi = np.minimum(lo, 0), np.maximum(hi, 0)      # keep 0 exactly representable
    s = (hi - lo) / (qmax - qmin)                       # scale
    z = np.round(qmin - lo / s)                         # zero point (integer)
    q = np.clip(np.round(x / s) + z, qmin, qmax)        # stored integers
    return s * (q - z)                                  # dequantized values

rng = np.random.default_rng(0)
# 64 output channels whose weight magnitudes differ by up to 100x
W = rng.normal(size=(64, 256)) * np.logspace(-2, 0, 64)[:, None]
for bits in [8, 4]:
    e_t = np.abs(quantize(W, bits) - W).mean()
    e_c = np.abs(quantize(W, bits, axis=1) - W).mean()
```

```
    print(f"{bits}-bit mean |error|: per-tensor {e_t:.5f}  per-channel {e_c:.5f}")
print("zero maps back to:", quantize(np.array([-1.0, 0.0, 3.0]))[1])
```

Output:

```
8-bit mean |error|: per-tensor 0.00632  per-channel 0.00124
4-bit mean |error|: per-tensor 0.06718  per-channel 0.02112
zero maps back to: 0.0
```

When channels have very different ranges, one shared scale wastes most of the grid on the largest channel and rounds small channels coarsely. Per-channel scales cut the error about fivefold at 8 bits here. In large language models, a few outlier activation channels cause the same problem at a larger scale, which is what methods such as GPTQ (layer-wise error-compensating weight rounding) and AWQ (protecting salient weight channels identified from activations) address for 3–4-bit weight-only quantization [42], [43].

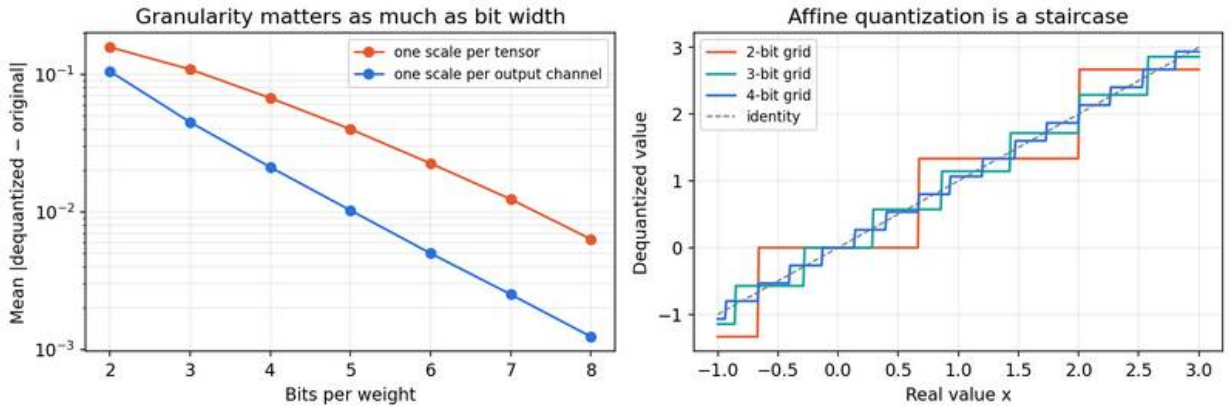


Figure 6 — Quantization error falls roughly by half for each extra bit, and the choice of scale granularity shifts the whole curve.

Pruning

Pruning removes weights, channels, attention heads or blocks that contribute little. *Unstructured* pruning zeros individual weights; it can remove a large fraction of parameters, as in Deep Compression, which combined pruning, weight sharing via quantization, and Huffman coding to shrink networks substantially [26]. But scattered zeros save time only if the runtime has sparse kernels for that pattern. *Structured* pruning removes whole channels or heads, producing smaller dense tensors that any runtime can exploit. Blalock et al. reviewed 81 pruning papers and found that inconsistent baselines and metrics made it hard to tell which methods actually work, and proposed a standardized benchmark [27]. Hoefler et al. survey sparsity in training and inference in depth [28].

Knowledge distillation

Distillation trains a small *student* to imitate a large *teacher*. Hinton et al. proposed matching the teacher's softened output distribution [29]. With temperature T, logits ℓ^t (teacher) and ℓ^s (student), softened probabilities $p^{(T)} = \mathrm{softmax}(\ell/T)$, and weight $\lambda \in [0, 1]$:

$$\mathcal{L} = (1 - \lambda)\, \mathcal{L}_{\mathrm{CE}}(y,\, p^{s,(1)}) + \lambda\, T^2\, D_{\mathrm{KL}}(p^{t,(T)} \,\|\, p^{s,(T)}).$$

The softened teacher distribution carries "dark knowledge" about which wrong classes are similar. The gradients of the soft term scale as $1/T^2$, so multiplying by T^2 keeps the two terms balanced when T changes [29]. Gou et al. organize later work by *what* is transferred (responses, intermediate features, relations between examples) and *how* (offline, online, self-distillation) [30]. A student inherits the teacher's mistakes and biases as well as its skill.

Code: weight-only int8, latency and export

This sketch stores the weights of a small MLP as int8 with one scale per output channel and measures size and latency on the CPU. It uses only core PyTorch.

```
import io, time
import numpy as np
import torch
from torch import nn

torch.manual_seed(0)
model = nn.Sequential(nn.Linear(512, 2048), nn.ReLU(),
                      nn.Linear(2048, 2048), nn.ReLU(),
                      nn.Linear(2048, 10)).eval()
x = torch.randn(32, 512)

class Int8WeightLinear(nn.Module):
    """Weight-only int8 storage, symmetric per output channel."""
    def __init__(self, lin):
        super().__init__()
        w = lin.weight.detach()
        scale = w.abs().amax(dim=1, keepdim=True) / 127
        self.register_buffer("q", torch.round(w / scale).to(torch.int8))
        self.register_buffer("scale", scale)
        self.register_buffer("bias", lin.bias.detach().clone())
    def forward(self, x):                       # dequantize, then fp32 matmul
        return nn.functional.linear(x, self.q.float() * self.scale, self.bias)

int8 = nn.Sequential(*[Int8WeightLinear(m) if isinstance(m, nn.Linear) else m
                       for m in model]).eval()

def size_mb(m):
    buf = io.BytesIO()
    torch.save(m.state_dict(), buf)
    return buf.getbuffer().nbytes / 1e6
```

```
def latency_ms(m, x, warmup=10, runs=100):
    with torch.inference_mode():
        for _ in range(warmup):                # first calls pay one-off costs
            m(x)
        times = []
        for _ in range(runs):
            t0 = time.perf_counter()
            m(x)
            times.append((time.perf_counter() - t0) * 1e3)
    return np.percentile(times, [50, 90]).round(2)

with torch.inference_mode():
    gap = (model(x) - int8(x)).abs().max().item()
    agree = (model(x).argmax(1) == int8(x).argmax(1)).float().mean().item()
print(f"fp32 file: {size_mb(model):.1f} MB  p50/p90 latency {latency_ms(model, x)} ms")
print(f"int8 file: {size_mb(int8):.1f} MB  p50/p90 latency {latency_ms(int8, x)} ms")
print(f"max |logit difference| = {gap:.4f}, top-1 agreement = {agree:.3f}")

# Ahead-of-time graph capture: a portable program, independent of this Python class.
exported = torch.export.export(model, (x,))
ops = [n.target for n in exported.graph.nodes if n.op == "call_function"]
print(type(exported).__name__, "->", [str(o).split(".")[1] for o in ops])
```

One CPU run (four threads) printed:

```
fp32 file: 21.1 MB  p50/p90 latency [2.74 4.49] ms
int8 file: 5.3 MB  p50/p90 latency [ 9.23 11.06] ms
max |logit difference| = 0.0018, top-1 agreement = 1.000
ExportedProgram -> ['linear', 'relu', 'linear', 'relu', 'linear']
```

Your latencies will differ; the pattern is the point. The file is four times smaller and the outputs barely change. Yet the int8 version is *slower*, because it converts the weights back to fp32 on every call and then runs the same fp32 matrix multiplication. Lower precision becomes faster only when the runtime has integer (or low-precision) kernels for that operation, or when the workload is limited by memory bandwidth so that reading fewer bytes dominates. This is exactly the gap between "compressed" and "accelerated" that runtimes such as TensorRT, ONNX Runtime and PyTorch's own `torchao` library [46] are built to close. The FLOP count of this MLP is $2 \times (512 \cdot 2048 + 2048 \cdot 2048 + 2048 \cdot 10) \approx 10.5$ MFLOPs per example, identical for both versions.

Monitoring after deployment

Plain version. A model can get worse without any code changing, because the data changes. Watch the inputs and outputs, collect fresh labels when possible, and decide in advance what triggers an investigation.

Sculley et al. describe how ML systems accumulate hidden "technical debt": entangled features, undeclared consumers of predictions, feedback loops where the model's outputs influence its future training data, and dependence on external data that changes without notice [33]. Paleyes et al. collected published deployment case studies and found problems at every stage of the workflow [39]. Useful signals, from cheapest to most informative:

- **System health:** latency percentiles, error rates, memory, throughput.
- **Input statistics:** image brightness, resolution, class of device, missing fields; compared with a reference window from training time.
- **Output statistics:** distribution of predicted classes and of confidence. These need no labels.
- **Delayed labels:** a small, regularly labelled sample of live traffic gives the only direct estimate of accuracy.

To compare a live window with the reference, a two-sample test is a simple starting point. The

Kolmogorov–Smirnov statistic for one-dimensional samples with empirical cumulative distribution functions F_{ref} and F_{live} is

$$D = \sup_t |F_{\text{ref}}(t) - F_{\text{live}}(t)|,$$

the largest vertical gap between the two curves. Rabanser et al. compared shift detectors and found that a two-sample test applied after reducing the inputs with a pre-trained classifier (for example to its softmax outputs) performed best across the shifts they studied [32].

```
import numpy as np
from scipy.stats import ks_2samp
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X, y = load_digits(return_X_y=True)
X = X / 16.0
Xtr, Xlive, ytr, ylive = train_test_split(X, y, test_size=0.5, random_state=0)
Xtr, Xref, ytr = Xtr[:600], Xtr[600:], ytr[:600]   # reference window from training time
clf = LogisticRegression(max_iter=2000).fit(Xtr, ytr)
ref_conf = clf.predict_proba(Xref).max(1)     # label-free signal: top-class confidence

rng = np.random.default_rng(0)
for name, noise in [("week 1 (clean)", 0.0), ("week 2 (new camera)", 0.25)]:
    batch = rng.choice(len(Xlive), 200, replace=False)
    Xb = np.clip(Xlive[batch] + noise * rng.normal(size=Xlive[batch].shape), 0, 1)
    conf = clf.predict_proba(Xb).max(1)
    stat, p = ks_2samp(ref_conf, conf)
    acc = (clf.predict(Xb) == ylive[batch]).mean()   # usually NOT available live
    print(f"{name}: mean confidence {conf.mean():.3f}, KS={stat:.3f}, p={p:.2g}, "
          f"(hidden) accuracy {acc:.3f}")
```

Output:

```
week 1 (clean): mean confidence 0.846, KS=0.089, p=0.28, (hidden) accuracy 0.950
week 2 (new camera): mean confidence 0.757, KS=0.277, p=1.3e-08, (hidden) accuracy 0.885
```

The confidence distribution flags week 2 without a single label, and the hidden accuracy confirms that something real happened. A detector says *that* the data changed, not *whether* accuracy fell: harmless shifts also trigger it, and with large windows even tiny changes become "significant". Use it to trigger investigation and labelling, not automatic retraining.

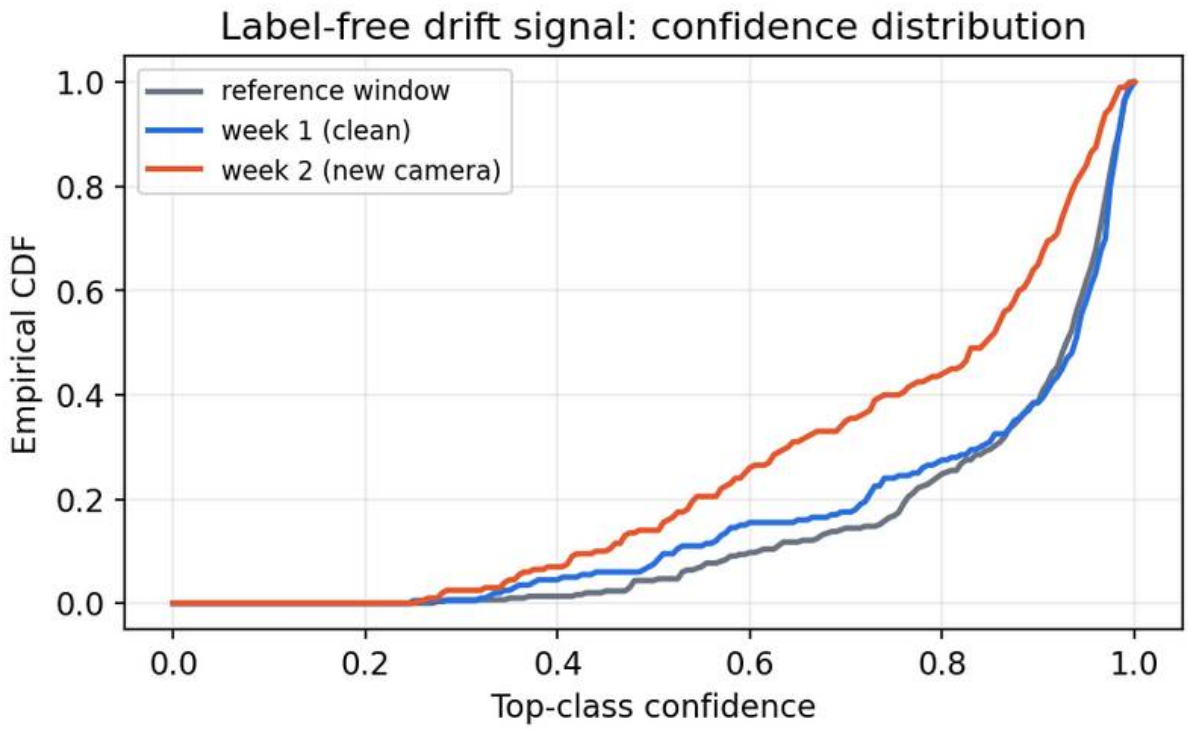


Figure 7 — The Kolmogorov–Smirnov statistic is the largest vertical gap between the reference curve and a live window's curve.

Reproducibility and documentation

Plain version. Someone else (including you in six months) should be able to rerun your experiment and get the same numbers, and a reader should be able to tell what the model is for and where it should not be used.

Making runs repeatable

Fix and record random seeds for every library, but know their limits: the PyTorch documentation states that completely reproducible results are not guaranteed across releases, commits or platforms, and recommends seeding, `torch.use_deterministic_algorithms(True)` and disabling cuDNN benchmarking when bit-exact repetition matters [34]. Beyond seeds, record:

- the dataset version, split IDs (the actual lists of example IDs) and deduplication rule;
- preprocessing and augmentation code, pretrained checkpoint and its source;
- the full configuration (optimizer, schedule, stopping rule), software environment and hardware;
- every validation decision made, and the single final test evaluation.

The NeurIPS 2019 reproducibility program, reported by Pineau et al., introduced a code-submission policy, a community reproducibility challenge and a checklist covering exactly these items [37].

Documenting models and data

Model cards are short documents that accompany a trained model and report its intended use, the evaluation data, performance broken down by relevant groups and conditions, and known limitations [35]. **Datasheets for datasets** ask dataset creators to document motivation, composition, collection process, preprocessing, recommended uses and maintenance [36]. Both make the evaluation decisions in this chapter visible to the people who will rely on them.

A capstone experiment

Choose one task with a modest, legally usable dataset. Define the deployment population and the split unit. Compare three candidates: a classical baseline, frozen pretrained features with a small head, and one fine-tuned network. For each, report the test metric with a bootstrap interval, a paired comparison against the best baseline, a reliability diagram, results on one shifted set, and latency percentiles on the target device. Write a one-page model card. Open the final test set once; if it reveals a failure and you revise the model, treat the old result as development evidence and obtain a fresh evaluation for the revised claim.

Modern view

The engineering of evaluation and deployment is now a research field in its own right. Several surveys and reviews map it.

Leakage and reproducibility. Kapoor and Narayanan's review of ML-based science documents leakage in 329 papers across 17 fields and proposes "model info sheets" that make authors argue

why their train/test separation is valid [4]. Combined with the variance study of Bouthillier et al. [10] and the NeurIPS reproducibility program [37], the consensus is that most over-optimistic results come from protocol errors, not from exotic statistics.

Uncertainty and calibration. Gawlikowski et al. survey uncertainty estimation in deep networks: they separate reducible model (epistemic) uncertainty from irreducible data (aleatoric) uncertainty, organize methods into deterministic networks, Bayesian networks, ensembles and test-time augmentation, and review calibration methods and their practical limits [38]. Ovadia et al. add the empirical warning that post-hoc calibration falls short under shift while methods that marginalize over models, such as ensembles, hold up better [16].

Deployment in practice. Paleyes et al. survey published case studies, map the challenges onto the stages of the deployment workflow and show that practitioners hit problems at every stage [39], extending the technical-debt view of Sculley et al. [33].

Compression. Cheng et al. give the classic overview of pruning, low-rank factorization, compact convolutions and distillation for CNNs [31]; Gholami et al. [25] and Nagel et al. [24] focus on quantization; Hoefler et al. on sparsity [28]; Gou et al. on distillation [30]. For large language models, Zhu et al. survey quantization, pruning, distillation and low-rank methods specific to models with billions of parameters [41]. A visible 2023–2026 trend is that weight-only quantization to 4 bits (GPTQ [42], AWQ [43]) and low-precision formats such as FP8 have moved from papers into production toolkits: TensorRT lists FP8, INT8, FP4 and INT4 among its supported precisions [22], and PyTorch's quantization tooling is consolidating in the `torchao` library [46].

Holistic evaluation. For foundation models, HELM evaluates many models on many scenarios and on several metrics at once (accuracy, calibration, robustness, fairness, bias, toxicity and efficiency) and argues that a single leaderboard number hides these trade-offs [40]. The same multi-metric habit applies to vision models.

Scaling. Scaling laws fit empirical relationships among model size, data, compute and loss. Hoffmann et al. found that, for a fixed compute budget, language models should scale parameters and training tokens roughly equally [44]; Zhai et al. studied how ViT accuracy scales with model size, data and compute [45]. Fitted constants from one domain should not be copied into another; the transferable lesson is to report the compute and data behind every result.

Open problems. Evaluation sets leaking into web-scale pretraining data (contamination) is hard to rule out; calibration under unknown shift remains unsolved; latency claims rarely transfer between hardware generations; and evaluation of generative and interactive systems (see DLCV 07) still lacks agreed protocols.

Key takeaways

- A baseline and a clean split decide whether a result means anything. Split by the unit that will be new in deployment, and fit all preprocessing inside the training fold.

- Cross-validation gives a more stable estimate on small data; use group or time-ordered folds when examples are correlated, and nested CV when tuning and reporting from the same data.
- Every test score needs an uncertainty. Compare models on the same examples with a paired bootstrap or McNemar's test.
- Accuracy, calibration and robustness are different properties. Temperature scaling fixes in-distribution overconfidence but not shift-induced overconfidence.
- FLOPs, parameters, memory and latency can disagree. Measure latency on the target hardware with warm-up and percentiles.
- Quantization, pruning and distillation save resources only when the runtime exploits them; re-evaluate accuracy after every export or compression step.
- Monitor inputs, outputs and a trickle of fresh labels after deployment, and document models and data so others can reproduce and judge them.

Exercises

1. **Leakage audit.** A colleague fits PCA on all 1,797 digits, then splits into train and test and reports test accuracy. Explain the leak, and why it exists even though PCA never sees labels. How would you fix the code?

   Hint

   The principal directions are computed using the test images, so the representation is tuned to the test distribution. Put `PCA` inside a `Pipeline` so it is fitted only on each training fold. The bias is often small for PCA, but for supervised preprocessing (feature selection using labels) it can be large.

2. **Rare positives.** A dataset has 1% positive images. What accuracy, positive recall and balanced accuracy does an always-negative classifier achieve? Which metric would you report to a doctor, and why?

   Hint

   Accuracy 99%, recall 0, balanced accuracy 0.5 (mean of recall 0 on positives and 1 on negatives). Report recall (sensitivity) and specificity or precision at a clinically chosen operating point, plus their intervals, because the cost of a missed positive is what matters.

3. **How big must the test set be?** You expect accuracy around 90% and want a 95% confidence interval of about ± 1 percentage point. Using the normal approximation, roughly how many independent test examples do you need? What changes if the images come from 50 patients?

   Hint

   Solve $1.96\sqrt{0.9 \cdot 0.1/n} = 0.01$, giving $n \approx 3{,}460$. With 50 patients the examples are not independent; the effective sample size is closer to 50 than to the number of images, so the interval will be much wider. Use a bootstrap that resamples patients.

4. **Temperature by hand.** A three-class model outputs logits $(4, 1, 0)$. Compute the top-class probability with $T = 1$ and $T = 2$. Does the predicted class change? When would a fitted T be less than 1?

Hint

$T = 1$: $e^4/(e^4 + e^1 + e^0) \approx 0.936$. $T = 2$: $e^2/(e^2 + e^{0.5} + 1) \approx 0.736$. The argmax is unchanged. $T < 1$ is fitted when the model is underconfident, for example after heavy regularization or label smoothing.

5. **Compression claim.** A pruned model has half the parameters of the original but identical latency on your GPU. Give three possible explanations and one experiment to tell them apart.

Hint

Unstructured zeros are still multiplied by dense kernels; the remaining cost is dominated by activations, memory traffic or unchanged layers; or the batch is so small that kernel-launch overhead dominates. Profile per-layer time, and compare with a structured-pruned model whose tensors are actually smaller.

6. **Design a monitor.** Your defect detector runs on a factory line; labels arrive two weeks late from manual inspection. Design a monitoring plan: what you log, which statistics you compare with which reference, what threshold triggers an alert, and what happens after an alert.

Hint

Log latency, image statistics (brightness, blur), predicted-defect rate and confidence histograms daily; compare with a reference window from validation using KS or a population-stability measure; alert on sustained change rather than single days. After an alert, label a sample immediately, compute accuracy with an interval, and only then decide whether to recalibrate, fine-tune or roll back.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 11 “Practical Methodology”. book
2. G. James, D. Witten, T. Hastie, R. Tibshirani and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*, Springer, 2023. book
3. scikit-learn developers, “Common pitfalls and recommended practices,” scikit-learn documentation. docs
4. S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, 2023. doi
5. S. Varma and R. Simon, “Bias in error estimation when using cross-validation for model selection,” *BMC Bioinformatics*, vol. 7, art. 91, 2006. doi

6. G. C. Cawley and N. L. C. Talbot, “On over-fitting in model selection and subsequent selection bias in performance evaluation,” *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010. paper
7. M. Bińkowski, D. J. Sutherland, M. Arbel and A. Gretton, “Demystifying MMD GANs,” ICLR, 2018. arXiv
8. T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” *Neural Computation*, vol. 10, no. 7, 1998. doi
9. B. Efron, “Bootstrap methods: another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, 1979. doi
10. X. Bouthillier et al., “Accounting for variance in machine learning benchmarks,” MLSys, 2021. arXiv
11. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On calibration of modern neural networks,” ICML, 2017. arXiv
12. M. Minderer et al., “Revisiting the calibration of modern neural networks,” NeurIPS, 2021. arXiv
13. D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” ICLR, 2019. arXiv
14. B. Recht, R. Roelofs, L. Schmidt and V. Shankar, “Do ImageNet classifiers generalize to ImageNet?,” ICML, 2019. paper
15. P. W. Koh et al., “WILDS: A benchmark of in-the-wild distribution shifts,” ICML, 2021. paper
16. Y. Ovadia et al., “Can you trust your model’s uncertainty? Evaluating predictive uncertainty under dataset shift,” NeurIPS, 2019. arXiv
17. Z. C. Lipton, Y.-X. Wang and A. Smola, “Detecting and correcting for label shift with black box predictors,” ICML, 2018. arXiv
18. M. Dehghani, A. Arnab, L. Beyer, A. Vaswani and Y. Tay, “The efficiency misnomer,” ICLR, 2022. arXiv
19. PyTorch, “torch.export,” PyTorch documentation. docs
20. ONNX, “Open Neural Network Exchange: the open standard for machine learning interoperability.” site
21. ONNX Runtime, “Welcome to ONNX Runtime,” documentation. docs
22. NVIDIA, “NVIDIA TensorRT documentation.” docs
23. B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” CVPR, 2018. doi
24. M. Nagel et al., “A white paper on neural network quantization,” arXiv:2106.08295, 2021. arXiv
25. A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” book chapter in *Low-Power Computer Vision*; arXiv:2103.13630, 2021. arXiv
26. S. Han, H. Mao and W. J. Dally, “Deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding,” ICLR, 2016. arXiv
27. D. Blalock, J. J. Gonzalez Ortiz, J. Frankle and J. Guttag, “What is the state of neural network pruning?,” MLSys, 2020. arXiv
28. T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden and A. Peste, “Sparsity in deep learning: pruning and growth for efficient inference and training in neural networks,” *Journal of Machine Learning Research*, vol. 22, 2021. arXiv

29. G. Hinton, O. Vinyals and J. Dean, “Distilling the knowledge in a neural network,” NIPS Deep Learning Workshop, 2014; arXiv:1503.02531. arXiv
30. J. Gou, B. Yu, S. J. Maybank and D. Tao, “Knowledge distillation: a survey,” *International Journal of Computer Vision*, vol. 129, 2021. doi
31. Y. Cheng, D. Wang, P. Zhou and T. Zhang, “Model compression and acceleration for deep neural networks: the principles, progress, and challenges,” *IEEE Signal Processing Magazine*, vol. 35, no. 1, 2018. doi arXiv
32. S. Rabanser, S. Günnemann and Z. C. Lipton, “Failing loudly: an empirical study of methods for detecting dataset shift,” NeurIPS, 2019. arXiv
33. D. Sculley et al., “Hidden technical debt in machine learning systems,” NeurIPS, 2015. paper
34. PyTorch, “Reproducibility,” PyTorch documentation. docs
35. M. Mitchell et al., “Model cards for model reporting,” ACM FAT*, 2019. arXiv
36. T. Gebru et al., “Datasheets for datasets,” *Communications of the ACM*, vol. 64, no. 12, 2021. doi
37. J. Pineau et al., “Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program),” *Journal of Machine Learning Research*, vol. 22, 2021. paper
38. J. Gawlikowski et al., “A survey of uncertainty in deep neural networks,” *Artificial Intelligence Review*, vol. 56, 2023. doi
39. A. Paleyes, R.-G. Urma and N. D. Lawrence, “Challenges in deploying machine learning: a survey of case studies,” *ACM Computing Surveys*, vol. 55, no. 6, 2022. doi
40. P. Liang et al., “Holistic evaluation of language models,” *Transactions on Machine Learning Research*, 2023. arXiv
41. X. Zhu, J. Li, Y. Liu, C. Ma and W. Wang, “A survey on model compression for large language models,” *Transactions of the Association for Computational Linguistics*, vol. 12, 2024. doi
42. E. Frantar, S. Ashkboos, T. Hoefler and D. Alistarh, “GPTQ: accurate post-training quantization for generative pre-trained transformers,” ICLR, 2023. arXiv
43. J. Lin et al., “AWQ: activation-aware weight quantization for LLM compression and acceleration,” MLSys, 2024. arXiv
44. J. Hoffmann et al., “Training compute-optimal large language models,” arXiv:2203.15556, 2022; NeurIPS 2022 version titled “An empirical analysis of compute-optimal large language model training.” arXiv
45. X. Zhai, A. Kolesnikov, N. Houlsby and L. Beyer, “Scaling vision transformers,” CVPR, 2022. doi
46. PyTorch, “torchao documentation.” docs

Part II

Deep Learning for Computer Vision

DLCV 01 • Deep Learning for Image Processing and Computer Vision

How deep learning reorganizes image processing, image analysis and computer vision: from the classical pattern-recognition pipeline to CNNs, Vision Transformers, self-supervised backbones and promptable foundation models.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-01-deep-learning-for-image-processing/

What you'll learn

- How the classical three levels of image work (image processing, image analysis, computer vision) map onto the two families that deep learning actually uses: **low-level** and **high-level** vision tasks.
- Why the textbook pattern-recognition pipeline (sensor → features → classifier → evaluation) was replaced by **end-to-end learning**, and how it quietly came back as "pretrained backbone + light head".
- How a convolutional layer is a bank of **learned spatial filters**, and how kernel size, stride, padding and pooling set the **receptive field** and the feature hierarchy.
- The design reasons behind the landmark CNNs (LeNet-5, AlexNet, VGG, GoogLeNet, ResNet) and the encoder–decoder (FCN, U-Net) used for dense prediction.
- How the **Vision Transformer** works, why it needs large-scale pretraining, and how Swin, ConvNeXt and self-supervised backbones (DINO, MAE, DINOv2/v3) answered it.
- A map of today's tasks and representative models: classification, detection, segmentation (the SAM family), restoration and enhancement, generation (diffusion, latent diffusion, DiT) and vision–language models (CLIP, LLaVA).

The big picture

This tutorial is the entry point of the deep-learning track. It does not go deep into any one task. Instead it gives you the map: what kinds of image problems exist, which network designs solve them, and why those designs look the way they do. The later tutorials then zoom in on image classification, object detection, image restoration, low-light enhancement and object tracking.

Explain like I'm five

Think of a photo lab with two kinds of workers. The **repair crew** takes a scratched, blurry or dark photo and hands back a cleaner photo: picture in, picture out. The **detective** looks at a photo and tells you *what* is in it: "a cat, sitting on a sofa, on the left." Old-style computer vision gave both workers a fixed toolbox designed by experts. Deep learning lets them build their own tools by looking at millions of examples. Today there is even a "super-worker" who learned from billions of pictures and can be told what to do with a few words.

Why it matters: almost every modern camera pipeline, medical-imaging product, driver-assistance system and photo app is built from the pieces in this tutorial. Knowing the map lets you pick a sensible starting point for a new problem, read papers critically, and understand which classical ideas from the Digital Image Processing series survive inside the networks (many do).

About this tutorial

This tutorial is expanded from the author's earlier post, written in Traditional Chinese, "深度學習的數位影像處理介紹" (An introduction to digital image processing with deep learning) [1]. It keeps the post's structure and viewpoints and adds the details, math and code.

Three levels of image tasks

Plain version. "Image processing" in the broad sense covers three kinds of work. Some operations turn an image into a better image. Some pull out a particular part or property of the image. Some try to *understand* the image the way a person would.

Precise version. Following Gonzalez and Woods [2], we can sort image work into three levels by what goes in and what comes out.

Level	Input → output	Typical operations	Classical examples
Image processing (narrow sense; low level)	image → image	primitive, pixel- or neighborhood-wise steps	histogram equalization, filtering, Fourier transform (IP 03, IP 04)
Image analysis (mid level)	image → attributes or highlighted regions	segmentation, feature extraction, description, using prior knowledge	skin-color regions in a face image, vessel regions in an X-ray (IP 10, IP 12)
Computer vision (high level; understanding)	image → meaning	recognition, interpretation, decision	classification, "is there a pedestrian?" (IP 13)

The boundaries are soft. Real systems mix all three. A vessel-segmentation tool, for instance, first denoises (low level), then extracts a vessel map (mid level), and may then flag a stenosis (high level). The levels describe *kinds of computation*, not separate departments.

How deep learning redraws the map

In deep-learning papers you will almost never see the word "mid-level". The field splits tasks into just two families, according to the **shape of the output** [1]:

- **Low-level vision**: the output is an image of (roughly) the same size as the input, and the target is a pixel-accurate signal. Denoising, deblurring, super-resolution, dehazing, low-light enhancement, image-to-image translation.
- **High-level vision**: the output is a *semantic* description. A label, boxes with labels, or a label per pixel.

The old "image analysis" level is absorbed into one of the two. Segmentation produces a per-pixel map, so it *looks* like an image-to-image task. But its values are class labels that require recognition, so it is grouped with high-level vision. Edge or vessel enhancement that outputs a cleaner intensity image is grouped with low-level vision.

The reason for the split is practical. The two families differ in the loss (pixel regression versus classification), the architecture (full-resolution encoder–decoders versus downsampling backbones), the training data (pairs of degraded and clean images versus human labels) and the metrics (fidelity versus accuracy). Figure 1 shows the reorganization.

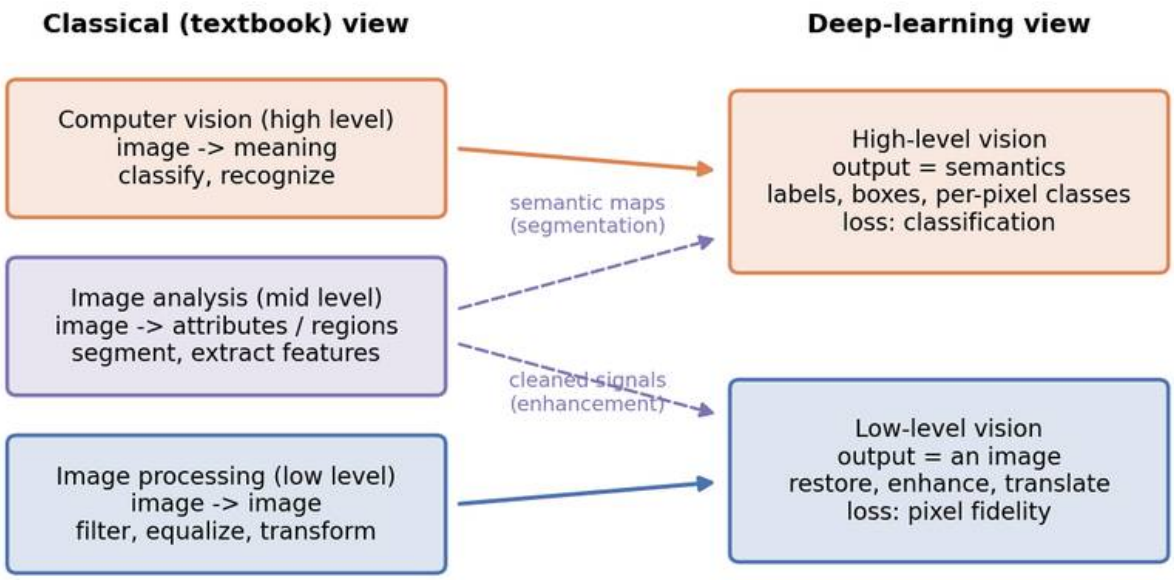


Figure 1 — The classical three levels (left) and the two families used in deep learning (right). Mid-level image analysis is split between them according to whether the output is a cleaned-up signal or a semantic map.

From the pattern-recognition pipeline to end-to-end learning

Plain version. Before deep learning, a recognition system was built like an assembly line: a camera, then a hand-designed step that measures useful properties, then a classifier that decides. Deep learning fuses the middle steps into one network that learns its own measurements.

Precise version. The classical pipeline [3] has four stages:

1. **Sensor.** Capture the pattern (camera, scanner, microscope).
2. **Feature generation and selection.** Compute a vector $\mathbf{x} = \phi(I) \in \mathbb{R}^d$ from image I, using a hand-designed map ϕ (moments, texture statistics, edge histograms, keypoint descriptors), then keep the most informative entries.
3. **Classifier design.** Learn a decision rule $\hat{y} = g(\mathbf{x}; \theta)$ with parameters θ, for example a Bayes classifier, a nearest-neighbor rule or a support-vector machine.
4. **System evaluation.** Measure the error rate on held-out data and go back to step 2 or 3 if it is too high.

Only θ is learned. The feature map ϕ is fixed by an engineer, so the whole system is only as good as ϕ. The chapter on feature extraction (IP 12) shows how much ingenuity went into those maps.

End-to-end learning makes the feature map learnable too. A deep network computes

$$\hat{y} = g(\phi(I; \mathbf{w}); \theta),$$

where $\phi(\cdot\,; \mathbf{w})$ is a stack of layers with weights $\mathbf{w}$, $g(\cdot\,; \theta)$ is a small head, and both $\mathbf{w}$ and θ are fitted jointly by minimizing a loss $\mathcal{L}(\hat{y}, y)$ over labeled pairs (I, y) with gradient descent and backpropagation. The input is raw pixels; nothing is hand-designed except the architecture and the loss.

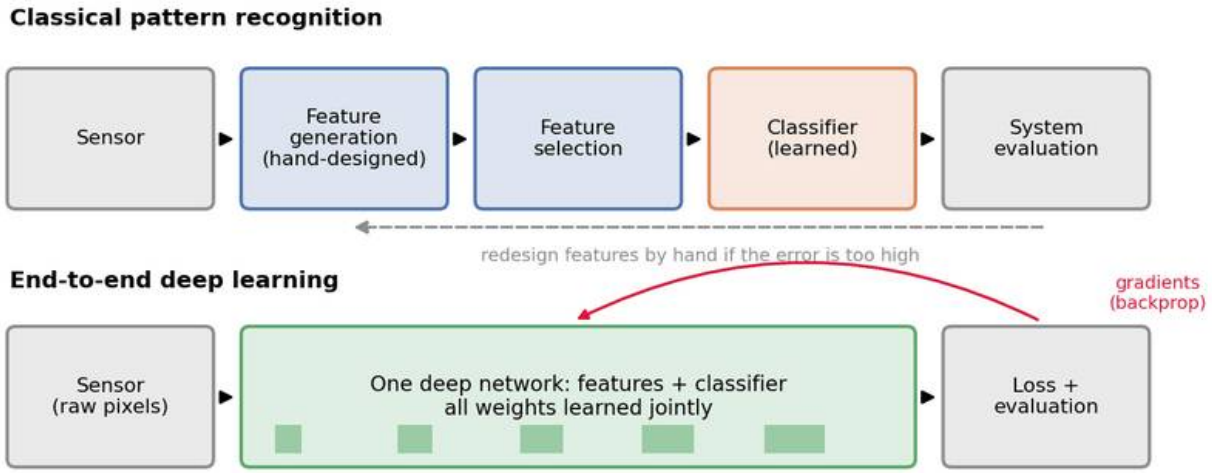


Figure 2 — Classical pattern recognition (top) versus end-to-end learning (bottom). In the deep network the functional split into 'features' and 'classifier' still exists, but there is no clean boundary between them.

The functional split still exists inside a network: early layers behave like feature extractors and the last layers like a classifier. But in practice you usually cannot point at a layer and say "here the features end" [1]. Two consequences follow.

- **Data replaces design.** End-to-end systems need many labeled examples; the hand-designed ϕ encoded prior knowledge that the network must now learn from data.
- **The split came back at a larger scale.** Since about 2021 the standard recipe is a **pretrained backbone** (trained once, often without labels, on a huge dataset) plus a **light task head** trained on your small dataset. That is "feature extractor + classifier" again, except the feature extractor is learned rather than designed. We return to this in the section on self-supervised backbones.

Convolutional neural networks

Plain version. A convolutional layer slides a small window over the image and computes a weighted sum at every position, exactly like the spatial filters of IP 03. The difference is that a CNN *learns* the weights of its filters, uses many filters at once, and stacks dozens of such layers with nonlinearities in between.

A convolutional layer is a bank of learned filters

Precise version. Let the input feature map be $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ (channels, height, width). A convolutional layer with C_{out} filters of size $k \times k$ computes

$$Y_o(x, y) = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{i=-r}^{r} \sum_{j=-r}^{r} W_{o,c}(i, j)\, X_c(x + i,\, y + j), \qquad o = 1, \dots, C_{\text{out}},$$

where $r = (k-1)/2$ for odd k, $W_{o,c}(i, j)$ is the weight of filter o on input channel c at offset (i, j), and b_o is a bias. Strictly this is a **correlation** (no kernel flip), as explained in the convolution tutorial; since the weights are learned, the flip makes no difference. An elementwise nonlinearity such as $\text{ReLU}(z) = \max(0, z)$ follows. Without it, a stack of convolutions would collapse into one big linear filter [1].

Two properties make this layer the right tool for images:

- **Local connectivity.** Each output depends only on a $k \times k$ neighborhood, because nearby pixels are strongly related and distant ones much less so.

- **Weight sharing.** The same filter is used at every position. This makes the layer **translation-equivariant** (shift the input, and the output shifts the same way) and slashes the number of parameters.

The saving is huge. A fully connected layer from a $224 \times 224 \times 3$ image to 64 units needs $224 \cdot 224 \cdot 3 \cdot 64 + 64 \approx 9.6$ million parameters. A 3×3 convolution from 3 to 64 channels needs $3 \cdot 3 \cdot 3 \cdot 64 + 64 = 1{,}792$, and it produces 64 *full-resolution* feature maps instead of 64 numbers.

The code below sets the two filters of a PyTorch convolution by hand to Sobel kernels, which turns the layer into an edge detector. Training would instead set these numbers by gradient descent.

```
import torch, torch.nn as nn, torch.nn.functional as F
from skimage import data, img_as_float

x = torch.tensor(img_as_float(data.camera()), dtype=torch.float32)[None, None]  # (1, 1,
 ↪  512, 512)
conv = nn.Conv2d(1, 2, kernel_size=3, padding=1, bias=False)
sobel_x = torch.tensor([[-1., 0., 1.], [-2., 0., 2.], [-1., 0., 1.]])
with torch.no_grad():
    conv.weight[0, 0] = sobel_x        # channel 0: responds to vertical edges
    conv.weight[1, 0] = sobel_x.T      # channel 1: responds to horizontal edges
    y = F.relu(conv(x))                # (1, 2, 512, 512): two feature maps
```

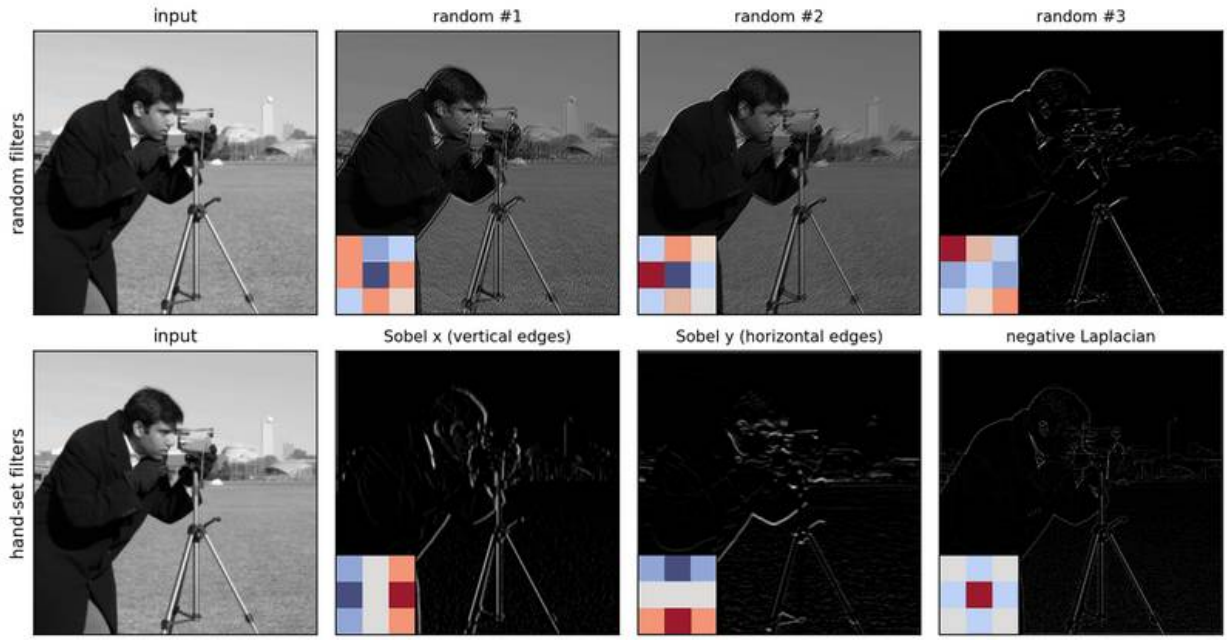


Figure 3 — Feature maps (after ReLU) of 3×3 convolutions on the camera image. Top: random initial weights, which give noisy mixtures of smoothing and edges. Bottom: hand-set Sobel and Laplacian weights. Training moves the random filters toward useful ones.

What do trained first-layer filters look like? We can train a tiny CNN on the 8×8 handwritten digits that ship with scikit-learn [36] in a few seconds on a CPU.

```
import torch, torch.nn as nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
X, y = load_digits(return_X_y=True)                    # 1,797 images of 8x8 pixels
X = torch.tensor(X / 16.0, dtype=torch.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, torch.tensor(y), test_size=0.25, random_state=0)

net = nn.Sequential(
    nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),     # 8x8 -> 4x4
    nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),    # 4x4 -> 2x2
    nn.Flatten(), nn.Linear(16 * 2 * 2, 10),
)
opt = torch.optim.Adam(net.parameters(), lr=1e-2)
for epoch in range(30):
    for i in range(0, len(Xtr), 64):
        loss = nn.functional.cross_entropy(net(Xtr[i:i + 64]), ytr[i:i + 64])
        opt.zero_grad(); loss.backward(); opt.step()
print((net(Xte).argmax(1) == yte).float().mean())       # about 0.97
filters = net[0].weight.detach()[:, 0]                   # (8, 3, 3) learned kernels
```

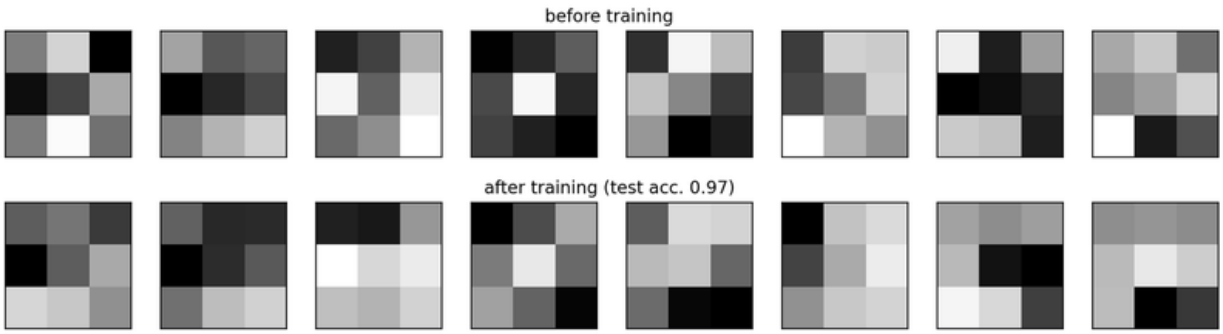


Figure 4 — The eight first-layer 3×3 filters of the tiny digit CNN, before (top) and after (bottom) training. Several drift toward oriented light-to-dark transitions, the derivative-like patterns a hand designer would choose; nobody told the network to do so. (Tiny 8×8 inputs and 3×3 kernels keep the effect modest; large networks trained on natural images show it much more clearly.)

Stride, padding and output size

Plain version. Padding adds a border so the window can sit on edge pixels. Stride makes the window jump more than one pixel at a time, which shrinks the output.

Precise version. For an input of width W, kernel size k, padding p (pixels added on each side) and stride s, the output width is

$$W_{\text{out}} = \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1.$$

With $k = 3$, $p = 1$, $s = 1$ the size is preserved ("same" padding); with $s = 2$ it is halved. Zero padding is the default; reflect or replicate padding, the same boundary choices discussed in IP 03, are preferred in low-level networks because zero borders create dark artifacts at image edges.

Pooling

Pooling summarizes each small window by one number, usually the maximum (max pooling) or the mean (average pooling), typically with a 2×2 window and stride 2. It halves resolution, makes the representation tolerant to small shifts, and cheaply enlarges the receptive field. Many modern networks replace it with strided convolutions, which learn how to downsample. At the very end of a classification network, **global average pooling** turns a $C \times H \times W$ map into a C-vector for the classifier head.

Receptive field

Plain version. The receptive field of a neuron is the patch of the input image that can influence it. One 3×3 layer sees 3×3 pixels. Stack two, and each output sees 5×5. Downsampling makes it grow much faster.

Precise version. Number the layers $\ell = 1, \dots, L$ with kernel size k_ℓ and stride s_ℓ. Let j_ℓ be the **jump**, the distance in input pixels between two neighboring units of layer ℓ, and r_ℓ the receptive-field size, with $r_0 = 1$, $j_0 = 1$ for the input. Then

$$r_\ell = r_{\ell-1} + (k_\ell - 1)\, j_{\ell-1}, \qquad j_\ell = j_{\ell-1}\, s_\ell.$$

Each layer adds $k_\ell - 1$ units of its input, and each input unit is worth $j_{\ell-1}$ pixels. With stride 1 everywhere, L layers of 3×3 give $r_L = 2L + 1$: linear growth. Every stride-2 step doubles the jump, so later layers grow the field twice as fast, giving roughly exponential growth in depth.

```
def receptive_field(layers):
    r, j = 1, 1                       # receptive field and jump, measured in input pixels
    for k, s in layers:               # (kernel size, stride) of each layer
        r, j = r + (k - 1) * j, j * s
    return r

print(receptive_field([(3, 1)] * 3))                                              # 7
print(receptive_field([(3, 1), (3, 1), (2, 2), (3, 1), (3, 1), (2, 2), (3, 1)])) # 24
```

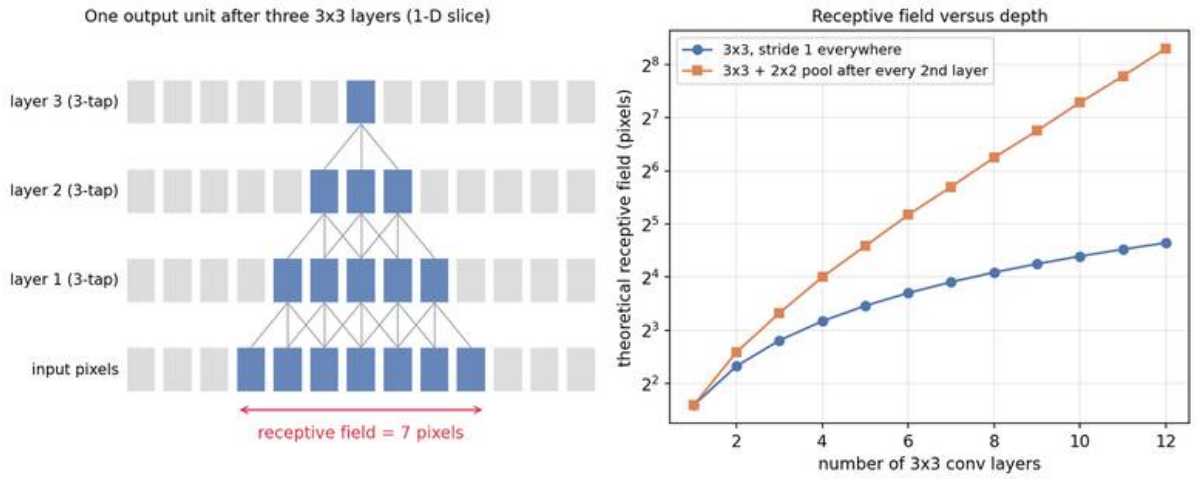


Figure 5 — Left: three stacked 3-tap layers give one output a 7-pixel receptive field. Right: theoretical receptive field versus depth, for stride 1 everywhere and for 2× downsampling after every second layer.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Two refinements matter in practice. First, the formula gives the **theoretical** receptive field. Luo et al. [12] showed that the **effective** receptive field, measured by how much each input pixel actually affects the output gradient, is roughly Gaussian and occupies only a fraction of the theoretical one. Pixels near the center dominate. Second, the receptive field decides what a network *can* use: a denoiser whose receptive field is smaller than the noise correlation length cannot remove that noise, and a classifier whose last layer sees only part of an object cannot recognize it from context.

Feature hierarchies

Because the receptive field grows with depth while resolution shrinks, a CNN builds a **feature hierarchy**: early layers respond to edges and color blobs (Figure 4), middle layers to textures and parts, late layers to object-level patterns over large regions. A typical backbone is organized in **stages**, each ending with a 2× downsampling and doubling the channel count, so features go from "high-resolution, few channels, local" to "low-resolution, many channels, semantic". High-level tasks read the late stages. Dense prediction and low-level tasks need the early, high-resolution stages too, which is why encoder–decoders exist (below).

Landmark CNN architectures

Plain version. Between 2012 and 2016 a series of networks, each deeper than the last, won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) [6]. Each one introduced a design idea that is still in use.

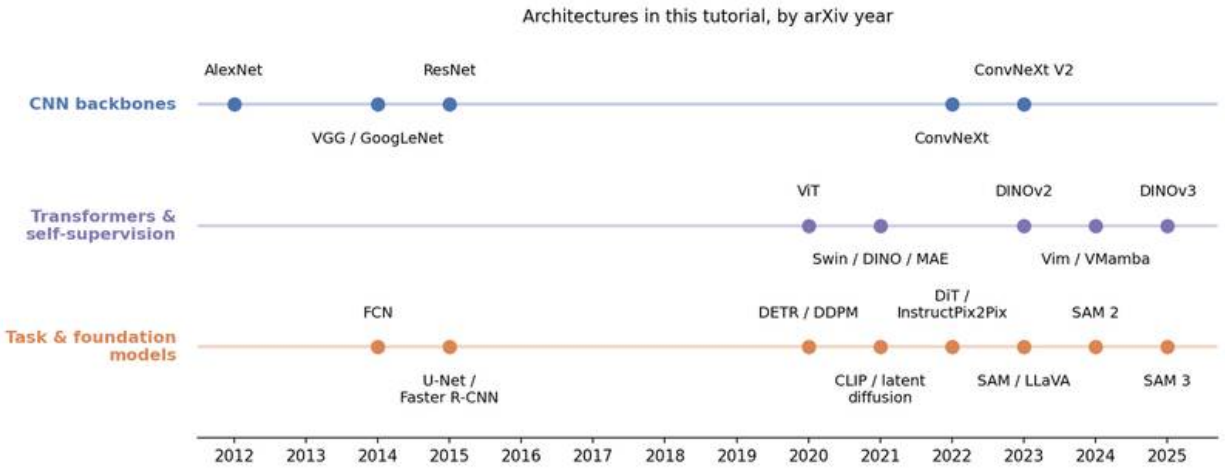


Figure 6 — A timeline of the architectures discussed in this tutorial, by arXiv year. LeNet-5 (1998) and the earlier convolutional networks of the late 1980s sit to the left of this axis.

LeNet-5 and where CNNs really began

The post's original heading said "CNN (1998~)". That year is when LeCun, Bottou, Bengio and Haffner published LeNet-5 [5], not when CNNs started: convolutional networks trained by back-propagation were already reading handwritten ZIP-code digits in 1989 [4].

LeNet-5 is a textbook pattern-recognition system in network form [1]. Layers C1 to C5 are the

feature extractor: convolutions (C1, C3, C5) alternating with subsampling layers (S2, S4) that turn a 32×32 digit image into a 120-dimensional feature vector. F6 and the output layer are the

classifier. Everything, features included, is trained by gradient descent on the classification loss. This was the end-to-end idea, already in place, but it needed 15 more years of data and compute to dominate.

AlexNet (2012)

AlexNet [7] has five convolutional layers and three fully connected layers (eight learned layers, about 60 million parameters). Its ingredients were not individually new, but the combination was: ReLU activations, which train much faster than saturating sigmoids; an efficient GPU implementation; heavy data augmentation; and dropout in the fully connected layers. It won ILSVRC

2012 by a wide margin, which is why 2012 is usually given as the start of the deep-learning era in vision.

VGG (2014)

Simonyan and Zisserman [8] asked a single question: what happens if you use only 3×3 convolutions and make the network deeper? Their best configurations had 16 and 19 weight layers. The reason 3×3 is enough is the receptive-field formula above: two stacked 3×3 layers see 5×5, three see 7×7. For C channels in and out, three 3×3 layers cost $3 \cdot 9C^2 = 27C^2$ weights versus $49C^2$ for one 7×7 layer, and they insert two extra nonlinearities. VGG's uniform design made it the default pretrained feature extractor for years.

GoogLeNet / Inception (2014)

GoogLeNet [9], 22 layers deep, won ILSVRC 2014. Its **Inception module** runs 1×1, 3×3 and 5×5 convolutions and a pooling branch in parallel and concatenates their outputs, so each block sees several scales at once. To keep this affordable, cheap 1×1 convolutions first reduce the number of channels. A 1×1 convolution mixes channels at each pixel without looking at neighbors; it is a per-pixel fully connected layer, and it remains a basic building block everywhere.

ResNet (2015) and why residual connections work

Deeper should be better, but in practice it was not: He et al. [10] observed that a plain 56-layer network had *higher training error* than a 20-layer one. That is not overfitting. It is an optimization failure: the deeper net could in principle copy the shallow one and set the extra layers to the identity, but gradient descent does not find that solution.

The fix is the **residual block** (Figure 7). Instead of asking a stack of layers to learn a mapping $H(\mathbf{x})$ directly, let it learn the residual $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$ and add the input back:

$$\mathbf{y} = \sigma(\mathbf{x} + F(\mathbf{x}; W)),$$

where $\mathbf{x}$ is the block input, F is typically conv–BN–ReLU–conv–BN with weights W (BN is batch normalization), and σ is ReLU. When channel counts or resolution change, the shortcut uses a 1×1 convolution instead of the identity.

Why it works:

- **Identity is easy.** If extra depth is not useful, the block only has to drive F toward zero, which is far easier than making a stack of nonlinear layers imitate the identity. Initializing the last BN scale of F at zero makes each block start as the identity.

- **Gradients have a highway.** Ignoring the final nonlinearity, $\partial \mathbf{y}/\partial \mathbf{x} = I + \partial F/\partial \mathbf{x}$. The identity term carries the gradient from the loss to early layers without passing through every weight matrix, so it does not vanish with depth.
- **An ensemble view.** Unrolling L blocks gives a sum over many paths of different lengths, so a deep ResNet behaves partly like a collection of shallower networks.

```
class ResidualBlock(nn.Module):
    """Basic block: y = ReLU(x + F(x)), with F = conv-BN-ReLU-conv-BN."""
    def __init__(self, c):
        super().__init__()
        self.f = nn.Sequential(
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c), nn.ReLU(),
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c),
        )
    def forward(self, x):
        return torch.relu(x + self.f(x))

blk = ResidualBlock(16)
nn.init.zeros_(blk.f[4].weight)                 # zero the last BN scale, so F(x) = 0
x = torch.randn(2, 16, 32, 32)
print(torch.allclose(blk(x), torch.relu(x)))    # True: the block starts as the identity
 ↪  (then ReLU)
```

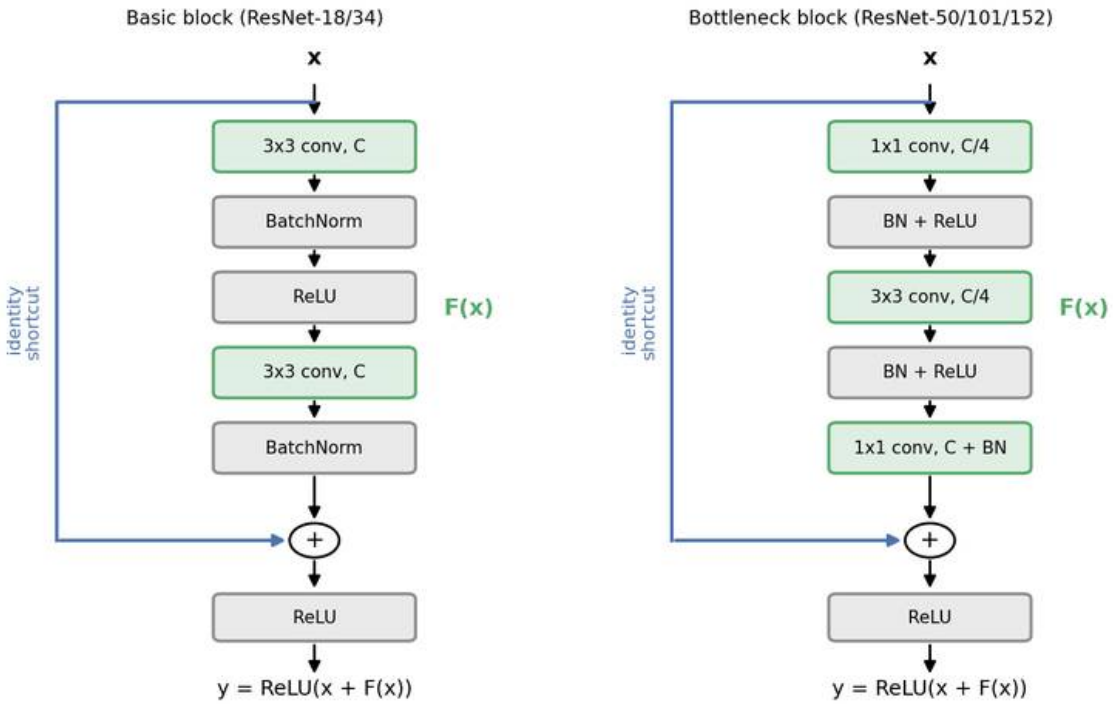


Figure 7 — Left: the basic residual block. Right: the bottleneck block used in ResNet-50/101/152, where

1×1 convolutions shrink and restore the channel count around a cheap 3×3 convolution.

On ImageNet the paper trained ResNets with **18, 34, 50, 101 and 152 layers**; the deeper three use the bottleneck block. The famous **1202-layer** network appears only in the paper's CIFAR-10 experiments [10], [11] (32×32 images), where it trained fine but overfit and tested worse than the 110-layer version. The ResNet family won ILSVRC 2015, and the residual connection is now in almost every deep network, Transformers included.

Encoder–decoders for dense prediction

Plain version. A classification backbone throws away resolution to gain meaning. For segmentation or restoration we need an answer at every pixel, so we add a second half that brings the resolution back.

Precise version. Long et al.'s **fully convolutional network (FCN)** [13] replaced the fully connected layers of a classifier with convolutions, so the network outputs a coarse class-score map for an input of any size, then upsampled it with learned transposed convolutions and fused it with scores from earlier, finer layers. **U-Net** [14] made this symmetric: a contracting **encoder** (conv blocks and downsampling) and an expanding **decoder** (upsampling and conv blocks), with **skip connections** that concatenate each encoder stage's feature map to the decoder stage of the same resolution. The encoder supplies *what*; the skips supply *where*. U-Net was designed for biomedical images with few labels and strong augmentation, and the same shape now underlies medical segmentation, almost every restoration network covered in DLCV 04, and the denoiser inside many diffusion models.

The Vision Transformer

Plain version. The Transformer [15] was built for sentences: it treats the input as a sequence of tokens and lets every token look at every other token. The Vision Transformer (ViT) [16] cuts an image into small square patches, treats each patch as a "word", and feeds the sequence to an unmodified Transformer encoder.

Patchify, embed, attend

Precise version. For an image $I \in \mathbb{R}^{C\times H\times W}$ and patch size P:

1. **Patchify.** Split I into $N = HW/P^2$ non-overlapping patches and flatten each into a vector $\mathbf{p}_i \in \mathbb{R}^{CP^2}$. For a 224×224 RGB image and $P = 16$, $N = 196$ and each vector has 768 entries.

2. **Embed.** Map each patch linearly to a D-dimensional token, $\mathbf{z}_i = E\,\mathbf{p}_i$ with $E \in \mathbb{R}^{D\times CP^2}$. Prepend a learnable class token $\mathbf{z}_{\text{cls}}$ and add learnable **position embeddings** $\mathbf{e}_i$, because attention by itself has no notion of where a token came from: $Z^{(0)} = [\mathbf{z}_{\text{cls}}; \mathbf{z}_1; \dots; \mathbf{z}_N] + [\mathbf{e}_0; \dots; \mathbf{e}_N]$.
3. **Encode.** Apply L Transformer encoder blocks, each a multi-head self-attention (MSA) sublayer and an MLP sublayer, both with layer normalization (LN) and residual connections:

$$Z' = Z^{(\ell-1)} + \mathrm{MSA}(\mathrm{LN}(Z^{(\ell-1)})),$$
$$Z^{(\ell)} = Z' + \mathrm{MLP}(\mathrm{LN}(Z')).$$

4. **Head.** Feed the final class token (or the average of all tokens) to an MLP head, which plays the role of the pattern-recognition classifier [1].

A single attention head computes, for token matrix $Z \in \mathbb{R}^{(N+1)\times D}$,

$$\mathrm{Attn}(Z) = \mathrm{softmax}\!\left(\frac{QK^{\top}}{\sqrt{d}}\right)V, \qquad Q = ZW_Q,\; K = ZW_K,\; V = ZW_V,$$

where $W_Q, W_K, W_V \in \mathbb{R}^{D\times d}$ are learned projections and d is the head dimension. Row i of the softmax matrix says how much token i draws from every other token. Compared with a convolution, the "filter" is computed from the content and covers the whole image from the first layer on. The cost is quadratic in the number of tokens, $O(N^2 d)$, which is why patches rather than pixels are used.

```
img = torch.randn(1, 3, 224, 224)
P, D = 16, 768
patches = img.unfold(2, P, P).unfold(3, P, P)                    # (1, 3, 14, 14, 16, 16)
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(1, -1, 3 * P * P)
print(patches.shape)                                              # (1, 196, 768)
embed = nn.Conv2d(3, D, kernel_size=P, stride=P)                  # patchify + linear embed
 ↪  in one op
tokens = embed(img).flatten(2).transpose(1, 2)                    # (1, 196, 768)
```

The last two lines show a useful identity: patch embedding *is* a convolution with kernel size and stride both equal to P.

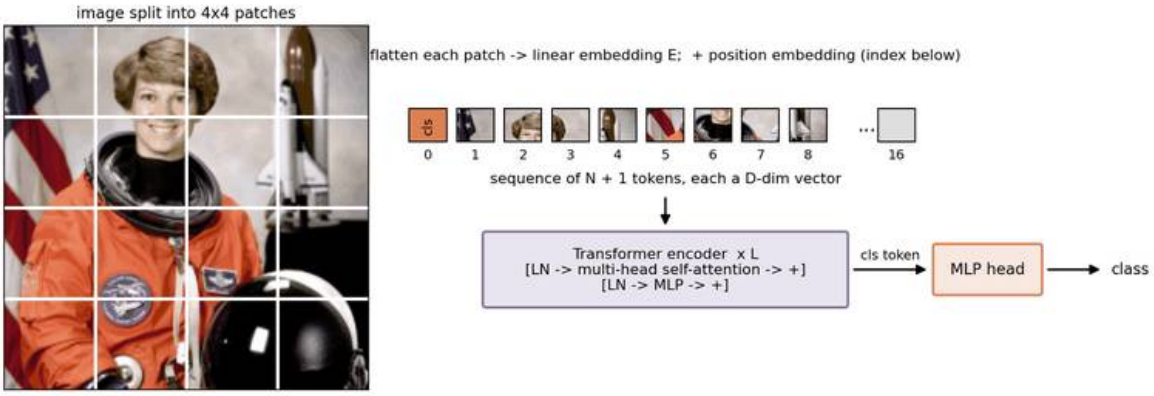


Figure 8 — ViT in one picture: patchify, linearly embed, add position embeddings and a class token, run a Transformer encoder, classify from the class token. (A 4×4 grid is drawn for legibility; ViT-B/16 on 224×224 inputs uses 14×14 patches.)

Inductive bias and the need for scale

The original post said that ViT is "harder to converge". The more precise statement [1], [16] is about **inductive bias**, the assumptions built into an architecture before it sees data. A CNN assumes locality and weight sharing, so it already "knows" that nearby pixels matter together and that a pattern means the same thing anywhere in the image. ViT assumes almost nothing: apart from the patch split, all spatial structure, even the 2-D layout of the position embeddings, must be learned.

That is a trade-off, not a defect. With little data, the CNN's assumptions are a valuable head start, and ViT trained from scratch on ImageNet-1k alone underperforms comparable ResNets. With enough data, the assumptions become a limitation, and ViT pretrained on very large datasets matched or beat the best CNNs when fine-tuned [16]. The practical lesson is that ViTs are used **pretrained**, either on huge labeled sets or, increasingly, with self-supervision.

Backbones after ViT

Plain version. After ViT, research went three ways: make Transformers more image-friendly (Swin), show that CNNs can catch up when modernized (ConvNeXt), and pretrain without labels so that one backbone serves every task (DINO, MAE, DINOv2/v3).

Swin Transformer: locality returns

Swin [17] computes attention only inside non-overlapping local windows (for example 7×7 tokens), which makes the cost linear in image size. In alternate blocks the window grid is **shifted** by half a window, so information crosses window borders. Between stages, neighboring tokens are merged to halve resolution. The result is a hierarchical, multi-scale feature pyramid shaped like a CNN backbone, which is exactly what detection and segmentation heads expect. Swin put locality and hierarchy, the CNN's inductive biases, back into a Transformer.

ConvNeXt: the CNN counterpoint

Liu et al. [18] started from a ResNet-50 and modernized it one step at a time, using the training recipe and design choices of Transformers: a "patchify" stem (a 4×4 convolution with stride 4), large 7×7 depthwise convolutions, an inverted-bottleneck MLP, fewer activation and normalization layers, and layer normalization. The resulting pure CNN, **ConvNeXt**, matched Swin in accuracy and scaling on classification, detection and segmentation. The lesson: much of ViT's advantage came from training recipes and macro design, not from attention itself. **ConvNeXt V2** [18] then added masked-autoencoder pretraining adapted to convolutions (the fully convolutional masked autoencoder, FCMAE) and a global response normalization layer. Today large foundation models mostly use ViT backbones, but convolutions remain common in low-level vision and on edge devices [1].

Self-supervised pretraining: DINO versus MAE

Self-supervised learning creates a training signal from the images themselves, so no labels are needed. Two recipes dominate, and they are often confused.

DINO: self-distillation [19]. A student network g_s and a teacher g_t with the same architecture each output a probability vector over K prototype dimensions. Several random crops of an image are made: two large "global" views and several small "local" views. The teacher sees only global views, the student sees all of them, and the student is trained to match the teacher's output:

$$\min_{\theta_s} \sum_{x \in \{x_1^g, x_2^g\}} \sum_{\substack{x' \in V \\ x' \neq x}} H(P_t(x),\, P_s(x')), \qquad H(a, b) = -a^\top \log b,$$

where V is the set of all views, P_t and P_s are the softmax outputs of teacher and student, and θ_s are the student's weights. The teacher is not trained by gradients. Its weights are an exponential moving average of the student's, $\theta_t \leftarrow \lambda \theta_t + (1-\lambda)\theta_s$ with λ close to 1, and its outputs are centered and sharpened so that training does not collapse to a constant output. Matching "small crop" to "whole image" forces the network to recognize objects from parts. A remarkable emergent

property is that the attention maps of a DINO ViT segment the main objects without ever seeing a mask.

MAE: masked prediction [20]. The masked autoencoder hides a large random subset of patches (75 % works best), encodes only the visible ones with a ViT, and asks a light decoder to reconstruct the missing pixels:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{\mathbf{p}}_i - \mathbf{p}_i\|_2^2,$$

where $\mathcal{M}$ is the set of masked patch indices, $\mathbf{p}_i$ the (normalized) pixels of patch i and $\hat{\mathbf{p}}_i$ the reconstruction. This "hide and predict" game is the image analogue of masked-word prediction in language models; DINO, by contrast, is a teacher–student distillation method [1]. MAE is cheap because the encoder processes only a quarter of the tokens, and it fine-tunes very well.

DINO-style features are better **frozen**, used as-is with a linear head.

DINOv2 and DINOv3: frozen general-purpose features

DINOv2 [21] scaled DINO-style self-distillation to a 1-billion-parameter ViT trained on a large, automatically **curated** image collection, then distilled it into smaller models. Its frozen features work well for both image-level tasks (classification, retrieval) and pixel-level tasks (segmentation, depth estimation) with only a light head. **DINOv3** [21] scaled data and model size further and introduced **Gram anchoring**, a regularizer that keeps dense patch features from degrading during very long training, so the same frozen backbone gives high-quality dense features.

This closes the loop with the pattern-recognition pipeline. "Feature extractor + classifier" has returned as **pretrained frozen backbone + light head**. The feature extractor is now learned once, at very large scale, without labels.

State-space backbones

Vision Mamba (Vim) and **VMamba** [22] replace attention with selective state-space models, which scan the token sequence with cost linear in its length. Images are not sequences, so these models scan the patch grid along more than one path (Vim is bidirectional; VMamba's 2-D selective scan traverses several routes). They are attractive for high-resolution inputs but are still mostly a research direction rather than a default backbone.

High-level vision tasks

Plain version. High-level tasks differ in how *where* the answer must be: one label for the whole image, a box per object, or a label for every pixel.

Task	Output	Typical loss	Typical metric	Representative models
Classification	one label (or a probability vector)	cross-entropy	top-1 / top-5 accuracy	ResNet, ViT, ConvNeXt [10], [16], [18]
Object detection	a set of (box, class, score)	classification + box regression, or set matching	average precision (AP) over IoU thresholds	Faster R-CNN, DETR [23]
Semantic segmentation	a class per pixel	per-pixel cross-entropy, Dice (an overlap loss, below)	mean IoU	FCN, U-Net [13], [14]
Instance segmentation	a mask and class per object	detection + mask losses	mask AP	promptable: SAM family [24]

Classification

Assign one label to the whole image. The network is a backbone plus global pooling plus a linear layer, trained with cross-entropy $\mathcal{L} = -\log \hat{p}_y$, where $\hat{p}_y$ is the predicted probability of the true class y. Classification is also the **pretraining task** that produced most backbones before self-supervision. DLCV 02 covers it in depth.

Object detection

Find every object, its class and its location as an axis-aligned box (a region of interest, ROI, given by its corner coordinates or center, width and height) [1]. Two design lineages illustrate the field [23]. **Faster R-CNN** is two-stage: a region proposal network suggests candidate boxes from shared CNN features, and a second head classifies and refines each one; duplicates are removed by non-maximum suppression. **DETR** treats detection as **set prediction**: a Transformer decoder emits a fixed set of predictions, and a bipartite matching between predictions and ground-truth objects defines the loss, which removes hand-designed anchors and non-maximum suppression. Box quality is measured by intersection over union,

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

where A and B are the predicted and true regions. The closely related Dice coefficient $2|A \cap B|/(|A| + |B|)$ is used, in a differentiable soft form, as a segmentation loss. Average precision summarizes

the precision–recall curve at one or more IoU thresholds. Details are in DLCV 03.

Semantic and instance segmentation

Here the region of interest is the object's exact set of pixels, not a box [1].

- **Semantic segmentation** labels each pixel with a class. Two overlapping people become one "person" region.
- **Instance segmentation** also separates objects of the same class. The two people become "person 1" and "person 2".

Semantic segmentation is the natural job of the encoder–decoders above [13], [14]. Instance segmentation adds a detection-like component that proposes one mask per object.

Promptable segmentation: SAM, SAM 2, SAM 3

Segmentation now has **promptable foundation models** [24]. **SAM** (Segment Anything) takes an image and a prompt (points, a box, or a rough mask) and returns a mask for the indicated object. A heavy ViT image encoder runs once per image, and a light prompt encoder and mask decoder run per prompt, so interaction is fast. It was trained on SA-1B, a dataset of more than 1 billion masks on 11 million images, built with a model-in-the-loop "data engine". **SAM 2** extends this to video with a streaming memory that carries the object from frame to frame, trained on a new video dataset (SA-V). **SAM 3** adds **promptable concept segmentation**: given a short noun phrase such as "yellow school bus", image exemplars, or both, it finds, segments and (in video) tracks *all* matching instances, rather than the one object a click points at. With SAM 3 the line between detection, instance segmentation and tracking becomes thin; tracking is covered in DLCV 06.

Low-level vision tasks

Plain version. Low-level tasks take an image and return an improved or transformed image of the same scene. The output is judged pixel by pixel, or by how natural it looks.

The general formulation is an inverse problem. A degraded observation $\mathbf{y}$ is modeled as

$$\mathbf{y} = \mathcal{D}(\mathbf{x}) + \mathbf{n},$$

where $\mathbf{x}$ is the clean image, $\mathcal{D}$ a degradation operator (blur, downsampling, haze, low exposure) and $\mathbf{n}$ noise, the same model as in IP 05. A network f_θ is trained on pairs to minimize, for example, $\|f_\theta(\mathbf{y}) - \mathbf{x}\|_1$. Because real paired data is scarce, pairs are usually **synthesized** by applying

a modeled degradation to clean images, and the gap between synthetic and real degradations is the field's central difficulty.

- **Image restoration** recovers the clean image: denoising, deblurring, super-resolution (small to large), dehazing (hazy to clear), deraining. Architectures are full-resolution encoder–decoders (U-Net-like) or residual networks without downsampling, now often with window attention. See DLCV 04.
- **Image enhancement** makes content easier to see without a single "true" target: low-light enhancement (bringing out detail in dark images, roughly a smart brightening; turning night into day would be translation instead) and underwater enhancement. See DLCV 05.
- **Image alignment** (registration) estimates the geometric transform or dense flow that maps one image onto another, a prerequisite for burst fusion, HDR and medical image comparison.
- **Image-to-image translation** maps an image from one domain or style to another: day to night, summer to winter, sketch to photo. Face-swapping "deepfakes" are a (misused) application of the same idea. Since diffusion models, translation and editing are often done by a generative model conditioned on the input image, as discussed next.

Generation and vision–language models

Plain version. Two newer families do not fit the old map well: models that **create** images from text, and models that **talk** about images. Both rely on large-scale pretraining and on connecting images with language.

Diffusion models: from noise to image

A **denoising diffusion probabilistic model (DDPM)** [27] defines a forward process that gradually adds Gaussian noise to an image $\mathbf{x}_0$ over T steps. In closed form,

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I),$$

where $t \in \{1, \dots, T\}$ is the step and $\bar{\alpha}_t \in (0,1)$ is a fixed schedule that decreases from nearly 1 to nearly 0. A network $\boldsymbol{\epsilon}_\theta$ is trained to predict the noise that was added,

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0, t, \boldsymbol{\epsilon}} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2,$$

and generation runs the process backwards, starting from pure noise and denoising step by step.

In other words, a generator is a **learned denoiser applied many times**, which ties generation directly to the restoration problems above.

- **Latent diffusion** [28] runs the diffusion in the compressed latent space of a pretrained autoencoder rather than on pixels, which cuts the cost enough for high-resolution text-to-image synthesis; text conditioning enters through cross-attention. It is the basis of Stable Diffusion.
- **DiT** (Diffusion Transformer) [28] replaces the U-Net denoiser with a Transformer on latent patches and shows that quality improves steadily as the model's compute grows.
- **InstructPix2Pix** [29] edits a given image according to a written instruction ("make it winter"), so classic image-to-image translation becomes a special case of conditional generation.

Vision–language models

Visual question answering (VQA), answering a free-form question about an image, once required a pipeline of image features, captioning and text question answering [1]. Two steps made it a single model.

CLIP [25] trains an image encoder f and a text encoder g on 400 million image–text pairs with a **contrastive** objective. In a batch of B pairs, with normalized embeddings $\mathbf{u}_i = f(I_i)$ and $\mathbf{v}_j = g(T_j)$ and temperature τ, the image-to-text loss is

$$\mathcal{L}_{I\to T} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(\mathbf{u}_i^\top \mathbf{v}_i/\tau)}{\sum_{j=1}^{B} \exp(\mathbf{u}_i^\top \mathbf{v}_j/\tau)},$$

and the total loss averages it with the symmetric text-to-image term. Matching pairs are pulled together and all other pairs in the batch pushed apart. Because the label space is now "any sentence", CLIP can classify zero-shot: embed the prompts "a photo of a cat", "a photo of a dog", ... and pick the one closest to the image.

LLaVA [26] connects a CLIP vision encoder to a large language model (LLM) through a projection layer that maps visual tokens into the LLM's word-embedding space, then fine-tunes on instruction-following conversations about images. Most current multimodal LLMs keep this **vision encoder + projector + LLM** pattern, so VQA, captioning and many recognition tasks are now handled by prompting one model.

Modern view

The trend across this tutorial is a move from "one task, one model" to "one foundation model + prompt or light head". The surveys below are a good map of how the field got there.

CNN architectures. Li et al. [30] review CNNs from their history through the convolution operation itself, classic and advanced architectures (the LeNet–ResNet line, lightweight and attention-augmented networks), 1-D, 2-D and multi-dimensional convolutions, and applications. They also

cover the improvements made to each component (layer design, activation and loss functions, regularization, optimization, fast computation), run experiments to derive rules of thumb for choosing functions and hyperparameters, and close with open issues and promising directions. It is the best single place to see the CNN design space laid out component by component.

Vision Transformers. Khan et al. [31] (ACM Computing Surveys) organize Transformer work by task: the foundations (self-attention, large-scale pretraining, bidirectional encoding), then recognition (classification, detection, segmentation, action recognition), generative modeling, multimodal tasks (VQA, visual reasoning, grounding), video, low-level vision (super-resolution, enhancement, colorization) and 3-D point clouds. Their central contrast matches this tutorial's: Transformers model long-range dependencies with minimal inductive bias, so they need large-scale pretraining or architectural priors (windows, hierarchies) to be data-efficient.

Self-supervised visual learning. Jing and Tian [32] (TPAMI) survey the pre-2020 generation of self-supervised methods, organized by pretext task (generation-based, context-based, free-semantic-label-based and cross-modal), along with the standard protocol of evaluating features by transfer to downstream tasks. Balestriero et al. [33] cover the modern generation in a practical "cookbook" and group methods into four families: deep metric learning (contrastive), self-distillation (the DINO family), canonical-correlation-style methods, and masked image modeling (MAE). Their emphasis on the many hidden "knobs" (augmentations, projector heads, collapse prevention) explains why reproducing self-supervised results is hard.

Foundation models for vision. Awais et al. [34] (TPAMI) categorize vision foundation models by modality pairing (vision with text, audio, depth), training objective (contrastive versus generative) and **prompt type**: textual, visual (points, boxes, masks, as in SAM) and heterogeneous. Their open challenges are evaluation and benchmarking, real-world understanding and contextual reasoning, bias, adversarial robustness and interpretability. Zhang et al. [35] (TPAMI) focus on vision–language models for visual recognition: architectures and pretraining objectives, transfer methods (prompt tuning, adapters) and knowledge distillation into detectors and segmenters. They argue that web-scale image–text data reduces the dependence on crowd-sourced labels and lets one model serve many tasks.

State of the art, 2024–2026. Without quoting benchmark numbers, the landscape looks like this:

- **Backbones.** Large ViTs pretrained with self-supervision or image–text data are the default for high-level tasks, with frozen DINOv2/DINOv3 features [21] a strong baseline for dense tasks. Modern CNNs (ConvNeXt V2 [18]) remain competitive and are preferred where latency, memory or high resolution dominate. State-space backbones [22] are an active alternative.
- **Segmentation and tracking** have converged on promptable models: SAM 2 for video objects and SAM 3 for open-vocabulary concepts [24].
- **Generation** has moved to latent diffusion with Transformer denoisers [28], and editing to instruction-conditioned diffusion [29].

- **Understanding** is increasingly done by multimodal LLMs in the LLaVA pattern [26].
- **Low-level vision** still relies heavily on task-specific encoder–decoders trained on synthetic degradations, now increasingly with generative (diffusion) priors; see DLCV 04 and DLCV 05.

Open problems. (1) *Data and evaluation*: benchmarks saturate and may overlap with web-scale pretraining data, so it is hard to tell generalization from memorization [34]. (2) *Real-world degradations*: models trained on synthetic noise or blur still fail on real camera pipelines. (3) *Fidelity versus hallucination*: generative priors produce sharp but sometimes invented detail, which is unacceptable in medical or forensic use. (4) *Efficiency*: attention is quadratic in tokens, and foundation models are hard to run on devices. (5) *Dense features at scale*: DINOv3's Gram anchoring [21] exists because dense feature quality can degrade as training scales. (6) *Bias, robustness and interpretability* of models trained on uncurated web data [34].

Key takeaways

- Deep learning reduces the three classical levels to two: **low-level** (image → image, judged by fidelity) and **high-level** (image → semantics, judged by accuracy). Mid-level analysis goes to whichever its output resembles.
- **End-to-end learning** replaced hand-designed features with learned ones. The split returned at scale as **pretrained backbone + light head**.
- A convolutional layer is a bank of **learned spatial filters** with local connectivity and weight sharing. Stride, padding and pooling set the output size; the **receptive field** grows as $r_\ell = r_{\ell-1} + (k_\ell - 1)j_{\ell-1}$, and the effective field is smaller than the theoretical one.
- LeNet-5 (1998) was not the first CNN; backprop-trained convolutional networks date to 1989. ResNet was trained at **18/34/50/101/152** layers on ImageNet; **1202** layers appears only on CIFAR-10. Residual connections make the identity easy and give gradients a direct path.
- Encoder–decoders with skip connections (**FCN, U-Net**) are the standard shape for dense prediction and restoration.
- **ViT** has little inductive bias, so it needs large-scale pretraining. Swin added locality back; ConvNeXt showed a modernized CNN matches it.
- **DINO** is teacher–student self-distillation; **MAE** is masked prediction. DINOv2/v3 provide frozen, general-purpose features.
- Tasks increasingly converge on promptable or language-connected **foundation models**: SAM 1/2/3 for segmentation, latent diffusion and DiT for generation, CLIP and LLaVA for vision–language.

Exercises

1. **Receptive field by hand.** A network has, in order: conv 7×7 stride 2; max-pool 3×3 stride 2; then four conv 3×3 stride 1. Compute the theoretical receptive field and the jump after each layer, and check your answer with the `receptive_field` function.

   Hint

   Start with $r = 1, j = 1$. After the first conv: $r = 7, j = 2$. After pooling: $r = 7 + 2 \cdot 2 = 11, j = 4$. Each following 3×3 layer adds $2 \cdot 4 = 8$, so $r = 19, 27, 35, 43$ with $j = 4$.

2. **Parameter budget.** For $C = 256$ input and output channels, compare the weights (ignore biases) of one 5×5 convolution, two stacked 3×3 convolutions, and a bottleneck (1×1 to 64 channels, 3×3 at 64, 1×1 back to 256). Which have the same receptive field?

   Hint

   $25C^2 \approx 1.64$M; $2 \cdot 9C^2 \approx 1.18$M; bottleneck $256 \cdot 64 + 9 \cdot 64^2 + 64 \cdot 256 \approx 0.07$M. The first two both see 5×5; the bottleneck sees 3×3 but is far cheaper, which is why ResNet-50 and deeper use it.

3. **Why the identity is hard without a shortcut.** Inside a network, a block's input usually comes from a previous ReLU, so it is non-negative. Show that for such inputs a residual block with $F \equiv 0$ outputs exactly $\mathbf{y} = \mathbf{x}$. Then explain what a plain block (conv–BN–ReLU–conv–BN–ReLU) would have to learn to output $\mathbf{y} = \mathbf{x}$. What does this imply for very deep plain networks?

   Hint

   For $\mathbf{x} \geq 0$, $\mathrm{ReLU}(\mathbf{x} + 0) = \mathbf{x}$. A plain block must instead learn weights that exactly undo two convolutions and two normalizations, a precise and fragile solution that gradient descent rarely finds. The residual block only needs $F = 0$, which weight decay and zero-initialization already push it toward. Deep plain networks therefore struggle to "do nothing" in extra layers, which matches the degradation the ResNet authors observed.

4. **ViT token count and cost.** For a 512×512 input, compute the number of tokens for patch sizes 32, 16 and 8, and the relative cost of one global attention layer ($\propto N^2$). How does Swin's windowed attention with 7×7 windows change the scaling?

   Hint

   $N = (512/P)^2$: 256, 1024, 4096 tokens; costs scale $1 : 16 : 256$. Windowed attention costs $N \cdot 49$ per layer (each token attends to 49 others), so it grows linearly with N.

5. **Learned versus hand-set filters.** Modify the digit CNN so that its first layer is frozen to four hand-set kernels (two Sobels, a Laplacian and a box blur) plus four random kernels. Train only the rest. Compare test accuracy with the fully learned network and inspect what happens to the random kernels if you unfreeze them.

   Hint

   Set `conv.weight.requires_grad_(False)` after assigning the weights, and pass only the remaining parameters to the optimizer. On this easy dataset both versions reach similar accuracy; the point is that hand-set derivative filters are a reasonable first layer, and that unfrozen random kernels tend to drift toward oriented, derivative-like patterns (compare Figure 4).

6. **Choose a recipe.** You have 2,000 labeled microscopy images and need (a) a per-image quality label, (b) per-pixel cell masks, (c) denoised images. For each, propose a backbone, a head and a training strategy using this tutorial's map, and say which would benefit most from a frozen self-supervised backbone.

   Hint

   (a) Frozen DINOv2/v3 or fine-tuned ConvNeXt + linear head. (b) U-Net, possibly with a pretrained encoder, or SAM prompted by points and then fine-tuned. (c) A full-resolution restoration network trained on synthetic noisy/clean pairs that match the microscope's noise. Frozen semantic features help (a) and (b) most; (c) depends on pixel-accurate statistics more than on semantics.

References

1. Shyandram, "深度學習的數位影像處理介紹 (An introduction to digital image processing with deep learning)," Medium post, 2024; updated 2026. link
2. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018. publisher page
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2008. publisher page
4. Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989. doi:10.1162/neco.1989.1.4.541
5. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi:10.1109/5.726791
6. O. Russakovsky, J. Deng, H. Su, et al., "ImageNet large scale visual recognition challenge," *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015. doi:10.1007/s11263-015-0816-y ·arXiv:1409.0575
7. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *NeurIPS*, 2012. proceedings
8. K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *ICLR*, 2015. arXiv:1409.1556

9. C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," *CVPR*, 2015. doi:10.1109/CVPR.2015.7298594 ·arXiv:1409.4842
10. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CVPR*, 2016. doi:10.1109/CVPR.2016.90 ·arXiv:1512.03385
11. A. Krizhevsky, "Learning multiple layers of features from tiny images," technical report, University of Toronto, 2009 (the CIFAR-10/100 datasets). dataset page
12. W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," *NeurIPS*, 2016. arXiv:1701.04128
13. J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," *CVPR*, 2015. arXiv:1411.4038
14. O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," *MICCAI*, 2015. arXiv:1505.04597
15. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *NeurIPS*, 2017. ML anthology
16. A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," *ICLR*, 2021. arXiv:2010.11929
17. Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical vision Transformer using shifted windows," *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00986 ·arXiv:2103.14030
18. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," *CVPR*, 2022, arXiv:2201.03545; and S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, "ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders," *CVPR*, 2023, arXiv:2301.00808.
19. M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision Transformers," *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00951 ·arXiv:2104.14294
20. K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," *CVPR*, 2022. doi:10.1109/CVPR52688.2022.01553 ·arXiv:2111.06377
21. M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024, arXiv:2304.07193; and O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, et al., "DINOv3," arXiv preprint, 2025, arXiv:2508.10104.
22. L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, "Vision Mamba: Efficient visual representation learning with bidirectional state space model," *ICML*, 2024, arXiv:2401.09417; and Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, J. Jiao, and Y. Liu, "VMamba: Visual state space model," *NeurIPS*, 2024, arXiv:2401.10166.
23. S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *NeurIPS*, 2015, arXiv:1506.01497; and N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with Transformers," *ECCV*, 2020, arXiv:2005.12872.
24. A. Kirillov, E. Mintun, N. Ravi, et al., "Segment anything," *ICCV*, 2023, arXiv:2304.02643; N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, et al., "SAM 2: Segment anything in images and videos,"

ICLR, 2025, arXiv:2408.00714; and N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, et al., "SAM 3: Segment anything with concepts," *ICLR*, 2026, arXiv:2511.16719.
25. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, et al., "Learning transferable visual models from natural language supervision," *ICML*, 2021. arXiv:2103.00020
26. H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," *NeurIPS*, 2023. arXiv:2304.08485
27. J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *NeurIPS*, 2020. arXiv:2006.11239
28. R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," *CVPR*, 2022, arXiv:2112.10752; and W. Peebles and S. Xie, "Scalable diffusion models with Transformers," *ICCV*, 2023, arXiv:2212.09748.
29. T. Brooks, A. Holynski, and A. A. Efros, "InstructPix2Pix: Learning to follow image editing instructions," *CVPR*, 2023. arXiv:2211.09800
30. Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A survey of convolutional neural networks: Analysis, applications, and prospects," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, 2022. doi:10.1109/TNNLS.2021.3084827 ·arXiv:2004.02806
31. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in vision: A survey," *ACM Computing Surveys*, vol. 54, no. 10s, 2022. doi:10.1145/3505244 ·arXiv:2101.01169
32. L. Jing and Y. Tian, "Self-supervised visual feature learning with deep neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, 2021. doi:10.1109/TPAMI.2020.2992393 ·arXiv:1902.06162
33. R. Balestriero, M. Ibrahim, V. Sobal, et al., "A cookbook of self-supervised learning," arXiv preprint, 2023. arXiv:2304.12210
34. M. Awais, M. Naseer, S. Khan, R. M. Anwer, H. Cholakkal, M. Shah, M.-H. Yang, and F. S. Khan, "Foundation models defining a new era in vision: A survey and outlook," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, 2025. doi:10.1109/TPAMI.2024.3506283 ·arXiv:2307.13721
35. J. Zhang, J. Huang, S. Jin, and S. Lu, "Vision-language models for vision tasks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, 2024. doi:10.1109/TPAMI.2024.3369699 ·arXiv:2304.00685
36. scikit-learn developers, "sklearn.datasets.load_digits" (a copy of the test set of the UCI optical recognition of handwritten digits dataset). docs

DLCV 02 • Deep Image Classification

How deep networks decide what an image shows: losses and metrics, datasets, the CNN and transformer architectures and why they were designed that way, modern training recipes, pre-training, robustness, calibration and interpretability.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-02-image-classification/

What you'll learn

- How image classification is posed precisely: single-label, multi-label and fine-grained tasks; logits, softmax and cross-entropy; top-1/top-5 accuracy; the confusion matrix and the metrics built on it (precision, recall, F1, MCC, macro and micro averages, ROC-AUC, average precision, mAP); and what it means for a classifier to be calibrated.
- What the standard training datasets (MNIST, CIFAR, ImageNet, iNaturalist) are for, which evaluation benchmarks test what (ImageNet-ReaL, ImageNetV2, robustness sets, fine-grained and transfer suites, long-tailed and zero-shot evaluation), and how to run an honest evaluation protocol.
- The design reasoning behind each major backbone, from AlexNet, VGG, Inception and ResNet to MobileNet, EfficientNet, RegNet, ConvNeXt, ViT, DeiT and Swin, with parameter and cost arithmetic you can check yourself.
- Which parts of a modern training recipe matter (augmentation, Mixup/CutMix, label smoothing, AdamW, warmup + cosine, stochastic depth, EMA) and why recipes alone moved ResNet-50 by several points.
- How pre-training changed the job: linear probes and fine-tuning, contrastive and masked self-supervision, DINOv2-style frozen features and CLIP zero-shot classification.
- How to evaluate honestly: distribution shift, adversarial examples, shortcut learning, temperature scaling, efficiency metrics and Grad-CAM.

The big picture

Image classification asks one question of a whole image: *which of these labels fits?* It is the oldest deep-learning benchmark task and still the place where most backbone networks are born. A detector, a segmenter or a restoration network usually starts from a backbone that was first trained, or at least evaluated, as an image classifier.

> **Explain like I'm five**
>
> Think of a child learning animals from a picture book. At first the child only sees blobs. With many pages, the child learns to spot ears, then faces, then "that's a cat". A deep classifier does the same in layers: early layers notice edges and colors, middle layers notice parts like eyes and wheels, and the

last layer votes for a label. Training is the parent saying "no, that's a dog" thousands of times. The rest of this tutorial is about building a better picture book (data), a better brain (architecture), a better way of teaching (training recipe), and better exams (evaluation), because a child who only ever saw cats on sofas may decide that "sofa" means "cat".

This tutorial assumes you know what a convolution, a CNN layer and backpropagation are. The classical side of classification (Bayes classifiers, perceptrons, a small CNN written in NumPy, softmax and cross-entropy derived by hand) is in IP 13. The broad history of deep learning for images is in DLCV 01. Here we go deep on classification itself. For a refresher on activation functions and optimizers, see ML 02 ·Neural Networks. A good single survey of the CNN side of this story is Li et al. [1].

The classification problem

Plain version. The network looks at an image and outputs one score per class. We turn scores into probabilities, compare them with the true label, and nudge the network so the right class gets more probability next time.

Single-label, multi-label and fine-grained

Let $\mathbf{x} \in \mathbb{R}^{H\times W\times 3}$ be an RGB image and $\mathcal{Y} = \{1,\dots,K\}$ a set of K classes. A network f_θ with parameters θ maps the image to a vector of **logits** $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$, one unnormalized score per class.

- **Single-label (multi-class)** classification assumes exactly one correct class $y \in \mathcal{Y}$. The logits are turned into a probability vector with the softmax,

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1,\dots,K,$$

where p_k is the model's estimate of $P(y = k \mid \mathbf{x})$. Training minimizes the **cross-entropy**, the negative log-probability of the true class, averaged over a mini-batch:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{B}\sum_{i=1}^{B} \log p_{y_i}(\mathbf{x}_i),$$

where B is the batch size and y_i the label of image $\mathbf{x}_i$. The gradient with respect to the logits is $\partial\mathcal{L}/\partial\mathbf{z} = \mathbf{p} - \mathbf{e}_y$ (per image), with $\mathbf{e}_y$ the one-hot vector of the true class; DIP Chapter 13 derives it.

- **Multi-label** classification allows any subset of labels (a street photo can contain "car", "person" and "bicycle"). Each class gets an independent sigmoid $\sigma(z_k) = 1/(1+e^{-z_k})$ and a binary cross-entropy, $-\sum_k [t_k \log \sigma(z_k) + (1 - t_k) \log(1 - \sigma(z_k))]$, where $t_k \in \{0, 1\}$ says whether class k is present. Metrics change too: mean average precision over classes replaces accuracy.
- **Fine-grained** classification separates visually similar subordinate classes, such as 200 bird species or 1,000 aircraft variants. The loss is the same as single-label, but the signal lives in small parts (a beak shape, a wing bar), classes are often long-tailed, and label noise from non-expert annotators is common. iNaturalist [6] is the standard large fine-grained benchmark.

Metrics: top-1, top-5 and beyond

Top-1 accuracy is the fraction of images whose highest-scoring class is the true one. **Top-k accuracy** counts an image as correct if the true class is among the k highest scores. The ImageNet challenge reported top-5 error because many images contain several objects, and some classes are nearly indistinguishable [2]. Top-5 hides real mistakes, so modern papers mostly report top-1. For imbalanced data, report **balanced accuracy** (the mean of per-class recalls) or per-class results, because a classifier can score high accuracy by ignoring rare classes.

```
import torch
import torch.nn.functional as F

logits = torch.tensor([[2.0, 1.0, 0.1, -1.0],    # 2 images, 4 classes
                       [0.5, 2.5, 2.4, 0.0]])
y = torch.tensor([0, 2])

p = logits.softmax(dim=1)
ce = F.cross_entropy(logits, y)                       # mean of -log p[i, y_i]
ce_ls = F.cross_entropy(logits, y, label_smoothing=0.1)
top1 = (logits.argmax(1) == y).float().mean()
top2 = (logits.topk(2, dim=1).indices == y[:, None]).any(1).float().mean()
print(p.round(decimals=3))
print(f"CE {ce:.3f}  CE+LS {ce_ls:.3f}  top-1 {top1:.2f}  top-2 {top2:.2f}")

# multi-label: one independent sigmoid per class, binary cross-entropy
targets = torch.tensor([[1., 0., 1., 0.], [0., 1., 1., 0.]])
print(f"BCE {F.binary_cross_entropy_with_logits(logits, targets):.3f}")
```

The second image is a near-tie between classes 1 and 2. Top-1 counts it as wrong; top-2 counts it as right. That one example is the whole argument for and against top-5.

The confusion matrix and the metrics built on it

Plain version. Accuracy collapses all mistakes into one number. A confusion matrix keeps them apart: it counts, for every true class, what the model said. Almost every other classification metric is a ratio of cells in that table, and each ratio answers a different question.

The binary case: TP, FP, FN, TN

Take one class as **positive** (say "sick") and the rest as **negative**. Each prediction lands in one of four cells:

	Predicted positive	Predicted negative
Actually positive	true positive (TP)	false negative (FN)
Actually negative	false positive (FP)	true negative (TN)

> **Explain like I'm five**
>
> A smoke alarm is a classifier. When there is a fire and it rings, that is a true positive. When you burn toast and it rings, that is a false positive: annoying, but nobody is hurt. When there is a fire and it stays quiet, that is a false negative: the dangerous one. When nothing happens and it stays quiet, that is a true negative. A good alarm is not just "right most of the time" (it is quiet almost every day, so that is easy). It must ring for fires and stay quiet for toast. Different metrics weigh those two goals differently.

With $N = \mathrm{TP} + \mathrm{FP} + \mathrm{FN} + \mathrm{TN}$ test examples, the standard ratios are

$$\text{accuracy} = \frac{\mathrm{TP} + \mathrm{TN}}{N}, \quad \text{precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \quad \text{recall (TPR)} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}},$$

$$\text{specificity (TNR)} = \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP}}, \quad \mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}} = 1 - \mathrm{TNR}.$$

- **Precision** answers "when the model says positive, how often is it right?" It matters when a false alarm is costly: a spam filter that hides real mail, an automatic product recall.
- **Recall**, also called **sensitivity** or the **true positive rate (TPR)**, answers "of all real positives, how many did we catch?" It matters when a miss is costly: cancer screening, a defect detector on a production line.
- **Specificity**, the **true negative rate (TNR)**, is recall for the negative class. The **false positive rate (FPR)** is its complement, the fraction of negatives wrongly flagged.

Precision and recall pull against each other: lowering the decision threshold flags more items, which raises recall and usually lowers precision. Single numbers that combine them are:

$$F_1 = \frac{2\,\mathrm{P}\,\mathrm{R}}{\mathrm{P}+\mathrm{R}} = \frac{2\,\mathrm{TP}}{2\,\mathrm{TP}+\mathrm{FP}+\mathrm{FN}}, \qquad F_\beta = \frac{(1+\beta^2)\,\mathrm{P}\,\mathrm{R}}{\beta^2\,\mathrm{P}+\mathrm{R}},$$

where P is precision, R recall and $\beta > 0$ says how many times more recall matters than precision (F_2 favours recall, $F_{0.5}$ favours precision). F_1 is the harmonic mean, so it is high only when *both* parts are high. Note what F_1 ignores: true negatives never appear in it.

Two further summaries use all four cells:

$$\text{balanced accuracy} = \frac{\mathrm{TPR}+\mathrm{TNR}}{2}, \qquad \mathrm{MCC} = \frac{\mathrm{TP}\cdot\mathrm{TN}-\mathrm{FP}\cdot\mathrm{FN}}{\sqrt{(\mathrm{TP}+\mathrm{FP})(\mathrm{TP}+\mathrm{FN})(\mathrm{TN}+\mathrm{FP})(\mathrm{TN}+\mathrm{FN})}}.$$

Balanced accuracy is the mean of the per-class recalls, so a classifier that ignores one class scores 0.5 however rare that class is. The **Matthews correlation coefficient** (MCC) is the Pearson correlation between the true and predicted binary labels: $+1$ is perfect, 0 is no better than chance, -1 is perfectly inverted. Chicco and Jurman [57] argue that it should be preferred over accuracy and F_1 for binary evaluation because it is high only when the classifier does well in all four cells. Sokolova and Lapalme [58] give a systematic comparison of these measures and of how each one reacts to changes in the confusion matrix.

Why accuracy misleads at 99:1. Suppose 10 of 1,000 scans are sick. A "model" that always answers *healthy* scores 99.0% accuracy. Model B flags 30 scans and catches 8 of the 10 sick ones; it scores *lower* accuracy (97.6%) yet is the only useful one:

```
import numpy as np
from sklearn.metrics import (accuracy_score, balanced_accuracy_score, confusion_matrix,
                             f1_score, matthews_corrcoef, precision_score, recall_score)

# 1,000 scans, 10 sick (label 1). Model A says "healthy" to everyone.
# Model B flags 30 scans and catches 8 of the 10 sick ones.
y = np.r_[np.ones(10), np.zeros(990)].astype(int)
pred_a = np.zeros(1000, dtype=int)
pred_b = np.r_[np.ones(8), np.zeros(2), np.ones(22), np.zeros(968)].astype(int)

for name, p in [("all-negative", pred_a), ("model B", pred_b)]:
    tn, fp, fn, tp = confusion_matrix(y, p).ravel()
    print(f"{name:12s} TP={tp} FP={fp} FN={fn} TN={tn}  "
          f"acc={accuracy_score(y, p):.3f} bal-acc={balanced_accuracy_score(y, p):.3f} "
          f"P={precision_score(y, p, zero_division=0):.3f} R={recall_score(y, p):.3f} "
          f"F1={f1_score(y, p, zero_division=0):.3f} MCC={matthews_corrcoef(y, p):.3f}")
# all-negative TP=0 FP=0 FN=10 TN=990  acc=0.990 bal-acc=0.500 P=0.000 R=0.000 F1=0.000
 ↪  MCC=0.000
# model B      TP=8 FP=22 FN=2 TN=968  acc=0.976 bal-acc=0.889 P=0.267 R=0.800 F1=0.400
 ↪  MCC=0.454
```

Balanced accuracy, F_1 and MCC all rank model B first; accuracy ranks it last. Rules of thumb: use accuracy only when classes are roughly balanced and all errors cost the same; report precision and recall (or F_β with a β chosen from the real costs) when one kind of error matters more; use balanced accuracy or MCC as a single imbalance-aware summary.

The multi-class confusion matrix

With K classes the confusion matrix $C \in \mathbb{N}^{K\times K}$ has entry C_{ij} = number of test images whose **true** class is i (row) and whose **predicted** class is j (column); this is the convention of scikit-learn [62]. The diagonal holds the correct predictions. Dividing each row by its sum gives the **row-normalized** matrix, whose diagonal is the per-class recall and whose off-diagonal entries show *which* classes are confused. Figure 1 shows one for a logistic regression trained on only 200 of scikit-learn's 8×8 digits: most mistakes are $3 \to 8$, $4 \to 1$ and $7 \to 9$, pairs that share strokes.

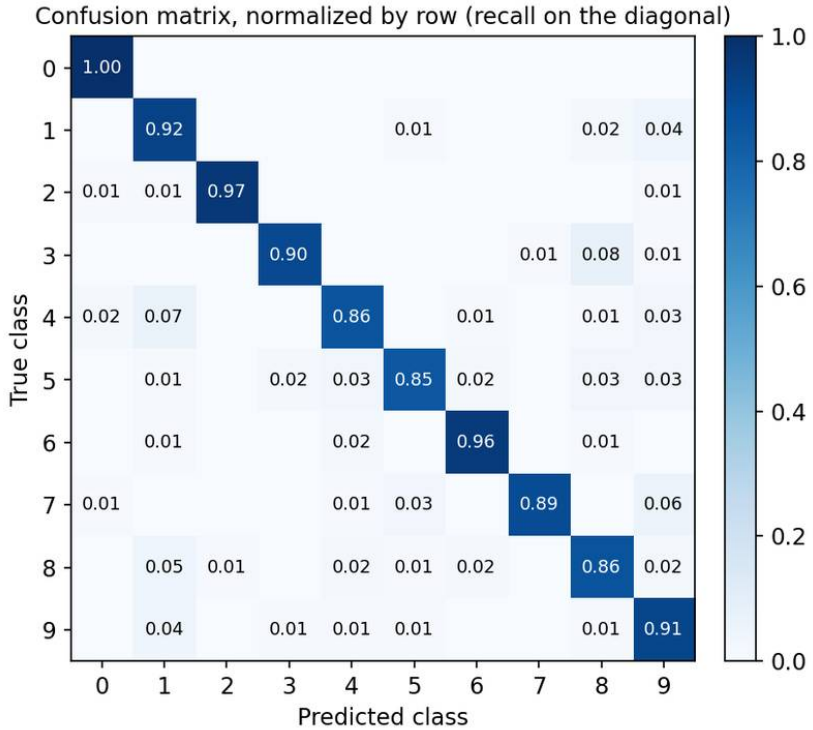


Figure 1 — Row-normalized confusion matrix of a logistic regression trained on 200 digits and tested on 899 (accuracy 91.1%). Each row sums to 1; the diagonal is the recall of each class, and the bright off-diagonal cells name the confusions worth fixing.

Treating each class k in turn as "positive" and all others as "negative" gives per-class counts: $\mathrm{TP}_k = C_{kk}$, $\mathrm{FP}_k = \sum_{i\neq k} C_{ik}$ (the rest of column k), $\mathrm{FN}_k = \sum_{j\neq k} C_{kj}$ (the rest of row k). Hence per-class precision $\mathrm{P}_k = C_{kk} / \sum_i C_{ik}$ and recall $\mathrm{R}_k = C_{kk} / \sum_j C_{kj}$. There are three common ways to turn K per-class numbers into one:

- **Macro average**: the plain mean over classes, $\mathrm{F}_1^{\text{macro}} = \frac{1}{K}\sum_k \mathrm{F}_{1,k}$. Every class counts equally, so rare classes weigh as much as common ones.
- **Micro average**: sum TP_k, FP_k, FN_k over classes first, then compute the metric once. In single-

label multi-class problems every error is one FP (for the predicted class) and one FN (for the true class), so micro precision = micro recall = micro F_1 = accuracy.

- **Weighted average**: the mean of per-class values weighted by each class's **support** n_k (number of true examples). It sits close to micro and hides rare-class failures almost as well.

A worked example. 100 test images from three classes with supports 80 (A), 15 (B) and 5 (C):

true \ predicted	A	B	C
A (80)	76	3	1
B (15)	4	10	1
C (5)	3	0	2

Column sums are 83, 13 and 4, so precision is $76/83 = 0.916$, $10/13 = 0.769$ and $2/4 = 0.500$; recall is $76/80 = 0.950$, $10/15 = 0.667$ and $2/5 = 0.400$; per-class F_1 is 0.933, 0.714 and 0.444. Then

$$\mathrm{F}_1^{\text{macro}} = \tfrac{1}{3}(0.933 + 0.714 + 0.444) = 0.697, \qquad \mathrm{F}_1^{\text{micro}} = \text{accuracy} = \tfrac{76+10+2}{100} = 0.880,$$

and the support-weighted F_1 is $0.80 \cdot 0.933 + 0.15 \cdot 0.714 + 0.05 \cdot 0.444 = 0.875$. The 18-point gap between micro and macro is the signal: the model is good on the common class and poor on the rare ones. Report macro when every class matters (rare diseases, rare species); report micro/accuracy when every *image* matters equally.

Cohen's kappa [59] corrects observed agreement for agreement expected by chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \qquad p_o = \frac{1}{N}\sum_k C_{kk}, \qquad p_e = \frac{1}{N^2}\sum_k \Big(\sum_j C_{kj}\Big)\Big(\sum_i C_{ik}\Big),$$

where p_o is the accuracy and p_e the accuracy a classifier would get by predicting each class at its observed rate independently of the input. It was designed for agreement between two human raters and is still common when comparing a model with annotators.

```
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (average_precision_score, classification_report,
 ↪  cohen_kappa_score,
                             confusion_matrix, precision_recall_fscore_support,
                              ↪  roc_auc_score)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import label_binarize

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.5, stratify=y, random_state=0)
clf = LogisticRegression(max_iter=2000).fit(X_tr[:200], y_tr[:200])   # only 200 labels
pred, prob = clf.predict(X_te), clf.predict_proba(X_te)

cm = confusion_matrix(y_te, pred)                     # rows = true, columns = predicted
```

```
cm_norm = confusion_matrix(y_te, pred, normalize="true")   # each row sums to 1 (Figure 1)
print(classification_report(y_te, pred, digits=3))    # per-class P, R, F1, support +
 ↪  averages
for avg in ["macro", "micro", "weighted"]:
    p, r, f, _ = precision_recall_fscore_support(y_te, pred, average=avg)
    print(f"{avg:8s} P={p:.3f} R={r:.3f} F1={f:.3f}")
print(f"kappa {cohen_kappa_score(y_te, pred):.3f}")
print(f"ROC-AUC (one-vs-rest, macro) {roc_auc_score(y_te, prob, multi_class='ovr'):.4f}")
Y = label_binarize(y_te, classes=range(10))           # one binary column per class
print(f"mean AP over classes {average_precision_score(Y, prob, average='macro'):.4f}")
# macro    P=0.914 R=0.911 F1=0.912
# micro    P=0.911 R=0.911 F1=0.911
# weighted P=0.915 R=0.911 F1=0.912
# kappa 0.901
# ROC-AUC (one-vs-rest, macro) 0.9958
# mean AP over classes 0.9742
```

Here macro and micro agree because the digit classes are balanced; on the A/B/C example above they would not.

Threshold-free evaluation: ROC and precision–recall curves

Every metric above needs a decision rule (argmax, or "positive if the score exceeds t"). A scoring classifier can be evaluated at *all* thresholds at once. Sweep t from high to low and record a point per threshold:

- The **ROC curve** plots TPR against FPR. Its area, **ROC-AUC**, equals the probability that a randomly chosen positive gets a higher score than a randomly chosen negative; 0.5 is chance, 1.0 is perfect separation. Fawcett [54] is the standard introduction.
- The **precision–recall (PR) curve** plots precision against recall. Its summary, **average precision** (AP), is the precision averaged over the recall levels at which a new positive is retrieved,

$$\mathrm{AP} = \sum_{n} (\mathrm{R}_n - \mathrm{R}_{n-1})\, \mathrm{P}_n,$$

where P_n and R_n are precision and recall at the n-th threshold, ordered by decreasing score (this is scikit-learn's definition [62]; benchmarks such as PASCAL VOC [61] have used interpolated variants, so check which one a paper reports).

Why PR under heavy imbalance. FPR divides by the number of negatives. When negatives outnumber positives 99 to 1, even thousands of false alarms barely move FPR, so the ROC curve looks excellent while precision is poor. Precision divides by the number of *flagged* items, so false alarms hit it directly, and the random baseline of AP equals the positive rate rather than 0.5. Davis and Goadrich [55] proved that a curve dominates in ROC space if and only if it dominates in PR space, and showed that optimizing ROC-AUC does not optimize AP; Saito and Rehmsmeier [56] showed with simulations and a literature review that PR plots are more informative than ROC

plots on imbalanced data. Figure 2 makes this concrete: two classifiers with the same ROC-AUC (0.95) on a task with 1% positives differ clearly in AP (0.45 against 0.36).

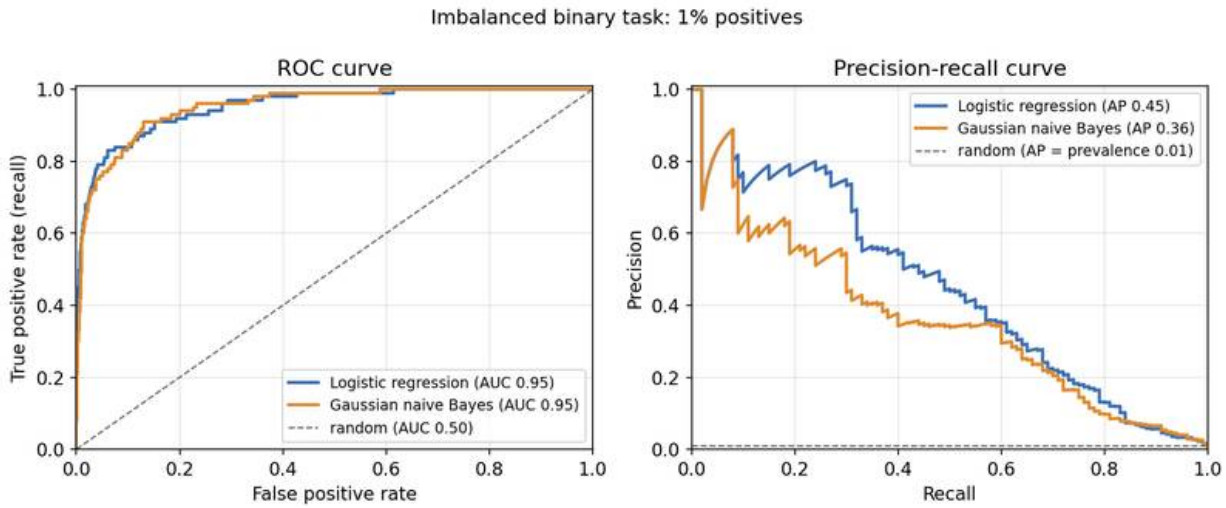


Figure 2 — ROC (left) and precision–recall (right) curves for two classifiers on a synthetic task with 1% positives (100 of 10,000 test points). Both have ROC-AUC 0.95, yet logistic regression keeps much higher precision at every recall level (AP 0.45 against 0.36). An all-negative classifier would score 99% accuracy here.

For $K > 2$ classes, both curves are computed **one-vs-rest**: class k is positive, all others negative, using the score p_k. The K areas are then macro-averaged (`roc_auc_score(..., multi_class="ovr")`), or micro-averaged by pooling all (k, image) pairs.

Multi-label metrics

In multi-label classification each image has a binary target vector $\mathbf{t} \in \{0,1\}^K$ and the model a score vector. Each label is its own binary problem, so the natural ranking metric is the **per-class AP** and its mean over classes, **mAP**, which became standard with the PASCAL VOC classification task [61]. Thresholded metrics come in two families, reviewed by Zhang and Zhou [60]:

- **Label-based** metrics compute a binary metric per label and macro- or micro-average over labels, exactly as in the multi-class case.
- **Example-based** metrics compute a score per *image* and average over images. Example-based F_1 is $\frac{1}{N}\sum_i \frac{2|\hat{Y}_i \cap Y_i|}{|\hat{Y}_i| + |Y_i|}$, where Y_i and $\hat{Y}_i$ are the true and predicted label sets of image i.
- **Hamming loss** is the fraction of wrong (image, label) bits, $\frac{1}{NK}\sum_i |\hat{Y}_i \,\triangle\, Y_i|$, where $\triangle$ is the symmetric difference. It is easy to read but, with many rare labels, an empty prediction already looks good.

```
import numpy as np
from sklearn.metrics import average_precision_score, f1_score, hamming_loss

# 4 street photos, 3 labels: car, person, bicycle
T = np.array([[1, 1, 0], [1, 0, 0], [0, 1, 1], [1, 1, 1]])            # ground truth
S = np.array([[.9, .6, .2], [.8, .4, .3], [.3, .7, .4], [.6, .2, .7]])  # sigmoid scores
P = (S >= 0.5).astype(int)                                             # threshold 0.5

print("per-class AP:", average_precision_score(T, S, average=None).round(3))
print(f"mAP {average_precision_score(T, S, average='macro'):.3f}")
print(f"example-based F1 {f1_score(T, P, average='samples'):.3f}  "
      f"macro F1 {f1_score(T, P, average='macro'):.3f}  micro F1 {f1_score(T, P,
      ↪ average='micro'):.3f}")
print(f"Hamming loss {hamming_loss(T, P):.3f}")
# per-class AP: [1.    0.917 1.   ]
# mAP 0.972
# example-based F1 0.867  macro F1 0.822  micro F1 0.857
# Hamming loss 0.167
```

Calibration: are the probabilities honest?

Accuracy only checks the argmax. **Calibration** checks whether the confidence means what it says: among all predictions made with confidence 0.8, about 80% should be right. Formally, a classifier is calibrated if $P(\hat{y} = y \mid \hat{p} = c) = c$ for every confidence c, where $\hat{y}$ is the predicted class and $\hat{p} = \max_k p_k$ its probability. The usual summary is the **expected calibration error**: split predictions into M confidence bins B_m and compute

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|,$$

where n is the number of test images, $\mathrm{acc}(B_m)$ the accuracy inside bin m and $\mathrm{conf}(B_m)$ the mean confidence there. Guo et al. [47] found that modern deep networks, trained long with cross-entropy, tend to be **overconfident**, and that a single scalar fix works remarkably well. We return to it in the robustness section.

Datasets and why they matter

Plain version. A benchmark is a frozen exam. It lets everyone compare methods fairly, but once a whole field studies for the same exam, scores start to measure the exam as much as the skill. It helps to keep two roles apart: datasets used to *train* a model and datasets used only to *evaluate* it.

Training datasets

Dataset	Images	Classes	Resolution	Role
MNIST [3]	60k train / 10k test	10 digits	28×28 gray	sanity check, teaching
CIFAR-10 / CIFAR-100 [4]	50k train / 10k test	10 / 100 (20 superclasses)	32×32 RGB	small-scale architecture and recipe studies
ImageNet-1k (ILSVRC 2012) [2]	about 1.28M train / 50k val	1,000	variable, usually trained at 224×224	the standard backbone benchmark
ImageNet-21k [5]	roughly 13–14M (versions differ)	about 21k	variable	larger-scale supervised pre-training
iNaturalist [6]	hundreds of thousands, by year	thousands of species	variable	fine-grained, long-tailed

MNIST is solved and now only catches bugs. CIFAR is small enough to run many controlled experiments, which is why regularization papers love it. ImageNet-1k became the reference after the 2012 challenge [2], [10]: architectures are usually reported as "top-1 on ImageNet-1k val at 224×224", and almost every pre-trained backbone in a model zoo has an ImageNet number attached. ImageNet-21k is the larger hierarchy from which ImageNet-1k was drawn; Ridnik et al. [5] made pre-training on it practical with a cleaned, semantic-hierarchy-aware recipe.

Evaluation benchmarks

Plain version. Training data teaches; evaluation data examines. A model that only ever sat one exam may have learned that exam. So modern papers grade a backbone on a *battery* of test sets, each built to probe one thing: the original distribution, a fresh sample of it, corrupted or stylized images, unusual viewpoints, fine-grained categories, small labelled budgets or rare classes.

The table lists the evaluation sets you will meet most often. Sizes are those stated by the papers or official pages cited.

Benchmark	What it measures	Size / classes	Usually reported
ImageNet-1k val [2]	in-distribution accuracy; the de facto test set, since ILSVRC test labels stay on the evaluation server	50,000 images, 1,000 classes	top-1 (top-5)
ImageNet-ReaL [8]	the same validation images with re-annotated, multi-label "reassessed" labels	46,837 images with 57,553 labels (3,163 images left without a label are discarded)	ReaL accuracy: top-1 counted correct if it is in the label set

Benchmark	What it measures	Size / classes	Usually reported
ImageNetV2 [7]	generalization to a new sample drawn by repeating the original collection process	10,000 images per variant (10 per class); three sampling variants, MatchedFrequency is the usual one	top-1
ImageNet-C [42]	robustness to synthetic corruptions	validation images × 15 corruptions × 5 severities	mean corruption error
ImageNet-R [43]	renditions: art, cartoons, sculpture, toys and more	30,000 images, 200 classes	top-1 over the 200 classes
ImageNet-A [44]	natural photos filtered to fool a ResNet-50	7,500 images, 200 classes	top-1 over the 200 classes
ImageNet-Sketch [63]	domain shift to black-and-white sketches	50,000 images, 1,000 classes (50 per class)	top-1
ObjectNet [64]	bias-controlled photos with varied rotation, background and viewpoint; test set only	50,000 images, 313 classes (113 overlap ImageNet)	top-1 on the overlapping classes
ImageNet-LT [73]	long-tailed training, balanced testing	115.8k training images, 1,000 classes, 5 to 1,280 per class; balanced test set	top-1, often split by class frequency
iNaturalist [6]	naturally long-tailed, fine-grained species	see the training table	top-1

Fine-grained and transfer benchmarks. A backbone is also judged by how well its features transfer to small, specialized datasets, either with a linear probe on frozen features or with fine-tuning (see the transfer-learning section below).

Dataset	Domain	Size / classes	Usually reported
CUB-200-2011 [65]	bird species, with part and attribute annotations	11,788 images, 200 classes	top-1
Stanford Cars [66]	car make, model and year	196 classes; 8,144 train / 8,041 test	top-1
FGVC-Aircraft [67]	aircraft variants in a three-level hierarchy	10,000 images, 100 variants	mean per-class accuracy
Oxford-IIIT Pets [68]	cat and dog breeds	37 classes, about 200 images each	mean per-class accuracy
Oxford Flowers-102 [69]	flower species	102 classes, 40 to 258 images each; 2,040 train / 6,149 test	mean per-class accuracy
Food-101 [70]	dishes; training labels deliberately left noisy	101,000 images, 101 classes (750 train / 250 test per class)	top-1
Caltech-101 [71]	generic objects	101 categories plus background, about 40 to 800 images each	mean per-class accuracy

Train/test sizes for Cars and Flowers follow the splits listed by Kornblith et al. [33]. Note that several of these benchmarks report **mean per-class accuracy** (macro-averaged recall) instead of

plain accuracy, because their test sets are not balanced; mixing the two in one comparison is a common error.

Two **suites** bundle such datasets so that one number summarizes transfer:

- Kornblith et al. [33] used **12 datasets**: Food-101, CIFAR-10, CIFAR-100, Birdsnap, SUN397, Stanford Cars, FGVC-Aircraft, PASCAL VOC 2007 classification, Describable Textures, Oxford-IIIT Pets, Caltech-101 and Oxford Flowers-102. They report top-1 accuracy, mean per-class accuracy, or 11-point mAP (VOC 2007) depending on the dataset, under linear probing, fine-tuning and training from scratch.
- **VTAB** [72] contains 19 tasks in three groups: 7 *natural* (standard camera images), 4 *specialized* (satellite and medical imagery) and 8 *structured* (scene geometry, such as counting or distance, mostly from simulated environments). In **VTAB-1k** each task provides only 1,000 labelled training examples (800 for training and 200 for validation), so it tests how well a representation adapts from few labels to tasks unlike its pre-training data.

Zero-shot evaluation. Image–text models such as CLIP [40] are evaluated without any training on the target dataset: the class names are turned into text prompts, and the image is assigned to the most similar prompt (see the CLIP section below). The CLIP paper benchmarked over 30 existing datasets this way, reported that zero-shot CLIP beat a linear probe on ResNet-50 features on 16 of 27 datasets, and found that an ensemble of 80 prompt templates improved ImageNet accuracy by 3.5 points over a single default prompt. Two cautions follow. Prompts and class-name wording are hyperparameters, so tuning them on the test set leaks test information exactly as tuning a learning rate on it would. And because the pre-training data are web-scale and mostly unpublished, "zero-shot" does not guarantee that the test images, or near-duplicates, were never seen.

How reliable are the numbers?

Three lines of work changed how we read ImageNet accuracy.

- **Reproduction.** Recht et al. [7] rebuilt test sets for CIFAR-10 and ImageNet by repeating the original collection procedure as closely as possible (ImageNetV2). Every model lost accuracy: 3–15% on CIFAR-10 and 11–14% on ImageNet. But the ranking was largely preserved, and gains on the old test set translated into gains on the new one. Their reading is that the drop comes from a subtle shift in difficulty between the old and new samples, not from the community overfitting to the test set by reusing it.
- **Labels.** Beyer et al. [8] re-annotated the ImageNet validation set with multiple labels per image and found that recent "progress" partly reflects fitting the idiosyncrasies of the original single-label annotations. Northcutt et al. [9] estimated that label errors are pervasive across common test sets and showed that they can change which model looks best.

- **Bias.** Every dataset has a "signature" in viewpoint, background and photographer style. DIP Chapter 13 reviews the dataset-bias studies; the practical lesson is to test on independently collected data whenever possible.

Read a benchmark number as a claim about a benchmark

"81.0% top-1" means: on these 50,000 particular validation images, with this preprocessing and this label set. Report the resolution, the number of crops, whether extra data or distillation was used, and the variance across seeds when differences are under a point.

Evaluation protocol

A benchmark number is only as honest as the protocol behind it.

- **Three splits, three jobs.** Fit parameters on the **training** set. Choose hyperparameters, early stopping, temperature, thresholds and prompts on a **validation** set (or by cross-validation inside the training data). Touch the **test** set once, at the end. When a benchmark only publishes a validation split, as with ImageNet-1k, hold out part of the training set for tuning.
- **Never tune on the test set.** Every decision made after looking at test results (an extra augmentation, another epoch, a different checkpoint) fits the test set a little. Repeated over a field's worth of papers, this is **test-set reuse**, or adaptive overfitting to a benchmark. ImageNetV2 [7] found that ranking was largely preserved on a fresh sample, which is reassuring, but the same study also shows that absolute numbers do not transfer and that a fresh test set is the only direct check.
- **Report variation.** Train with several random seeds (at least three, more for small datasets) and report mean $\pm$ standard deviation, or a confidence interval. With 10,000 test images, ImageNetV2 [7] quotes 95% intervals of about $\pm 1\%$; a 0.3-point gain on such a set is not a result.
- **Report the whole recipe.** Resolution, number of crops, extra data, distillation, which variant of a metric (top-1 or mean per-class accuracy, which AP definition) and which version of a label set (original or ReaL).

CNN architectures and the reasons behind them

Plain version. Each famous CNN fixed one specific problem of its predecessors: training too slow, too many parameters, too hard to optimize when deep, too expensive on a phone. Knowing the problem each one solved is more useful than memorizing their layer tables.

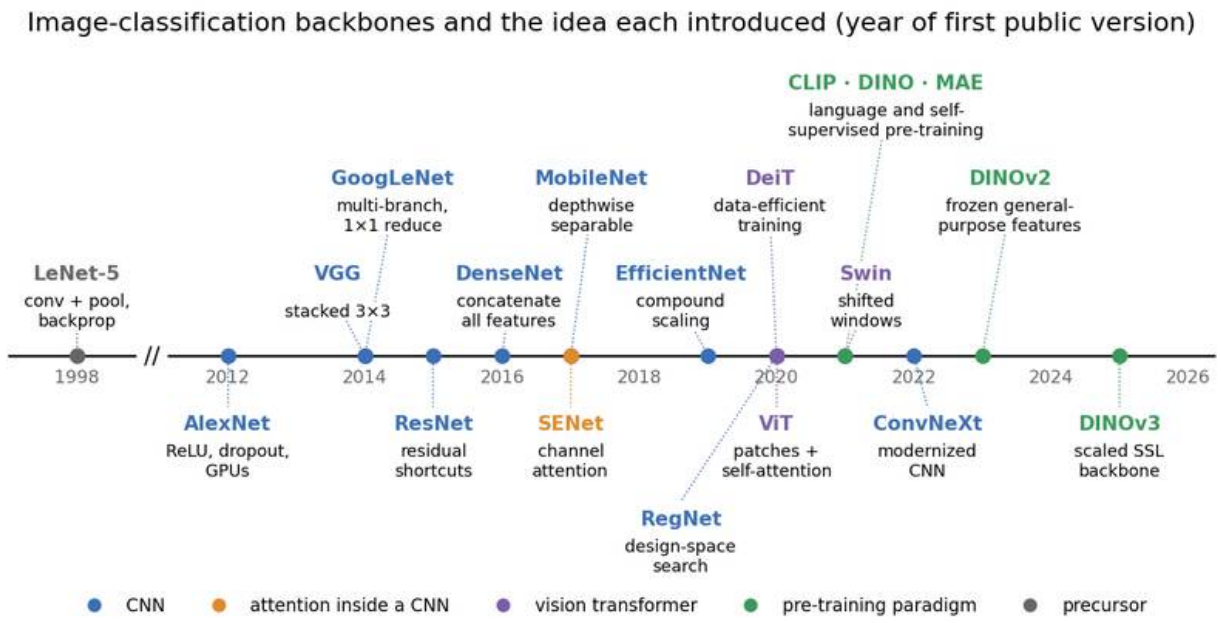


Figure 3 — Image-classification backbones and the main idea each introduced. Dates are the year of the first public version (usually arXiv). Colors separate pure CNNs, attention inside CNNs, vision transformers and pre-training paradigms.

AlexNet: depth becomes trainable

Before 2012, the best ImageNet systems used hand-crafted features (SIFT, Fisher vectors) with linear classifiers. AlexNet [10] was a five-convolution, three-fully-connected network trained end to end on ImageNet. Three ingredients made it work at that scale. **ReLU** activations, $\max(0, z)$, do not saturate for positive inputs, so gradients do not vanish the way they do with tanh, and the paper reports several times faster training. **Dropout** in the large fully connected layers fought overfitting; those layers held most of the parameters. **GPUs** made it feasible: the model was split across two GPUs. Heavy data augmentation (random crops and flips, color perturbation) was the fourth, often forgotten, ingredient. The design lesson: with enough data and compute, learned features beat engineered ones.

VGG: small kernels, stacked

VGG [11] asked a cleaner question: what happens if we only use 3×3 convolutions and go deeper (16–19 weight layers)? Two stacked 3×3 layers see a 5×5 region; three see 7×7. For C input and output channels,

$$\underbrace{2 \cdot 3^2 C^2}_{\text{two } 3\times3} = 18C^2 \;<\; \underbrace{5^2 C^2}_{\text{one } 5\times5} = 25C^2,$$

ignoring biases. So the stack has fewer parameters for the same receptive field, and it inserts an extra nonlinearity between the two layers. VGG's uniform design made it a favourite feature extractor for years. Its weakness is cost: the fully connected head and the wide early layers make it heavy.

GoogLeNet / Inception: width, branches and 1×1 bottlenecks

GoogLeNet [12] attacked cost instead. Each **Inception module** runs 1×1, 3×3, 5×5 convolutions and pooling in parallel and concatenates the outputs, so the next layer can choose the right scale.

Running big kernels on many channels would be expensive, so 1×1 **convolutions** first reduce the channel count. A 1×1 convolution is a per-pixel linear map across channels: it mixes channels without looking at neighbours, and it is the cheapest way to change width. GoogLeNet also replaced the large fully connected head with **global average pooling**. The follow-up Inception-v3 paper [13] factorized large kernels into stacks or into $1 \times n$ and $n \times 1$ pairs, and introduced **label smoothing**, which we meet again in the training section.

ResNet: learning residuals

The problem. Stacking more plain layers eventually made networks *worse*, even on the training set. This "degradation" is an optimization problem, not overfitting: a deeper network could in principle copy a shallower one and set the extra layers to identity, yet SGD did not find that solution.

The idea. He et al. [14] let a block learn a residual $\mathcal{F}$ and add its input back:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}; \{W_i\}) + \mathbf{x},$$

where $\mathbf{x}$ is the block input, $\mathcal{F}$ is two or three convolution layers with weights W_i, and $\mathbf{y}$ the output. If the best mapping is close to identity, the block only has to push $\mathcal{F}$ toward zero, which is easy. When the shapes differ (more channels, stride 2), the shortcut becomes a 1×1 projection.

Why it trains. Unroll L blocks of the pre-activation form $\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l)$, as analysed in the follow-up paper on identity mappings [15]:

$$\mathbf{x}_L = \mathbf{x}_l + \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i), \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L}\Big(\mathbf{I} + \frac{\partial}{\partial \mathbf{x}_l} \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i)\Big),$$

where $\mathcal{L}$ is the loss and $\mathbf{I}$ the identity matrix. The identity term carries the gradient from the top straight down to any block, so it cannot vanish through a product of many small Jacobians. That paper also found that keeping the shortcut a *pure* identity (moving BN and ReLU inside the residual branch, "pre-activation") trains even deeper networks more easily.

The bottleneck. For deeper ResNets (50 layers and up) each block becomes 1×1 (reduce) $\to 3\times3$ $\to 1\times1$ (expand), shown in Figure 4(c). At 256 channels, a basic block of two 3×3 convolutions has $2\cdot9\cdot256^2 \approx 1.18$M weights; a bottleneck that reduces to 64 channels has $256\cdot64+9\cdot64^2+64\cdot256 \approx$ 70k. The expensive 3×3 runs on a thin tensor, which is what makes 50–152 layers affordable.

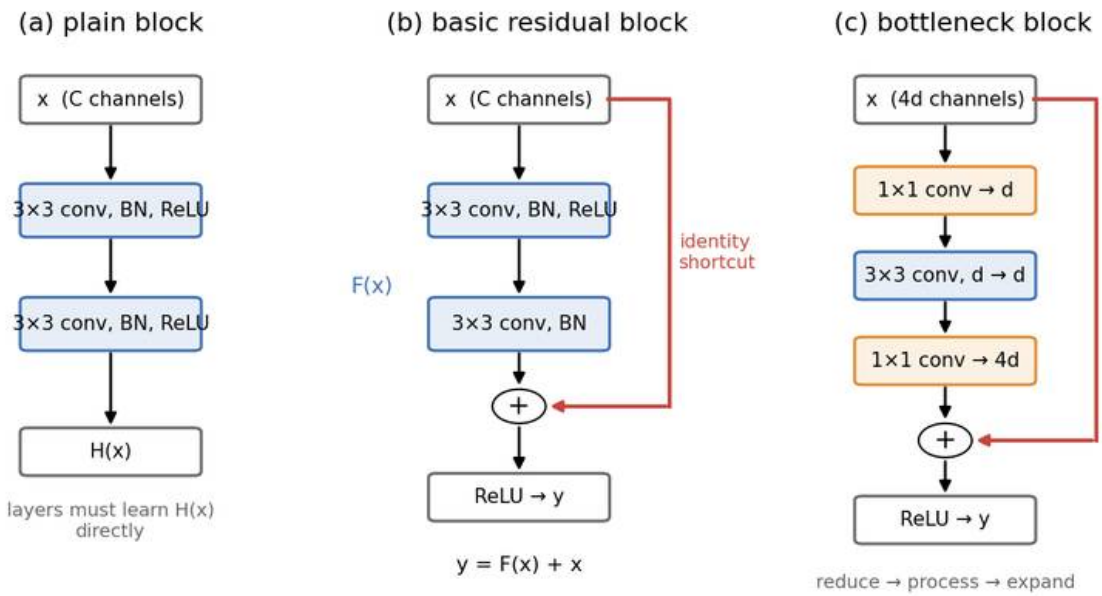


Figure 4 — (a) A plain block must learn the full mapping $H(x)$. (b) A residual block learns $F(x) = H(x) - x$ and adds the input back through an identity shortcut. (c) The bottleneck block runs the 3×3 convolution on a reduced width d and expands back to $4d$.

The basic block used in this tutorial's experiments, written in PyTorch:

```
import torch.nn as nn
import torch.nn.functional as F

class BasicBlock(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(cin, cout, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(cout)
        self.conv2 = nn.Conv2d(cout, cout, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(cout)
        self.short = nn.Identity()
        if stride != 1 or cin != cout:   # projection shortcut when shapes change
            self.short = nn.Sequential(nn.Conv2d(cin, cout, 1, stride, bias=False),
             ↪  nn.BatchNorm2d(cout))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
```

```
        return F.relu(out + self.short(x))   # y = F(x) + x
```

DenseNet: reuse everything

DenseNet [16] pushes feature reuse further: inside a dense block, layer ℓ receives the **concatenation** of all earlier feature maps, $\mathbf{x}_\ell = H_\ell([\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{\ell-1}])$, where $[\cdot]$ is channel concatenation and H_ℓ is BN–ReLU–conv. Each layer adds only a few channels (the "growth rate"), so the network is parameter-efficient, and every layer has a short path to the loss. The cost is memory: concatenated activations grow with depth, which made DenseNets slower in practice than their parameter count suggests. A useful lesson for later: parameters, FLOPs and wall-clock time are three different things.

MobileNet: factorize the convolution

MobileNet [17] targets phones. A standard $K \times K$ convolution from M to N channels on an $H \times W$ map costs HWK^2MN multiply–adds. A **depthwise separable** convolution splits it into a **depthwise** $K \times K$ convolution that filters each channel separately (HWK^2M) and a **pointwise** 1×1 convolution that mixes channels ($HWMN$). The cost ratio is

$$\frac{HWK^2M + HWMN}{HWK^2MN} = \frac{1}{N} + \frac{1}{K^2},$$

so with $K = 3$ and many output channels the separable version is roughly 8–9 times cheaper. The bet is that spatial filtering and channel mixing do not need to be learned jointly. That bet held up so well that depthwise convolutions reappear in EfficientNet and ConvNeXt.

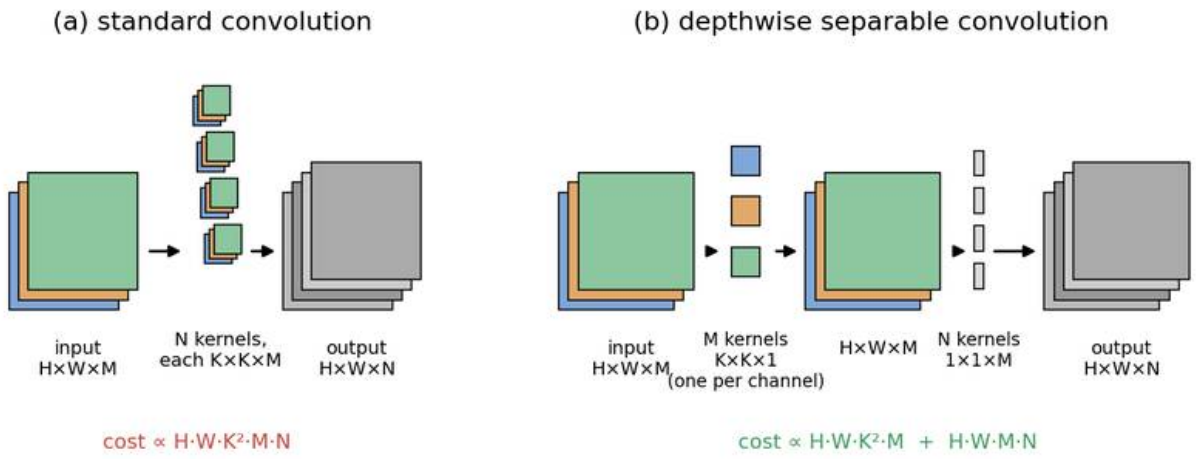


Figure 5 — Standard convolution filters and mixes channels in one step. Depthwise separable convolution filters each channel alone (depthwise) and then mixes channels with 1×1 kernels (pointwise).

```
import torch.nn as nn

def n_params(m):
    return sum(p.numel() for p in m.parameters())

M, N, K = 128, 128, 3
standard = nn.Conv2d(M, N, K, padding=1, bias=False)
separable = nn.Sequential(nn.Conv2d(M, M, K, padding=1, groups=M, bias=False),   # depthwise
                          nn.Conv2d(M, N, 1, bias=False))                          #
                           ↪ pointwise
print("standard:", n_params(standard), " separable:", n_params(separable),
      f" ratio {n_params(separable) / n_params(standard):.3f} vs 1/N + 1/K^2 = {1/N +
       ↪ 1/K**2:.3f}")
# standard: 147456  separable: 17536  ratio 0.119 vs 1/N + 1/K^2 = 0.119
```

EfficientNet: scale depth, width and resolution together

Given a good small network, how should we make it bigger? Deeper, wider, or higher input resolution? Tan and Le [18] argued these interact: a higher resolution image needs more depth (larger receptive field) and more width (finer patterns). **Compound scaling** grows all three with one coefficient ϕ:

$$d = \alpha^{\phi}, \quad w = \beta^{\phi}, \quad r = \gamma^{\phi}, \qquad \text{s.t. } \alpha\beta^2\gamma^2 \approx 2, \;\; \alpha, \beta, \gamma \geq 1,$$

where d, w, r multiply depth, width and resolution. FLOPs scale roughly as $d\,w^2 r^2$, so each unit increase of ϕ roughly doubles the cost. The constants are found by a small grid search on the base network (itself found by architecture search, using inverted-bottleneck blocks with depthwise convolutions and squeeze-and-excitation). The idea, "scale in a balanced way", outlived the specific constants.

RegNet: design spaces instead of single designs

Radosavovic et al. [19] studied populations of networks rather than single winners. Starting from a broad space of ResNet-like networks, they repeatedly constrained it (for example, shared bottleneck ratios, widths that increase across stages) and checked whether the *distribution* of errors improved. The surviving space, RegNet, describes stage widths with a simple quantized linear rule. The lesson is methodological: good architectures follow simple regularities, and comparing distributions of models is more reliable than comparing one tuned model against another.

ConvNeXt: a CNN modernized step by step

After vision transformers took over the leaderboards, Liu et al. [20] asked how much of their advantage came from attention and how much from everything else. They started from a ResNet-50 trained with a transformer-style recipe and changed one thing at a time: stage ratios like Swin's, a "patchify" stem (a 4×4 stride-4 convolution), depthwise convolutions with larger 7×7 kernels, an inverted bottleneck, fewer activations and normalizations, GELU and LayerNorm. The result, ConvNeXt, matched Swin-style transformers of similar cost on ImageNet and downstream tasks. The lesson: architecture families matter less than the many small design and training choices that come with them.

Attention and transformers

Plain version. A convolution only looks at a small neighbourhood with fixed weights. Attention lets each position decide, based on content, which other positions to listen to. Vision transformers apply this to image patches.

Squeeze-and-excitation: attention inside a CNN

The SE block [21] adds cheap, content-dependent **channel** weights to any CNN. It "squeezes" each channel to one number by global average pooling, passes the vector through a small two-layer MLP, and multiplies each channel by a sigmoid gate:

$$\mathbf{s} = \sigma(W_2\, \delta(W_1\, \bar{\mathbf{u}})), \qquad \tilde{\mathbf{u}}_c = s_c\, \mathbf{u}_c,$$

where $\mathbf{u}_c$ is channel c of the feature map, $\bar{\mathbf{u}}$ the vector of channel means, δ a ReLU, W_1 reduces the dimension by a ratio r and W_2 restores it. The network can now emphasise "fur" channels when the global context says "animal". SE blocks are part of EfficientNet and many mobile networks.

```
import torch
import torch.nn as nn

class SE(nn.Module):
    """Squeeze-and-excitation: reweight channels using global context."""
    def __init__(self, c, r=16):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(c, c // r), nn.ReLU(), nn.Linear(c // r, c),
         ↪  nn.Sigmoid())

    def forward(self, x):                       # x: (B, C, H, W)
        s = x.mean(dim=(2, 3))                  # squeeze: (B, C)
        w = self.fc(s)[:, :, None, None]        # excitation: (B, C, 1, 1) in (0, 1)
        return x * w

print(SE(64)(torch.randn(2, 64, 8, 8)).shape)
```

ViT: an image as a sequence of patches

The Vision Transformer [22] removes convolutions almost entirely. An image $\mathbf{x} \in \mathbb{R}^{H\times W\times C}$ is cut into $N = HW/P^2$ non-overlapping $P\times P$ patches. Each patch is flattened and linearly projected to a D-dimensional **token**. A learnable **class token** is prepended and learnable **position embeddings** are added:

$$\mathbf{z}_0 = [\,\mathbf{x}_{\text{class}};\ \mathbf{x}_p^1\mathbf{E};\ \dots;\ \mathbf{x}_p^N\mathbf{E}\,] + \mathbf{E}_{\text{pos}},$$

where $\mathbf{x}_p^i \in \mathbb{R}^{P^2C}$ is the i-th flattened patch, $\mathbf{E} \in \mathbb{R}^{P^2C\times D}$ the patch embedding and $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1)\times D}$ the position embeddings. A stack of L standard transformer encoder layers (multi-head self-attention and MLP, each with LayerNorm and a residual connection) processes the sequence, and a small head reads the final class token to produce logits. Self-attention for one head is

$$\text{Attn}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\Big)\mathbf{V},$$

with $\mathbf{Q} = \mathbf{Z}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{Z}\mathbf{W}_K$, $\mathbf{V} = \mathbf{Z}\mathbf{W}_V$ linear projections of the token matrix $\mathbf{Z}$, and d the key dimension. The attention matrix is $(N+1)\times(N+1)$: every patch can attend to every other patch from the first layer, so the receptive field is global immediately.

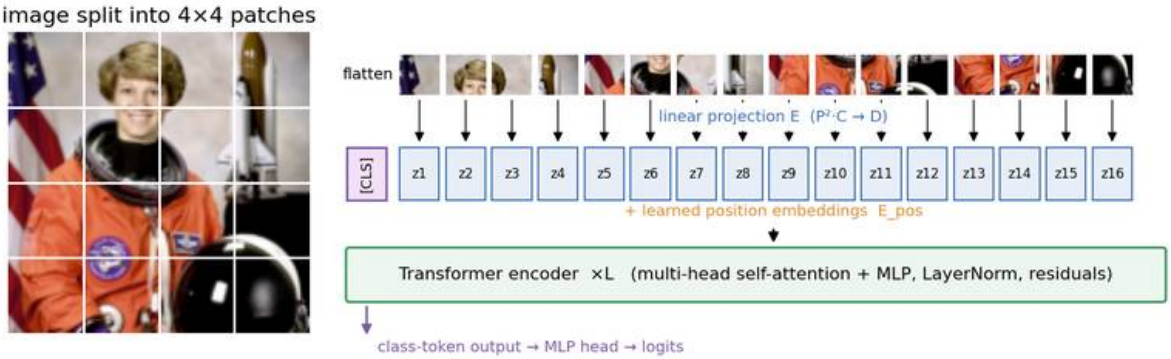


Figure 6 — The ViT input pipeline. Patches become tokens through one shared linear projection (equivalently, a $P\times P$ convolution with stride P). A class token and learned position embeddings are added, and the class token's final state is classified.

```
import torch
import torch.nn as nn

B, C, H, W, P, D = 2, 3, 32, 32, 8, 64
img = torch.randn(B, C, H, W)

# 1) patchify: (B, C, H, W) -> (B, N, P*P*C) with N = (H/P)*(W/P)
patches = img.unfold(2, P, P).unfold(3, P, P)                 # (B, C, H/P, W/P, P, P)
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(B, -1, C * P * P)
N = patches.shape[1]
```

```
# 2) linear patch embedding + class token + learned position embedding
embed = nn.Linear(C * P * P, D)          # same as nn.Conv2d(C, D, kernel_size=P, stride=P)
cls = nn.Parameter(torch.zeros(1, 1, D))
pos = nn.Parameter(torch.randn(1, N + 1, D) * 0.02)
z = torch.cat([cls.expand(B, -1, -1), embed(patches)], dim=1) + pos   # (B, N+1, D)

# 3) one head of scaled dot-product self-attention
Wq, Wk, Wv = (nn.Linear(D, D, bias=False) for _ in range(3))
q, k, v = Wq(z), Wk(z), Wv(z)
A = (q @ k.transpose(1, 2) / D ** 0.5).softmax(dim=-1)        # (B, N+1, N+1), rows sum to 1
out = A @ v
print("tokens:", N + 1, " attention:", tuple(A.shape), " class-token output:", tuple(out[:,
 ↪  0].shape))
```

Why ViT needs data. A CNN has built-in assumptions, called **inductive biases**: locality (a pixel mostly relates to its neighbours) and translation equivariance (the same filter everywhere). ViT has almost none; apart from patch extraction and position embeddings, it must learn spatial structure from data. The ViT paper [22] reports that, trained on ImageNet-1k alone, ViTs fell behind comparable ResNets, but pre-trained on much larger datasets (ImageNet-21k or the 300M-image JFT) they matched or beat them. Weak priors cost data at small scale and stop limiting the model at large scale. The same paper also tried **hybrid** models, feeding the feature map of a ResNet stem into the transformer instead of raw patches; hybrids helped at small compute budgets and the gap closed as models grew.

DeiT: training ViTs on ImageNet alone

DeiT [23] showed that the "ViT needs huge data" conclusion was partly a statement about recipes. With strong augmentation (RandAugment, Mixup, CutMix, random erasing), stochastic depth, repeated augmentation and AdamW, a ViT-B trained only on ImageNet-1k reached 83.1% top-1.

DeiT also added a **distillation token**: a second learnable token, like the class token, whose output is trained to match a teacher network's predictions (they found a convnet teacher worked best), while the class token is trained on the true label. This is a transformer-specific form of knowledge distillation [51]. At test time the two heads' outputs are combined.

Swin: windows, shifts and a hierarchy

Global attention costs grow quadratically with the number of tokens, which is a problem for high-resolution inputs and for dense tasks. Swin [24] computes attention only inside non-overlapping $M \times M$ windows, and shifts the window grid by half a window in alternating layers so information crosses window borders. Token count is reduced between stages by merging 2×2 neighbouring patches, giving a CNN-like pyramid with strides 4, 8, 16, 32. For an $h \times w$ token map with C channels, the paper gives

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C, \qquad \Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC,$$

so windowed attention is linear in the number of tokens hw for a fixed window size M. Swin brought back two CNN priors, locality and hierarchy, and became a general-purpose backbone for detection and segmentation as well as classification. The broad landscape of vision transformers is surveyed by Khan et al. [25].

Training recipes

Plain version. The same network can gain or lose several points of accuracy depending on how it is trained. A modern recipe is a bundle of small tricks that each make memorizing harder or optimization smoother.

Data augmentation

- **Random resized crop** samples a crop covering a random fraction of the image area and a random aspect ratio, then resizes it to the training resolution. It teaches scale and position invariance and is the default for ImageNet training. **Horizontal flips** are added when they are label-preserving (they are for most photos, not for text or digits).
- **Mixup** [26] blends two images and their labels: $\tilde{\mathbf{x}} = \lambda \mathbf{x}_i + (1-\lambda)\mathbf{x}_j$ and $\tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1-\lambda)\mathbf{y}_j$, with $\lambda \sim \text{Beta}(\alpha, \alpha)$ and $\mathbf{y}$ one-hot. It encourages linear behaviour between training examples.
- **CutMix** [27] pastes a random rectangle from image j into image i and mixes the labels in proportion to the pasted area. Unlike Mixup, every pixel stays natural, and the network must recognize objects from partial views.
- **RandAugment** [28] replaces learned augmentation policies with two numbers: apply N random operations (rotate, shear, posterize, contrast, ...) at a shared magnitude M. Its paper reports that this tiny search space matches or beats earlier automated policies, and that the right strength depends on model and dataset size.

```
import numpy as np
import torch
import torch.nn.functional as F

def mixup(x, y, num_classes, alpha=0.2):
    lam = float(np.random.beta(alpha, alpha))
    idx = torch.randperm(x.size(0))
    y1 = F.one_hot(y, num_classes).float()
    return lam * x + (1 - lam) * x[idx], lam * y1 + (1 - lam) * y1[idx]

def cutmix(x, y, num_classes, alpha=1.0):
    lam = float(np.random.beta(alpha, alpha))
```

```
    idx = torch.randperm(x.size(0))
    H, W = x.shape[-2:]
    h, w = int(H * np.sqrt(1 - lam)), int(W * np.sqrt(1 - lam))
    cy, cx = np.random.randint(H), np.random.randint(W)
    y0, y1_, x0, x1_ = max(cy - h // 2, 0), min(cy + h // 2, H), max(cx - w // 2, 0), min(cx
     ↪  + w // 2, W)
    x = x.clone()
    x[..., y0:y1_, x0:x1_] = x[idx][..., y0:y1_, x0:x1_]
    lam = 1 - (y1_ - y0) * (x1_ - x0) / (H * W)            # actual area kept
    y1 = F.one_hot(y, num_classes).float()
    return x, lam * y1 + (1 - lam) * y1[idx]

def soft_ce(logits, soft_targets):
    return -(soft_targets * logits.log_softmax(1)).sum(1).mean()
```

Label smoothing

Cross-entropy with one-hot targets keeps pushing the true logit up forever; the loss only reaches zero at infinite margin. **Label smoothing** [13] replaces the target with

$$\mathbf{q} = (1 - \varepsilon)\,\mathbf{e}_y + \frac{\varepsilon}{K}\mathbf{1},$$

where ε (often 0.1) is the smoothing strength, $\mathbf{e}_y$ the one-hot vector and $\mathbf{1}$ the all-ones vector. The optimum is now a finite logit gap, which regularizes the network and usually reduces overconfidence. In PyTorch it is one argument: `F.cross_entropy(logits, y, label_smoothing=0.1)`.

Optimizer, schedule and regularizers

- **AdamW** [29] decouples weight decay from the adaptive gradient step: instead of adding $\lambda\theta$ to the gradient (which Adam then rescales per parameter), it shrinks the weights directly, $\theta \leftarrow \theta - \eta(\hat{\mathbf{m}}/(\sqrt{\hat{\mathbf{v}}} + \epsilon) + \lambda\theta)$, where η is the learning rate, $\hat{\mathbf{m}}, \hat{\mathbf{v}}$ Adam's moment estimates and λ the decay. It is the default for transformers and for modern CNN recipes. Biases and normalization parameters are usually excluded from decay.
- **Warmup + cosine schedule.** Large learning rates are unstable in the first steps, when normalization statistics and Adam's moment estimates are still poor, so the learning rate rises linearly for a few epochs. It then follows a half cosine down to near zero, $\eta_t = \frac{1}{2}\eta_{\max}(1 + \cos(\pi t/T))$ with t the step after warmup and T the remaining steps, an idea popularized by SGDR [30].
- **Stochastic depth** [31] randomly skips whole residual branches during training (the block becomes identity), with a skip probability that grows with depth. It shortens the effective depth during training and acts like an ensemble of shallower networks.

- **EMA of weights** keeps an exponential moving average $\bar{\theta} \leftarrow \mu\bar{\theta} + (1-\mu)\theta$ (with μ close to 1, e.g. 0.999) and evaluates $\bar{\theta}$. It smooths out the noise of the last SGD steps.

```
import math
import torch

def warmup_cosine(step, total, warmup):
    if step < warmup:
        return (step + 1) / warmup                      # linear warmup
    p = (step - warmup) / max(1, total - warmup)
    return 0.5 * (1 + math.cos(math.pi * p))            # cosine decay to 0

model = torch.nn.Linear(10, 3)
decay = [p for n, p in model.named_parameters() if p.ndim > 1]      # weights
no_decay = [p for n, p in model.named_parameters() if p.ndim <= 1]  # biases, norm scales
opt = torch.optim.AdamW([{"params": decay, "weight_decay": 0.05},
                         {"params": no_decay, "weight_decay": 0.0}], lr=1e-3)
total = 1000
sched = torch.optim.lr_scheduler.LambdaLR(opt, lambda s: warmup_cosine(s, total, warmup=50))
ema = torch.optim.swa_utils.AveragedModel(model,
 ↪  multi_avg_fn=torch.optim.swa_utils.get_ema_multi_avg_fn(0.999))
for step in range(total):
    loss = model(torch.randn(16, 10)).pow(2).mean()
    opt.zero_grad(); loss.backward(); opt.step(); sched.step(); ema.update_parameters(model)
```

A small experiment

To see the pieces working together, we trained a 78k-parameter ResNet ("TinyResNet": a stem, three basic blocks with 16, 32 and 64 channels, global average pooling) on scikit-learn's 8×8 digits, upsampled to 32 × 32 so that the strided stages have room to work. To make overfitting visible we used only **100 training images**, 500 images for calibration and 1,197 for testing, with AdamW (weight decay 0.05), batch size 32, 40 epochs and two epochs of warmup before a cosine decay. One run used no augmentation. The other used random rotation ($\pm 15°$), scaling and shifts (a random-resized-crop-like affine; no flips, since a flipped digit is a different symbol) together with label smoothing 0.1. All code is in `scripts/figures/dl_02_image_classification.py` and runs on a CPU in about a minute.

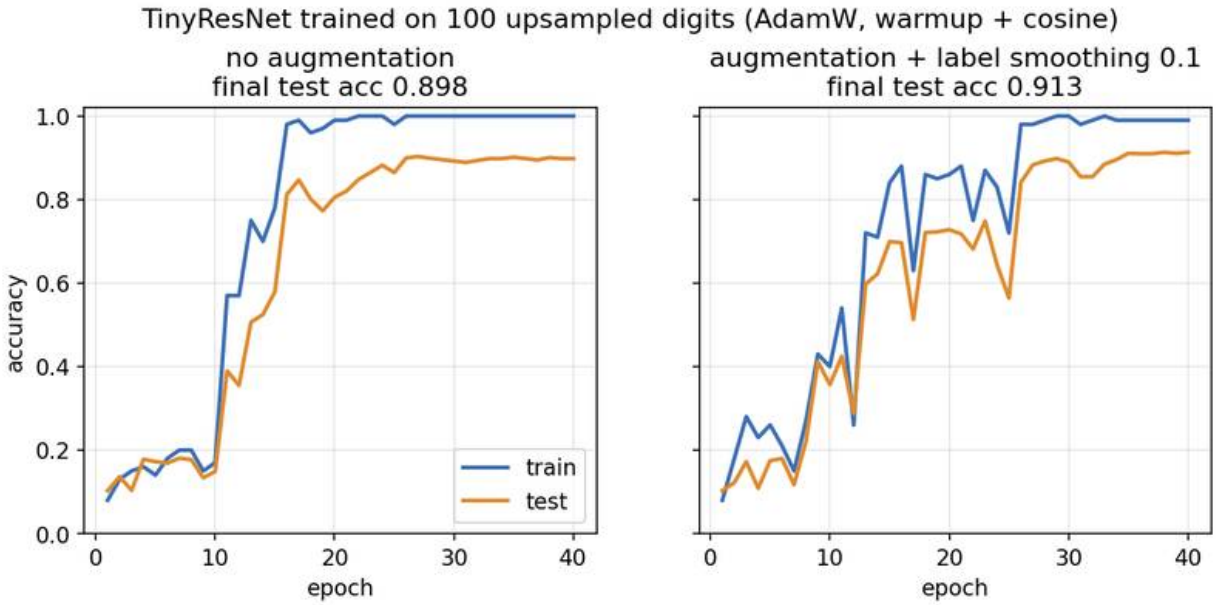


Figure 7 — Training curves on 100 training digits. Without augmentation (left) the network memorizes the training set and reaches 89.8% test accuracy. With augmentation and label smoothing (right) it reaches 91.3%. The gap is modest on such an easy task; on natural images the effect of the recipe is much larger.

The no-augmentation model reaches 100% training accuracy after about twenty epochs: with 100 images, memorization is easy. The augmented model sees a slightly different version of each digit every epoch, learns more slowly, and generalizes a little better (91.3% against 89.8%). Do not read too much into 1.5 points on one seed; the point is the mechanism.

"ResNet strikes back"

The most striking evidence for recipes comes from Wightman, Touvron and Jégou [32]. They kept the 2015 ResNet-50 architecture fixed and retrained it with a modern recipe: random resized crops and flips, RandAugment, Mixup and CutMix, the LAMB optimizer with a cosine schedule, a binary cross-entropy loss and longer training. The vanilla ResNet-50 reached **80.4% top-1** on ImageNet at 224×224 without extra data or distillation, against 76.1% for the standard torchvision weights they compare with. Two consequences follow. First, many "new architecture beats ResNet-50" claims compared a new model with a modern recipe against an old model with an old recipe. Second, when you compare architectures, train them with the same, tuned recipe, as ConvNeXt [20] and RegNet [19] did.

Transfer learning and fine-tuning

Plain version. Training a backbone from scratch needs a lot of data. Instead, start from a network already trained on a big dataset and adapt it to your task. You can either keep its features frozen and train only a new last layer, or gently retrain everything.

Let g_ϕ be a pre-trained backbone that maps an image to a feature vector $\mathbf{h} = g_\phi(\mathbf{x}) \in \mathbb{R}^D$, and $\mathbf{W} \in \mathbb{R}^{K \times D}$ a new linear head.

- **Linear probe.** Freeze ϕ and train only $\mathbf{W}$ (a logistic regression on fixed features). It is cheap, needs few labels, and is the standard way to *measure* how good a representation is.
- **Full fine-tuning.** Update ϕ and $\mathbf{W}$ together, usually with a learning rate for the backbone 10–100 times smaller than for the head, a short warmup, and sometimes layer-wise learning-rate decay (smaller rates for earlier layers). It adapts the features to the new domain and usually wins when there is enough labelled data.
- In between lie partial fine-tuning (only the last stages), and parameter-efficient methods that add small trainable modules to a frozen backbone.

Kornblith et al. [33] measured how ImageNet accuracy predicts transfer performance. For fine-tuning, better ImageNet models generally transferred better, with a strong correlation. For fixed features, the correlation was weaker and depended on regularization choices: some settings that improved ImageNet accuracy (for example, label smoothing and dropout) produced *worse* frozen features for other tasks. A good classifier and a good feature extractor are related, but not the same thing.

```
import torch
import torch.nn as nn

backbone = nn.Sequential(nn.Conv2d(3, 32, 3, 2, 1), nn.ReLU(), nn.Conv2d(32, 64, 3, 2, 1),
 ↪  nn.ReLU(),
                          nn.AdaptiveAvgPool2d(1), nn.Flatten())          # stand-in for a
                           ↪  pre-trained model
head = nn.Linear(64, 5)

# linear probe: freeze the backbone, train only the head
for p in backbone.parameters():
    p.requires_grad = False
probe_opt = torch.optim.AdamW(head.parameters(), lr=1e-3)

# full fine-tuning: unfreeze, use a smaller learning rate for the backbone than for the head
for p in backbone.parameters():
    p.requires_grad = True
ft_opt = torch.optim.AdamW([{"params": backbone.parameters(), "lr": 1e-5},
                            {"params": head.parameters(), "lr": 1e-3}], weight_decay=0.05)
```

Self-supervised and foundation-model classifiers

Plain version. Labels are expensive; images are not. Self-supervised learning invents a task whose answer is in the image itself (are these two crops from the same photo? what was under this mask?), learns features by solving it, and only then uses a few labels.

Contrastive learning: SimCLR and MoCo

Contrastive methods pull together the embeddings of two augmented **views** of the same image and push apart views of different images. SimCLR [34] takes a batch of B images, makes two random views of each, and minimizes the InfoNCE (NT-Xent) loss for each positive pair (i, j):

$$\ell_{i,j} = -\log \frac{\exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2B} \mathbb{1}_{[k \neq i]} \exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)},$$

where $\mathbf{z}$ are projected embeddings, sim is cosine similarity, τ a temperature and the other $2B - 2$ views in the batch act as negatives. It is a $2B - 1$-way softmax classification: "which of these is my partner?". SimCLR found that strong augmentation (especially crop plus color distortion), a non-linear projection head and large batches were essential. MoCo [35] avoids the need for huge batches by keeping a **queue** of negatives encoded by a slowly updated **momentum encoder**, whose weights are an EMA of the main encoder.

DINO and DINOv2: self-distillation

DINO [36] drops explicit negatives. A student network sees several crops (including small local ones) and is trained to match the output distribution of a teacher that sees global crops. The teacher is an EMA of the student; centering and sharpening of the teacher outputs prevent collapse to a constant. With ViTs, DINO features showed striking properties: the class token's attention maps segment objects without any segmentation labels, and a simple k-NN classifier on frozen features works well. DINOv2 [38] scaled the approach with an automatically curated dataset and a large ViT distilled into smaller ones, producing general-purpose **frozen features** that work across classification, segmentation and depth without fine-tuning. DINOv3 [39] continued the scaling and introduced "Gram anchoring" to keep dense patch features from degrading during long training.

MAE: masked image modeling

MAE [37] borrows the masked-language-model idea. It masks a large fraction of patches (75% in the paper), feeds only the visible patches to a ViT encoder, and asks a light decoder to reconstruct the missing pixels. Because the encoder skips masked tokens, pre-training is cheap. MAE features

are less linearly separable than contrastive ones (linear probes are weaker), but they **fine-tune** very well. This is a recurring pattern: different pre-training objectives favour different adaptation methods.

CLIP: zero-shot classification with language

CLIP [40] trains an image encoder g and a text encoder h on hundreds of millions of image–caption pairs with a symmetric contrastive loss, so that matching pairs have high cosine similarity.

A classifier is then built **without any training images**: write a prompt per class, such as `a photo of a dog.`, embed it, and predict

$$\hat{y} = \arg\max_k \frac{g(\mathbf{x})^\top h(t_k)}{\|g(\mathbf{x})\| \, \|h(t_k)\|},$$

where t_k is the prompt for class k. The text embeddings act as the weights of a linear classifier, computed from words. The paper reports that zero-shot CLIP matched the accuracy of a ResNet-50 trained on ImageNet's 1.28M labelled images, and that averaging several prompt templates ("a photo of a ...", "a blurry photo of a ...") helps. Zero-shot accuracy depends strongly on prompt wording, on whether the class names are visually meaningful, and on how well the web data covers the domain.

Robustness and evaluation pitfalls

Plain version. A classifier that scores 90% on its test set may fail badly when the photos come from a different camera, when someone adds invisible noise on purpose, or when the "easy clue" it learned (backgrounds, watermarks) is missing. And even when it is right, its confidence may be wrong.

Distribution shift: ImageNet-C, -R and -A

Hendrycks and colleagues built three widely used shifted test sets for ImageNet classifiers.

ImageNet-C [42] applies 15 algorithmic corruptions (noise, blur, weather, digital artefacts) at five severities and summarizes the error relative to a baseline (mean corruption error).

ImageNet-R [43] collects renditions (art, cartoons, sculptures, sketches, toys) of 200 ImageNet classes and tests whether a model understands shape and concept rather than photographic texture. **ImageNet-A** [44] contains natural, unmodified photos that were filtered to fool a ResNet-50; standard models do very poorly on it. These sets reveal different weaknesses, and improving one does not guarantee improving the others. Large-scale pre-training (as in CLIP and DINOv2) tends to improve robustness to such natural shifts much more than architecture changes at fixed data.

Adversarial examples: FGSM

Goodfellow et al. [45] explained adversarial examples by the **linearity** of networks in high dimensions: a tiny change to every input dimension, aligned with the weights, adds up to a large change in the output. The **fast gradient sign method** perturbs an input in one step:

$$\mathbf{x}_{\text{adv}} = \text{clip}\left(\mathbf{x} + \varepsilon \ \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}), y))\right),$$

where ε bounds the change per pixel (ℓ_∞ norm), $\mathcal{L}$ is the loss and clip keeps pixels in the valid range.

```
import torch.nn.functional as F

def fgsm(model, x, y, eps):
    x = x.clone().requires_grad_(True)
    F.cross_entropy(model(x), y).backward()
    return (x + eps * x.grad.sign()).clamp(-1, 1).detach()
```

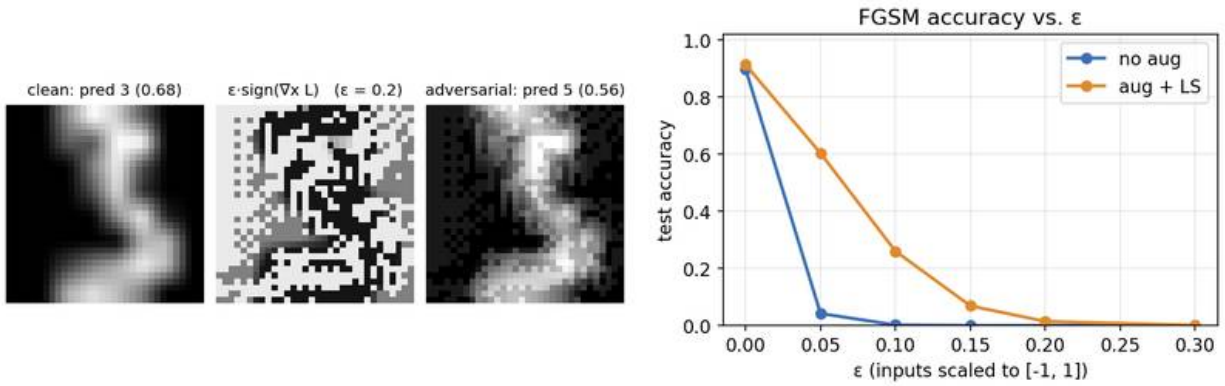


Figure 8 — FGSM on the TinyResNet models (inputs in $[-1, 1]$). At $\varepsilon = 0.05$ test accuracy falls from 89.8% to 4.2% for the model trained without augmentation and from 91.3% to 60.3% for the model trained with augmentation and label smoothing; at $\varepsilon = 0.2$ both are near zero.

Two cautions about Figure 8. First, the augmented model looks more robust at small ε, but label smoothing is known to flatten the loss surface around training points, which can weaken one-step gradient attacks without giving real robustness; stronger iterative attacks are the proper test. Second, $\varepsilon = 0.05$ on a $[-1, 1]$ scale is visible on an 8×8 digit; on natural images, imperceptible perturbations already fool standard ImageNet models.

Shortcut learning

Geirhos et al. [46] collect many failures under one name: **shortcut learning**. A network minimizes the training loss with whatever cue is easiest, which is often not the cue we intended: background

instead of animal, hospital tag instead of pathology, texture instead of shape. Shortcuts look fine on an i.i.d. test set, because the same shortcut exists there. They fail under distribution shift. The prescription is to test out of distribution on purpose, design tests that break the suspected shortcut, and use interpretability tools such as Grad-CAM (below) to check what the model attends to.

Calibration and temperature scaling

Temperature scaling [47] divides all logits by one scalar $T > 0$ before the softmax, $\mathbf{p} = \text{softmax}(\mathbf{z}/T)$. T is fitted by minimizing the negative log-likelihood on a held-out **calibration** split (never the test set). $T > 1$ softens overconfident predictions; $T < 1$ sharpens underconfident ones. Because dividing by T does not change the argmax, accuracy is unchanged.

```
import torch
import torch.nn.functional as F

def fit_temperature(logits, y, iters=300):
    """Learn one scalar T > 0 on a held-out calibration split."""
    log_t = torch.zeros(1, requires_grad=True)
    opt = torch.optim.Adam([log_t], lr=0.05)
    for _ in range(iters):
        loss = F.cross_entropy(logits / log_t.exp(), y)
        opt.zero_grad(); loss.backward(); opt.step()
    return log_t.exp().item()

def ece(probs, y, n_bins=10):
    conf, pred = probs.max(1)
    correct = (pred == y).float()
    edges = torch.linspace(0, 1, n_bins + 1)
    total = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            total += m.float().mean() * (correct[m].mean() - conf[m].mean()).abs()
    return float(total)

# toy check: an over-confident "model" = true logits multiplied by 3
torch.manual_seed(0)
true_logits = torch.randn(4000, 10)
y = torch.distributions.Categorical(logits=true_logits).sample()
over = 3 * true_logits
T = fit_temperature(over[:2000], y[:2000])
print(f"T = {T:.2f}  ECE {ece(over[2000:].softmax(1), y[2000:]):.3f} -> "
      f"{ece((over[2000:] / T).softmax(1), y[2000:]):.3f}")
# T = 2.90  ECE 0.345 -> 0.018
```

In this toy check the logits are three times too large, and the fitted temperature (2.90) recovers that

factor. On our real TinyResNet (the model trained without augmentation), the miscalibration went the *other* way.

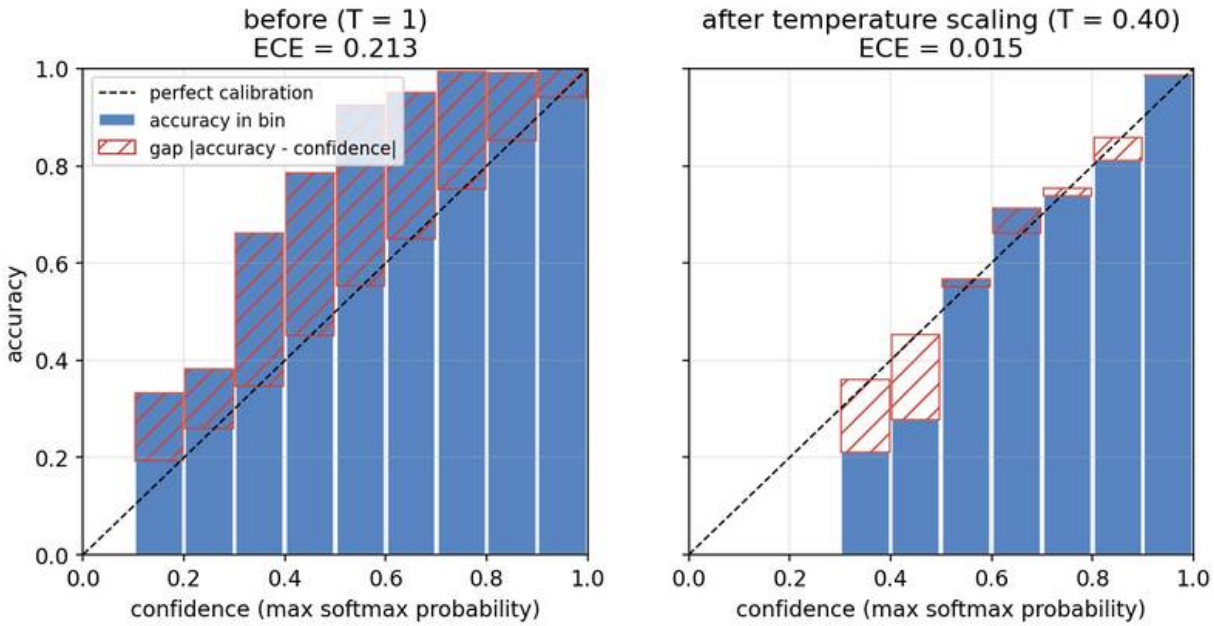


Figure 9 — Reliability diagrams for the TinyResNet trained without augmentation, on 1,197 test digits. Before scaling (left), accuracy exceeds confidence in most bins: the model is underconfident (ECE 0.213). A temperature $T = 0.40$ fitted on 500 separate calibration images brings ECE to 0.015 and test NLL from 0.515 to 0.298, without changing any prediction.

Why underconfident? Our tiny model was trained briefly with strong weight decay, which keeps logits small. Guo et al. [47] observed overconfidence in large, long-trained modern networks, and identified depth, width, batch normalization and *reduced* weight decay as contributing factors. Miscalibration has no fixed sign; measure it. Also note that calibration is itself fragile under distribution shift: a temperature fitted on in-distribution data does not guarantee calibrated confidence on ImageNet-C-style shifts.

Efficiency

Plain version. A model must fit the device and the time budget. Parameter count, compute, and actual speed are three different measurements, and only the last one is what users feel.

- **Parameters** determine model size on disk and in memory. A float32 parameter takes 4 bytes.
- **FLOPs** (or multiply–accumulates, MACs; one MAC is usually counted as two FLOPs) measure arithmetic per image. For a convolution with $K \times K$ kernels, C_{in} input and C_{out} output channels on an $H' \times W'$ output map, MACs $= H'W'K^2C_{\text{in}}C_{\text{out}}$. Compute grows with the square of

resolution.

- **Latency and throughput** are measured on target hardware. Depthwise convolutions have few FLOPs but low arithmetic intensity, so they often run slower on GPUs than their FLOP count suggests; DenseNet's concatenations cost memory bandwidth. Always benchmark.
- **Quantization** stores weights and activations as 8-bit integers instead of 32-bit floats. Jacob et al. [50] described integer-only inference with quantization-aware training, which simulates rounding during training so the network learns to tolerate it. Post-training quantization with a small calibration set is the cheap alternative.
- **Distillation** [51] trains a small student to match a large teacher's soft predictions, as DeiT did with its distillation token.

Interpretability: CAM and Grad-CAM

Plain version. A heat map over the image shows which regions pushed the score for a class up. It is a sanity check, not a proof of reasoning.

Class activation mapping (CAM) [48] works for networks that end with global average pooling and one linear layer. If A^k is the k-th feature map of the last convolutional layer and w_k^c the weight connecting it to class c, then the class score is $\sum_k w_k^c \cdot \text{mean}(A^k)$, and the map $\sum_k w_k^c A^k$ shows where that score comes from. **Grad-CAM** [49] generalizes this to any differentiable architecture by replacing the weights with spatially averaged gradients:

$$\alpha_k^c = \frac{1}{Z} \sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k}, \qquad L_{\text{Grad-CAM}}^c = \text{ReLU}\Big(\sum_k \alpha_k^c A^k \Big),$$

where y^c is the logit of class c, A_{ij}^k the activation of channel k at position (i, j), and Z the number of positions. The ReLU keeps only regions with a positive influence on the class. The coarse map (8×8 in our network) is upsampled to the image size.

```
import torch.nn.functional as F

def grad_cam(model, x, target=None):
    model.eval()
    feats = model.features(x)                          # (1, K, h, w) last conv stage
    feats.retain_grad()
    logits = model.fc(feats.mean(dim=(2, 3)))
    c = logits.argmax(1).item() if target is None else target
    logits[0, c].backward()
    alpha = feats.grad.mean(dim=(2, 3), keepdim=True)  # channel importance weights
    cam = F.relu((alpha * feats).sum(1, keepdim=True))
    cam = F.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)
    return (cam / (cam.max() + 1e-8))[0, 0].detach(), c
```

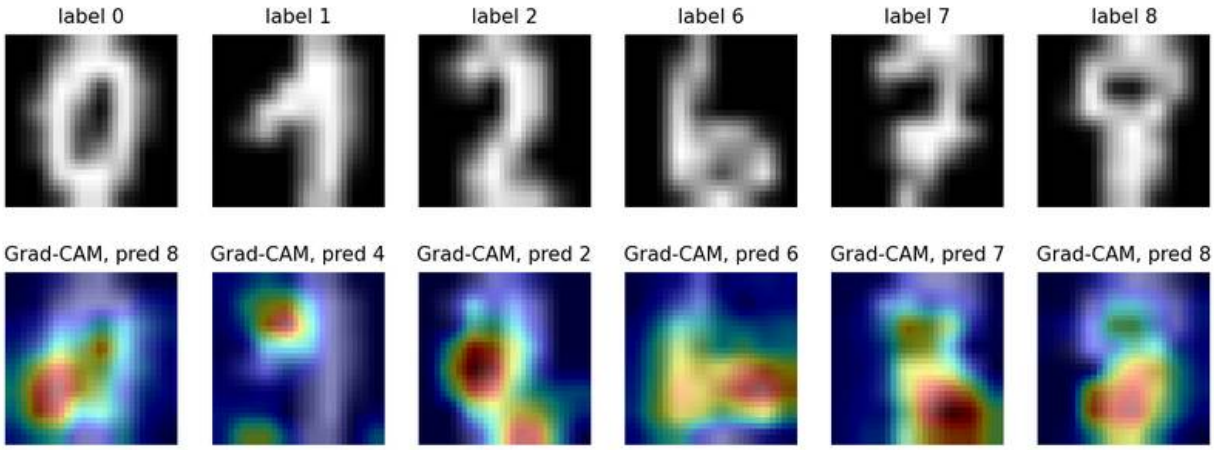


Figure 10 — Grad-CAM for the predicted class on the first test example of six digits, using the augmented TinyResNet. Two are misclassified: the 0 is called an 8, with heat on the lower-left part of the loop, and the 1 is called a 4, with heat on the hook at its upper left. The maps point to where the misleading evidence was found.

Explanations are most useful exactly when the model is wrong, as in Figure 10: they show which strokes looked like the wrong class. But treat saliency maps with care. They are coarse (limited by the last feature map's resolution), and they explain one class score for one input, not the model's general reasoning.

Modern view

Surveys worth reading. Li et al. [1] review CNNs from their history and basic building blocks through classic and advanced models, highlighting the key ideas behind each, and run experiments that compare activation functions, loss functions and optimizers to derive rules of thumb; they close with applications of 1-D, 2-D and multi-dimensional convolution and a list of open issues. Khan et al. [25] survey vision transformers across tasks. For classification they group them into uniform-scale models (ViT, DeiT), multi-scale hierarchical models (such as Swin), hybrids with convolutions, and self-supervised transformers (such as DINO), and they stress the reliance on large-scale pre-training and the cost of quadratic attention. Gui et al. [41] survey self-supervised learning and divide it into context-based methods (pretext tasks such as rotation prediction or jigsaw puzzles), contrastive methods (with negative pairs, self-distillation as in DINO, or feature decorrelation), generative methods (masked image modeling as in MAE), and hybrids of contrastive and generative objectives, before covering combinations with semi-supervised and multi-modal learning. On the reliability of benchmarks, the ImageNetV2 study [7], the re-labelling study of Beyer et al. [8] and the label-error analysis of Northcutt et al. [9] together argue for multiple test sets, multi-label evaluation and reporting variance.

The 2023–2026 picture. Five trends define current practice.

1. **Pre-train once, adapt cheaply.** Most new classifiers are not trained from scratch. They start from a large backbone trained with self-supervision (DINOv2 [38], DINOv3 [39]), image–text contrast (CLIP [40] and open reproductions), or masked modeling (MAE [37]), and adapt it with a linear probe, a k-NN rule, or a short fine-tune.
2. **Zero-shot and open-vocabulary classification.** Image–text models turn the label set into text. The class list can change at test time without retraining, which blurs the line between classification and retrieval.
3. **Scaling laws for vision.** Zhai et al. [52] scaled ViTs to billions of parameters and characterized how error falls with compute, data and model size, including the observation that larger models are more sample-efficient in few-shot transfer. Cherti et al. [53] fitted reproducible power laws for open CLIP training on public data, and found that the scaling behaviour depends on the training data distribution and the downstream task.
4. **Architectures converged.** ConvNeXt [20] and Swin [24] show that convolutions and attention, given the same recipe and scale, land in a similar place. The choice is now driven by hardware, resolution and downstream tasks (dense prediction favours hierarchical designs) more than by top-1 accuracy.
5. **Evaluation moved beyond ImageNet-1k top-1.** Robustness suites (ImageNet-C/R/A [42]–[44] and ImageNetV2 [7]), calibration, and transfer to many downstream datasets are now standard parts of a backbone paper.

Open problems. Robustness to natural distribution shift still lags clean accuracy, and gains from robust training methods are often specific to the shift they target. Shortcut learning [46] is hard to detect without counterfactual tests. Calibration under shift is unsolved. Long-tailed and fine-grained recognition remain hard because the rare classes have the fewest examples. Benchmark saturation and label noise make small accuracy differences on ImageNet-1k uninformative. And the dominant foundation models are trained on web-scale data whose composition, licensing and biases are hard to audit, which matters directly for what a "zero-shot" classifier will and will not recognize.

Key takeaways

- A classifier outputs logits; softmax + cross-entropy trains single-label tasks, independent sigmoids + binary cross-entropy train multi-label tasks. Report top-1 accuracy and, where it matters, calibration (ECE) and per-class results.
- Each landmark CNN fixed one bottleneck: ReLU/GPUs (AlexNet), small stacked kernels (VGG), 1×1 reductions and multi-branch modules (Inception), identity shortcuts for optimization (ResNet), depthwise separable convolutions for cost (MobileNet), balanced scaling (EfficientNet).
- ViTs trade convolutional inductive biases for flexibility; they need more data or a stronger recipe (DeiT), and windows plus hierarchy (Swin) make them practical backbones. ConvNeXt shows

that the recipe explains much of the gap.
- Training recipes matter as much as architecture: augmentation (random resized crop, Mixup, CutMix, RandAugment), label smoothing, AdamW, warmup + cosine, stochastic depth and EMA lifted an unchanged ResNet-50 to 80.4% top-1.
- Pre-trained backbones are the default starting point. Linear probes measure representations; fine-tuning adapts them. CLIP turns class names into classifier weights for zero-shot recognition.
- Accuracy hides *which* errors a model makes. Read the confusion matrix; under class imbalance report precision/recall, balanced accuracy or MCC, and prefer the precision–recall curve and AP over ROC-AUC. State whether an average is macro (every class equal) or micro (every image equal; equals accuracy for single-label tasks).
- Separate training data from evaluation data: judge a backbone on a battery of test sets (ImageNet val and ReaL, ImageNetV2, ImageNet-C/R/A/Sketch, ObjectNet, fine-grained and transfer suites), tune only on validation data, and report mean $\pm$ std over seeds.
- Benchmark accuracy is fragile: test on shifted data, beware shortcuts and adversarial inputs, fit a temperature on held-out data, and inspect Grad-CAM maps, especially on errors.

Exercises

1. **Receptive fields and parameters.** Show that three stacked 3×3 convolutions (stride 1) have a 7×7 receptive field. With C input and output channels and no biases, compare their parameter count with a single 7×7 convolution.

   Hint

   Each 3×3 layer adds 2 pixels to the receptive field: $3 \to 5 \to 7$. Parameters: $3 \cdot 9C^2 = 27C^2$ versus $49C^2$, about 45% fewer, plus two extra nonlinearities.

2. **Bottleneck arithmetic.** A ResNet bottleneck has $4d$ input channels and reduces to d. Count the weights (no biases) of the 1×1–3×3–1×1 branch as a function of d, and compare with a basic block of two 3×3 convolutions at width $4d$. What is the ratio?

   Hint

   Bottleneck: $4d \cdot d + 9d^2 + d \cdot 4d = 17d^2$. Basic block at width $4d$: $2 \cdot 9 \cdot 16d^2 = 288d^2$. The ratio is $17/288 \approx 0.059$, about 17 times fewer weights.

3. **Label smoothing optimum.** For one example with K classes and smoothing ε, assume all wrong-class logits are equal. Show that the cross-entropy with smoothed targets is minimized when the logit gap between the true class and each wrong class is $\log\left((1 - \varepsilon + \varepsilon/K) \,/\, (\varepsilon/K)\right)$. Evaluate it for $K = 1000$, $\varepsilon = 0.1$.

Hint

The minimum of cross-entropy over $\mathbf{p}$ is at $\mathbf{p} = \mathbf{q}$. So $p_y = 1 - \varepsilon + \varepsilon/K$ and every other $p_k = \varepsilon/K$, and the logit gap is the log of their ratio: $\log(0.9001/0.0001) \approx 9.1$. Without smoothing the optimal gap is infinite.

4. **ViT token count.** A ViT with 16×16 patches receives a 224×224 image. How many tokens enter the encoder? If the image is fine-tuned at 384 × 384, how many tokens are there, by what factor does the attention-matrix size grow, and what must be done with the position embeddings?

Hint

$(224/16)^2 = 196$ patches plus one class token: 197. At 384: $24^2 + 1 = 577$. The attention matrix grows by $(577/197)^2 \approx 8.6$. The learned position embeddings form a 14×14 grid; they are interpolated (2-D) to 24 × 24, which is what the ViT paper does when fine-tuning at higher resolution.

5. **Temperature does not change accuracy.** Prove that for any $T > 0$, $\arg\max_k \mathrm{softmax}(\mathbf{z}/T)_k = \arg\max_k z_k$. Then, using `scripts/figures/dl_02_image_classification.py`, fit the temperature on the *test* set instead of the calibration split and compare the reported ECE. Why is that number optimistic?

Hint

Division by a positive constant and the exponential are both strictly increasing, and the softmax denominator is shared. Fitting T on the test set uses test labels to tune a parameter, so the reported ECE is no longer an honest estimate of performance on new data.

6. **Build a shortcut.** Modify the digit experiment so that in the training set every "3" has a bright pixel in the top-left corner, but the test set does not. Train the TinyResNet, report test accuracy on the 3s, and look at Grad-CAM maps for a few training 3s. What do you expect, and how would you detect this problem if you did not know the shortcut was there?

Hint

The network can classify training 3s from the corner pixel alone, so test accuracy on 3s drops while training accuracy stays perfect, and Grad-CAM tends to highlight the corner. Without knowing the shortcut, look for heat maps that fall on background, evaluate on independently collected data, and run counterfactual tests (erase or move suspicious regions and see whether predictions change).

7. **Macro versus micro F1.** A three-class model is tested on 100 images. Rows are true classes, columns predictions: class A (90 images) → 88 A, 1 B, 1 C; class B (5 images) → 3 A, 2 B, 0 C; class C (5 images) → 4 A, 0 B, 1 C. Compute per-class precision, recall and F_1, then macro F_1 and micro F_1. Which number would you put in a report about detecting the rare classes B and C, and why?

Hint

Column sums are 95, 3 and 2. Precision: $88/95 = 0.926$, $2/3 = 0.667$, $1/2 = 0.500$. Recall: $88/90 = 0.978$, $2/5 = 0.400$, $1/5 = 0.200$. Using $F_1 = 2\text{TP}/(2\text{TP} + \text{FP} + \text{FN})$: $176/185 = 0.951$, $4/8 = 0.500$, $2/7 = 0.286$. Macro $F_1 = (0.951 + 0.500 + 0.286)/3 = 0.579$. Micro F_1 equals accuracy, $91/100 = 0.910$. Micro is dominated by class A; macro exposes that B and C are mostly missed, so macro (plus per-class recall) is the honest number for rare classes. You can check the result with `precision_recall_fscore_support(..., average="macro")`.

References

1. Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 12, pp. 6999–7019, 2022. doi:10.1109/TNNLS.2021.3084827
2. O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, et al., "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision*, vol. 115, pp. 211–252, 2015. doi:10.1007/s11263-015-0816-y
3. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi:10.1109/5.726791
4. A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," technical report, University of Toronto, 2009. dataset page
5. T. Ridnik, E. Ben-Baruch, A. Noy, and L. Zelnik-Manor, "ImageNet-21K Pretraining for the Masses," *NeurIPS*, 2021. arXiv
6. G. Van Horn, O. Mac Aodha, Y. Song, Y. Cui, C. Sun, A. Shepard, et al., "The iNaturalist Species Classification and Detection Dataset," *CVPR*, 2018. arXiv
7. B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do ImageNet Classifiers Generalize to ImageNet?," *ICML*, PMLR vol. 97, 2019. paper
8. L. Beyer, O. J. Hénaff, A. Kolesnikov, X. Zhai, and A. van den Oord, "Are We Done with ImageNet?," arXiv:2006.07159, 2020. arXiv
9. C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," *NeurIPS*, 2021. arXiv
10. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems 25 (NIPS)*, 2012. paper
11. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *ICLR*, 2015. arXiv
12. C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going Deeper with Convolutions," *CVPR*, 2015. doi:10.1109/CVPR.2015.7298594
13. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," *CVPR*, 2016. doi:10.1109/CVPR.2016.308
14. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016. doi:10.1109/CVPR.2016.90
15. K. He, X. Zhang, S. Ren, and J. Sun, "Identity Mappings in Deep Residual Networks," *ECCV*, 2016. arXiv

16. G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," *CVPR*, 2017. arXiv
17. A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017. arXiv
18. M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *ICML*, 2019. arXiv
19. I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He, and P. Dollár, "Designing Network Design Spaces," *CVPR*, 2020. arXiv
20. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," *CVPR*, 2022. arXiv
21. J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, "Squeeze-and-Excitation Networks," arXiv:1709.01507, 2017 (journal version in *IEEE TPAMI*). arXiv
22. A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021. arXiv
23. H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training Data-Efficient Image Transformers & Distillation through Attention," *ICML*, PMLR vol. 139, 2021. paper
24. Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00986
25. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in Vision: A Survey," *ACM Computing Surveys*, 2022. arXiv
26. H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond Empirical Risk Minimization," *ICLR*, 2018. arXiv
27. S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features," *ICCV*, 2019. arXiv
28. E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "RandAugment: Practical Automated Data Augmentation with a Reduced Search Space," *CVPR Workshops*, 2020. doi:10.1109/CVPRW50498.2020.00359
29. I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *ICLR*, 2019. arXiv
30. I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," *ICLR*, 2017. arXiv
31. G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, "Deep Networks with Stochastic Depth," *ECCV*, 2016. doi:10.1007/978-3-319-46493-0_39
32. R. Wightman, H. Touvron, and H. Jégou, "ResNet Strikes Back: An Improved Training Procedure in timm," arXiv:2110.00476, 2021. arXiv
33. S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?," *CVPR*, 2019. arXiv
34. T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," *ICML*, 2020. arXiv
35. K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum Contrast for Unsupervised Visual Representation Learning," *CVPR*, 2020. arXiv
36. M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging

Properties in Self-Supervised Vision Transformers," *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00951
37. K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked Autoencoders Are Scalable Vision Learners," *CVPR*, 2022. doi:10.1109/CVPR52688.2022.01553
38. M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. arXiv
39. O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, C. Jose, et al., "DINOv3," arXiv:2508.10104, 2025. arXiv
40. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, et al., "Learning Transferable Visual Models From Natural Language Supervision," *ICML*, PMLR vol. 139, 2021. paper
41. J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo, and D. Tao, "A Survey on Self-Supervised Learning: Algorithms, Applications, and Future Trends," *IEEE Transactions on Pattern Analysis and Machine Intelligence* (arXiv:2301.05712, 2023). arXiv
42. D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," *ICLR*, 2019. arXiv
43. D. Hendrycks, S. Basart, N. Mu, S. Kadavath, F. Wang, E. Dorundo, et al., "The Many Faces of Robustness: A Critical Analysis of Out-of-Distribution Generalization," *ICCV*, 2021. arXiv
44. D. Hendrycks, K. Zhao, S. Basart, J. Steinhardt, and D. Song, "Natural Adversarial Examples," *CVPR*, 2021. arXiv
45. I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and Harnessing Adversarial Examples," in *Proc. International Conference on Learning Representations* (*ICLR*), 2015. arXiv
46. R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut Learning in Deep Neural Networks," *Nature Machine Intelligence*, 2020. arXiv
47. C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," *ICML*, 2017. arXiv
48. B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning Deep Features for Discriminative Localization," *CVPR*, 2016. doi:10.1109/CVPR.2016.319
49. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," *ICCV*, 2017. doi:10.1109/ICCV.2017.74
50. B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *CVPR*, 2018. doi:10.1109/CVPR.2018.00286
51. G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015. arXiv
52. X. Zhai, A. Kolesnikov, N. Houlsby, and L. Beyer, "Scaling Vision Transformers," *CVPR*, 2022. arXiv
53. M. Cherti, R. Beaumont, R. Wightman, M. Wortsman, G. Ilharco, C. Gordon, et al., "Reproducible Scaling Laws for Contrastive Language-Image Learning," *CVPR*, 2023. arXiv
54. T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. doi:10.1016/j.patrec.2005.10.010
55. J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," *Proc. 23rd International Conference on Machine Learning* (*ICML*), pp. 233–240, 2006. doi:10.1145/

1143844.1143874
56. T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. doi:10.1371/journal.pone.0118432
57. D. Chicco and G. Jurman, "The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation," *BMC Genomics*, vol. 21, article 6, 2020. doi:10.1186/s12864-019-6413-7
58. M. Sokolova and G. Lapalme, "A Systematic Analysis of Performance Measures for Classification Tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009. doi:10.1016/j.ipm.2009.03.002
59. J. Cohen, "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960. doi:10.1177/001316446002000104
60. M.-L. Zhang and Z.-H. Zhou, "A Review on Multi-Label Learning Algorithms," *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 8, pp. 1819–1837, 2014. doi:10.1109/TKDE.2013.39
61. M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010. doi:10.1007/s11263-009-0275-4
62. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. paper
63. H. Wang, S. Ge, E. P. Xing, and Z. C. Lipton, "Learning Robust Global Representations by Penalizing Local Predictive Power," *NeurIPS*, 2019 (introduces ImageNet-Sketch). arXiv
64. A. Barbu, D. Mayo, J. Alverio, W. Luo, C. Wang, D. Gutfreund, J. Tenenbaum, and B. Katz, "ObjectNet: A Large-Scale Bias-Controlled Dataset for Pushing the Limits of Object Recognition Models," *NeurIPS*, 2019. dataset page
65. C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "The Caltech-UCSD Birds-200-2011 Dataset," Technical Report CNS-TR-2011-001, California Institute of Technology, 2011. dataset page
66. J. Krause, M. Stark, J. Deng, and L. Fei-Fei, "3D Object Representations for Fine-Grained Categorization," *IEEE International Conference on Computer Vision Workshops (ICCVW)*, pp. 554–561, 2013 (introduces Stanford Cars). doi:10.1109/ICCVW.2013.77
67. S. Maji, E. Rahtu, J. Kannala, M. Blaschko, and A. Vedaldi, "Fine-Grained Visual Classification of Aircraft," arXiv:1306.5151, 2013. arXiv
68. O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and Dogs," *CVPR*, 2012 (introduces Oxford-IIIT Pets). dataset page
69. M.-E. Nilsback and A. Zisserman, "Automated Flower Classification over a Large Number of Classes," *Indian Conference on Computer Vision, Graphics and Image Processing (ICVGIP)*, 2008 (introduces Oxford Flowers-102). dataset page
70. L. Bossard, M. Guillaumin, and L. Van Gool, "Food-101 – Mining Discriminative Components with Random Forests," *ECCV*, 2014. dataset page
71. L. Fei-Fei, R. Fergus, and P. Perona, "One-Shot Learning of Object Categories," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2006 (the reference given for Caltech-101). dataset page

72. X. Zhai, J. Puigcerver, A. Kolesnikov, et al., “A Large-scale Study of Representation Learning with the Visual Task Adaptation Benchmark,” arXiv:1910.04867, 2019. arXiv
73. Z. Liu, Z. Miao, X. Zhan, J. Wang, B. Gong, and S. X. Yu, “Large-Scale Long-Tailed Recognition in an Open World,” *CVPR*, 2019 (introduces ImageNet-LT). arXiv

DLCV 03 • Object Detection: From R-CNN to DETR

How deep networks find and label every object in an image: box regression, IoU, anchors, NMS and feature pyramids; two-stage, one-stage, anchor-free and transformer detectors; open-vocabulary models; and how detection is measured.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-03-object-detection/

What you'll learn

- What a detector outputs (boxes, class labels, confidences), and how detection differs from classification, semantic segmentation and instance segmentation.
- The building blocks every modern detector shares, in math and in code: box parameterization and regression targets, IoU and its GIoU/DIoU/CIoU variants, anchors and label assignment, non-maximum suppression, feature pyramids and the focal loss.
- How the four big families work and why each was invented: two-stage (R-CNN to Cascade R-CNN), one-stage (YOLO, SSD, RetinaNet), anchor-free (CornerNet, CenterNet, FCOS) and set prediction with transformers (DETR to RT-DETR).
- How open-vocabulary and grounding detectors (OWL-ViT, Grounding DINO, YOLO-World) and promptable foundation models change what "the class list" means.
- How detection is evaluated (precision–recall, AP, VOC and COCO protocols, AP50/AP75/APs/m/l), which datasets matter, and what limits speed in deployment.

The big picture

A classifier answers one question about a whole image: *what is this?* A detector answers a harder one: *what is where?* It must find every object of interest, draw a tight rectangle (a **bounding box**) around each, name it, and say how sure it is. The number of objects is not known in advance: there may be zero, one or three hundred.

> **Explain like I'm five**
>
> Imagine a "Where's Waldo?" page. A **classifier** only tells you "this page has Waldo on it." A **detector** puts a sticker on Waldo, another sticker on every dog and every balloon, and writes on each sticker what it is and how sure it is ("dog, pretty sure"). The tricky parts are the ones you would also find tricky: tiny things far away, things hiding behind other things, and not putting two stickers on the same dog.

Detection is the entry point of most "real" vision systems. Self-driving cars detect pedestrians and vehicles; phones detect faces before focusing; factories detect defects; trackers (see the tracking

chapter linked at the end) mostly *link detections over time*. The field also produced many ideas that now appear everywhere in deep learning: multi-task losses, feature pyramids, the focal loss, and set prediction with bipartite matching.

This tutorial assumes you know CNNs and image classification (DLCV 02) and the classical feature ideas of IP 12. Two surveys frame the whole story: Zou et al., *Object Detection in 20 Years* [1], and Liu et al.'s IJCV survey of deep generic detection [2]. We return to both in the Modern view.

The detection problem

Plain version. The input is an image. The output is a short list of (box, class, score) triples. The training data is a list of human-drawn boxes with class names.

Precise version. Given an image $I \in \mathbb{R}^{H\times W\times 3}$, a detector outputs a set

$$\hat{Y} = \{(\hat{b}_i, \hat{c}_i, \hat{s}_i)\}_{i=1}^{N},$$

where $\hat{b}_i \in \mathbb{R}^4$ is a box, $\hat{c}_i \in \{1, \dots, K\}$ is one of K classes, $\hat{s}_i \in [0, 1]$ is a confidence score, and N varies from image to image. The ground truth is a set $Y = \{(b_j, c_j)\}_{j=1}^{M}$ of M annotated objects.

Two box formats are common: corners (x_1, y_1, x_2, y_2) and center–size (c_x, c_y, w, h). Boxes are **axis-aligned**; rotated boxes are a separate variant used for aerial images and text.

Three properties make detection harder than classification:

1. **Variable-size output.** The network must decide *how many* things to output. Every architecture in this tutorial is, at heart, a different answer to this problem.
2. **Localization and recognition at once.** The model must be invariant to *what* changes about an object's appearance but sensitive to *where* it is, a tension that shapes the design of every head.
3. **Extreme imbalance.** A typical image has a handful of objects but tens of thousands of candidate positions and sizes, nearly all background.

Detection versus its neighbours

Figure 1 compares the four classic recognition tasks on one synthetic scene. **Classification** gives one label per image. **Detection** gives a box and a label per object. **Semantic segmentation** labels every pixel with a class, so two touching balls merge into one blob. **Instance segmentation** gives each object its own pixel mask, which is detection plus a mask per box; Mask R-CNN [16] is the standard way to get it, as we will see.

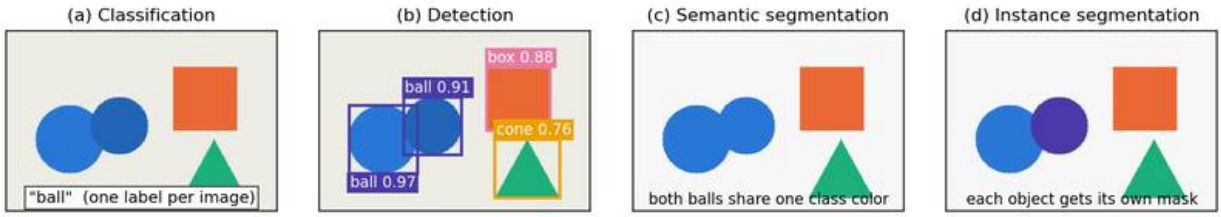


Figure 1 — The same scene seen by four tasks: (a) classification, (b) detection, (c) semantic segmentation, (d) instance segmentation.

Classical detection: sliding windows and hand-made features

Plain version. Before deep learning, detection was "classification in a loop": cut out a window at every position and size, ask a classifier whether it contains the object, and keep the windows that say yes.

Sliding windows. For a window of fixed size $w \times h$ slid with stride s over an image pyramid with scale factor λ, the number of windows is roughly

$$N_{\text{win}} \approx \sum_{k \geq 0} \frac{(H\lambda^{-k} - h)(W\lambda^{-k} - w)}{s^2},$$

where k indexes the pyramid levels. For a 640×480 image, a 64×128 window, $s = 8$ and $\lambda = 1.2$, that is on the order of 10^4 windows for one aspect ratio, all of which must be classified. This is the template matching of IP 13 scaled up, and it has the same weaknesses: one template shape, many scales, and a lot of wasted work on background.

Viola–Jones (2001) [5] made real-time face detection possible with three tricks. *Haar-like features* (differences of sums over adjacent rectangles) can be evaluated in constant time using an **integral image** $S(x,y) = \sum_{x' \leq x, y' \leq y} I(x',y')$, so any rectangle sum needs four lookups. *AdaBoost* selects a small set of informative features out of a huge pool. An *attentional cascade* of increasingly complex classifiers rejects most background windows after only a few feature evaluations, which is the first big idea for beating the imbalance problem: spend computation only where it might matter.

HOG + linear SVM (2005) [6] replaced raw intensities with **histograms of oriented gradients**, close relatives of the gradient-orientation descriptors in IP 12. The detection window is divided into small cells; each cell stores a histogram of gradient orientations weighted by magnitude; overlapping blocks of cells are contrast-normalized; the concatenated vector is scored by a linear SVM. HOG is robust to small shifts and illumination changes, and it made pedestrian detection practical.

Deformable part models (DPM) [7] added structure. An object is a coarse *root* HOG filter plus

several higher-resolution *part* filters that may shift relative to the root, paying a quadratic deformation cost:

$$\text{score}(p_0) = F_0 \cdot \phi(p_0) + \sum_{i=1}^{n} \max_{p_i} \left[F_i \cdot \phi(p_i) - d_i \cdot \psi(p_i - p_0) \right] + \beta,$$

where p_0 is the root position, p_i the position of part i, F_i the learned filters, $\phi(\cdot)$ the HOG features at a position, ψ the displacement features (dx, dy, dx^2, dy^2), d_i learned deformation weights and β a bias. Part positions are latent variables, trained with a *latent SVM*. DPM and its variants dominated the PASCAL VOC benchmark [3] until 2012.

Why this era ended. Classical detectors were *slow* (an exhaustive scan over positions, scales and aspect ratios, each scored by a hand-designed pipeline) and *brittle* (one hand-designed feature type, a few rigid templates, and poor transfer to new categories, viewpoints and clutter). They also had no way to share features across classes. Deep detectors kept the useful ideas, such as scanning in a feature pyramid, cascades, parts and hard-negative mining, and replaced the hand-made features with learned ones.

Core building blocks

Every detector in the rest of this tutorial is assembled from the same parts. We define each one carefully, with code you can run.

Box parameterization and regression targets

Plain version. A network is bad at outputting absolute pixel coordinates, but good at outputting small corrections. So we start from a **reference box** (a proposal, an anchor or a point) and predict how to nudge it.

Given a reference box $P = (P_x, P_y, P_w, P_h)$ in center–size form and its matched ground-truth box $G = (G_x, G_y, G_w, G_h)$, the standard targets introduced with R-CNN [8] are

$$t_x = \frac{G_x - P_x}{P_w}, \qquad t_y = \frac{G_y - P_y}{P_h}, \qquad t_w = \log \frac{G_w}{P_w}, \qquad t_h = \log \frac{G_h}{P_h}.$$

Dividing the shift by the reference size makes the target **scale-invariant** (a 10-pixel error matters more for a 20-pixel box than for a 400-pixel box). The **log** for width and height makes scaling symmetric (doubling and halving are $+\log 2$ and $-\log 2$) and keeps the decoded size positive: $\hat{G}_w = P_w \exp(\hat{t}_w)$. In practice the targets are often divided by fixed standard deviations, such as $(0.1, 0.1, 0.2, 0.2)$, so that all four have similar magnitude.

```
import numpy as np
```

```
def to_cxcywh(b):
    return np.stack([(b[:, 0] + b[:, 2]) / 2, (b[:, 1] + b[:, 3]) / 2,
                     b[:, 2] - b[:, 0], b[:, 3] - b[:, 1]], axis=1)

def to_xyxy(c):
    return np.stack([c[:, 0] - c[:, 2] / 2, c[:, 1] - c[:, 3] / 2,
                     c[:, 0] + c[:, 2] / 2, c[:, 1] + c[:, 3] / 2], axis=1)

def encode(anchors, gt, std=(0.1, 0.1, 0.2, 0.2)):
    """Regression targets (tx, ty, tw, th) of gt relative to anchors (both xyxy)."""
    P, G = to_cxcywh(anchors), to_cxcywh(gt)
    t = np.stack([(G[:, 0] - P[:, 0]) / P[:, 2],
                  (G[:, 1] - P[:, 1]) / P[:, 3],
                  np.log(G[:, 2] / P[:, 2]),
                  np.log(G[:, 3] / P[:, 3])], axis=1)
    return t / np.array(std)          # normalize so all four have similar scale

def decode(anchors, t, std=(0.1, 0.1, 0.2, 0.2)):
    P = to_cxcywh(anchors)
    t = t * np.array(std)
    c = np.stack([P[:, 0] + t[:, 0] * P[:, 2],
                  P[:, 1] + t[:, 1] * P[:, 3],
                  P[:, 2] * np.exp(t[:, 2]),
                  P[:, 3] * np.exp(t[:, 3])], axis=1)
    return to_xyxy(c)

anchors = np.array([[0, 0, 32, 32], [100, 100, 228, 164]], float)
gt      = np.array([[4, 2, 40, 30], [90, 110, 250, 170]], float)
t = encode(anchors, gt)
print(np.round(t, 3))
print(np.allclose(decode(anchors, t), gt))   # round trip
```

```
[[ 1.875  0.     0.589 -0.668]
 [ 0.469  1.25   1.116 -0.323]]
True
```

These targets are usually trained with the **smooth L1** (Huber) loss of Fast R-CNN [9],

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5\,x^2 & \text{if } |x| < 1, \\ |x| - 0.5 & \text{otherwise,} \end{cases}$$

applied to each coordinate difference $x = \hat{t} - t$. It behaves like L2 for small errors (smooth gradients) and like L1 for large ones (robust to outliers early in training). Its weakness is that the four coordinates are treated as independent numbers, while the evaluation metric, IoU, treats the box as a whole. That mismatch motivated IoU-based losses.

IoU and its variants

Plain version. IoU asks: of all the area covered by either box, what fraction is covered by both? It is 1 for a perfect match and 0 for boxes that do not touch.

For two boxes A and B,

$$\mathrm{IoU}(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|},$$

where $|\cdot|$ is area. IoU is scale-invariant, and it is the yardstick for evaluation (a detection counts as correct if its IoU with an unmatched ground-truth box exceeds a threshold), for label assignment and for NMS. Figure 2 shows how demanding the thresholds are: shifting a square by only about 18% of its side along both axes already drops IoU to 0.5.

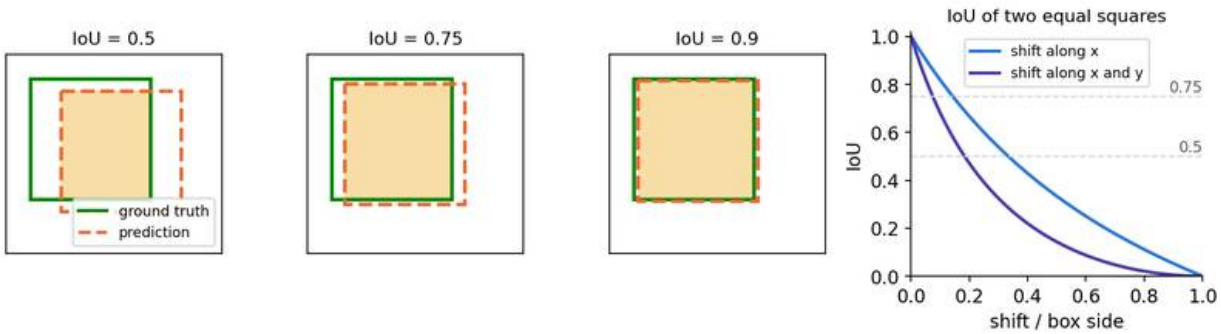


Figure 2 — What IoU thresholds look like. Left: a prediction at IoU 0.5, 0.75 and 0.9 with the ground truth. Right: IoU of two equal squares as one is shifted along x, or along x and y.

As a loss, $1 - \mathrm{IoU}$ has a flaw: when two boxes do not overlap, IoU is 0 no matter how far apart they are, so the gradient is zero. Three fixes are widely used:

- **GIoU** [11] adds a penalty based on the smallest box C that encloses both: $\mathrm{GIoU} = \mathrm{IoU} - \frac{|C \setminus (A \cup B)|}{|C|}$, which lies in $(-1, 1]$ and keeps decreasing as non-overlapping boxes drift apart.
- **DIoU** [12] penalizes the distance between centers directly: $\mathrm{DIoU} = \mathrm{IoU} - \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2}$, where ρ is the Euclidean distance between the box centers $\mathbf{b}$ and $\mathbf{b}^{gt}$, and c is the diagonal length of C. It converges faster than GIoU because it pulls the centers together even when one box contains the other, where the GIoU penalty vanishes.
- **CIoU** [12] adds an aspect-ratio term: $\mathrm{CIoU} = \mathrm{DIoU} - \alpha v$, with $v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2$ measuring aspect-ratio mismatch and $\alpha = \frac{v}{(1-\mathrm{IoU})+v}$ a trade-off weight.

The corresponding losses are $1 - \mathrm{GIoU}$, $1 - \mathrm{DIoU}$ and $1 - \mathrm{CIoU}$.

```
import numpy as np
```

```
def iou_family(a, b, eps=1e-9):
    """a, b: (N, 4) boxes as (x1, y1, x2, y2). Returns IoU, GIoU, DIoU, CIoU."""
    ix1, iy1 = np.maximum(a[:, 0], b[:, 0]), np.maximum(a[:, 1], b[:, 1])
    ix2, iy2 = np.minimum(a[:, 2], b[:, 2]), np.minimum(a[:, 3], b[:, 3])
    inter = np.clip(ix2 - ix1, 0, None) * np.clip(iy2 - iy1, 0, None)
    wa, ha = a[:, 2] - a[:, 0], a[:, 3] - a[:, 1]
    wb, hb = b[:, 2] - b[:, 0], b[:, 3] - b[:, 1]
    union = wa * ha + wb * hb - inter
    iou = inter / (union + eps)
    # smallest enclosing box C
    cx1, cy1 = np.minimum(a[:, 0], b[:, 0]), np.minimum(a[:, 1], b[:, 1])
    cx2, cy2 = np.maximum(a[:, 2], b[:, 2]), np.maximum(a[:, 3], b[:, 3])
    area_c = (cx2 - cx1) * (cy2 - cy1)
    giou = iou - (area_c - union) / (area_c + eps)
    # center distance over enclosing-box diagonal
    rho2 = ((a[:, 0] + a[:, 2]) - (b[:, 0] + b[:, 2])) ** 2 / 4 \
         + ((a[:, 1] + a[:, 3]) - (b[:, 1] + b[:, 3])) ** 2 / 4
    c2 = (cx2 - cx1) ** 2 + (cy2 - cy1) ** 2
    diou = iou - rho2 / (c2 + eps)
    v = 4 / np.pi**2 * (np.arctan(wb / hb) - np.arctan(wa / ha)) ** 2
    alpha = v / (1 - iou + v + eps)
    ciou = diou - alpha * v
    return iou, giou, diou, ciou

gt = np.array([[10, 10, 50, 50]] * 3, float)
pred = np.array([[20, 20, 60, 60],    # overlapping, shifted
                 [60, 10, 100, 50],   # disjoint, nearby
                 [200, 10, 240, 50]], float)  # disjoint, far away
for name, v in zip(["IoU", "GIoU", "DIoU", "CIoU"], iou_family(pred, gt)):
    print(f"{name:5s}", np.round(v, 3))
```

```
IoU   [0.391 0.    0.   ]
GIoU  [ 0.311 -0.111 -0.652]
DIoU  [ 0.351 -0.258 -0.662]
CIoU  [ 0.351 -0.258 -0.662]
```

The two disjoint predictions both have IoU 0, but GIoU and DIoU correctly say that the far one is worse. CIoU equals DIoU here because all boxes are squares, so there is no aspect-ratio mismatch.

Anchors and label assignment

Plain version. Instead of searching freely, a dense detector places a fixed set of reference boxes, **anchors**, at every cell of a feature map, and each anchor asks: "is there an object roughly like me here, and how should I move to fit it?"

At a feature map with stride s (each cell covers $s \times s$ input pixels), anchors are centered at $((i+\frac{1}{2})s, (j+\frac{1}{2})s)$ with a few scales σ and aspect ratios $r = h/w$, giving widths $w = \sigma/\sqrt{r}$ and heights

$h = \sigma\sqrt{r}$ so that the area stays σ^2. Faster R-CNN uses 3 scales × 3 ratios = 9 anchors per location [10] (Figure 3, left).

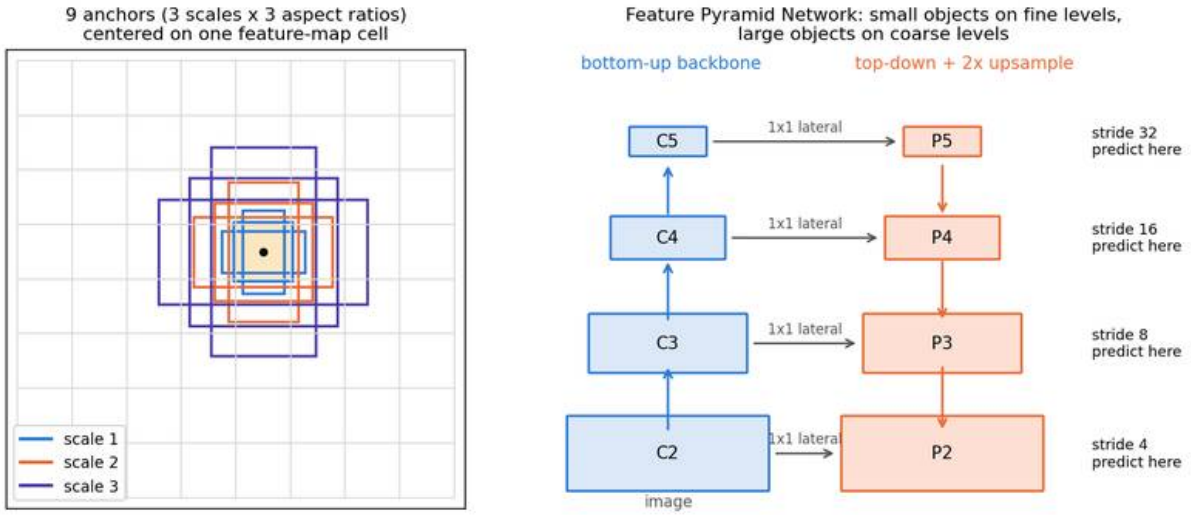


Figure 3 — Left: nine anchors (3 scales × 3 aspect ratios) at one feature-map cell. Right: a Feature Pyramid Network; each level predicts objects of a matching size range.

Label assignment decides which anchors are trained as which object. The classic rule is IoU-based: in the region proposal network of Faster R-CNN an anchor is *positive* if its IoU with some ground-truth box is at least 0.7 (or if it is the best anchor for that box), *negative* if its IoU with every box is below 0.3, and ignored otherwise [10]; RetinaNet uses 0.5 and 0.4 [15]. Each positive anchor regresses toward its matched box with the targets above. Assignment turned out to matter as much as architecture: anchor-free detectors (FCOS [29]) assign by *location* and object size instead, and DETR [30] replaces all fixed rules by a learned one-to-one matching.

Anchors have real costs: they introduce hyperparameters (scales, ratios, thresholds) that must be tuned per dataset, and they multiply the number of predictions. YOLO9000 [19] chose anchor shapes by running k-means on the training boxes, using $1 - \text{IoU}$ as the distance.

Non-maximum suppression (NMS) and Soft-NMS

Plain version. A dense detector fires many times on the same object (neighboring anchors all see it). NMS keeps the most confident box and deletes the others that overlap it too much.

Greedy NMS, per class: sort the boxes by score; repeatedly take the highest-scoring remaining box M, keep it, and remove every box b_i with $\text{IoU}(M, b_i) > N_t$ (typically $N_t = 0.5$ to 0.7). The weakness is crowded scenes: two real people standing close may overlap by more than N_t, and the lower-scoring one is deleted, losing recall.

Soft-NMS [13] replaces deletion with a score decay. With the Gaussian variant,

$$s_i \leftarrow s_i \exp\!\left(-\frac{\mathrm{IoU}(M, b_i)^2}{\sigma}\right),$$

where s_i is the score of box b_i and σ controls how fast overlapping boxes are penalized. A heavy overlap still pushes the score down a lot, but a genuine neighbor survives with a reduced score instead of disappearing.

```
import numpy as np

def iou_one_to_many(box, boxes):
    x1 = np.maximum(box[0], boxes[:, 0]); y1 = np.maximum(box[1], boxes[:, 1])
    x2 = np.minimum(box[2], boxes[:, 2]); y2 = np.minimum(box[3], boxes[:, 3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[..., 2] - b[..., 0]) * (b[..., 3] - b[..., 1])
    return inter / (area(box) + area(boxes) - inter)

def nms(boxes, scores, thr=0.5):
    order = np.argsort(-scores)
    keep = []
    while order.size:
        i = order[0]
        keep.append(i)
        ious = iou_one_to_many(boxes[i], boxes[order[1:]])
        order = order[1:][ious <= thr]          # drop everything that overlaps too much
    return np.array(keep)

def soft_nms(boxes, scores, sigma=0.5, score_thr=0.001):
    scores = scores.copy()
    idx = np.arange(len(scores))
    keep = []
    while idx.size:
        j = np.argmax(scores[idx]); i = idx[j]
        keep.append(i)
        idx = np.delete(idx, j)
        ious = iou_one_to_many(boxes[i], boxes[idx])
        scores[idx] *= np.exp(-ious**2 / sigma)  # decay instead of delete
        idx = idx[scores[idx] > score_thr]
    return np.array(keep), scores

# two people standing close together + duplicates
boxes = np.array([[10, 10, 60, 110], [14, 12, 62, 112], [8, 6, 58, 104],
                  [40, 15, 90, 115], [43, 18, 92, 118]], float)
scores = np.array([0.95, 0.90, 0.80, 0.85, 0.70])
print("greedy NMS keeps:", nms(boxes, scores, 0.5))
keep, s = soft_nms(boxes, scores)
print("Soft-NMS order:", keep, "rescored:", np.round(s[keep], 3))
```

```
greedy NMS keeps: [0 3]
Soft-NMS order: [0 3 2 4 1] rescored: [0.95  0.761 0.183 0.145 0.058]
```

Greedy NMS keeps one box per person. Soft-NMS keeps everything but ranks the duplicates far

below the two real detections; a final score threshold or the AP computation then treats them as low-confidence guesses. Figure 4 shows the same effect on a synthetic scene.

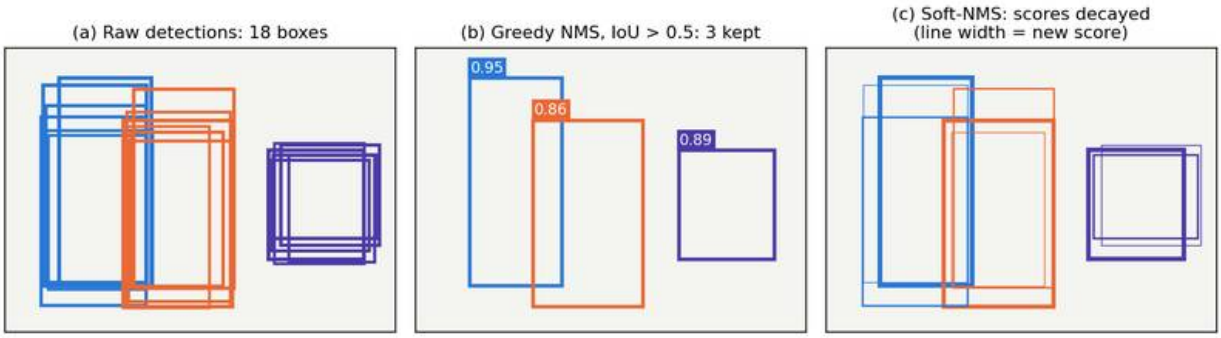


Figure 4 — NMS on synthetic detections. (a) 18 raw boxes around three objects. (b) Greedy NMS at IoU 0.5 keeps one box per object. (c) Soft-NMS keeps all boxes but decays the scores of overlapping ones (thinner line = lower score).

NMS has three further problems: it has a hand-tuned threshold; it is a sequential, data-dependent step that is awkward to run on accelerators; and it is not part of training, so the network is never taught to avoid duplicates. Removing it is one of the main motivations behind DETR and YOLOv10 [24].

Feature pyramids (FPN)

Plain version. Small objects need high-resolution feature maps; large objects need features that have seen a lot of context. A feature pyramid gives you both, at every scale.

A CNN backbone naturally produces a pyramid $\{C_2, C_3, C_4, C_5\}$ with strides $4, 8, 16, 32$, but its shallow levels are semantically weak. The **Feature Pyramid Network** [14] adds a top-down path: starting from the coarsest level, it upsamples by 2 and adds a 1×1-projected lateral connection from the backbone,

$$P_l = \mathrm{Conv}_{3\times3}\big(\mathrm{Conv}_{1\times1}(C_l) + \mathrm{Up}_{2\times}(P_{l+1})\big),$$

so every output level P_l has the same channel width and strong semantics. In a two-stage detector with FPN, a region of interest of size $w \times h$ is pooled from level $l = \lfloor l_0 + \log_2(\sqrt{wh}/224) \rfloor$ (with $l_0 = 4$ in [14]), so a region twice as large moves one level up; dense detectors use a similar size-to-level rule. Later necks (extra bottom-up paths, weighted fusion) refine the same idea; nearly every detector after 2017 uses one (Figure 3, right).

Focal loss and class imbalance

Plain version. If 99.9% of your candidates are background, a normal loss is dominated by millions of easy "nothing here" examples. The focal loss turns down the volume on examples the model

already gets right.

Let $p \in [0,1]$ be the predicted probability of the foreground class and $y \in \{0,1\}$ the label. Define $p_t = p$ if $y = 1$ and $p_t = 1 - p$ otherwise. The **focal loss** [15] is

$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t),$$

where $\gamma \geq 0$ is the focusing parameter and α_t a class-balancing weight (α for foreground, $1-\alpha$ for background). With $\gamma = 0$ this is weighted cross-entropy. With the paper's default $\gamma = 2, \alpha = 0.25$, an easy negative with $p_t = 0.99$ is down-weighted by $(0.01)^2 = 10^{-4}$ relative to cross-entropy, while hard examples keep almost their full weight. The loss is normalized by the number of positive anchors. Two-stage detectors avoid the problem differently (the proposal stage removes most background, then a fixed foreground:background sampling ratio), and SSD [26] uses **hard negative mining** (keeping only the highest-loss negatives at about 3:1). The focal loss made a *dense* one-stage detector as accurate as two-stage ones for the first time.

Two-stage detectors: the R-CNN family

Plain version. First propose a few thousand places where *something* might be; then look closely at each one and decide *what* it is and exactly where its edges are.

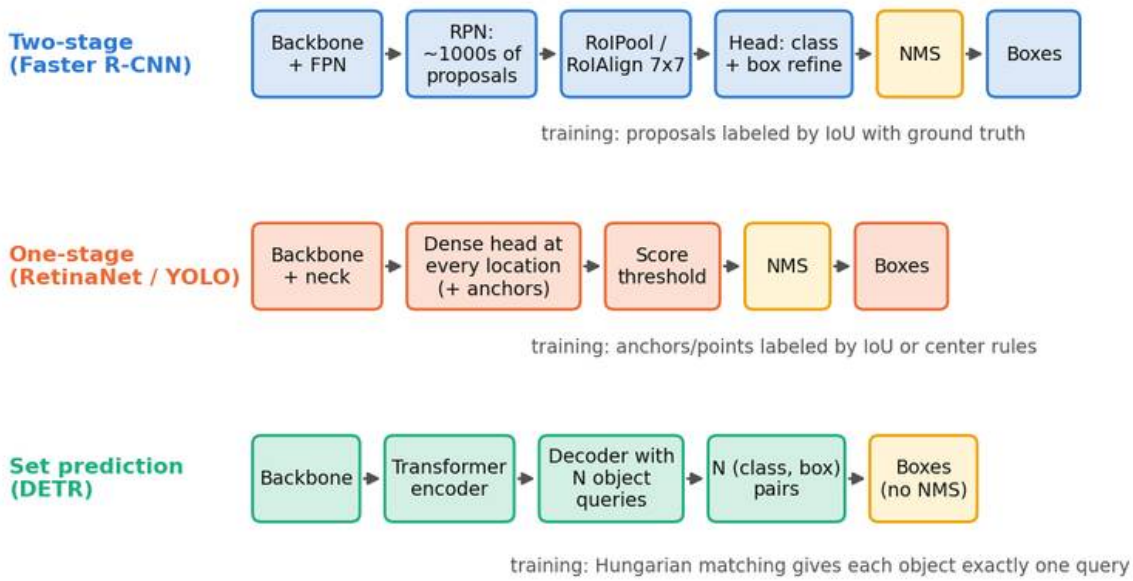


Figure 5 — Three detector pipelines. NMS (yellow) is a post-processing step for two-stage and one-stage detectors; DETR-style set prediction removes it by learning one-to-one assignments.

R-CNN (2014) [8] ran a bottom-up grouping algorithm to get about 2,000 class-agnostic region proposals per image, warped each one to a fixed size, passed it through a classification-pretrained CNN, scored the features with per-class linear SVMs and refined the box with the regression targets above. It improved mean AP on PASCAL VOC 2012 by more than 30% relative to the previous best result, and it established the recipe "pretrain on classification, fine-tune on detection" that is still standard. It was very slow, because the CNN ran separately on every proposal.

Fast R-CNN (2015) [9] runs the CNN *once* on the whole image and then crops each proposal from the shared feature map with **RoI pooling**: the proposal's projection onto the feature map is divided into a fixed $H \times W$ grid (e.g. 7×7) and each bin is max-pooled, so every proposal yields the same-size feature regardless of its shape. A single network is trained with a multi-task loss,

$$L = L_{\text{cls}}(p, u) + \lambda \, [u \geq 1] \, L_{\text{loc}}(t^u, v),$$

where p is the softmax over $K + 1$ classes, u the true class (0 for background), $[u \geq 1]$ is 1 for foreground proposals only, t^u the predicted box offsets for class u, v the target offsets and L_{loc} the smooth L1 loss.

Faster R-CNN (2015) [10] made the proposals themselves learned. A small **Region Proposal Network** (RPN) slides over the shared feature map and, for each of the 9 anchors per location, predicts an objectness score and box offsets. The top proposals (after NMS) feed the Fast R-CNN head. Because both stages share the backbone, proposals become almost free, and the whole detector is one network trained end to end.

Mask R-CNN (2017) [16] adds a small fully convolutional branch that predicts a binary mask per class inside each RoI, giving instance segmentation. Its key detail is **RoIAlign**: RoI pooling rounds the proposal boundaries and bin edges to the feature grid, which shifts features by up to half a stride. RoIAlign keeps real-valued coordinates and samples each bin with bilinear interpolation. This sounds minor, but pixel-accurate masks need it, and box accuracy at high IoU improves too.

Cascade R-CNN (2018) [17] addresses a subtle problem. A head trained with positives at IoU ≥ 0.5 learns to produce boxes that are "good at 0.5" but not at 0.75. Simply raising the threshold leaves too few positives and overfits. Cascade R-CNN chains several heads trained at increasing IoU thresholds (0.5, 0.6, 0.7), each refining the boxes of the previous one, so each stage receives a training distribution of higher quality. It was a large gain on the strict COCO metrics.

Two-stage detectors are accurate, flexible (the RoI head can predict masks, keypoints or attributes) and still the default for many instance-segmentation tasks, but the per-RoI head and NMS cost time.

One-stage detectors: YOLO, SSD and RetinaNet

Plain version. Skip the proposal step. Make one dense prediction at every location of the feature map in a single forward pass, then clean it up with NMS.

YOLO v1 (2016) [18] divided the image into an $S \times S$ grid ($S = 7$). Each cell predicted $B = 2$ boxes, each with a confidence, plus one set of C class probabilities, so the output was a single $S \times S \times (5B + C)$ tensor ($7 \times 7 \times 30$ for PASCAL VOC's 20 classes). The cell containing an object's center was responsible for it. It ran in real time and, because it saw the whole image, made fewer background errors than Fast R-CNN, but it localized less precisely and struggled with small objects and groups, since each cell could only predict a fixed number of boxes and one class.

The **YOLO family** then evolved quickly. The table lists only versions with a paper we could verify; many widely used releases are maintained as software packages without papers and are not covered here.

Version	Year	Main ideas (from the paper)
YOLO [18]	2016	Single-pass grid regression; real time
YOLO9000 / v2 [19]	2016 (arXiv)	Anchors chosen by k-means with an IoU distance; batch norm; higher-resolution training
YOLOv3 [20]	2018 (tech report)	Deeper backbone; predictions at three scales; independent logistic class scores
YOLOv4 [21]	2020 (arXiv)	Systematic study of training tricks ("bag of freebies") and cheap architectural add-ons ("bag of specials")
YOLOv7 [22]	2023 (CVPR)	Trainable bag-of-freebies: re-parameterized modules, improved label assignment, efficient layer aggregation
YOLOv9 [23]	2024 (ECCV)	Programmable gradient information (PGI) to keep information through deep layers; GELAN backbone
YOLOv10 [24]	2024 (NeurIPS)	NMS-free training with consistent dual (one-to-many and one-to-one) assignments; efficiency-driven design
YOLOv12 [25]	2025 (arXiv)	Attention-centric design that keeps real-time speed

A recurring theme is visible: the core "dense grid + NMS" idea stayed, while label assignment, necks, training recipes and, most recently, attention and NMS-free heads changed.

SSD (2016) [26] attached small convolutional predictors to *several* feature maps of different resolutions, each with its own set of default boxes (anchors) of several aspect ratios, which handled scale far better than YOLO v1. It trained with hard negative mining and heavy data augmentation.

RetinaNet (2017) [15] combined an FPN backbone, dense anchors (9 per location) and two small subnetworks for classification and box regression, trained with the focal loss. The paper's analysis showed that class imbalance, not the architecture, had been the main reason one-stage detectors lagged behind two-stage ones.

Anchor-free detectors: points instead of boxes

Plain version. Anchors are guesses about what objects look like. Anchor-free detectors drop them and predict boxes from *points*: corners, centers, or every pixel inside the object.

CornerNet (2018) [27] detects an object as a pair of keypoints: a heatmap for top-left corners, a heatmap for bottom-right corners, and an **embedding** per corner. Corners from the same object are trained to have similar embeddings, so pairing is done by embedding distance. Because a corner often lies outside the object, the paper introduced **corner pooling**, which looks along rows and columns to collect evidence for a corner.

CenterNet, "Objects as Points" (2019) [28] represents each object by its center. The network outputs a heatmap $\hat{Y} \in [0,1]^{\frac{W}{R} \times \frac{H}{R} \times K}$ (output stride R, one channel per class), plus a size (w, h) and a sub-pixel offset at each location. Ground-truth centers are splatted with a Gaussian, and the heatmap is trained with a pixel-wise focal loss. At inference, peaks are simply local maxima, found with a 3×3 max-pooling comparison, so **no IoU-based NMS** is needed. The same design extends to 3D boxes and human pose by regressing more quantities at the center.

FCOS (2019) [29] treats detection like segmentation. Every feature-map location that falls inside a ground-truth box is a positive sample and regresses the four distances to the box sides,

$$l^* = x - x_1, \qquad t^* = y - y_1, \qquad r^* = x_2 - x, \qquad b^* = y_2 - y,$$

where (x, y) is the location in image coordinates and (x_1, y_1, x_2, y_2) the box. Each FPN level only handles objects whose largest regression distance falls in its own range, which resolves most ambiguity when boxes overlap. Locations far from the object's center produce poor boxes, so FCOS adds a **centerness** branch,

$$\text{centerness} = \sqrt{\frac{\min(l^*, r^*)}{\max(l^*, r^*)} \cdot \frac{\min(t^*, b^*)}{\max(t^*, b^*)}},$$

that is 1 at the center and decays toward the edges; it multiplies the class score at test time so off-center boxes rank lower before NMS. FCOS removed every anchor hyperparameter and matched or beat anchor-based RetinaNet.

Set prediction with transformers: DETR and its descendants

Plain version. Instead of predicting thousands of candidates and deleting duplicates, predict a small *set* of answers directly, and train so that each real object is claimed by exactly one prediction.

DETR (2020) [30] uses a CNN backbone, a transformer encoder over the feature map, and a transformer decoder that takes N learned **object queries** (100 in the paper) and outputs N (class, box) predictions in parallel. There are no anchors, no proposals and no NMS. Since N exceeds the number of objects, a special "no object" class $\emptyset$ pads the ground truth to size N.

The key is training with **bipartite matching**. Let $y = \{y_j\}$ be the padded ground truth and $\hat{y} = \{\hat{y}_i\}_{i=1}^{N}$ the predictions. Find the permutation $\hat{\sigma}$ of N elements with lowest total matching cost,

$$\hat{\sigma} = \arg\min_{\sigma \in \mathfrak{S}_N} \sum_{j=1}^{N} \mathcal{L}_{\text{match}}(y_j, \hat{y}_{\sigma(j)}), \qquad \mathcal{L}_{\text{match}} = -\mathbb{1}_{\{c_j \neq \emptyset\}}\, \hat{p}_{\sigma(j)}(c_j) + \mathbb{1}_{\{c_j \neq \emptyset\}}\, \mathcal{L}_{\text{box}}(b_j, \hat{b}_{\sigma(j)}),$$

where $\mathfrak{S}_N$ is the set of permutations, c_j and b_j are the class and box of target j, $\hat{p}_{\sigma(j)}(c_j)$ is the predicted probability of class c_j for the matched query, and

$\mathcal{L}_{\text{box}} = \lambda_{L1}\|b_j - \hat{b}_{\sigma(j)}\|_1 + \lambda_{\text{iou}}(1 - \text{GIoU}(b_j, \hat{b}_{\sigma(j)}))$. This assignment problem is solved exactly by the **Hungarian algorithm** [31] in polynomial time. The training loss (the **set loss**) then uses the matched pairs:

$$\mathcal{L}_{\text{Hungarian}} = \sum_{j=1}^{N} \Big[-\log \hat{p}_{\hat{\sigma}(j)}(c_j) + \mathbb{1}_{\{c_j \neq \emptyset\}}\, \mathcal{L}_{\text{box}}(b_j, \hat{b}_{\hat{\sigma}(j)}) \Big],$$

with the log-probability term down-weighted by a factor of 10 for $c_j = \emptyset$ to balance the many "no object" slots. Because each object is matched to exactly one query, a duplicate prediction is *penalized* as a false "no object", and the decoder's self-attention learns to suppress duplicates, which is why NMS is unnecessary.

The matching step is short with SciPy. The cost below combines the three terms with DETR's weights; `linear_sum_assignment` accepts the rectangular $N \times M$ matrix directly (DETR computes the L1 term on normalized (c_x, c_y, w, h); corner format is used here for brevity).

```
import numpy as np
from scipy.optimize import linear_sum_assignment

def giou_matrix(a, b):
    x1 = np.maximum(a[:, None, 0], b[None, :, 0]); y1 = np.maximum(a[:, None, 1], b[None, :,
     ↪  1])
    x2 = np.minimum(a[:, None, 2], b[None, :, 2]); y2 = np.minimum(a[:, None, 3], b[None, :,
     ↪  3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[:, 2] - b[:, 0]) * (b[:, 3] - b[:, 1])
```

```
    union = area(a)[:, None] + area(b)[None, :] - inter
    cx1 = np.minimum(a[:, None, 0], b[None, :, 0]); cy1 = np.minimum(a[:, None, 1], b[None,
     ↪  :, 1])
    cx2 = np.maximum(a[:, None, 2], b[None, :, 2]); cy2 = np.maximum(a[:, None, 3], b[None,
     ↪  :, 3])
    c = (cx2 - cx1) * (cy2 - cy1)
    return inter / union - (c - union) / c

rng = np.random.default_rng(1)
N, C = 6, 3                                      # 6 queries, 3 real classes (+ "no object")
logits = rng.normal(size=(N, C + 1))
prob = np.exp(logits) / np.exp(logits).sum(1, keepdims=True)
pred_boxes = np.array([[.10, .10, .40, .50], [.55, .20, .90, .60], [.12, .08, .42, .48],
                       [.30, .60, .60, .95], [.00, .00, .20, .20], [.50, .50, .70, .70]])
gt_boxes  = np.array([[.10, .10, .40, .50], [.32, .62, .62, .93], [.56, .22, .88, .62]])
gt_labels = np.array([0, 2, 1])

w_cls, w_l1, w_giou = 1.0, 5.0, 2.0              # weights used by DETR
cost = (-w_cls * prob[:, gt_labels]                          # (N, M) class term
        + w_l1 * np.abs(pred_boxes[:, None] - gt_boxes[None]).sum(-1)
        - w_giou * giou_matrix(pred_boxes, gt_boxes))
rows, cols = linear_sum_assignment(cost)         # works on rectangular N x M matrices
print("query -> gt:", dict(zip(rows.tolist(), cols.tolist())))
print("unmatched queries (trained as 'no object'):", sorted(set(range(N)) -
 ↪  set(rows.tolist())))
```

```
query -> gt: {0: 0, 1: 2, 3: 1}
unmatched queries (trained as 'no object'): [2, 4, 5]
```

Query 2 is a near-duplicate of query 0, yet it is left unmatched and will be trained toward "no object". That is the mechanism that replaces NMS.

The price. DETR needed very long training schedules (hundreds of epochs, compared with roughly a dozen for Faster R-CNN) and was weak on small objects, because global attention over a high-resolution feature map is expensive, so it used only a single coarse level. The descendants fixed these problems one by one.

- **Deformable DETR (2021)** [32] replaces dense attention with **deformable attention**: each query attends to only a few learned sampling points around a reference point, across multiple feature levels. This makes multi-scale features affordable and, according to the paper, reaches better accuracy than DETR (especially on small objects) with 10 times fewer training epochs.
- **DINO (2022)** [33] (not to be confused with the self-supervised method of the same name) combines *contrastive denoising training* (noised copies of ground-truth boxes are fed as extra queries and must be reconstructed, while slightly more noised ones must be rejected), *mixed query selection* (initializing anchor positions from encoder features) and a *look-forward-twice* box update. It reports 49.4 AP after 12 epochs and 51.3 AP after 24 epochs on COCO with a standard

50-layer CNN backbone, and 63.2 AP on COCO val2017 with a large backbone and Objects365 pretraining.

- **RT-DETR (2024)** [34] targets real-time use. An *efficient hybrid encoder* applies attention only to the coarsest level and fuses scales with convolutions, and *IoU-aware query selection* picks better initial queries. The paper reports 53.1 / 54.3 AP on COCO at 108 / 74 FPS on a T4 GPU for its smaller and larger variants, faster and more accurate than YOLO models of comparable size at the time, without NMS.

Today the DETR line and the YOLO line are converging: YOLOv10 borrows one-to-one assignment to drop NMS, and real-time DETRs borrow efficient convolutional necks.

Open-vocabulary and grounding detectors

Plain version. A closed-set detector can only find the 80 or 365 classes it was trained on. An open-vocabulary detector takes the class list *as text at test time*, so you can ask for "red fire hydrant" or "forklift" without retraining.

The trick is to replace the last classification layer, a fixed matrix of K class weight vectors, with **text embeddings**. For a region feature $\mathbf{z}_i$ and the text embedding $\mathbf{e}_k$ of a prompt such as "a photo of a forklift", the class score is a scaled cosine similarity,

$$s_{ik} = \tau \cdot \frac{\mathbf{z}_i^\top \mathbf{e}_k}{\|\mathbf{z}_i\| \, \|\mathbf{e}_k\|},$$

where τ is a learned temperature. Any phrase you can embed becomes a class. The vision side must learn to align regions with language, which requires image–text data far larger than any box-annotated dataset.

- **OWL-ViT (2022)** [35] starts from a transformer image encoder and a text encoder pretrained contrastively on image–text pairs, removes the final pooling, and attaches light box and class heads to every output token. The class head is the text embedding above. It supports zero-shot (text query) and one-shot (image-exemplar query) detection, and the paper shows consistent gains with model scale.
- **Grounding DINO (2024)** [36] fuses language deeply into a DINO detector: a feature enhancer with cross-attention between image and text, language-guided query selection, and a cross-modality decoder. It is pretrained on detection, grounding (phrase-to-region) and caption data, and reports 52.5 AP on COCO zero-shot transfer (with no COCO training data) and a 26.1 mean AP on a benchmark collection of diverse real-world detection datasets.
- **YOLO-World (2024)** [37] brings open vocabulary to real-time detectors with a vision–language path aggregation network (RepVL-PAN) and region–text contrastive pretraining. Its *prompt-then-detect* mode encodes the user's vocabulary once offline, so inference costs about the same as a closed-set YOLO. It reports 35.4 AP on LVIS at 52.0 FPS on a V100 GPU.

Promptable foundation models go one step further. SAM 3 [38] (2025) accepts a short noun phrase or image exemplars as a *concept prompt* and returns masks with identities for **all** matching instances in images and videos; the authors built a data engine producing a dataset with 4M unique concept labels and report doubling the accuracy of existing systems on their promptable concept segmentation benchmark. In such systems, detection becomes one output of a general "find this concept" model rather than a separately trained network.

Small objects and dense scenes

Plain version. Tiny objects have very few pixels, and stride-32 features may squeeze them into less than one cell. Crowds are hard because objects hide each other and NMS confuses neighbors with duplicates.

Small objects. COCO defines "small" as area below 32^2 pixels [40]. After a stride-16 backbone, such an object covers at most about 2×2 feature cells, and its appearance is mostly lost in downsampling. IoU also becomes harsh: a 2-pixel error on a 10×10 box can drop IoU below 0.7. Common remedies are high-resolution feature levels (P_2 with stride 4), feature pyramids, image tiling or slicing at inference, larger input resolutions, copy-paste and scale augmentation, and assignment rules that do not starve small boxes of positive samples. The survey and benchmark of Cheng et al. [39] reviews these strategies and introduces two large small-object benchmarks: SODA-D (24,828 traffic images, 278,433 instances, 9 categories) and SODA-A (2,513 high-resolution aerial images, 872,069 instances, 9 classes).

Dense and occluded scenes. In crowds, true objects overlap heavily, so a fixed NMS threshold trades recall against duplicates. Soft-NMS [13] helps; set-prediction detectors avoid the threshold altogether by learning which predictions are duplicates. Occlusion also makes box annotation ambiguous (full extent or visible part?), and evaluation then depends on the protocol. These issues return in tracking, where objects disappear and reappear.

Evaluating detectors

Plain version. Sort all detections by confidence. Walk down the list: each detection is either a hit (it matches an unclaimed real object well enough) or a false alarm. Track how precision and recall change; the area under that curve is the Average Precision.

Precision, recall and AP

For one class and one IoU threshold θ, sort all detections in the test set by descending score. A detection is a **true positive** (TP) if its IoU with a not-yet-matched ground-truth box of the same

class is at least θ; otherwise it is a **false positive** (FP). Each ground-truth box can be matched only once, which is how duplicates are punished. After the top k detections,

$$\text{precision}(k) = \frac{\text{TP}(k)}{k}, \qquad \text{recall}(k) = \frac{\text{TP}(k)}{M},$$

where M is the number of ground-truth objects of that class. To make the curve monotone, precision is **interpolated** with its upper envelope, $p_{\text{interp}}(r) = \max_{r' \geq r} p(r')$. Then:

- **VOC 11-point AP** (PASCAL VOC up to 2009 [3]): $\text{AP} = \frac{1}{11} \sum_{r \in \{0, 0.1, \ldots, 1\}} p_{\text{interp}}(r)$.
- **VOC all-point AP** (VOC 2010 onwards [41]): the exact area under the interpolated curve, $\text{AP} = \sum_n (r_{n+1} - r_n)\, p_{\text{interp}}(r_{n+1})$ over the distinct recall values.
- **COCO AP** [40]: the mean of p_{interp} at 101 recall points $0, 0.01, \ldots, 1$.

Figure 6 shows a PR curve, its envelope and the AP area.

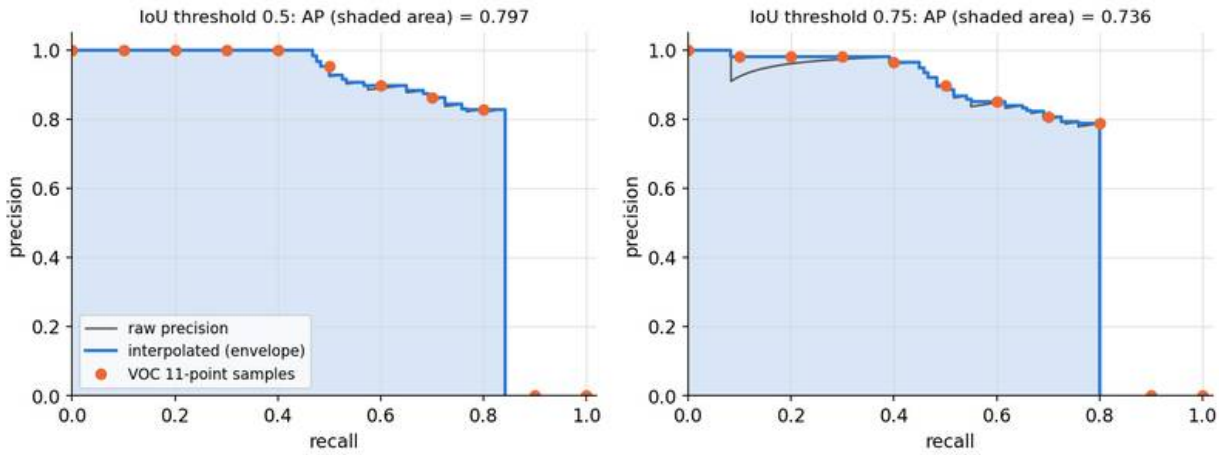


Figure 6 — Precision–recall on synthetic detections at two IoU thresholds. The shaded area under the interpolated envelope is the all-point AP; orange dots are the VOC 11-point samples. Precision drops to 0 where recall ends, because some objects were never detected.

The COCO protocol

COCO's primary metric, written **AP** or **mAP@[.5:.95]**, averages AP over 10 IoU thresholds $\theta \in \{0.50, 0.55, \ldots, 0.95\}$ and over all 80 categories [40]:

$$\text{AP}_{\text{COCO}} = \frac{1}{10\,K} \sum_{\theta} \sum_{k=1}^{K} \text{AP}_k(\theta).$$

It rewards precise localization, not just finding the object. Companion metrics are AP_{50} (the VOC-style metric), AP_{75} (strict), and AP_S, AP_M, AP_L for small (area $< 32^2$), medium (32^2 to 96^2) and large ($> 96^2$) objects. At most 100 detections per image are scored. Average recall (AR) at 1, 10 and 100 detections per image is also reported. Note that COCO calls its category-averaged metric "AP"; "mAP" in papers usually means the same thing.

The code below computes everything from scratch on synthetic predictions (jittered ground truth, about 15% misses and some random false positives).

```
import numpy as np

def box_iou(a, b):
    """Pairwise IoU between (N,4) and (M,4) xyxy boxes -> (N, M)."""
    x1 = np.maximum(a[:, None, 0], b[None, :, 0]); y1 = np.maximum(a[:, None, 1], b[None, :,
     ↪  1])
    x2 = np.minimum(a[:, None, 2], b[None, :, 2]); y2 = np.minimum(a[:, None, 3], b[None, :,
     ↪  3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[:, 2] - b[:, 0]) * (b[:, 3] - b[:, 1])
    return inter / (area(a)[:, None] + area(b)[None, :] - inter)

def match(dets, gts, thr):
    """dets: list over images of (boxes, scores); gts: list of gt boxes.
    Returns scores, TP flags (sorted by score), and number of GT."""
    recs = []
    for img, ((db, ds), gb) in enumerate(zip(dets, gts)):
        for k in range(len(ds)):
            recs.append((ds[k], img, k))
    recs.sort(key=lambda r: -r[0])                      # global ranking by confidence
    used = [np.zeros(len(g), bool) for g in gts]
    tp = np.zeros(len(recs))
    for n, (s, img, k) in enumerate(recs):
        gb = gts[img]
        if len(gb) == 0:
            continue
        ious = box_iou(dets[img][0][k:k + 1], gb)[0]
        ious[used[img]] = -1                            # each GT can be matched once
        j = ious.argmax()
        if ious[j] >= thr:
            tp[n] = 1; used[img][j] = True
    return tp, sum(len(g) for g in gts)

def pr_curve(tp, n_gt):
    ctp, cfp = np.cumsum(tp), np.cumsum(1 - tp)
    return ctp / n_gt, ctp / (ctp + cfp)                # recall, precision

def ap(rec, prec, mode="all"):
    r = np.concatenate([[0], rec, [1]]); p = np.concatenate([[0], prec, [0]])
    p = np.maximum.accumulate(p[::-1])[::-1]            # monotone envelope
    if mode == "all":                                   # VOC2010+: exact area
        i = np.where(r[1:] != r[:-1])[0]
        return np.sum((r[i + 1] - r[i]) * p[i + 1])
    pts = np.linspace(0, 1, 11 if mode == "voc11" else 101)
```

```
    return np.mean([p[r >= t].max() if np.any(r >= t) else 0 for t in pts])

# synthetic data: 50 images, 1-4 objects each; detector = jittered GT + false positives
rng = np.random.default_rng(0)
gts, dets = [], []
for _ in range(50):
    n = rng.integers(1, 5)
    xy = rng.uniform(0, 200, (n, 2)); wh = rng.uniform(20, 80, (n, 2))
    g = np.hstack([xy, xy + wh]); gts.append(g)
    keep = rng.random(n) > 0.15                          # miss ~15% of objects
    jit = rng.normal(0, 0.06, (n, 4)) * np.hstack([wh, wh])
    db = (g + jit)[keep]; ds = rng.uniform(0.4, 1.0, keep.sum())
    m = rng.integers(0, 3)                               # 0-2 false positives
    fxy = rng.uniform(0, 200, (m, 2)); fwh = rng.uniform(20, 80, (m, 2))
    db = np.vstack([db, np.hstack([fxy, fxy + fwh])]); ds = np.concatenate([ds,
     ↪  rng.uniform(0, 0.7, m)])
    dets.append((db, ds))

for thr in (0.5, 0.75):
    tp, n_gt = match(dets, gts, thr)
    rec, prec = pr_curve(tp, n_gt)
    print(f"IoU {thr}: VOC11 {ap(rec, prec, 'voc11'):.3f}  all-point {ap(rec, prec):.3f}"
          f"  COCO-101 {ap(rec, prec, 'coco'):.3f}")
aps = [ap(*pr_curve(*match(dets, gts, t)), "coco") for t in np.arange(0.5, 0.96, 0.05)]
print("AP@[.5:.95] =", round(float(np.mean(aps)), 3))
```

```
IoU 0.5: VOC11 0.776  all-point 0.797  COCO-101 0.797
IoU 0.75: VOC11 0.749  all-point 0.736  COCO-101 0.738
AP@[.5:.95] = 0.532
```

Three lessons are visible. The three interpolation rules give similar but *not identical* numbers, so AP values computed with different protocols must never be compared. AP@[.5:.95] is much lower than AP50 because jittered boxes fail the strict thresholds. And this is a single class; real mAP averages over classes, so rare classes count as much as frequent ones.

What AP does not tell you

AP summarizes ranking quality over the whole test set, so it does not choose an operating threshold, and it does not separate *types* of errors: localization errors, confusion with similar classes, duplicates and background false positives all look the same. When diagnosing a model, break errors down by type and by object size, and look at the images.

Datasets and benchmarks

Plain version. Detection progress has been driven by a few public datasets; each one changed what "good" meant.

Dataset	Classes	Scale (from the papers)	Annotations	Why it matters
PASCAL VOC [3]	20	Small by today's standards	Boxes (+ some masks)	The 2005–2012 benchmark; AP50
MS COCO [4]	91 types labeled; 80 used for detection	328k images, 2.5M labeled instances	Boxes + instance masks	Cluttered everyday scenes, many small objects; AP@[.5:.95]
Objects365 [42]	365	Large-scale, high-quality boxes	Boxes	Common pretraining set for strong detectors
Open Images V4 [43]	600 boxable classes	9.2M images; 15.4M boxes	Boxes, image labels, relationships	Very large; 19.8k concepts at image level
LVIS [44]	over 1000	~2M masks in 164k images	Instance masks	Long-tailed vocabulary; rare-class evaluation

Two trends stand out. Category counts grew (20, then 80, then hundreds and over a thousand), and with them came **long-tail** problems: in LVIS many categories have only a handful of training examples, so a loss or sampler that ignores frequency fails on rare classes. And the biggest models are now pretrained on Objects365 or on image–text data before fine-tuning on COCO, which makes "trained on COCO only" and "with extra data" separate leaderboards.

Speed, accuracy and deployment

Plain version. In practice you pick a point on a curve: more accuracy usually means more computation. What counts is the latency on *your* hardware, end to end.

- **FLOPs are not latency.** FLOPs count arithmetic, but runtime also depends on memory traffic, operator support and parallelism. Depth-wise convolutions and attention can be cheap in FLOPs but slow on some accelerators.
- **Measure end to end.** Report latency including preprocessing, the network *and* post-processing. NMS cost grows with the number of candidate boxes (worst case quadratic in their number), is data-dependent, and often runs on the CPU. RT-DETR's paper [34] analyses exactly this and is one reason NMS-free designs such as RT-DETR and YOLOv10 [24] are attractive for deployment.
- **Input resolution** is the strongest knob: halving the side length cuts compute by about 4 times but hurts small objects most.
- **Quantization** (e.g. 8-bit integer weights and activations) and **re-parameterization** (training a multi-branch block, then folding it into a single convolution, as used by YOLOv7 [22]) reduce latency with small accuracy loss. Box-regression outputs and the score head are often the most sensitive to quantization, so calibrate with real data and check AP75, not just AP50.

- **Batch size 1** is the usual edge setting; throughput numbers measured with large batches do not transfer.

A tiny detector you can train

To see box regression and an IoU loss work end to end, the PyTorch script below trains a small CNN to localize one rectangle or ellipse in noisy 32×32 images. It is a single-object detector: the network directly regresses (c_x, c_y, w, h) in $[0, 1]$ through a sigmoid, trained with L1 plus a GIoU loss, as DETR does for its matched queries.

```
import time, torch, torch.nn as nn
torch.manual_seed(0)
S = 32
torch.set_num_threads(4)

def make_batch(n):
    """One white rectangle or ellipse per 32x32 noisy image; target = (cx, cy, w, h) in [0,
    ↪ 1]."""
    img = 0.1 * torch.randn(n, 1, S, S)
    wh = torch.rand(n, 2) * 0.4 + 0.15
    c = wh / 2 + torch.rand(n, 2) * (1 - wh)
    yy, xx = torch.meshgrid(torch.arange(S), torch.arange(S), indexing="ij")
    xx, yy = (xx[None] + 0.5) / S, (yy[None] + 0.5) / S
    dx = (xx - c[:, 0, None, None]) / (wh[:, 0, None, None] / 2)
    dy = (yy - c[:, 1, None, None]) / (wh[:, 1, None, None] / 2)
    rect = (dx.abs() <= 1) & (dy.abs() <= 1)
    ell = dx**2 + dy**2 <= 1
    shape = torch.where((torch.rand(n) < 0.5)[:, None, None], rect, ell)
    return img + shape[:, None].float(), torch.cat([c, wh], 1)

def xyxy(b):
    return torch.cat([b[:, :2] - b[:, 2:] / 2, b[:, :2] + b[:, 2:] / 2], 1)

def giou(p, g):
    p, g = xyxy(p), xyxy(g)
    lt, rb = torch.max(p[:, :2], g[:, :2]), torch.min(p[:, 2:], g[:, 2:])
    inter = (rb - lt).clamp(min=0).prod(1)
    area = lambda b: (b[:, 2:] - b[:, :2]).prod(1)
    union = area(p) + area(g) - inter
    c = (torch.max(p[:, 2:], g[:, 2:]) - torch.min(p[:, :2], g[:, :2])).prod(1)
    return inter / union, inter / union - (c - union) / c

net = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 16
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 8
    nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 4
    nn.Flatten(), nn.Linear(32 * 4 * 4, 128), nn.ReLU(),
    nn.Linear(128, 4), nn.Sigmoid())                               # (cx, cy, w, h) in [0, 1]
opt = torch.optim.Adam(net.parameters(), 2e-3)
```

```
t0 = time.time()
for step in range(500):
    x, y = make_batch(64)
    pred = net(x)
    iou, g = giou(pred, y)
    loss = (pred - y).abs().sum(1).mean() + (1 - g).mean()        # L1 + GIoU loss
    opt.zero_grad(); loss.backward(); opt.step()

with torch.no_grad():
    x, y = make_batch(1000)
    iou, _ = giou(net(x), y)
print(f"train time {time.time() - t0:.1f}s | test mean IoU {iou.mean():.3f}"
      f" | IoU>=0.5: {(iou >= 0.5).float().mean():.1%} | IoU>=0.75: {(iou >=
      ↪ 0.75).float().mean():.1%}")
```

```
train time 21.1s | test mean IoU 0.885 | IoU>=0.5: 100.0% | IoU>=0.75: 97.8%
```

(Training time depends on your CPU.) Because there is exactly one object, there is no assignment and no NMS. Exercise 6 asks you to extend this to several objects, which is where every hard problem in this tutorial reappears.

Modern view

Figure 7 places the methods of this tutorial on a timeline, using the year of each cited publication.

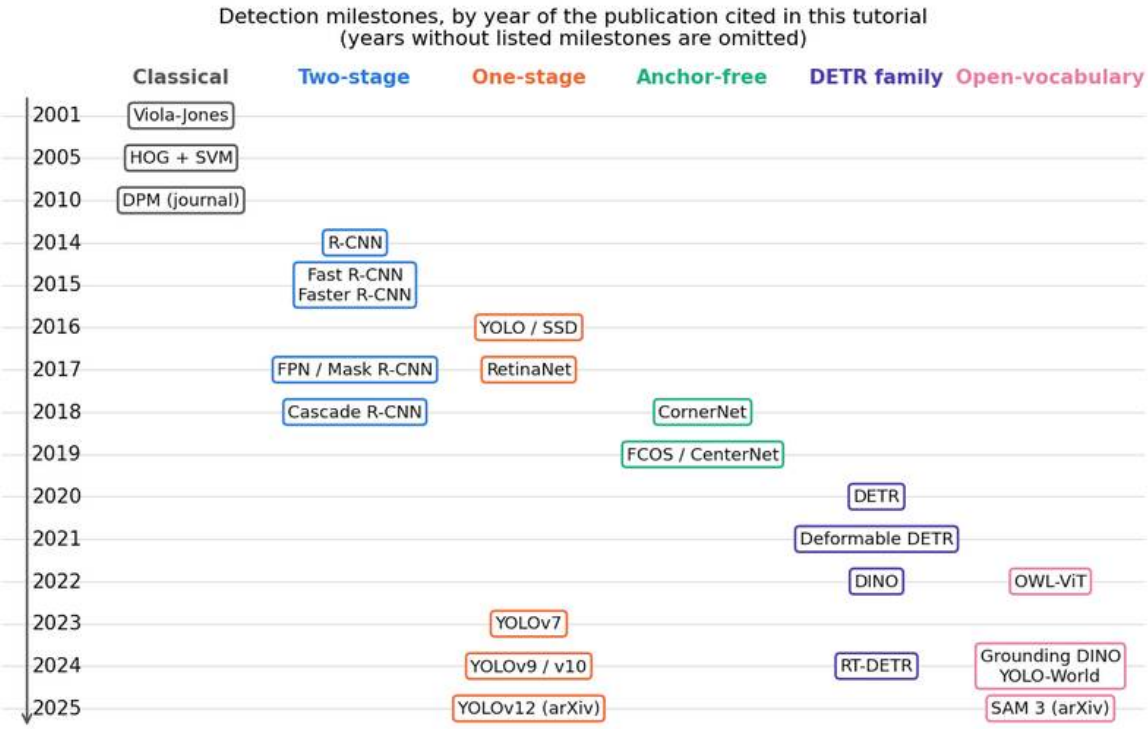


Figure 7 — Milestones by year of the publication cited in this tutorial, grouped by family. Years without a listed milestone are omitted.

What the surveys say

Object Detection in 20 Years (Zou et al., *Proceedings of the IEEE*, 2023) [1] is the broadest historical review. It covers milestone detectors from the traditional era (Viola–Jones, HOG, DPM) to deep two-stage and one-stage methods, detection datasets and metrics, fundamental building blocks (multi-scale detection, context, hard-negative mining, NMS, box regression), speed-up techniques, and recent state-of-the-art methods. Its main message is that most "new" ideas have older roots: multi-scale scanning became feature pyramids, cascades became cascaded heads, hard-negative mining became the focal loss, and NMS has been rethought several times.

Deep Learning for Generic Object Detection: A Survey (Liu et al., *IJCV*, 2020) [2] reviews over 300 contributions in depth and organizes them by detection framework, object feature representation, object proposal generation, context modeling, training strategies and evaluation metrics. It is the best single reference for the pre-transformer CNN era, and it stresses that progress came as much from training strategies, data and evaluation as from architectures.

Object Detection with Transformers: A Review (Shehzadi et al., *Sensors*, 2025) [45] analyses 21 recent improvements to DETR, grouped by changes to the backbone, query design and attention

mechanism, and compares them on common benchmarks. Its taxonomy tracks the fixes described above: multi-scale and sparse attention for small objects and speed, and better query initialization and denoising for convergence.

A Survey on Open-Vocabulary Detection and Segmentation: Past, Present, and Future (Zhu and Chen, *IEEE TPAMI*, 2024) [46] organizes open-vocabulary methods by how they use weak supervision into six families: visual–semantic space mapping, novel visual feature synthesis, region-aware training, pseudo-labeling, knowledge distillation and transfer learning. It covers 2D, 3D and video tasks, and it highlights how evaluation of "novel" classes is complicated by vocabulary overlap between pretraining and test data.

Towards Large-Scale Small Object Detection (Cheng et al., *IEEE TPAMI*, 2023) [39] focuses on the most stubborn failure mode, reviews the remedies by category and contributes the SODA benchmarks described earlier. It shows that small-object accuracy remains far below that of larger objects even for strong detectors.

The state of the art, 2023–2026

- **Real-time closed-set detection** is now a contest between the YOLO line (YOLOv9 [23], YOLOv10 [24], YOLOv12 [25]) and real-time DETRs (RT-DETR [34]). The two lines borrow from each other: one-to-one assignment and NMS-free heads in YOLO, convolutional necks and hybrid encoders in DETRs. Attention is now affordable at real-time budgets.
- **High-accuracy closed-set detection** is dominated by DETR-style models such as DINO [33] with large backbones and large-scale pretraining (Objects365 [42] and beyond). Gains on COCO have become small, and the benchmark is close to saturated for the largest models, which is pushing attention to harder benchmarks: LVIS [44], open vocabulary and cross-domain evaluation.
- **Open-vocabulary and grounding detection** (OWL-ViT [35], Grounding DINO [36], YOLO-World [37]) is moving into practice: in many applications you now *prompt* a detector rather than train one, and then optionally fine-tune it.
- **Promptable foundation models** (SAM 3 [38]) unify detection, segmentation and tracking under concept prompts.

Open problems

1. **Localization quality versus confidence.** Classification score and box quality are trained separately and are often poorly correlated, so the "best" box can be suppressed by a more confident but worse one.
2. **Small, crowded and occluded objects** remain far from solved [39].

3. **Long-tail and open-world recognition.** Rare classes and truly unseen objects, and knowing *when* the model does not know.
4. **Evaluation.** COCO AP hides error types, is saturated at the top, and open-vocabulary evaluation depends on prompts and on what the pretraining data already contained [46].
5. **Efficiency on real hardware**, including NMS-free designs, quantization-friendly heads and fair latency reporting.

Key takeaways

- Detection outputs a variable-size set of (box, class, score) triples; every architecture is a different answer to "how many, and which one is responsible for which object?"
- Boxes are regressed as scale-normalized offsets from a reference $(t_x, t_y$, log-scale $t_w, t_h)$; IoU-based losses (GIoU, DIoU, CIoU) align training with the IoU-based metric.
- Anchors and assignment rules decide which predictions learn which object; NMS removes duplicates afterwards, and Soft-NMS softens its failure in crowds.
- FPN handles scale; the focal loss handles the foreground–background imbalance that made dense detectors weak.
- Two-stage (R-CNN, Fast, Faster, Mask, Cascade) refine proposals; one-stage (YOLO, SSD, RetinaNet) predict densely in one pass; anchor-free (CornerNet, CenterNet, FCOS) predict from points; DETR predicts a set with Hungarian matching and needs no NMS.
- Open-vocabulary detectors turn the class list into text prompts, and promptable foundation models merge detection, segmentation and tracking.
- Measure with the right protocol: VOC 11-point, VOC all-point and COCO 101-point AP differ; COCO AP averages IoU 0.50–0.95, and APs/m/l expose scale weaknesses.

Exercises

1. **Encode by hand.** An anchor has center $(100, 100)$ and size 64×32. A ground-truth box has center $(110, 96)$ and size 80×40. Compute (t_x, t_y, t_w, t_h) without normalization, then decode them back.

   Hint

   $t_x = 10/64 = 0.156$, $t_y = -4/32 = -0.125$, $t_w = \log(80/64) = 0.223$, $t_h = \log(40/32) = 0.223$. Decoding: $c_x = 100 + 0.156 \cdot 64 = 110$, $w = 64e^{0.223} = 80$, and similarly for y and h.

2. **IoU versus shift.** Show that two equal $a \times a$ squares shifted by d along one axis have IoU $= (a - d)/(a + d)$. What shift gives IoU 0.5 and 0.75? Repeat for an equal shift along both axes (Figure 2, right).

Hint

Intersection $a(a-d)$, union $2a^2 - a(a-d) = a(a+d)$. IoU 0.5 at $d = a/3$; IoU 0.75 at $d = a/7$. For a diagonal shift, IoU $= (a-d)^2/(2a^2-(a-d)^2)$, so IoU 0.5 at $d = a(1-\sqrt{2/3}) \approx 0.18a$.

3. **When greedy NMS fails.** Modify the NMS example so that two different people overlap with IoU 0.6, and run greedy NMS at thresholds 0.5 and 0.7, and Soft-NMS. Which recovers both people, and what new problem does a high threshold create?

Hint

At 0.5 the lower-scored person is deleted. At 0.7 both survive, but duplicates of the same person with IoU between 0.5 and 0.7 also survive, adding false positives. Soft-NMS keeps the second person with a reduced score ($e^{-0.36/0.5} \approx 0.49$ times its original score), which still counts toward recall in AP.

4. **Focal loss numbers.** With $\gamma = 2$ and $\alpha = 0.25$, compute the focal loss and the plain cross-entropy for a background anchor with $p = 0.01$ (easy) and $p = 0.6$ (hard). By what factor does each change?

Hint

For background, $p_t = 1-p$ and $\alpha_t = 0.75$. Easy: CE $= -\log 0.99 = 0.01005$; FL $= 0.75 \cdot 0.01^2 \cdot 0.01005 \approx 7.5\times10^{-7}$, a factor of about 1.3×10^4 smaller. Hard: CE $= -\log 0.4 = 0.916$; FL $= 0.75 \cdot 0.6^2 \cdot 0.916 \approx 0.247$, only about 3.7 times smaller.

5. **Protocols disagree.** Using the AP code, find (by changing the random seed or the jitter) a case where VOC 11-point AP is *higher* than all-point AP, and one where it is lower. Explain why neither is always larger.

Hint

The 11-point rule samples the envelope at fixed recalls, so it overestimates when the envelope is high just to the right of a sample point and drops right after it, and underestimates in the opposite case. The output above already shows both: at IoU 0.5 the 11-point value is lower, at 0.75 it is higher.

6. **From one object to many.** Extend the tiny PyTorch detector to images with up to three shapes. Option A: predict an 8×8 grid of (objectness, box) like YOLO v1, assigning each object to the cell containing its center. Option B: predict 5 (class, box) slots and train with Hungarian matching like DETR. Compare the failure modes.

Hint

Option A fails when two centers fall in the same cell and needs NMS to remove duplicates from neighboring cells. Option B needs no NMS, but trains more slowly, and early in training the matching is unstable (the same object may be assigned to different slots from step to step). Use `scipy.optimize.linear_sum_assignment` on a detached cost matrix.

Detections are the raw material of tracking. The next step, linking boxes across video frames and keeping identities, is covered in DLCV 06 ·Object Tracking.

References

1. Z. Zou, K. Chen, Z. Shi, Y. Guo and J. Ye, "Object detection in 20 years: A survey," *Proceedings of the IEEE*, vol. 111, no. 3, 2023. doi
2. L. Liu, W. Ouyang, X. Wang, P. Fieguth, J. Chen, X. Liu and M. Pietikäinen, "Deep learning for generic object detection: A survey," *International Journal of Computer Vision*, 2020. doi
3. M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn and A. Zisserman, "The PASCAL Visual Object Classes (VOC) challenge," *International Journal of Computer Vision*, vol. 88, 2010. doi
4. T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick et al., "Microsoft COCO: Common objects in context," in *Proc. ECCV*, 2014. doi
5. P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE CVPR*, 2001. doi
6. N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE CVPR*, 2005. doi
7. P. F. Felzenszwalb, R. B. Girshick, D. McAllester and D. Ramanan, "Object detection with discriminatively trained part-based models," *IEEE TPAMI*, 2010. doi
8. R. Girshick, J. Donahue, T. Darrell and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE CVPR*, 2014. arXiv
9. R. Girshick, "Fast R-CNN," in *Proc. IEEE ICCV*, 2015. arXiv
10. S. Ren, K. He, R. Girshick and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE TPAMI*, 2017 (conference version 2015). doi
11. H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid and S. Savarese, "Generalized intersection over union: A metric and a loss for bounding box regression," in *Proc. IEEE/CVF CVPR*, 2019. arXiv
12. Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proc. AAAI*, 2020. arXiv
13. N. Bodla, B. Singh, R. Chellappa and L. S. Davis, "Soft-NMS — Improving object detection with one line of code," in *Proc. IEEE ICCV*, 2017. arXiv
14. T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE CVPR*, 2017. doi
15. T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE ICCV*, 2017. doi
16. K. He, G. Gkioxari, P. Dollár and R. Girshick, "Mask R-CNN," in *Proc. IEEE ICCV*, 2017. doi
17. Z. Cai and N. Vasconcelos, "Cascade R-CNN: Delving into high quality object detection," in *Proc. IEEE/CVF CVPR*, 2018. doi
18. J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016. doi
19. J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," arXiv:1612.08242, 2016. arXiv
20. J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," tech. report, arXiv:1804.02767, 2018. arXiv

21. A. Bochkovskiy, C.-Y. Wang and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," arXiv:2004.10934, 2020. arXiv
22. C.-Y. Wang, A. Bochkovskiy and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF CVPR*, 2023. doi
23. C.-Y. Wang, I-H. Yeh and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in *Proc. ECCV*, 2024. doi
24. A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin et al., "YOLOv10: Real-time end-to-end object detection," in *Proc. NeurIPS*, 2024. arXiv
25. Y. Tian, Q. Ye and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," arXiv:2502.12524, 2025. arXiv
26. W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu and A. C. Berg, "SSD: Single shot multibox detector," in *Proc. ECCV*, 2016. arXiv
27. H. Law and J. Deng, "CornerNet: Detecting objects as paired keypoints," in *Proc. ECCV*, 2018. doi
28. X. Zhou, D. Wang and P. Krähenbühl, "Objects as points," arXiv:1904.07850, 2019. arXiv
29. Z. Tian, C. Shen, H. Chen and T. He, "FCOS: Fully convolutional one-stage object detection," in *Proc. IEEE/CVF ICCV*, 2019. arXiv
30. N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. ECCV*, 2020. doi
31. H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, 1955. doi
32. X. Zhu, W. Su, L. Lu, B. Li, X. Wang and J. Dai, "Deformable DETR: Deformable transformers for end-to-end object detection," in *Proc. ICLR*, 2021. arXiv
33. H. Zhang, F. Li, S. Liu, L. Zhang, H. Su, J. Zhu, L. M. Ni and H.-Y. Shum, "DINO: DETR with improved denoising anchor boxes for end-to-end object detection," arXiv:2203.03605, 2022. arXiv
34. Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang et al., "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF CVPR*, 2024. doi
35. M. Minderer, A. Gritsenko, A. Stone, M. Neumann, D. Weissenborn et al., "Simple open-vocabulary object detection with vision transformers," in *Proc. ECCV*, 2022. arXiv
36. S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proc. ECCV*, 2024. doi
37. T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang et al., "YOLO-World: Real-time open-vocabulary object detection," in *Proc. IEEE/CVF CVPR*, 2024. doi
38. N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu et al., "SAM 3: Segment anything with concepts," arXiv:2511.16719, 2025. arXiv
39. G. Cheng, X. Yuan, X. Yao, K. Yan, Q. Zeng et al., "Towards large-scale small object detection: Survey and benchmarks," *IEEE TPAMI*, vol. 45, no. 11, 2023. arXiv
40. COCO Consortium, "COCO detection evaluation," official evaluation description (12 metrics, 101-point interpolation). link
41. M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn and A. Zisserman, "The PASCAL Visual Object Classes challenge: A retrospective," *International Journal of Computer Vision*, vol. 111, 2015. doi
42. S. Shao, Z. Li, T. Zhang, C. Peng et al., "Objects365: A large-scale, high-quality dataset for object detection," in *Proc. IEEE/CVF ICCV*, 2019. doi

43. A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin et al., “The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale,” *International Journal of Computer Vision*, 2020. arXiv
44. A. Gupta, P. Dollár and R. Girshick, “LVIS: A dataset for large vocabulary instance segmentation,” in *Proc. IEEE/CVF CVPR*, 2019. doi
45. T. Shehzadi, K. A. Hashmi, M. Liwicki, D. Stricker and M. Z. Afzal, “Object detection with transformers: A review,” *Sensors*, vol. 25, no. 19, 2025. doi
46. C. Zhu and L. Chen, “A survey on open-vocabulary detection and segmentation: Past, present, and future,” *IEEE TPAMI*, 2024. doi

DLCV 04 • Deep Image Restoration: Low-Level Vision

One degradation model, many tasks: how deep networks undo blur, noise, low resolution, haze, rain, snow and darkness, from SRCNN and DnCNN to Restormer, all-in-one models and diffusion priors.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-04-image-restoration/

What you'll learn

- What "low-level vision" means, and why super-resolution, denoising, deblurring, dehazing, deraining, desnowing and low-light enhancement are really **one problem**: invert a degradation $y = \mathcal{D}(x) + n$.
- How deep restoration is set up in practice: paired data, **synthetic degradation pipelines**, and the gap to real-world (blind) degradations. You will build a pipeline and train a tiny residual denoiser in PyTorch.
- The milestones and current families for each task, from SRCNN, DnCNN, DehazeNet and the GoPro deblurring benchmark to SwinIR, Restormer, NAFNet, MambaIR and diffusion-based SR (SUPIR, OSEDiff).
- How **all-in-one** models (AirNet, TransWeather, PromptIR, AdaIR, DA-CLIP, InstructIR) handle unknown or mixed degradations with one network.
- Which losses and metrics to use, why PSNR and "looks real" disagree (the **perception–distortion trade-off**), and how NTIRE challenges track the field.

The big picture

Low-level vision is about the pixels themselves. Before a system can recognize a face, read a sign or detect a car, the image has to be clear enough to look at. Restoration models are the part of the pipeline that takes a damaged photo (too small, noisy, blurred, foggy, rainy, dark) and returns a better estimate of the clean scene.

> **Explain like I'm five**
>
> Imagine someone hands you a photo that went through the washing machine: it is smudged, speckled and faded. You have seen thousands of clean photos in your life, so you can guess what this one *used to* look like: faces have two eyes, edges of buildings are straight, the sky is smooth. A restoration network does the same thing. It has looked at millions of "before and after" pairs, so it learned what clean pictures look like and how washing-machine damage changes them, and it uses both to undo the damage.

Why it matters: every phone camera runs denoising, demosaicking and super-resolution before you ever see a picture. Satellite, microscope, medical and surveillance images are restored before they are measured. Video streaming services upscale low-bitrate frames. Self-driving cars need their detectors to work in rain and fog. And from a research point of view, restoration is where many architectural ideas were tested first: residual learning, channel attention, efficient transformers and diffusion priors all have well-known restoration variants.

About this tutorial

This tutorial is expanded from the author's earlier post Low-Level Vision Task — Image Restoration 簡介 (written in Traditional Chinese) [1]. It keeps the post's order and viewpoints, folds in its 2026 updates and corrections, and adds the math, code and survey literature.

We assume you know CNNs and backpropagation (DLCV 01) and the classical restoration chapter (IP 05), which covers noise models, Wiener filtering and the degradation model in detail. We link to it rather than repeat it.

What is low-level vision?

Plain version. High-level vision asks "what is in the picture?" (classification, detection). Low-level vision asks "what does the picture actually look like, pixel by pixel?" One is about *understanding*, the other about *perceiving*.

The analogy in the author's post [1] comes from how people interact with the world: we first *sense* (light hits the retina and we see edges, colors and contrast) and then *interpret* (that is a cup). Both are necessary. If the sensing is poor (the cup is blurry), the interpretation suffers. Low-level vision tasks exist to make the sensing good again: when we say an object looks "blurry", our perception of it is not sharp, and a deblurring task is created to make it "clear".

Precise version. A low-level vision task maps an image to an image of (usually) the same content:

$$f_\theta : \mathbb{R}^{H\times W\times C} \to \mathbb{R}^{H'\times W'\times C'},$$

where the output is a pixel-wise estimate rather than a label. H, W, C are the input height, width and channels, and H', W', C' those of the output ($H' = sH$ for super-resolution by a factor s, otherwise usually equal). This has three consequences that shape the whole field:

1. **Dense, high-resolution outputs.** Every pixel matters, so architectures cannot downsample aggressively and must keep fine detail, which makes them expensive at high resolution.
2. **Supervision is cheap to synthesize.** Take a clean image, damage it with a known process, and you have an input–target pair. This is a huge advantage over labeled data for recognition, and also the source of the field's main weakness (synthetic damage is not real damage).

3. **"Correct" is ambiguous.** Many clean images are consistent with one damaged image. Which one is "best" depends on whether you value fidelity to the truth or natural appearance, as we will see in the evaluation section.

The family of tasks is broad: deblurring, super-resolution, denoising, inpainting, color calibration, infrared–visible fusion and more. Papers with Code used to be the standard place to browse these task lists and leaderboards, but it has shut down and its address now redirects to Hugging Face's trending-papers page, so its per-task leaderboards are no longer available. A reliable way to follow progress today is the yearly **NTIRE** challenge reports (New Trends in Image Restoration and Enhancement, a CVPR workshop), which we discuss near the end [2], [3].

One problem, many faces: the degradation model

Plain version. Every task in this tutorial can be written as "the camera saw a clean scene x, something damaged it, and we got y." Only the damage changes. If you can write the damage down, you can name the task.

The unified view

$$y = \mathcal{D}(x) + n,$$

where x is the unknown clean image, $\mathcal{D}$ is a (possibly nonlinear, possibly unknown) degradation operator, n is noise, and y is what we observe. Restoration means estimating x from y. Here are the specific forms used in the post [1] and in the rest of this tutorial. Figure 1 shows each one applied to the same photo.

Blur (the textbook model). As in IP 05,

$$g(u,v) = (h * f)(u,v) + n(u,v),$$

where f is the sharp image, h the point spread function (blur kernel), $*$ 2-D convolution, n additive noise and g the observed image, all indexed by pixel coordinates (u,v). Denoising is the special case $h = \delta$ (a unit impulse, so no blur).

Super-resolution. The low-resolution image is a blurred, subsampled and noisy version of the high-resolution one:

$$y = (x * k)\downarrow_s + n,$$

where k is a blur (anti-aliasing) kernel, $\downarrow_s$ keeps every s-th pixel in each direction, and s is the scale factor ($2, 3, 4, \ldots$). The classic "bicubic" benchmark setting replaces all of this with MATLAB-style bicubic downsampling and no noise.

Compression. $y = \mathrm{JPEG}_q(x)$, where q is the quality factor. JPEG quantizes 8×8 DCT coefficients (see IP 08), producing blocking and ringing. It is not additive and not linear, so it is usually just added to the pipeline as a black box.

Haze (the atmospheric scattering model).

$$I(p) = J(p)\,t(p) + A\,(1 - t(p)), \qquad t(p) = e^{-\beta d(p)},$$

where p is a pixel, I the hazy image, J the haze-free scene radiance, A the global atmospheric light (the color of the haze), $t \in (0, 1]$ the transmission, β the scattering coefficient and d the scene depth. The first term is the attenuated scene; the second is "airlight" scattered toward the camera. Far objects (large d) get small t and fade into A.

Rain. For rain streaks the common model is additive,

$$I = B + R,$$

where B is the rain-free background and R the rain layer. Raindrops stuck on a lens or windshield behave differently: they block and refract the background, so a popular model is $I = (1 - M) \odot B + R$, where M is a binary raindrop mask, $\odot$ the element-wise product and R the light passing through the drops [39].

Low light (Retinex). Retinex theory [5] separates an image into reflectance and illumination:

$$S = R \cdot L,$$

where S is the observed image, R the reflectance (the intrinsic color and texture of surfaces, what we want), L the illumination and $\cdot$ the element-wise product. A dark photo has small L; enhancement estimates L and brightens it, while noise that was hidden in the dark gets amplified.



Figure 1 — One clean image x pushed through the degradation models of this section. Each restoration task tries to undo one (or several) of these.

Although none of these models is exact, they all have the same shape: a clean image goes in, an operator plus noise comes out. That is the post's central point [1]: seen through the degradation model, these are all **the same kind of problem**, and the same tools (and often the same networks) apply to all of them. Figure 2 organizes them by where the damage comes from.

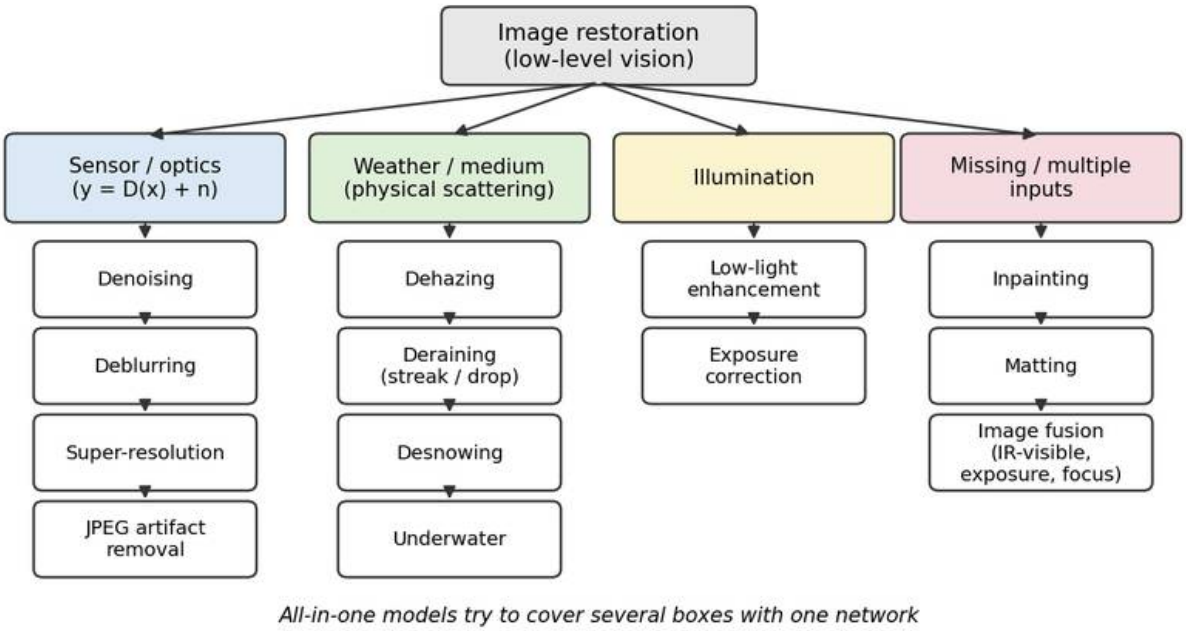


Figure 2 — A task taxonomy of low-level vision by the source of degradation. All-in-one models try to cover several boxes with a single network.

Why the problem is hard: ill-posedness

For almost every $\mathcal{D}$ above, many different x produce the same y. Downsampling by 4 throws away 15 of every 16 pixels; blur suppresses high frequencies to near zero; haze with $t \approx 0$ erases distant objects. The inverse problem is **ill-posed**, and we need a **prior** on what clean images look like. In the Bayesian (MAP) form,

$$\hat{x} = \arg\min_x \ \tfrac{1}{2}\|y - \mathcal{D}(x)\|_2^2 + \lambda\, \Phi(x),$$

where the first term (data fidelity) says "$\hat{x}$ must explain the observation", Φ is a regularizer encoding the prior (small Φ for natural-looking images), and $\lambda > 0$ balances the two. The whole history of the field is a history of better priors Φ.

Before deep learning: the era of hand-crafted priors

Before deep learning, each task had its own carefully reasoned prior, and success depended on having enough physical or statistical knowledge of the problem. Denoising used local smoothness, non-local self-similarity and sparsity; deblurring used Wiener and constrained least-squares filters plus heavy-tailed gradient priors; dehazing used the dark channel prior [25]; super-resolution used interpolation and example-based dictionaries; low-light enhancement

used histogram equalization and Retinex decompositions. IP 05 covers the core of this toolbox (noise models, mean/median/adaptive filters, inverse, Wiener and CLS filtering) and ends with a short history of the move to learned denoisers. The author's view in [1] is that once deep learning surpassed what these models could do, the decisive factor stopped being a strong theoretical prior for each task and became a strong **learning mechanism and feature extractor**.

The degradation model did not disappear: it moved from the solver into the **data generator**.

The deep learning formulation

Plain version. Make lots of (damaged, clean) pairs, and train a network to map damaged to clean. Most of the hard work is in making the damage realistic.

Supervised training on pairs

Given a dataset of clean images $\{x_i\}$ and a degradation sampler, we build pairs (y_i, x_i) with $y_i = \mathcal{D}_i(x_i) + n_i$, and fit

$$\theta^\star = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(y_i),\, x_i),$$

where f_θ is the restoration network with parameters θ, N the number of training pairs and ℓ a loss (L1, L2, perceptual, adversarial, ...; see the losses section). Training is done on random crops ("patches", e.g. 64×64 to 256×256) with flips and rotations, because convolutional and windowed-attention networks are translation-equivariant and full-resolution images do not fit in GPU memory.

A useful fact explains a lot of what follows: with the L2 loss, the optimal network outputs the **posterior mean** $f^\star(y) = \mathbb{E}[x \mid y]$. When many sharp images are consistent with y, their average is blurry. This is why MSE-trained super-resolution looks smooth, and why GANs and diffusion models were brought in.

Synthetic degradation pipelines

Pairs come from three sources. (1) **Synthetic**: degrade clean images with a known model (bicubic downsampling, Gaussian noise, Gaussian blur, JPEG). Cheap and unlimited, but limited by how realistic the model is. (2) **Captured**: photograph the same scene twice, e.g. a short and a long exposure, or a burst averaged into a clean reference (as in the SIDD real-noise dataset [22]), or averaging high-speed video frames to create real motion blur (GoPro [44]). Realistic but expensive

and specific to one camera. (3) **Unpaired / self-supervised**: learn from degraded images alone, using GANs, cycle consistency or noise statistics. Most strong models today train on synthetic pairs from a rich degradation pipeline and evaluate on both synthetic and captured data.

A typical "classic" pipeline is blur → downsample → noise → JPEG, mirroring the order in which a camera and an image-sharing site would damage a photo:

```
import numpy as np, cv2, torch, torch.nn.functional as F
from skimage import data, img_as_float32

def gaussian_kernel(sigma, size=21):
    ax = torch.arange(size) - size // 2
    g = torch.exp(-ax**2 / (2 * sigma**2))
    k = torch.outer(g, g)
    return k / k.sum()

def degrade(x, scale=4, sigma_blur=1.6, sigma_noise=0.03, jpeg_q=40, rng=None):
    """x: float tensor (C,H,W) in [0,1]. Returns the low-quality image y = D(x) + n."""
    rng = rng or np.random.default_rng(0)
    C = x.shape[0]
    k = gaussian_kernel(sigma_blur).to(x)[None, None].repeat(C, 1, 1, 1)
    y = F.conv2d(F.pad(x[None], (10,) * 4, mode="reflect"), k, groups=C)   # 1) blur  h * x
    y = y[..., ::scale, ::scale]                                           # 2) downsample
     ↪  (.)↓s
    y = y + sigma_noise * torch.from_numpy(rng.standard_normal(y.shape).astype(np.float32))
     ↪  # 3) noise n
    y = y.clamp(0, 1)[0]
    img = (y.permute(1, 2, 0).numpy() * 255).round().astype(np.uint8)      # 4) JPEG round
     ↪  trip
    ok, buf = cv2.imencode(".jpg", img[..., ::-1], [cv2.IMWRITE_JPEG_QUALITY, jpeg_q])
    img = cv2.imdecode(buf, cv2.IMREAD_COLOR)[..., ::-1]
    return torch.from_numpy(img.copy()).permute(2, 0, 1).float() / 255

x = torch.from_numpy(img_as_float32(data.astronaut())).permute(2, 0, 1)    # (3, 512, 512)
y = degrade(x)
print(x.shape, "->", y.shape)        # torch.Size([3, 512, 512]) -> torch.Size([3, 128, 128])
```

In training, every parameter (kernel width and shape, noise level and type, JPEG quality, even the order) is **sampled at random per image**, so the network sees a distribution of degradations instead of a single one.

Real-world and blind degradations

A model trained on one fixed $\mathcal{D}$ (say, bicubic ×4) breaks on real photos, whose degradation is unknown, spatially varying and a mix of several effects. This is the **blind** (or real-world) restoration problem. Two broad strategies exist:

- **Make the synthetic distribution wide enough to cover reality.** Real-ESRGAN [4] applies the classic pipeline *twice* in sequence ("high-order" degradation, with randomized blur kernels, resizing, noise and JPEG in each round) and adds sinc filters to simulate ringing and overshoot. Trained on purely synthetic data, it generalizes much better to real low-quality images than bicubic-trained models.
- **Bring in a strong generative prior.** Pretrained text-to-image diffusion models already know what natural images look like. DiffBIR [16], SUPIR [17] and OSEDiff [18] condition such a model on the degraded input, so that the prior "fills in" plausible detail. This is the main recent trend in real-world SR and is discussed below.

A third, complementary strategy is to **estimate the degradation** (noise level, kernel, degradation type) and feed it to the network, as FFDNet's noise-level map [21] and the all-in-one models do. For blind super-resolution, where the unknown is mostly the blur kernel k in $y = (x * k)\downarrow_s + n$, this line of work developed its own family of methods:

- **Condition the network on the kernel.** SRMD [87] gives a single SR network the degradation as extra input: the blur kernel, reduced to a low-dimensional vector, and the noise level are stretched into maps of the same size as the low-resolution image and concatenated with it. One network then handles many kernels and noise levels, provided someone supplies them.

  USRNet [91] goes further with **deep unfolding**: it unrolls a half-quadratic-splitting MAP solver into a network whose data module uses the given kernel, scale and noise level in closed form and whose prior module is a learned denoiser, so one model serves arbitrary known kernels.

- **Estimate the kernel, then correct it.** IKC [88] observes that a wrong kernel causes characteristic artifacts (too large gives over-sharpening, too small gives over-smoothing), and uses them to iteratively correct an initial kernel estimate while a kernel-conditioned SR network (SFTMD) restores the image. DAN [90] unfolds the alternation into one end-to-end network: an *estimator* predicts the kernel from both the low-resolution input and the current SR result, a *restorer* super-resolves with the current kernel, and the two alternate, the same idea as the classical deblurring loop above.

- **Estimate the kernel from the image alone.** KernelGAN [89] trains a small GAN on the single test image at test time: its generator learns to downscale the image so that the discriminator cannot tell the patches of the downscaled image from those of the original. Because patches recur across scales in natural images, the learned downscaler is an estimate of the image's own SR kernel, which is then given to a non-blind SR method.

Explicit-kernel methods are interpretable and accurate when the degradation really is "blur, downsample, noise" with a roughly uniform kernel. They are fragile when it is not (JPEG, camera processing, spatially varying blur), and errors in the estimated kernel propagate into the result. That motivated the **implicit** route: BSRGAN [92] trains on a practical degradation model with randomly shuffled blur (isotropic and anisotropic Gaussian kernels), downsampling, noise

and JPEG steps, and Real-ESRGAN [4] uses its high-order pipeline; neither estimates a kernel at test time. Diffusion-prior methods [16]–[18] are implicit as well. The survey by Liu et al. [93] organizes blind SR methods by how they model the degradation and which data (external datasets or the single test image) they use, and compares them on synthetic and real images.

Super-resolution

Plain version. Make a small image bigger *and* sharper, i.e. predict the missing high-resolution pixels rather than just stretching the existing ones.

Problem setup

Single-image super-resolution (SISR) estimates $x \in \mathbb{R}^{sH \times sW \times C}$ from $y \in \mathbb{R}^{H \times W \times C}$ under $y = (x * k) \downarrow_s + n$. The traditional answer is **interpolation**: nearest-neighbor, bilinear and bicubic. Bicubic, which fits a cubic polynomial through a 4×4 neighborhood, is the best known and is still used everywhere, both as a baseline and as the standard way to create benchmark low-resolution inputs. Interpolation only redistributes existing information; it cannot recreate the high frequencies that $\downarrow_s$ removed.

How a network gets bigger: three upsampling strategies

Because the output is larger than the input, every SR network must decide *where* the resolution increases.

a. Pre-upsampling (interpolate first). SRCNN [6] bicubically enlarges the input to the target size, then applies three convolution layers (patch extraction, non-linear mapping, reconstruction). Simple and intuitive, but every layer runs at high resolution, which is slow, and the network must clean up interpolation artifacts.

b. Transposed convolution (deconvolution). FSRCNN [7] works at low resolution and enlarges only at the end with a learned transposed convolution, giving a large speed-up over SRCNN. Transposed convolutions with a stride that does not divide the kernel size overlap unevenly and tend to produce **checkerboard artifacts** [9], so they are now rare as the final SR upsampler, although they remain common elsewhere in neural networks (for example in decoders of segmentation and generative models).

c. Sub-pixel convolution (pixel shuffle). ESPCN [8] computes Cs^2 channels at low resolution and rearranges them into an image s times larger:

$$\mathrm{PS}(T)_{c,\,sh+i,\,sw+j} = T_{cs^2+si+j,\,h,\,w}, \qquad 0 \le i, j < s,$$

where T is the low-resolution feature tensor with Cs^2 channels, (h, w) a low-resolution position, (i, j) the sub-pixel offset and c the output channel. All learning happens at low resolution, and the rearrangement itself has no parameters. It is the dominant choice in modern SR networks (`torch.nn.PixelShuffle`).

Milestones

- **SRCNN** (2014/2016) [6]: the first end-to-end CNN for SR, three layers on a bicubic-upsampled input.
- **FSRCNN** (2016) [7] and **ESPCN** (2016) [8]: post-upsampling for real-time speed.
- **EDSR** (2017) [10]: a deep residual network with batch normalization removed (BN hurt the range flexibility needed for restoration), residual scaling for stable training, and a multi-scale variant. It won the NTIRE 2017 SR challenge.
- **RCAN** (2018) [11]: very deep "residual-in-residual" structure with **channel attention**, which re-weights feature channels using global statistics, so the network can focus on high-frequency information.
- **SRGAN** (2017) [12] and **ESRGAN** (2018) [13]: adversarial and perceptual (VGG feature) losses produce sharp, realistic textures instead of the blurry MSE average. ESRGAN improved the generator (Residual-in-Residual Dense Blocks without BN), used a relativistic discriminator and computed the perceptual loss on features before activation.

By around 2018 these models already produced results that looked much better than interpolation to human observers, and the field started to question how to *judge* SR at all (see the perception–distortion section).

Transformers, real-world SR and diffusion (current families)

- **Transformers.** SwinIR [14] (2021) uses residual Swin Transformer blocks (self-attention inside shifted local windows) for SR, denoising and JPEG artifact removal. HAT [15] (CVPR 2023) combines channel attention with window self-attention and an overlapping cross-attention module to "activate more pixels" for each output. Both are standard baselines.
- **State-space models.** MambaIR [49] and MambaIRv2 [50] replace attention with selective state-space (Mamba) scans that give a global receptive field at linear cost; see the backbones section.
- **Real-world (blind) SR.** Real-ESRGAN [4] and its high-order degradation pipeline, described above.
- **Diffusion priors.** DiffBIR [16] removes degradations first and then regenerates detail with a pretrained latent diffusion model; SUPIR [17] scales this up with a large diffusion backbone, a 20-million-image training set with text descriptions and negative-quality prompts; OSEDiff

[18] distills the process into a **single** diffusion step that starts from the low-quality image itself, using variational score distillation to keep outputs realistic. These methods produce far more convincing textures on real photos, but they can also invent detail that was not in the scene.

Datasets and metrics

The standard training set is **DIV2K** [19] (2K-resolution images introduced for NTIRE 2017). Evaluation uses small classic test sets of natural, urban and comic images, reporting PSNR and SSIM on the luminance (Y) channel, usually after cropping s border pixels. Real-world SR is evaluated with perceptual and no-reference metrics (LPIPS, NIQE and others; see Evaluation).

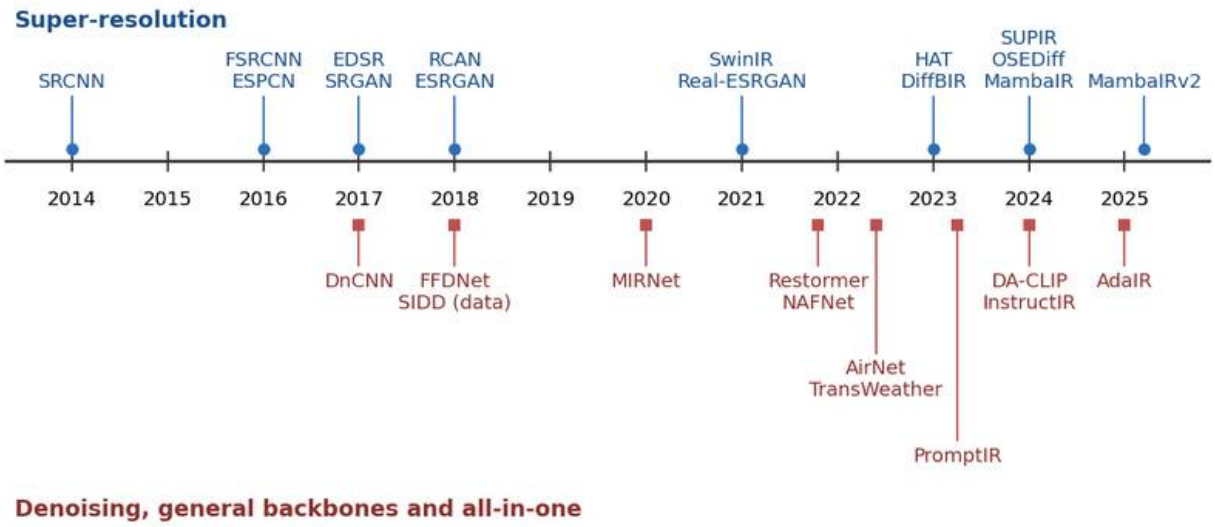


Figure 3 — Milestones in super-resolution (top) and in denoising, general backbones and all-in-one restoration (bottom), placed by publication year (positions within a year are only for legibility).

Denoising

Plain version. Remove the random grain from a photo without wiping out the real texture. Noise is the simplest degradation, so denoising became the testbed where most deep restoration ideas appeared first.

Problem setup

$y = x + n$. In the synthetic setting $n \sim \mathcal{N}(0, \sigma^2 I)$ (additive white Gaussian noise, AWGN, with standard deviation σ, usually quoted on a 0–255 scale such as $\sigma = 25$). Real camera noise is

different: it is signal-dependent (photon shot noise grows with brightness), spatially correlated after demosaicking, and reshaped by the camera's processing pipeline.

Milestones

DnCNN [20] made two choices that became standard. It uses **residual learning**: instead of predicting x, the network $\mathcal{R}_\theta$ predicts the noise, and the output is

$$\hat{x} = y - \mathcal{R}_\theta(y),$$

where $\mathcal{R}_\theta(y) \approx n$. The residual (noise) is a "simpler" function to learn than the clean image, which made deep plain CNNs with batch normalization train well. A single DnCNN can also be trained for a range of noise levels (blind Gaussian denoising), and the same network handles SR and JPEG deblocking.

FFDNet [21] feeds a **noise-level map** M (same size as the image, value σ at each pixel) to the network along with a downsampled version of the input. Changing M at test time trades noise removal against detail preservation, and a non-uniform M handles spatially varying noise. It is fast enough to run on a CPU.

Real noise. The **SIDD** dataset [22] provides real smartphone noisy images with clean references obtained by careful capture and processing of image bursts. Models trained only on AWGN do poorly on it, which pushed the field toward realistic noise synthesis and captured training data.

General backbones. MIRNet [30], **Restormer** [23] and **NAFNet** [24] are general restoration architectures that reported state-of-the-art results on SIDD as well as on deblurring and other tasks. They are covered in the backbones section.

Code: a tiny residual denoiser

The snippet trains a five-layer DnCNN-style network on random 32×32 patches cut from four `skimage` images, with fresh Gaussian noise every step, then tests on an image it has never seen. On a laptop CPU it takes well under a minute (we measured about one minute on a heavily loaded machine).

```
import time, numpy as np, torch, torch.nn as nn
from skimage import data, img_as_float32
from skimage.color import rgb2gray

torch.manual_seed(0); torch.set_num_threads(2)
rng = np.random.default_rng(0)
sigma = 25 / 255                                    # noise level (on a [0,1] scale)
train_imgs = [img_as_float32(im) for im in (data.camera(), data.coins(), data.moon(),
 ↪  data.page())]
test_img = img_as_float32(rgb2gray(data.astronaut()))   # held out
```

```
def random_patches(n=32, p=32):
    batch = []
    for _ in range(n):
        im = train_imgs[rng.integers(len(train_imgs))]
        i, j = rng.integers(0, im.shape[0] - p), rng.integers(0, im.shape[1] - p)
        batch.append(im[i:i + p, j:j + p])
    return torch.from_numpy(np.stack(batch))[:, None]   # (n, 1, p, p)

class TinyDnCNN(nn.Module):
    """conv-ReLU, (depth-2) x conv-BN-ReLU, conv. Predicts the NOISE, not the image."""
    def __init__(self, depth=5, ch=32):
        super().__init__()
        layers = [nn.Conv2d(1, ch, 3, padding=1), nn.ReLU(inplace=True)]
        for _ in range(depth - 2):
            layers += [nn.Conv2d(ch, ch, 3, padding=1), nn.BatchNorm2d(ch),
                       nn.ReLU(inplace=True)]
        layers += [nn.Conv2d(ch, 1, 3, padding=1)]
        self.body = nn.Sequential(*layers)
    def forward(self, y):
        return y - self.body(y)                         # x_hat = y - R(y): residual learning

net = TinyDnCNN()
opt = torch.optim.Adam(net.parameters(), lr=2e-3)
t0 = time.time()
for step in range(300):
    x = random_patches()
    y = x + sigma * torch.randn_like(x)                 # synthetic pairs, made on the fly
    loss = nn.functional.l1_loss(net(y), x)
    opt.zero_grad(); loss.backward(); opt.step()
print(f"trained in {time.time() - t0:.0f} s, last L1 = {loss.item():.4f}")

net.eval()
with torch.no_grad():
    x = torch.from_numpy(test_img)[None, None]
    y = x + sigma * torch.randn_like(x)
    x_hat = net(y).clamp(0, 1)
mse = lambda a, b: torch.mean((a - b) ** 2).item()
print(f"PSNR noisy {10*np.log10(1/mse(y.clamp(0,1), x)):.2f} dB -> denoised
 ↪  {10*np.log10(1/mse(x_hat, x)):.2f} dB")
# e.g. PSNR noisy 20.74 dB -> denoised 28.64 dB
```

Even this toy model, trained for 300 small steps on four images, gains about 8 dB on an unseen photo and beats a hand-tuned Gaussian filter (Figure 4). Drag the slider to compare the noisy input with the tiny network's output.



Noisy ($\sigma = 25$)



Tiny residual CNN

Slider comparing a grainy grayscale astronaut face with the same crop after denoising by a small CNN, which is clean but slightly smoothed.

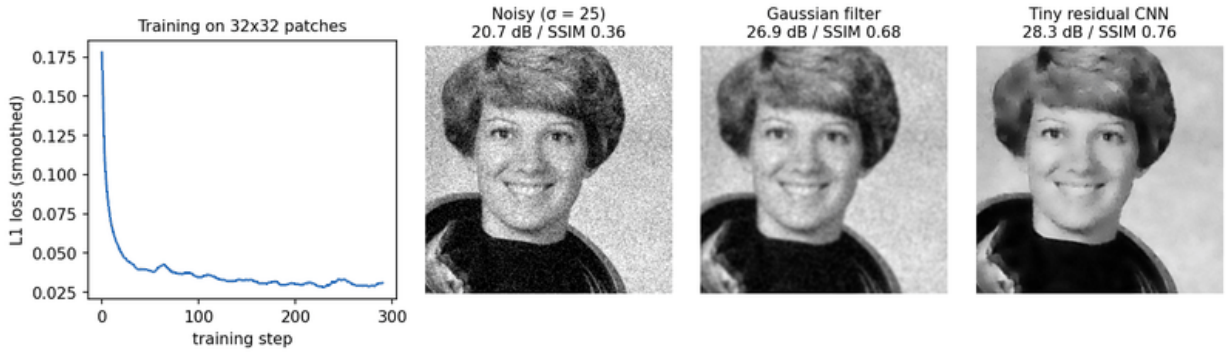


Figure 4 — The tiny residual denoiser. PSNR and SSIM are computed on the full 512 × 512 test image. The CNN removes more noise than a Gaussian filter while keeping edges sharper, but it also flattens fine texture, a typical MSE/L1 behavior. Values vary slightly between runs and machines.

Dehazing

Plain version. Remove the gray-white veil of fog or smog, restoring contrast and color, especially for distant objects. In the post's words it is an extension of denoising and deblurring, with a physical model attached.

The classical recipe and the dark channel prior

Classical methods estimate the unknowns of $I = Jt + A(1 - t)$ (the transmission map t and the airlight A), then invert the model. The hard part is finding a reliable relationship that pins those unknowns down. The most famous one is He et al.'s **dark channel prior** [25]. In haze-free outdoor images, most local patches contain some pixel that is very dark in at least one color channel (shadows, dark or colorful objects):

$$J^{\text{dark}}(p) = \min_{c \in \{r,g,b\}} \Big(\min_{q \in \Omega(p)} J^c(q) \Big) \approx 0,$$

where $\Omega(p)$ is a small patch around p and J^c is color channel c. Applying the same min–min operation to the haze model and using $J^{\text{dark}} \approx 0$ gives a transmission estimate

$$\tilde{t}(p) = 1 - \omega \min_c \Big(\min_{q \in \Omega(p)} \frac{I^c(q)}{A^c} \Big),$$

where $\omega < 1$ (0.95 in the paper) keeps a little haze for depth perception. Then $\hat{J} = (I - A)/\max(\tilde{t}, t_0) + A$, with a small t_0 to avoid dividing by zero. The prior fails on large bright regions similar

to the airlight (white walls, sky), and the coarse patch-based t must be refined to follow edges.

Deep dehazing: two schools

The post [1] divides deep dehazing into two schools, and the split still describes the field well.

1. Physics-guided ("apply the formula"). Learn the unknown parameters, then invert the model. **DehazeNet** [26] uses a CNN (with Maxout units and a bounded ReLU) to regress the transmission t from hazy patches. **AOD-Net** [27] simplifies the scattering model further by folding t and A into a single function K:

$$J(p) = K(p)\, I(p) - K(p) + b, \qquad K(p) = \frac{\frac{1}{t(p)}(I(p) - A) + (A - b)}{I(p) - 1},$$

where b is a constant bias. A light-weight CNN predicts K, so the whole network is end-to-end, and it can be attached in front of a detector and trained jointly.

2. Strong general architectures ("big learner"). Skip the physics and learn $I \mapsto J$ directly with a powerful network. **GridDehazeNet** [28] uses an attention-based multi-scale grid; **FFA-Net** [29] stacks channel- and pixel-attention blocks with adaptive feature fusion. These designs transfer well to other degradations, which is exactly the route taken by general restoration networks such as MIRNet [30]. Transformer dehazers followed: **DehazeFormer** [31] adapts the Swin Transformer to dehazing with a modified normalization layer, activation and spatial aggregation.

Datasets

- **RESIDE** [32]: a large benchmark with synthetic indoor and outdoor hazy images (generated from depth with the scattering model) plus real hazy images for task-driven evaluation. Its SOTS test set is the most quoted dehazing benchmark.
- **O-HAZE** [33] and **Dense-Haze** [34]: real hazy/haze-free pairs captured with professional haze machines, outdoor and with dense haze respectively. They are small but expose the synthetic-to-real gap.
- **KITTI** [35] is *not* a dehazing dataset. It is an autonomous-driving benchmark (stereo, optical flow, odometry, 3-D detection, tracking) recorded around Karlsruhe. Some dehazing papers synthesize fog on its images (it has depth from LiDAR and stereo) or use it to measure how dehazing affects downstream detection.

Desnowing

Plain version. Remove falling snow. It looks like a cousin of deraining, but it also behaves like dehazing.

Snow is harder than it looks. Snowflakes vary hugely in size and transparency, from opaque white blobs to semi-transparent streaks, and heavy snow also creates a **veiling effect** much like haze. So a desnowing model has to handle both the flakes and the fog-like veil. **DesnowNet** [36] introduced the **Snow100K** dataset (synthetic snow over real images, plus real snowy images) and a network that estimates both the translucency of snow and the residual it leaves. **JSTASR** [37] explicitly models snow size and transparency and adds a veiling-effect removal step. Desnowing is now usually studied inside "adverse weather" all-in-one models such as TransWeather [52].

Deraining

Plain version. Remove rain from photos. "Rain" means two very different things: thin streaks falling through the air, and drops sitting on the lens or windshield.

Rain streaks versus raindrops

- **Rain streaks** are thin, bright, oriented lines overlaid on the scene, modeled as $I = B + R$. Distant streaks accumulate into a haze-like veil. Methods learn to separate the two layers. An early influential deep method jointly detects and removes rain [38] and comes with widely used synthetic datasets of light and heavy rain.
- **Raindrops** on a glass surface act as tiny lenses: they hide and refract the background, are often out of focus, and have no consistent orientation. The attentive GAN of Qian et al. [39] uses an attention map (where are the drops?) to guide both the generator and the discriminator, and introduced a paired raindrop dataset captured through glass with and without water drops.

Toward real scenes

Most deraining datasets are synthetic, and models trained on synthetic streaks often fail on real rain. The **NTIRE 2025 Day and Night Raindrop Removal for Dual-Focused Images** challenge [40] reflects the shift to real data: it uses the real-captured **Raindrop Clarity** dataset, covering daytime and nighttime scenes and both raindrop-focused and background-focused shots (14,139 training, 240 validation and 731 test images). The focus setting matters: when the camera focuses on the drops, the background is blurred too, so raindrop removal becomes partly deblurring.

Low-light enhancement (brief)

Plain version. Make a dark photo look properly exposed, with visible details and natural colors, without blowing up the noise.

Training compares dark images with normally exposed references of the same scene, for example the **LOL** dataset of paired low/normal-light images introduced with **RetinexNet** [41]. Classical methods rely on contrast adjustment (histogram equalization) and Retinex-based models (and, less often, an inverted atmospheric scattering model). Deep methods follow two lines: Retinex-inspired networks that decompose $S = R \cdot L$ and adjust L (RetinexNet [41]), and general well-designed networks. **Retinexformer** [42] (ICCV 2023), which its authors describe as the first Transformer-based low-light enhancement method, combines both: a one-stage Retinex formulation with illumination-guided attention. Low-light enhancement has enough of its own (zero-reference methods, curves, noise, color, video) to deserve a separate chapter: see DLCV 05.

Image fusion (brief)

Plain version. Combine several images of the same scene, each good at something different, into one image that has the best of each.

Three common settings appear in the post [1]: **infrared and visible** fusion (thermal targets from IR, textures and color from the visible camera), **multi-exposure** fusion (detail from under- and over-exposed shots, an HDR-like result) and **multi-focus** fusion (an all-in-focus image from shots focused at different depths). Unlike the other tasks, there is usually no ground-truth fused image, so deep fusion relies on unsupervised losses that keep gradients and intensities from the sources, or on GANs. The survey by Zhang et al. [43] organizes deep fusion methods by these scenarios and by learning strategy.

Deblurring

Plain version. Recover a sharp image from one smeared by camera shake, object motion or defocus.

Problem setup

Deblurring is the direct instance of $g = h * f + n$. If the kernel h is known it is **non-blind** deblurring (the setting of the classical Wiener and CLS filters in DIP 5, and of learned non-blind solvers); if h

is unknown it is **blind** deblurring. Real motion blur in dynamic scenes is worse: different objects move differently and depth varies, so there is no single global h.

Blind deblurring: estimating the kernel

Plain version. If you knew exactly how the camera shook, undoing the blur would be a well-studied filtering problem. In blind deblurring you do not know the shake, so you must guess the shake *and* the sharp photo together. The trick is to guess them in turns: first sharpen with your current guess of the shake, then use the sharpest edges you got to improve the guess of the shake, and repeat.

The joint problem. For a spatially uniform blur, blind deconvolution solves

$$(\hat{f},\hat{h}) = \arg\min_{f,h} \|h * f - g\|_2^2 + \lambda_f\, \phi(f) + \lambda_h\, \psi(h) \quad \text{subject to} \quad h \geq 0,\ \ \textstyle\sum_{u,v} h(u,v) = 1,$$

where g is the blurred image, f the unknown sharp image, h the unknown kernel (supported on a small window, e.g. 15×15 to 75×75 pixels for camera shake), ϕ an image prior (typically a sparse gradient penalty $\sum_p |\nabla f(p)|^\alpha$ with $0 < \alpha \leq 1$, because natural images have mostly flat regions and a few strong edges), ψ a kernel prior (e.g. $\|h\|_2^2$ or $\|h\|_1$, favoring compact, sparse motion paths), and $\lambda_f, \lambda_h > 0$ their weights. The constraints say a blur kernel only redistributes light: it is non-negative and preserves total brightness. They also remove one ambiguity, because $(c\,h, f/c)$ explains g as well as (h, f) for any $c > 0$. A second ambiguity cannot be removed: shifting f one way and h the other leaves $h*f$ unchanged, so the kernel is only recovered up to a translation.

Why it is badly ill-posed. Far more unknowns (f and h) than equations, and many pairs explain g exactly. Worse, the most obvious pair is wrong: $h = \delta$ (no blur) and $f = g$ fits the data perfectly. Levin et al. [74] showed that with the usual sparse gradient priors, the blurry image often has a *lower* prior cost than the sharp one, because blur shrinks gradients and ϕ rewards small gradients. So naive joint MAP estimation over (f, h) tends to return the no-blur solution. Their analysis points to a better-posed target: estimate only the kernel, integrating out the image,

$$\hat{h} = \arg\max_h\ p(h \mid g) = \arg\max_h\ p(h) \int p(g \mid f, h)\, p(f)\, df,$$

where $p(g \mid f, h)$ is the Gaussian likelihood given by the data term, $p(f)$ and $p(h)$ the priors behind ϕ and ψ, and the integral averages over all sharp images. A kernel has few unknowns compared with the many pixel observations, so its posterior is much better determined than that of the full image. Once $\hat{h}$ is known, a non-blind deconvolution recovers f.

Classical remedies. The methods that worked in practice either approximate that marginalization or change the prior so the sharp solution wins:

- **Variational Bayes.** Fergus et al. [75] fit a heavy-tailed mixture-of-Gaussians model of image

gradients and approximate the kernel posterior with variational Bayes, estimating the kernel coarse-to-fine; the image is then recovered by a standard non-blind method.

- **Edge prediction and selection.** Shan et al. [76] alternate kernel and image estimation with a natural-image gradient prior and a careful noise model to suppress ringing. Cho and Lee [77] made the alternation fast: they *predict* sharp edges from the current latent image with bilateral and shock filtering, and estimate the kernel in the gradient domain with FFTs. Xu and Jia [78] observed that thin edges narrower than the kernel can mislead estimation, so they select only informative edges in a first phase and refine the kernel's support in a second phase.
- **Priors that prefer sharp images.** Krishnan et al. [79] replace $\|\nabla f\|_1$ by the normalized sparsity $\|\nabla f\|_1 / \|\nabla f\|_2$, which is scale-invariant and *increases* with blur, so it no longer favors the blurry solution. Pan et al. [80] use the dark channel (the same quantity as in He et al.'s dehazing prior [25]): blur averages dark pixels with bright neighbors, so the dark channel becomes less sparse, and an L_0 penalty on it favors sharp images.

The standard recipe. Almost all classical blind deblurring methods follow the same loop. Starting from a delta kernel at the coarsest level of an image pyramid, alternate

$$f^{(t+1)} = \arg\min_{f} \|h^{(t)} * f - g\|_2^2 + \lambda_f \, \phi(f),$$

$$h^{(t+1)} = \arg\min_{h \geq 0,\ \sum h = 1} \|h * \nabla \tilde{f}^{(t+1)} - \nabla g\|_2^2 + \lambda_h \|h\|_2^2,$$

where t is the iteration, ∇ the image gradient, and $\tilde{f}$ the latent image after edge prediction or selection (sharpened, with weak gradients set to zero). The kernel step works in the gradient domain because convolution commutes with differentiation, $\nabla(h * f) = h * \nabla f$, and gradients are where the blur information sits; with a quadratic penalty it has a closed form in the Fourier domain, followed by projection onto the constraints. After a few iterations the kernel is upsampled to the next pyramid level (**coarse-to-fine**), which avoids poor local minima because a large blur is only a few pixels wide at coarse scales. Finally, with $\hat{h}$ fixed, a **non-blind deconvolution** produces the output image: Wiener or constrained least squares filtering as in IP 05, or a hyper-Laplacian or learned non-blind solver.

Deep learning and the kernel. Networks entered blind deblurring in three ways.

- **Predict the kernel (or a filter), then deconvolve.** Sun et al. [81] trained a CNN to classify the local motion kernel of each patch among a set of linear motions, smoothed the resulting non-uniform motion field with a Markov random field, then deconvolved. Schuler et al. [82] built a deep network with stages that mimic the classical loop (feature extraction, kernel estimation, image estimation) and trained it end-to-end on synthetic blur. Chakrabarti [83] trained a network to predict, for each patch, the Fourier coefficients of a deconvolution filter, used the result to estimate one global kernel, and finished with standard non-blind deconvolution.

- **Self-supervised, per-image estimation.** Deep Image Prior [84] showed that the structure of an untrained generator network is itself a useful image prior. SelfDeblur [85] uses two generators, an encoder–decoder for the image and a fully connected network with a softmax output for the kernel (the softmax enforces $h \geq 0$ and $\sum h = 1$), and fits both to the single blurred image. No training data is needed, but each image takes an optimization run.
- **Kernel-free, end-to-end.** The multi-scale network of Nah et al. [44], DeblurGAN [86] (a conditional GAN with a content loss) and the general backbones Restormer and NAFNet map blurred to sharp images directly, never forming h.

Which is preferable? Kernel-based methods fit **spatially uniform** blur (camera shake of a distant, static scene), give an **interpretable** intermediate result that can be checked and reused (for example to deblur other frames from the same exposure), and generalize to new kernels as long as the physics holds; they are slower and break when the blur varies across the image or the image has few edges. Kernel-free networks handle **non-uniform** blur from object motion and depth, are fast at test time, and are the current state of the art on dynamic-scene benchmarks such as GoPro, but they generalize only as far as their training blur distribution and give no explicit kernel. The survey of Zhang et al. [69] summarizes this shift.

Code: a toy alternating blind deconvolution. The snippet blurs a 256×256 `camera` image with a curved 15×15 camera-shake kernel, then alternates a closed-form image step (a CLS-like filter with a quadratic gradient penalty) and a kernel step (least squares on the strongest 2% of shock-filtered gradients, projected onto $h \geq 0$, $\sum h = 1$). It runs in about a second on a CPU.

```
import numpy as np
from numpy.fft import fft2, ifft2
from scipy.ndimage import convolve, gaussian_filter, laplace
from skimage import data, img_as_float, transform

rng = np.random.default_rng(0)
f_true = transform.resize(img_as_float(data.camera()), (256, 256), anti_aliasing=True)
K = 15                                              # kernel support K x K
h_true = np.zeros((K, K))
for t in np.linspace(0, 1, 40):                     # a curved "camera-shake" path
    h_true[round(3 + 8 * t), round(2 + 10 * t + 2 * np.sin(3 * t))] += 1
h_true = gaussian_filter(h_true, 0.5); h_true /= h_true.sum()
g = convolve(f_true, h_true, mode="wrap") + 0.005 * rng.standard_normal(f_true.shape)

def otf(h, shape):                                  # centered kernel -> transfer function
    p = np.zeros(shape); p[:h.shape[0], :h.shape[1]] = h
    return fft2(np.roll(p, (-(h.shape[0] // 2), -(h.shape[1] // 2)), (0, 1)))

G = fft2(g)
Dx, Dy = otf(np.array([[0, -1, 1]]), g.shape), otf(np.array([[0], [-1], [1]]), g.shape)
```

```
def image_step(h, lam):        # f = argmin ||h*f - g||^2 + lam ||grad f||^2  (closed form,
 ↪  CLS-like)
    H = otf(h, g.shape)
    return np.real(ifft2(np.conj(H) * G / (np.abs(H)**2 + lam * (np.abs(Dx)**2 +
     ↪  np.abs(Dy)**2))))

def shock(f, n=5, dt=0.5):     # edge prediction: a shock filter turns soft edges into steps
    f = gaussian_filter(f, 0.7)
    for _ in range(n):
        gy, gx = np.gradient(f)
        f = f - dt * np.sign(laplace(f)) * np.hypot(gx, gy)
    return f

def kernel_step(f, keep=0.02, beta=1e-3):   # least squares in the gradient domain, strong
 ↪  edges only
    num = den = 0
    for D in (Dx, Dy):
        fx = np.real(ifft2(D * fft2(f)))
        fx[np.abs(fx) < np.quantile(np.abs(fx), 1 - keep)] = 0
        Fx = fft2(fx)
        num, den = num + np.conj(Fx) * D * G, den + np.abs(Fx)**2
    h = np.roll(np.real(ifft2(num / (den + beta))), (K // 2, K // 2), (0, 1))[:K, :K]
    h = np.maximum(h, 0)                            # project onto h >= 0 ...
    return h / h.sum()                              # ... and sum(h) = 1

def best_shift(fn):            # blind deconvolution recovers h (and f) only up to a small
 ↪  translation
    return min(fn(dy, dx) for dy in range(-3, 4) for dx in range(-3, 4))
kernel_error = lambda h: best_shift(lambda dy, dx: np.linalg.norm(np.roll(h, (dy, dx), (0,
 ↪  1)) - h_true)) / np.linalg.norm(h_true)
psnr = lambda f: -best_shift(lambda dy, dx: 10 * np.log10(np.mean((np.clip(np.roll(f, (dy,
 ↪  dx), (0, 1)), 0, 1) - f_true)[8:-8, 8:-8] ** 2)))

h = np.zeros((K, K)); h[K // 2, K // 2] = 1
h = gaussian_filter(h, 1.0); h /= h.sum()           # start from an almost-delta kernel
 ↪  ("no blur")
print(f"kernel error at start: {kernel_error(h):.2f}")
for _ in range(30):                                 # alternate: image update, then kernel
 ↪  update
    h = kernel_step(shock(image_step(h, lam=0.02)))
print(f"kernel error after 30 alternations: {kernel_error(h):.2f}")
print(f"PSNR  blurry {psnr(g):.2f} dB | estimated kernel {psnr(image_step(h, 1e-2)):.2f} dB"
      f" | true kernel {psnr(image_step(h_true, 1e-2)):.2f} dB")
# kernel error at start: 1.19
# kernel error after 30 alternations: 0.57
# PSNR  blurry 22.59 dB | estimated kernel 27.11 dB | true kernel 29.67 dB
```

The relative kernel error roughly halves, and the final non-blind step with the estimated kernel recovers most of the gap between the blurry input and the true-kernel result. Be honest about what this toy is: the blur is synthetic, uniform and circular (so the FFT model is exact), the noise is low, and the parameters (`lam`, `keep`, the number of shock iterations) were tuned on this one image. There is no image pyramid, so larger kernels would fail, and with real photos you would also need

boundary handling, saturated-pixel handling and robust losses. Both errors are measured after the best small shift, because of the translation ambiguity explained above.

Data and milestones

Nah et al. [44] (CVPR 2017) addressed this with the **GoPro** dataset: they recorded sharp video with a high-speed camera and averaged consecutive frames to produce realistically blurred images, keeping the middle frame as the sharp target. Their network is **multi-scale**: it restores a coarse version first, then refines at finer scales, like the coarse-to-fine schemes of classical blind deblurring, but without estimating a kernel at all. GoPro is still the most common deblurring benchmark.

Today the most common baselines are the general backbones **Restormer** [23] and **NAFNet** [24]. NAFNet's striking finding is that the usual nonlinear activations (ReLU, GELU, Sigmoid) are not needed: they can be replaced by a simple element-wise multiplication of two feature halves ("SimpleGate") or removed, giving a "nonlinear activation free" network that is both simpler and stronger on GoPro and SIDD. The deblurring survey by Zhang et al. [69] covers the wider landscape (blind and non-blind, video, defocus, losses).

Inpainting (brief)

Plain version. Fill in missing or damaged regions (scratches, removed objects, holes) with plausible content.

Inpainting is the restoration task closest to image **generation**: inside a large hole there is nothing to restore, only something to invent that matches the surroundings. The degradation is a mask, $y = m \odot x$, where m is 0 in the hole and 1 elsewhere. **Masked autoencoders** (MAE) [45] can be seen as inpainting used as a pre-training task: hide 75% of image patches and train a ViT to reconstruct them, which turns out to teach strong visual representations. For practical inpainting, **LaMa** [46] uses fast Fourier convolutions with an image-wide receptive field and training on large masks to fill big holes at high resolution; today, diffusion-based inpainting is the default for large, semantic edits.

Matting (brief)

Plain version. Cut out the foreground (a person, their hair) with soft, partially transparent edges so it can be pasted on a new background.

Matting assumes each pixel is a blend of foreground and background,

$$I = \alpha F + (1 - \alpha)B,$$

where $\alpha \in [0, 1]$ is the opacity (alpha matte), F the foreground color and B the background color. With seven unknowns per pixel (α and two RGB colors) and three observations, the problem is badly underdetermined, so classical methods ask the user for a **trimap** (definite foreground, definite background, unknown band). It can be seen as a soft variant of segmentation. **Background**

Matting [47] replaces the trimap with an extra photo of the background without the subject, and trains networks first on synthetic composites and then on real images with an adversarial loss that judges how natural the new composites look.

General backbones

Plain version. Across all these tasks, a handful of network designs keep winning. They are general "image-to-image engines" that you can train on any of the degradations above.

Backbone	Core idea	Why it suits restoration
U-Net [48]	Encoder–decoder with skip connections at every scale	Large receptive field from the downsampling path, fine detail from the skips
SwinIR [14]	Residual groups of shifted-window self-attention blocks at full resolution	Content-adaptive weights; strong for SR, denoising, JPEG
Restormer [23]	U-Net of transformer blocks; attention computed across **channels** (transposed attention) plus gated feed-forward	Cost linear in the number of pixels, so it scales to large images
NAFNet [24]	U-Net of simple conv blocks with LayerNorm, SimpleGate and simplified channel attention; no nonlinear activations	Very strong accuracy per FLOP; the default "simple baseline"
MambaIR / MambaIRv2 [49], [50]	Selective state-space (Mamba) scans with local enhancement and channel attention; v2 adds "attentive" non-causal scanning	Global receptive field at linear cost

Design reasons that recur. (1) **Global residual**: predict $x - y$ (or x minus an upsampled y) so the network learns only the missing part. (2) **No batch normalization** in most SR/restoration nets since EDSR [10]: BN statistics differ between training patches and full test images and hurt color/range fidelity. (3) **Large receptive field** at manageable cost: multi-scale U-Nets, windowed or channel attention, state-space scans. Restoration needs context (to tell a texture from noise) *and* full-resolution detail. (4) **Attention or gating** for content-adaptive processing: channel attention (RCAN), gating (NAFNet, Restormer). The post's "Extension" list predicted new architectures

from ViT, flow, GAN and diffusion; by 2026 the state-space model joined CNNs and transformers as a third backbone option [49], [50].

All-in-one and universal restoration

Plain version. Instead of training one model for rain, another for haze and another for noise, train *one* model that figures out what is wrong with the image and fixes it.

Motivation

The post's "Extension" section lists all-in-one restoration as one of the hottest topics of 2023, starting around 2021. The motivation is practical: in a deployed system we do not know in advance which degradation (or mixture) a given image has, and weather can change or combine at any moment. A bank of task-specific models needs a reliable classifier in front and wastes memory; a single model that adapts is more convenient. The difficulty is **task interference**: different degradations need different, sometimes opposite operations (denoising smooths, deblurring sharpens), so naive multi-task training underperforms specialists.

Representative methods

- **AirNet** [51] (CVPR 2022), the "airnet" of the original post: a contrastive-learning degradation encoder extracts a representation of *how* the image is degraded, without labels, and that representation guides the restoration network. It handles unknown corruption types with one model; its noise + rain + haze setting became the standard "three-task" benchmark.
- **TransWeather** [52] (CVPR 2022): a single transformer encoder–decoder for rain, fog and snow, with learnable **weather-type queries** in the decoder.
- **PromptIR** [53] (NeurIPS 2023): a Restormer-style network with lightweight **prompt blocks**. Learnable prompt components are combined with weights predicted from the input features and injected into the decoder, encoding degradation-specific information. Tested on denoising, deraining and dehazing.
- **AdaIR** [54] (ICLR 2025): observes that different degradations affect **different frequency subbands**, then mines low- and high-frequency features from the input and modulates them adaptively. Covers five degradations: noise, haze, rain, motion blur and low light.
- **DA-CLIP** [55] (ICLR 2024): adapts the vision–language model CLIP with a trained controller so that its image encoder yields both a clean-content embedding and a degradation embedding from a corrupted image; these guide a restoration network through cross-attention.

- **InstructIR** [56] (ECCV 2024): the user writes a natural-language instruction ("remove the rain", "make it brighter"), a text encoder embeds it, and the embedding modulates the restoration network.

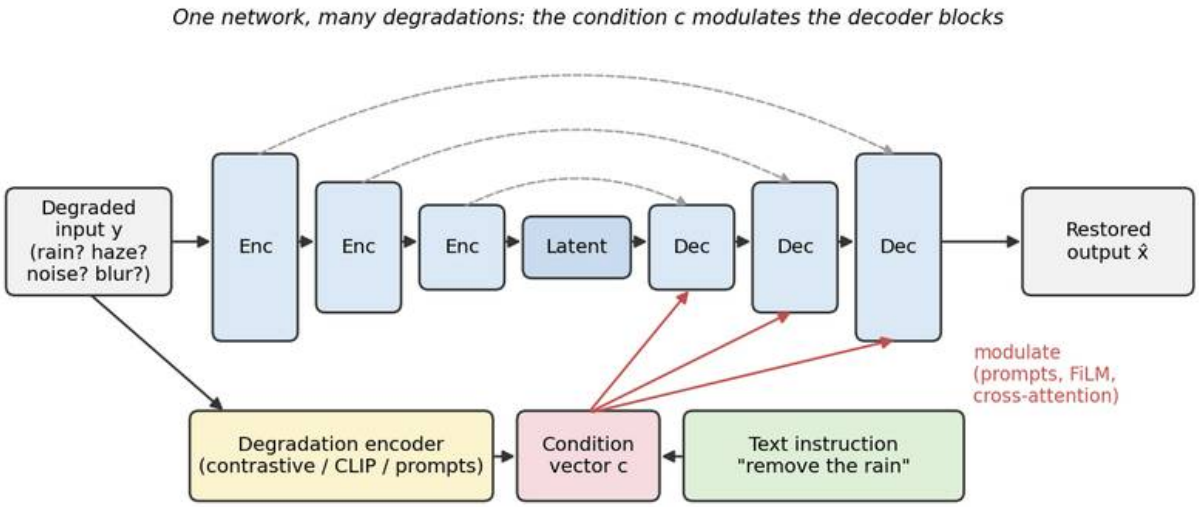


Figure 5 — The common template behind all-in-one restoration: an encoder–decoder backbone plus a degradation-aware condition c, inferred from the image (AirNet, PromptIR, AdaIR, DA-CLIP) or given as text (InstructIR), that modulates the decoder.

The post's original motivation is still the core; what changed is *who decides* the degradation type. It moved from fixed task labels to learned prompts, to vision–language embeddings and finally to human instructions, often backed by large pretrained models. The all-in-one survey [72] discusses this in detail.

Loss functions

Plain version. The loss decides what "good" means. Pixel losses reward being close to the truth on average; perceptual and adversarial losses reward looking real; frequency losses reward getting fine detail.

Pixel losses. With $\hat{x} = f_\theta(y)$,

$$\ell_2 = \frac{1}{P}\sum_p (\hat{x}_p - x_p)^2, \qquad \ell_1 = \frac{1}{P}\sum_p |\hat{x}_p - x_p|, \qquad \ell_\varepsilon = \frac{1}{P}\sum_p \sqrt{(\hat{x}_p - x_p)^2 + \varepsilon^2},$$

where P is the number of pixel values and ε a small constant that makes the last one a smooth, differentiable version of L1. L2 directly optimizes PSNR but penalizes large errors heavily and converges to the posterior mean. Zhao et al. [57] showed that the loss alone changes results noticeably with the architecture fixed, and that L1 and perceptually motivated losses (including

combinations with SSIM) often beat L2 even on L2-related metrics. L1 is the default in most modern SR and restoration papers.

Perceptual (feature) loss. Compare images in the feature space of a pretrained classification network ϕ (usually VGG) [58]:

$$\ell_{\text{feat}} = \frac{1}{C_j H_j W_j} \|\phi_j(\hat{x}) - \phi_j(x)\|_2^2,$$

where ϕ_j is the feature map of layer j with C_j channels and size $H_j \times W_j$. It tolerates small misalignments and rewards matching edges and textures rather than exact values.

Adversarial loss. A discriminator D_ψ learns to tell restored from real images and the restorer learns to fool it, e.g. $\ell_{\text{adv}} = -\log D_\psi(\hat{x})$. This pushes outputs onto the manifold of natural images (SRGAN [12], ESRGAN [13], Real-ESRGAN [4]) at the cost of possible hallucinated texture. In practice GAN-based restorers use a weighted sum such as $\ell = \ell_1 + \lambda_p \ell_{\text{feat}} + \lambda_a \ell_{\text{adv}}$ with weights λ_p, λ_a.

Frequency losses. Networks tend to fit low frequencies first and under-fit fine detail. Adding a loss on the Fourier transform $\mathcal{F}$, such as $\|\mathcal{F}(\hat{x}) - \mathcal{F}(x)\|_1$ on complex values or on amplitude, directly penalizes missing high frequencies. The **focal frequency loss** [59] goes further and adaptively up-weights the frequency components that are currently hardest to reconstruct. Frequency terms are now common in deblurring and all-in-one models.

Evaluation metrics

Plain version. "Is the restored image close to the truth?" and "Does it look like a real photo?" are different questions, answered by different metrics, and you cannot maximize both at once.

Full-reference metrics

PSNR (peak signal-to-noise ratio):

$$\text{PSNR}(x, \hat{x}) = 10 \log_{10} \frac{\text{MAX}^2}{\text{MSE}(x, \hat{x})},$$

where MAX is the largest possible pixel value (1 or 255) and MSE is the mean squared error. It is measured in dB; +1 dB is a 21% reduction in MSE.

SSIM (structural similarity) [60] compares local means, variances and covariance:

$$\text{SSIM}(a, b) = \frac{(2\mu_a\mu_b + C_1)(2\sigma_{ab} + C_2)}{(\mu_a^2 + \mu_b^2 + C_1)(\sigma_a^2 + \sigma_b^2 + C_2)},$$

computed over local windows a, b of the two images and averaged, where μ are local means, σ^2 local variances, σ_{ab} the local covariance and C_1, C_2 small constants for stability. It correlates better with perceived structure than PSNR but still prefers smooth results.

LPIPS [61] measures distance between deep-network features (calibrated on human similarity judgments). Lower is better, and it rewards realistic textures that PSNR penalizes.

```
import numpy as np
from scipy import ndimage as ndi
from skimage import data, img_as_float32
from skimage.metrics import peak_signal_noise_ratio, structural_similarity

x = img_as_float32(data.camera())
y = np.clip(x + np.random.default_rng(0).normal(0, 0.1, x.shape), 0, 1).astype(np.float32)
x_hat = ndi.gaussian_filter(y, 1.0)

def psnr(ref, est, peak=1.0):
    mse = np.mean((ref.astype(np.float64) - est) ** 2)
    return 10 * np.log10(peak ** 2 / mse)

print(f"PSNR  ours {psnr(x, x_hat):.2f} dB | skimage {peak_signal_noise_ratio(x, x_hat,
 ↪  data_range=1):.2f} dB")
print(f"SSIM  {structural_similarity(x, x_hat, data_range=1):.3f}")
# PSNR  ours 27.20 dB | skimage 27.20 dB
# SSIM  0.639
```

Always report the conventions (RGB or Y channel, border crop, data range, 8-bit rounding): differences in these alone can shift published PSNR by tenths of a dB.

No-reference metrics

Plain version. A no-reference (or *blind*) metric looks at a single image and guesses how good a person would say it is, without a "correct answer" to compare against. It is like a teacher grading an essay without an answer key: the teacher relies on what good essays usually look like. Restoration needs such metrics because the photos we most want to fix (real blurry, noisy, rainy, dark photos) have no clean version anywhere.

Full-reference, reduced-reference and no-reference

Image quality assessment (IQA) methods are grouped by how much they know about the pristine image x:

- **Full-reference (FR)**: the whole x is available, and the metric compares $\hat{x}$ with it pixel by pixel or feature by feature (PSNR, SSIM [60], LPIPS [61]). This is what synthetic benchmarks use.

- **Reduced-reference (RR)**: only a few summary features of x are available (for example statistics sent alongside a compressed video), and the metric compares the same features computed on $\hat{x}$.
- **No-reference (NR, blind)**: only $\hat{x}$ is available. The metric must carry its own model of what "good" looks like.

Restoration needs NR metrics for three reasons. (1) **Real test images have no ground truth**: real-world SR, deraining and dehazing are evaluated on photos where nobody has the clean version.

(2) **The reference itself can be imperfect or only one of many valid answers**: a GAN or diffusion output can look excellent yet have low PSNR because its texture differs from the reference's (the perception–distortion trade-off below). (3) **NR scores can steer systems at run time**, for example to choose among outputs or parameters when no reference exists. The price: an NR metric is a *model of human opinion*, and every model has blind spots.

How NR metrics are validated: correlation with human opinion

Plain version. We trust an NR metric only if it ranks images the way people do. So researchers collect many human ratings on many images and check how well the metric's scores agree with them.

Mean opinion score. In a subjective study, each image i is rated by several observers, and the average rating is its **mean opinion score**, $\mathrm{MOS}_i = \frac{1}{R}\sum_{r=1}^{R} s_{ir}$, where s_{ir} is the score of rater r and R the number of raters. Some datasets publish a **differential** score (DMOS), based on the difference between the ratings of the distorted image and its reference; for DMOS, lower usually means better.

SRCC (Spearman rank-order correlation) measures whether the metric puts the images in the same *order* as people:

$$\mathrm{SRCC} = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n\,(n^2-1)},$$

where n is the number of test images and d_i the difference between the rank of image i by predicted score and its rank by MOS (this short form assumes no tied ranks; with ties one computes the Pearson correlation of the ranks). SRCC is unchanged by any monotonic transformation of the scores, so it measures **prediction monotonicity**.

PLCC (Pearson linear correlation) measures how *linearly* the scores agree:

$$\mathrm{PLCC} = \frac{\sum_{i=1}^{n}(q_i - \bar{q})(m_i - \bar{m})}{\sqrt{\sum_{i=1}^{n}(q_i - \bar{q})^2}\sqrt{\sum_{i=1}^{n}(m_i - \bar{m})^2}},$$

where q_i is the metric's score for image i, m_i its MOS, and $\bar{q}$, $\bar{m}$ their means. Because metric scales are arbitrary, it is common practice to first pass q_i through a fitted monotonic (usually logistic) function and compute PLCC on the mapped scores, so PLCC measures **prediction accuracy** after calibration. Both lie in $[-1, 1]$. For "lower is better" metrics such as NIQE the raw correlation with MOS is negative, and papers report its absolute value. A metric is typically trained on part of a dataset and tested on the rest, and its cross-dataset correlation is the more honest test of generalization.

Human-rated datasets. NR models are trained and tested on IQA datasets. They differ in *where the distortions come from*, which matters a lot for restoration:

Dataset	Distortions	What it contains
LIVE (Release 2) [104]	Synthetic	Images with five distortion types (JPEG, JPEG2000, Gaussian blur, white noise, JPEG2000 bit errors), with DMOS
TID2013 [105]	Synthetic	25 reference images × 24 distortion types × 5 levels = 3,000 distorted images; scores from 971 experiments with observers in five countries
KADID-10k [106]	Synthetic	81 pristine images × 25 distortion types × 5 levels = 10,125 images, 30 crowdsourced ratings each
KonIQ-10k [107]	Authentic (in the wild)	10,073 photos with about 1.2 million ratings from 1,459 crowd workers
SPAQ [108]	Authentic (smartphones)	11,125 photos from 66 smartphones, with quality, attribute (brightness, colorfulness, contrast, noisiness, sharpness) and scene labels plus EXIF data
PIPAL [109]	Restoration outputs	Outputs of image restoration algorithms, including GAN-based ones, with over 1.13 million human judgments converted to scores by an Elo system

Synthetic datasets control the distortion exactly but contain only "textbook" damage. Authentic datasets contain real camera problems but no restoration outputs. Only datasets such as PIPAL contain the kind of images a restoration paper actually needs to score.

Family 1: natural-scene-statistics (NSS) models

Plain version. Clean photos share hidden statistical habits. If you remove local brightness and local contrast from a photo, the leftovers follow a bell-shaped histogram of a typical shape, and neighboring leftovers are only weakly related. Blur, noise and compression change that shape in characteristic ways. NSS metrics measure the shape and ask how unusual it is.

MSCN coefficients. For a grayscale image I, the **mean-subtracted contrast-normalized** (MSCN) coefficients are [94]

$$\hat{I}(i,j) = \frac{I(i,j) - \mu(i,j)}{\sigma(i,j) + C}, \qquad \mu(i,j) = \sum_{k,l} w_{k,l}\, I(i+k, j+l), \qquad \sigma(i,j) = \sqrt{\sum_{k,l} w_{k,l}\, (I(i+k,j+l) - \mu(i,j))^2},$$

where (i,j) is a pixel position, w a small circularly symmetric Gaussian window whose weights sum to 1, μ and σ the local weighted mean and standard deviation, and C a small constant (1 for 8-bit images) that prevents division by zero in flat regions. This is a divisive normalization, loosely modeled on contrast gain control in early human vision.

The generalized Gaussian fit. The histogram of $\hat{I}$ is modeled by a zero-mean generalized Gaussian distribution (GGD),

$$f(t;\, \alpha, \sigma^2) = \frac{\alpha}{2\beta\,\Gamma(1/\alpha)} \exp\!\Big(-(|t|/\beta)^{\alpha}\Big), \qquad \beta = \sigma\sqrt{\frac{\Gamma(1/\alpha)}{\Gamma(3/\alpha)}},$$

where t is an MSCN value, $\alpha > 0$ the **shape** ($\alpha = 2$ is a Gaussian, $\alpha = 1$ a Laplacian; smaller α means a peakier histogram with heavier tails, larger α a flatter top), σ^2 the variance, β the corresponding scale and Γ the gamma function. A simple way to estimate α is **moment matching**: the ratio $\mathbb{E}[t^2]/(\mathbb{E}|t|)^2 = \Gamma(1/\alpha)\,\Gamma(3/\alpha)/\Gamma(2/\alpha)^2$ depends only on α, so one computes the left side from the data and searches for the α that matches it.

BRISQUE [94] (Blind/Referenceless Image Spatial Quality Evaluator) computes the GGD shape and variance of the MSCN coefficients, plus parameters of *asymmetric* GGDs fitted to products of neighboring MSCN coefficients in four orientations (horizontal, vertical and the two diagonals), at two image scales. A support-vector regressor trained on human scores of synthetically distorted images maps these features to a quality score. Lower is better. It is **opinion-aware**: it needs human scores for training.

NIQE [62] (Natural Image Quality Evaluator) is **opinion-unaware**: it never sees human scores or distorted images. It computes similar NSS features on sharp patches, fits a multivariate Gaussian (MVG) with mean ν_1 and covariance Σ_1 to features from a corpus of pristine images, fits another MVG (ν_2, Σ_2) to the test image's patches, and reports

$$D(\nu_1, \nu_2, \Sigma_1, \Sigma_2) = \sqrt{(\nu_1 - \nu_2)^{\top} \Big(\frac{\Sigma_1 + \Sigma_2}{2}\Big)^{-1} (\nu_1 - \nu_2)},$$

a Mahalanobis-like distance between the two Gaussians ($^\top$ is the transpose). Lower is better; zero means "statistically like the pristine corpus".

PIQE [96] (Perception-based Image Quality Evaluator) also uses MSCN coefficients but works **block by block**: it divides the image into 16×16 blocks, flags blocks that show blocking artifacts or noise, and pools the distortion of the flagged blocks into one score. It needs no training on opinion scores, and it gives a local map of where the image looks distorted. In the common MATLAB implementation the score lies between 0 and 100, lower is better.

Ma et al.'s score and the Perceptual Index. Ma et al. [95] ran human studies on super-resolved images and learned a no-reference metric for SR from those scores, using statistical features in spatial and frequency domains and a two-stage regression; their score ("Ma", higher is better) lies roughly between 0 and 10. The 2018 PIRM challenge on perceptual SR [63] combined it with NIQE into the **Perceptual Index**,

$$\mathrm{PI} = \tfrac{1}{2}\big((10 - \mathrm{Ma}) + \mathrm{NIQE}\big),$$

so that lower PI means better perceptual quality. PIRM ranked methods by PI within three regions of the perception–distortion plane, defined by RMSE at most 11.5, 12.5 and 16, so that realism was compared at a fixed distortion budget. PI is still reported in many perceptual SR papers.

Code: MSCN statistics in practice. The snippet computes MSCN coefficients, the GGD shape by moment matching, the MSCN variance and the correlation between horizontal neighbors (a simplified version of BRISQUE's pairwise features) on `camera` and on blurred, noisy and over-sharpened versions:

```
import numpy as np
from scipy.ndimage import gaussian_filter
from scipy.special import gamma
from skimage import data, img_as_float

def mscn(img, sigma=7/6, C=1.0):
    """Mean-subtracted contrast-normalized coefficients of a grayscale image in [0, 255]."""
    mu = gaussian_filter(img, sigma)
    sd = np.sqrt(np.abs(gaussian_filter(img * img, sigma) - mu * mu))
    return (img - mu) / (sd + C)

def ggd_shape(x):
    """Moment-matching estimate of the generalized Gaussian shape alpha (2 = Gaussian)."""
    alphas = np.arange(0.2, 10.0, 0.001)
    r_theory = gamma(1 / alphas) * gamma(3 / alphas) / gamma(2 / alphas) ** 2
    r_data = np.mean(x ** 2) / np.mean(np.abs(x)) ** 2
    return alphas[np.argmin((r_theory - r_data) ** 2)], np.mean(x ** 2)

rng = np.random.default_rng(0)
clean = img_as_float(data.camera()) * 255
versions = {
    "clean": clean,
```

```
    "blur (sigma=2)": gaussian_filter(clean, 2),
    "noise (sigma=20)": np.clip(clean + rng.normal(0, 20, clean.shape), 0, 255),
    "over-sharpened": np.clip(clean + 2.0 * (clean - gaussian_filter(clean, 2)), 0, 255),
}
for name, img in versions.items():
    m = mscn(img)
    alpha, var = ggd_shape(m)
    rho = np.mean(m[:, :-1] * m[:, 1:]) / var          # correlation of horizontal neighbors
    print(f"{name:17s} GGD shape {alpha:4.2f} | MSCN variance {var:4.2f} | neighbor corr.
     ↪  {rho:+.2f}")
# clean             GGD shape 1.56 | MSCN variance 0.28 | neighbor corr. +0.12
# blur (sigma=2)    GGD shape 0.87 | MSCN variance 0.04 | neighbor corr. +0.89
# noise (sigma=20)  GGD shape 3.12 | MSCN variance 0.68 | neighbor corr. -0.10
# over-sharpened    GGD shape 2.35 | MSCN variance 0.43 | neighbor corr. +0.09
```

Each distortion pushes the features in its own direction (Figure 6). **Blur** removes local contrast, so most MSCN values collapse toward zero: tiny variance, a very peaky histogram (small α) and strongly correlated neighbors. **Noise** adds independent energy: larger variance, a flat-topped histogram (α above 2) and neighbor correlation near zero (slightly negative, because subtracting the local mean couples neighbors). **Over-sharpening** moves the features *in the same direction as noise*. That is the core weakness of NSS metrics for restoration: they cannot tell restored texture from invented high-frequency energy. Note also that the absolute numbers depend on content and on C (the clean `camera` image is not a unit Gaussian), which is why BRISQUE needs a trained regressor and NIQE a pristine reference corpus rather than fixed thresholds.

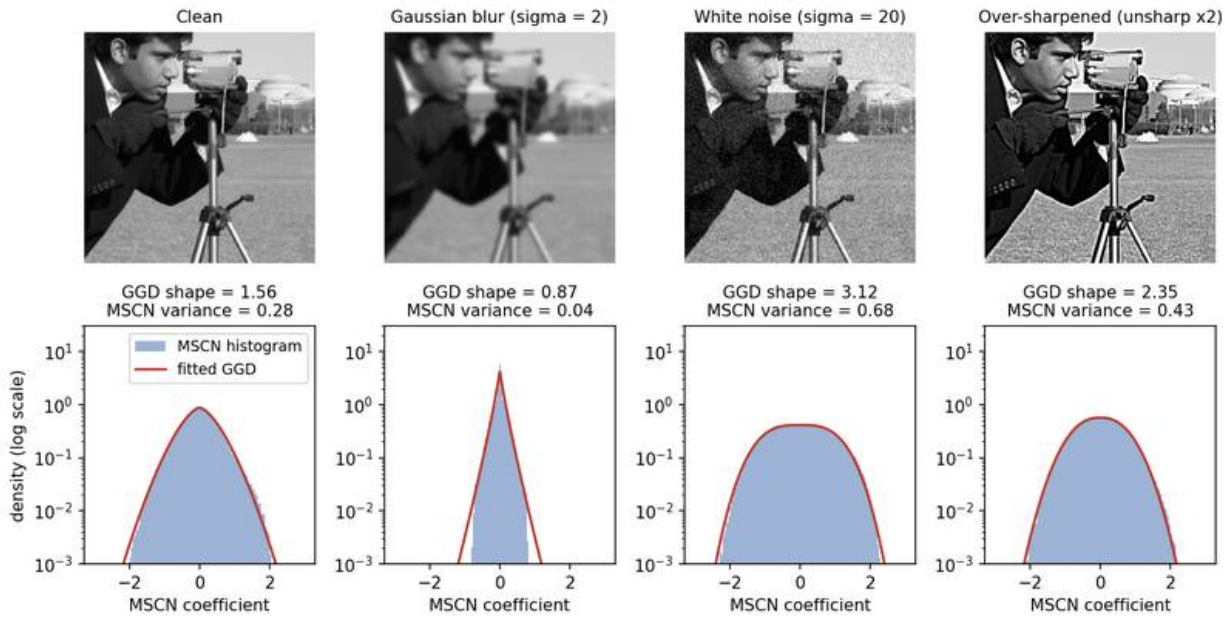


Figure 6 — MSCN statistics, the basis of BRISQUE, NIQE and PIQE. Blur makes the histogram narrow and peaky; noise and over-sharpening both make it wider and flatter, so an NSS metric sees them as similar

changes.

Family 2: learned deep models

Plain version. Instead of hand-made statistics, train a neural network to predict human ratings directly. Deep features capture content, semantics and complex artifacts far better, but the model only knows the kinds of images and opinions it was trained on.

- **NIMA** [97] predicts the whole *distribution* of human ratings with a CNN, not a single number. With p_k the predicted probability of score k (on a 1–10 scale), the reported score is the mean $\hat{s} = \sum_k k\, p_k$, and the spread of the distribution says how much people would disagree. Checkpoints trained on aesthetic ratings judge "is this a nice photo", not "is this a faithful restoration".
- **HyperIQA** [98] targets authentic, in-the-wild distortions. It splits assessment into three stages:

  a backbone first *understands the content*, a **hyper network** then generates the weights of the quality-prediction network from that content (the "perception rule" adapts to what is in the picture, since the same blur is judged differently on a sky and on a face), and the generated network predicts the score.

- **MUSIQ** [99] is a multi-scale transformer that takes the image at its **native resolution and aspect**

  ratio plus downscaled versions, using hash-based 2-D spatial and scale embeddings, so no resizing or cropping changes the very artifacts being judged.

- **MANIQA** [100] combines vision-transformer features with transposed (channel) attention blocks and scale Swin-transformer blocks, and predicts the score with a dual-branch, patch-weighted head. It won the no-reference track of the NTIRE 2022 perceptual IQA challenge, whose data include outputs of restoration and GAN-based algorithms, so it is closer to the restoration setting than metrics trained on camera distortions.
- **TOPIQ** [101] follows a **top-down** idea: high-level semantic features decide where to look, and a cross-scale attention mechanism uses them to weight lower-level, distortion-sensitive features (its network is called CFANet, with a ResNet-50 backbone). The same design works for FR and NR assessment and is light compared with transformer-based IQA models.

All of these are **opinion-aware** and output "higher is better" scores, and each depends on its training dataset: a model trained on KonIQ-10k and the same architecture trained on PIPAL can rank restoration methods differently.

Family 3: vision-language and large multimodal models

Plain version. Models trained on huge collections of images and text already "know" what a good or bad photo looks like. You can ask them, either by comparing the image with text prompts or

by literally asking a chatbot-style model to rate it.

CLIP-IQA [102] compares the CLIP embedding of the image with the embeddings of an antonym prompt pair such as "Good photo." and "Bad photo.". With $\mathbf{x}$ the image embedding, $\mathbf{t}_1, \mathbf{t}_2$ the two text embeddings and $s_i = \mathbf{x}^\top \mathbf{t}_i / (\|\mathbf{x}\| \, \|\mathbf{t}_i\|)$ their cosine similarities, the score is

$$\text{CLIP-IQA} = \frac{e^{s_1}}{e^{s_1} + e^{s_2}} \in [0, 1],$$

higher is better. The basic version is **zero-shot** (no opinion scores at all), and other prompt pairs ("Noisy photo." / "Clean photo.", "Sharp photo." / "Blurry photo.") probe single attributes. The wording of the prompts changes the scores.

Q-Align [103] fine-tunes a large multimodal model to answer the way human raters do, with a **text rating level** (bad, poor, fair, good, excellent) instead of a number. At test time, with X_{l_i} the model's logit for level l_i, the score is the expected level under a softmax restricted to the five words,

$$S = \sum_{i=1}^{5} i \cdot \frac{e^{X_{l_i}}}{\sum_{j=1}^{5} e^{X_{l_j}}},$$

where $i = 1, \ldots, 5$ indexes the levels from bad to excellent. Such models are strong across datasets but heavy, and they inherit the biases of their base model.

Distribution-level metrics

Some metrics score a *set* of outputs rather than one image. **FID** [64] compares the distribution of Inception features of the restored set with that of a set of real images; it was designed for generative models and is used for generative restoration such as diffusion SR, but it needs many images and says nothing about any single one. **IS** (Inception Score), mentioned in the post, measures the confidence and diversity of class predictions; it suits class-conditional generation rather than restoration and is rarely used there today.

Pitfalls

- **Metrics can be gamed.** A restorer that adds grain or sharpens edges can improve NSS scores, as the code above shows, without becoming more faithful. Any differentiable metric (and any learned IQA network) used as a training loss or for model selection can be optimized directly, and the network may learn to please the metric rather than people. If a method uses CLIP or an IQA network in its loss, evaluating it with the same model is not independent.

- **Domain mismatch.** Most opinion-aware metrics were trained on synthetic distortions (LIVE, TID2013, KADID-10k) or on camera photos (KonIQ-10k, SPAQ). Restoration outputs contain different artifacts: hallucinated textures, over-smoothing, GAN patterns. This mismatch was the explicit motivation for PIPAL [109], whose authors found that existing IQA methods could not fairly evaluate GAN-based restoration. For low light, the problems are exposure and color, which most NSS features normalize away; see the low-light-specific discussion in DLCV 05.
- **Metrics disagree.** NSS, learned and vision-language metrics often rank the same methods differently, and all can disagree with people. Scores also depend on the implementation, checkpoint, color space, input size and whether the image is resized, so numbers copied from different papers are often not comparable.
- **"Higher NR quality" is not "more correct".** An NR metric cannot know whether a sharp face or line of text matches the real scene. A hallucinated license plate can score perfectly.

Recommended practice

1. When a reference exists, lead with FR metrics (PSNR, SSIM, LPIPS) and add NR metrics as a complement; place your method on the perception–distortion plane (next subsection).
2. On real images without references, report **several NR metrics from different families** (for example NIQE or PI, one or two learned models such as MUSIQ, MANIQA or TOPIQ, and a vision-language one such as CLIP-IQA or Q-Align), and say which way each one points.
3. Use one shared implementation and state it, with versions and checkpoints. The open-source

   IQA-PyTorch toolbox (`pyiqa`) [110] reimplements most metrics in this section (NIQE, BRISQUE, PIQE, the Ma score and PI, NIMA, HyperIQA, MUSIQ, MANIQA, TOPIQ, CLIP-IQA, Q-Align, and FR metrics) behind one interface.
4. Do not tune on the judge: keep at least one metric that played no role in training or model selection.
5. Back important claims with a **user study** (mean opinion scores or paired comparisons, blind and randomized); DLCV 05 describes MOS and Bradley–Terry paired-comparison protocols. When metrics disagree with each other or with people, say so.

Summary table

Metric	Family	Needs training?	Better	Mainly sensitive to
NIQE [62]	NSS	Pristine images only (opinion-unaware)	Lower	Deviations of local statistics: blur, noise, compression
BRISQUE [94]	NSS + regression	Yes, human scores on synthetic distortions	Lower	The same statistics, mapped to human scores

Metric	Family	Needs training?	Better	Mainly sensitive to
PIQE [96]	NSS, block-wise	No opinion scores	Lower	Blocking and noise in local blocks; gives a distortion map
Ma [95]	Handcrafted features + regression	Yes, human scores on SR outputs	Higher	SR artifacts, texture and sharpness
PI [63]	NSS + learned	Through its Ma part	Lower	Perceptual quality of SR; rewards texture, including amplified noise
NIMA [97]	Learned CNN	Yes, rating distributions	Higher	Technical or aesthetic quality, depending on checkpoint
HyperIQA [98]	Learned CNN + hyper network	Yes	Higher	Authentic distortions, judged in the context of content
MUSIQ [99]	Learned transformer	Yes	Higher	Multi-scale quality at native resolution
MANIQA [100]	Learned transformer	Yes	Higher	Restoration and GAN artifacts (trained on such data)
TOPIQ [101]	Learned CNN, top-down attention	Yes	Higher	Distortions in semantically important regions
CLIP-IQA [102]	Vision-language	No (zero-shot version)	Higher	Overall "good photo" impression; depends on prompts
Q-Align [103]	Large multimodal model	Yes, ratings as text levels	Higher	Human-like overall quality judgments
FID [64]	Distribution-level	Pretrained Inception features	Lower	Distribution gap between a set of outputs and real images

The perception–distortion trade-off

Blau and Michaeli [65] formalized the tension. Let $\Delta(x, \hat{x})$ be any distortion measure (MSE, $1 - $ SSIM, ...) and $d(p_X, p_{\hat{X}})$ a divergence between the distribution of real images and that of restored images (perceptual quality: small d means outputs are statistically indistinguishable from real images). The **perception–distortion function** is

$$P(D) = \min_{p_{\hat{X}|Y}} d(p_X, p_{\hat{X}}) \quad \text{subject to} \quad \mathbb{E}[\Delta(X, \hat{X})] \leq D,$$

where the minimum is over all (possibly random) restoration methods. They proved that $P(D)$ is non-increasing and (for divergences convex in their second argument) convex: there is an unattainable region near the origin. A method cannot be both minimal-distortion and perfectly realistic. In particular, the MMSE estimator $\mathbb{E}[x \mid y]$ has the lowest distortion but a blurry, unrealistic output distribution; to look real you must accept higher MSE. GANs (and now diffusion

models) are principled ways to move along the bound.

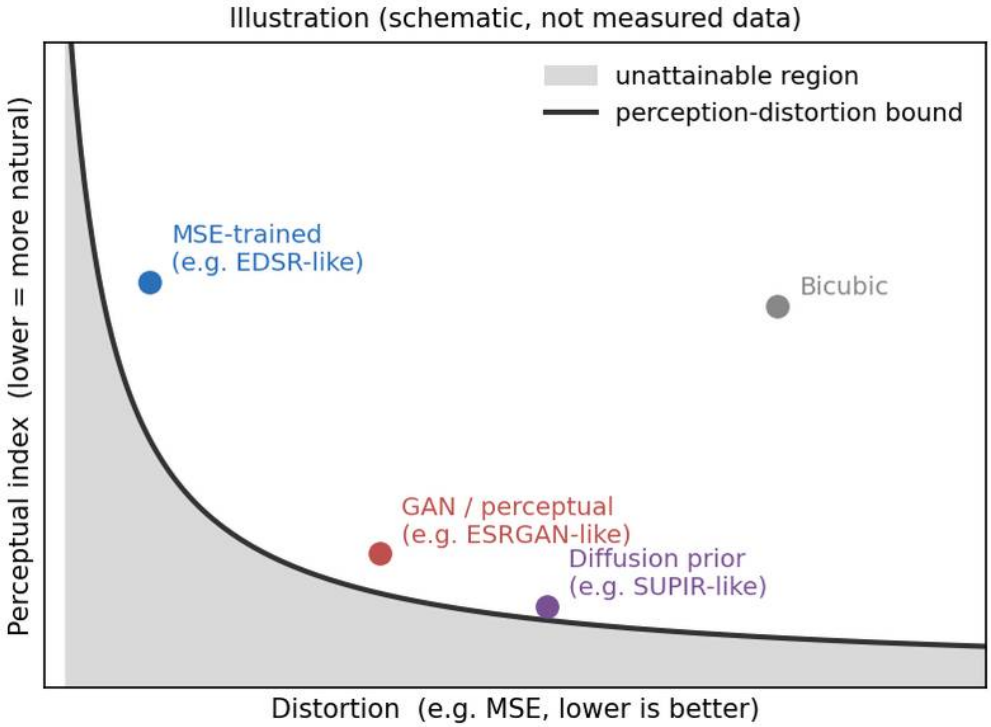


Figure 7 — Schematic of the perception–distortion trade-off [65] (*not measured data*). *The bound is convex and decreasing; methods trade fidelity* (*left*) *against realism* (*bottom*).

This is the update in the post's metric section: since diffusion models can generate plausible but unfaithful detail, the gap between fidelity (PSNR/SSIM) and perceptual quality has become central. The **NTIRE 2025** ×4 SR challenge therefore ran two tracks: a restoration track ranked by PSNR and a perceptual track ranked by a perceptual score [3], whereas the 2024 edition ranked by PSNR [2]. Practical advice: report both a distortion and a perceptual metric, and say where on the trade-off your method aims.

NTIRE challenges

Plain version. Each year a CVPR workshop runs public competitions on fixed datasets and publishes a report describing the winning ideas: the easiest way to see what currently works.

The NTIRE challenge reports are a good way to follow restoration now that per-task leaderboards are gone. Each report describes the task, data, evaluation protocol and the methods of top teams. Examples cited in this tutorial:

- **NTIRE 2017 SR** [19]: introduced DIV2K; EDSR [10] won.
- **NTIRE 2024** ×4 **SR** [2]: bicubic ×4 on DIV2K, ranked by PSNR.

- **NTIRE 2025 ×4 SR** [3]: separate restoration (PSNR) and perceptual tracks.
- **NTIRE 2025 raindrop removal** [40]: real day and night raindrop images with dual focus.

Two patterns appear across recent reports: winning entries are usually ensembles or large hybrids of the backbones above (HAT-, Restormer-, NAFNet- or Mamba-style, sometimes with diffusion priors) trained with heavy data and compute, and challenges increasingly use real captured data and perceptual tracks.

Modern view

The post closes with an extension list (medical images, multimodal, unsupervised, new architectures, all-in-one, low-level plus high-level, real-time, underwater). Most of these are now active areas. Here we summarize the survey literature task by task, then the 2023–2026 state of the art and the open problems.

Survey papers

- **Wang, Chen and Hoi, "Deep Learning for Image Super-resolution: A Survey" (TPAMI 2021)** [66]. Organizes SR into supervised, unsupervised and domain-specific SR; within supervised SR it dissects models by upsampling position (pre-, post-, progressive, iterative up-and-down), upsampling operator (interpolation, transposed convolution, sub-pixel), network design (residual, recursive, dense, attention, multi-path), losses and training strategies. Takeaway: most SR progress can be described as choices along these axes, and unsupervised/real-world SR was flagged as the main open direction, which the field then pursued.
- **Tian et al., "Deep Learning on Image Denoising: An Overview" (Neural Networks 2020)** [67]. Groups CNN denoisers into additive white noise, real noise, blind denoising, and hybrid noise with blur or low resolution, and compares them on public benchmarks. Takeaway: the realism of the noise model, not the network, is often the bottleneck.
- **Elad, Kawar and Vaksman, "Image Denoising: The Deep Learning Revolution and Beyond" (SIAM J. Imaging Sciences 2023)** [68]. Traces denoising from classical priors to deep networks, then argues that denoisers are now building blocks for other inverse problems (plug-and-play, regularization by denoising) and for diffusion-based image generation, and that restoration should embrace multiple valid solutions (posterior sampling) instead of one answer. Takeaway: the field's frontier moved from "better MMSE denoisers" to "sampling from the posterior".
- **Zhang et al., "Deep Image Deblurring: A Survey" (IJCV 2022)** [69]. Covers causes of blur, datasets and metrics, and a taxonomy of CNN deblurring by architecture, loss and application (faces, text, stereo), for both blind and non-blind settings. Takeaway: dynamic-scene deblur-

ring moved from kernel estimation to end-to-end, multi-scale networks trained on captured pairs such as GoPro.

- **Gui et al., "A Comprehensive Survey and Taxonomy on Single Image Dehazing Based on Deep Learning" (ACM Computing Surveys 2023)** [70]. Splits methods into supervised, semi-supervised and unsupervised, and within each by whether and how they use the scattering model; also reviews datasets, network modules, losses and metrics, with experiments on baselines. Takeaway: physics-guided and physics-free designs coexist, and the synthetic-to-real gap is the main open problem.
- **Yang et al., "Single Image Deraining: From Model-Based to Data-Driven and Beyond" (TPAMI 2021)** [71]. Reviews rain appearance models, model-based methods (priors and optimization) and data-driven methods (architectures, constraints, losses, datasets). Takeaway: realistic rain models and real-world evaluation matter more than further architecture tweaks on synthetic data.
- **Jiang et al., "A Survey on All-in-One Image Restoration: Taxonomy, Evaluation and Future Trends" (TPAMI, 2025)** [72]. Organizes all-in-one methods by architectural design, learning paradigm and core innovation (degradation representations, prompts, mixture-of-experts, language guidance), and consolidates datasets and evaluation protocols. Takeaway: comparisons are complicated by inconsistent task sets, and generalization to unseen and mixed degradations remains open.
- **Li et al., "Diffusion Models for Image Restoration and Enhancement: A Comprehensive Survey" (IJCV 2025)** [73]. Categorizes diffusion-based restoration by learning paradigm (supervised versus zero-shot use of a pretrained prior), conditioning strategy, framework design and modeling strategy, with focus on SR, deblurring, inpainting and blind/real-world restoration. Takeaway: diffusion gives the best perceptual quality but at high compute cost and with fidelity/hallucination concerns; distillation to few steps and better evaluation are key directions.

State of the art, 2023–2026

1. **All-in-one and blind restoration became the mainstream setting**, not a niche topic. Methods moved from learned degradation representations (AirNet) to prompts (PromptIR), frequency-aware adaptation (AdaIR) and language guidance (DA-CLIP, InstructIR).
2. **Pretrained diffusion models as generative priors** transformed real-world SR and blind restoration (DiffBIR, SUPIR), and one-step variants (OSEDiff) cut the inference cost from tens or hundreds of steps to one.
3. **State-space models** (MambaIR, MambaIRv2) joined CNNs and transformers as backbone op-

tions with global receptive fields at linear cost.

4. **Evaluation caught up with generation**: challenges separate fidelity from perceptual tracks [3], and real captured datasets (SIDD, O-HAZE, Dense-Haze, Raindrop Clarity) are standard alongside synthetic benchmarks.

Open problems

- **Real-world generalization.** Synthetic pipelines still miss real camera processing, spatially varying blur and mixed weather; models degrade on unseen combinations.
- **Fidelity versus hallucination.** Generative priors can invent text, faces or textures. For medical, forensic and scientific use this is unacceptable, and we lack good metrics for "faithful but sharp".
- **Evaluation.** PSNR/SSIM reward blur; no-reference metrics can be gamed; human studies are costly. Agreement between metrics and human preference for generative restoration is still weak.
- **Efficiency.** High-resolution, real-time (and video) restoration on phones and cameras remains a strong constraint, for transformers and diffusion models especially.
- **Task interference and scaling in all-in-one models**, and how to add a new degradation without retraining.
- **Low-level plus high-level.** Restoration that helps downstream detection or segmentation, rather than human viewers, is a different objective (task-driven evaluation, as in RESIDE [32]).

The author's perspective

The post ends with two observations that still hold. First, because each task is simple to state, the race for state of the art is largely won by special architectures, which means a lot of compute and complicated experiments; but these tasks are very practical and easy to apply, so there is much room in **applications** and in **combining tasks** (exactly where all-in-one restoration went). Second, compared with high-level tasks the learning curve is gentle: a beginner can easily picture what the model is doing (a damaged image goes in, a cleaner one comes out), which makes low-level vision a good door into deep learning research.

Key takeaways

- Low-level vision is about *perceiving* pixels well; almost every task is an instance of $y = \mathcal{D}(x) + n$ with a different $\mathcal{D}$ (blur, downsampling, JPEG, scattering, rain layer, Retinex illumination, mask).

- Deep learning moved the degradation model from the solver into the **data generator**: realistic, randomized synthetic pipelines (and captured pairs) are as important as the network.
- Upsampling in SR evolved from bicubic pre-upsampling (SRCNN) through transposed convolution (FSRCNN) to sub-pixel convolution (ESPCN), which is the standard today.
- A few general backbones dominate: U-Net-style CNNs (NAFNet), transformers (SwinIR, Restormer, HAT) and state-space models (MambaIR); key design habits are global residuals, no batch norm, large receptive fields and gating/attention.
- All-in-one models (AirNet, TransWeather, PromptIR, AdaIR, DA-CLIP, InstructIR) handle unknown degradations by conditioning one network on a learned, prompted or text-given degradation description.
- L2/L1 losses give faithful but smooth results; perceptual, adversarial and diffusion-based methods give realistic but less faithful ones. The **perception–distortion trade-off** says you cannot have both at the limit, so report both kinds of metric.
- Blind deblurring and blind SR must estimate the **kernel** as well as the image. Naive joint MAP prefers the no-blur solution [74]; working methods marginalize over the image, use edge prediction or sharpness-favoring priors, alternate image and kernel updates coarse-to-fine and finish with non-blind deconvolution. Deep methods either predict or condition on the kernel (SRMD, IKC, DAN, KernelGAN, SelfDeblur) or skip it entirely (GoPro-trained networks, BSRGAN, Real-ESRGAN).
- KITTI is a driving benchmark, not a dehazing dataset; for dehazing use RESIDE plus real pairs such as O-HAZE and Dense-Haze.
- No-reference metrics are models of human opinion, validated by SRCC/PLCC against mean opinion scores. NSS metrics (BRISQUE, NIQE, PIQE, PI) measure deviations of MSCN statistics; learned (NIMA, HyperIQA, MUSIQ, MANIQA, TOPIQ) and vision-language (CLIP-IQA, Q-Align) models learn from ratings. All can be gamed by sharpening or noise and suffer from the gap between their training distortions and restoration artifacts, so report several, together with FR metrics and a user study.

Exercises

1. **Degradation order.** Using the `degrade` function, compare (a) blur → downsample → noise → JPEG with (b) noise → blur → downsample → JPEG at the same parameters. Visually and with the noise standard deviation measured on a flat patch, explain why the order matters for what the network has to learn.

   Hint

   In (b), the blur and downsampling filter the noise: its standard deviation drops and it becomes spatially correlated (smooth blotches rather than per-pixel grain). In (a), the noise stays white at the low resolution. A network trained only on (a) will under-remove correlated noise at test time. This is one reason Real-ESRGAN randomizes and repeats the stages.

2. **Pixel shuffle by hand.** A feature tensor has shape $(Cs^2, H, W) = (4, 2, 2)$ with $s = 2, C = 1$, and channel k is filled with the constant k. Write the 4×4 output of pixel shuffle using the formula in the SR section, then check with `torch.nn.PixelShuffle(2)`.

Hint

Output pixel $(2h + i, 2w + j)$ takes channel $2i + j$. So every 2×2 block is $\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$, and the 4×4 output repeats that block four times. Each output block gathers one value from each of the four channels at the same low-resolution position.

3. **Residual or direct?** Modify `TinyDnCNN.forward` to return `self.body(y)` (predict the clean image directly) and train both versions with the same seed and steps. Compare PSNR and the loss curves. Then repeat with $\sigma = 5/255$. When does residual learning help most?

Hint

With few training steps, the residual version usually converges faster and reaches a higher PSNR, because at initialization it already outputs roughly y (good PSNR) and only needs to learn a small correction. At low noise the direct version struggles most, since it must learn a near-identity mapping through several ReLU layers.

4. **Dark channel by numbers.** A hazy pixel has $I = (0.75, 0.78, 0.80)$, the darkest value of I^c/A^c over its patch and channels is 0.70, and $A = (0.9, 0.9, 0.9)$. With $\omega = 0.95$ and $t_0 = 0.1$, compute $\tilde{t}$ and the recovered $\hat{J}$ for this pixel.

Hint

$\tilde{t} = 1 - 0.95 \times 0.70 = 0.335$. Then $\hat{J}^c = (I^c - 0.9)/0.335 + 0.9$, giving about $(0.452, 0.542, 0.601)$. The output is darker and more saturated than the input, as expected when the veil is removed.

5. **Perception versus distortion.** Take the tiny denoiser's output and add a small amount of fresh Gaussian noise ($\sigma = 3/255$) to it. Compute PSNR and SSIM before and after. The image may look *more* natural (less plastic) although both metrics drop. Relate this to the perception–distortion function $P(D)$.

Hint

The MMSE-like output lies at low distortion but far from the natural-image distribution (too smooth). Adding grain moves the output distribution closer to real photos (lower d) at the price of higher MSE: a move along, or toward, the convex bound, not past it.

6. **Design an all-in-one experiment.** You must build a single model for noise ($\sigma \in [0, 50]$), rain streaks and haze. Describe the training data mix, how you would condition the network (no labels at test time), the baselines you would compare against, and how you would detect task interference.

Hint

Mix datasets per task with balanced sampling; add a degradation encoder (contrastive as in AirNet, or prompts as in PromptIR) trained jointly. Compare with three single-task models of the same backbone and size, and with a version without conditioning. Interference shows up as the all-in-one model falling behind the single-task model on one task while matching it on others; also test on mixed degradations (rain + haze) unseen in training.

7. **Kernel estimation by hand.** In the blind-deconvolution snippet, (a) remove the `shock(...)` call and set `keep=1.0` (use all gradients) and compare the kernel error and final PSNR; (b) start from a 3×3 box kernel instead of the near-delta; (c) replace `h_true` by a straight 25-pixel motion path in a 31×31 window. Explain each result with the discussion of Levin et al. [74] and the coarse-to-fine strategy.

Hint

(a) With all gradients, the many weak gradients of the smoothed latent image dominate the least-squares fit, and the estimate no longer improves (in our run the kernel error grew instead of shrinking and the final PSNR stayed at the blurry input's level). Selecting a few strong, well-predicted edges is what makes the kernel step informative, which is the practical answer to the "no-blur" bias of naive joint MAP. (b) The loop is a non-convex alternation, so the start matters; a different start can converge to a different local minimum. (c) A large kernel at full resolution is hard to find from a delta start: in our run the error dropped much less and most of the true-kernel PSNR was not recovered. Estimating the kernel on a downsampled image first (where it is only a few pixels long) and upsampling the result is what the image pyramid is for.

8. **Validating an NR metric.** Five restored images have mean opinion scores MOS $= (4.1, 3.2, 2.5, 1.8, 3.9)$ and NIQE scores $(3.4, 4.2, 5.1, 6.3, 3.0)$. (a) Compute SRCC by hand and interpret its sign. (b) Would SRCC change if you reported $\log(\text{NIQE})$ instead? Would PLCC? (c) Run the MSCN snippet on an over-sharpened version of a *denoised* image and explain why an NSS metric might rate it better than the plain denoised output.

Hint

(a) Ranking from smallest to largest, the MOS ranks of images 1–5 are $(5, 3, 2, 1, 4)$ and the NIQE ranks are $(2, 3, 4, 5, 1)$. The rank differences are $(3, 0, -2, -4, 3)$, so $\sum d_i^2 = 38$ and SRCC $= 1 - 6 \cdot 38/(5 \cdot 24) = -0.9$. The sign is negative because lower NIQE means better quality; one reports $|\text{SRCC}| = 0.9$. Only images 1 and 5 are swapped. (b) SRCC depends only on ranks, so any monotonic transform such as log leaves it unchanged; PLCC changes, which is why PLCC is usually computed after a fitted monotonic mapping. (c) Denoising leaves a smooth image whose MSCN

statistics resemble mild blur (low variance, peaky histogram). Sharpening restores high-frequency energy and pushes the statistics back toward those of natural images, whether or not the added detail is real.

References

1. Shyandram, "Low-Level Vision Task-Image Restoration 簡介," Medium post (in Traditional Chinese), 2023; updated 2026. link
2. Z. Chen, Z. Wu, E. Zamfir, K. Zhang, Y. Zhang, R. Timofte et al., "NTIRE 2024 Challenge on Image Super-Resolution (×4): Methods and Results," in *Proc. IEEE/CVF CVPR Workshops*, 2024. arXiv
3. Z. Chen, K. Liu, J. Gong, J. Wang, L. Sun et al., "NTIRE 2025 Challenge on Image Super-Resolution (×4): Methods and Results," in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv
4. X. Wang, L. Xie, C. Dong and Y. Shan, "Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data," in *Proc. IEEE/CVF ICCV Workshops*, 2021. arXiv
5. E. H. Land and J. J. McCann, "Lightness and Retinex Theory," *Journal of the Optical Society of America*, vol. 61, no. 1, pp. 1–11, 1971. doi
6. C. Dong, C. C. Loy, K. He and X. Tang, "Image Super-Resolution Using Deep Convolutional Networks," *IEEE TPAMI*, 2016 (conference version ECCV 2014). arXiv
7. C. Dong, C. C. Loy and X. Tang, "Accelerating the Super-Resolution Convolutional Neural Network," in *Proc. ECCV*, 2016. arXiv
8. W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert and Z. Wang, "Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network," in *Proc. IEEE CVPR*, 2016. arXiv
9. A. Odena, V. Dumoulin and C. Olah, "Deconvolution and Checkerboard Artifacts," *Distill*, 2016. link
10. B. Lim, S. Son, H. Kim, S. Nah and K. M. Lee, "Enhanced Deep Residual Networks for Single Image Super-Resolution," in *Proc. IEEE CVPR Workshops*, 2017. arXiv
11. Y. Zhang, K. Li, K. Li, L. Wang, B. Zhong and Y. Fu, "Image Super-Resolution Using Very Deep Residual Channel Attention Networks," in *Proc. ECCV*, 2018. arXiv
12. C. Ledig, L. Theis, F. Huszár, J. Caballero, A. Cunningham, A. Acosta, A. Aitken, A. Tejani, J. Totz, Z. Wang and W. Shi, "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *Proc. IEEE CVPR*, 2017. arXiv
13. X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, C. C. Loy, Y. Qiao and X. Tang, "ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks," in *Proc. ECCV Workshops*, 2018. arXiv
14. J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool and R. Timofte, "SwinIR: Image Restoration Using Swin Transformer," in *Proc. IEEE/CVF ICCV Workshops*, 2021. arXiv
15. X. Chen, X. Wang, J. Zhou, Y. Qiao and C. Dong, "Activating More Pixels in Image Super-Resolution Transformer," in *Proc. IEEE/CVF CVPR*, 2023. arXiv
16. X. Lin, J. He, Z. Chen, Z. Lyu, B. Dai, F. Yu, W. Ouyang, Y. Qiao and C. Dong, "DiffBIR: Towards Blind Image Restoration with Generative Diffusion Prior," arXiv:2308.15070, 2023. arXiv

17. F. Yu, J. Gu, Z. Li, J. Hu, X. Kong, X. Wang, J. He, Y. Qiao and C. Dong, "Scaling Up to Excellence: Practicing Model Scaling for Photo-Realistic Image Restoration In the Wild," in *Proc. IEEE/CVF CVPR*, 2024. arXiv
18. R. Wu, L. Sun, Z. Ma and L. Zhang, "One-Step Effective Diffusion Network for Real-World Image Super-Resolution," in *Proc. NeurIPS*, 2024. arXiv
19. E. Agustsson and R. Timofte, "NTIRE 2017 Challenge on Single Image Super-Resolution: Dataset and Study," in *Proc. IEEE CVPR Workshops*, 2017. doi
20. K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising," *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142–3155, 2017. arXiv
21. K. Zhang, W. Zuo and L. Zhang, "FFDNet: Toward a Fast and Flexible Solution for CNN-Based Image Denoising," *IEEE Transactions on Image Processing*, 2018. arXiv
22. A. Abdelhamed, S. Lin and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *Proc. IEEE/CVF CVPR*, 2018. doi
23. S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan and M.-H. Yang, "Restormer: Efficient Transformer for High-Resolution Image Restoration," in *Proc. IEEE/CVF CVPR*, 2022. arXiv
24. L. Chen, X. Chu, X. Zhang and J. Sun, "Simple Baselines for Image Restoration," in *Proc. ECCV*, 2022. arXiv
25. K. He, J. Sun and X. Tang, "Single Image Haze Removal Using Dark Channel Prior," in *Proc. IEEE CVPR*, 2009. doi
26. B. Cai, X. Xu, K. Jia, C. Qing and D. Tao, "DehazeNet: An End-to-End System for Single Image Haze Removal," *IEEE Transactions on Image Processing*, 2016. arXiv
27. B. Li, X. Peng, Z. Wang, J. Xu and D. Feng, "AOD-Net: All-in-One Dehazing Network," in *Proc. IEEE ICCV*, 2017. doi ·arXiv
28. X. Liu, Y. Ma, Z. Shi and J. Chen, "GridDehazeNet: Attention-Based Multi-Scale Network for Image Dehazing," in *Proc. IEEE/CVF ICCV*, 2019. arXiv
29. X. Qin, Z. Wang, Y. Bai, X. Xie and H. Jia, "FFA-Net: Feature Fusion Attention Network for Single Image Dehazing," in *Proc. AAAI*, 2020. arXiv
30. S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan, M.-H. Yang and L. Shao, "Learning Enriched Features for Real Image Restoration and Enhancement," in *Proc. ECCV*, 2020. arXiv
31. Y. Song, Z. He, H. Qian and X. Du, "Vision Transformers for Single Image Dehazing," *IEEE Transactions on Image Processing*, vol. 32, 2023. arXiv
32. B. Li, W. Ren, D. Fu, D. Tao, D. Feng, W. Zeng and Z. Wang, "Benchmarking Single-Image Dehazing and Beyond," *IEEE Transactions on Image Processing*, 2019. arXiv
33. C. O. Ancuti, C. Ancuti, R. Timofte and C. De Vleeschouwer, "O-HAZE: A Dehazing Benchmark with Real Hazy and Haze-Free Outdoor Images," in *Proc. IEEE/CVF CVPR Workshops*, 2018. doi
34. C. O. Ancuti, C. Ancuti, M. Sbert and R. Timofte, "Dense-Haze: A Benchmark for Image Dehazing with Dense-Haze and Haze-Free Images," in *Proc. IEEE ICIP*, 2019. doi
35. A. Geiger, P. Lenz and R. Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite," in *Proc. IEEE CVPR*, 2012. project page
36. Y.-F. Liu, D.-W. Jaw, S.-C. Huang and J.-N. Hwang, "DesnowNet: Context-Aware Deep Network for Snow Removal," *IEEE Transactions on Image Processing*, 2018. arXiv
37. W.-T. Chen, H.-Y. Fang, J.-J. Ding, C.-C. Tsai and S.-Y. Kuo, "JSTASR: Joint Size and

Transparency-Aware Snow Removal Algorithm Based on Modified Partial Convolution and Veiling Effect Removal," in *Proc. ECCV*, 2020. doi
38. W. Yang, R. T. Tan, J. Feng, J. Liu, Z. Guo and S. Yan, "Deep Joint Rain Detection and Removal from a Single Image," in *Proc. IEEE CVPR*, 2017. arXiv
39. R. Qian, R. T. Tan, W. Yang, J. Su and J. Liu, "Attentive Generative Adversarial Network for Raindrop Removal from a Single Image," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
40. X. Li, Y. Jin, X. Jin, Z. Wu, B. Li, Y. Wang et al., "NTIRE 2025 Challenge on Day and Night Raindrop Removal for Dual-Focused Images: Methods and Results," in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv
41. C. Wei, W. Wang, W. Yang and J. Liu, "Deep Retinex Decomposition for Low-Light Enhancement," in *Proc. BMVC*, 2018. arXiv
42. Y. Cai, H. Bian, J. Lin, H. Wang, R. Timofte and Y. Zhang, "Retinexformer: One-stage Retinex-based Transformer for Low-light Image Enhancement," in *Proc. IEEE/CVF ICCV*, 2023. arXiv
43. H. Zhang, H. Xu, X. Tian, J. Jiang and J. Ma, "Image Fusion Meets Deep Learning: A Survey and Perspective," *Information Fusion*, vol. 76, pp. 323–336, 2021. doi
44. S. Nah, T. H. Kim and K. M. Lee, "Deep Multi-scale Convolutional Neural Network for Dynamic Scene Deblurring," in *Proc. IEEE CVPR*, 2017. doi ·arXiv
45. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, "Masked Autoencoders Are Scalable Vision Learners," in *Proc. IEEE/CVF CVPR*, 2022. arXiv
46. R. Suvorov, E. Logacheva, A. Mashikhin, A. Remizova, A. Ashukha, A. Silvestrov, N. Kong, H. Goka, K. Park and V. Lempitsky, "Resolution-robust Large Mask Inpainting with Fourier Convolutions," in *Proc. IEEE/CVF WACV*, 2022. arXiv
47. S. Sengupta, V. Jayaram, B. Curless, S. Seitz and I. Kemelmacher-Shlizerman, "Background Matting: The World is Your Green Screen," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
48. O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Proc. MICCAI*, 2015. arXiv
49. H. Guo, J. Li, T. Dai, Z. Ouyang, X. Ren and S.-T. Xia, "MambaIR: A Simple Baseline for Image Restoration with State-Space Model," in *Proc. ECCV*, 2024. arXiv
50. H. Guo, Y. Guo, Y. Zha, Y. Zhang, W. Li, T. Dai, S.-T. Xia and Y. Li, "MambaIRv2: Attentive State Space Restoration," in *Proc. IEEE/CVF CVPR*, 2025. arXiv
51. B. Li, X. Liu, P. Hu, Z. Wu, J. Lv and X. Peng, "All-In-One Image Restoration for Unknown Corruption," in *Proc. IEEE/CVF CVPR*, 2022. code & paper
52. J. M. J. Valanarasu, R. Yasarla and V. M. Patel, "TransWeather: Transformer-based Restoration of Images Degraded by Adverse Weather Conditions," in *Proc. IEEE/CVF CVPR*, 2022. arXiv
53. V. Potlapalli, S. W. Zamir, S. Khan and F. S. Khan, "PromptIR: Prompting for All-in-One Image Restoration," in *Proc. NeurIPS*, 2023. arXiv
54. Y. Cui, S. W. Zamir, S. Khan, A. Knoll, M. Shah and F. S. Khan, "AdaIR: Adaptive All-in-One Image Restoration via Frequency Mining and Modulation," in *Proc. ICLR*, 2025. arXiv
55. Z. Luo, F. K. Gustafsson, Z. Zhao, J. Sjölund and T. B. Schön, "Controlling Vision-Language Models for Multi-Task Image Restoration," in *Proc. ICLR*, 2024. arXiv
56. M. V. Conde, G. Geigle and R. Timofte, "InstructIR: High-Quality Image Restoration Following Human Instructions," in *Proc. ECCV*, 2024. arXiv
57. H. Zhao, O. Gallo, I. Frosio and J. Kautz, "Loss Functions for Image Restoration with Neural Networks," *IEEE Transactions on Computational Imaging*, 2017 (arXiv title: "Loss Functions for

Neural Networks for Image Processing"). arXiv
58. J. Johnson, A. Alahi and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *Proc. ECCV*, 2016. doi ·arXiv
59. L. Jiang, B. Dai, W. Wu and C. C. Loy, "Focal Frequency Loss for Image Reconstruction and Synthesis," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
60. Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. doi
61. R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang, "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
62. A. Mittal, R. Soundararajan and A. C. Bovik, "Making a 'Completely Blind' Image Quality Analyzer," *IEEE Signal Processing Letters*, 2013. doi
63. Y. Blau, R. Mechrez, R. Timofte, T. Michaeli and L. Zelnik-Manor, "The 2018 PIRM Challenge on Perceptual Image Super-resolution," in *Proc. ECCV Workshops*, 2018. arXiv
64. M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter, "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *Proc. NeurIPS*, 2017. arXiv
65. Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
66. Z. Wang, J. Chen and S. C. H. Hoi, "Deep Learning for Image Super-Resolution: A Survey," *IEEE TPAMI*, 2021. arXiv
67. C. Tian, L. Fei, W. Zheng, Y. Xu, W. Zuo and C.-W. Lin, "Deep Learning on Image Denoising: An Overview," *Neural Networks*, vol. 131, pp. 251–275, 2020. arXiv
68. M. Elad, B. Kawar and G. Vaksman, "Image Denoising: The Deep Learning Revolution and Beyond — A Survey Paper," *SIAM Journal on Imaging Sciences*, vol. 16, no. 3, pp. 1594–1654, 2023. doi ·arXiv
69. K. Zhang, W. Ren, W. Luo, W.-S. Lai, B. Stenger, M.-H. Yang and H. Li, "Deep Image Deblurring: A Survey," *International Journal of Computer Vision*, 2022. arXiv
70. J. Gui, X. Cong, Y. Cao, W. Ren, J. Zhang, J. Zhang, J. Cao and D. Tao, "A Comprehensive Survey and Taxonomy on Single Image Dehazing Based on Deep Learning," *ACM Computing Surveys*, 2023. doi ·arXiv
71. W. Yang, R. T. Tan, S. Wang, Y. Fang and J. Liu, "Single Image Deraining: From Model-Based to Data-Driven and Beyond," *IEEE TPAMI*, 2021. doi ·arXiv
72. J. Jiang, Z. Zuo, G. Wu, K. Jiang and X. Liu, "A Survey on All-in-One Image Restoration: Taxonomy, Evaluation and Future Trends," *IEEE TPAMI*, 2025. arXiv
73. X. Li, Y. Ren, X. Jin, C. Lan, X. Wang, W. Zeng, X. Wang and Z. Chen, "Diffusion Models for Image Restoration and Enhancement: A Comprehensive Survey," *International Journal of Computer Vision*, 2025. arXiv
74. A. Levin, Y. Weiss, F. Durand and W. T. Freeman, "Understanding and Evaluating Blind Deconvolution Algorithms," in *Proc. IEEE CVPR*, 2009, pp. 1964–1971. doi
75. R. Fergus, B. Singh, A. Hertzmann, S. T. Roweis and W. T. Freeman, "Removing Camera Shake from a Single Photograph," *ACM Transactions on Graphics* (Proc. SIGGRAPH), vol. 25, no. 3, pp. 787–794, 2006. doi
76. Q. Shan, J. Jia and A. Agarwala, "High-quality Motion Deblurring from a Single Image," *ACM Transactions on Graphics* (Proc. SIGGRAPH), vol. 27, no. 3, 2008. doi

77. S. Cho and S. Lee, "Fast Motion Deblurring," *ACM Transactions on Graphics* (Proc. SIGGRAPH Asia), vol. 28, no. 5, 2009. doi
78. L. Xu and J. Jia, "Two-Phase Kernel Estimation for Robust Motion Deblurring," in *Proc. ECCV*, 2010, pp. 157–170. doi
79. D. Krishnan, T. Tay and R. Fergus, "Blind Deconvolution Using a Normalized Sparsity Measure," in *Proc. IEEE CVPR*, 2011, pp. 233–240. doi
80. J. Pan, D. Sun, H. Pfister and M.-H. Yang, "Blind Image Deblurring Using Dark Channel Prior," in *Proc. IEEE CVPR*, 2016, pp. 1628–1636. doi
81. J. Sun, W. Cao, Z. Xu and J. Ponce, "Learning a Convolutional Neural Network for Non-uniform Motion Blur Removal," in *Proc. IEEE CVPR*, 2015. arXiv
82. C. J. Schuler, M. Hirsch, S. Harmeling and B. Schölkopf, "Learning to Deblur," *IEEE TPAMI*, vol. 38, no. 7, pp. 1439–1451, 2016. doi ·arXiv
83. A. Chakrabarti, "A Neural Approach to Blind Motion Deblurring," in *Proc. ECCV*, 2016. arXiv
84. D. Ulyanov, A. Vedaldi and V. Lempitsky, "Deep Image Prior," in *Proc. IEEE/CVF CVPR*, 2018, pp. 9446–9454. doi ·arXiv
85. D. Ren, K. Zhang, Q. Wang, Q. Hu and W. Zuo, "Neural Blind Deconvolution Using Deep Priors," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
86. O. Kupyn, V. Budzan, M. Mykhailych, D. Mishkin and J. Matas, "DeblurGAN: Blind Motion Deblurring Using Conditional Adversarial Networks," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
87. K. Zhang, W. Zuo and L. Zhang, "Learning a Single Convolutional Super-Resolution Network for Multiple Degradations," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
88. J. Gu, H. Lu, W. Zuo and C. Dong, "Blind Super-Resolution With Iterative Kernel Correction," in *Proc. IEEE/CVF CVPR*, 2019. arXiv
89. S. Bell-Kligler, A. Shocher and M. Irani, "Blind Super-Resolution Kernel Estimation using an Internal-GAN," in *Proc. NeurIPS*, 2019. link ·arXiv
90. Z. Luo, Y. Huang, S. Li, L. Wang and T. Tan, "Unfolding the Alternating Optimization for Blind Super Resolution," in *Proc. NeurIPS*, 2020. arXiv
91. K. Zhang, L. Van Gool and R. Timofte, "Deep Unfolding Network for Image Super-Resolution," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
92. K. Zhang, J. Liang, L. Van Gool and R. Timofte, "Designing a Practical Degradation Model for Deep Blind Image Super-Resolution," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
93. A. Liu, Y. Liu, J. Gu, Y. Qiao and C. Dong, "Blind Image Super-Resolution: A Survey and Beyond," *IEEE TPAMI*, 2022 (early access). doi ·arXiv
94. A. Mittal, A. K. Moorthy and A. C. Bovik, "No-Reference Image Quality Assessment in the Spatial Domain," *IEEE Transactions on Image Processing*, vol. 21, no. 12, pp. 4695–4708, 2012. doi
95. C. Ma, C.-Y. Yang, X. Yang and M.-H. Yang, "Learning a No-Reference Quality Metric for Single-Image Super-Resolution," *Computer Vision and Image Understanding*, vol. 158, pp. 1–16, 2017. doi ·arXiv
96. N. Venkatanath, D. Praneeth, M. C. Bh, S. S. Channappayya and S. S. Medasani, "Blind Image Quality Evaluation Using Perception Based Features," in *Proc. Twenty First National Conference on Communications* (NCC), 2015, pp. 1–6. doi ·MATLAB documentation
97. H. Talebi and P. Milanfar, "NIMA: Neural Image Assessment," *IEEE Transactions on Image Processing*, vol. 27, no. 8, pp. 3998–4011, 2018. doi ·arXiv

98. S. Su, Q. Yan, Y. Zhu, C. Zhang, X. Ge, J. Sun and Y. Zhang, "Blindly Assess Image Quality in the Wild Guided by a Self-Adaptive Hyper Network," in *Proc. IEEE/CVF CVPR*, 2020, pp. 3664–3673. doi
99. J. Ke, Q. Wang, Y. Wang, P. Milanfar and F. Yang, "MUSIQ: Multi-scale Image Quality Transformer," in *Proc. IEEE/CVF ICCV*, 2021, pp. 5128–5137. doi ·arXiv
100. S. Yang, T. Wu, S. Shi, S. Lao, Y. Gong, M. Cao, J. Wang and Y. Yang, "MANIQA: Multi-dimension Attention Network for No-Reference Image Quality Assessment," in *Proc. IEEE/CVF CVPR Workshops*, 2022, pp. 1190–1199. doi ·arXiv
101. C. Chen, J. Mo, J. Hou, H. Wu, L. Liao, W. Sun, Q. Yan and W. Lin, "TOPIQ: A Top-Down Approach From Semantics to Distortions for Image Quality Assessment," *IEEE Transactions on Image Processing*, vol. 33, pp. 2404–2418, 2024. doi ·arXiv
102. J. Wang, K. C. K. Chan and C. C. Loy, "Exploring CLIP for Assessing the Look and Feel of Images," in *Proc. AAAI Conference on Artificial Intelligence*, 2023. arXiv
103. H. Wu, Z. Zhang, W. Zhang, C. Chen, L. Liao, C. Li, Y. Gao, A. Wang, E. Zhang, W. Sun, Q. Yan, X. Min, G. Zhai and W. Lin, "Q-Align: Teaching LMMs for Visual Scoring via Discrete Text-Defined Levels," in *Proc. ICML*, PMLR 235, pp. 54015–54029, 2024. link ·arXiv
104. H. R. Sheikh, M. F. Sabir and A. C. Bovik, "A Statistical Evaluation of Recent Full Reference Image Quality Assessment Algorithms," *IEEE Transactions on Image Processing*, vol. 15, no. 11, pp. 3440–3451, 2006 (LIVE Image Quality Assessment Database). doi ·database
105. N. Ponomarenko, L. Jin, O. Ieremeiev, V. Lukin, K. Egiazarian, J. Astola, B. Vozel, K. Chehdi, M. Carli, F. Battisti and C.-C. J. Kuo, "Image Database TID2013: Peculiarities, Results and Perspectives," *Signal Processing: Image Communication*, vol. 30, pp. 57–77, 2015. doi ·database
106. H. Lin, V. Hosu and D. Saupe, "KADID-10k: A Large-scale Artificially Distorted IQA Database," in *Proc. Eleventh International Conference on Quality of Multimedia Experience (QoMEX)*, 2019. doi ·database
107. V. Hosu, H. Lin, T. Sziranyi and D. Saupe, "KonIQ-10k: An Ecologically Valid Database for Deep Learning of Blind Image Quality Assessment," *IEEE Transactions on Image Processing*, vol. 29, pp. 4041–4056, 2020. arXiv
108. Y. Fang, H. Zhu, Y. Zeng, K. Ma and Z. Wang, "Perceptual Quality Assessment of Smartphone Photography," in *Proc. IEEE/CVF CVPR*, 2020, pp. 3674–3683. doi
109. J. Gu, H. Cai, H. Chen, X. Ye, J. Ren and C. Dong, "PIPAL: A Large-Scale Image Quality Assessment Dataset for Perceptual Image Restoration," in *Proc. ECCV*, 2020. arXiv
110. C. Chen and J. Mo, "IQA-PyTorch: PyTorch Toolbox for Image Quality Assessment," software (pip package `pyiqa`), 2022. GitHub

DLCV 05 • Low-Light Image Enhancement

From histogram equalization and Retinex to curve estimation, Retinex transformers and diffusion models: how to brighten dark photos without amplifying noise, shifting colors or fooling the metrics.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-05-low-light-enhancement/

What you'll learn

- Why low-light image enhancement (LLIE) is two problems at once, **tone enhancement** and **detail and noise restoration**, and why "just make it brighter" fails.
- How the classical toolbox works: brightness channels in HSV, HSI, YCbCr and Lab; histogram equalization and CLAHE; gamma curves; Retinex (SSR, MSR, LIME); and the dehazing-inversion trick.
- How deep LLIE methods are organized by their **training signal** (supervised, unrolled, unpaired, zero-reference, generative), with the math of the Retinex-Net decomposition losses and the Zero-DCE curve.
- How to evaluate enhancement honestly: full-reference metrics, no-reference metrics (NIQE, BRISQUE, PI, lightness order error, learned assessors) and user studies, the **GT-mean** pitfall, and task-driven evaluation.
- Which datasets and challenges (LOL, LOL-v2, LSRW, SDSD, SID, FiveK, ExDark, NTIRE 2024/2025) are used for what, and what remains unsolved.

The big picture

A photo taken in the dark is not just a normal photo with the brightness turned down. The sensor collected very few photons, so the picture is dim, noisy, and often has a color cast. Low-light image enhancement aims to produce the image you *would* have gotten with good light: well exposed, with natural colors, clean details, and no new artifacts.

> **Explain like I'm five**
>
> Imagine you are drawing in a room with the lights almost off. Your drawing comes out faint and smudgy, and you are not sure which crayon was which color. Turning on the lights later and photocopying the drawing "brighter" won't help: the smudges get brighter too, and the wrong colors stay wrong. A good low-light enhancer is like a friend who looks at your faint drawing and *redraws* it as it would look in daylight: same picture, right colors, without the smudges.

Why it matters: phones shoot in the dark every day, cars and robots must see at night, and surveillance, astronomy-like scientific imaging and medical endoscopy all fight the same physics. Low light also hurts downstream models: a detector trained on daytime photos loses accuracy at night. That makes LLIE both a picture-quality task and a robustness task for computer vision.

About this tutorial

This tutorial is expanded from the author's earlier Medium post on LLIE [1], which is written in Traditional Chinese (read it here). It follows the post's outline and adds math, code, figures and a survey of the literature.

Goal and difficulty

Plain version. We want to map a dark image to a normally exposed one. The hard part is that brightening and repairing are different jobs: brightening changes *tone* (how light or dark each region is), while repairing recovers *details and clean colors* that the darkness and noise destroyed.

Precise version. Unlike image-to-image translation, which changes an image's *attributes* (summer to winter, photo to painting), LLIE should keep the scene and its content exactly as they are and only undo the damage caused by insufficient light. The post [1] splits that damage into two difficulties:

1. **Tone enhancement.** Lift the exposure, globally and locally, without crushing highlights or flattening contrast. Bright regions such as lamps must stay bright, dark regions must open up, and the ordering of brightness between regions should be preserved.
2. **Detail restoration.** Recover texture and color that are buried in noise and quantization, while suppressing that noise rather than amplifying it.

In deep learning, detail restoration is what a network does anyway once it is trained to output a clean image, so most method design goes into the *tone* part (which image properties or physical model to build in) and the *learning mechanism* (which supervision and losses to use), rather than into a dedicated detail module.

Why it is not simple brightening

A linear gain $y = c\,x$ with $c > 1$ fails in four ways, all visible in Figure 1:

- **Noise.** Photon arrival is random. With few photons, the signal-to-noise ratio is low, and a gain multiplies signal and noise equally. The image gets brighter, not cleaner.
- **Color cast.** Low light is often colored (tungsten, sodium street lamps), and the camera's white balance and the three color channels have different noise levels. Scaling the channels equally keeps the cast; scaling them separately may produce strange colors.

- **Saturation and over-exposure.** Pixels already near the top (lamps, windows, screens) clip at 1. A global gain big enough for the shadows destroys the highlights.
- **Quantization.** A dark 8-bit image may use only 20–40 distinct gray levels. Stretching them produces visible banding.

A model of low-light capture

It helps to have an explicit degradation model, both for understanding and for **synthesizing training data**. Work in *linear* light (values proportional to photon counts), not on gamma-encoded pixels (see IP 03 on gamma). A widely used model is the Poisson–Gaussian model of Foi et al. [7]:

$$z = \frac{1}{\eta}\,\mathcal{P}(\eta\, k\, x) + n, \qquad n \sim \mathcal{N}(0, \sigma_r^2),$$

where $x \in [0,1]$ is the clean linear image, $k \in (0,1]$ the fraction of light that reached the sensor, η a scale that converts intensity to photon counts (a larger η means more photons and less relative noise), $\mathcal{P}(\cdot)$ a Poisson random variable modeling **shot noise**, and n Gaussian **read noise** with standard deviation σ_r. The camera then applies a tone curve and quantizes. The variance of z is approximately $kx/\eta + \sigma_r^2$: it grows with the signal, so noise is *signal-dependent*, which is why "add Gaussian noise" is a poor stand-in for real low light.

```
import numpy as np
from skimage import data, img_as_float

rng = np.random.default_rng(0)
clean = img_as_float(data.coffee())            # H x W x 3, sRGB-encoded, in [0, 1]

def synth_low_light(x, k=0.05, peak=2000.0, read_std=8e-4, gamma=2.2):
    lin = k * x ** gamma                              # 1) less light, in linear units
    lin = rng.poisson(lin * peak) / peak              # 2) shot noise (Poisson)
    lin = lin + rng.normal(0, read_std, lin.shape) # 3) read noise (Gaussian)
    out = np.clip(lin, 0, 1) ** (1 / gamma)           # 4) re-encode for display
    return np.round(out * 255) / 255                  # 5) 8-bit quantization

dark = synth_low_light(clean)
print(clean.mean(), dark.mean())                      # about 0.39 -> 0.10
```

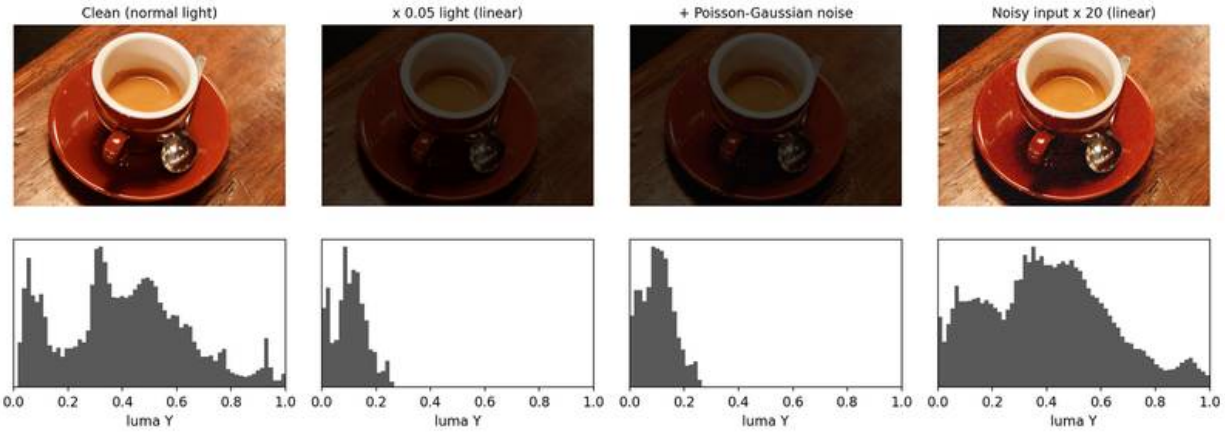


Figure 1 — Synthetic low light. Removing 95% of the light squeezes the histogram into the bottom quarter. Noise is hard to see in the dark image, but undoing the exposure with a perfect gain (right) reveals it: brightening and denoising are separate problems.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

The right panel is the most important picture in this tutorial: even the *ideal* brightness correction leaves the noise, which is why LLIE is a restoration problem (see DLCV 04 ·Image Restoration) and not only a tone-mapping problem.

Non-deep-learning LLIE and the characteristics of low-light images

Plain version. Before deep learning there were two main families: **histogram-based** methods,

which redistribute gray levels, and **Retinex-based** methods, which split an image into "lighting" and "objects" and fix the lighting. A third idea, borrowed from dehazing, fits into the Retinex family, and statistical methods extend all of them.

Low-light images in color space

Many color spaces separate a "how bright" channel from "what color" channels. Different spaces call it value, intensity, luma or lightness, and each defines it differently. With $R, G, B \in [0, 1]$ (see IP 06 for the spaces themselves):

$$\begin{aligned} \text{HSV:} \quad & V = \max(R, G, B), \\ \text{HSI:} \quad & I = \tfrac{1}{3}(R + G + B), \\ \text{YCbCr (BT.601):} \quad & Y' = 0.299\,R' + 0.587\,G' + 0.114\,B', \\ \text{CIELAB:} \quad & L^* = 116\,f(Y_{\text{lin}}/Y_n) - 16. \end{aligned}$$

Here R', G', B' are gamma-encoded values; Y_{lin} is linear-light luminance and Y_n that of the reference white; $f(t) = t^{1/3}$ above a small threshold and linear below it. Some remarks:

- V **is the "bright channel".** It is the per-pixel maximum, the counterpart of the *dark channel* (a per-patch minimum) used in dehazing [14]. In a dark image, V is the channel least damaged by the darkness, which is why LIME [17] uses it to initialize the illumination.
- **The luma weights.** ITU-R BT.601 [8] defines luma as $Y' = 0.299R' + 0.587G' + 0.114B'$. That full-range form is what JPEG/JFIF uses. For 8-bit *limited-range* ("studio") video, BT.601 places black at 16 and white at 235: $Y = 16 + 219\,Y'$. With $R', G', B' \in [0, 255]$ this becomes $Y = 16 + 0.257R' + 0.504G' + 0.098B'$, because $219/255 \times (0.299, 0.587, 0.114) \approx (0.257, 0.504, 0.098)$. Both formulas describe the same luma; they differ only in range and offset.
- L^* is designed to be perceptually uniform, so equal steps look roughly equal to a person. That makes it a good channel for contrast enhancement.

The practical lesson: since brightness can be separated from color, you can **enhance only the brightness channel** and keep the chroma. Figure 2 shows that the choice of channel matters a lot.

```
R, G, B = dark[..., 0], dark[..., 1], dark[..., 2]
V = dark.max(axis=2)                          # HSV value
I = dark.mean(axis=2)                         # HSI intensity
Y = 0.299 * R + 0.587 * G + 0.114 * B         # BT.601 luma, full range (JPEG/JFIF)
Y_limited = 16 + 219 * Y                      # 8-bit limited ("studio") range, 16..235
print(np.allclose(Y_limited, 16 + (65.481 * R + 128.553 * G + 24.966 * B)))  # True
```

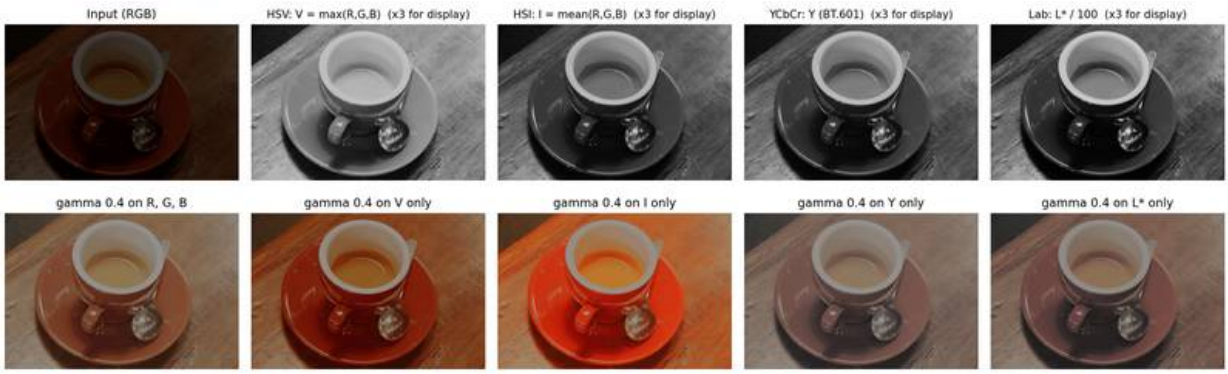


Figure 2 — The same gamma curve applied in different color spaces. Raising V or I scales all three channels by one factor, which keeps hue but strengthens saturation (very orange). Raising Y or L with fixed chroma desaturates the result. Neither is 'correct': tone and color are coupled.*

Two effects are visible. Scaling RGB by a common factor (HSV, HSI) preserves hue and keeps the ratio between channels, so saturated objects become *very* saturated. Changing only Y' or L^* while keeping Cb/Cr or a*/b* fixed adds brightness without adding color, so the image looks washed

out. This coupling of tone and color is exactly what later methods such as HVI-CIDNet [27] try to handle with a learned color space.

Histogram equalization and CLAHE

Plain version. Histogram equalization (HE) spreads the gray levels of an image so that all brightness values are used about equally. A dark image, whose values are packed near zero, gets stretched across the full range.

Precise version. For an image with L levels, normalized histogram $p_r(r_k) = n_k/(MN)$, HE uses the scaled cumulative distribution function as the mapping:

$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_j), \qquad k = 0, \dots, L-1,$$

where n_k is the number of pixels at level r_k and MN the image size. The slope of T is proportional to the histogram, so heavily populated gray levels get pulled apart. The derivation is in IP 03. For color images, the usual recipe is to convert to a space with a brightness channel, equalize that channel only, and convert back, for example RGB → Lab → equalize L^* → RGB.

Global HE has two problems in low light: it over-stretches large flat regions (including their noise), and one mapping cannot serve both a dark corner and a bright lamp. **Adaptive HE** computes a mapping per tile, and **CLAHE** (contrast-limited AHE) clips each tile's histogram at a limit before computing the CDF, which caps the slope of T and therefore the noise gain, then interpolates between neighboring tiles to avoid seams [9], [10].

Gamma correction

A power law $s = r^\gamma$ with $\gamma < 1$ lifts shadows more than highlights; its slope $\gamma r^{\gamma-1}$ is large near $r = 0$. It is cheap and monotonic but global: one γ for the whole picture, no denoising, and the same desaturation-or-oversaturation choice as above depending on where it is applied.

```
from skimage import color, exposure

gamma_out = dark ** 0.4                                    # power-law curve on every
 ↪  channel
lab = color.rgb2lab(dark)
L = lab[..., 0] / 100                                      # L* scaled to [0, 1]
he = lab.copy();    he[..., 0] = 100 * exposure.equalize_hist(L)
clahe = lab.copy(); clahe[..., 0] = 100 * exposure.equalize_adapthist(L, clip_limit=0.01)
he_out, clahe_out = color.lab2rgb(he), color.lab2rgb(clahe)
for name, im in [("gamma", gamma_out), ("HE", he_out), ("CLAHE", clahe_out)]:
    print(f"{name:6s} mean {im.mean():.3f}   std {im.std():.3f}")
```



Figure 3 — Classical enhancers on the noisy input of Figure 1. All of them brighten; none of them removes noise, and the Retinex outputs show strong color noise where the input was darkest.

Retinex theory

Plain version. Retinex says that what the eye (or camera) receives is "the object's own colors" multiplied by "the light falling on it". If we can estimate the light, we can divide it out and see the object as it really is, then put back a better light.

Precise version. Land's Retinex theory [11] models an observed image S as

$$S(\mathbf{x}) = R(\mathbf{x}) \circ L(\mathbf{x}),$$

where $\mathbf{x}$ is the pixel position, $R \in [0,1]$ is the **reflectance** (a property of the surfaces, assumed constant under any lighting), L is the **illumination** (the light falling on the scene, which varies smoothly in space except at shadow boundaries), and $\circ$ is element-wise multiplication. Note the word: L is *illumination*, not luminance. "Luminance" usually refers to a brightness channel of a color space, such as Y above.

Recovering two unknowns from one product is ill-posed, so every Retinex method adds a prior. The classic one is "illumination is smooth".

Single-scale Retinex (SSR) [12] estimates L with a Gaussian blur G_σ and works in the log domain, where the product becomes a sum:

$$\log R_c(\mathbf{x}) = \log S_c(\mathbf{x}) - \log\left(G_\sigma * S_c\right)(\mathbf{x}),$$

for each color channel c, with $*$ convolution and σ the surround scale. Small σ gives strong local

contrast and halos at edges; large σ gives better tone but weaker detail. **Multi-scale Retinex (MSR)** [13] averages SSR outputs at several scales (typically three) with weights w_n:

$$\log R_c^{\text{MSR}} = \sum_{n=1}^{N} w_n \left[\log S_c - \log(G_{\sigma_n} * S_c)\right].$$

Because SSR and MSR work per channel, they tend to gray out the colors. **MSRCR** (MSR with color restoration) [13] multiplies the result by a color-restoration factor based on each channel's share of $R + G + B$; other variants apply MSR to an intensity channel only and keep the input's chromaticity.

```
from scipy import ndimage as ndi

def ssr(img, sigma=40.0, eps=1e-3):
    log_s = np.log(img + eps)
    log_l = np.log(ndi.gaussian_filter(img, sigma=(sigma, sigma, 0)) + eps)  # L ~ Gaussian
     ↪  blur
    r = log_s - log_l                                          # log R = log S - log L
    lo, hi = np.percentile(r, [1, 99])                         # robust stretch for display
    return np.clip((r - lo) / (hi - lo), 0, 1)

msr = np.mean([ssr(dark, s) for s in (15, 80, 250)], axis=0)   # MSR, equal weights
 ↪  (simplified)
```

For *enhancement* we usually do not want R alone, which looks flat and unnatural. We want to brighten L and recombine: $\hat{S} = R \circ \hat{L}$ with, for example, $\hat{L} = L^{\gamma}$ and $\gamma < 1$. This "decompose, adjust L, recombine" pattern is the template for most Retinex-based deep networks.



Figure 4 — A LIME-style Retinex decomposition. The illumination is initialized with $\max(R, G, B)$ and smoothed; the reflectance is S / L. Brightening L with a gamma of 0.35 and multiplying back gives a natural result, while R alone looks flat and over-saturated.

LIME: Retinex as optimization

LIME [17] estimates only the illumination. It initializes it with the bright channel, $\hat{T}(\mathbf{x}) = \max_c S_c(\mathbf{x})$, then refines it by solving a structure-aware smoothing problem: keep T close to $\hat{T}$, but make it smooth except where the image has strong structure. The enhanced image is S/T^{γ}, and because dividing amplifies noise, LIME optionally denoises afterward. LIME's images

(often called the LIME set) became a standard test set, and the method is a good example of the classical recipe: a physical model, a hand-designed prior, and an optimization.

Another model: dehazing an inverted image

The Retinex model is not the only image-formation model. Dehazing uses the **atmospheric scattering model**:

$$I(\mathbf{x}) = J(\mathbf{x})\, t(\mathbf{x}) + A\,(1 - t(\mathbf{x})),$$

where I is the hazy observation, J the haze-free scene radiance, $t \in (0, 1]$ the transmission (how much scene light survives), and A the global atmospheric light. This model cannot be applied to a dark image directly, because a dark image does not look like a hazy one.

Dong et al. observed that an **inverted** low-light image, $1 - S$, *does* look hazy: dark regions become bright and low-contrast, like fog. They therefore invert the image, run a dehazing algorithm (in the spirit of the dark channel prior [14]), and invert the result back [15], [16]. In the notation above,

$$I_{\text{haze}} = 1 - S_{\text{Retinex}}, \qquad J_{\text{haze}} = 1 - \hat{S}_{\text{enhanced}}.$$

From the "what is reliable" angle, dehazing assumes the *dark* channel marks haze-free content; in a low-light image the dark channel is mostly damaged pixels, so attention shifts to the *bright* channel, the region least affected by the darkness. The LIME paper [17] analyzes this inversion approach and shows that it can be rewritten in Retinex form, so the two models make essentially the same assumption, with Retinex being the simpler statement.

Statistical approaches

Many classical methods add statistics or priors on top of these models: naturalness constraints that keep the lightness order of a scene (NPE [18]), layered or two-dimensional histogram models (the method behind the DICM set [59]), and fusion of several synthetic exposures. They are hand-crafted, need no training data, and generalize in a predictable way; their weak point is noise, which they usually ignore or hand to a separate denoiser.

Deep-learning LLIE

Plain version. Deep networks learn the mapping from dark to bright from data. The question is not whether to use a network but **what training signal** to use: paired dark/bright photos, unpaired photos, no reference at all, or a generative model of what bright images look like. Physics (usually Retinex) can be built into any of these.

Deep learning is itself a large optimization, and its black-box nature lets us drop many hand-made assumptions. A common first split is **theory-based** methods (usually Retinex) versus **theory-free** methods (strong generic networks). In practice the two mix, so the split is not binary. Figure 5 groups methods mainly by training signal instead, following the surveys [2], [3].

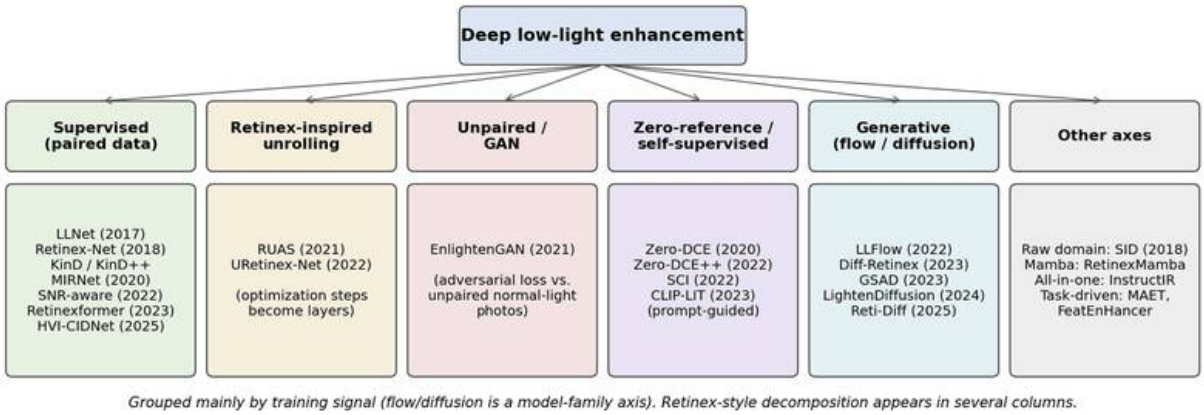


Figure 5 — A taxonomy of deep LLIE, mainly by training signal (the flow/diffusion column is a model-family axis). Retinex decomposition is an architectural choice that appears in several columns, not a column of its own.

Supervised methods (paired data)

Given pairs (x_i, y_i) of low-light and normal-light images, train f_θ to minimize a reconstruction loss, typically $\mathcal{L} = \|f_\theta(x) - y\|_1$ plus perceptual or structural terms. Paired data is expensive (two exposures of a static scene, or synthesis), and the network learns the *specific* brightness and color style of the reference images, which matters for evaluation later.

- **LLNet** [19] (2017) was one of the first deep LLIE models: a stacked sparse denoising autoencoder trained on synthetically darkened and noised patches, learning brightening and denoising jointly.
- **CNN + bright channel prior** (Tao et al. [20], 2017) handled the two difficulties in sequence: a CNN denoises first, then contrast is enhanced with an atmospheric-scattering-style model driven by the bright channel, an early example of combining learning with the inversion idea above.
- **MIRNet** [24] (2020) is a general restoration backbone that keeps a full-resolution stream alongside parallel lower-resolution streams, exchanges information between them, fuses them with selective-kernel attention, and was evaluated on enhancement as well as denoising and super-resolution.
- **SNR-aware** [25] (2022) estimates a per-pixel signal-to-noise map. Where SNR is high, local convolution is enough; where SNR is very low, the local neighborhood is unreliable, so the model

uses long-range transformer attention to borrow information from elsewhere in the image.

- **HVI-CIDNet** [27] (2025) attacks the color-space problem of Figure 2: sRGB is very sensitive to color and gives color bias and brightness artifacts, and HSV fixes brightness but creates red and black noise. The authors propose the HVI space, built from polarized hue/saturation maps and a *learnable* intensity, plus a network that decouples color and intensity.

Retinex-based networks

Retinex-Net [21] (2018) changed the goal from "estimate R" to "**both the input and the output have their own R and L**". The pipeline is: decompose the low and normal images, enhance L to $\hat{L}$, recombine with R. It also introduced the LOL dataset (below). Its decomposition network is trained only through consistency, without ground-truth R or L, using

$$\mathcal{L}_{\text{decom}} = \mathcal{L}_{\text{recon}} + \lambda_{ir}\,\mathcal{L}_{ir} + \lambda_{is}\,\mathcal{L}_{is},$$

$$\mathcal{L}_{\text{recon}} = \sum_{i\in\{\text{low,normal}\}} \sum_{j\in\{\text{low,normal}\}} \lambda_{ij} \left\| R_i \circ I_j - S_j \right\|_1,$$

$$\mathcal{L}_{ir} = \|R_{\text{low}} - R_{\text{normal}}\|_1, \qquad \mathcal{L}_{is} = \sum_i \left\| \nabla I_i \circ \exp\left(-\lambda_g \nabla R_i\right) \right\|.$$

In the paper's notation $S = R \circ I$, so I here is the illumination (L in our notation). S_j is the input image j, R_i and I_i the decomposed reflectance and illumination of image i, and ∇ the horizontal plus vertical gradient. The terms say:

- $\mathcal{L}_{\text{recon}}$: each decomposition must reproduce its own image ($\lambda_{ii} = 1$), and crossing reflectance and illumination between the pair should still roughly work ($\lambda_{ij} = 0.001$ for $i \neq j$).
- $\mathcal{L}_{ir}$: the two images show the same scene, so their reflectances should be equal; this is how the network learns what "reflectance" means.
- $\mathcal{L}_{is}$: illumination should be smooth, *except* where the reflectance has a strong edge. The weight $\exp(-\lambda_g \nabla R)$ relaxes the smoothness penalty across object boundaries. This is the learned version of LIME's structure-aware prior.

The paper uses $\lambda_{ir} = 0.001$, $\lambda_{is} = 0.1$ and $\lambda_g = 10$. The enhancement network is trained with $\|R_{\text{low}} \circ \hat{I} - S_{\text{normal}}\|_1$ plus the same smoothness term.

The pattern spread quickly, and many names contain "Retinex":

- **KinD** [22] (2019) splits the work into three subnetworks: decomposition, *reflectance restoration* (because the dark image's reflectance is noisy and color-shifted, the "R is unchanged" assumption is relaxed), and illumination adjustment with a user-controllable brightness ratio.

  KinD++ [23] (2021) refines the design to reduce visual defects.

- **Retinexformer** [26] (2023) notes that both capture and brightening corrupt the components, and rewrites the model with perturbations: $S = (R + \hat{R}) \circ (L + \hat{L})$, where $\hat{R}$ and $\hat{L}$ model noise, artifacts and color distortion. It is trained in **one stage** (instead of Retinex-Net's multi-stage training) with an L1 loss, and its **illumination-guided multi-head self-attention** uses the illumination estimate to steer attention, so regions with different exposure interact. It became a common baseline.
- **RetinexMamba** [42] (2024 preprint) keeps the Retinex-transformer structure and replaces attention with Mamba state-space blocks, which scale linearly with the number of pixels. "Retinex decomposition plus a new backbone" has been a stable design pattern for years.

A less common route builds on the atmospheric scattering model rather than plain Retinex. CPGA-Net [28] combines channel priors with gamma correction in a lightweight network that processes global and local information, and CPGA-Net+ [29] revisits the theoretical illumination model for efficiency. That line of work also reports that the features learned by different functional modules follow the physical roles of the priors, which suggests that a network guided by a model is not a completely opaque black box.

Retinex-inspired unrolling

Unrolling (or unfolding) takes an iterative optimization algorithm for the Retinex problem and turns each iteration into a network stage with learnable parameters. The architecture inherits the algorithm's structure and interpretability, and training replaces hand-tuned priors.

- **RUAS** [30] (2021) unrolls optimization models for illumination estimation and noise removal, searches the architecture of the prior modules, and is trained with a cooperative *reference-free* strategy, so it also belongs in the unsupervised column. It is very small and fast.
- **URetinex-Net** [31] (2022) unfolds a Retinex decomposition optimization into a network with learned modules for initialization, the unfolded updates of R and L, and illumination adjustment.

Unpaired training: EnlightenGAN

EnlightenGAN [32] (2021) uses a generative adversarial network (GAN) so that it needs only a set of dark images and an *unrelated* set of well-lit images. A U-Net generator, guided by a self-regularized attention map derived from the input's illumination, produces the enhanced image. A **global** discriminator judges the whole image and a **local** discriminator judges random patches, so the output looks like a well-lit photo both overall and in detail. A **self-feature-preserving** loss

compares VGG features of input and output, keeping content in place since there is no ground truth to do so.

Zero-reference methods and curve estimation

Plain version. Zero-DCE does not output a new image. It outputs a *brightening curve per pixel*, applies it, and is trained with losses that judge the result without any reference photo: "is it well exposed, is the color balanced, is the curve smooth, is local contrast kept?"

The LE curve. Zero-DCE [33] (2020) defines a quadratic **light-enhancement curve**

$$\mathrm{LE}(x;\alpha) = x + \alpha\, x\,(1-x), \qquad \alpha \in [-1,1],$$

where $x \in [0,1]$ is a pixel value. The curve has three useful properties: $\mathrm{LE}(0) = 0$ and $\mathrm{LE}(1) = 1$, so the range is preserved and no clipping is needed; it is monotonic for $|\alpha| \le 1$, because $\mathrm{d}\,\mathrm{LE}/\mathrm{d}x = 1 + \alpha(1-2x) \ge 0$, so brightness order is kept; and it is differentiable, so it can be trained by backpropagation. One quadratic is not flexible enough, so it is applied **iteratively**:

$$\mathrm{LE}_n(\mathbf{x}) = \mathrm{LE}_{n-1}(\mathbf{x}) + \mathcal{A}_n(\mathbf{x})\,\mathrm{LE}_{n-1}(\mathbf{x})(1 - \mathrm{LE}_{n-1}(\mathbf{x})),$$

with LE_0 the input, $n = 1, \ldots, 8$, and $\mathcal{A}_n$ a **per-pixel, per-channel** parameter map instead of a single α. A seven-layer CNN (DCE-Net, 32 channels per layer, about 79k parameters, no downsampling) predicts $8 \times 3 = 24$ maps with a tanh output. Each pixel thus gets its own high-order curve, which is how local adjustment arises from a global-looking formula.

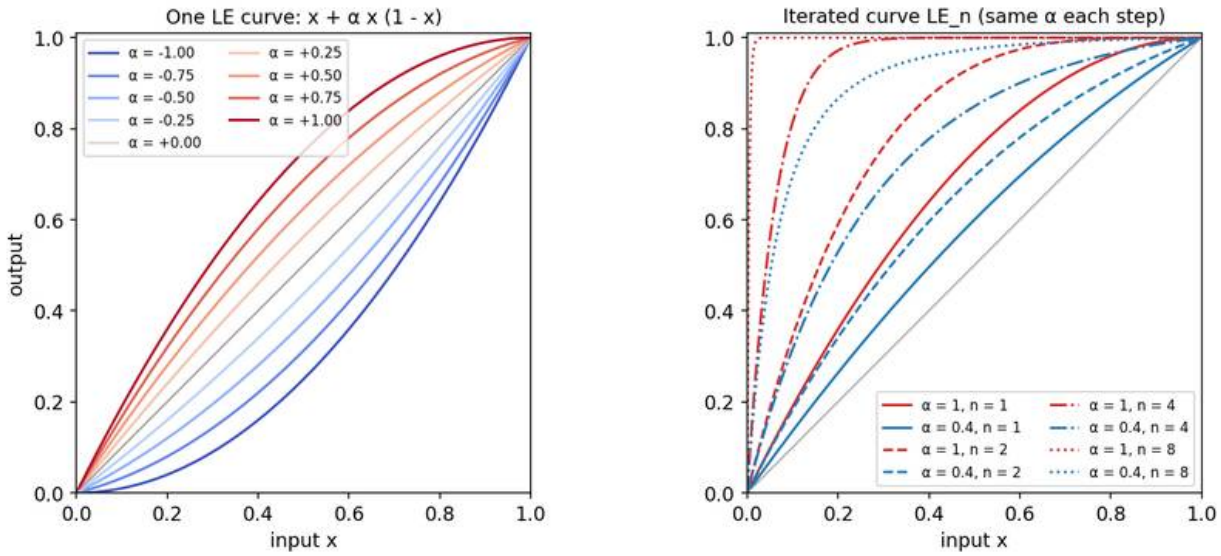


Figure 6 — The LE curve. Left: one iteration for α from −1 to 1. Right: iterating sharpens the curve; with $\alpha = 1$ and $n = 8$, a value of 0.02 is already mapped to almost 1, so tiny noisy values in the shadows are amplified enormously.

```
def le_iterate(x, alphas):
    """Zero-DCE curve: LE_n = LE_{n-1} + a_n * LE_{n-1} * (1 - LE_{n-1})."""
    y = x.copy()
    for a in alphas:              # a can be a scalar or a per-pixel map
        y = y + a * y * (1 - y)
    return y

x = np.array([0.0, 0.02, 0.1, 0.3, 0.6, 1.0])
print(le_iterate(x, [1.0] * 8).round(3))   # [0. 0.994 1. 1. 1. 1.]
print(le_iterate(x, [0.4] * 8).round(3))   # [0. 0.245 0.695 0.93 0.985 1.]
```

The first line shows the danger: for small x, each step multiplies by about $1 + \alpha$, so eight steps with $\alpha = 1$ give a gain near $2^8 = 256$. Whatever noise sits in the shadows is amplified by the same factor.

Non-reference losses. Zero-DCE is trained with four losses that need no ground truth:

$$\mathcal{L}_{spa} = \frac{1}{K}\sum_{i=1}^{K}\sum_{j\in\Omega(i)}\left(|Y_i - Y_j| - |I_i - I_j|\right)^2,$$

$$\mathcal{L}_{exp} = \frac{1}{M}\sum_{k=1}^{M}|Y_k - E|, \qquad \mathcal{L}_{col} = \sum_{(p,q)\in\varepsilon}(J^p - J^q)^2,$$

$$\mathcal{L}_{tv_A} = \frac{1}{N}\sum_{n=1}^{N}\sum_{c\in\{R,G,B\}}\left(|\nabla_x A_n^c| + |\nabla_y A_n^c|\right)^2.$$

In $\mathcal{L}_{spa}$ (**spatial consistency**), K is the number of 4×4 regions, $\Omega(i)$ the four neighbors of region i, and Y and I the mean intensities of a region in the enhanced and input images: contrast between neighboring regions should be preserved. In $\mathcal{L}_{exp}$ (**exposure control**), Y_k is the mean intensity of the k-th of M non-overlapping 16×16 regions and $E = 0.6$ a target well-exposedness level. In $\mathcal{L}_{col}$ (**color constancy**), J^p is the mean of channel p over the image and $\varepsilon = \{(R,G),(R,B),(G,B)\}$: the gray-world assumption that the average scene color is gray. In $\mathcal{L}_{tv_A}$ (**illumination smoothness**), N is the number of iterations and ∇_x, ∇_y horizontal and vertical gradients: neighboring pixels should get similar curves. The total is $\mathcal{L}_{spa} + \mathcal{L}_{exp} + W_{col}\mathcal{L}_{col} + W_{tv_A}\mathcal{L}_{tv_A}$ with $W_{col} = 0.5$ and $W_{tv_A} = 20$ in the paper; training uses multi-exposure images from SICE [56], which provide both under- and over-exposed inputs.

Follow-ups refined the idea:

- **Zero-DCE++** [34] (2022) uses depthwise separable convolutions, predicts only **3** curve maps and reuses them across all iterations, and estimates them on a downsampled input before re-

sizing them back. This cuts the model to about 10k parameters and makes it fast enough to run at interactive rates even on a CPU.

- **SCI** [35] (2022) learns illumination with a cascade of weight-sharing stages and a **self-calibrated** module that makes the stages' outputs converge during training, so that at test time a **single** basic block suffices. It is trained with unsupervised losses.
- **CLIP-LIT** [36] (2023) targets *backlit* images. It uses the vision-language model CLIP and learns a pair of text prompts, one for backlit and one for well-lit images, then refines the prompts and the enhancement network iteratively, so the CLIP similarity to the "well-lit" prompt acts as the training signal.

A zero-reference toy in PyTorch

To see these losses at work, the code below trains a tiny Zero-DCE-style network for about half a minute on CPU. It trains on random crops of four `skimage` images, each darkened by a random factor in linear light, and is tested on the coffee image, which it never sees during training. No ground truth is used. (The loss weights differ from the paper's because the terms are normalized differently: squared rather than absolute exposure error and mean rather than summed gradients.)

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from skimage import data, img_as_float
from skimage.transform import resize

torch.manual_seed(0); rng = np.random.default_rng(0)
darken = lambda x, k: (k * x ** 2.2) ** (1 / 2.2)          # less light, in linear units
train = [resize(img_as_float(f()), (256, 256), anti_aliasing=True)
         for f in (data.astronaut, data.chelsea, data.rocket, data.immunohistochemistry)]

def batch(n=4, size=64):                                    # random crops, random darkness
    xs = []
    for _ in range(n):
        img = train[rng.integers(len(train))]
        i, j = rng.integers(0, 256 - size, 2)
        xs.append(darken(img[i:i + size, j:j + size], rng.uniform(0.02, 0.3)))
    return torch.tensor(np.stack(xs), dtype=torch.float32).permute(0, 3, 1, 2)

class TinyDCE(nn.Module):
    def __init__(self, n_iter=8, w=16):
        super().__init__()
        self.n_iter = n_iter
        self.body = nn.Sequential(
            nn.Conv2d(3, w, 3, padding=1), nn.ReLU(), nn.Conv2d(w, w, 3, padding=1),
            ↪ nn.ReLU(),
```

```
            nn.Conv2d(w, w, 3, padding=1), nn.ReLU(), nn.Conv2d(w, 3 * n_iter, 3,
             ↪  padding=1), nn.Tanh())
    def forward(self, x):
        alphas = self.body(x).chunk(self.n_iter, dim=1)    # 8 maps of shape B x 3 x H x W
        y = x
        for a in alphas:
            y = y + a * y * (1 - y)                         # the LE curve
        return y, torch.cat(alphas, 1)

def L_exp(y, E=0.6, p=16):   # mean brightness of 16x16 patches should be near E
    return (F.avg_pool2d(y.mean(1, keepdim=True), p) - E).pow(2).mean()
def L_col(y):                # gray-world: channel means should agree
    r, g, b = y.mean((2, 3)).unbind(1)
    return ((r - g) ** 2 + (r - b) ** 2 + (g - b) ** 2).mean()
def L_tv(a):                 # curve parameters should vary smoothly in space
    return (a[..., 1:, :] - a[..., :-1, :]).pow(2).mean() + (a[..., :, 1:] - a[..., :,
     ↪  :-1]).pow(2).mean()
def L_spa(y, x, p=4):        # keep the contrast between neighbouring 4x4 regions
    Y, X = (F.avg_pool2d(t.mean(1, keepdim=True), p) for t in (y, x))
    return sum(((torch.diff(Y, dim=d) - torch.diff(X, dim=d)) ** 2).mean() for d in (2, 3))

net = TinyDCE()
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
for step in range(600):                                  # ~20-40 s on a laptop CPU
    x = batch()
    y, a = net(x)
    loss = L_spa(y, x) + 10 * L_exp(y) + 5 * L_col(y) + 200 * L_tv(a)
    opt.zero_grad(); loss.backward(); opt.step()

test = darken(img_as_float(data.coffee()), 0.05)          # an image never used in training
with torch.no_grad():
    out, _ = net(torch.tensor(test, dtype=torch.float32).permute(2, 0, 1)[None])
print(f"mean brightness: {test.mean():.3f} -> {out.mean():.3f}")   # about 0.10 -> 0.50
```

Drag the slider to compare the dark input and the toy's output:



Dark input



Zero-reference toy

The dark coffee-cup image and the toy network's output, which is much brighter, with visible wood grain and a lighter, slightly greenish and less saturated color than the original photo.

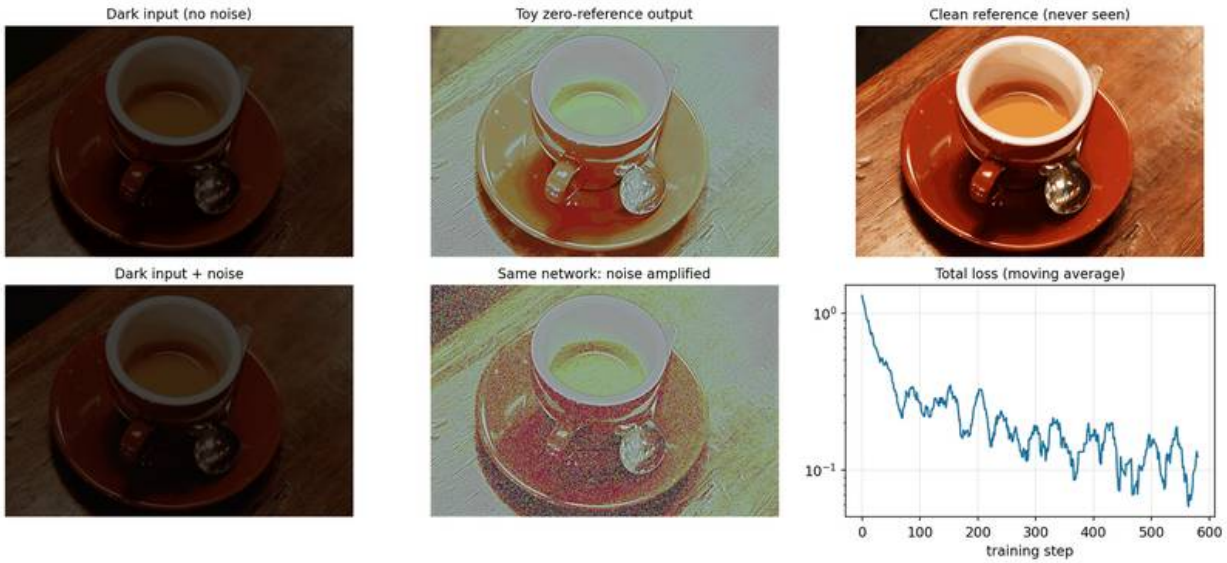


Figure 7 — What the zero-reference losses do and don't do. The exposure loss gives good brightness, but the gray-world color loss pulls a red-brown scene toward gray (top middle versus top right), and with real noise the per-pixel curves amplify it (bottom middle).

The toy shows the strengths and the limits of the zero-reference idea in one picture. Without a single reference image, the network learns a sensible brightening that transfers to an unseen photo. But each loss is a *prior*, and priors can be wrong: gray-world fails for scenes that really are red or brown, a fixed exposure target E flattens deliberately dark regions, and nothing in the losses asks for denoising. Real zero-reference methods are trained on many more images, and some add explicit noise handling, but these failure modes are visible in published results too.

Normalizing flows and diffusion models

LLFlow [37] (2022) observes that one dark image has *many* acceptable bright versions, and a pixel-wise loss averages them into a blurry compromise. It learns a conditional normalizing flow: an invertible network that maps normal-light images to a Gaussian distribution, conditioned on features of the dark input. Those features are guided by priors that echo the classical methods: a histogram-equalized version of the input, a color map (close to the Retinex reflectance) and a noise map.

From 2023, **diffusion models** entered LLIE. They generate the output by iteratively denoising random noise, conditioned on the dark image, and so can synthesize plausible detail that regression networks blur out:

- **Diff-Retinex** [38] (2023) first decomposes the image with a Retinex transformer, then uses conditional diffusion models to generate the restored reflectance and illumination.
- **GSAD** [39] (2023) adds a global structure-aware regularization that uses the image structure to guide a curvature-based constraint on the diffusion trajectory, plus an uncertainty-guided term for hard regions.
- **LightenDiffusion** [40] (2024) performs Retinex decomposition in a latent space and trains with *unpaired* data.
- **Reti-Diff** [41] (2025) runs a latent diffusion model to produce compact Retinex priors that guide a restoration transformer, which reduces the cost of running diffusion at full resolution.

The trade-off is cost and fidelity: many denoising steps make inference slow, and generated details may look right without being *true*. The survey by Adhikarla et al. [6] organizes this sub-field into six families: intrinsic decomposition, spectral and latent, accelerated, guided, multimodal, and autonomous.

Raw-domain low light

Everything above works on processed sRGB images, where the camera's pipeline (demosaicking, white balance, denoising, tone curve, compression) has already discarded information. **Learning to See in the Dark** [44] (2018) instead feeds **raw** sensor data to a U-Net that replaces the whole pipeline. Its SID dataset has 5,094 short-exposure raw images paired with 424 distinct long-exposure references, from a Sony α7S II (Bayer sensor) and a Fujifilm X-T2 (X-Trans sensor), with exposure ratios of ×100, ×250 and ×300. Raw data keeps the linear relation between photons and values and has more bits, so extreme low light that is hopeless in sRGB becomes recoverable. The cost is that the model is tied to one sensor.

All-in-one restoration

Low-light enhancement is also one task inside general restoration models. **InstructIR** [43] (2024) takes a natural-language instruction ("make this photo brighter", "remove the noise") and restores the image accordingly, covering denoising, deblurring, dehazing and low-light enhancement in one network.

Low light for downstream tasks

If the goal is *detection* rather than pretty pictures, the usual baseline is "enhance, then detect" (see DLCV 03 ·Object Detection). But enhancement tuned for people is not necessarily what a detector needs. **MAET** [45] (2021) models the low-light degradation physically and trains the detector to

also predict the degradation parameters, so that features become aware of it. **FeatEnHancer** [46] (2023) enhances hierarchical *features* inside the network, trained directly with the downstream loss, instead of enhancing pixels.



Figure 8 — A timeline of the methods in this tutorial (by publication year). Colors mark Retinex-based or unrolled designs, unpaired or zero-reference training, flow and diffusion models, and challenges.

Evaluation

Plain version. There are two kinds of scores. **Full-reference** metrics compare the output to a ground-truth photo; **no-reference** metrics judge the output alone. A third approach asks whether the enhanced image helps a downstream task. Each answers a different question, and each can be gamed.

Full-reference metrics

With a reference y and output $\hat{y}$ (values in $[0, 1]$), **PSNR** is

$$\mathrm{PSNR} = 10 \log_{10} \frac{1}{\frac{1}{n}\|\hat{y} - y\|_2^2},$$

where n is the number of values. It is a pure pixel error. **SSIM** [47] compares local luminance, contrast and structure statistics, which correlates better with perception. **LPIPS** [48] measures the distance between deep features of the two images; lower is better, and it is sensitive to texture and blur that PSNR ignores.

No-reference metrics: why low light depends on them

Plain version. A no-reference (NR) metric looks at one image and guesses how good it is, with no "correct answer" to compare against. Low-light papers need these metrics because their most realistic test photos have no correct answer, and because even when one exists, it is only one of many acceptable brightness levels.

Precise version. Two facts push LLIE toward NR evaluation:

1. **Real test sets are unpaired.** LIME [17], MEF [58], NPE [18], DICM [59] and VV [60] are small sets of real, often non-uniformly lit photos with no ground truth. Paired sets such as LOL [21] exist, but they were captured under controlled conditions (a static scene, two exposures), and a model can do well on them without generalizing (the generalization gap reported in [2]). Results on the unpaired sets are therefore shown in almost every LLIE paper, and only NR metrics and human judgments can score them.
2. **The "correct" brightness is ambiguous.** Even with a reference, the normal-light image is one photographer's choice of exposure and color. An output that is a little brighter or warmer can look as good, yet lose several dB of PSNR (this is the reason for the GT-mean protocol below). An NR metric at least does not punish a different but valid exposure.

The price is that an NR metric is a *model of human preference*, and every model has blind spots. The general theory of NR image quality assessment (IQA), including how NIQE, BRISQUE, MUSIQ and CLIP-IQA are built, is covered in DLCV 04 ·Image Restoration. The subsections below summarize what each family measures and focus on how it behaves on *enhanced low-light* images.

Classical natural-scene-statistics metrics: NIQE, BRISQUE and PI

Plain version. Natural photos share statistical regularities: after you remove local brightness and contrast, what remains looks like a bell-shaped distribution of a particular shape. Blur, noise and compression change that shape. Natural-scene-statistics (NSS) metrics measure how far an image's statistics are from those of clean photos.

MSCN coefficients. Both NIQE and BRISQUE start from **mean-subtracted contrast-normalized** (MSCN) coefficients of the grayscale image I [50]:

$$\hat{I}(i,j) = \frac{I(i,j) - \mu(i,j)}{\sigma(i,j) + C}, \qquad \mu(i,j) = \sum_{k,l} w_{k,l}\, I(i+k, j+l), \qquad \sigma(i,j) = \sqrt{\sum_{k,l} w_{k,l} \big(I(i+k,j+l) - \mu(i,j)\big)^2},$$

where (i,j) is a pixel, w a small circularly symmetric Gaussian window normalized to sum to 1, μ and σ the local weighted mean and standard deviation, and C a small constant ($C = 1$ for 8-bit images) that avoids division by zero in flat regions. For pristine images the histogram of $\hat{I}$ is close to a unit Gaussian; distortions make it peakier (blur) or heavier-tailed (noise, blocking). The

products of neighboring MSCN coefficients in four directions capture the remaining structure.

BRISQUE [50] fits a generalized Gaussian distribution to $\hat{I}$ and asymmetric generalized Gaussians to the neighbor products at two scales, and feeds the fitted shape and variance parameters to a support-vector regressor trained on human opinion scores for synthetic distortions. Lower is better.

NIQE [49] needs no opinion scores. It selects sharp patches, computes similar NSS features, fits a multivariate Gaussian (MVG) with mean ν_1 and covariance Σ_1 to a corpus of pristine images, fits another MVG (ν_2, Σ_2) to the test image, and reports

$$D(\nu_1, \nu_2, \Sigma_1, \Sigma_2) = \sqrt{(\nu_1 - \nu_2)^\top \left(\frac{\Sigma_1 + \Sigma_2}{2}\right)^{-1} (\nu_1 - \nu_2)},$$

a Mahalanobis-like distance between the two Gaussians. Lower is better: zero means "statistically like the pristine corpus".

PI. The Perceptual Index was introduced for the 2018 PIRM super-resolution challenge [63]:

$$\mathrm{PI} = \tfrac{1}{2}((10 - \mathrm{Ma}) + \mathrm{NIQE}),$$

where Ma is the learned no-reference score of Ma et al. [64] (higher is better, roughly 0 to 10, trained on human ratings of super-resolved images). Lower PI is better. Zero-DCE [33], for example, reports PI on the unpaired sets alongside a user study.

What they reward, and how they mislead on exposure. All three were designed for *structural* distortions (blur, noise, compression, ringing), not for exposure or color:

- MSCN normalization divides out local mean and contrast by construction, so a global brightness change barely moves the features. An under-enhanced image and a well-exposed one can get similar scores.
- Grayscale features ignore color casts and saturation, the very errors of Figure 2.
- Noise and over-sharpening add high-frequency energy, which can make a smooth, noise-amplified output look "more natural" to these statistics than a clean but slightly soft one. HE and CLAHE outputs, with strong local contrast, are a typical case.
- The pristine corpus and training distortions contain few dark, night or backlit scenes, so "natural" means "daylight-like".

Low-light-specific measures: lightness order, colorfulness and noise

Plain version. Some measures target what goes wrong in enhancement specifically: Did the method swap which regions are brighter than others? Are the colors vivid or washed out? How much noise did brightening reveal?

Lightness Order Error (LOE). The NPE paper [18] argues that a natural enhancement should keep the *relative order* of lightness in the scene: if a lamp was brighter than a wall before, it should

still be after, because that order carries the direction of the light and the sense of depth. LOE counts how often the order of two pixels flips. With $L(x) = \max_{c\in\{R,G,B\}} I_c(x)$ the lightness of the input and $L_e(x)$ that of the enhanced image,

$$\mathrm{LOE} = \frac{1}{m}\sum_{x=1}^{m} \mathrm{RD}(x), \qquad \mathrm{RD}(x) = \sum_{y=1}^{m} U(L(x), L(y)) \oplus U(L_e(x), L_e(y)),$$

where m is the number of pixels, $U(p,q) = 1$ if $p \geq q$ and 0 otherwise, and $\oplus$ is exclusive-or, so $\mathrm{RD}(x)$ is the number of pixels whose order relative to x changed. Lower is better. Three practical points:

- The cost is $O(m^2)$, so implementations evaluate LOE on a subsampled image. The raw value grows with m, so numbers are comparable only at the same subsampling. Many papers do not state theirs.
- The reference is the **input**, not a clean image. Any monotonic tone curve applied in the max channel gets LOE = 0, including doing nothing at all. And because the noise in the dark input already scrambles the order of nearby pixels, a method that *removes* noise is penalized.
- LOE says nothing about whether the result is bright enough. It is a guard against unnatural local tone mapping, not a quality score.

Colorfulness. Hasler and Süsstrunk [65] fit a simple colorfulness measure to perceptual ratings, using opponent-color axes on 8-bit RGB:

$$rg = R - G, \qquad yb = \tfrac{1}{2}(R + G) - B, \qquad M = \sqrt{\sigma_{rg}^2 + \sigma_{yb}^2} + 0.3\sqrt{\mu_{rg}^2 + \mu_{yb}^2},$$

where μ and σ are the mean and standard deviation over the image, and the weight 0.3 comes from the authors' fit. Higher means more colorful. It is useful as a descriptive statistic for desaturation (luma-only enhancement, Figure 2) or oversaturation, but it has no notion of the *correct* color, and noise in the chroma channels inflates σ_{rg} and σ_{yb}.

Noise level of the output. Since brightening reveals noise (Figure 1), it helps to report the noise of the output. Immerkær's fast estimator [66] filters the image with a mask that cancels smooth structure and keeps noise,

$$N = \begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}, \qquad \hat{\sigma} = \sqrt{\frac{\pi}{2}}\,\frac{1}{6\,(W-2)(H-2)} \sum_{i,j} |(I * N)(i,j)|,$$

where $W \times H$ is the image size and the sum runs over interior pixels. It assumes white Gaussian noise, so it overestimates on textured images and underestimates spatially correlated noise, which is what real camera pipelines and many networks produce. Use it to compare outputs of the *same* input, not across scenes.

The snippet below recreates the dark coffee image of Figure 1 and compares five "enhancements" plus the clean photo.

```
import numpy as np
from scipy import ndimage as ndi
from skimage import color, data, exposure, img_as_float

rng = np.random.default_rng(0)
clean = img_as_float(data.coffee())

def synth_low_light(x, k=0.05, peak=2000.0, read_std=8e-4, gamma=2.2):   # as in Figure 1
    lin = k * x ** gamma
    lin = rng.poisson(lin * peak) / peak
    lin = lin + rng.normal(0, read_std, lin.shape)
    return np.round(np.clip(lin, 0, 1) ** (1 / gamma) * 255) / 255

def loe(ref, out, step=8):
    """Lightness Order Error on a strided subsample (m = 50 x 75 = 3750 pixels here)."""
    L, Le = (im.max(axis=2)[::step, ::step].ravel() for im in (ref, out))
    U, Ue = L[:, None] >= L[None, :], Le[:, None] >= Le[None, :]
    return (U ^ Ue).sum(axis=1).mean()                  # mean over x of RD(x)

def colorfulness(im):
    """Hasler-Suesstrunk colorfulness, RGB scaled to 0..255."""
    R, G, B = (255 * im[..., c] for c in range(3))
    rg, yb = R - G, 0.5 * (R + G) - B
    return np.hypot(rg.std(), yb.std()) + 0.3 * np.hypot(rg.mean(), yb.mean())

def noise_sigma(im):
    """Immerkaer's noise estimate on BT.601 luma, in 8-bit units."""
    Y = 255 * (0.299 * im[..., 0] + 0.587 * im[..., 1] + 0.114 * im[..., 2])
    N = np.array([[1, -2, 1], [-2, 4, -2], [1, -2, 1]])
    H, W = Y.shape
    return np.sqrt(np.pi / 2) / (6 * (W - 2) * (H - 2)) * np.abs(ndi.convolve(Y, N)[1:-1,
     ↪  1:-1]).sum()

dark = synth_low_light(clean)
lab = color.rgb2lab(dark); L = lab[..., 0] / 100
he = lab.copy(); he[..., 0] = 100 * exposure.equalize_hist(L)
cl = lab.copy(); cl[..., 0] = 100 * exposure.equalize_adapthist(L, clip_limit=0.01)
outputs = {
    "dark input": dark,
    "gain x20 (linear)": np.clip(20 * dark ** 2.2, 0, 1) ** (1 / 2.2),
    "gamma 0.4": dark ** 0.4,
    "HE on L*": np.clip(color.lab2rgb(he), 0, 1),
    "CLAHE on L*": np.clip(color.lab2rgb(cl), 0, 1),
    "clean reference": clean,
}
for name, im in outputs.items():
    print(f"{name:18s} mean {im.mean():.2f}  LOE {loe(dark, im):6.1f}  "
          f"colorfulness {colorfulness(im):5.1f}  noise {noise_sigma(im):5.2f}")
# dark input         mean 0.10  LOE    0.0  colorfulness  23.1  noise  3.32
# gain x20 (linear)  mean 0.39  LOE    0.3  colorfulness  90.0  noise 12.93
# gamma 0.4          mean 0.34  LOE    0.0  colorfulness  61.6  noise 11.19
# HE on L*           mean 0.48  LOE  334.7  colorfulness  28.3  noise 16.17
# CLAHE on L*        mean 0.40  LOE  657.6  colorfulness  27.1  noise 23.06
```

```
# clean reference    mean 0.39  LOE  322.0  colorfulness  76.9  noise  4.69
```



Figure 9 — Brighter is not automatically better. HE is the brightest output but the most order-breaking after CLAHE and nearly colorless; the noisy linear gain is more 'colorful' than the clean photo; and the clean photo itself has a high LOE, because LOE is measured against the noisy input.

Read the numbers together, not one at a time:

- **Brightness alone ranks the wrong way.** HE has the highest mean (0.48) but also a large LOE, the second-highest noise and almost no color left. CLAHE's local remapping breaks lightness order the most.
- **Colorfulness is fooled by noise.** The linear gain scores 90, *higher* than the clean photo's 77, partly because amplified chroma noise raises σ_{rg} and σ_{yb}.
- **LOE penalizes restoration.** The clean photo, the ideal answer, gets LOE 322 against the noisy input, while doing nothing at all gets 0. Low LOE is necessary for a natural tone curve, not sufficient for quality.
- **Only the noise estimate separates the clean photo** (4.7) from the bright but noisy outputs (11 to 23). No single NR number captures "well exposed, natural color, and clean".

Learned no-reference assessors

Plain version. Instead of hand-made statistics, a neural network is trained to predict human ratings, or a large vision-language model is asked how good a photo looks. These models capture more of what people care about, including content and aesthetics, but they inherit the biases of their training data.

The main families, all of which can be applied to LLIE outputs (and which share common implementations, e.g. the IQA-PyTorch toolbox [75]):

- **NIMA** [67] (2018) predicts the *distribution* of human ratings over the score bins with a CNN, rather than a single number. The reported score is the mean of that distribution, $\hat{s} = \sum_k k\, p_k$, where p_k is the predicted probability of score k. Versions trained on aesthetic ratings measure "is this a nice photo", which is not the same as "is this a faithful enhancement".
- **MUSIQ** [68] (2021) is a multi-scale transformer that takes the image at its native resolution and aspect ratio, plus downscaled versions, using hash-based 2D position and scale embeddings. Avoiding resizing matters for LLIE, where noise and fine texture are exactly what resizing would hide. Higher is better.
- **CLIP-IQA** [69] (2023) uses the vision-language model CLIP with an antonym prompt pair, "Good photo." and "Bad photo.". With $\mathbf{x}$ the image embedding and $\mathbf{t}_1, \mathbf{t}_2$ the two text embeddings, $s_i = \mathbf{x}^\top \mathbf{t}_i / (\|\mathbf{x}\|\,\|\mathbf{t}_i\|)$ and the score is $e^{s_1}/(e^{s_1} + e^{s_2})$, in $[0, 1]$, higher is better. The basic version is zero-shot: it needs no opinion scores. Other prompt pairs ("Bright photo." / "Dark photo.", "Noisy photo." / "Clean photo.") probe single attributes, which is convenient for exposure, but the exact wording changes the score. The same CLIP-similarity idea is used as a *training* signal by CLIP-LIT [36], so a CLIP-based metric should not be the only judge of a CLIP-trained enhancer.
- **MANIQA** [70] (2022) combines a vision transformer with transposed (channel) attention and Swin-transformer blocks and a dual-branch patch-weighted score. It won the NTIRE 2022 perceptual IQA challenge, whose images include outputs of restoration and GAN-based algorithms, so it is closer to the "algorithm output" setting than metrics trained on camera distortions.
- **Q-Align** [71] (2024) fine-tunes a large multimodal model to answer with a *text rating level* (bad, poor, fair, good, excellent), mimicking how human raters work. With X_{l_i} the model's logit for level l_i, the score is $S = \sum_{i=1}^{5} i \cdot e^{X_{l_i}} / \sum_{j=1}^{5} e^{X_{l_j}}$, the expected level under a softmax restricted to the five words.
- **LLIE-specific assessors.** Kannan et al. [72] (2023) collected human opinion scores for images restored by different LLIE methods, benchmarked existing NR-IQA models on them, and proposed an *opinion-unaware* model that learns distortion-aware features of restored low-light images by self-supervised contrastive learning. Datasets of this kind are the right test bed for deciding which metric to trust on enhancement outputs.

Caveats for low light. Learned assessors are trained mostly on daylight photos. They tend to rate brighter, sharper and more saturated images higher, which favors over-enhancement, and a deliberately dark night scene can be penalized even when it is the faithful result. Their scores depend on the checkpoint (which training set), the input size and the implementation, so report those. And when a method is tuned with a learned metric or the same backbone in its loss, that metric is no longer an independent judge.

User studies and paired comparisons

Plain version. In the end, people decide what looks good. The two standard ways to ask them are to rate images on a scale or to choose the better of two.

Mean opinion score (MOS). Each of N raters gives an image a score s_n on a fixed scale (often 1 to 5), and $\text{MOS} = \frac{1}{N}\sum_{n=1}^{N} s_n$. ITU-R BT.500 [73] standardizes such tests: viewing conditions, rating scales, single- and double-stimulus and comparison methods, and screening of unreliable raters. Zero-DCE [33], for example, had 15 subjects rate results on a 1–5 scale over the 202 images of NPE, LIME, MEF, DICM and VV, with instructions to look for over- or under-exposure, color deviation, and unnatural texture or noise.

Paired comparison. Absolute scales drift between raters; choosing between two images is easier and more consistent. The **Bradley–Terry** model [74] turns many such choices into a score per method: each method i has a strength $\pi_i > 0$ and

$$P(i \text{ preferred over } j) = \frac{\pi_i}{\pi_i + \pi_j},$$

with the π_i fitted by maximum likelihood to the observed wins. EnlightenGAN [32] used this protocol: 9 subjects compared pairs of outputs from five methods on 23 images, judging visible noise, exposure artifacts and unrealistic color or texture, and a Bradley–Terry fit gave the ranking.

Good practice: hide method names, randomize order and left/right position, show the input for context, use enough images *and* raters, report confidence intervals or significance, and state the display and viewing conditions. A study with a handful of raters and hand-picked images is a demonstration, not evidence.

Choosing and reporting no-reference metrics

Metric	Type	Needs training data?	Better	Sensitive to	Caveat for low light
NIQE [49]	NSS	Pristine images only (no opinion scores)	Lower	Blur, noise, compression in grayscale structure	Nearly blind to global exposure and color; can prefer noisy, high-contrast outputs
BRISQUE [50]	NSS + regression	Yes, opinion scores on synthetic distortions	Lower	Same features as NIQE, mapped to human scores	Trained on daylight distortions; ignores color cast
PI [63], [64]	NSS + learned	Ma score trained on super-resolution ratings	Lower	Texture and sharpness	Designed for super-resolution; rewards detail, including amplified noise
LOE [18]	Task-specific	No	Lower	Flips in lightness order relative to the input	Depends on subsampling; 0 for identity; penalizes denoising
Colorfulness [65]	Task-specific (descriptive)	No (fitted constants)	Context-dependent	Spread and offset of opponent colors	No notion of correct color; inflated by chroma noise
Noise estimate [66]	Task-specific (descriptive)	No	Lower	High-frequency residual	Assumes white noise; confused by texture and correlated noise
NIMA [67]	Learned	Yes, rating distributions	Higher	Aesthetics or technical quality, depending on checkpoint	Aesthetic models reward attractive, not faithful, results
MUSIQ [68]	Learned	Yes, opinion scores	Higher	Multi-scale quality at native resolution	Trained mostly on daylight photos; checkpoint-dependent
CLIP-IQA [69]	Learned (vision-language)	No for zero-shot	Higher	Semantic "good photo" impression; prompt-dependent	Prompt wording matters; not independent of CLIP-trained enhancers
MANIQA [70]	Learned	Yes, opinion scores	Higher	Restoration and GAN artifacts	Not trained on exposure errors specifically
Q-Align [71]	Learned (large multimodal model)	Yes, opinion scores as text levels	Higher	Overall human-like quality judgment	Heavy; biases of the base model; version-dependent

Metric	Type	Needs training data?	Better	Sensitive to	Caveat for low light
MOS / paired comparison [73], [74]	Human	Human raters	Higher MOS / higher strength	What people actually prefer	Costly; small studies are noisy; protocol must be reported

Practical recommendations:

1. **On paired sets**, lead with full-reference metrics (PSNR, SSIM, LPIPS) under the standard protocol, and add NR scores only as a complement.
2. **On unpaired sets**, report *several* NR metrics from different families (one NSS, one or two learned, LOE), state the implementation, checkpoint and image size, and show many visual examples, including failures.
3. **Expect disagreement.** NSS and learned metrics often rank methods differently, and both can disagree with people; the survey of Li et al. [2] notes the frequent mismatch between metric scores and visual quality, and Liu and Fan [5] find a further gap between image quality and downstream task performance. When metrics disagree, say so instead of picking the favorable one.
4. **Do not optimize the judge.** If a method's loss uses a metric or the same backbone (for example CLIP), evaluate it with something else.
5. **Back claims with people.** For a new method, a properly designed user study or paired comparison is the strongest NR evidence.

The GT-mean pitfall

Plain version. On paired benchmarks, a small difference in *overall brightness* changes PSNR a lot. Some papers rescale each output so its average brightness matches the ground truth before scoring. That is information you never have in practice.

Precise version. "GT-mean" evaluation scales the output by $\lambda = \mathbb{E}[y]/\mathbb{E}[f(x)]$ and computes the metric on $\lambda f(x)$, where $\mathbb{E}[\cdot]$ is the image mean. Because LLIE is ambiguous about the "right" brightness, this can raise PSNR substantially. The Retinexformer authors provide this mode in their code for comparability with methods such as LLFlow and KinD and recent diffusion models, while explicitly saying they do *not* recommend it, because it uses the ground truth to correct the output [26]. Liao et al. [51] (2025) study the brightness mismatch between outputs and references and propose a GT-mean *loss* for training, reporting both standard and GT-mean metrics. When reading tables: check which setting is used, never mix the two, and prefer methods that are good under the standard protocol.

Task-driven evaluation

Since LLIE is often a preprocessing step, another test is to run a detector or segmenter on the enhanced images (on ExDark [57], for example) and report its accuracy. Liu and Fan's 2025 review [5] finds a disconnect: supervised methods often produce images with high perceptual quality but only modest gains for vision tasks, while zero-shot methods, despite lower image-quality scores, give more consistent gains across tasks. The benchmark of Liu et al. [4] also evaluates face detection in low light. A complete evaluation reports fidelity, no-reference quality, task performance, and runtime.

Benchmark datasets

Plain version. Datasets differ in whether they have pairs, whether the darkness is real or synthetic, whether they are raw or sRGB, and whether they carry labels for detection.

Dataset	Type	Content and what it is for
LOL (v1) [21]	Paired, real, sRGB	500 low/normal pairs captured by changing exposure (485 train / 15 test), 400×600. The most common LLIE benchmark; small test set.
LOL-v2 [52]	Paired, real + synthetic subsets	A larger *Real* subset of captured pairs and a *Synthetic* subset of synthesized pairs. Report the two subsets separately.
LSRW [53]	Paired, real	5,650 pairs from a Nikon D7500 (3,170) and a Huawei P40 Pro (2,480); 5,600 train, 50 test. Tests generalization across devices.
SDSD [54]	Paired video	Dynamic indoor and outdoor scenes captured twice with a mechatronic system for alignment; often split into frames for image tests.
SID [44]	Paired, raw	5,094 short-exposure raw images with 424 long-exposure references (Sony and Fuji). Extreme low light, raw-to-sRGB.
MIT-Adobe FiveK [55]	Paired, retouching	5,000 raw photos, each retouched by five experts. Used for tone and color enhancement (photo retouching).
SICE [56]	Multi-exposure sequences	Under- to over-exposed captures of each scene with a reference; used to train exposure correction (e.g. Zero-DCE).
VE-LOL [4]	Paired + detection	VE-LOL-L for enhancement and VE-LOL-H with face annotations for high-level evaluation.
ExDark [57]	Unpaired, labeled	7,363 low-light images, 12 object classes, 10 lighting conditions. Task-driven evaluation (detection).

Dataset	Type	Content and what it is for
LIME [17], MEF [58], NPE [18], DICM [59], VV [60]	Unpaired test sets	Small collections of real low-light or non-uniformly lit photos for visual comparison and no-reference metrics.

Challenges: NTIRE 2024 and 2025

The NTIRE workshop at CVPR ran LLIE challenges in consecutive years. The **NTIRE 2024** challenge [61] covered ultra-high-resolution (4K and beyond), non-uniform illumination, backlighting, extreme darkness and night scenes; it had 428 registered participants and 22 teams with valid submissions. The **NTIRE 2025** challenge [62] had 762 registered participants and 28 teams with valid entries. The reports summarize the winning designs and are a good snapshot of what works under a fixed, fair protocol.

Modern view

The field has moved from hand-made priors to learned models in under a decade, but the core questions in the post [1] are the same: how to model the light, how to handle noise, and how to know whether an output is good. The surveys below are the best entry points.

Li et al., TPAMI 2022 [2]. The most cited deep-LLIE survey. It organizes methods by learning strategy (supervised, reinforcement, unsupervised, zero-shot and semi-supervised learning), network structure, use of the Retinex model, data format (sRGB or raw), loss functions, and training and test data. It also introduces LLIV-Phone, an image and video dataset captured with many different phone cameras under diverse illumination, and an online platform for comparing methods. Its main lessons are a generalization gap (many methods struggle on the phone data) and a frequent mismatch between metric scores and visual quality.

Zheng et al., "Low-Light Image and Video Enhancement: A Comprehensive Survey and Beyond" [3]. It reviews traditional and deep methods with a similar taxonomy and goes further on data: it proposes SICE_Grad and SICE_Mix, which mix under- and over-exposure inside one image, and Night Wenzhou, an aerial and street night video set, to test methods on cases that existing benchmarks miss. It was the source of the post's own overview figures.

Liu et al., IJCV 2021 [4]. "Benchmarking low-light image enhancement and beyond" is a benchmark more than a survey: the VE-LOL dataset with paired data and face annotations, a comparison using full-reference, no-reference and semantic metrics, and a joint enhancement-plus-detection model. It made task-driven evaluation mainstream.

Liu and Fan, Neurocomputing 2025 [5]. This review asks which enhancement actually helps downstream vision and finds the disconnect described above between image-quality metrics and task performance, with zero-shot methods giving the most consistent task gains.

Adhikarla et al., 2025 [6]. A survey of diffusion models for LLIE with a six-family taxonomy and a performance analysis. Its message is that diffusion brings realism and flexibility but at a high inference cost, so acceleration, guidance and latent-space designs are the active topics.

NTIRE reports [61], [62]. These are not surveys but show the current practice of top teams under a common protocol, with high-resolution and difficult illumination.

State of the art (2023–2026). Three directions dominate. First, **Retinex-guided backbones**: transformers (Retinexformer [26]) and state-space models (RetinexMamba [42]) that keep a light-versus-reflectance structure. Second, **generative models**: diffusion (Diff-Retinex [38], GSAD [39], LightenDiffusion [40], Reti-Diff [41]) and flows (LLFlow [37]). Third, **better representations and protocols**: learned color spaces (HVI-CIDNet [27]), brightness-mismatch-aware training and evaluation (GT-Mean loss [51]), language-guided and all-in-one restoration (CLIP-LIT [36], InstructIR [43]), and task-aware enhancement (MAET [45], FeatEnHancer [46]).

Open problems.

- **Real noise.** Most sRGB methods train on data whose noise does not match real sensors after a camera pipeline. Better noise models [7], raw-domain training [44], or joint enhancement and denoising remain central.
- **Video consistency.** Frame-by-frame enhancement flickers. Paired video data (SDSD [54]) is scarce, and temporal consistency is still hard, especially with generative models.
- **Evaluation.** The ill-posed brightness target, GT-mean protocols, unreliable no-reference metrics for exposure, and the gap between perceptual and task metrics make comparisons fragile.
- **Efficiency on edge devices.** Phones and cars need real-time, low-power models. Curve methods (Zero-DCE++ [34], SCI [35]) and lightweight prior-based networks [28] show that tiny models can work, while diffusion models remain far too slow for this use.
- **Generalization.** Models trained on one dataset or camera often fail on another (the generalization gap reported in [2]); mixed over- and under-exposure, backlight and colored artificial light are still difficult.

Key takeaways

- LLIE is **tone enhancement plus restoration**: a perfect brightness correction still leaves noise, color cast and quantization (Figure 1).
- Brightness can be enhanced in a separate channel (V, I, Y', L^*), but the choice couples to saturation. BT.601 luma is $0.299R' + 0.587G' + 0.114B'$; $16 + 0.257R' + 0.504G' + 0.098B'$ is the same

luma in 8-bit limited range.

- Retinex models an image as reflectance times **illumination**, $S = R \circ L$; SSR/MSR/LIME estimate L with smoothness priors, and the inverted-image dehazing approach rests on essentially the same assumption.
- Deep methods are best organized by **training signal**: supervised, unrolled, unpaired (EnlightenGAN), zero-reference (Zero-DCE, SCI, CLIP-LIT) and generative (LLFlow, diffusion). Retinex decomposition appears across them.
- Zero-DCE's LE curve $x + \alpha x(1-x)$ preserves range and order; its non-reference losses are priors (exposure, gray world, smoothness, contrast) that can fail on real noise and colorful scenes.
- Report full-reference, no-reference and task-driven results, **never mix GT-mean and standard numbers**, and remember that higher PSNR does not guarantee better detection.
- No-reference metrics each see one aspect: NIQE/BRISQUE/PI see structure but are nearly blind to exposure and color, LOE sees lightness order but not brightness or noise, colorfulness is inflated by chroma noise, and learned assessors inherit daylight biases. **Brighter is not automatically better** (Figure 9): report several NR metrics from different families, plus a well-designed user study.

Exercises

1. **Monotonic curves.** Show that $\mathrm{LE}(x;\alpha) = x + \alpha x(1-x)$ is monotonically non-decreasing on $[0, 1]$ if and only if $|\alpha| \leq 1$. What goes wrong for $\alpha = 1.5$?

   Hint

   The derivative is $1 + \alpha(1-2x)$, which is linear in x, so its minimum on $[0,1]$ is at an endpoint: $1+\alpha$ at $x = 0$ or $1-\alpha$ at $x = 1$. Both are ≥ 0 exactly when $|\alpha| \leq 1$. For $\alpha = 1.5$ the slope at $x = 1$ is -0.5, so near-white pixels would get darker as the input gets brighter, reversing the brightness order and creating artifacts in highlights.

2. **Gain of the iterated curve.** For a small input x, show that eight iterations with a constant α multiply x by approximately $(1+\alpha)^8$. Use this to explain why zero-reference curve methods amplify shadow noise, and suggest one change to the losses or architecture that would reduce the problem.

   Hint

   For small x, $x(1-x) \approx x$, so each step gives $x \mapsto (1+\alpha)x$. With $\alpha = 1$ that is $2^8 = 256$. Noise with standard deviation σ becomes 256σ. Possible fixes: add a denoising branch or a noise-aware loss, limit α where the estimated SNR is low (in the spirit of SNR-aware), or penalize high-frequency energy in the output that is not present in a smoothed input.

3. **Two luma formulas.** A pixel has $(R', G', B') = (200, 100, 50)$ in 8 bits. Compute its full-range BT.601 luma and its limited-range luma. Show that they encode the same quantity.

Hint

Full range: $0.299 \cdot 200 + 0.587 \cdot 100 + 0.114 \cdot 50 = 124.2$. Limited range: $16 + 0.257 \cdot 200 + 0.504 \cdot 100 + 0.098 \cdot 50 \approx 122.7$. Converting back: $(122.7 - 16) \cdot 255/219 \approx 124.2$. The only differences are the scale $219/255$ and the offset 16.

4. **Retinex losses without ground truth.** In Retinex-Net, why is $\mathcal{L}_{ir} = \|R_{\text{low}} - R_{\text{normal}}\|_1$ needed? Describe a trivial decomposition that zeroes the self-reconstruction terms of $\mathcal{L}_{\text{recon}}$ and $\mathcal{L}_{is}$ but is useless, and explain how $\mathcal{L}_{ir}$ rules it out.

Hint

Set $I_i \equiv 1$ (perfectly smooth) and $R_i = S_i$. Then each image is reconstructed exactly and the smoothness loss is zero, but nothing has been decomposed. Since $S_{\text{low}} \neq S_{\text{normal}}$, this gives $R_{\text{low}} \neq R_{\text{normal}}$, so $\mathcal{L}_{ir}$ is large. Requiring equal reflectances forces the brightness difference between the pair into the illumination.

5. **GT-mean by hand.** Your model outputs an image with mean 0.40; the ground truth has mean 0.50, and otherwise the output is perfect up to that global scale. What is the GT-mean scale factor, and what is the PSNR before and after rescaling if the image is a constant? Why is the rescaled number not a fair comparison with a method evaluated normally?

Hint

$\lambda = 0.50/0.40 = 1.25$. Before: MSE $= 0.1^2 = 0.01$, PSNR $= 20$ dB. After rescaling the output equals the reference, so the error is 0 and PSNR is unbounded. A real method never has the ground-truth mean at test time, so GT-mean numbers reward the right *structure* while ignoring the hardest part of the task, choosing the brightness.

6. **Change the color prior.** In the PyTorch toy, replace the gray-world loss with a loss that keeps the *chromaticity* of the input, for example $\|y/\sum_c y_c - x/\sum_c x_c\|_1$. Train again and compare the coffee result with Figure 7. When is each prior preferable?

Hint

Add a small epsilon to the sums. Preserving input chromaticity keeps the red-brown cup red, but it also keeps any color cast of the light source (for example a sodium lamp). Gray world removes casts but desaturates scenes that are genuinely colorful. Real methods often combine a weak color prior with learned color correction from data.

7. **LOE by hand.** A three-pixel input has lightness $L = (0.1, 0.2, 0.3)$, and an enhancer outputs $L_e = (0.5, 0.4, 0.9)$. Compute $\mathrm{RD}(x)$ for each pixel and the LOE. Then explain why, in Figure 9, the clean reference gets a *higher* LOE (322) than the gamma curve (0), and what this implies for comparing a denoising enhancer with a plain tone curve using LOE alone.

Hint

Only the pair $(1, 2)$ changes order: $0.1 < 0.2$ in the input but $0.5 > 0.4$ in the output. That flips both $U(L_1, L_2)$ and $U(L_2, L_1)$, so $\mathrm{RD} = (1, 1, 0)$ (self-pairs never flip) and $\mathrm{LOE} = 2/3$. A gamma curve applied per channel is monotonic, so it keeps the order of $\max(R, G, B)$ exactly and gets 0. The clean image does not share the *noise* of the dark input, and that noise has scrambled the order of many nearby, nearly equal pixels, so the clean image disagrees with the input on those pairs. LOE therefore rewards keeping the input's noise and penalizes real restoration; it should be read together with a noise or quality measure, never alone.

References

1. Shyandram, "簡介 Deep Learning Low-Light Image Enhancement (LLIE)" (Introduction to deep-learning low-light image enhancement, in Chinese), Medium post, 2024; updated 2026. link
2. C. Li, C. Guo, L. Han, J. Jiang, M.-M. Cheng, J. Gu and C. C. Loy, "Low-light image and video enhancement using deep learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 12, pp. 9396–9416, 2022. arXiv
3. S. Zheng, Y. Ma, J. Pan, C. Lu and G. Gupta, "Low-light image and video enhancement: A comprehensive survey and beyond," arXiv:2212.10772, 2022. arXiv
4. J. Liu, D. Xu, W. Yang, M. Fan and H. Huang, "Benchmarking low-light image enhancement and beyond," *International Journal of Computer Vision*, vol. 129, no. 4, pp. 1153–1184, 2021. doi project
5. F. Liu and L. Fan, "A review of advancements in low-light image enhancement using deep learning," *Neurocomputing*, vol. 652, art. 131052, 2025. arXiv
6. E. Adhikarla, Y. Liu and B. D. Davison, "Diffusion models for low-light image enhancement: A multi-perspective taxonomy and performance analysis," arXiv:2510.05976, 2025. arXiv
7. A. Foi, M. Trimeche, V. Katkovnik and K. Egiazarian, "Practical Poissonian-Gaussian noise modeling and fitting for single-image raw-data," *IEEE Transactions on Image Processing*, vol. 17, no. 10, pp. 1737–1754, 2008. doi
8. ITU-R, "Recommendation BT.601-7: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios," 2011. ITU
9. S. M. Pizer, E. P. Amburn, J. D. Austin, et al., "Adaptive histogram equalization and its variations," *Computer Vision, Graphics, and Image Processing*, vol. 39, no. 3, pp. 355–368, 1987. doi
10. K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, Academic Press, 1994, pp. 474–485. doi
11. E. H. Land, "The retinex theory of color vision," *Scientific American*, vol. 237, no. 6, pp. 108–128, 1977. doi

12. D. J. Jobson, Z. Rahman and G. A. Woodell, “Properties and performance of a center/surround retinex,” *IEEE Transactions on Image Processing*, vol. 6, pp. 451–462, 1997. doi
13. D. J. Jobson, Z. Rahman and G. A. Woodell, “A multiscale retinex for bridging the gap between color images and the human observation of scenes,” *IEEE Transactions on Image Processing*, vol. 6, no. 7, pp. 965–976, 1997. doi
14. K. He, J. Sun and X. Tang, “Single image haze removal using dark channel prior,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 12, pp. 2341–2353, 2011. doi
15. X. Dong, Y. Pang and J. Wen, “Fast efficient algorithm for enhancement of low lighting video,” *ACM SIGGRAPH 2010 Posters*, 2010. SIGGRAPH history doi
16. X. Dong, G. Wang, Y. Pang, W. Li, J. Wen, W. Meng and Y. Lu, “Fast efficient algorithm for enhancement of low lighting video,” in *Proc. IEEE International Conference on Multimedia and Expo* (*ICME*), 2011. doi
17. X. Guo, Y. Li and H. Ling, “LIME: Low-light image enhancement via illumination map estimation,” *IEEE Transactions on Image Processing*, vol. 26, no. 2, pp. 982–993, 2017. doi
18. S. Wang, J. Zheng, H.-M. Hu and B. Li, “Naturalness preserved enhancement algorithm for non-uniform illumination images,” *IEEE Transactions on Image Processing*, vol. 22, no. 9, pp. 3538–3548, 2013. doi
19. K. G. Lore, A. Akintayo and S. Sarkar, “LLNet: A deep autoencoder approach to natural low-light image enhancement,” *Pattern Recognition*, vol. 61, pp. 650–662, 2017. doi
20. L. Tao, C. Zhu, J. Song, T. Lu, H. Jia and X. Xie, “Low-light image enhancement using CNN and bright channel prior,” in *Proc. IEEE International Conference on Image Processing* (*ICIP*), 2017, pp. 3215–3219. doi
21. C. Wei, W. Wang, W. Yang and J. Liu, “Deep retinex decomposition for low-light enhancement,” in *Proc. British Machine Vision Conference* (*BMVC*), 2018. arXiv project
22. Y. Zhang, J. Zhang and X. Guo, “Kindling the darkness: A practical low-light image enhancer,” in *Proc. ACM International Conference on Multimedia* (*ACM MM*), 2019, pp. 1632–1640. doi
23. Y. Zhang, X. Guo, J. Ma, W. Liu and J. Zhang, “Beyond brightening low-light images,” *International Journal of Computer Vision*, vol. 129, no. 4, pp. 1013–1037, 2021. doi
24. S. W. Zamir, A. Arora, S. Khan, et al., “Learning enriched features for real image restoration and enhancement,” in *Proc. European Conference on Computer Vision* (*ECCV*), 2020, pp. 492–511. arXiv
25. X. Xu, R. Wang, C.-W. Fu and J. Jia, “SNR-aware low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2022, pp. 17693–17703. doi
26. Y. Cai, H. Bian, J. Lin, H. Wang, R. Timofte and Y. Zhang, “Retinexformer: One-stage Retinex-based transformer for low-light image enhancement,” in *Proc. IEEE/CVF ICCV*, 2023. arXiv code
27. Q. Yan, Y. Feng, C. Zhang, et al., “HVI: A new color space for low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2025. arXiv
28. S.-E. Weng, S.-G. Miaou and R. Christanto, “A lightweight low-light image enhancement network via channel prior and gamma correction,” *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 39, no. 12, art. 2554013, 2025. doi code
29. S.-E. Weng, C.-Y. Hsiao, L.-W. Lu, et al., “Rethinking theoretical illumination for efficient low-light image enhancement” (CPGA-Net+), arXiv:2409.05274, 2024. arXiv
30. R. Liu, L. Ma, J. Zhang, X. Fan and Z. Luo, “Retinex-inspired unrolling with cooperative

prior architecture search for low-light image enhancement," in *Proc. IEEE/CVF CVPR*, 2021, pp. 10556–10565. arXiv
31. W. Wu, J. Weng, P. Zhang, X. Wang, W. Yang and J. Jiang, "URetinex-Net: Retinex-based deep unfolding network for low-light image enhancement," in *Proc. IEEE/CVF CVPR*, 2022, pp. 5891–5900. doi
32. Y. Jiang, X. Gong, D. Liu, et al., "EnlightenGAN: Deep light enhancement without paired supervision," *IEEE Transactions on Image Processing*, vol. 30, pp. 2340–2349, 2021. arXiv
33. C. Guo, C. Li, J. Guo, et al., "Zero-reference deep curve estimation for low-light image enhancement," in *Proc. IEEE/CVF CVPR*, 2020, pp. 1777–1786. arXiv
34. C. Li, C. Guo and C. C. Loy, "Learning to enhance low-light image via zero-reference deep curve estimation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 8, 2022. arXiv
35. L. Ma, T. Ma, R. Liu, X. Fan and Z. Luo, "Toward fast, flexible, and robust low-light image enhancement," in *Proc. IEEE/CVF CVPR*, 2022, pp. 5627–5636. arXiv
36. Z. Liang, C. Li, S. Zhou, R. Feng and C. C. Loy, "Iterative prompt learning for unsupervised backlit image enhancement," in *Proc. IEEE/CVF ICCV*, 2023. arXiv
37. Y. Wang, R. Wan, W. Yang, H. Li, L.-P. Chau and A. C. Kot, "Low-light image enhancement with normalizing flow," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 36, no. 3, pp. 2604–2612, 2022. arXiv
38. X. Yi, H. Xu, H. Zhang, L. Tang and J. Ma, "Diff-Retinex: Rethinking low-light image enhancement with a generative diffusion model," in *Proc. IEEE/CVF ICCV*, 2023. arXiv
39. J. Hou, Z. Zhu, J. Hou, H. Liu, H. Zeng and H. Yuan, "Global structure-aware diffusion process for low-light image enhancement," in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2023. arXiv
40. H. Jiang, A. Luo, X. Liu, S. Han and S. Liu, "LightenDiffusion: Unsupervised low-light image enhancement with latent-Retinex diffusion models," in *Proc. European Conference on Computer Vision* (*ECCV*), 2024. arXiv
41. C. He, C. Fang, Y. Zhang, et al., "Reti-Diff: Illumination degradation image restoration with Retinex-based latent diffusion model," in *Proc. International Conference on Learning Representations* (*ICLR*), 2025. proceedings
42. J. Bai, Y. Yin, Q. He, Y. Li and X. Zhang, "RetinexMamba: Retinex-based Mamba for low-light image enhancement," arXiv:2405.03349, 2024. arXiv
43. M. V. Conde, G. Geigle and R. Timofte, "InstructIR: High-quality image restoration following human instructions," in *Proc. European Conference on Computer Vision* (*ECCV*), 2024. arXiv
44. C. Chen, Q. Chen, J. Xu and V. Koltun, "Learning to see in the dark," in *Proc. IEEE/CVF CVPR*, 2018, pp. 3291–3300. arXiv
45. Z. Cui, G.-J. Qi, L. Gu, S. You, Z. Zhang and T. Harada, "Multitask AET with orthogonal tangent regularity for dark object detection," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
46. K. A. Hashmi, G. Kallempudi, D. Stricker and M. Z. Afzal, "FeatEnHancer: Enhancing hierarchical features for object detection and beyond under low-light vision," in *Proc. IEEE/CVF ICCV*, 2023. arXiv
47. Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. doi

48. R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *Proc. IEEE/CVF CVPR*, 2018. arXiv
49. A. Mittal, R. Soundararajan and A. C. Bovik, “Making a ‘completely blind’ image quality analyzer,” *IEEE Signal Processing Letters*, vol. 20, no. 3, pp. 209–212, 2013. doi
50. A. Mittal, A. K. Moorthy and A. C. Bovik, “No-reference image quality assessment in the spatial domain,” *IEEE Transactions on Image Processing*, vol. 21, no. 12, pp. 4695–4708, 2012. doi
51. J. Liao, S. Hao, R. Hong and M. Wang, “GT-Mean loss: A simple yet effective solution for brightness mismatch in low-light image enhancement,” in *Proc. IEEE/CVF ICCV*, 2025. arXiv
52. W. Yang, W. Wang, H. Huang, S. Wang and J. Liu, “Sparse gradient regularized deep Retinex network for robust low-light image enhancement,” *IEEE Transactions on Image Processing*, vol. 30, pp. 2072–2086, 2021. doi
53. J. Hai, Z. Xuan, R. Yang, et al., “R2RNet: Low-light image enhancement via Real-low to Real-normal Network,” *Journal of Visual Communication and Image Representation*, vol. 90, art. 103712, 2023. arXiv
54. R. Wang, X. Xu, C.-W. Fu, et al., “Seeing dynamic scene in the dark: A high-quality video dataset with mechatronic alignment,” in *Proc. IEEE/CVF ICCV*, 2021, pp. 9680–9689. doi code
55. V. Bychkovsky, S. Paris, E. Chan and F. Durand, “Learning photographic global tonal adjustment with a database of input/output image pairs,” in *Proc. IEEE CVPR*, 2011, pp. 97–104. dataset
56. J. Cai, S. Gu and L. Zhang, “Learning a deep single image contrast enhancer from multi-exposure images,” *IEEE Transactions on Image Processing*, vol. 27, no. 4, pp. 2049–2062, 2018. doi
57. Y. P. Loh and C. S. Chan, “Getting to know low-light images with the Exclusively Dark dataset,” *Computer Vision and Image Understanding*, vol. 178, pp. 30–42, 2019. arXiv
58. K. Ma, K. Zeng and Z. Wang, “Perceptual quality assessment for multi-exposure image fusion,” *IEEE Transactions on Image Processing*, vol. 24, no. 11, pp. 3345–3356, 2015. doi
59. C. Lee, C. Lee and C.-S. Kim, “Contrast enhancement based on layered difference representation of 2D histograms,” *IEEE Transactions on Image Processing*, vol. 22, no. 12, pp. 5372–5384, 2013. doi
60. V. Vonikakis, “Datasets” (source of the VV test images), personal web page. link
61. X. Liu, Z. Wu, A. Li, et al., “NTIRE 2024 challenge on low light image enhancement: Methods and results,” in *Proc. IEEE/CVF CVPR Workshops*, 2024. arXiv
62. X. Liu, Z. Wu, F.-A. Vasluianu, et al., “NTIRE 2025 challenge on low light image enhancement: Methods and results,” in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv
63. Y. Blau, R. Mechrez, R. Timofte, T. Michaeli and L. Zelnik-Manor, “The 2018 PIRM challenge on perceptual image super-resolution,” in *Proc. European Conference on Computer Vision (ECCV) Workshops*, 2018. arXiv
64. C. Ma, C.-Y. Yang, X. Yang and M.-H. Yang, “Learning a no-reference quality metric for single-image super-resolution,” *Computer Vision and Image Understanding*, vol. 158, pp. 1–16, 2017. doi arXiv
65. D. Hasler and S. E. Süsstrunk, “Measuring colorfulness in natural images,” in *Proc. SPIE 5007, Human Vision and Electronic Imaging VIII*, 2003. doi
66. J. Immerkær, “Fast noise variance estimation,” *Computer Vision and Image Understanding*, vol. 64, no. 2, pp. 300–302, 1996. doi

67. H. Talebi and P. Milanfar, “NIMA: Neural image assessment,” *IEEE Transactions on Image Processing*, 2018. arXiv
68. J. Ke, Q. Wang, Y. Wang, P. Milanfar and F. Yang, “MUSIQ: Multi-scale image quality transformer,” in *Proc. IEEE/CVF ICCV*, 2021. arXiv
69. J. Wang, K. C. K. Chan and C. C. Loy, “Exploring CLIP for assessing the look and feel of images,” in *Proc. AAAI Conference on Artificial Intelligence*, 2023. arXiv
70. S. Yang, T. Wu, S. Shi, et al., “MANIQA: Multi-dimension attention network for no-reference image quality assessment,” in *Proc. IEEE/CVF CVPR Workshops*, 2022, pp. 1190–1199. doi arXiv
71. H. Wu, Z. Zhang, W. Zhang, C. Chen, et al., “Q-Align: Teaching LMMs for visual scoring via discrete text-defined levels,” in *Proc. International Conference on Machine Learning (ICML)*, PMLR 235, 2024. arXiv
72. V. Kannan, S. Malik, N. C. Babu and R. Soundararajan, “Quality assessment of low-light restored images: A subjective study and an unsupervised model,” *IEEE Access*, vol. 11, pp. 68216–68230, 2023. doi arXiv
73. ITU-R, “Recommendation BT.500-15: Methodologies for the subjective assessment of the quality of television images,” 2023. ITU
74. R. A. Bradley and M. E. Terry, “Rank analysis of incomplete block designs: The method of paired comparisons,” *Biometrika*, vol. 39, no. 3–4, pp. 324–345, 1952. doi
75. C. Chen and J. Mo, “IQA-PyTorch: PyTorch toolbox for image quality assessment,” software (pip package `pyiqa`). GitHub

DLCV 06 • Object Tracking: From Single-Object to Multi-Object

How computer vision follows objects through video: re-identification, single-object trackers from correlation filters to transformers and SAM 2, and multi-object tracking with Kalman filters, Hungarian matching, end-to-end models and the MOTA/IDF1/HOTA metrics.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-06-object-tracking/

What you'll learn

- Where tracking sits after classification and detection, and the two jobs every tracker does: follow motion across frames and keep each object's identity.
- A clear taxonomy: re-identification versus temporal tracking, single-object (SOT/VOT) versus multi-object tracking (MOT), and multi-target multi-camera tracking (MTMCT).
- How single-object trackers evolved: correlation filters (MOSSE, KCF), Siamese networks (SiamFC, SiamRPN), one-stream transformers (OSTrack, SeqTrack) and video segmentation models (SAM 2, SAMURAI, SAM 3).
- Multi-object tracking in depth: the tracking-by-detection pipeline (detector, Kalman filter, appearance embedding, Hungarian assignment), the SORT family (SORT, DeepSORT, ByteTrack, OC-SORT, BoT-SORT), joint detection-and-embedding (JDE, FairMOT), end-to-end query-based trackers (MOTR, MOTRv2, MOTIP) and open-vocabulary tracking.
- How trackers are measured: CMC and mAP for Re-ID, success and precision for SOT, and MOTA, IDF1 and HOTA for MOT, including why HOTA was introduced.
- A working NumPy tracker you can run, with measured ID switches and metrics on a synthetic scene.

The big picture

A detector looks at one picture and says "there are three people here". A tracker watches a video and says "this is the *same* person as one second ago, and that one just walked behind a pillar and will come out again". Tracking adds time and memory to detection.

> **Explain like I'm five**
>
> Imagine you are watching a soccer game with your friend, and you each pick a favorite player. Finding every player on the field in one photo is **detection**. Keeping your eyes on *your* player as she runs, turns and gets hidden behind others for a moment is **single-object tracking**. Putting a numbered sticker on *every* player and making sure the stickers never get swapped, even when two players crash into each other, is **multi-object tracking**. And recognizing your player again when the TV switches to a different camera is **re-identification**.

Why it matters: tracking is how a self-driving car knows that a pedestrian is *walking toward* the road rather than just standing near it, how sports analytics measures how far each player ran, how a warehouse counts forklifts and robots, and how a video editor keeps a mask glued to a moving object. Each of these needs not just *where* things are, but *which one is which* over time.

About this tutorial

This tutorial is expanded from the author's earlier Medium post (written in Traditional Chinese), "物件追蹤 Object Tracking 簡介" [1], keeping its structure and adding the technical depth, a runnable tracker and an up-to-date survey. You can read the original post here.

From classification to detection to tracking

Plain version. Classification answers *what* is in an image. Detection answers *what and where*. Tracking answers *what, where, and which one, over time*.

The progression in [1] is a useful map of computer vision tasks:

1. **Image classification** maps an image I to a label y.
2. **Object detection** maps an image to a set of boxes with labels and confidence scores, $\mathcal{D} = \{(b_j, c_j, s_j)\}_{j=1}^{N}$, where b_j is a bounding box, c_j a class and $s_j \in [0, 1]$ a score. Detection imitates the human ability to locate targets in a scene; it is covered in depth in DLCV 03 ·Object Detection.
3. **Object tracking** takes a sequence of frames $I_1, \dots, I_T$ and outputs **trajectories**: for each object k, a sequence of boxes (or masks) $(b_t^k)_{t \in \mathcal{T}_k}$ over the set of frames $\mathcal{T}_k$ in which it is visible, all carrying the same identity k.

The key new ingredient is the identity label. Detection is stateless: frame t and frame $t + 1$ are processed independently. Tracking is stateful: it carries information forward.

What tracking does

Plain version. Tracking imitates how people perceive moving things: we predict where something will go, look there, and keep treating it as the same object even when its appearance changes or it is briefly hidden.

More precisely, every tracker solves two coupled sub-problems:

- **Localization over time** (motion perception): estimate where each target is in each frame, ideally using the past to predict the future. This is what lets a tracker survive blur, missed detections and short occlusions.

- **Identity maintenance** (data association): decide which observation in frame t belongs to which target from frame $t-1$. This is what prevents two people who cross paths from swapping labels.

Single-object tracking puts most of its effort into the first problem, since there is only one identity. Multi-object tracking spends most of its effort on the second. Re-identification is the second problem with *no* motion information at all.

The difficulties that [1] lists for each family come from these two jobs: appearance changes (pose, lighting, scale, viewpoint), occlusion by other objects or the scene, similar-looking distractors, fast or non-linear motion, camera motion, and targets entering and leaving the scene.

A taxonomy of tracking

Plain version. You can sort trackers by *what* they follow (one object or many, one camera or many) and by *how* they keep identity (by appearance alone, or by following motion frame to frame).

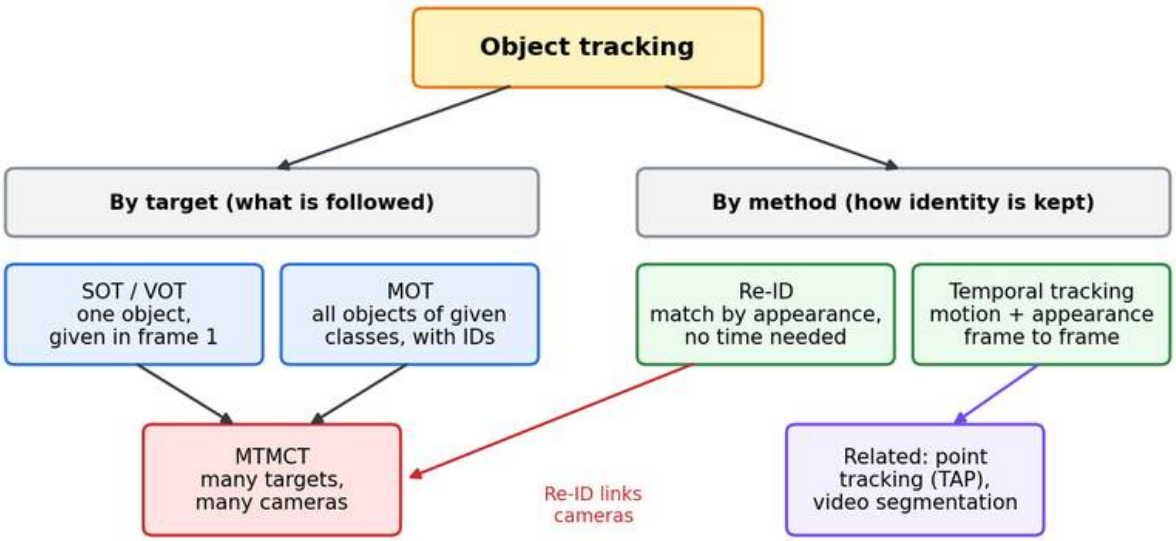


Figure 1 — A taxonomy of tracking. By target: single-object (SOT, also called VOT) and multi-object (MOT). By method: appearance-only re-identification and temporal tracking. The hardest combined setting is multi-target multi-camera tracking (MTMCT).

Following [1]:

- **By method.** *Re-identification* (*Re-ID*) matches a target across images that may be far apart in time or come from different cameras, using appearance (and sometimes pose) alone. *Temporal*

object tracking follows targets frame by frame in one video stream, where motion continuity is available.

- **By target.** *Visual object tracking* (*VOT*), also called *single-object tracking* (*SOT*), follows one arbitrary object specified by a box in the first frame. *Multi-object tracking* (*MOT*) follows all objects of given classes (for example, every pedestrian) through consecutive frames and maintains a consistent ID for each.
- **The combined goal.** *Multi-target multi-camera tracking* (*MTMCT*) tracks many targets across a network of cameras and keeps the same identity when a person leaves one camera's view and appears in another. It combines MOT within each camera with Re-ID between cameras.

Most of the classical literature, and most benchmarks, use **pedestrians** as targets [1], because surveillance and autonomous driving made people the most valuable class to track. Modern benchmarks add dancers, athletes, vehicles and open-vocabulary categories.

Person re-identification

Plain version. Re-ID is a search problem: given a photo of a person from one camera, find the same person in photos taken by other cameras, ranked from most to least likely.

Problem formulation

Re-ID is posed as **retrieval** [7]. A *query* image q and a *gallery* $\mathcal{G} = \{g_1, \dots, g_M\}$ of detected person crops from other cameras are mapped by a network f_θ to embeddings. The gallery is ranked by a distance, usually cosine distance

$$d(q, g) = 1 - \frac{f_\theta(q)^\top f_\theta(g)}{\|f_\theta(q)\| \, \|f_\theta(g)\|},$$

where $f_\theta(\cdot) \in \mathbb{R}^D$ is the D-dimensional embedding produced by the network with parameters θ. Training makes embeddings of the same identity close and of different identities far apart. A typical recipe combines an identity classification loss (cross-entropy over the C training identities) with a **triplet loss**

$$\mathcal{L}_{\text{tri}} = \left[\, d(a, p) - d(a, n) + m \,\right]_+,$$

where a is an anchor image, p a positive (same identity), n a negative (different identity), $m > 0$ a margin and $[x]_+ = \max(x, 0)$. Test identities never appear in training, so Re-ID is an open-set problem: the network must learn a general notion of "same person".

Challenges

[1] groups the difficulties into **data factors** and **environment factors**. Data factors: limited labeled identities, noisy boxes from the detector, partial bodies, low resolution. Environment factors: different viewpoints and camera color responses, lighting, occlusion, background clutter, and people who change clothes. Re-ID also raises clear **privacy** concerns, which is part of why surveillance-style datasets have been withdrawn (below). The original post names city-scale surveillance systems as the main application; today Re-ID is just as common as the appearance module inside MOT trackers and in retail and sports analytics.

Representative models

- **Strong baselines.** Luo et al. [14] showed that a plain ResNet-50 with a set of training "tricks" — warm-up learning rate, random erasing augmentation, label smoothing, last-stride 1 for a larger feature map, and a **BNNeck** (a batch-norm layer that separates the triplet-loss feature from the classification feature) — is a strong, reproducible baseline.
- **Centroid triplet loss (CTL).** The model the original post cites (ResNet-50 at 256 × 128 input) [15] replaces instance-to-instance triplets with distances to *class centroids*, and at retrieval time compares a query to identity centroids rather than to every gallery image.
- **CLIP-ReID.** Li et al. [16] adapt the vision-language model CLIP to Re-ID, where no text labels exist. In a first stage they learn identity-specific text tokens with the encoders frozen; in a second stage they fine-tune the image encoder using those text features as extra supervision.

Datasets

Dataset	Content	Status
Market-1501 [12]	1,501 identities, 6 cameras, detector-cropped boxes	available
MSMT17 [13]	4,101 identities, 15 indoor and outdoor cameras	available
DukeMTMC [10] (and DukeMTMC-reID)	8 synchronized campus cameras	**withdrawn in 2019**

DukeMTMC was introduced with the IDF1 metric for multi-camera tracking [10], and a Re-ID subset was widely used. Its creators withdrew it in 2019 after privacy and ethics concerns about recording people on a campus, and it should no longer be downloaded or used; Peng et al. [11] trace how derived copies kept circulating afterwards and argue for better dataset stewardship.

Metrics: CMC rank-k and mAP

The **cumulative matching characteristic** at rank k is the fraction of queries whose first correct match appears in the top k of the ranked gallery:

$$\mathrm{CMC}(k) = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{1}\left[\, r_q \le k \,\right],$$

where Q is the set of queries and r_q is the rank of the first true match for query q. *Rank-1 accuracy* is $\mathrm{CMC}(1)$. Because a query usually has several true matches in the gallery, rank-k alone ignores how well the *others* are ranked. **Mean average precision** fixes this:

$$\mathrm{AP}(q) = \frac{1}{G_q} \sum_{i=1}^{G_q} \frac{i}{r_{q,i}}, \qquad \mathrm{mAP} = \frac{1}{|Q|} \sum_{q} \mathrm{AP}(q),$$

where G_q is the number of true matches of q in the gallery and $r_{q,i}$ is the rank of the i-th one. So $i / r_{q,i}$ is the precision at the moment the i-th true match is found.

```
import numpy as np

def cmc_map(dist, q_ids, g_ids, max_rank=5):
    """dist: (Q, G) distances between query and gallery embeddings."""
    order = np.argsort(dist, axis=1)                # closest gallery items first
    match = g_ids[order] == q_ids[:, None]          # (Q, G) True where same person
    cmc = (np.cumsum(match, axis=1) > 0).mean(axis=0)[:max_rank]
    aps = []
    for row in match:
        hits = np.flatnonzero(row)                  # ranks (0-based) of true matches
        precision_at_hits = np.arange(1, len(hits) + 1) / (hits + 1)
        aps.append(precision_at_hits.mean())
    return cmc, np.mean(aps)

rng = np.random.default_rng(0)
centres = rng.normal(size=(50, 64))                 # 50 identities, 64-D embeddings
g_ids = np.repeat(np.arange(50), 4)                 # 4 gallery images per identity
q_ids = np.arange(50)
gallery = centres[g_ids] + 1.6 * rng.normal(size=(200, 64))
query = centres[q_ids] + 1.6 * rng.normal(size=(50, 64))
norm = lambda x: x / np.linalg.norm(x, axis=1, keepdims=True)
dist = 1 - norm(query) @ norm(gallery).T            # cosine distance
cmc, mAP = cmc_map(dist, q_ids, g_ids)
print("rank-1 %.2f  rank-5 %.2f  mAP %.2f" % (cmc[0], cmc[4], mAP))
```

On this synthetic gallery (4 images per identity) the script prints `rank-1 0.76  rank-5 0.92  mAP 0.57`: most queries find *one* correct image near the top, but the other three are scattered, which only mAP reveals.

Single-object tracking

Plain version. You draw a box around something in the first frame; the tracker must keep a box on that same thing for the rest of the video, without knowing in advance what kind of object it is.

Problem formulation

Given frames $I_1, \dots, I_T$ and an initial box b_1, a single-object tracker outputs $\hat{b}_2, \dots, \hat{b}_T$. The target is **class-agnostic**: it could be a person, a bird or a coffee cup. This makes SOT a form of *one-shot* learning: the only example of the target is the first-frame patch, the **template** z. In each new frame the tracker searches a region x around the last position. The challenges [1] lists are complex appearance changes, unusual motion patterns, and environmental change such as lighting and occlusion.

Classical: correlation filters

Before deep learning, the fastest trackers learned a **correlation filter** online. MOSSE [17] learns a filter H in the Fourier domain so that correlating it with training patches F_i produces sharp Gaussian peaks G_i at the target center:

$$\min_{H^*} \sum_i |F_i \odot H^* - G_i|^2 \quad \Rightarrow \quad H^* = \frac{\sum_i G_i \odot F_i^*}{\sum_i F_i \odot F_i^*},$$

where capital letters are 2-D discrete Fourier transforms, $\odot$ is element-wise multiplication, * is complex conjugation and the division is element-wise. Because correlation becomes multiplication in the frequency domain (see IP 04), training and detection cost only a few FFTs, so these trackers run at hundreds of frames per second on a CPU. KCF [18] showed that training on *all cyclic shifts* of a patch gives a circulant data matrix that the DFT diagonalizes, which turns kernel ridge regression into element-wise operations and allows multi-channel features such as HOG. Their weaknesses are a fixed search window, boundary effects from the cyclic assumption, and hand-crafted features.

Siamese trackers

Plain version. Train one network to answer a single question — "where in this search image does the template appear?" — on huge amounts of video offline, then use it frozen at test time.

SiamFC [19] applies the *same* fully convolutional network φ to the template z and the search region x and cross-correlates the two feature maps:

$$f(z, x) = \varphi(z) \star \varphi(x) + b\,\mathbb{1},$$

where $\star$ is cross-correlation (the template features act as a convolution kernel), b is a learned bias and $\mathbb{1}$ is a map of ones. The output is a score map whose peak gives the new position. It is trained with a logistic loss on score-map positions labeled positive near the true center and negative elsewhere. Because nothing is updated online, it is fast and robust to drift, but it only predicts position, with scale handled by searching a small pyramid.

SiamRPN [20] adds a **region proposal network** after the Siamese features: one branch classifies anchors as target or background, another regresses box offsets. Tracking becomes "one-shot detection" with the template as the class definition, which yields accurate boxes with changing aspect ratios.

Transformer one-stream trackers

From 2022 the mainstream moved from Siamese CNNs to transformers. The key change is *where* the template and the search region interact (Figure 2). In a two-stream design they are encoded separately and compared once. In a **one-stream** design such as OSTrack [21], the image patches of z and x are concatenated into one token sequence and processed by a single Vision Transformer, so that every attention layer both extracts features and relates template to search. OSTrack also drops search tokens that are clearly background in early layers to save computation. SeqTrack [22] goes further in simplicity: an encoder-decoder transformer generates the four box coordinates as a sequence of discrete tokens, autoregressively, trained with plain cross-entropy — the same recipe as language modeling.

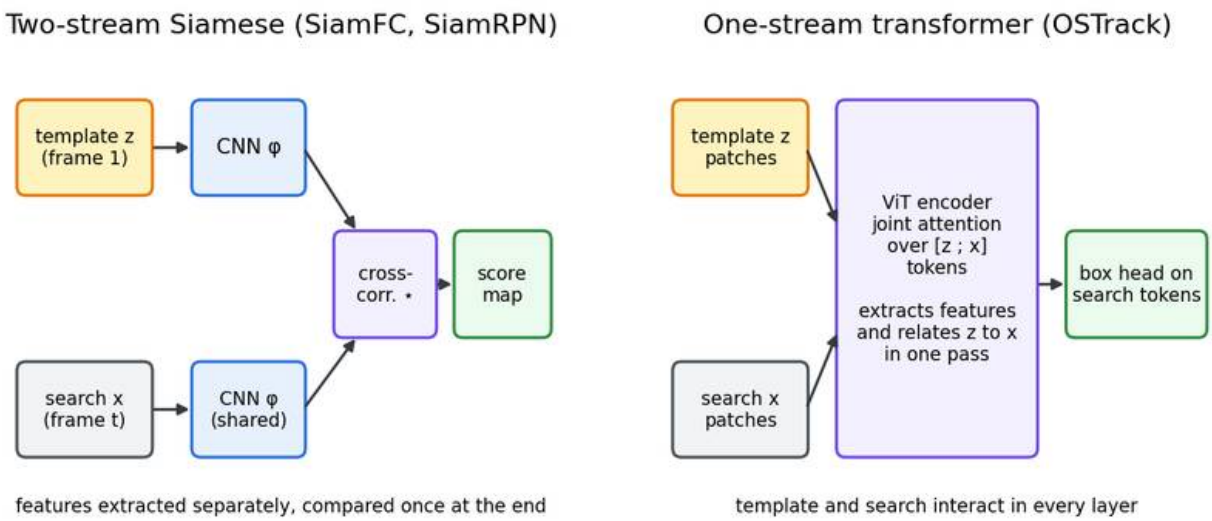


Figure 2 — Two-stream Siamese trackers compare template and search features once, at the end. One-stream transformer trackers let them interact in every layer.

Tracking with video segmentation models

Plain version. New "segment anything in video" models can be told "this object" with one click, and then follow it for the whole video as a pixel mask. That *is* single-object tracking, with a mask instead of a box.

SAM 2 [23] extends the Segment Anything model to video with a **streaming memory**: features and predicted masks from past frames, plus the prompted frames, are stored in a memory bank that the current frame attends to. Prompting one frame with a click, box or mask yields a "masklet" through the whole video, so it behaves like a mask-output SOT tracker. SAMURAI [24] adapts SAM 2 for tracking without any retraining: it adds a Kalman-filter motion model to choose among SAM 2's candidate masks and a motion-aware rule for which past frames enter memory, which helps in crowded scenes with similar distractors. SAM 3 [25] adds **promptable concept segmentation**: given a short noun phrase such as "yellow school bus" or an image exemplar, it detects, segments and tracks *all* matching instances in a video and gives each its own identity. That is a form of open-vocabulary MOT, and it blurs the old SOT/MOT boundary.

Benchmarks and metrics

Benchmark	What makes it distinct
OTB [26]	early standard benchmark with per-sequence attribute labels (occlusion, fast motion, ...); introduced the success and precision plots
VOT challenge [53]	yearly challenge series since 2013, with reset-based protocols; now VOTS, tracking *and* segmentation
LaSOT [27]	long-term: 1,400 sequences, more than 3.5 million frames, each frame manually annotated
GOT-10k [28]	one-shot protocol: training and test object classes do not overlap
TrackingNet [29]	large-scale, in-the-wild videos with a held-out evaluation server

The two classic measures, from OTB [26], are:

$$\text{Success}(\tau) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{1}[\,\text{IoU}(\hat{b}_t, b_t) > \tau\,], \qquad \text{AUC} = \int_0^1 \text{Success}(\tau)\, d\tau,$$

$$\text{Precision}(\delta) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{1}[\|\hat{c}_t - c_t\|_2 \le \delta],$$

where $\hat{b}_t, b_t$ are the predicted and ground-truth boxes, τ is an IoU threshold, $\hat{c}_t, c_t$ are their centers and δ is a pixel distance, conventionally 20. The AUC of the success plot is the headline number

on OTB and LaSOT. GOT-10k reports **average overlap** (the mean IoU) and success rates at $\tau = 0.5$ and 0.75.

```
import numpy as np

def iou_xywh(a, b):
    """IoU between rows of a and b, boxes as [x, y, w, h] (top-left corner)."""
    x1 = np.maximum(a[:, 0], b[:, 0]); y1 = np.maximum(a[:, 1], b[:, 1])
    x2 = np.minimum(a[:, 0] + a[:, 2], b[:, 0] + b[:, 2])
    y2 = np.minimum(a[:, 1] + a[:, 3], b[:, 1] + b[:, 3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    return inter / (a[:, 2] * a[:, 3] + b[:, 2] * b[:, 3] - inter)

def success_auc(pred, gt, thresholds=np.linspace(0, 1, 21)):
    ious = iou_xywh(pred, gt)
    curve = np.array([(ious > t).mean() for t in thresholds])
    return curve.mean()                              # area under the success plot

def precision_at(pred, gt, px=20):
    c_pred = pred[:, :2] + pred[:, 2:] / 2; c_gt = gt[:, :2] + gt[:, 2:] / 2
    return (np.linalg.norm(c_pred - c_gt, axis=1) <= px).mean()

rng = np.random.default_rng(0)
gt = np.c_[np.linspace(50, 400, 300), np.full(300, 80.0), np.full(300, 40.0), np.full(300,
 ↪  60.0)]
pred = gt + rng.normal(0, 4, gt.shape)
pred[200:230, :2] += 120                             # 30 frames lost on a distractor
print("success AUC %.3f   precision@20px %.3f" % (success_auc(pred, gt), precision_at(pred,
 ↪  gt)))
```

This prints `success AUC 0.647   precision@20px 0.900`: the 30 frames spent on a distractor cost precision 10 points, while the small jitter on the other frames costs AUC much more, because AUC also rewards tight boxes.

Multi-object tracking

Plain version. Find every object of interest in every frame, and give each one a number that stays the same for as long as it is in view — even when objects cross, hide behind each other, or leave and come back.

MOT tracks multiple targets **across consecutive frames** and maintains a consistent ID for each; a single frame on its own only supports detection. The challenges [1] lists are frequent occlusion, deciding when trajectories start and end, similar appearance between targets, and interaction between targets (people in a crowd move together and occlude each other).

Formally, given detections $\mathcal{D}_t$ in each frame, MOT seeks a set of trajectories $\mathcal{T} = \{\tau_k\}$ that partitions the true detections, discards false positives, and fills in missed ones. Offline methods solve

this over the whole video, often as a graph or network-flow problem; **online** methods, which this section focuses on, must decide at frame t using only frames $1..t$, which is what real-time applications require.

The tracking-by-detection pipeline

The dominant design for a decade has been **tracking-by-detection** [2], [4]: run a detector on every frame, then link detections over time (Figure 3). Each frame goes through four steps:

1. **Detect.** A detector (Faster R-CNN, YOLO-family, DETR-family; see DLCV 03) gives boxes and scores.
2. **Predict.** A motion model moves every existing track forward to where it should be now.
3. **Describe.** Optionally, a Re-ID network embeds each detection crop.
4. **Associate.** Build a cost matrix between predicted tracks and detections and solve the assignment problem. Matched tracks are updated; unmatched detections may start new tracks (*birth*); tracks unmatched for too long are deleted (*death*).

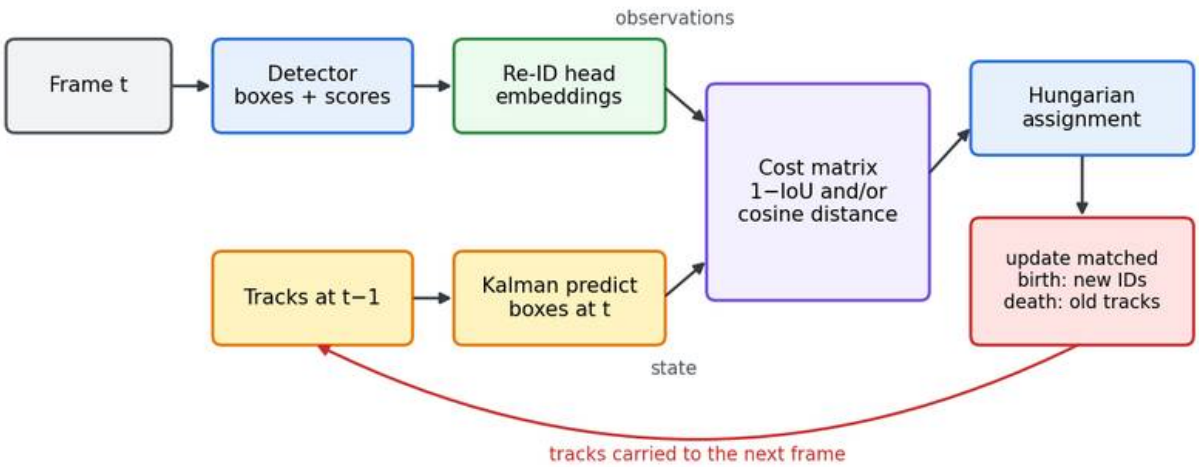


Figure 3 — Online tracking-by-detection. Observations (boxes, embeddings) and state (predicted tracks) meet in a cost matrix; the assignment result updates the state for the next frame.

Motion model: the Kalman filter

Plain version. Guess where the object is now from where it was and how fast it was moving; then nudge that guess toward the detection, trusting whichever of the two is less uncertain.

Nearly all tracking-by-detection systems use a **linear Kalman filter** [30] with a constant-velocity model. In our code the state of one track is

$$\mathbf{x} = [\, c_x,\ c_y,\ w,\ h,\ v_x,\ v_y \,]^\top,$$

where (c_x, c_y) is the box center, w, h its width and height, and (v_x, v_y) the center velocity in pixels per frame. (SORT uses the center, area and aspect ratio instead of w, h [32]; BoT-SORT returns to w and h [36].) The model has two steps.

Predict (time update):

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}\,\hat{\mathbf{x}}_{t-1|t-1},$$
$$\mathbf{P}_{t|t-1} = \mathbf{F}\,\mathbf{P}_{t-1|t-1}\,\mathbf{F}^\top + \mathbf{Q},$$

where $\mathbf{F}$ is the 6×6 transition matrix (identity plus 1s that add v_x, v_y to c_x, c_y, since $\Delta t = 1$ frame), $\mathbf{P}$ is the state covariance (our uncertainty) and $\mathbf{Q}$ is the process noise covariance (how much the true motion can deviate from constant velocity).

Update (measurement update) with a matched detection $\mathbf{z}_t = [c_x, c_y, w, h]^\top$:

$$\mathbf{S}_t = \mathbf{H}\,\mathbf{P}_{t|t-1}\,\mathbf{H}^\top + \mathbf{R}, \qquad \mathbf{K}_t = \mathbf{P}_{t|t-1}\,\mathbf{H}^\top\,\mathbf{S}_t^{-1},$$
$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t(\mathbf{z}_t - \mathbf{H}\,\hat{\mathbf{x}}_{t|t-1}), \qquad \mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\,\mathbf{P}_{t|t-1},$$

where $\mathbf{H} = [\, \mathbf{I}_4 \ \ \mathbf{0} \,]$ picks the observed part of the state, $\mathbf{R}$ is the detector's measurement noise covariance, $\mathbf{z}_t - \mathbf{H}\hat{\mathbf{x}}_{t|t-1}$ is the **innovation** (surprise), $\mathbf{S}_t$ its covariance, and $\mathbf{K}_t$ the **Kalman gain**. If the prediction is very uncertain ($\mathbf{P}$ large), the update moves the estimate almost all the way to the detection; if the detector is noisy ($\mathbf{R}$ large), it trusts the prediction. When a detection is missing, the tracker runs only the predict step ("coasting"), and $\mathbf{P}$ grows (Figure 4).

```
import numpy as np
from scipy.optimize import linear_sum_assignment

# ---------- 1. constant-velocity Kalman filter ----------
class KalmanBox:
    """State x = [cx, cy, w, h, vx, vy]; measurement z = [cx, cy, w, h]."""
    def __init__(self, z, q=1.0, r=4.0):
        self.x = np.r_[z, 0.0, 0.0]
        self.P = np.diag([r, r, r, r, 100.0, 100.0])    # unknown velocity: large variance
        self.F = np.eye(6); self.F[0, 4] = self.F[1, 5] = 1.0   # dt = 1 frame
        self.H = np.eye(4, 6)
        self.Q = q * np.diag([1, 1, 0.1, 0.1, 0.5, 0.5])
        self.R = r * np.eye(4)

    def predict(self):
        self.x = self.F @ self.x
        self.P = self.F @ self.P @ self.F.T + self.Q
        return self.x[:4]

    def update(self, z):
```

```
        S = self.H @ self.P @ self.H.T + self.R          # innovation covariance
        K = self.P @ self.H.T @ np.linalg.inv(S)          # Kalman gain
        self.x = self.x + K @ (z - self.H @ self.x)
        self.P = (np.eye(6) - K @ self.H) @ self.P
```

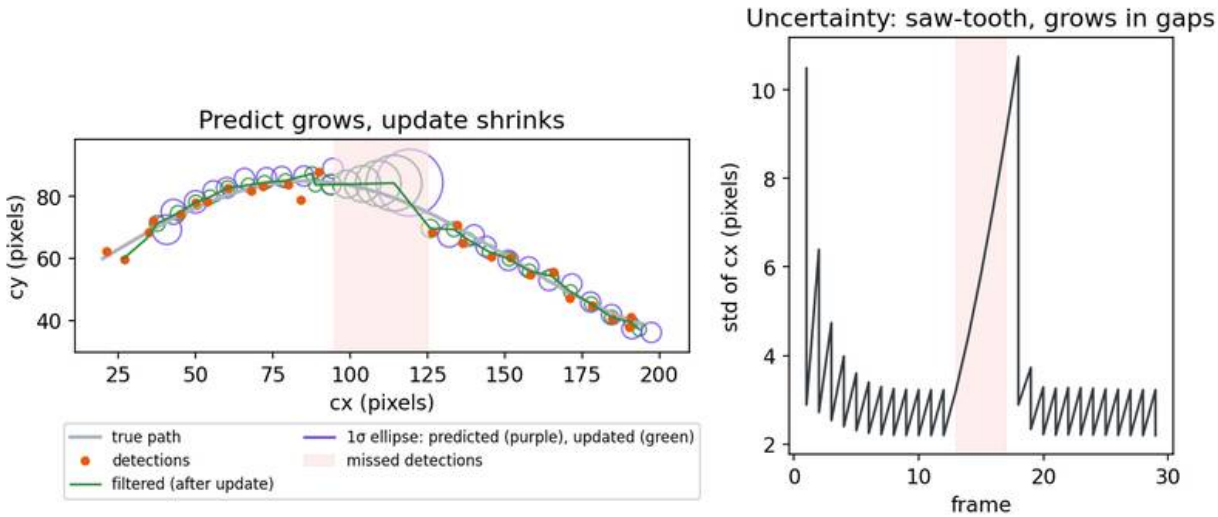


Figure 4 — Kalman predict and update on a synthetic track. Every predict step inflates the uncertainty, every update shrinks it. During five missed detections (pink) the filter coasts in a straight line and its uncertainty grows; this is exactly when identity errors happen.

The squared **Mahalanobis distance** $d_M^2 = \boldsymbol{\nu}^\top \mathbf{S}^{-1} \boldsymbol{\nu}$ of an innovation $\boldsymbol{\nu}$ follows a χ^2 distribution with 4 degrees of freedom under the model, so DeepSORT [33] uses it to *gate* (forbid) implausible matches. The constant-velocity assumption is also the main weakness: dancers, athletes and anything filmed by a moving camera violate it.

Appearance embeddings

A Re-ID network (previous section) turns each detection crop into a unit vector $\mathbf{e}_j$. Each track keeps an appearance memory, for example an exponential moving average $\mathbf{m}_k \leftarrow \alpha\, \mathbf{m}_k + (1-\alpha)\, \mathbf{e}_j$ followed by renormalization, with $\alpha \in [0, 1)$ (we use 0.9). The appearance cost is the cosine distance $1 - \mathbf{m}_k^\top \mathbf{e}_j$. Appearance is what lets a tracker recover an identity after a long occlusion, when motion prediction has become useless.

Association: cost matrices and the Hungarian algorithm

Plain version. Make a table of "how bad would it be to match track k with detection j" for every pair, then choose the one-to-one matching with the smallest total badness.

With K predicted tracks and N detections, build a cost matrix $\mathbf{C} \in \mathbb{R}^{K \times N}$, for example

$$C_{kj} = \lambda \, (1 - \mathrm{IoU}(\hat{b}_k, b_j)) + (1 - \lambda) \, (1 - \mathbf{m}_k^\top \mathbf{e}_j),$$

where $\hat{b}_k$ is the predicted box of track k, b_j detection j's box and $\lambda \in [0, 1]$ the weight of motion versus appearance ($\lambda = 1$ is pure IoU, as in SORT). Then solve the **linear assignment problem**

$$\min_{\mathbf{X} \in \{0,1\}^{K \times N}} \sum_{k,j} C_{kj} X_{kj} \quad \text{s.t.} \quad \sum_j X_{kj} \le 1, \;\; \sum_k X_{kj} \le 1,$$

so each track takes at most one detection and vice versa. The **Hungarian algorithm** [31] solves it exactly in polynomial time ($O(n^3)$ for $n = \max(K, N)$); SciPy's `linear_sum_assignment` implements a fast variant. Finally, any matched pair whose cost exceeds a threshold is rejected, which is how gating is applied in practice.

```
# ---------- 2. IoU and Hungarian association ----------
def iou_matrix(a, b):
    """a: (N,4), b: (M,4) boxes as [cx, cy, w, h] -> (N,M) IoU."""
    a1, a2 = a[:, None, :2] - a[:, None, 2:] / 2, a[:, None, :2] + a[:, None, 2:] / 2
    b1, b2 = b[None, :, :2] - b[None, :, 2:] / 2, b[None, :, :2] + b[None, :, 2:] / 2
    wh = np.clip(np.minimum(a2, b2) - np.maximum(a1, b1), 0, None)
    inter = wh[..., 0] * wh[..., 1]
    area = lambda x: x[..., 2] * x[..., 3]
    return inter / (area(a)[:, None] + area(b)[None, :] - inter + 1e-9)

def associate(cost, max_cost):
    """Hungarian assignment; pairs whose cost exceeds max_cost are rejected."""
    if cost.size == 0:
        return [], list(range(cost.shape[0])), list(range(cost.shape[1]))
    rows, cols = linear_sum_assignment(cost)
    pairs = [(r, c) for r, c in zip(rows, cols) if cost[r, c] <= max_cost]
    mr = {r for r, _ in pairs}; mc = {c for _, c in pairs}
    return (pairs, [r for r in range(cost.shape[0]) if r not in mr],
            [c for c in range(cost.shape[1]) if c not in mc])
```

A complete tracker on a synthetic scene

To see what each idea buys, we generate a scene with five pedestrians over 80 frames, designed to contain the classic failure cases:

- objects 0 and 1 **cross**, and while they overlap, object 1 (behind) gets only **low-confidence** detections;
- objects 2 and 3 meet and **bounce back** (non-linear motion, like dancers), with the rear one also low-confidence during the overlap;
- object 4 walks **behind a pole** for 8 frames, so its detections are low-confidence there;

- every detection is dropped with 5% probability, and there are occasional low-score false positives.

Each detection also carries a 16-D appearance vector: its object's fixed unit vector plus noise.

```
# ---------- 3. synthetic scene ----------
def make_scene(T=80, seed=0):
    rng = np.random.default_rng(seed)
    W, H = 30.0, 70.0
    t = np.arange(T, dtype=float)
    tr = {}
    # pair 1: plain crossing, B is behind A (occluded while they overlap)
    tr[0] = np.c_[80 + 5 * t, np.full(T, 120.0)]
    tr[1] = np.c_[480 - 5 * t, np.full(T, 128.0)]
    # pair 2: they meet and bounce back (non-linear motion, like dancers)
    m = 40
    x2 = np.where(t < m, 120 + 4 * t, 120 + 4 * m - 4 * (t - m))
    x3 = np.where(t < m, 440 - 4 * t, 440 - 4 * m + 4 * (t - m))
    tr[2] = np.c_[x2 + 10, np.full(T, 260.0)]
    tr[3] = np.c_[x3 - 10, np.full(T, 266.0)]
    # walker passing behind a pole (frames 30-37): detections become low-score
    tr[4] = np.c_[60 + 3 * t, 40 + 1.0 * t]
    gt = []                                    # per frame list of (id, box)
    dets = []                                  # per frame list of (box, score, gt_id)
    feats = {k: (lambda v: v / np.linalg.norm(v))(rng.normal(size=16)) for k in tr}
    for f in range(T):
        g, d = [], []
        boxes = {k: np.r_[tr[k][f], W, H] for k in tr}
        for k, b in boxes.items():
            g.append((k, b))
            score = rng.uniform(0.6, 0.95)
            if k == 1 and iou_matrix(b[None], boxes[0][None])[0, 0] > 0.2:
                score = rng.uniform(0.15, 0.45)       # occluded by object 0
            if k == 3 and iou_matrix(b[None], boxes[2][None])[0, 0] > 0.2:
                score = rng.uniform(0.15, 0.45)       # occluded by object 2
            if k == 4 and 30 <= f <= 37:
                score = rng.uniform(0.15, 0.45)       # behind the pole
            if rng.random() < 0.05:
                continue                               # random missed detection
            noisy = b + np.r_[rng.normal(0, 2.0, 2), rng.normal(0, 1.0, 2)]
            e = feats[k] + 0.15 * rng.normal(size=16)
            d.append((noisy, score, k, e / np.linalg.norm(e)))
        if rng.random() < 0.3:                         # low-score false positive
            fp = np.r_[rng.uniform(50, 550), rng.uniform(50, 320), W, H]
            e = rng.normal(size=16)
            d.append((fp, rng.uniform(0.1, 0.4), -1, e / np.linalg.norm(e)))
        gt.append(g); dets.append(d)
    return gt, dets
```

The tracker below has three switches. With both off it is **SORT-style**: Kalman prediction, IoU cost, Hungarian matching on high-score detections (score ≥ 0.5), tracks deleted after 2 missed

frames. `use_low=True` adds a **BYTE**-style second pass: tracks left unmatched try the low-score detections, with IoU only. `use_app=True` adds the **appearance** cost to the first pass and a third pass that re-identifies tracks lost for up to 30 frames by appearance alone.

```
# ---------- 4. SORT-style tracker with optional BYTE and appearance ----------
class Track:
    def __init__(self, tid, z, emb):
        self.id, self.kf, self.emb = tid, KalmanBox(z), emb
        self.lost = 0

def run_tracker(dets, use_low=False, use_app=False, hi=0.5, max_age=2,
                app_age=30, lam=0.5):
    tracks, out, next_id = [], [], 0
    for d in dets:
        for tk in tracks:
            tk.kf.predict()
        boxes = np.array([x[0] for x in d]).reshape(-1, 4)
        scores = np.array([x[1] for x in d])
        embs = np.array([x[3] for x in d]).reshape(-1, 16)
        high = np.where(scores >= hi)[0]
        low = np.where((scores < hi) & (scores >= 0.1))[0]
        active = [tk for tk in tracks if tk.lost <= max_age]
        pred = np.array([tk.kf.x[:4] for tk in active]).reshape(-1, 4)
        # stage 1: high-score detections vs active tracks
        cost = 1 - iou_matrix(pred, boxes[high])
        gate = 0.7
        if use_app and len(active) and len(high):
            tE = np.array([tk.emb for tk in active])
            app = 1 - tE @ embs[high].T            # cosine distance
            cost = np.where(cost > 0.9, 9.0, lam * cost + (1 - lam) * app)
            gate = 0.6
        pairs, ut, ud = associate(cost, gate)
        matched = [(active[r], high[c]) for r, c in pairs]
        rest_tracks = [active[r] for r in ut]
        rest_high = [high[c] for c in ud]
        # stage 2 (BYTE): remaining tracks vs low-score detections, IoU only
        if use_low and rest_tracks and len(low):
            p2 = np.array([tk.kf.x[:4] for tk in rest_tracks])
            pairs2, ut2, _ = associate(1 - iou_matrix(p2, boxes[low]), 0.5)
            matched += [(rest_tracks[r], low[c]) for r, c in pairs2]
            rest_tracks = [rest_tracks[r] for r in ut2]
        # stage 3 (appearance only): long-lost tracks vs leftover high-score dets
        if use_app and rest_high:
            lost = [tk for tk in tracks if max_age < tk.lost <= app_age]
            if lost:
                app = 1 - np.array([tk.emb for tk in lost]) @ embs[rest_high].T
                pairs3, _, ud3 = associate(app, 0.3)
                matched += [(lost[r], rest_high[c]) for r, c in pairs3]
                rest_high = [rest_high[c] for c in ud3]
        for tk, j in matched:
            tk.kf.update(boxes[j]); tk.lost = 0
            tk.emb = 0.9 * tk.emb + 0.1 * embs[j]; tk.emb /= np.linalg.norm(tk.emb)
        hit = {id(tk) for tk, _ in matched}
```

```
        for tk in tracks:
            if id(tk) not in hit:
                tk.lost += 1
        for j in rest_high:                          # birth: unmatched high-score
            tracks.append(Track(next_id, boxes[j], embs[j])); next_id += 1
        tracks = [tk for tk in tracks if tk.lost <= (app_age if use_app else max_age)]
        out.append([(tk.id, tk.kf.x[:4].copy()) for tk in tracks if tk.lost == 0])
    return out
```

The metric code (MOTA, IDF1 and HOTA, explained below) and the experiment:

```
# ---------- 5. metrics: MOTA, IDF1, HOTA ----------
def frame_match(g, p, thr):
    if not g or not p:
        return []
    iou = iou_matrix(np.array([b for _, b in g]), np.array([b for _, b in p]))
    r, c = linear_sum_assignment(-iou)
    return [(g[i][0], p[j][0]) for i, j in zip(r, c) if iou[i, j] >= thr]

def mota(gt, hyp, thr=0.5):
    fn = fp = idsw = 0; ngt = 0; last = {}
    for g, p in zip(gt, hyp):
        m = frame_match(g, p, thr)
        ngt += len(g); fn += len(g) - len(m); fp += len(p) - len(m)
        for gi, pi in m:
            if gi in last and last[gi] != pi:
                idsw += 1
            last[gi] = pi
    return 1 - (fn + fp + idsw) / ngt, idsw

def idf1(gt, hyp, thr=0.5):
    gids = sorted({i for g in gt for i, _ in g}); pids = sorted({i for p in hyp for i, _ in
    ↪  p})
    gi = {k: n for n, k in enumerate(gids)}; pi = {k: n for n, k in enumerate(pids)}
    ov = np.zeros((len(gids), len(pids)))
    for g, p in zip(gt, hyp):
        if g and p:
            iou = iou_matrix(np.array([b for _, b in g]), np.array([b for _, b in p]))
            for a in range(len(g)):
                for b in range(len(p)):
                    ov[gi[g[a][0]], pi[p[b][0]]] += iou[a, b] >= thr
    r, c = linear_sum_assignment(-ov)
    idtp = ov[r, c].sum()
    n_gt = sum(len(g) for g in gt); n_p = sum(len(p) for p in hyp)
    return 2 * idtp / (n_gt + n_p)

def hota(gt, hyp, alphas=np.arange(0.05, 0.96, 0.05)):
    res = []
    for a in alphas:
        pairs = []; n_gt = n_p = 0
        for g, p in zip(gt, hyp):
            pairs += frame_match(g, p, a); n_gt += len(g); n_p += len(p)
```

```
        tp = len(pairs)
        if tp == 0:
            res.append((0, 0, 0)); continue
        cnt = {}
        for q in pairs:
            cnt[q] = cnt.get(q, 0) + 1
        gcount = {}; pcount = {}
        for g in gt:
            for i, _ in g: gcount[i] = gcount.get(i, 0) + 1
        for p in hyp:
            for i, _ in p: pcount[i] = pcount.get(i, 0) + 1
        ass = sum(cnt[q] / (gcount[q[0]] + pcount[q[1]] - cnt[q]) for q in pairs) / tp
        det = tp / (n_gt + n_p - tp)
        res.append((np.sqrt(det * ass), det, ass))
    return np.array(res).mean(axis=0)   # HOTA, DetA, AssA averaged over alpha

gt, dets = make_scene(seed=3)
for name, kw in [("IoU only", {}), ("+ low-score pass", dict(use_low=True)),
                 ("+ appearance", dict(use_low=True, use_app=True))]:
    hyp = run_tracker(dets, **kw)
    m, sw = mota(gt, hyp)
    print(f"{name:18s} MOTA {m:.2f}  IDF1 {idf1(gt, hyp):.2f}  "
          f"HOTA {hota(gt, hyp)[0]:.2f}  ID switches {sw}")
```

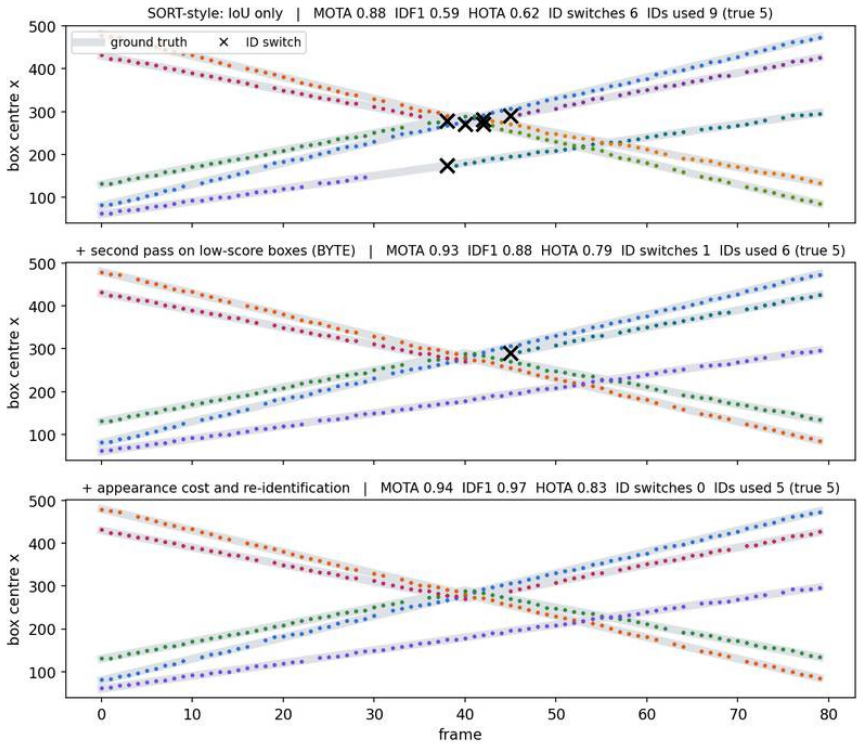


Figure 5 — The same synthetic detections tracked three ways. Colors are predicted IDs; black crosses mark ID switches. Low-score association removes fragmentation from occlusion; appearance fixes the bounce,

where constant-velocity prediction points the wrong way.

The script prints:

Tracker	MOTA	IDF1	HOTA	ID switches	IDs used (true: 5)
SORT-style, IoU only	0.88	0.59	0.62	6	9
+ BYTE low-score pass	0.93	0.88	0.79	1	6
+ appearance and re-identification	0.94	0.97	0.83	0	5

Read it carefully. The IoU-only tracker loses the occluded objects because their detections fall below the score threshold, so their tracks die and come back with **new IDs** (fragmentation). The low-score pass keeps them alive, which fixes almost everything, but it cannot fix the bounce: the Kalman filter predicts the two dancers will keep going, so after the overlap one of them is not matched and later restarts with a new ID. Only appearance resolves that. Notice also how little MOTA moves (0.88 → 0.94) compared with IDF1 (0.59 → 0.97): MOTA is mostly a detection measure, a point we return to in the metrics section.

The SORT family

These ideas map directly onto the most cited online trackers:

Tracker	Key idea
SORT [32]	Kalman filter + IoU cost + Hungarian assignment; deliberately minimal and very fast. Showed that detector quality dominates tracking quality.
DeepSORT [33]	Adds a CNN appearance descriptor trained on a person Re-ID dataset, Mahalanobis gating, and a *matching cascade* that gives priority to recently seen tracks. Fewer ID switches through longer occlusions.
ByteTrack [34]	Associates **every** detection box: high-score boxes first, then the remaining tracks against **low-score** boxes using IoU only. Occluded objects often get low scores, so they are recovered instead of discarded, while low-score background boxes stay unmatched and are thrown away.
OC-SORT [35]	"Observation-centric": when a lost track is re-found, it re-runs the Kalman updates along a virtual trajectory between the last and new observations (fixing error accumulated while coasting), and adds a motion-direction consistency term to the cost. Strong on non-linear motion such as DanceTrack.

Tracker	Key idea
BoT-SORT [36]	ByteTrack-style association plus camera-motion compensation (global image registration), a Kalman state with w, h instead of aspect ratio, and a fused IoU/Re-ID cost.

The lesson of this family is that *carefully engineered association on top of a strong detector* is very hard to beat on pedestrian benchmarks. The 2025 survey of Adžemović [4] reaches the same conclusion: heuristic methods lead on dense scenes with mostly linear motion, while learned association does better when motion is complex.

Joint detection and embedding, and detector-based tracking

Running a separate Re-ID network on every crop is expensive. **JDE** [37] adds an embedding head to a one-stage detector so that boxes and appearance vectors come out of one forward pass. **FairMOT** [38] argued that this sharing had been "unfair" to Re-ID: anchor-based detectors produce features that are ambiguous for identity, and detection dominates training. It uses an anchor-free, CenterNet-style detector with two equal branches (detection and Re-ID) on a high-resolution feature map. **Tracktor** [39], which the original post also cites, removes explicit association for most cases: it feeds the previous frame's boxes to the detector's box-regression head, which pulls each box onto the object's new position and keeps its identity.

Tracking by query: end-to-end MOT

Plain version. Instead of hand-writing "predict, compare, match", give a transformer a memory slot per object and let it learn to update the slot each frame.

DETR-style detectors (see DLCV 03) represent objects as **queries**. MOTR [40] extends this to video: each detected object's output embedding becomes a **track query** that is passed to the next frame and keeps predicting the same object, while new *detect queries* find newborn objects. Identity is carried implicitly by the query; there is no Kalman filter and no Hungarian matching at inference. Training uses a tracklet-aware label assignment and a loss averaged over clips. The weakness was detection quality: one decoder must both detect and associate, and the two tasks conflict. MOTRv2 [41] bootstraps MOTR with proposals from a separate pretrained detector (YOLOX) used as anchors, which eases that conflict. MOTIP [42] (CVPR 2025) reformulates association as **ID prediction**: given the trajectories so far, each with an ID label drawn from a learnable ID dictionary, a decoder classifies each new detection into one of the existing IDs or "new". This is in-context, end-to-end trainable, and works with plain object-level features.

Open-vocabulary MOT

Classic MOT only tracks the classes in its training set. **OVTrack** [43] (CVPR 2023) tracks arbitrary categories by distilling knowledge from a vision-language model (CLIP) for both classification and association, and by learning robust appearance features from *hallucinated* image pairs produced with a diffusion model. SAM 3's concept prompts [25] approach the same goal from the segmentation side.

Datasets

Dataset	Domain	What it tests
MOT17 [44]	pedestrians, static and moving cameras	the standard pedestrian benchmark; the MOT16 sequences re-released as MOT17
MOT20 [45]	very crowded pedestrian scenes	heavy occlusion, small targets
DanceTrack [46]	group dancing	uniform appearance and diverse, non-linear motion; appearance cannot separate people
SportsMOT [47]	basketball, volleyball, football (240 sequences)	fast, variable-speed motion with similar but distinguishable appearance
BDD100K MOT [48]	driving video	multi-class tracking (cars, pedestrians, cyclists, ...) from a moving vehicle

The shift from MOT17 to DanceTrack and SportsMOT changed the field's priorities: on MOT17 a good detector and IoU matching go far, but on DanceTrack the motion model and association learning matter most.

Metrics: MOTA, IDF1, HOTA

Plain version. A tracker can fail in two ways: it can miss objects or invent them (detection errors), or it can mix up who is who (association errors). Different metrics weigh these two differently.

All three metrics first match predictions to ground truth in each frame at an IoU threshold. Let GT_t, FN_t, FP_t and IDSW_t be the number of ground-truth objects, misses, false positives and identity switches in frame t. An identity switch occurs when a ground-truth object is matched to a different predicted ID than at its previous match.

MOTA (CLEAR MOT [9]):

$$\mathrm{MOTA} = 1 - \frac{\sum_t (\mathrm{FN}_t + \mathrm{FP}_t + \mathrm{IDSW}_t)}{\sum_t \mathrm{GT}_t}.$$

It ranges from $-\infty$ to 1. Because there are usually far more detections than switches, FN and FP dominate it: MOTA is mostly a detection score. A switch is also counted only at the moment it happens, so a tracker that swaps two IDs once and keeps them swapped for 1,000 frames pays the same as one that swaps back after 1 frame.

IDF1 [10] computes a *global* one-to-one matching between ground-truth trajectories and predicted trajectories (again with the Hungarian algorithm) that maximizes the number of frames where they agree, then

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}},$$

where IDTP counts detections matched to the trajectory assigned to their ground-truth identity, and IDFP, IDFN the remaining predicted and ground-truth detections. IDF1 measures how long the tracker keeps the right identity, so it is mostly an association score — but because the matching is global, it can behave unintuitively, for example dropping when detection improves.

HOTA (Higher Order Tracking Accuracy [8]) was introduced because neither metric is balanced or decomposable. For a localization threshold α, with per-frame matches giving true positives (TP), FN and FP:

$$\mathrm{DetA}_\alpha = \frac{|\mathrm{TP}|}{|\mathrm{TP}| + |\mathrm{FN}| + |\mathrm{FP}|}, \qquad \mathrm{AssA}_\alpha = \frac{1}{|\mathrm{TP}|} \sum_{c \in \mathrm{TP}} \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|},$$

$$\mathrm{HOTA}_\alpha = \sqrt{\mathrm{DetA}_\alpha \cdot \mathrm{AssA}_\alpha}, \qquad \mathrm{HOTA} = \frac{1}{19} \sum_{\alpha \in \{0.05,\, 0.10,\, \ldots,\, 0.95\}} \mathrm{HOTA}_\alpha.$$

For a matched pair c (ground-truth ID g, predicted ID p), $\mathrm{TPA}(c)$ is the set of all TPs in the video that also match g to p; $\mathrm{FNA}(c)$ the detections of g matched to another ID or missed; $\mathrm{FPA}(c)$ the detections of p matched to another object or to nothing. The geometric mean means a tracker must be good at *both* detection and association to score well, and DetA and AssA (and a localization accuracy, LocA) can be reported separately to diagnose failures. Averaging over α also rewards precise boxes. (Our code uses a simplified matching that maximizes IoU per frame; the official definition matches with a score that also favors pairs that have been associated before.)

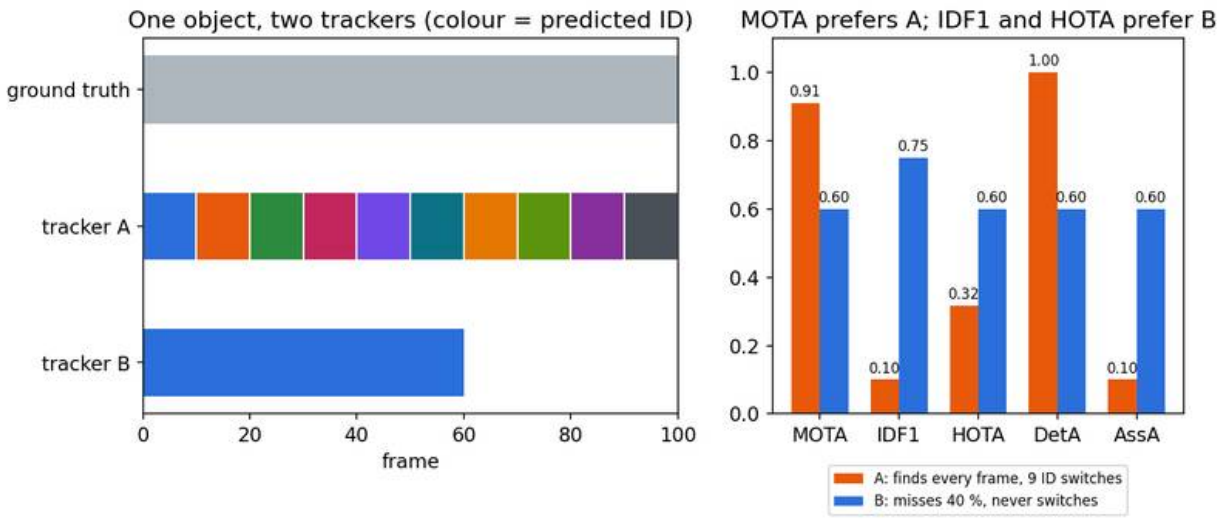


Figure 6 — Why HOTA was introduced. Tracker A detects every frame but switches identity nine times; tracker B misses 40% of frames but never switches. MOTA ranks A far ahead; IDF1 and HOTA rank B ahead, and DetA/AssA show exactly why.

Figure 6 is computed with the functions above: tracker A gets MOTA $= 1 - 9/100 = 0.91$ and AssA $= 0.10$, giving HOTA $= \sqrt{1.0 \times 0.10} \approx 0.32$; tracker B gets 0.60 on MOTA, DetA, AssA and HOTA, and IDF1 $= 2 \cdot 60/(100 + 60) = 0.75$. Today, papers on MOT17, MOT20, DanceTrack and SportsMOT report HOTA, IDF1 and MOTA together, and HOTA is usually the primary ranking metric.

Point tracking: a related trend

Plain version. Instead of following whole objects, follow individual *points* — a freckle on a cheek, a corner of a box — through the whole video, even when they are briefly hidden.

Tracking any point (TAP) asks, for a query point (x, y) in frame t_0, for its position and visibility in every other frame. It generalizes optical flow (two frames) to long, occlusion-aware trajectories, and is a natural successor to the classical keypoint tracking from IP 12. TAPIR [49] first matches the query feature independently in every frame to get a rough trajectory, then refines it with temporal convolutions. CoTracker [50] tracks many points *jointly* with a transformer, so points help one another: a point that becomes occluded can be inferred from its visible neighbors. CoTracker3 [51] simplifies the architecture and trains on real videos pseudo-labeled by existing trackers, instead of only synthetic data. Point trackers are now used as motion cues inside object trackers, video editing and robotics.

Multi-target multi-camera tracking

Plain version. Many cameras watch one place. Track everyone in each camera, then figure out that the woman leaving camera 3 is the same woman entering camera 5.

MTMCT is the ultimate goal named in [1]. A typical pipeline runs single-camera MOT to produce *tracklets*, embeds each tracklet with Re-ID features, and clusters tracklets across cameras using appearance, camera topology (which cameras connect, and travel times between them) and, with calibrated cameras, 3D positions on a ground plane. IDF1 was originally proposed for this setting [10]. The **AI City Challenge** is the main recurring benchmark. Its ninth edition (2025) [52] made Track 1 a multi-class **3D** multi-camera tracking task in warehouse-style environments, with people, humanoid robots, autonomous mobile robots and forklifts, calibrated cameras and 3D box annotations. That shift — from image boxes to calibrated 3D tracks across many views — is where MTMCT is heading.

Modern view

What the surveys say

- **Luo et al., "Multiple object tracking: A literature review" [3]** (*Artificial Intelligence*, 2021; first released 2014). The classical reference. It defines the MOT problem formally, categorizes methods by initialization (detection-based versus detection-free), processing mode (online versus offline) and output type (deterministic versus probabilistic), and analyzes the components — appearance model, motion model, interaction model, exclusion constraints, occlusion handling — and the inference methods. Read it to understand the problem independent of any network.
- **Ciaparrone et al., "Deep learning in video multi-object tracking: A survey" [2]** (*Neurocomputing*, 2020). Organizes deep MOT around four stages — detection, feature extraction/motion prediction, affinity computation and association — and reviews how deep learning entered each. Its main takeaway, from comparisons on MOTChallenge, is that detection quality and learned appearance features drive most gains.
- **Adžemović, "Deep learning-based multi-object tracking: A comprehensive survey from foundations to state-of-the-art" [4]** (arXiv, 2025). Splits tracking-by-detection into joint detection-and-embedding, heuristic-based, motion-based, affinity-learning and offline methods, and contrasts them with end-to-end trackers. It identifies 2022 (ByteTrack and MOTR) as the acceleration point and concludes that heuristic trackers lead on crowded, mostly linear scenes while learned association wins on complex motion.
- **Marvasti-Zadeh et al., "Deep learning for visual tracking: A comprehensive survey" [5]** (*IEEE T-ITS*, 2022). For SOT. Analyzes deep trackers along nine aspects (network architecture,

exploitation, training, objective, output, use of correlation filters, aerial-view, long-term and online tracking) and compares them on OTB, VOT, LaSOT and aerial benchmarks.

- **Thangavel (Kugarajeevan) et al., "Transformers in single object tracking: An experimental survey" [6]** (*IEEE Access*, 2023). Classifies transformer trackers as CNN-Transformer hybrids, two-stream two-stage, and one-stream one-stage fully-transformer trackers, and evaluates their robustness and efficiency; the one-stream design (OSTrack-style) emerges as the dominant pattern.
- **Ye et al., "Deep learning for person re-identification: A survey and outlook" [7]** (*IEEE TPAMI*, 2022). Separates closed-world Re-ID (feature learning, metric learning, ranking) from open-world Re-ID (heterogeneous data, raw images, noisy labels, unsupervised and open-set settings), proposes the AGW baseline and the mINP metric, which measures the cost of finding *all* correct matches.
- **Luiten et al., "HOTA" [8]** (*IJCV*, 2020). Not a survey, but it re-examines the evaluation of the entire field, shows the biases of MOTA and IDF1 with worked examples and user studies, and is the reason current leaderboards rank by HOTA.

State of the art, 2024–2026

- **SOT** is dominated by one-stream transformer trackers in the OSTrack/SeqTrack lineage and, increasingly, by segmentation foundation models: SAM 2 with motion-aware memory (SAMURAI) gives strong zero-shot tracking without tracking-specific training [23], [24].
- **MOT** has two coexisting camps. Heuristic tracking-by-detection (ByteTrack, OC-SORT, BoT-SORT and descendants) remains the default in practice: simple, fast and modular, with a strong detector doing most of the work. End-to-end trackers (MOTR, MOTRv2, MOTIP) lead where motion is complex and appearance is ambiguous, such as DanceTrack and SportsMOT [4], [42].
- **Open-vocabulary and promptable tracking** is moving into mainstream models: OVTrack established the task, and SAM 3 detects, segments and tracks instances of a text concept with identities [25], [43].
- **Point tracking** (TAPIR, CoTracker, CoTracker3) matured into a general motion primitive [49]–[51].
- **MTMCT** is moving to calibrated, multi-class 3D tracking (AI City Challenge 2025) [52].

Open problems

- **Long-term occlusion and re-entry.** Motion models are useless after a few seconds; appearance must carry identity, but appearance is unreliable for uniform clothing (DanceTrack, sports) and across cameras.

- **Non-linear motion and camera motion.** Constant-velocity Kalman filters remain the default and are the weak point on dance, sports and driving data; learned motion models are not yet clearly better everywhere.
- **End-to-end versus heuristic association.** End-to-end trackers are elegant and win on complex motion, but need lots of video training data and compute and still lose to simple heuristics on crowded pedestrian scenes. A unified design that is best on both is open.
- **Open-vocabulary and long-tail tracking.** Rare classes, and appearance features that generalize to classes never seen in training.
- **Evaluation.** HOTA fixed much, but benchmarks are pedestrian-heavy, and ID-level evaluation over hours of video and many cameras remains hard. **Privacy and dataset ethics** (the DukeMTMC lesson) constrain what data can be collected at all.

Key takeaways

- Tracking = detection + time: every tracker must both follow motion and keep identities, and the balance between the two defines SOT, MOT and Re-ID.
- Re-ID is retrieval with learned embeddings; report both CMC rank-k and mAP; DukeMTMC was withdrawn in 2019 and should not be used.
- SOT went from correlation filters (fast, hand-crafted) to Siamese matching (offline-trained) to one-stream transformers; SAM 2-style video segmentation now does SOT with masks, and SAM 3 extends it to many instances from a text prompt.
- Online MOT is a loop of detect → Kalman predict → cost matrix → Hungarian assignment → update/birth/death. Most practical gains come from what goes into the cost matrix and which detections are allowed to match.
- ByteTrack's low-score pass fixes occlusion fragmentation; appearance fixes non-linear motion and long gaps; end-to-end query trackers learn association directly.
- MOTA mostly measures detection, IDF1 mostly association; HOTA $= \sqrt{\text{DetA} \cdot \text{AssA}}$ (averaged over IoU thresholds) balances and decomposes both.
- The open frontier is long-term identity, non-linear motion, open-vocabulary categories and multi-camera 3D tracking.

Exercises

1. **Kalman gain intuition.** For a 1-D constant-position model with prior variance P and measurement noise R, show that the Kalman gain is $K = P/(P + R)$ and that the posterior variance is $PR/(P + R)$. What happens to K after five frames without detections if each predict step adds Q to P?

   Hint

With $H = 1$, $S = P + R$ and $K = P/S$. The posterior variance is $(1 - K)P = PR/(P + R)$, smaller than both P and R. After five coasting steps the prior is $P + 5Q$, so K moves toward 1: the filter will trust the next detection much more, which is why a wrong match after an occlusion moves the track a lot.

2. **Hungarian by hand.** Tracks A and B have predicted IoUs with detections 1 and 2 of $\begin{bmatrix} 0.6 & 0.5 \\ 0.55 & 0.0 \end{bmatrix}$. What does greedy matching (highest IoU first) return, and what does the Hungarian algorithm on cost $1 - \text{IoU}$ return? Which total IoU is larger?

Hint

Greedy takes A–1 (0.6) first, leaving B–2 with IoU 0 (rejected), total 0.6. Hungarian picks A–2 and B–1, total $0.5 + 0.55 = 1.05$. Greedy leaves track B unmatched and starts a new ID for detection 2; this is one way ID switches appear.

3. **Break the tracker.** In the synthetic scene, change the bounce in `make_scene` so that objects 2 and 3 pass through each other instead (no velocity reversal). Which of the three trackers improves the most, and why?

Hint

With linear motion the constant-velocity prediction is correct through the overlap. With `seed=3`, the low-score tracker's IDF1 rises from 0.88 to about 0.96 and it uses the correct 5 IDs, almost matching the appearance tracker (about 0.96 as well); the IoU-only tracker still fragments occluded objects. Appearance helps most exactly when motion is non-linear, which is the design rationale of DanceTrack.

4. **Metric arithmetic.** A video has one object for 200 frames. A tracker detects it in all frames but uses ID 1 for frames 1–100 and ID 2 for frames 101–200. Compute MOTA, IDF1, DetA, AssA and HOTA at a single threshold, assuming perfect boxes.

Hint

$\text{MOTA} = 1 - 1/200 = 0.995$. The best global ID match covers 100 frames: $\text{IDF1} = 2 \cdot 100/(200 + 200) = 0.5$. $\text{DetA} = 1$. For each TP, $|\text{TPA}| = 100$, $|\text{FNA}| = 100$, $|\text{FPA}| = 0$, so $\text{AssA} = 0.5$ and $\text{HOTA} = \sqrt{0.5} \approx 0.71$. One swap barely moves MOTA but halves AssA.

5. **Re-ID metrics.** In the Re-ID snippet, change the noise level from 1.6 to 2.2 and then to 1.0. How do rank-1 and mAP change, and which one is more sensitive? Explain using the definition of AP.

Hint

mAP falls faster than rank-1 as noise grows, because rank-1 only needs the single easiest true match at the top, while AP penalizes every true match that is pushed down the list. With 4 true matches per query, the hardest one usually decides how much AP is lost.

6. **Design question.** You must track forklifts and workers across 12 calibrated cameras in a warehouse. Sketch a pipeline using components from this tutorial, and say which metric you would report and why.

Hint

Per-camera detection plus a ByteTrack/BoT-SORT-style tracker; project boxes to the ground plane using calibration; cross-camera association by 3D position and time first, Re-ID appearance second (workers may wear identical vests); handle the two classes separately. Report HOTA (balanced, decomposable into DetA/AssA) and IDF1 (long-term identity across cameras), as in the AI City Challenge setting.

References

1. Shyandram, "物件追蹤 Object Tracking 簡介" (Introduction to object tracking), Medium post (in Traditional Chinese), 2023; updated 2026. link
2. G. Ciaparrone, F. Luque Sánchez, S. Tabik, L. Troiano, R. Tagliaferri and F. Herrera, "Deep learning in video multi-object tracking: A survey," *Neurocomputing*, 2020. arXiv:1907.12740
3. W. Luo, J. Xing, A. Milan, X. Zhang, W. Liu and T.-K. Kim, "Multiple object tracking: A literature review," *Artificial Intelligence*, vol. 293, 2021 (arXiv 2014). arXiv:1409.7618
4. M. Adžemović, "Deep learning-based multi-object tracking: A comprehensive survey from foundations to state-of-the-art," arXiv:2506.13457, 2025. arXiv
5. S. M. Marvasti-Zadeh, L. Cheng, H. Ghanei-Yakhdan and S. Kasaei, "Deep learning for visual tracking: A comprehensive survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, 2022. arXiv:1912.00535
6. J. Thangavel (Kugarajeevan), T. Kokul, A. Ramanan and S. Fernando, "Transformers in single object tracking: An experimental survey," *IEEE Access*, vol. 11, 2023. arXiv:2302.11867
7. M. Ye, J. Shen, G. Lin, T. Xiang, L. Shao and S. C. H. Hoi, "Deep learning for person re-identification: A survey and outlook," *IEEE TPAMI*, vol. 44, 2022. arXiv:2001.04193
8. J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé and B. Leibe, "HOTA: A higher order metric for evaluating multi-object tracking," *International Journal of Computer Vision*, 2020. arXiv:2009.07736
9. K. Bernardin and R. Stiefelhagen, "Evaluating multiple object tracking performance: The CLEAR MOT metrics," *EURASIP Journal on Image and Video Processing*, 2008. doi
10. E. Ristani, F. Solera, R. S. Zou, R. Cucchiara and C. Tomasi, "Performance measures and a data set for multi-target, multi-camera tracking," *ECCV Workshop on Benchmarking Multi-Target Tracking*, 2016. arXiv:1609.01775
11. K. Peng, A. Mathur and A. Narayanan, "Mitigating dataset harms requires stewardship: Lessons from 1000 papers," arXiv:2108.02922, 2021. arXiv
12. L. Zheng, L. Shen, L. Tian, S. Wang, J. Wang and Q. Tian, "Scalable person re-identification: A benchmark," *ICCV*, 2015. doi
13. L. Wei, S. Zhang, W. Gao and Q. Tian, "Person transfer GAN to bridge domain gap for person re-identification," *CVPR*, 2018. arXiv:1711.08565

14. H. Luo, Y. Gu, X. Liao, S. Lai and W. Jiang, "Bag of tricks and a strong baseline for deep person re-identification," *CVPR Workshops*, 2019. arXiv:1903.07071
15. M. Wieczorek, B. Rychalska and J. Dąbrowski, "On the unreasonable effectiveness of centroids in image retrieval," arXiv:2104.13643, 2021. arXiv
16. S. Li, L. Sun and Q. Li, "CLIP-ReID: Exploiting vision-language model for image re-identification without concrete text labels," *AAAI*, 2023. arXiv:2211.13977
17. D. S. Bolme, J. R. Beveridge, B. A. Draper and Y. M. Lui, "Visual object tracking using adaptive correlation filters," *CVPR*, 2010. doi
18. J. F. Henriques, R. Caseiro, P. Martins and J. Batista, "High-speed tracking with kernelized correlation filters," *IEEE TPAMI*, vol. 37, 2015. arXiv:1404.7584
19. L. Bertinetto, J. Valmadre, J. F. Henriques, A. Vedaldi and P. H. S. Torr, "Fully-convolutional Siamese networks for object tracking," arXiv:1606.09549, 2016 (ECCV 2016 workshops). arXiv
20. B. Li, J. Yan, W. Wu, Z. Zhu and X. Hu, "High performance visual tracking with Siamese region proposal network," *CVPR*, 2018. doi
21. B. Ye, H. Chang, B. Ma, S. Shan and X. Chen, "Joint feature learning and relation modeling for tracking: A one-stream framework" (OSTrack), *ECCV*, 2022. arXiv:2203.11991
22. X. Chen, H. Peng, D. Wang, H. Lu and H. Hu, "SeqTrack: Sequence to sequence learning for visual object tracking," *CVPR*, 2023 (the arXiv entry was later extended as SeqTrackv2). arXiv:2304.14394
23. N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma et al., "SAM 2: Segment anything in images and videos," *ICLR*, 2025. arXiv:2408.00714
24. C.-Y. Yang, H.-W. Huang, W. Chai, Z. Jiang and J.-N. Hwang, "SAMURAI: Adapting segment anything model for zero-shot visual tracking with motion-aware memory," arXiv:2411.11922, 2024. arXiv
25. N. Carion, L. Gustafson, Y.-T. Hu et al., "SAM 3: Segment anything with concepts," *ICLR*, 2026. arXiv:2511.16719
26. Y. Wu, J. Lim and M.-H. Yang, "Object tracking benchmark," *IEEE TPAMI*, vol. 37, 2015. doi
27. H. Fan, L. Lin, F. Yang, P. Chu, G. Deng, S. Yu et al., "LaSOT: A high-quality benchmark for large-scale single object tracking," *CVPR*, 2019. arXiv:1809.07845
28. L. Huang, X. Zhao and K. Huang, "GOT-10k: A large high-diversity benchmark for generic object tracking in the wild," *IEEE TPAMI*, vol. 43, 2021. arXiv:1810.11981
29. M. Müller, A. Bibi, S. Giancola, S. Al-Subaihi and B. Ghanem, "TrackingNet: A large-scale dataset and benchmark for object tracking in the wild," arXiv:1803.10794, 2018 (ECCV 2018). arXiv
30. R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960. doi
31. H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, pp. 83–97, 1955. doi
32. A. Bewley, Z. Ge, L. Ott, F. Ramos and B. Upcroft, "Simple online and realtime tracking," *ICIP*, 2016. arXiv:1602.00763
33. N. Wojke, A. Bewley and D. Paulus, "Simple online and realtime tracking with a deep association metric," *ICIP*, 2017. doi
34. Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," *ECCV*, 2022. arXiv:2110.06864

35. J. Cao, J. Pang, X. Weng, R. Khirodkar and K. Kitani, “Observation-centric SORT: Rethinking SORT for robust multi-object tracking,” *CVPR*, 2023. arXiv:2203.14360
36. N. Aharon, R. Orfaig and B.-Z. Bobrovsky, “BoT-SORT: Robust associations multi-pedestrian tracking,” arXiv:2206.14651, 2022. arXiv
37. Z. Wang, L. Zheng, Y. Liu, Y. Li and S. Wang, “Towards real-time multi-object tracking” (JDE), *ECCV*, 2020. arXiv:1909.12605
38. Y. Zhang, C. Wang, X. Wang, W. Zeng and W. Liu, “FairMOT: On the fairness of detection and re-identification in multiple object tracking,” *IJCV*, 2021. arXiv:2004.01888
39. P. Bergmann, T. Meinhardt and L. Leal-Taixé, “Tracking without bells and whistles” (Tracktor), *ICCV*, 2019. arXiv:1903.05625
40. F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang and Y. Wei, “MOTR: End-to-end multiple-object tracking with transformer,” *ECCV*, 2022. arXiv:2105.03247
41. Y. Zhang, T. Wang and X. Zhang, “MOTRv2: Bootstrapping end-to-end multi-object tracking by pretrained object detectors,” *CVPR*, 2023. arXiv:2211.09791
42. R. Gao, J. Qi and L. Wang, “Multiple object tracking as ID prediction” (MOTIP), *CVPR*, 2025. arXiv:2403.16848
43. S. Li, T. Fischer, L. Ke, H. Ding, M. Danelljan and F. Yu, “OVTrack: Open-vocabulary multiple object tracking,” *CVPR*, 2023. arXiv:2304.08408
44. A. Milan, L. Leal-Taixé, I. Reid, S. Roth and K. Schindler, “MOT16: A benchmark for multi-object tracking,” arXiv:1603.00831, 2016 (MOT16/MOT17). arXiv
45. P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler and L. Leal-Taixé, “MOT20: A benchmark for multi object tracking in crowded scenes,” arXiv:2003.09003, 2020. arXiv
46. P. Sun, J. Cao, Y. Jiang, Z. Yuan, S. Bai, K. Kitani and P. Luo, “DanceTrack: Multi-object tracking in uniform appearance and diverse motion,” *CVPR*, 2022. arXiv:2111.14690
47. Y. Cui, C. Zeng, X. Zhao, Y. Yang, G. Wu and L. Wang, “SportsMOT: A large multi-object tracking dataset in multiple sports scenes,” *ICCV*, 2023. arXiv:2304.05170
48. F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan and T. Darrell, “BDD100K: A diverse driving dataset for heterogeneous multitask learning,” *CVPR*, 2020. arXiv:1805.04687
49. C. Doersch, Y. Yang, M. Vecerik, D. Gokay, A. Gupta, Y. Aytar et al., “TAPIR: Tracking any point with per-frame initialization and temporal refinement,” *ICCV*, 2023. arXiv:2306.08637
50. N. Karaev, I. Rocco, B. Graham, N. Neverova, A. Vedaldi and C. Rupprecht, “CoTracker: It is better to track together,” *ECCV*, 2024. arXiv:2307.07635
51. N. Karaev, I. Makarov, J. Wang, N. Neverova, A. Vedaldi and C. Rupprecht, “CoTracker3: Simpler and better point tracking by pseudo-labelling real videos,” arXiv:2410.11831, 2024. arXiv
52. Z. Tang, S. Wang, D. C. Anastasiu et al., “The 9th AI City Challenge,” *ICCV Workshops*, 2025. arXiv:2508.13564
53. VOT Challenge organizers, “The Visual Object Tracking (VOT) challenge series,” website, 2013–present. votchallenge.net

DLCV 07 • World Models: Video, Latent Dynamics and Planning

What a world model is, how model-based reinforcement learning uses one (World Models, the Dreamer line, MuZero), why video generators are proposed as world simulators, how latent dynamics and JEPA-style predictive representations work, and how to evaluate all of them, with complete CPU experiments.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-07-world-models/

What you'll learn

- What a world model is, and the difference between an observation, a hidden state, a belief and an action.
- How model-based reinforcement learning uses a learned model in three different ways: planning online, learning a policy in imagination, and tree search with a value-equivalent model, with World Models, PlaNet, the Dreamer line, TD-MPC2 and MuZero as examples.
- Why video prediction and generation are proposed as "world simulators", why squared-error predictors produce blur, and what interactive generators such as Genie can and cannot show.
- How recurrent state-space models and JEPA-style predictive representations learn latent dynamics without (or with) reconstructing pixels.
- How to fit a small dynamics model, measure multi-step error, plan with it, and detect when a planner exploits model errors, all on a CPU.
- How to evaluate a world model, and which problems remain open.

The big picture

Tracking asks where an object went. A world model asks what could happen next, and especially what would happen **if an agent took a particular action**. A robot approaching a cup needs more than an object label: it needs to predict how its gripper and the cup will move, which outcomes are uncertain, and which action is likely to reach its goal.

> **Explain like I'm five**
>
> Before pushing a toy car, you imagine where it will go. You can imagine a gentle push and a strong push, pick the better one, push, and then look again to see what really happened. A world model is a learned version of that imagination. A planner is the part that tries out pushes in imagination and picks one. Looking again after each push is what keeps small imagination mistakes from piling up.

The term "world model" covers several research traditions. Ding et al. organize the literature around two functions: building internal representations that capture how the world works, and predicting future states to simulate and guide decisions [1]. This chapter uses an operational definition: **a world model is a learned model that represents enough of an environment to predict relevant changes under specified conditions, usually including the agent's own actions.** Its usefulness depends on the task. A model for rendering a scene, a model for learning representations, and a model for controlling a robot may need very different information.

Observations, states, beliefs and actions

Plain version. A camera image is not the world; it is a partial view of it. A world model has to keep track of what it cannot currently see, like the speed of a ball or an object behind an occluder.

Let s_t denote the underlying environment state at time t, o_t the observation and a_t the executed action:

$$s_{t+1} \sim p(s_{t+1} \mid s_t, a_t), \qquad o_t \sim p(o_t \mid s_t).$$

The first distribution is the **transition** (dynamics) model; the second is the **observation** model. The state might contain object positions, velocities and contact conditions. An image shows only a projection, with occlusions and sensor noise; Szeliski covers the geometry and motion side of this observation process [3]. Two identical images can have different futures: a ball at the same position might be moving left or right.

A learned model therefore builds a representation from history:

$$z_t = E_\theta(o_{0:t}, a_{0:t-1}), \qquad p_\theta(z_{t+1} \mid z_t, a_t),$$

where E_θ is an encoder with parameters θ, $o_{0:t}$ is the sequence of observations up to t, and $p_\theta(z_{t+1} \mid z_t, a_t)$ is a learned latent transition. An encoder does not automatically make z_t a sufficient state. If it discards velocity, hidden objects or a relevant material property, the transition model cannot recover that information. Recurrent memory or a short video context helps, but must be tested under the actual occlusions and delays.

When the state is only partially observed, the principled object is a **belief**, a probability distribution over possible states given the history. Bishop's treatment of sequential latent-variable models (hidden Markov models and linear dynamical systems) gives the probabilistic background [2]. A compact latent vector can approximate useful aspects of this distribution; it need not map one-to-one onto human-readable physical variables.

A prediction factorization

For a proposed action sequence, the probability of future observations factorizes as

$$p_\theta(o_{t+1:t+H} \mid o_{\le t}, a_{0:t+H-1}) = \int p_\theta(z_t \mid o_{\le t}, a_{<t}) \prod_{k=t}^{t+H-1} p_\theta(z_{k+1} \mid z_k, a_k)\, p_\theta(o_{k+1} \mid z_{k+1})\, dz_{t:t+H},$$

where H is the prediction horizon. The three factors are **inference** (what is the current latent state given the past?), **transition** (how does it evolve under the actions?) and **rendering** (what would the camera see?). A planner may work directly with z and skip the pixel decoder if its goal and cost are defined in latent space. Much of the design space in this chapter consists of choosing which of these factors to learn, and how.

Model-based reinforcement learning

Plain version. Reinforcement learning (RL) learns to act by trial and error. If the agent also learns how the world responds to its actions, it can practice in its head, which needs far fewer real trials.

The setting

An RL agent interacts with an environment over discrete steps. At each step it observes, chooses a_t from a **policy** $\pi(a_t \mid s_t)$, receives a reward r_t, and the environment moves on. The goal is to maximize the expected **return**

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k},$$

where $\gamma \in [0, 1)$ is a discount factor that weights near rewards more than distant ones. The **value** $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ is the expected return from state s when following π.

Model-free methods learn a policy or value directly from real transitions. **Model-based** methods also learn a model of transitions and rewards and use it to reduce real interaction. The idea is old: Sutton's Dyna architecture (1991) already combined learning from real experience with planning on experience simulated by a learned model, using the same value updates for both [4]. Moerland et al. survey the field and organize it along two axes: how the dynamics model is learned (handling stochasticity, uncertainty, partial observability, temporal abstraction) and how planning and learning are integrated (where to start planning, how much budget to give it, how to plan, and how to feed the results back into acting and learning) [5].

Three ways to use a model

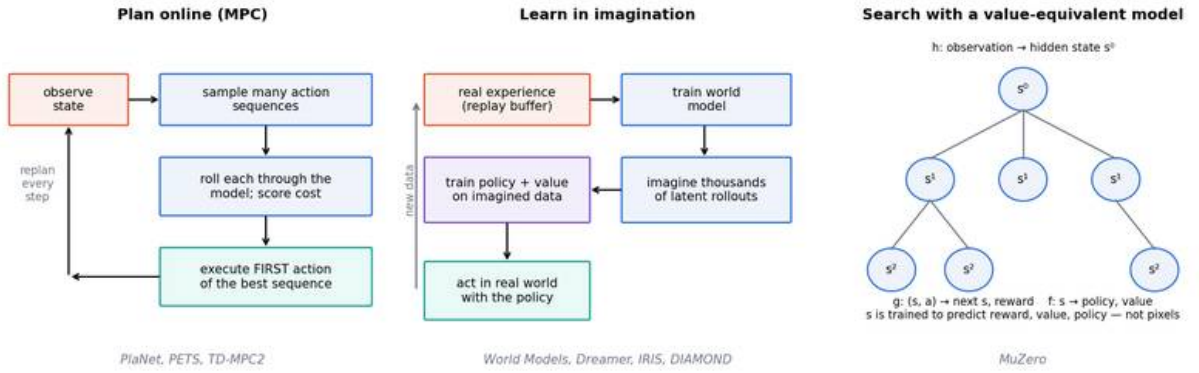


Figure 1 — Three ways to use a learned model. The same model can serve more than one role; the difference is where the decision is computed.

1. **Plan online (model-predictive control).** At every real step, search for a good action sequence by simulating candidates in the model, execute only the first action, observe, and repeat. PlaNet [7], PETS [29] and TD-MPC2 [11] work this way.
2. **Learn in imagination.** Train a policy (and usually a value function) on trajectories generated by the model, then act with that policy without searching at run time. World Models [6], the Dreamer line [8], [9], [10], IRIS [12] and DIAMOND [13] work this way.
3. **Search with a value-equivalent model.** Learn a model that predicts only what planning needs (rewards, values, policy), not observations, and run tree search in its latent space. MuZero [14] is the canonical example.

All three share one danger: the model is wrong somewhere, and an optimizer is very good at finding exactly those places. We return to this in the section on uncertainty and exploitation.

Why errors compound

Suppose the learned transition $\hat{f}$ has one-step error at most ε, and the true dynamics f are Lipschitz with constant L (a small change in state changes the next state by at most L times as much). Starting from the same state and applying the same actions, the gap e_k between predicted and true states after k steps obeys $e_{k+1} \le L e_k + \varepsilon$, so

$$e_H \le \varepsilon \sum_{k=0}^{H-1} L^k = \varepsilon \, \frac{L^H - 1}{L - 1} \quad (L \neq 1).$$

For $L > 1$ the bound grows exponentially with the horizon H; for $L < 1$ it saturates at $\varepsilon/(1 - L)$. This is a worst-case bound, but it explains two common design choices: replanning often (MPC) and using short model rollouts. Janner et al. made the second idea concrete: their MBPO

algorithm branches short model rollouts from real states rather than imagining long trajectories from the start, and they argue both theoretically and empirically that this keeps the benefit of model data while limiting its bias [30].

World Models: compress, predict, control

Plain version. First learn to summarize each frame in a few numbers, then learn to predict the next summary, then train a tiny controller on top. The controller can even be trained entirely inside the model's "dream".

Ha and Schmidhuber's *World Models* (published at NeurIPS 2018 as "Recurrent World Models Facilitate Policy Evolution", with an interactive online version) made the idea popular in deep learning [6]. Their agent has three parts:

- **V (vision):** a variational autoencoder that compresses each 64×64 frame into a small latent vector z_t.
- **M (memory):** a recurrent network with a mixture-density output that predicts a distribution over z_{t+1} given z_t, the action and its hidden state h_t.
- **C (controller):** a deliberately tiny linear policy $a_t = W_c[z_t; h_t] + b_c$, trained with an evolution strategy rather than backpropagation.

In a car-racing task, the controller using V and M features solved the task; in a VizDoom task, the controller was trained entirely inside the environment simulated by M and then transferred to the real game [6]. The design separates *perception*, *prediction* and *decision*, and shows that a controller can be very small when the representation already carries the relevant dynamics. It also showed the central risk: a controller trained inside an imperfect dream can learn to exploit the dream's mistakes.

Latent dynamics: from PlaNet to the Dreamer line

Plain version. Instead of predicting future images, predict a compact hidden state and only decode images when training. A hidden state that mixes a deterministic memory with a random part can represent both what the agent remembers and what it is unsure about.

Recurrent state-space models

Hafner et al.'s PlaNet learned environment dynamics from images and chose actions by fast online planning in latent space; its key modeling idea is a latent dynamics model with both deterministic and stochastic components, the **recurrent state-space model** (RSSM) [7]. The RSSM keeps a deterministic memory h_t and a stochastic state z_t:

$$h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1}), \qquad z_t \sim p_\theta(z_t \mid h_t) \ \text{(prior)}, \qquad z_t \sim q_\theta(z_t \mid h_t, o_t) \ \text{(posterior)}.$$

Here f_θ is a recurrent cell, the **prior** predicts z_t without seeing the current image (this is what is used when imagining), and the **posterior** also sees the image o_t (used during training and when filtering real observations). Heads read the model state (h_t, z_t) to predict the image $\hat{o}_t$, the reward $\hat{r}_t$ and a continuation flag (whether the episode goes on).

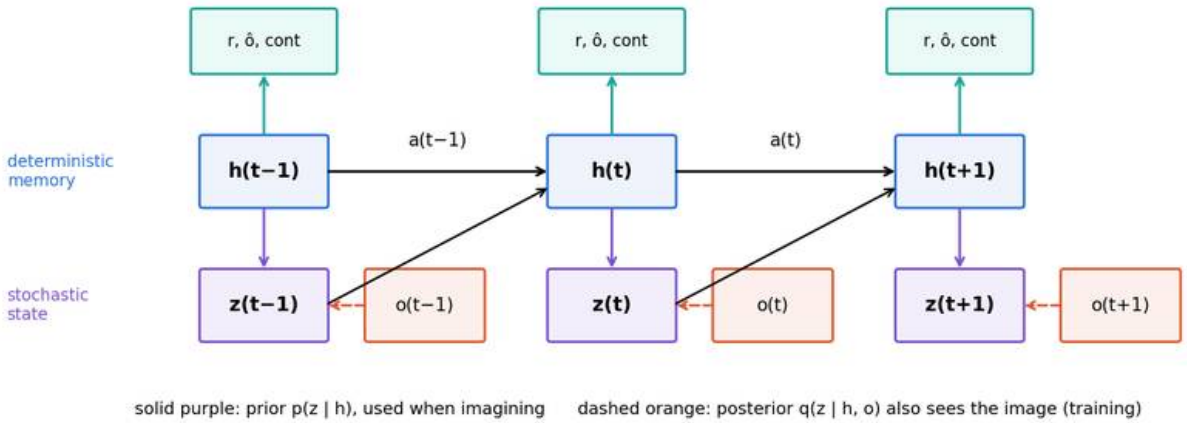


Figure 2 — An unrolled RSSM. Imagination uses only the prior path (solid); training additionally conditions z on the real image (dashed).

Training maximizes a variational lower bound on the likelihood of observed sequences. A simplified per-step loss is

$$\mathcal{L}_t = -\log p_\theta(o_t \mid h_t, z_t) - \log p_\theta(r_t \mid h_t, z_t) + \beta\, D_{\mathrm{KL}}(q_\theta(z_t \mid h_t, o_t) \,\|\, p_\theta(z_t \mid h_t)),$$

with $\beta > 0$ weighting the KL term. The reconstruction terms make the state informative; the KL term teaches the prior to predict what the posterior will see, which is exactly the skill needed for imagination. Real implementations add loss balancing and other stabilizers, so treat this as a teaching outline, not the exact loss of any one paper.

Dreamer: learning behavior in imagination

Dreamer (2020) kept the RSSM but replaced online planning with an actor–critic trained entirely on imagined latent trajectories. Its key move was to propagate analytic gradients of learned state values back through trajectories imagined in the compact latent space [8]. Starting from latent states inferred from real data, the agent imagines H steps with its actor, scores them with a learned critic, and trains the critic on bootstrapped targets such as the λ-return

$$V_t^\lambda = \hat{r}_t + \gamma\, \hat{c}_t \big[(1-\lambda)\, v_\psi(s_{t+1}) + \lambda\, V_{t+1}^\lambda \big],$$

where $\hat{r}_t$ and $\hat{c}_t$ are the predicted reward and continuation, v_ψ is the critic, and $\lambda \in [0,1]$ trades

off one-step bootstrapping ($\lambda = 0$) against longer imagined returns ($\lambda = 1$). The actor is updated to increase these values.

The line continued:

- **DreamerV2** (2021) switched to *discrete* (categorical) latent variables and became the first agent to reach human-level performance on the 55-game Atari benchmark by learning behaviors inside a separately trained world model [9].
- **DreamerV3** (Nature, 2025) used robustness techniques based on normalization, balancing and transformations so that a single configuration outperformed specialized methods across more than 150 tasks, and was the first algorithm to collect diamonds in Minecraft from scratch without human data or curricula [10].

Two related lines show that the RSSM is one choice among several. **TD-MPC2** plans in the latent space of an *implicit*, decoder-free world model (no image reconstruction), reported strong results across 104 online RL tasks with a single set of hyperparameters, and trained a single 317M-parameter agent on 80 tasks [11]. **IRIS** replaced the RSSM with a discrete autoencoder plus an autoregressive Transformer over image tokens and learned from the equivalent of two hours of

Atari gameplay [12]. **DIAMOND** went further toward pixels: it trains the agent inside a *diffusion* world model, arguing that discrete latents can drop visual details that matter for control, and also demonstrated the model as an interactive neural game engine for a 3D game [13].

MuZero: a model that predicts only what planning needs

Plain version. You do not have to predict what the screen will look like to play a good game. You only need to predict how good positions are, which moves look promising, and what rewards you will get. MuZero learns exactly that, and then searches.

AlphaZero-style tree search needs a simulator of the game rules. MuZero removes that requirement by learning a model that, applied repeatedly, predicts the quantities most directly relevant to planning: reward, action-selection policy and value [14]. It has three functions:

$$s^0 = h_\theta(o_1, \dots, o_t), \qquad (r^k, s^k) = g_\theta(s^{k-1}, a^k), \qquad (\mathbf{p}^k, v^k) = f_\theta(s^k),$$

where h_θ (representation) encodes the observation history into a hidden state s^0, g_θ (dynamics) maps a hidden state and a hypothetical action a^k to a predicted reward r^k and next hidden state, and f_θ (prediction) outputs a policy vector $\mathbf{p}^k$ and a value v^k. A Monte Carlo tree search unrolls g_θ to choose actions. Training unrolls the model for K steps along real trajectories and minimizes

$$\mathcal{L} = \sum_{k=0}^{K} \Big[\ell^r(u_{t+k}, r^k) + \ell^v(z_{t+k}, v^k) + \ell^p(\boldsymbol{\pi}_{t+k}, \mathbf{p}^k) \Big] + c\|\theta\|^2,$$

where u are observed rewards, z are value targets (bootstrapped returns), $\boldsymbol{\pi}$ are the visit-count distributions produced by the search, ℓ^r, ℓ^v, ℓ^p are per-quantity losses and c weights an L2 penalty.

The hidden state s^k is never asked to reconstruct an observation. It only has to be **value-equivalent**: planning with it must give the same rewards, values and policies as planning in the real environment would. With this design MuZero reached a new state of the art on 57 Atari games and, without being given the rules, matched AlphaZero's superhuman performance in Go, chess and shogi [14]. The lesson for vision is that a world model's target should be chosen by its use; predicting every pixel is one option, not a requirement.

Video prediction and generation as world simulators

Plain version. If a model can generate realistic future video, perhaps it has learned how the world works. Sometimes it has; often it has only learned what videos usually look like. Testing which is the case is the hard part.

Action-conditioned video prediction

Finn et al. trained an **action-conditioned video prediction** model on a dataset of 59,000 robot pushing interactions; by predicting pixel motion rather than raw pixels, the model generalized partly to unseen objects, and conditioning on different future actions produced a "visual imagination" of different futures [15]. This is a world model in the most literal sense: image in, action in, image out.

Why squared-error predictors blur

Real futures are uncertain. Suppose that from the same past, a ball goes left with probability $\frac{1}{2}$ and right with probability $\frac{1}{2}$. For squared error, the best single prediction is the **conditional mean**:

$$\hat{o}^\star = \arg\min_{\hat{o}} \; \mathbb{E}[\|o_{t+1} - \hat{o}\|^2 \mid o_{\le t}] = \mathbb{E}[o_{t+1} \mid o_{\le t}],$$

which here is two half-bright balls, an image that never happens. A generative model that **samples** one of the two futures produces a realistic image but, on average, a *higher* squared error.

```
import numpy as np

x = np.arange(64)
def frame(center):                      # a 1-D "image" with one bright ball
    return np.exp(-0.5 * ((x - center) / 2.0) ** 2)

# Same past, two equally likely futures: the ball bounces left or right.
futures = np.stack([frame(20), frame(44)])
mean_pred = futures.mean(axis=0)        # what an MSE-trained predictor converges to
```

```
sample_pred = futures[0]                # what a good generative sampler might output

def expected_mse(pred):                 # average over the two possible futures
    return np.mean([(pred - f) ** 2 for f in futures])

print(f"peak brightness: true future 1.00, mean prediction {mean_pred.max():.2f}")
print(f"expected MSE of mean prediction:   {expected_mse(mean_pred):.4f}")
print(f"expected MSE of a sharp sample:    {expected_mse(sample_pred):.4f}")
```

Output:

```
peak brightness: true future 1.00, mean prediction 0.50
expected MSE of mean prediction:   0.0277
expected MSE of a sharp sample:    0.0554
```

The sharp sample has exactly twice the expected error of the blurry mean. This is why pixel MSE rewards blur, why stochastic video models were introduced, and why evaluating a world model with a single squared-error number can prefer the wrong model.

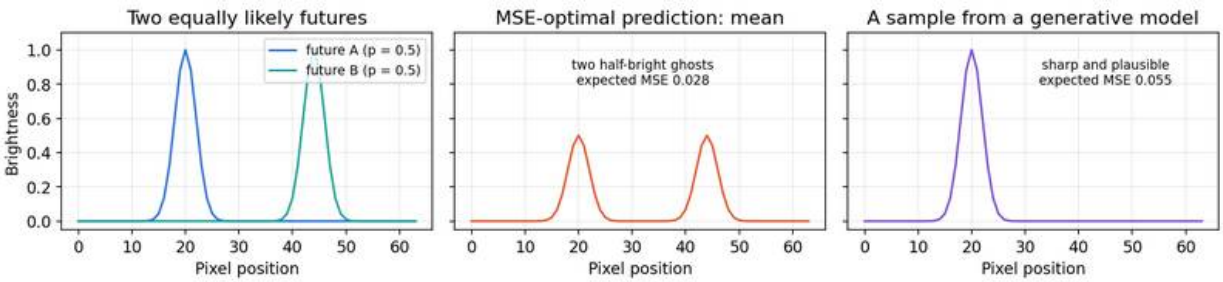


Figure 3 — Output of the code above. The MSE-optimal prediction averages incompatible futures; a sample is plausible but scores worse on MSE.

Babaeizadeh et al.'s SV2P addressed this by predicting a different possible future for each sample of a latent variable, and found substantially better predictions than the same model without stochasticity [16]. Modern video generators use the same principle at a much larger scale, with autoregressive token models or diffusion and flow models (see ML 07 and ML 08).

Large generative world models

- **GAIA-1** casts driving world modeling as next-token prediction over discrete tokens of video, text and action, producing realistic driving scenarios with control over ego-vehicle behavior and scene features [17].
- **UniSim** learns a "universal simulator" of real-world interaction by combining datasets that are rich along different axes (objects in images, dense actions in robot data, motion in navigation data); it simulates both high-level instructions and low-level controls, and policies trained purely in it were deployed zero-shot on real robots [18].

- **Genie** is a generative interactive environment trained without action labels on unlabelled Internet video. It consists of a video tokenizer, an autoregressive dynamics model and a *latent action model* that infers actions from changes between frames, so users can act frame by frame in generated worlds even though no action labels were used in training [19]. Such latent actions need not correspond to calibrated physical actuators.
- **Genie 3**, announced by Google DeepMind in August 2025, generates worlds from text that can be navigated in real time at 24 frames per second and 720p, keeping consistency for a few minutes. The announcement lists limitations: a constrained action space, difficulty modeling several independent agents, inaccurate real-world locations, and interaction limited to minutes rather than hours [20].
- **Cosmos** is NVIDIA's open platform of "world foundation models": pretrained video models, video tokenizers, a data-curation pipeline and post-training examples meant to be fine-tuned into task-specific world models for robots and vehicles [21].

Does realistic video mean physical understanding?

Two careful studies say: not necessarily. Motamed et al.'s **Physics-IQ** benchmark tests fluid dynamics, optics, solid mechanics, magnetism and thermodynamics; across current video models (including Sora, Runway, Pika, Lumiere, Stable Video Diffusion and VideoPoet), physical understanding was severely limited and unrelated to visual realism [22]. Kang et al. trained diffusion video models on a 2D simulator governed by classical mechanics and found perfect in-distribution generalization, measurable scaling for combinatorial generalization, but failure out of distribution; the models behaved as if retrieving the closest training example rather than abstracting a rule [23].

For robot control, also ask what an "action" means: force, joint target, gripper command, camera movement or an inferred token. A wrong unit, timing offset or control frequency changes the conditional problem even if the video looks convincing.

Predictive representations: the JEPA family

Plain version. Predicting every pixel wastes effort on details that do not matter, like leaf texture. Instead, predict an abstract *description* of the missing or future part. If the description is good, the prediction is useful for planning.

A **joint-embedding predictive architecture** (JEPA) encodes a visible *context* x and a hidden *target* y with encoders, and trains a predictor to map the context embedding to the target embedding:

$$\mathcal{L} = \left\| P_\phi(E_\theta(x),\, m) - \mathrm{sg}[\bar{E}(y)] \right\|^2,$$

where E_θ is the context encoder, $\bar{E}$ the target encoder, P_ϕ a predictor conditioned on information m

about where the target is (for example its position in the image or time), and sg (stop-gradient) blocks gradients into the target branch. Nothing is reconstructed in pixel space. The danger is **collapse**: if both encoders output a constant, the loss is zero. Asymmetric designs (a predictor on one side only, stop-gradient, and a target encoder that is not trained by the same gradient) are used to prevent it; ML 05 discusses these self-supervised mechanisms in more detail.

- **I-JEPA** predicts the representations of several large target blocks of an image from one informative context block, and found the masking strategy (large, semantic targets; spatially distributed context) crucial for learning semantic features without hand-crafted augmentations [24].
- **V-JEPA** applies feature prediction as a stand-alone objective to video, trained on 2 million videos without pretrained image encoders, text, negatives or reconstruction, and produced frozen features that work for both motion- and appearance-based tasks [25].
- **V-JEPA 2** pretrains an action-free JEPA on over one million hours of Internet video, then post-trains a latent **action-conditioned** world model, V-JEPA 2-AC, on less than 62 hours of unlabeled robot video; planning toward image goals with it enabled zero-shot pick-and-place on Franka arms in two labs without data from those environments, task-specific training or reward [26]. The split between passive video and action data makes the observation/action distinction of this chapter concrete.
- **V-JEPA 2.1** adds a dense predictive loss in which both visible and masked tokens contribute, deep self-supervision across intermediate layers, and multi-modal tokenizers for images and videos, aiming at spatially structured, temporally consistent dense features [27].
- **DINO-WM** shows the same idea with an off-the-shelf encoder: it predicts future DINOv2 patch features from offline trajectories and plans by optimizing action sequences toward the features of a goal image, without reconstructing pixels or learning a reward model [28].

Goals in feature space need care. A distance $\|z - z_{\text{goal}}\|^2$ rewards only what the representation preserves. If the representation ignores gripper closure, a planner may reach a visually similar state without actually grasping the object. Better feature quality alone does not establish that an encoder predicts the consequences of arbitrary actions.

How the families relate

Family	Prediction target	Main benefit	Question to test
Latent dynamics (RSSM, TD-MPC2)	compact state, reward, value	cheap, repeated rollouts	does the latent retain task-relevant dynamics?
Value-equivalent (MuZero)	reward, value, policy only	no effort on irrelevant detail	does it transfer when the reward changes?
Autoregressive video (GAIA-1, Genie, IRIS)	next frame or token	explicit sequential factorization	do errors drift over long rollouts?
Diffusion/flow video (DIAMOND, Cosmos)	jointly generated frames or chunks	expressive, uncertain futures	can actions steer outcomes at control latency?

Family	Prediction target	Main benefit	Question to test
Predictive embeddings (JEPA, DINO-WM)	target features	structure without pixel reconstruction	are actions represented and goals meaningful?

These are compatible choices. A Transformer can parameterize any of them; a latent world model can decode through diffusion; a video generator can use an autoregressive outer loop and a diffusion inner loop. Architecture, prediction target, temporal factorization and training objective are separate design decisions.

Learning dynamics from data

Training examples should retain ordering and action alignment:

$$(o_t, a_t, o_{t+1}, r_t, \text{done}_{t+1}),$$

where done_{t+1} marks the end of an episode. Record whether each action happens before or after each camera timestamp. Split by whole episodes, environment instances and relevant subjects; randomly splitting adjacent frames leaks nearly identical scenes and makes a transition predictor look better than it is (see ML 10 for leakage in general).

The data-collection policy matters. A model trained on slow, gentle interactions may fail for abrupt motion. Observational correlations are especially problematic when actions are absent: a model may learn that a cup moves when a hand approaches, yet have no evidence about what alternative pushes would do. This is the difference between passive video and interventional data.

A **one-step loss** measures prediction from real history. A **multi-step loss** unrolls the model's own predictions and penalizes their accumulated error. Unrolling changes training difficulty and gradient paths; it does not remove distribution shift outside the observed interactions.

Complete CPU experiment: fit, predict, plan

We use a two-dimensional state $x_t = (\text{position}, \text{velocity})$, a scalar action u_t and a damped linear system:

$$x_{t+1} = Ax_t + Bu_t + \epsilon_t,$$

where A is the 2×2 state-transition matrix, B the action-input vector and ϵ_t small Gaussian noise. The system is deliberately fully observed. It isolates dynamics and planning before adding image encoders and partial observability. Training and test episodes are generated independently. We fit only on training transitions and evaluate both one-step prediction and open-loop rollouts with the same held-out actions.

The planner uses the **cross-entropy method** (CEM): sample candidate action sequences from a Gaussian, keep the lowest-cost "elite" sequences under the model, refit the Gaussian's mean and

spread to them, and repeat. It executes one action, receives a fresh real state and replans. This is model-predictive control.

```
import numpy as np

rng = np.random.default_rng(7)
A = np.array([[1.0, 0.1], [0.0, 0.94]])
B = np.array([0.005, 0.1])

def collect(n, steps=40):
    x = np.zeros((n, steps + 1, 2))
    x[:, 0] = rng.normal(size=(n, 2)) * [0.8, 0.3]
    u = rng.uniform(-1, 1, size=(n, steps))
    for t in range(steps):
        x[:, t + 1] = (x[:, t] @ A.T + u[:, t, None] * B
                       + rng.normal(scale=0.002, size=(n, 2)))
    return x, u

train, train_u = collect(120)
test, test_u = collect(80)
phi = np.column_stack([train[:, :-1].reshape(-1, 2), train_u.ravel()])
target = train[:, 1:].reshape(-1, 2)
W = np.linalg.lstsq(phi, target, rcond=None)[0]
W_no_action = np.linalg.lstsq(phi[:, :2], target, rcond=None)[0]

def predict(x, u):
    return np.concatenate([x, np.asarray(u)[..., None]], axis=-1) @ W

one_step = predict(test[:, :-1], test_u)
rollout = np.zeros_like(test)
rollout[:, 0] = test[:, 0]
for t in range(test_u.shape[1]):
    rollout[:, t + 1] = predict(rollout[:, t], test_u[:, t])
rmse = lambda a, b: np.sqrt(np.mean((a - b) ** 2))
print("One-step:", rmse(one_step, test[:, 1:]))
print("Without actions:", rmse(test[:, :-1] @ W_no_action, test[:, 1:]))
print("Final-horizon rollout:", rmse(rollout[:, -1], test[:, -1]))

def plan(x, mean, horizon=20, population=256):
    std = np.ones(horizon)
    for _ in range(4):
        candidates = np.clip(rng.normal(mean, std, (population, horizon)), -1, 1)
        imagined = np.broadcast_to(x, (population, 2)).copy()
        costs = np.zeros(population)
        for t in range(horizon):
            imagined = predict(imagined, candidates[:, t])
            costs += (imagined[:, 0] ** 2 + 0.15 * imagined[:, 1] ** 2
                      + 0.02 * candidates[:, t] ** 2)
        costs += 20 * imagined[:, 0] ** 2
        elite = candidates[np.argsort(costs)[:32]]
        mean = elite.mean(axis=0)
        std = elite.std(axis=0) + 0.05
    return mean
```

```
real = np.array([1.5, 0.0])
mean = np.zeros(20)
for _ in range(40):
    sequence = plan(real, mean)
    real = A @ real + B * sequence[0]  # real system; never used inside plan()
    mean = np.r_[sequence[1:], 0.0]
print("Controlled final state:", real)
assert rmse(one_step, test[:, 1:]) < 0.01
assert abs(real[0]) < 0.1
```

Output:

```
One-step: 0.0019866110218121832
Without actions: 0.04106321931941812
Final-horizon rollout: 0.013772759824274517
Controlled final state: [-0.00453985 -0.0210205 ]
```

The one-step RMSE (0.0020) is at the noise level; dropping the action input raises it twentyfold (0.0411), so the action genuinely matters. The open-loop rollout error at horizon 40 (0.0138) is about seven times the one-step error even for this nearly perfect linear model, a mild case of the compounding derived above. MPC drives the position from 1.5 to about −0.0045, compared with 1.5 under zero action. These are synthetic results from this script, not published comparisons.

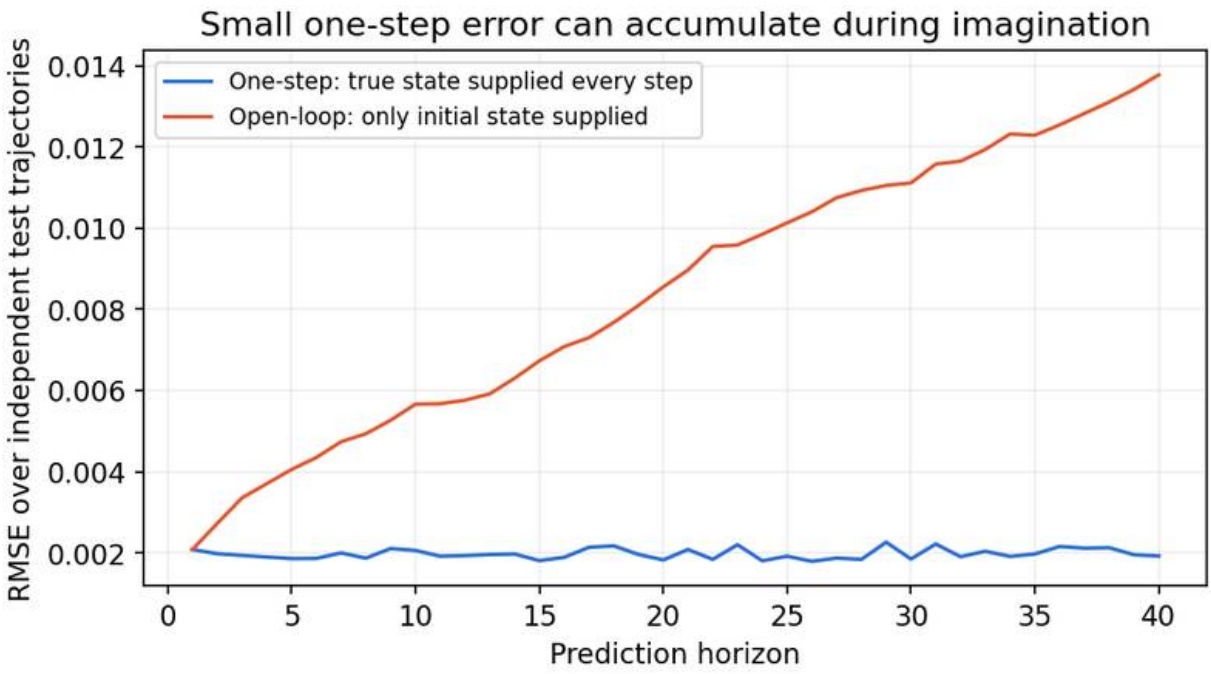


Figure 4 — Independent synthetic test episodes. One-step evaluation receives the true state each time; open-loop evaluation receives only the initial state and the held-out actions.

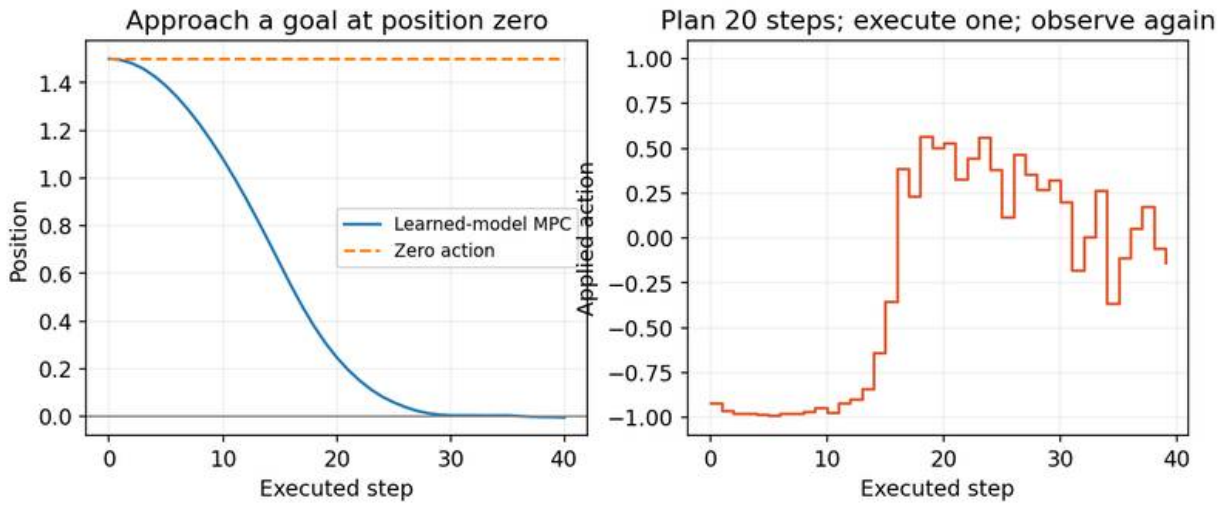


Figure 5 — CEM plans with the learned transition model. Only its first action is executed in the synthetic environment before replanning from a new observation.

Replacing x_t with an image embedding does not reproduce Dreamer or V-JEPA 2-AC: it adds an encoder-learning problem, latent uncertainty and the need for a valid goal or reward.

Planning, uncertainty and model exploitation

Plain version. A planner looks for the action the model likes best. If the model is wrong somewhere, the planner will happily go there. Knowing *where* the model is unsure is what keeps it honest.

A finite-horizon planning objective can be written as

$$a^*_{t:t+H-1} = \arg\min_{a_{t:t+H-1}} \mathbb{E}_\theta\Big[\sum_{k=1}^{H} c(z_{t+k}, a_{t+k-1}) + \lambda_{\mathrm{T}}\, c_{\mathrm{T}}(z_{t+H})\Big],$$

where c is a per-step cost, c_{T} a terminal cost weighted by λ_{T}, and the expectation is over the model's predicted latent states z. **Model-predictive control** executes the first action, observes again and replans. Feedback corrects drift, but cannot rescue an action whose immediate consequence was badly modeled.

Two kinds of uncertainty

Aleatoric uncertainty is inherent randomness (a coin flip, the ball that may go left or right); more data does not remove it. **Epistemic** uncertainty comes from lack of data (the model has never seen this situation); more data does remove it. Sampling a generative model can represent aleatoric

uncertainty, but sample diversity is not proof of calibration. Epistemic uncertainty is commonly approximated with **ensembles**: several models trained on bootstrap resamples or with different initializations agree where data is dense and tend to disagree where it is not. Chua et al.'s PETS combined probabilistic networks (for aleatoric noise) in an ensemble (for epistemic uncertainty) with sampling-based uncertainty propagation and CEM planning, matching the final performance of strong model-free methods on several benchmarks with far fewer samples [29].

Code: a planner exploiting a learned model

The agent has logged data only for actions in $[-1, 1]$, but its planner may choose any action in $[-3, 3]$.

```
import numpy as np
from sklearn.neural_network import MLPRegressor

rng = np.random.default_rng(0)
true_reward = lambda a: np.sin(3 * a) * np.exp(-0.3 * a ** 2)   # unknown to the agent

# Logged interactions only cover actions in [-1, 1].
a_data = rng.uniform(-1, 1, 200)
r_data = true_reward(a_data) + 0.05 * rng.normal(size=200)

ensemble = []
for k in range(5):                        # bootstrap + different initialization
    idx = rng.integers(0, 200, 200)
    m = MLPRegressor(hidden_layer_sizes=(64, 64), max_iter=3000, random_state=k)
    ensemble.append(m.fit(a_data[idx, None], r_data[idx]))

grid = np.linspace(-3, 3, 601)
preds = np.stack([m.predict(grid[:, None]) for m in ensemble])
mean, std = preds.mean(0), preds.std(0)

for name, score in [("single model", preds[0]),
                    ("ensemble mean", mean),
                    ("pessimistic: mean - 2 std", mean - 2 * std)]:
    a_star = grid[np.argmax(score)]       # the "planner": pick the best-looking action
    print(f"{name:26s} picks a = {a_star:+.2f}, predicted {score.max():.2f}, "
          f"true {true_reward(a_star):.2f}")
print(f"mean std inside data {std[np.abs(grid) <= 1].mean():.3f}, "
      f"outside {std[np.abs(grid) > 1.5].mean():.3f}")
```

Output:

```
single model               picks a = -3.00, predicted 1.79, true -0.03
ensemble mean              picks a = -3.00, predicted 1.58, true -0.03
pessimistic: mean - 2 std  picks a = +0.43, predicted 0.87, true 0.91
mean std inside data 0.025, outside 0.321
```

Both the single model and the ensemble mean extrapolate a rising line outside the data, and the planner runs to the edge of the action space, where the true reward is about zero. Ensemble disagreement is thirteen times larger outside the data than inside, and a *pessimistic* score (mean minus two standard deviations) keeps the planner inside the data, where it finds an action with true reward 0.91. Restricting actions, shortening horizons, penalizing disagreement and testing with real feedback are the practical responses. Agreement among ensemble members is not proof of correctness, though: all five can share the same bias.

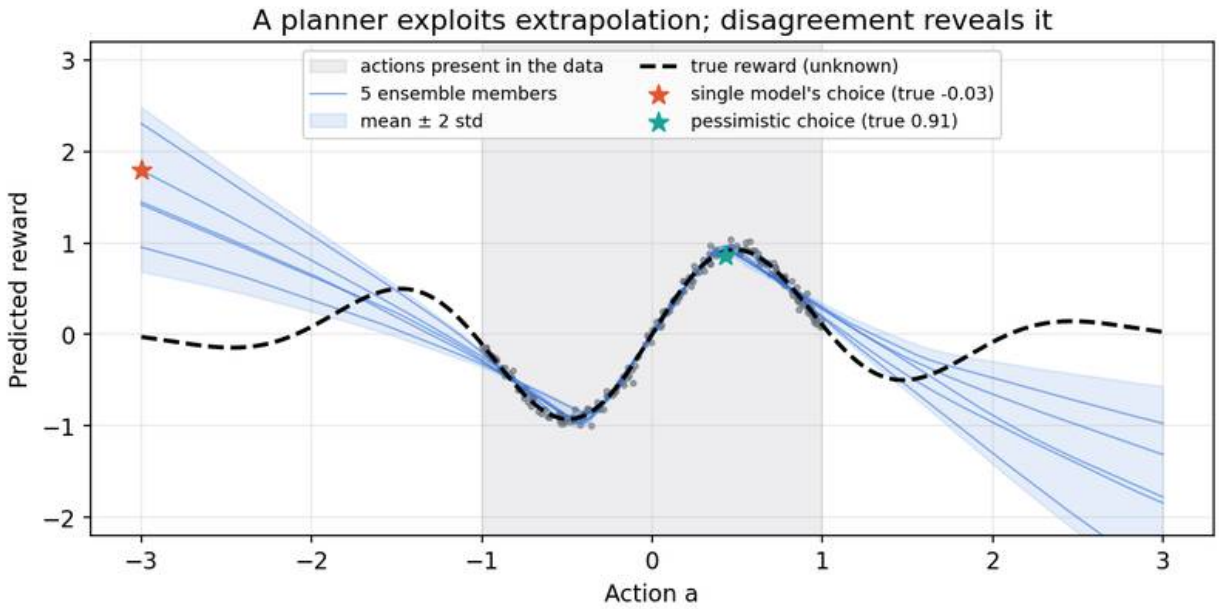


Figure 6 — Output of the code above. Optimizing a learned model drives the planner to where the model is least reliable; ensemble spread exposes those regions.

How to evaluate a world model

Question	Measurement	Important control
Does it predict known dynamics?	one-step and horizon-indexed state or feature error	independent episodes; fixed action sequence
Does appearance stay coherent?	reconstruction and video-quality metrics, identity consistency	compare camera motion and occlusions
Do actions matter correctly?	matched initial context with different specified actions	real action-conditioned reference trajectories
Are physical constraints respected?	collisions, object permanence, conservation; benchmarks such as Physics-IQ [22]	task-specific tolerances; out-of-distribution cases [23]
Does it help decisions?	success rate, return, interventions, control latency	test in the real or a trusted environment

Pixel fidelity and decision usefulness can disagree, as the blur example showed. An averaged prediction can have low squared error while showing an impossible scene; an accurate latent predictor can omit visually irrelevant texture. Report the target, horizon, conditions and output resolution with every metric.

For comparison, include a persistence baseline (predict "nothing changes"), an action-free predictor and a simple task-appropriate dynamics model. Evaluate held-out objects, viewpoints, action ranges and environment properties separately. Report closed-loop control results in addition to imagined ones, and account for encoder and sampler latency: a planner that needs ten seconds per action may be useless for a robot running at 10 Hz.

Modern view

Surveys. Moerland et al. give the most systematic treatment of model-based RL: dynamics-model learning (stochasticity, uncertainty, partial observability, temporal abstraction), a taxonomy of how planning and learning are integrated, and implicit (value-equivalent, MuZero-style) approaches [5]. Ding et al. (ACM Computing Surveys) organize the broader "world model" literature by function, understanding the world versus predicting its future, and review applications in generative games, autonomous driving, robotics and social simulation [1]. Zhu et al. survey "general world models" from the video-generation side, covering video generators, autonomous-driving world models and world models inside autonomous agents, and discuss their limitations [31]. Guan et al. give an initial survey of world models specifically for autonomous driving [32].

State of the art, 2023–2026.

- *Model-based RL matured.* DreamerV3 showed a single configuration working across more than 150 tasks and was published in Nature in 2025 [10]; TD-MPC2 showed decoder-free latent planning scaling to a multi-task 317M-parameter agent [11]; IRIS and DIAMOND showed Transformer and diffusion world models competitive on the Atari 100k benchmark [12], [13].
- *Video generators were reframed as simulators.* GAIA-1, UniSim, Genie, Genie 3 and Cosmos trained large models on video with actions, text or inferred latent actions [17], [18], [19], [20], [21]. The strongest evidence that they help agents is UniSim's zero-shot transfer of policies trained inside the simulator [18]; the strongest caution is that realism does not imply physics [22], [23].
- *Predictive representations moved toward control.* The JEPA line went from images (I-JEPA) to video (V-JEPA) to action-conditioned planning on real robots (V-JEPA 2-AC) and denser features (V-JEPA 2.1) [24], [25], [26], [27]; DINO-WM showed that frozen self-supervised features can already support zero-shot planning [28].

Open problems.

- *Long-horizon consistency.* Errors compound; generated worlds drift; object identity and off-screen state are hard to keep for minutes, let alone hours [20].
- *Causal and physical correctness.* Models trained on passive video learn correlations; out-of-distribution physics fails [22], [23]. Interventional (action) data is scarce and expensive.

- *Action interfaces.* Latent actions inferred from video, robot joint commands and text instructions are different things; mapping between them reliably is unsolved.
- *Exploitation and uncertainty.* Planners and policies trained in imagination exploit model errors; calibrated epistemic uncertainty for large video models is largely missing.
- *Evaluation.* There is no agreed benchmark that ties prediction quality to downstream decision quality across domains; most papers still report either video metrics or task returns, rarely both.
- *Cost.* Sampling a large video model can be far slower than real time, which conflicts with closed-loop control.

Key takeaways

- A world model predicts how an environment changes, usually under the agent's actions; its prediction target and action interface determine what it can support.
- Observations are not states. Partial observability requires memory or belief estimation.
- A model can be used to plan online (MPC), to train a policy in imagination (World Models, Dreamer), or to search with a value-equivalent latent (MuZero).
- RSSMs combine deterministic memory with stochastic latents; the KL term trains the prior to imagine what the posterior will see.
- Squared-error video prediction averages possible futures into blur; stochastic and generative models trade lower likelihood of blur for realistic samples, but realism is not physical understanding.
- JEPA-style models predict features instead of pixels; they need anti-collapse design and meaningful goals in feature space.
- One-step errors compound over long horizons, and optimizers exploit model mistakes; ensembles, pessimism, short rollouts and replanning are the standard defenses.
- The strongest evidence connects controlled action interventions and held-out predictions to successful behavior in the real environment.

Exercises

1. **Action ablation.** In the CPU experiment, remove the action column. Compare one-step error, horizon-40 error and control behavior. Why can a short-term action-free predictor still look reasonable?

   Hint

   Imagined actions no longer affect the action-free predictor, so a planner cannot rank their consequences at all: every candidate gets the same predicted cost. Over one step the damped dynamics change the state only slightly, so persistence-like predictions still score moderately well, which is why one-step error alone is a weak test.

2. **Partial observations.** Hide velocity and fit position-to-position dynamics. Then use the last two positions and the previous action as features. When does this history help, and where does observation noise hurt?

Hint

The difference of two positions estimates velocity (a finite difference), so the two-position model recovers most of the state. Noise in each position is amplified when divided by the time step, so the benefit shrinks as observation noise grows.

3. **Planning horizon.** Compare horizons 5, 20 and 40, adjusting the initial mean to the chosen length. Measure final distance, action effort and runtime across several independent seeds.

Hint

Keep the cost definition fixed and count all model calls. Very short horizons cannot "see" that braking early avoids overshoot; very long horizons cost more and accumulate more model error, although for this linear system the error is small.

4. **Domain shift.** Collect training episodes with velocity damping 0.94, then execute control in an environment with damping 0.75. Compare open-loop execution of a plan with replanning at every step. Identify an action range where the training data provide weak coverage.

Hint

Separate the learned dynamics (fitted on 0.94) from the true execution dynamics (0.75); do not refit on evaluation transitions. Replanning uses fresh observations, so it partially corrects the biased model; open-loop execution cannot.

5. **Mean versus sample.** Extend the blur example to three equally likely futures at positions 10, 32 and 54. Compute the expected MSE of the mean prediction and of a single sharp sample. Then propose a metric that would prefer the sample.

Hint

The mean now has three one-third-height bumps; the expected MSEs are about 0.037 (mean) and 0.074 (sample), again a factor of two. That factor is general: if X and Y are independent draws from the same distribution, $\mathbb{E}\|X-Y\|^2 = 2\,\mathbb{E}\|X-\mathbb{E}X\|^2$, so a perfect sampler always has twice the MSE of the mean. A distribution-level metric (for example, comparing sets of generated samples with sets of real futures, as FID-style metrics do for images) or a best-of-N error over several samples would reward sharp, plausible predictions.

6. **Pessimism trade-off.** In the ensemble code, replace the factor 2 in "mean − 2 std" by 0, 0.5, 1 and 4. Plot the chosen action and its true reward for each. What goes wrong when the factor is too large?

Hint

At 0 the planner exploits extrapolation; as the factor grows it stays inside the data. Too much pessimism makes the planner avoid any action with even slight disagreement, so it may miss the best action inside the data and, in an RL setting, stop exploring.

References

1. J. Ding et al., "Understanding world or predicting future? A comprehensive survey of world models," *ACM Computing Surveys*, vol. 58, no. 3, 2025. doi arXiv
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, ch. 13 "Sequential Data". book
3. R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022. book
4. R. S. Sutton, "Dyna, an integrated architecture for learning, planning, and reacting," *ACM SIGART Bulletin*, vol. 2, no. 4, 1991. doi
5. T. M. Moerland, J. Broekens, A. Plaat and C. M. Jonker, "Model-based reinforcement learning: a survey," *Foundations and Trends in Machine Learning*, vol. 16, no. 1, 2023. doi arXiv
6. D. Ha and J. Schmidhuber, "Recurrent world models facilitate policy evolution," NeurIPS, 2018; interactive version "World Models," arXiv:1803.10122. paper interactive
7. D. Hafner et al., "Learning latent dynamics for planning from pixels," ICML, 2019. paper
8. D. Hafner, T. Lillicrap, J. Ba and M. Norouzi, "Dream to control: learning behaviors by latent imagination," ICLR, 2020. arXiv
9. D. Hafner, T. Lillicrap, M. Norouzi and J. Ba, "Mastering Atari with discrete world models," ICLR, 2021. arXiv
10. D. Hafner, J. Pasukonis, J. Ba and T. Lillicrap, "Mastering diverse control tasks through world models," *Nature*, vol. 640, 2025. doi arXiv
11. N. Hansen, H. Su and X. Wang, "TD-MPC2: scalable, robust world models for continuous control," ICLR, 2024. arXiv
12. V. Micheli, E. Alonso and F. Fleuret, "Transformers are sample-efficient world models," ICLR, 2023. arXiv
13. E. Alonso et al., "Diffusion for world modeling: visual details matter in Atari," NeurIPS, 2024. arXiv
14. J. Schrittwieser et al., "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, vol. 588, 2020. doi arXiv
15. C. Finn, I. Goodfellow and S. Levine, "Unsupervised learning for physical interaction through video prediction," NIPS, 2016. arXiv
16. M. Babaeizadeh, C. Finn, D. Erhan, R. H. Campbell and S. Levine, "Stochastic variational video prediction," arXiv:1710.11252, 2017. arXiv
17. A. Hu et al., "GAIA-1: a generative world model for autonomous driving," technical report, arXiv:2309.17080, 2023. arXiv
18. S. Yang et al., "Learning interactive real-world simulators," arXiv:2310.06114, 2023. arXiv
19. J. Bruce et al., "Genie: generative interactive environments," arXiv:2402.15391, 2024. arXiv
20. Google DeepMind, "Genie 3: a new frontier for world models," blog post, August 5, 2025. blog

21. NVIDIA (N. Agarwal et al.), “Cosmos world foundation model platform for physical AI,” arXiv:2501.03575, 2025. arXiv
22. S. Motamed, L. Culp, K. Swersky, P. Jaini and R. Geirhos, “Do generative video models understand physical principles?,” arXiv:2501.09038, 2025. arXiv
23. B. Kang et al., “How far is video generation from world model: a physical law perspective,” ICML, 2025. arXiv
24. M. Assran et al., “Self-supervised learning from images with a joint-embedding predictive architecture,” CVPR, 2023. doi arXiv
25. A. Bardes et al., “Revisiting feature prediction for learning visual representations from video,” arXiv:2404.08471, 2024. arXiv
26. M. Assran et al., “V-JEPA 2: self-supervised video models enable understanding, prediction and planning,” arXiv:2506.09985, 2025. arXiv
27. L. Mur-Labadia et al., “V-JEPA 2.1: unlocking dense features in video self-supervised learning,” arXiv:2603.14482, 2026. arXiv
28. G. Zhou, H. Pan, Y. LeCun and L. Pinto, “DINO-WM: world models on pre-trained visual features enable zero-shot planning,” arXiv:2411.04983, 2024. arXiv
29. K. Chua, R. Calandra, R. McAllister and S. Levine, “Deep reinforcement learning in a handful of trials using probabilistic dynamics models,” NeurIPS, 2018. arXiv
30. M. Janner, J. Fu, M. Zhang and S. Levine, “When to trust your model: model-based policy optimization,” NeurIPS, 2019. arXiv
31. Z. Zhu et al., “Is Sora a world simulator? A comprehensive survey on general world models and beyond,” arXiv:2405.03520, 2024. arXiv
32. Y. Guan et al., “World models for autonomous driving: an initial survey,” arXiv:2403.02622, 2024. arXiv